

Hidden Surface Removal



Hidden Surface Removal

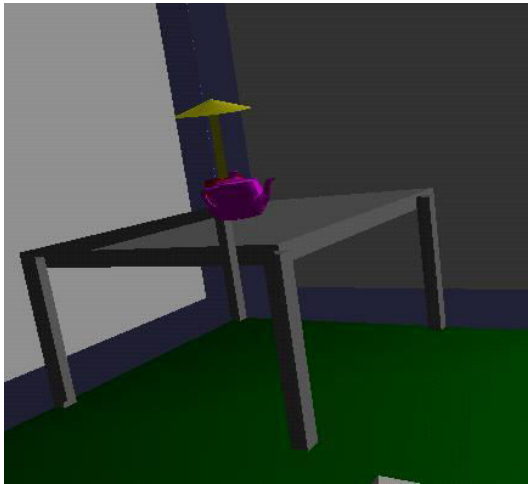
2

- Drawing polygon faces on screen consumes CPU cycles
- We cannot see every surface in scene
- To save time, draw only surfaces we see
- Surfaces we cannot see and their elimination methods:
 - Occluded surfaces: hidden surface removal (visibility)
 - Back faces: back face culling
 - Faces outside view volume: viewing frustum culling/clipping
- Definitions:
 - Object space: before vertices are mapped to pixels
 - Image space: after vertices have been rasterized

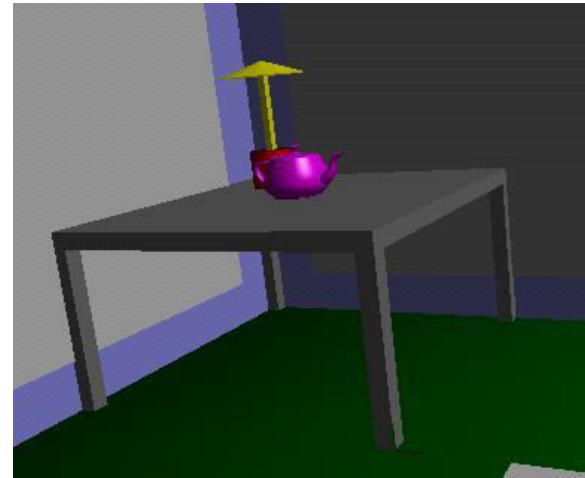
Visibility (hidden surface removal)

3

- A correct rendering requires correct visibility calculations
- Correct visibility – when multiple opaque polygons cover the same screen space, only the front most one is visible (remove the hidden surfaces)



wrong visibility



Correct visibility

Visibility (hidden surface removal)

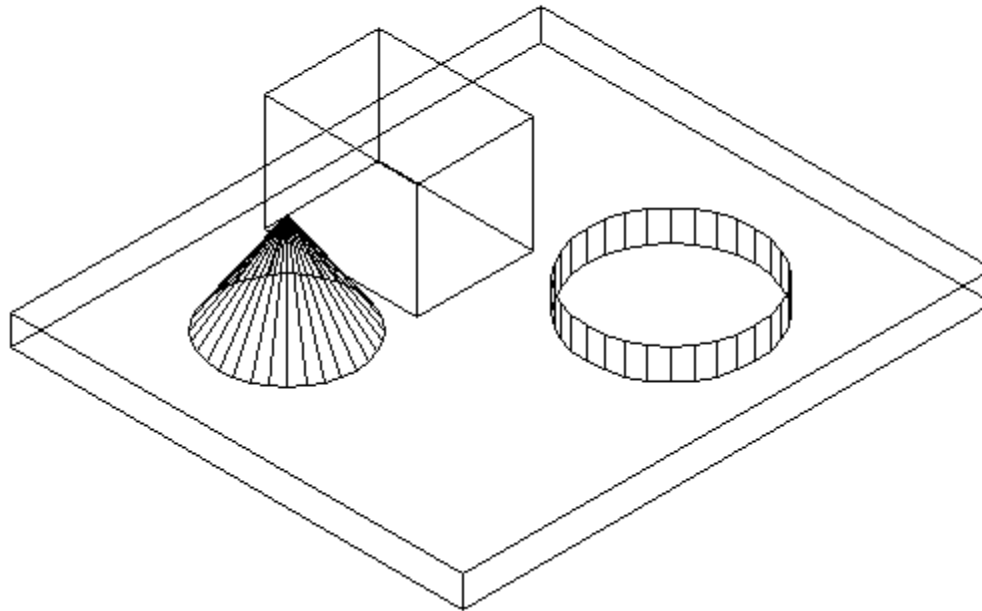
4

- Goal: determine which objects are visible to the eye
 - Determine what colors to use to paint the pixels
- Active research subject - lots of algorithms have been proposed in the past (and is still a hot topic)

No Lines Removed

5

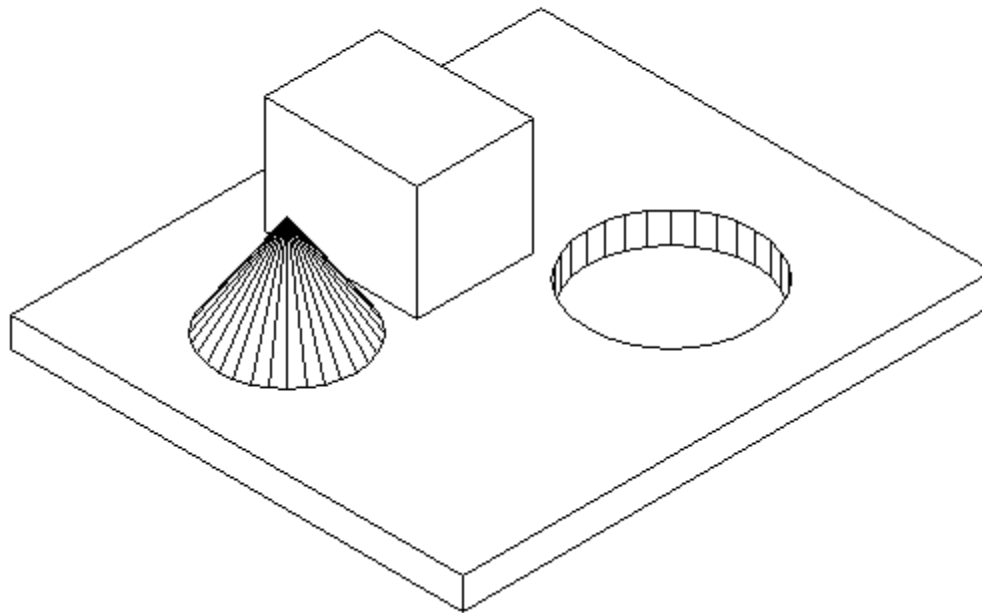
Wireframe



Hidden Lines Removed

6

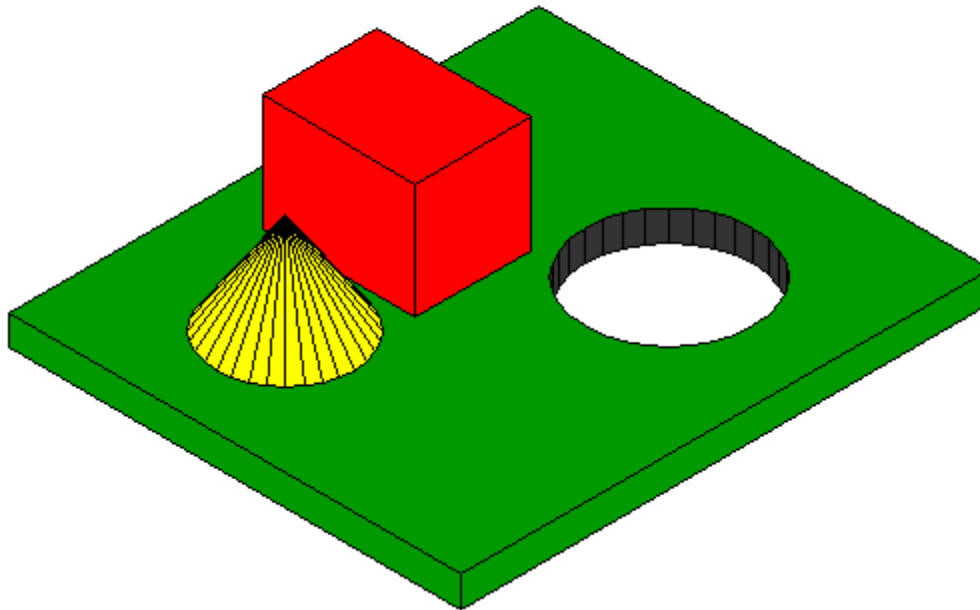
Hidden Line Removal



Hidden Surfaces Removed

7

Hidden Surface Removal

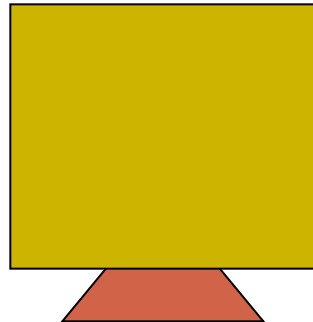


Occlusion: Full, Partial, None

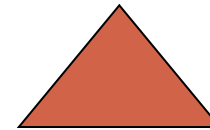
8



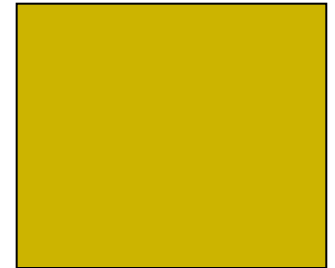
Full



Partial



None

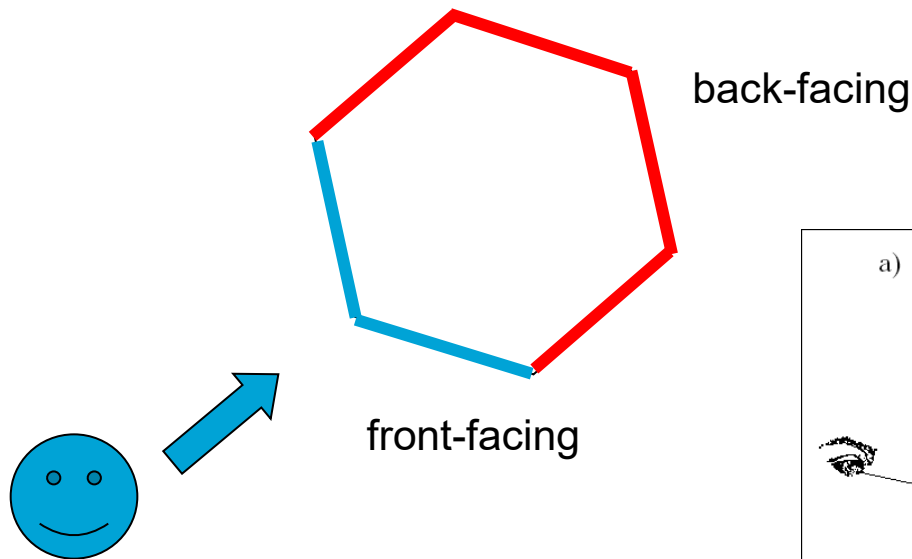


- The rectangle is closer than the triangle
- Should appear in front of the triangle

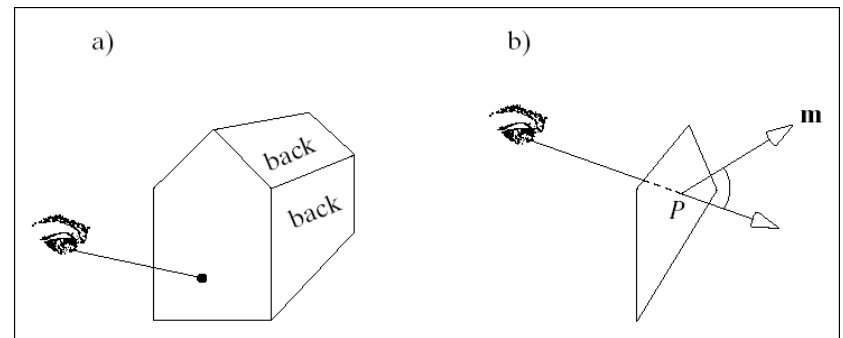
Backface Culling

9

- Avoid drawing polygons facing away from the viewer
 - Front-facing polygons occlude these polygons in a closed polyhedron
- Test if a polygon is front- or back-facing?



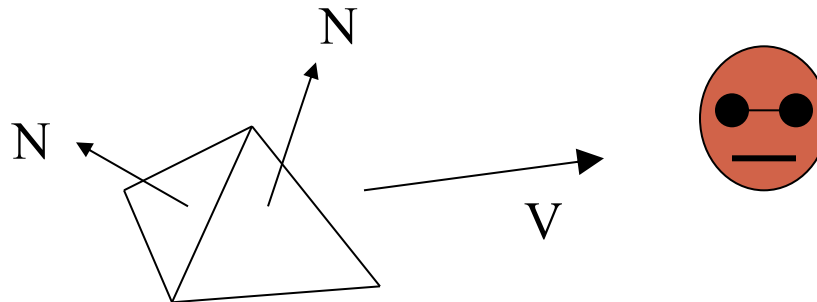
Ideas?



Backface Culling

10

- If we find backface, do not draw, save rendering resources
- There must be other forward face(s) closer to eye
- F is face of object we want to test if backface
- P is a point on F
- Form view vector, V as (eye - P)
- N is normal to face F



Detecting Back-face Polygons

11

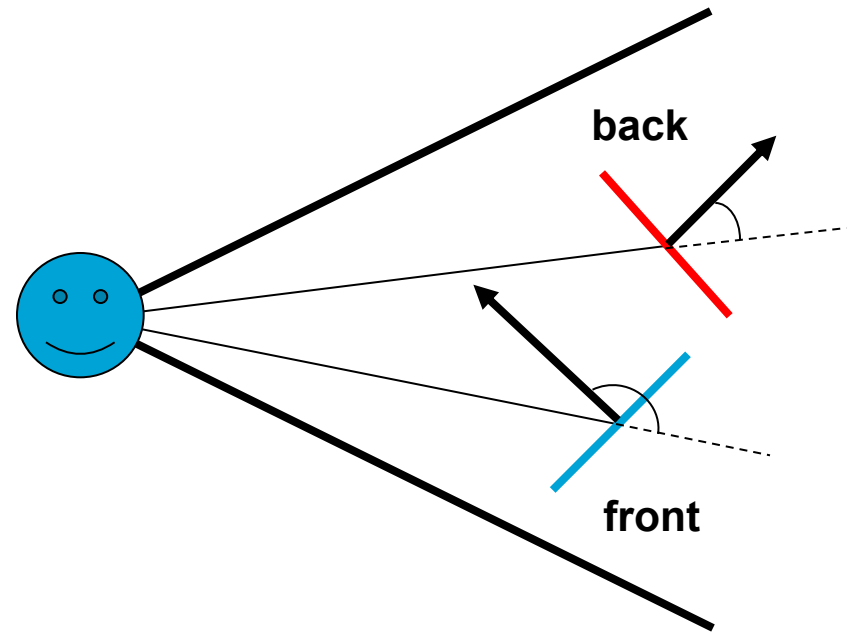
- The polygon normal of a ...
 - front-facing polygon points **towards** the viewer
 - back-facing polygon points **away** from the viewer

If $(\mathbf{n} \cdot \mathbf{v}) > 0 \Rightarrow$ “back-face”

If $(\mathbf{n} \cdot \mathbf{v}) \leq 0 \Rightarrow$ “front-face”

$\mathbf{v}$ = view vector

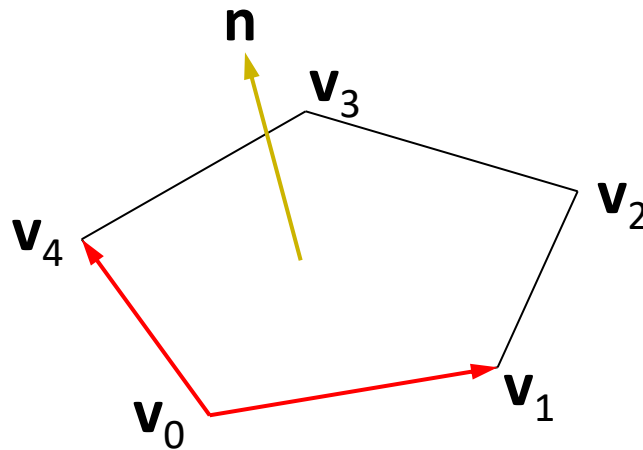
- Eye-space test ... EASY!
 - “back-face” if $\mathbf{n}_z < 0$



Polygon Normals

12

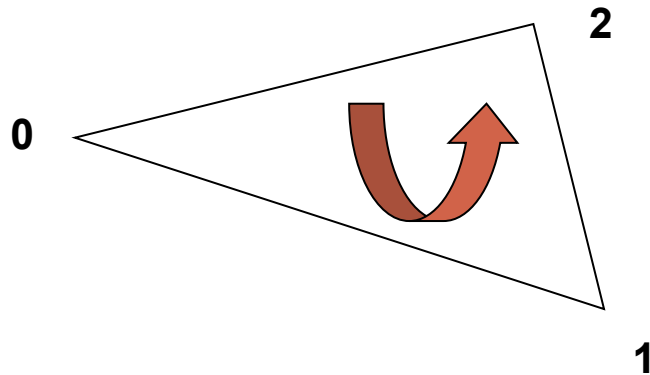
- Let polygon vertices $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_{n-1}$ be in counterclockwise order and co-planar
- Calculate normal with cross product:
$$\mathbf{n} = (\mathbf{v}_1 - \mathbf{v}_0) \times (\mathbf{v}_{n-1} - \mathbf{v}_0)$$
- Normalize to unit vector with $\mathbf{n} / \|\mathbf{n}\|$



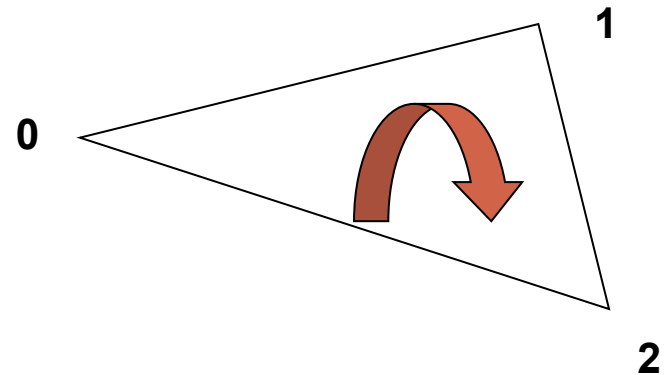
Normal Direction

13

- Vertices counterclockwise $\Rightarrow$ Front-facing
- Vertices clockwise $\Rightarrow$ Back-facing



Front facing



Back facing

Visibility

14

- How do we ensure that closer polygons overwrite further ones in general?

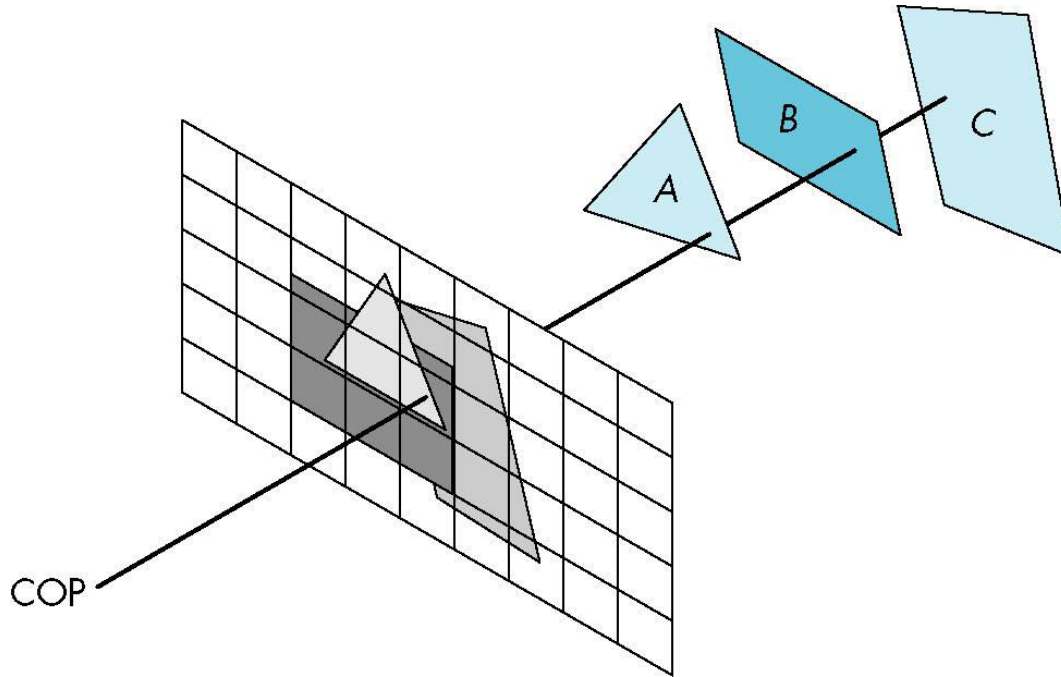


Image Space Approach – Z-buffer

15

- Method used in most of graphics hardware (and thus OpenGL): **Z-buffer algorithm**
- Requires lots of memory
- Basic idea:
 - rasterize every input polygon
 - Recall that we have z at polygon vertices
 - Maintains 2 buffers
 - For every pixel in the polygon interior, calculate its corresponding z value (by interpolation) (Z buffer)
 - Choose the color of the polygon whose z value is the closest to the eye to paint the pixel. (Refresh Buffer)

Image Space Approach – Z-buffer

16

- Recall: after projection transformation
- In viewport transformation
 - x, y used to draw screen image
 - z component is mapped to pseudo-depth with range $[0, 1]$
- However, objects/polygons are made up of vertices
- Hence z is known at vertices
- Point in object seen through pixel may be between vertices
- Need to interpolate to find z

Z (depth) buffer algorithm

17

- How to choose the polygon that has the closet Z for a given pixel?
- Assumption for example: eye at $z = 0$, farther objects have increasingly negative values
 - Initialize (clear) every pixel in the z buffer to a very large negative value
 - Track polygon z's.
 - As we rasterize polygons, check to see if polygon's z through this pixel is less than current minimum z through this pixel
 - Run the following loop:

Z (depth) Buffer Algorithm

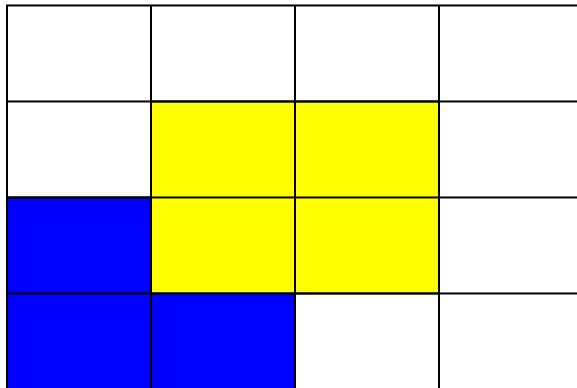
18

```
For each polygon {  
    for each pixel (x,y) inside the polygon projection area {  
        if (z_polygon_pixel(x,y) > depth_buffer(x,y) ) {  
            depth_buffer(x,y) = z_polygon_pixel(x,y);  
            color_buffer(x,y) = polygon color at (x,y)  
        }  
    }  
}
```

Note: we have depths at vertices. Interpolate for interior depths

Z buffer example

19



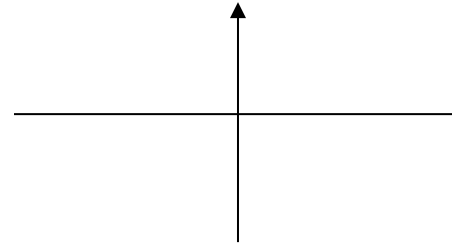
Final image



$Z = -0.5$



$Z = -0.3$



eye

Top View

Z buffer example

20

Step 1: Initialize the depth buffer

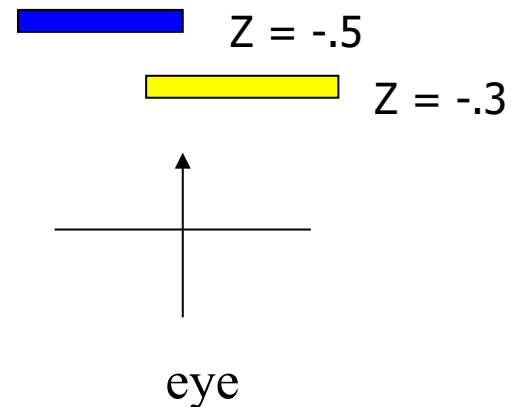
-999	-999	-999	-999
-999	-999	-999	-999
-999	-999	-999	-999
-999	-999	-999	-999

Z buffer example

21

Step 2: Draw the blue polygon (assuming the OpenGL program draws blue polygon first – the order does not affect the final result any way).

-999	-999	-999	-999
-999	-999	-999	-999
-.5	-.5	-999	-999
-.5	-.5	-999	-999

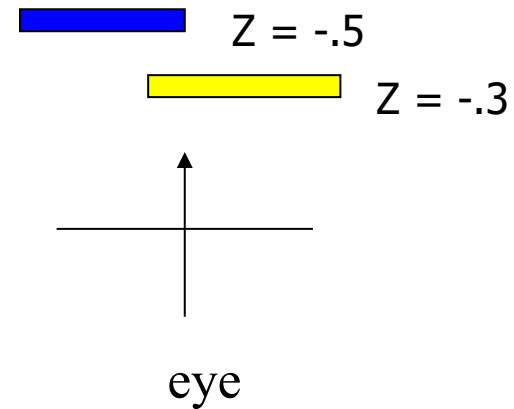


Z buffer example



Step 3: Draw the yellow polygon

-999	-999	-999	-999
-999	-.3	-.3	-999
-.5	-.3	-.3	-999
-.5	-.5	-999	-999



Calculating depth values efficiently

23

- We know the depth values at the vertices. How can we calculate the depth at any other point on the surface of the polygon.
- Using the polygon surface equation:

$$z = \frac{-Ax - By - D}{C}$$

Calculating depth values efficiently

24

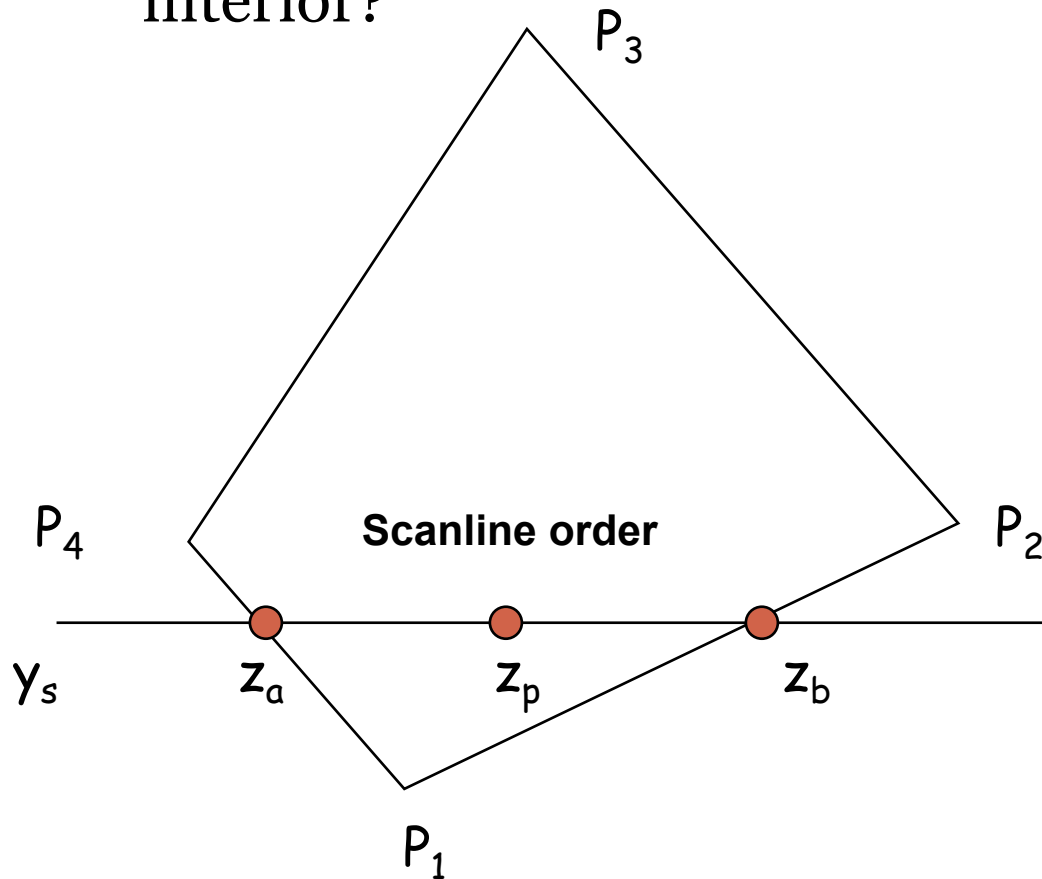
- For any scan line adjacent horizontal x positions or vertical y positions differ by 1 unit.
- The depth value of the next position $(x+1, y)$ on the scan line can be obtained using.

$$\begin{aligned} z' &= \frac{-A(x+1) - By - D}{C} \\ &= z - \frac{A}{C} \end{aligned}$$

Z-Buffer



- How do we calculate the depth values on the polygon interior?



$$z_a = z_1 + (z_4 - z_1) \frac{(y_1 - y_s)}{(y_1 - y_4)}$$

$$z_b = z_1 + (z_2 - z_1) \frac{(y_1 - y_s)}{(y_1 - y_2)}$$

$$z_p = z_a + (z_b - z_a) \frac{(x_a - x_p)}{(x_a - x_b)}$$

Bilinear Interpolation

Z-Buffer Example



∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞

Z-buffer

Screen

[6,7,5]

5	5	5	5	5	5	5
5	5	5	5	5	5	
5	5	5	5	5		
5	5	5	5			
5	5	5				
5	5					
5						

[illegible]

A 10x10 grid with a 4x4 square highlighted in purple in the top-left corner.

Not Parallel

7					
6	7				
5	6	7			
4	5	6	7		
3	4	5	6	7	
2	3	4	5	6	7

[5,1,7]

[illegible]

A 10x10 grid with a purple path and an orange path. The purple path starts at (0,9) and ends at (4,4). The orange path starts at (0,4) and ends at (5,0). The paths are composed of horizontal and vertical segments.

Limitation of Z-buffer Algorithm

29

- This algorithm do not work on **transparent surface**, this is only for opaque surface.
- Hence here no method is available to compare the Z values of the surfaces which are transparent.
- For working on transparent surface another method is available which is called A-buffer Algorithm.
- Extra memory is required to store depth value.

A-Buffer Method

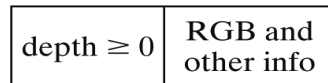
30

- Extends the depth-buffer algorithm so that each position in the buffer can reference a linked list of surfaces.
- More memory is required.
- However, we can correctly compose different surface colors and handle transparent surfaces.

A-Buffer Method

31

- Each position in the A-buffer has two fields:
 - a depth field
 - surface data field which can be either surface data or a pointer to a linked list of surfaces that contribute to that pixel position
 - Many other information can be stored in the link list such as : % of transparency, % of area coverage, surface rendering factor.



(a)



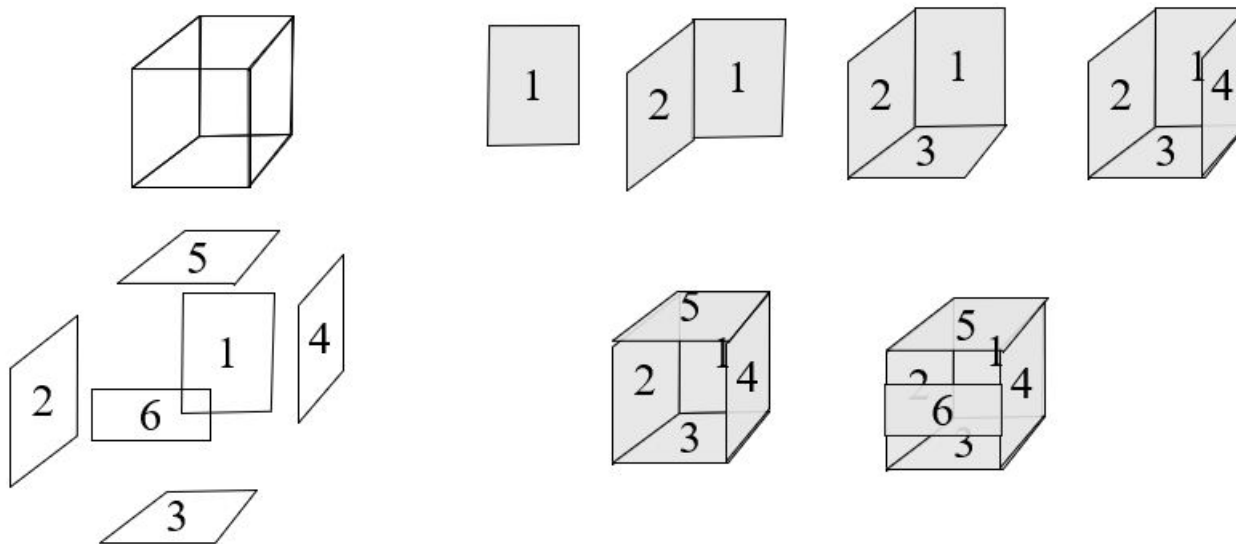
(b)

Object Space Methods

Depth Sorting

33

- Also known as **painters algorithm**. First draw the distant objects than the closer objects. Pixels of each object overwrites the previous objects.



Depth-sort algorithm

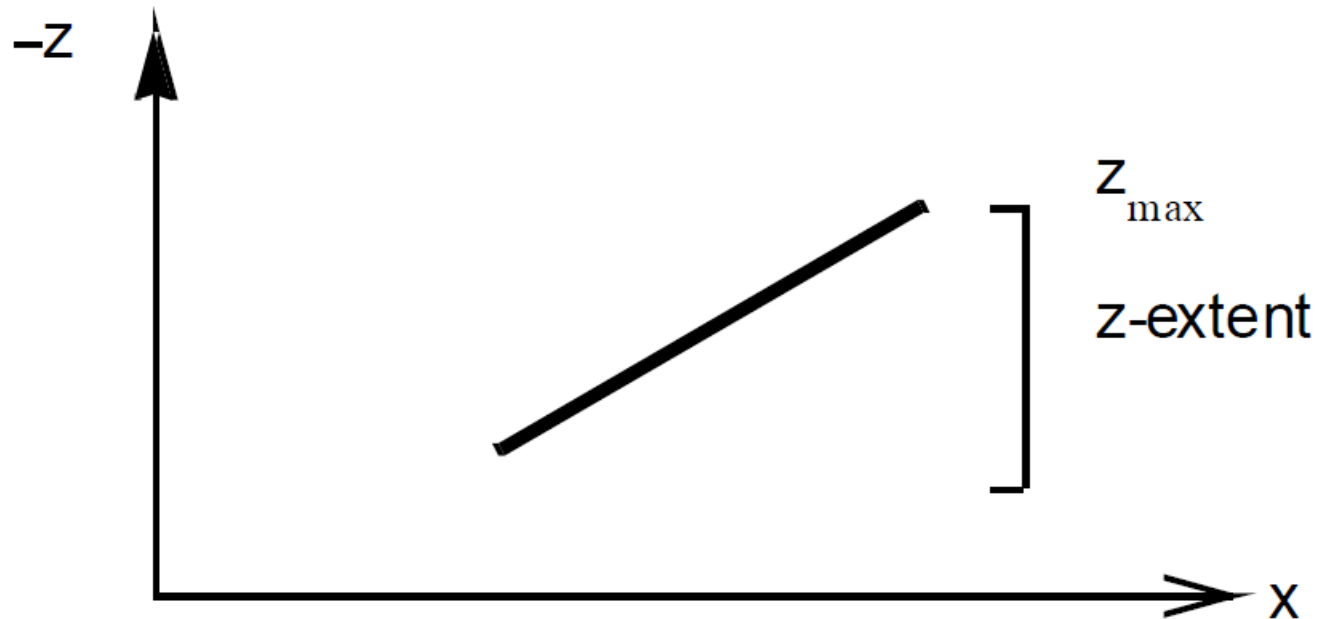
34

- The idea here is to go back to front drawing all the objects into the frame buffer with nearer objects being drawn over top of objects that are further away.
- Simple algorithm:
 - Sort all polygons based on their farthest z coordinate
 - Resolve ambiguities
 - Draw the polygons in order from back to front
- This algorithm would be very simple if the z coordinates of the polygons were guaranteed never to overlap. Unfortunately that is usually not the case, which means that step 2 can be somewhat complex.

Depth-sort algorithm

35

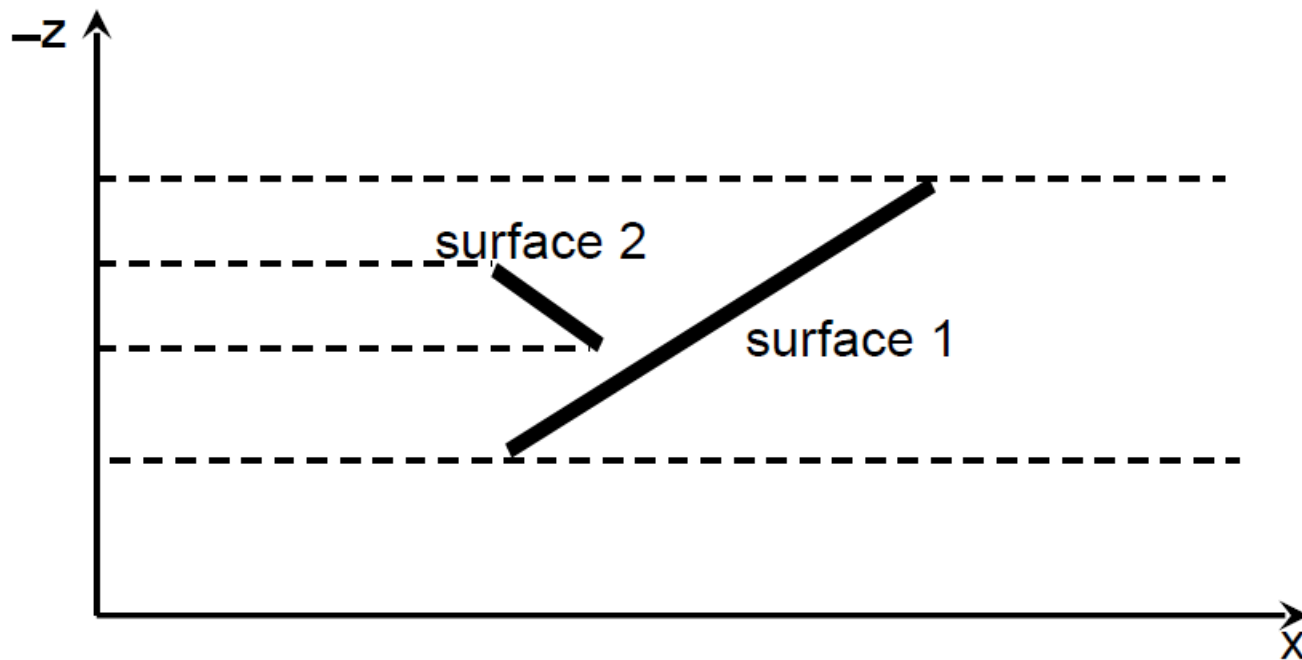
- First must determine z-extent for each polygon



Depth-sort algorithm

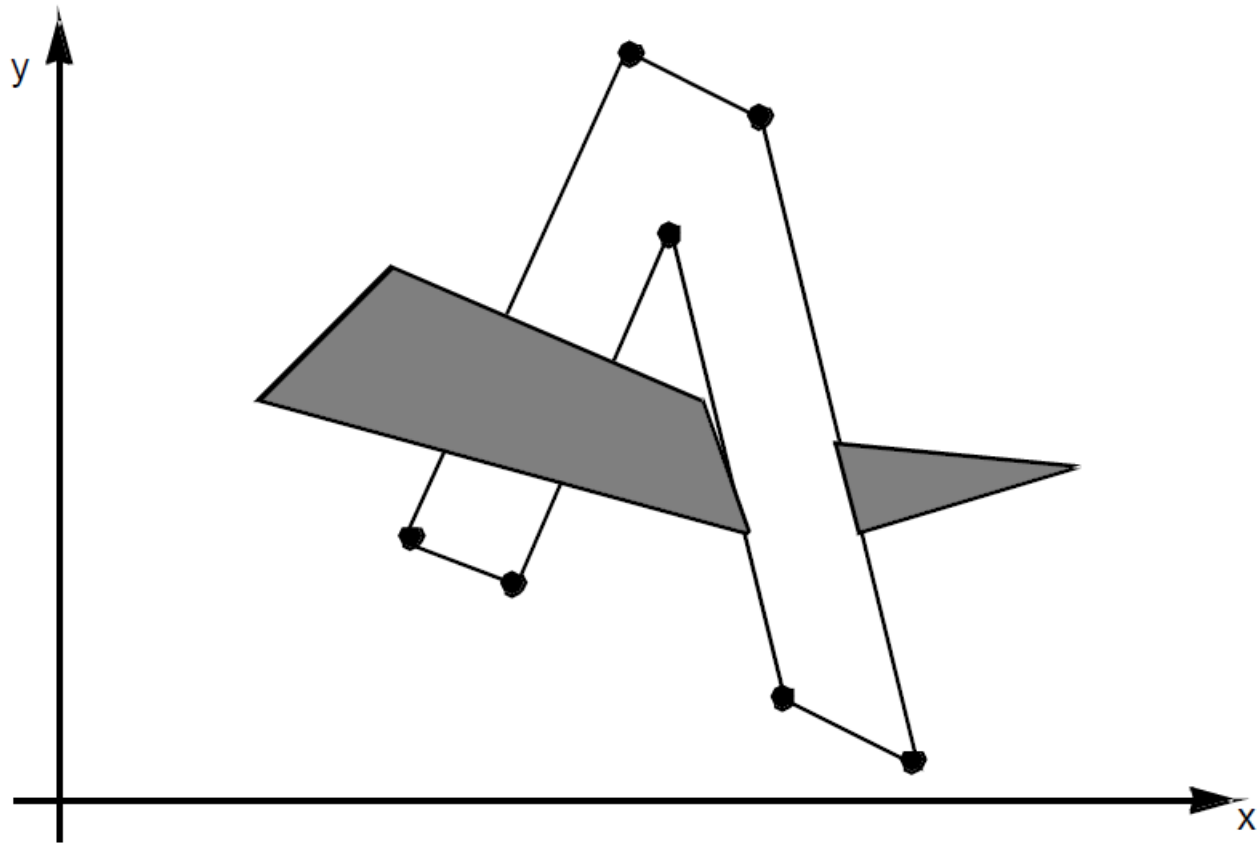
36

- Ambiguities arise when the z-extents of two surfaces overlap.



Depth-sort algorithm

37



Depth-sort algorithm

38

- All polygons whose z extents overlap must be tested against each other.
- We start with the furthest polygon and call it P. Polygon P must be compared with every polygon Q whose z extent overlaps P's z extent. 5 comparisons are made. If any comparison is true then P can be written before Q. If at least one comparison is true for each of the Qs then P is drawn and the next polygon from the back is chosen as the new P.

Depth-sort algorithm

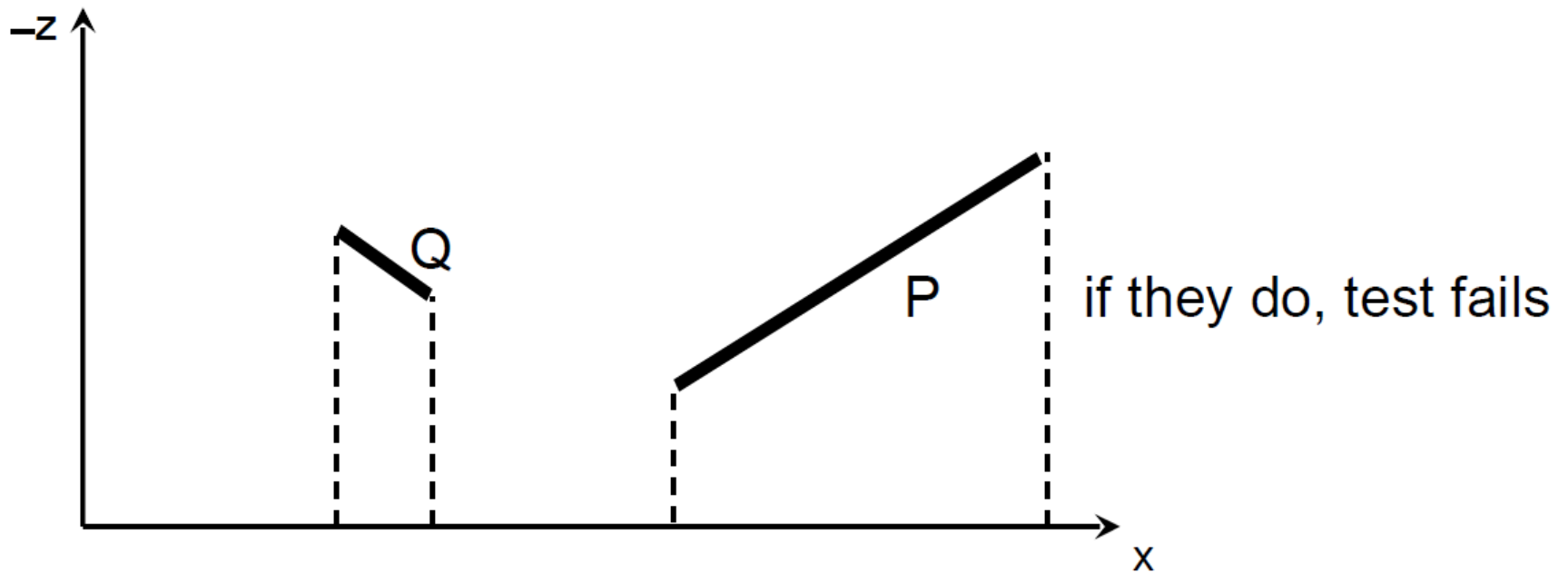
39

1. Do P and Q's x-extents not overlap?
 2. Do P and Q's y-extents not overlap?
 3. Is P entirely on the opposite side of Q's plane from the viewport?
 4. Is Q entirely on the same side of P's plane as the viewport?
 5. Do the projections of P and Q onto the (x,y) plane not overlap?
- If all 5 tests fail we quickly check to see if switching P and Q will work. Tests 1, 2, and 5 do not differentiate between P and Q but 3 and 4 do. So we rewrite 3 and 4 as:
 - 3'. Is Q entirely on the opposite side of P's plane from the viewport?
 - 4'. Is P entirely on the same side of Q's plane as the viewport?

Depth-sort algorithm

40

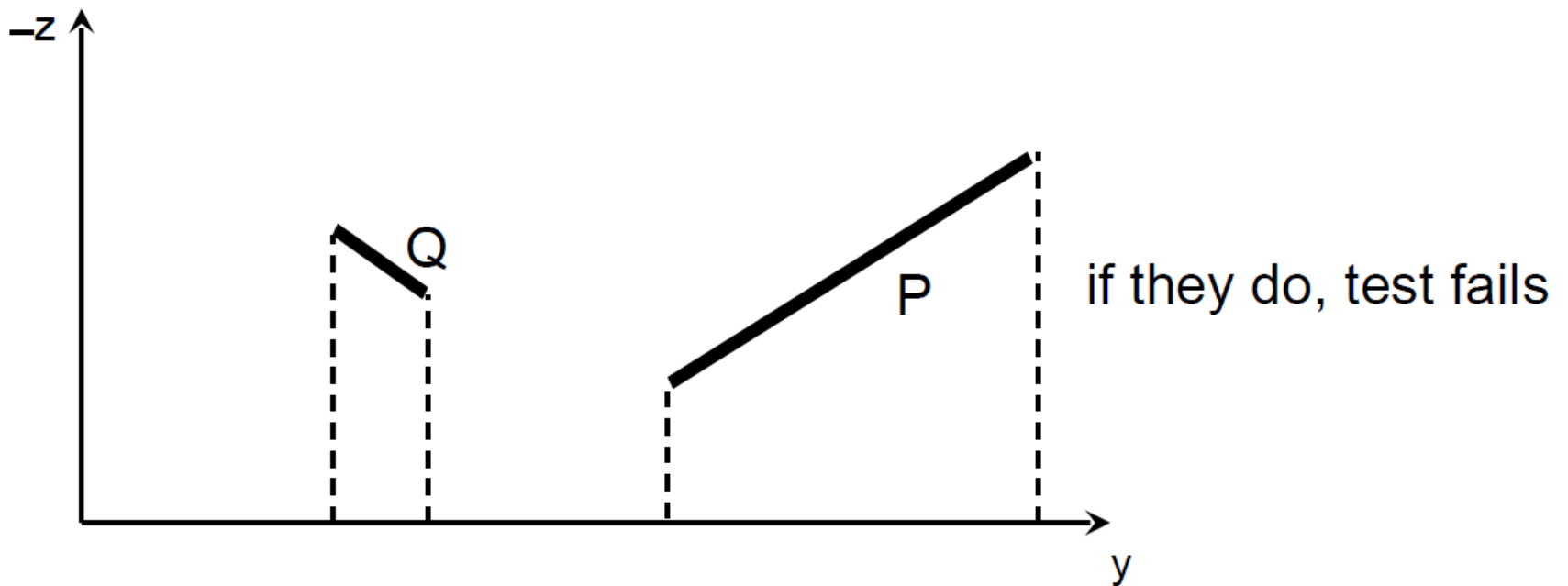
- x - extents not overlap?



Depth-sort algorithm

41

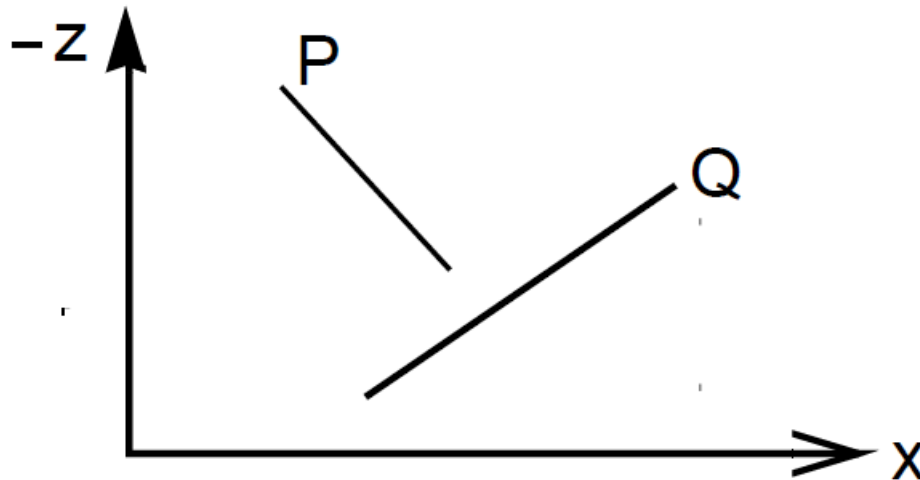
- y - extents not overlap?



Depth-sort algorithm

42

- Is P entirely behind the surface Q relative to the viewing position (i.e., behind Q's plane with respect to the viewport)?

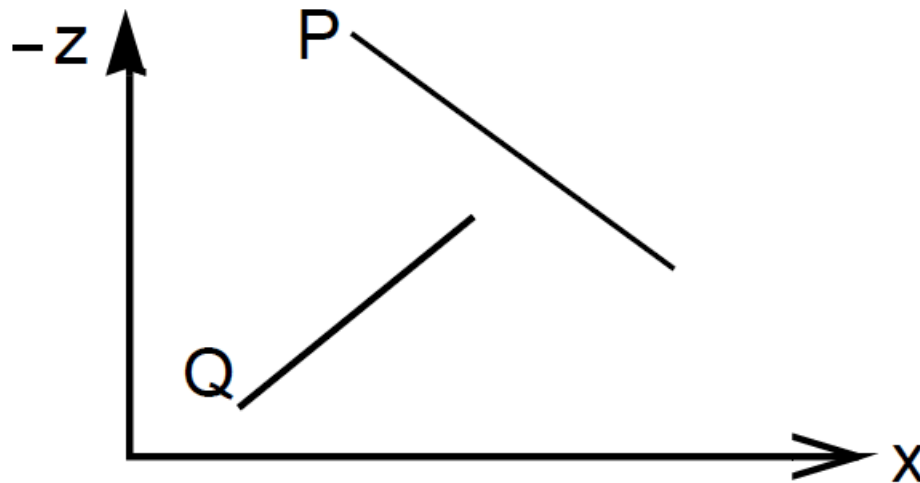


Test is true...

Depth-sort algorithm

43

- Is Q entirely in front of P's plane relative to the viewing position (i.e., the viewport)?

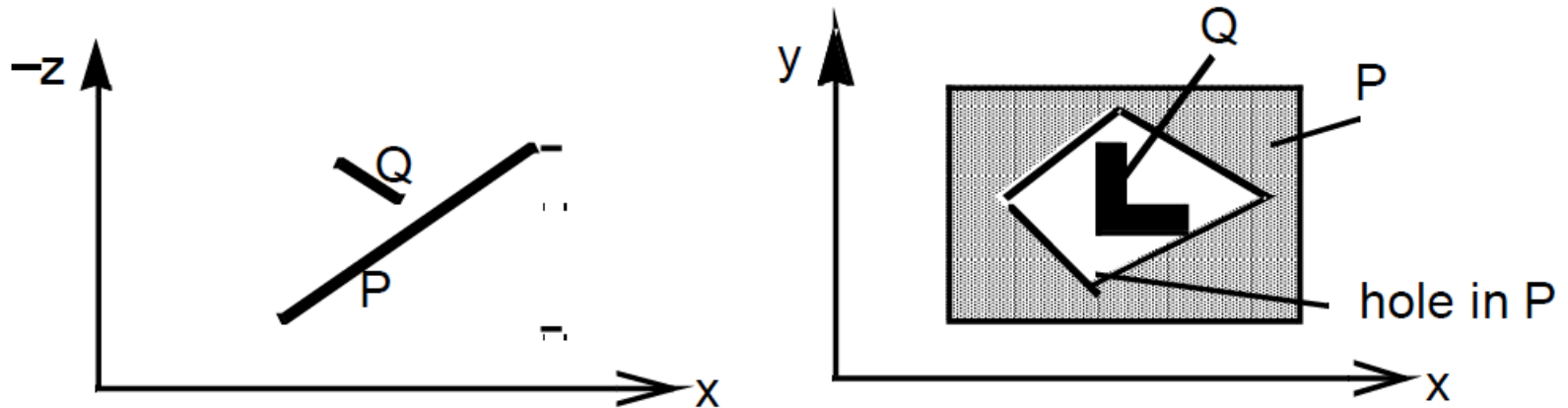


Test is true...

Depth-sort algorithm

44

- Do the projections of P and Q onto the (x,y) plane not overlap?



Test is true...

Depth-sort algorithm

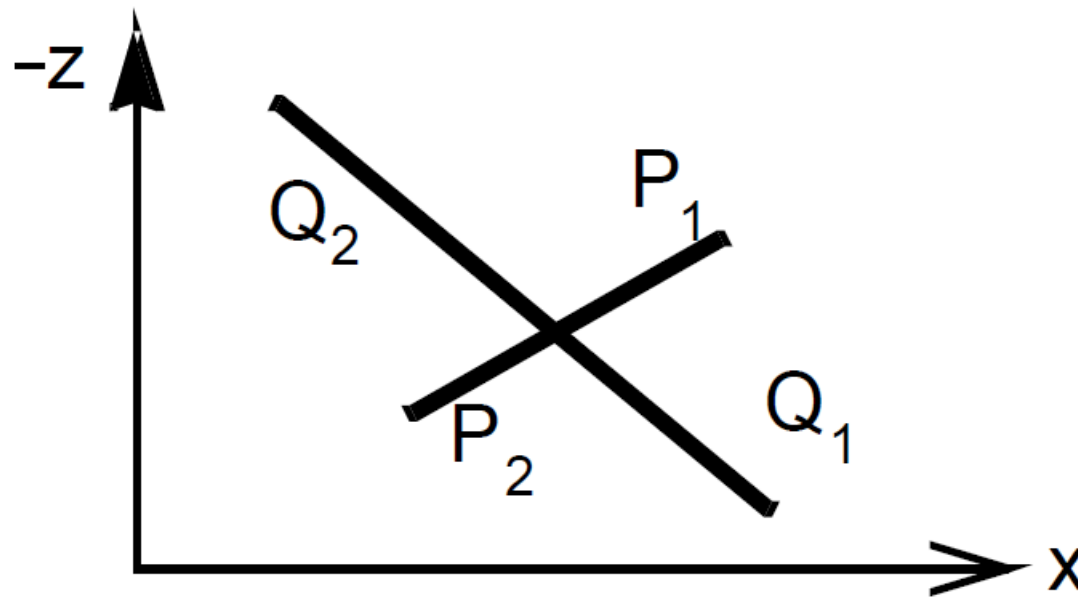
45

- If all tests fail...
 - ... then reverse P and Q in the list of surfaces sorted by maximum depth
 - set a flag to say that the test has been performed once.
 - If the tests fail a second time, then it is necessary to split the surfaces and repeat the algorithm on the 4 new split surfaces

Depth-sort algorithm

46

- Example:
 - We end up processing with order Q_2, P_1, P_2, Q_1



Reference

47

- **Computer Graphics**
 - Donald Hearn, M Pauline Baker
 - Pearson Education

CSE 4207: Computer Graphics

Introduction

DR. SK. MD. MASUDUL AHSAN

PROFESSOR

DEPT. OF CSE, KUET

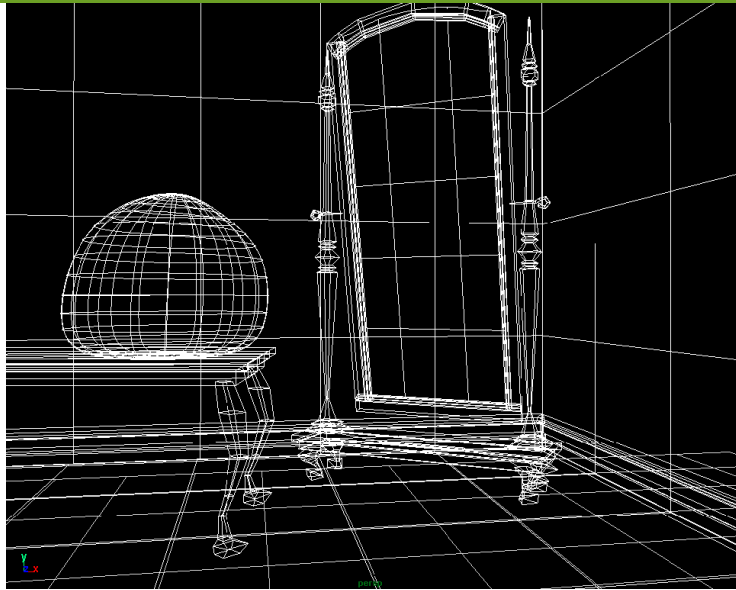
Computer Graphics

- ❑ Computer Graphics commonly mean the creation, storage and manipulation of geometric objects (modeling) and their images (rendering)
- ❑ ~ is concerned with the creation and manipulation of **synthetic** images and animations using the aid of computer.

Computer graphics are pictures and movies created using computers – usually referring to image data created by a computer specifically with help from specialized graphical hardware and software.

- wikipedia

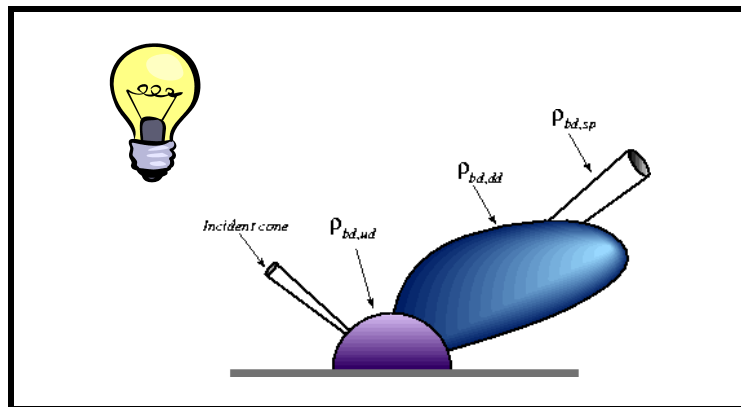
Computer graphics?



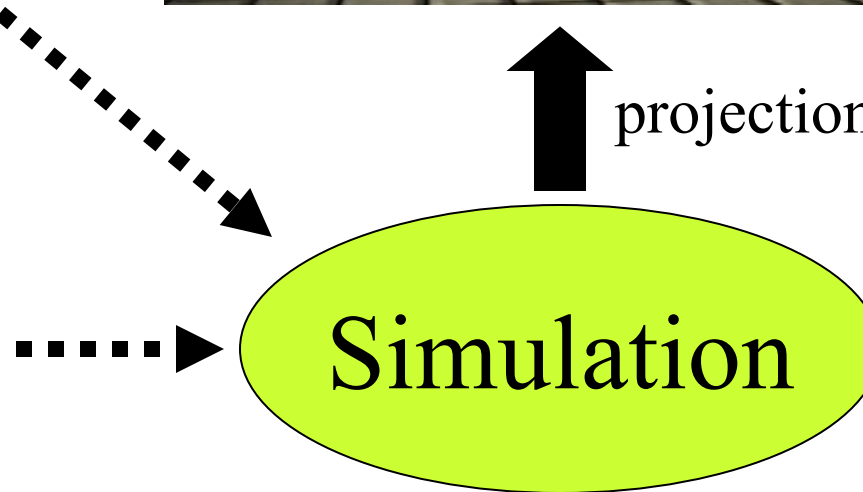
3D geometry



projection



physics



Why Study Computer Graphics?

❑ Graphics is cool

- I like to see what I'm doing
- I like to show people what I'm doing

❑ Graphics is interesting

- Involves *simulation, AI, algorithms, architecture...*

❑ I'll never get an Oscar for my acting

- But maybe I'll get one for my CG special effects

❑ Graphics is fun

❑ Very interdisciplinary

- Within CS: systems, compilers, languages, computer architecture, algorithms, numerical techniques
- Math, physics, art, perception, architecture, manufacturing

❑ Helps you understand why the world looks the way it does

- You can “see” the result

What drives computer graphics?

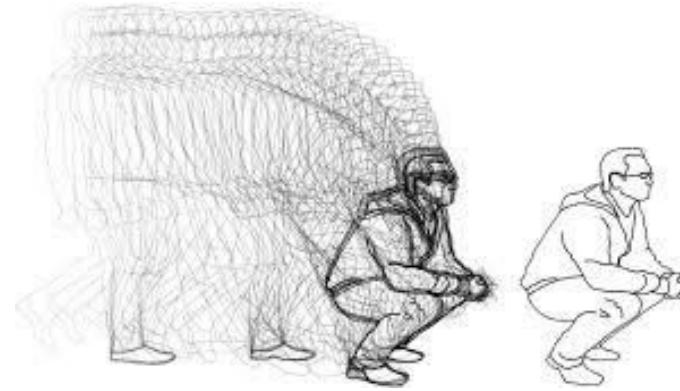
❑ Movie Industry

- Movie production, Animations, Special Effect (super reality)
- Big budgets and tight schedules
- Defines our expectations



What drives computer graphics?

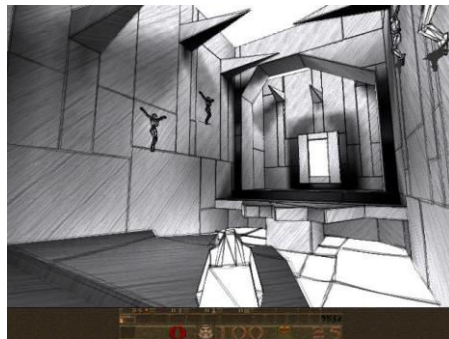
- ❑ Movie production (special effects & full-length feature films)
 - Matte (traveling Matte – blue/green screen)
 - Rotoscoping to computer animation



What drives computer graphics?

❑ Game Industry

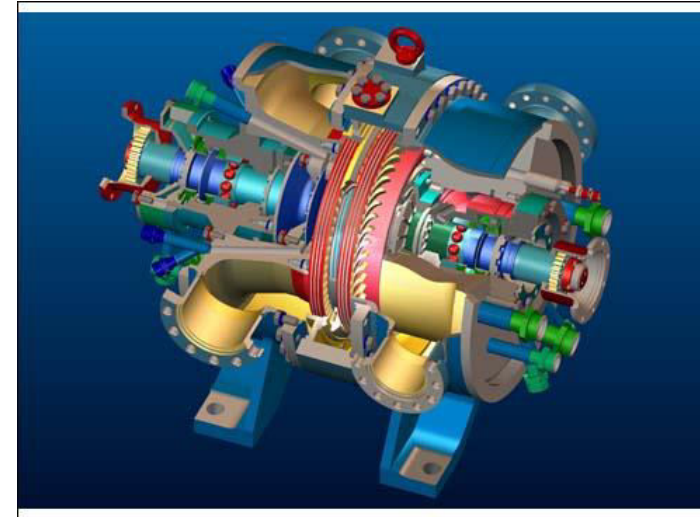
- The newest driving force in CG
 - Why? Volume and Profit
 - This is why we have commodity GPUs
- Focus on interactivity
- Cost effective solutions



What drives computer graphics?

❑ Computer Aided Design

- Structural design – design of buildings, bridges, car, aircraft, many more
- Electrical circuit, mechanical design
- Integration of computing and display resources
- Reduced design cycles == faster systems, sooner



Important Application Areas

- ❑ Human-computer Interaction (HCI)
 - Augmented Reality

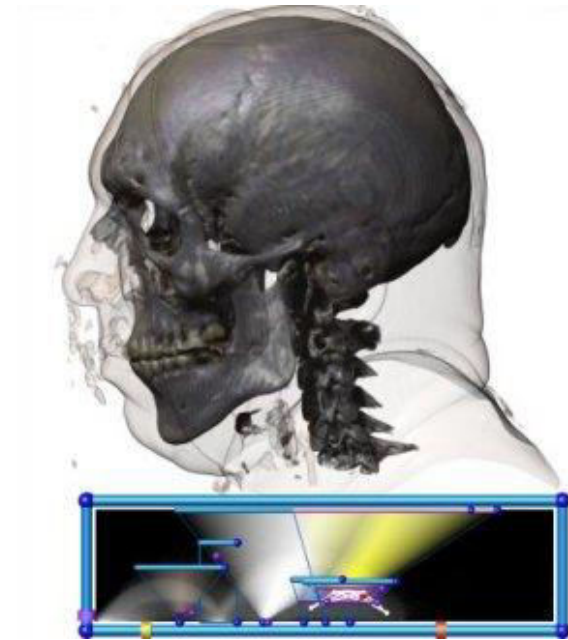


<http://augmentedrealityoverview.blogspot.com/2011/11/edibear.html>

Graphics Applications

❑ Medical Visualization

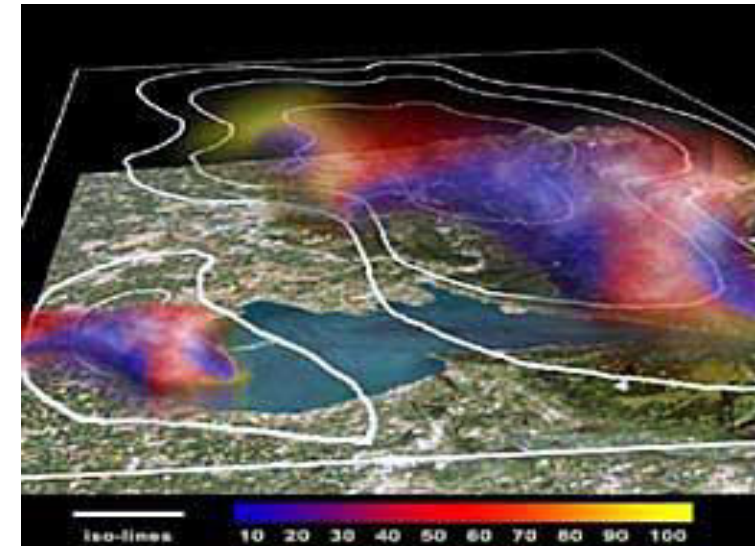
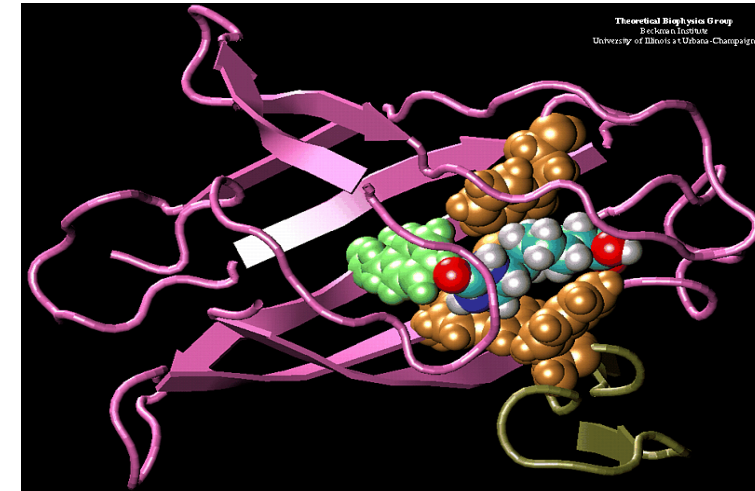
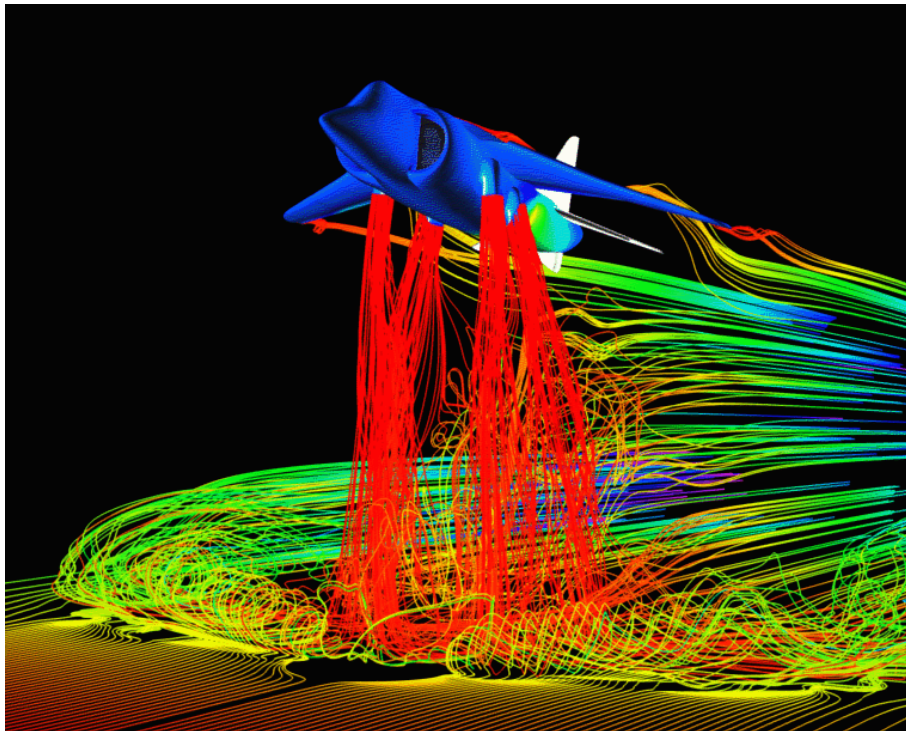
- Tools for visualizing diagnosis
 - No cheating or tricks allowed
- Drive issues of precision and correctness
- Focus on presentation and interpretation of data
- Construction of models from acquired data



Graphics Applications

❑ Scientific Data Visualization

- CG is superb tool for presenting complex scientific data in a way that can be easily grasped



Airflow around a Harrier Jet (NASA Ames) PROF. DR. SMM AHSAN, CSE, KUET

Computer Graphics - Applications

❑ Education and Training

- Computer generated model can aid easy understanding (education)
- Various simulator can be used for practice / training session for aircraft pilot, heavy equipment operators.

❑ Computer Art, publishing

- Fine and commercial art
- Slide, Book, Magazine design



What's Our Scope?

- ❑ Not a Tutorial on Commercial Software

- 3DMax, Maya, Photoshop, etc.

- ❑ Not about Graphics Business

- 3D online-game, E-commerce, etc.

- ❑ Graphics = Algorithm for Visual Simulation

- Modeling, Rendering, Animation

Computer Graphics - Main Themes

❑ Imaging

- Representing 2D images

❑ Modeling

- Representing 3D objects
 - Surface Modeling
 - Solid Modeling
 - Procedure Modeling
 - Simplification

❑ Rendering

- Constructing 2D images from 3D models
 - Polygonal Rendering
 - Global Rendering
 - Texture Mapping

❑ Animation

- Simulating changes over time
 - Keyframe Animation
 - Inverse Kinematics
 - Dynamics
 - Motion Capture

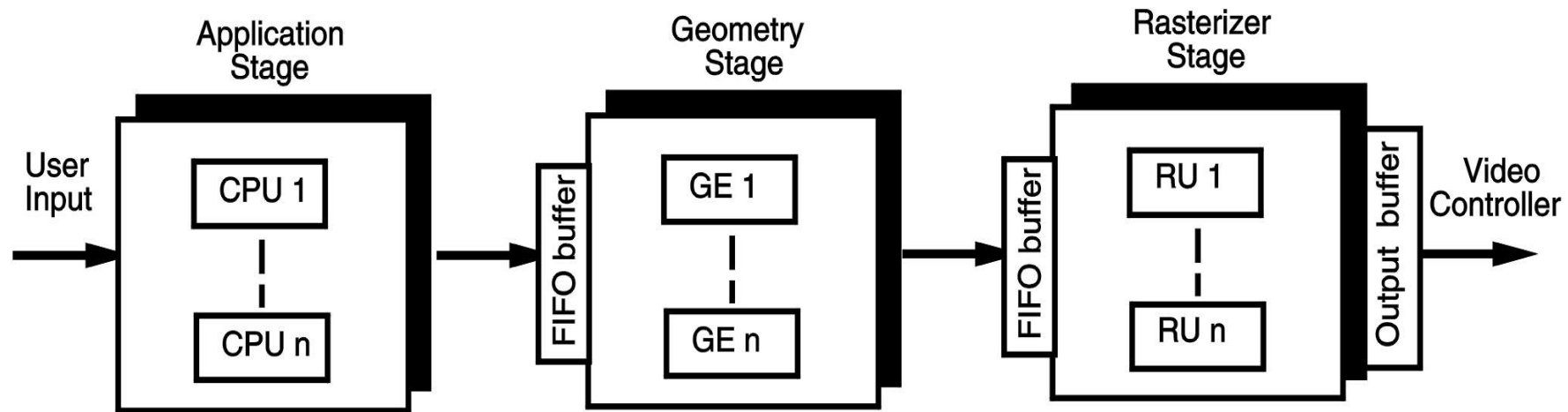
Graphics / Rendering Pipeline

There are three stages

Application Stage

Geometry Stage

Rasterization Stage



Application stage

Entirely done in software by the CPU

Read Data

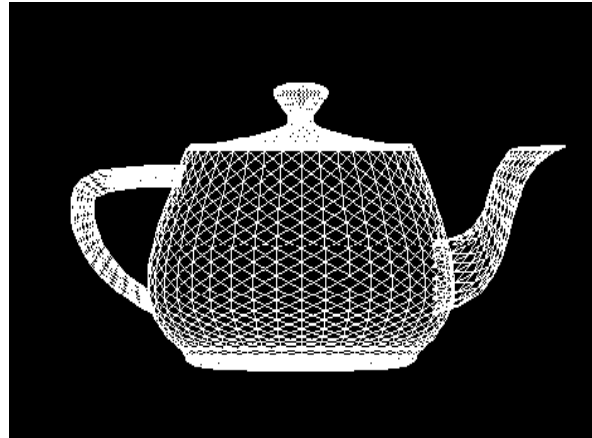
the world geometry database,

User's input by mice, trackballs, trackers, or sensing gloves

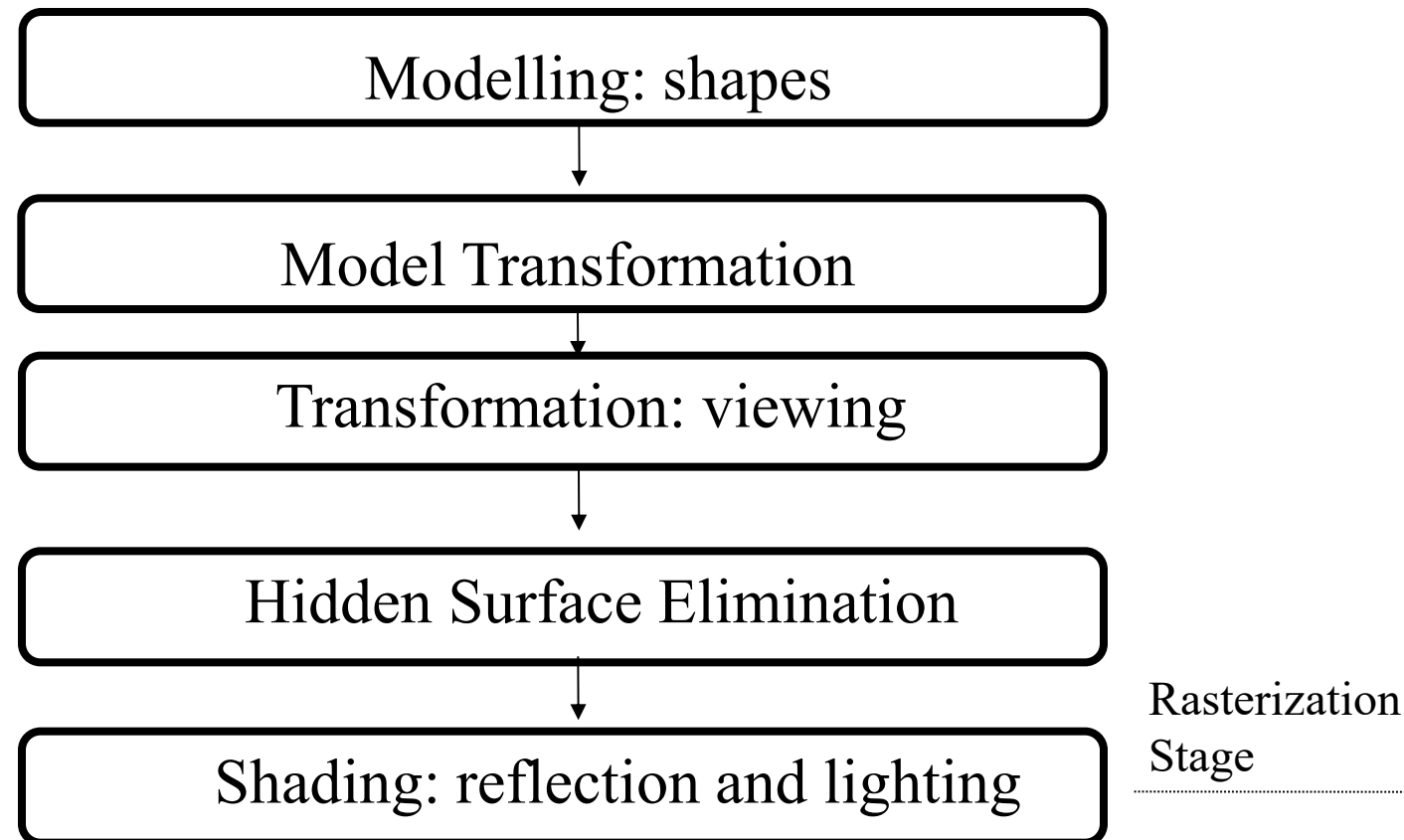
In response to the user's input, the application stage change the view or scene

```
✓ 3. 382035 2. 446498 -0. 064692
✓ 3. 382035 2. 446498 0. 064692
✓ 3. 392006 2. 474995 -0. 050004
✓ 3. 392006 2. 474995 0. 050004
✓ 3. 400000 2. 446800 0. 000000
✓ 3. 406947 2. 462176 -0. 061668
✓ 3. 406947 2. 462176 0. 061668
✓ 3. 408000 2. 475600 0. 000000
✓ 3. 408000 2. 475600 0. 000000
✓ 3. 411237 2. 476753 -0. 054000
✓ 3. 411237 2. 476753 0. 054000
✓ 3. 416450 2. 472371 -0. 057996
✓ 3. 416450 2. 472371 0. 057996
✓ 3. 424875 2. 462506 0. 000000
✓ 3. 428125 2. 477344 0. 000000
✓ 3. 428125 2. 477344 0. 000000
✓ 3. 434000 2. 472900 0. 000000

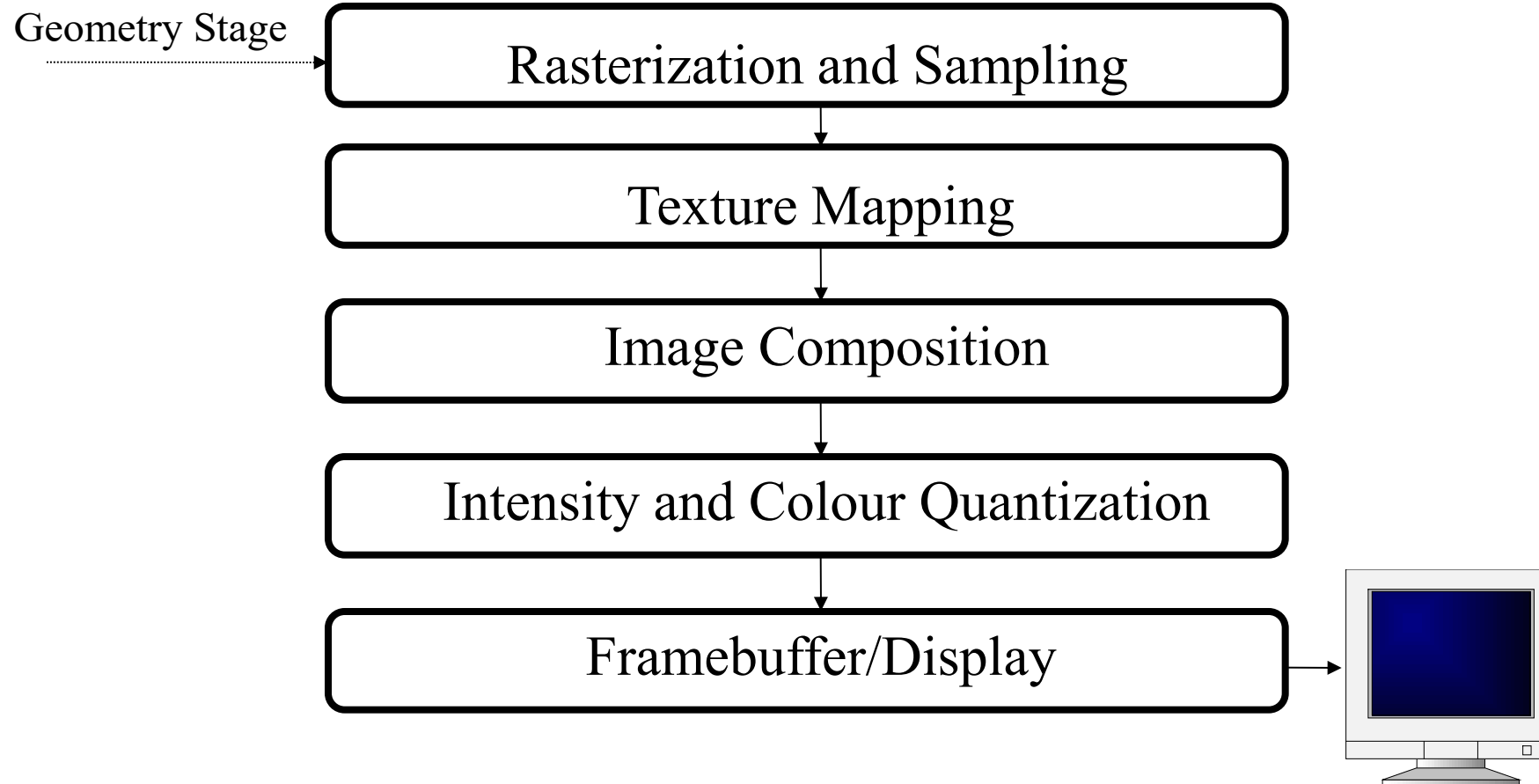
2909 2921 2939
2939 2931 2909
2869 2877 2921
2921 2909 2869
2819 2827 2877
2877 2869 2819
2737 2747 2827
2827 2819 2737
2669 2673 2747
2747 2737 2669
2567 2575 2673
2673 2669 2567
```



Geometry Stage



Rasterization Stage



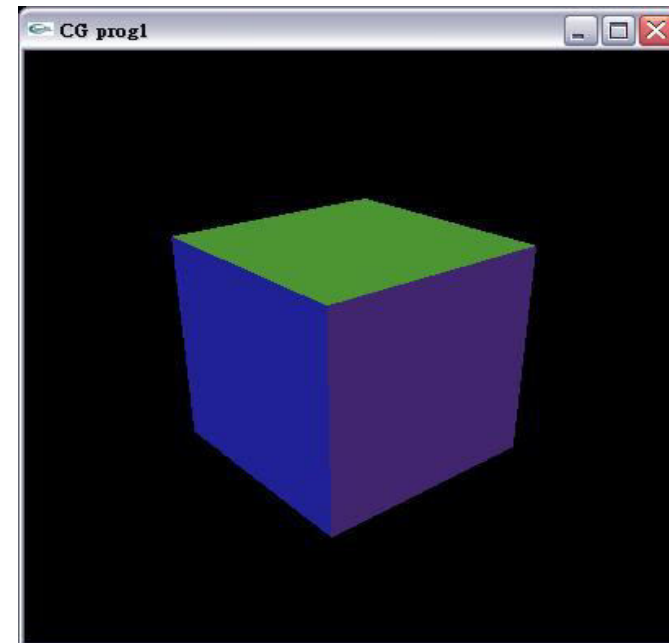
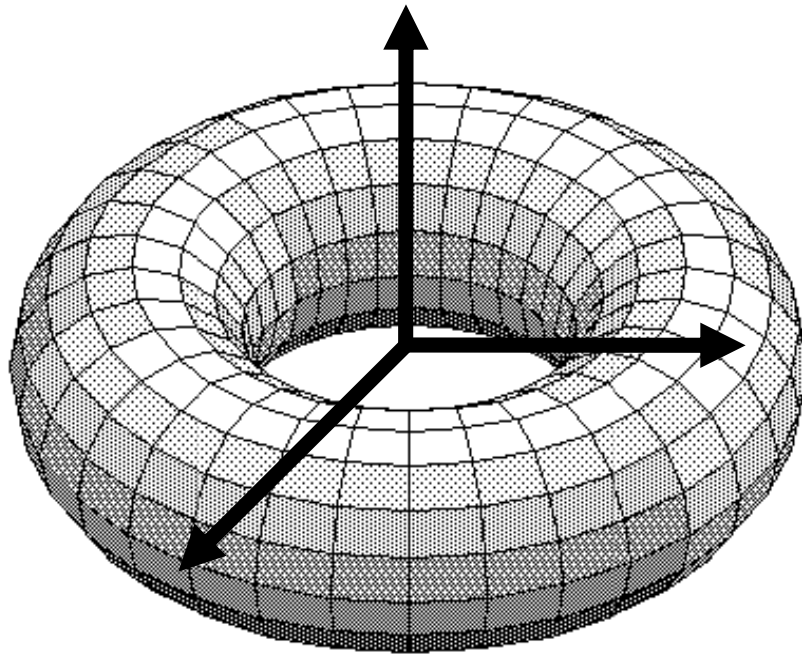
An example thro' the pipeline...

The scene we are trying to represent:



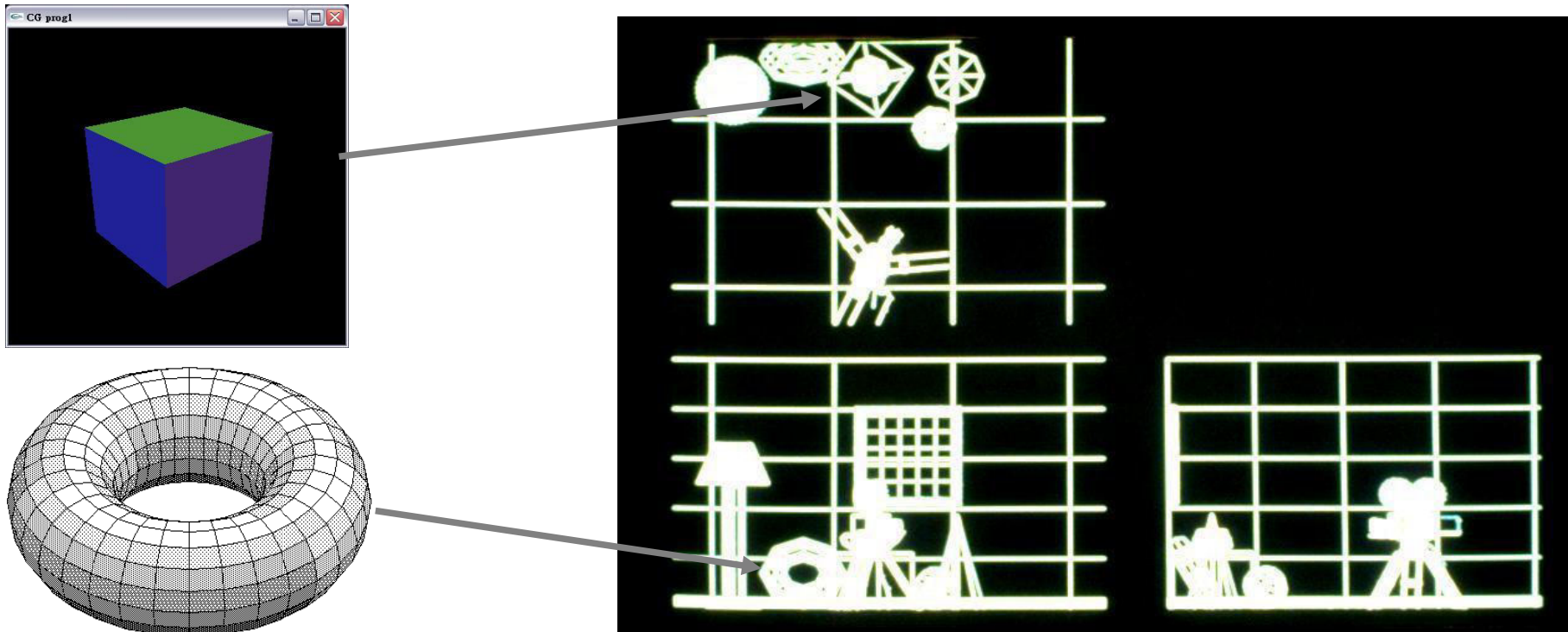
Preparing Shape Models

- ❑ Designed by polygons, parametric curves/surfaces, implicit surfaces and etc.
- ❑ Defined in its own coordinate system



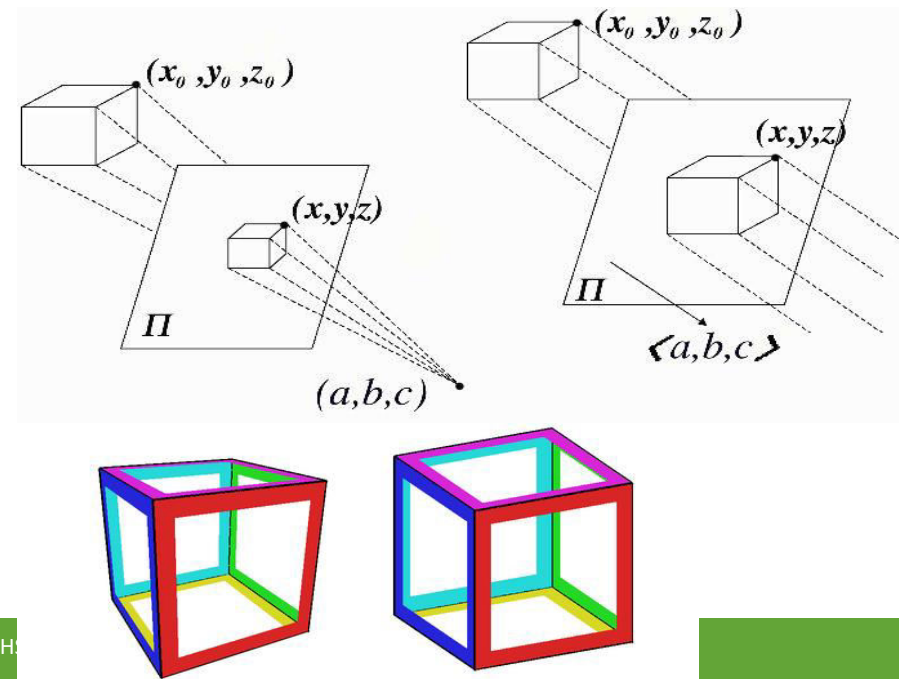
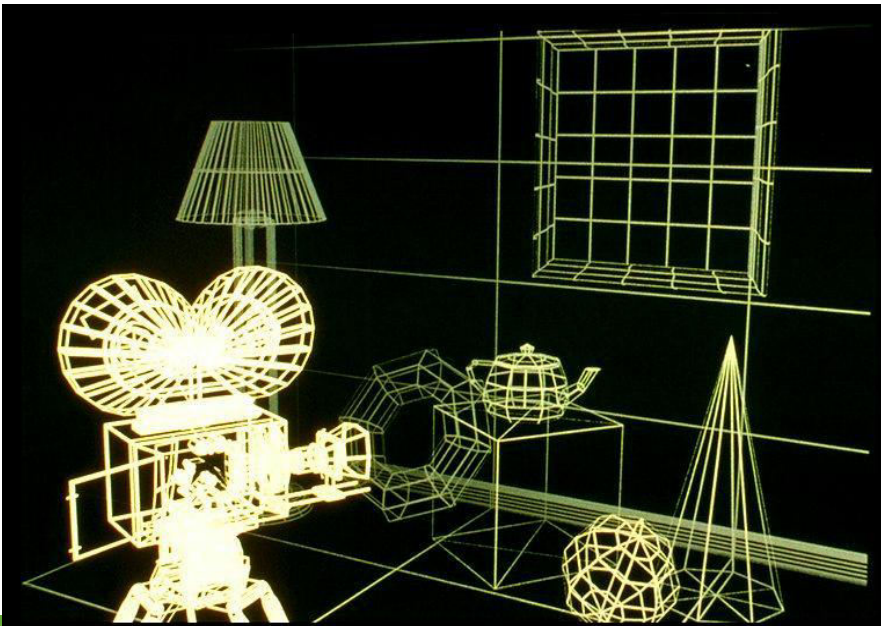
Model Transformation

- ✓ Objects put into the scene by applying translation, scaling and rotation
- ✓ The location of all the vertices are updated by this transformation



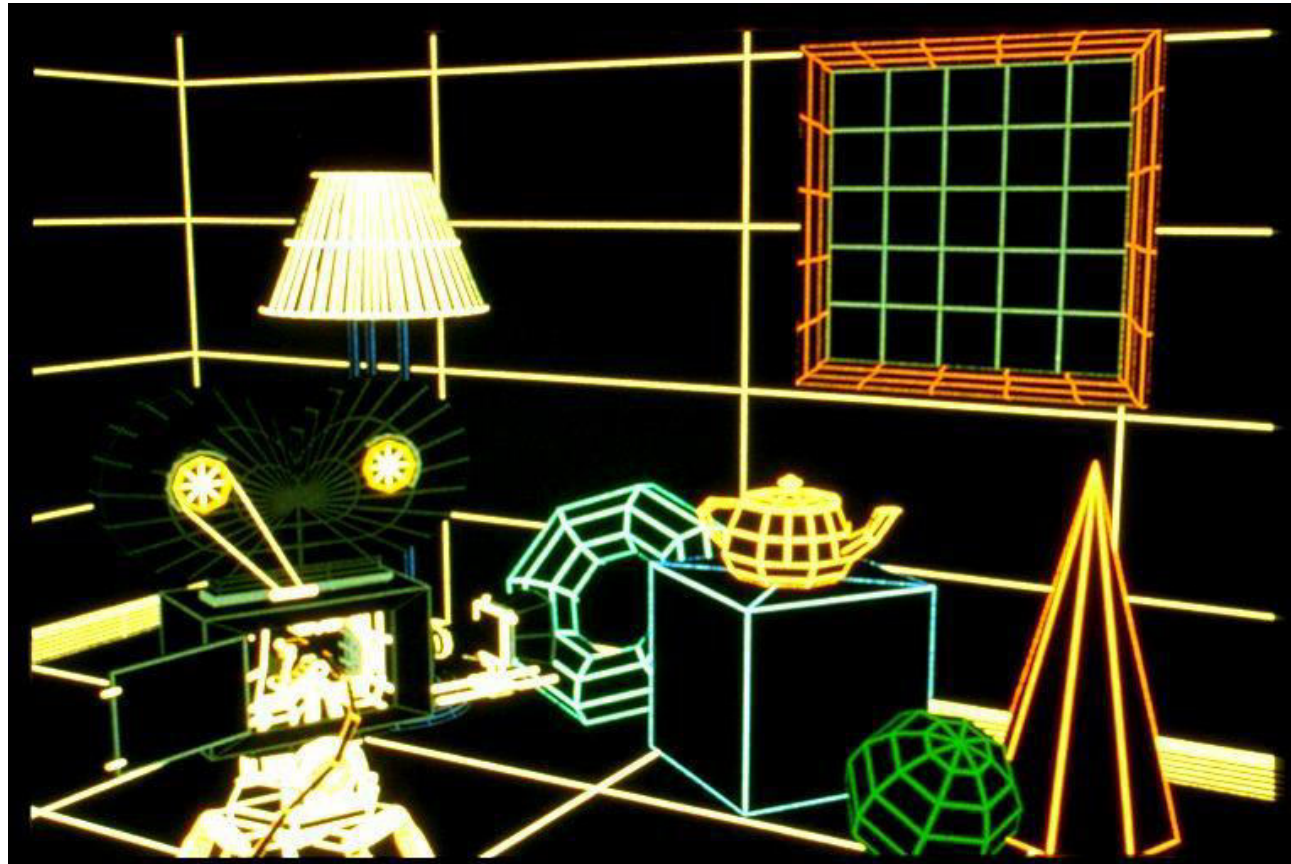
Perspective Projection

- ❑ We want to create a picture of the scene viewed from the camera
- ❑ We apply a perspective transformation to convert the 3D coordinates to 2D coordinates of the screen
- ❑ Objects far away appear smaller, closer objects appear bigger



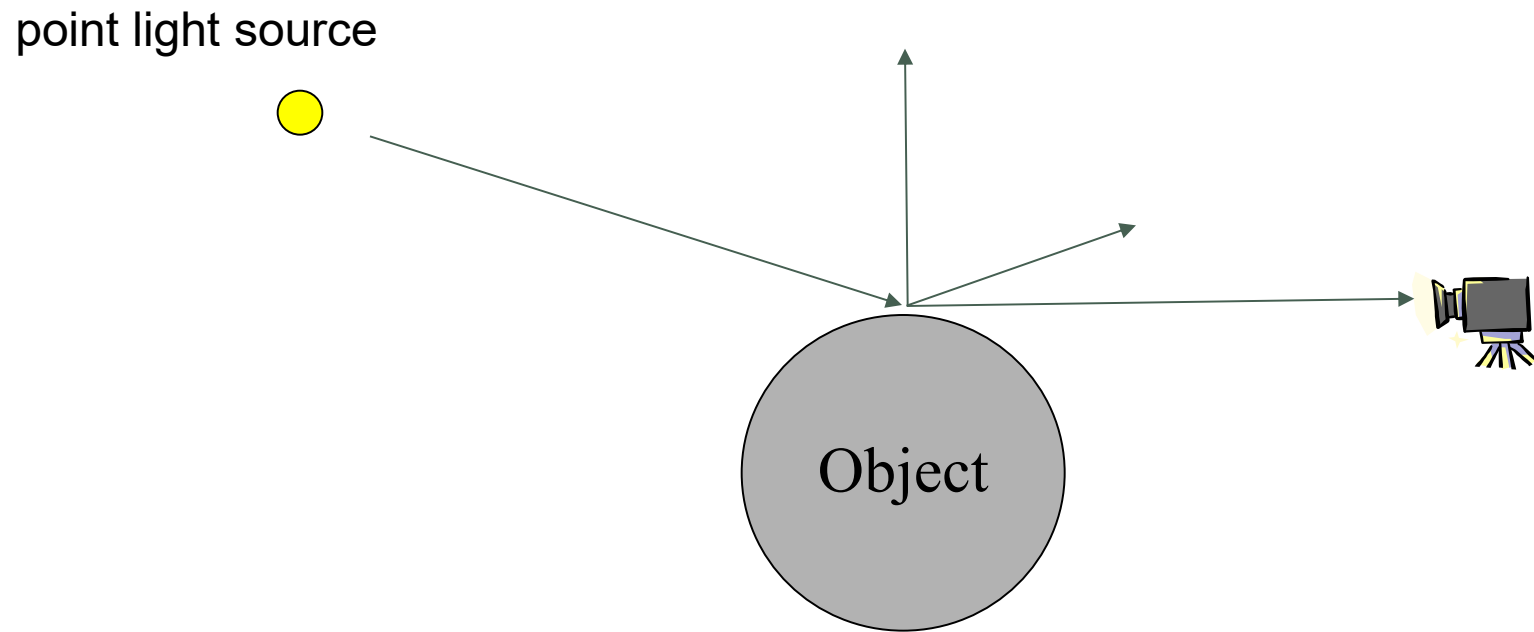
Hidden Surface Removal

- ❑ Objects occluded by other objects must not be drawn



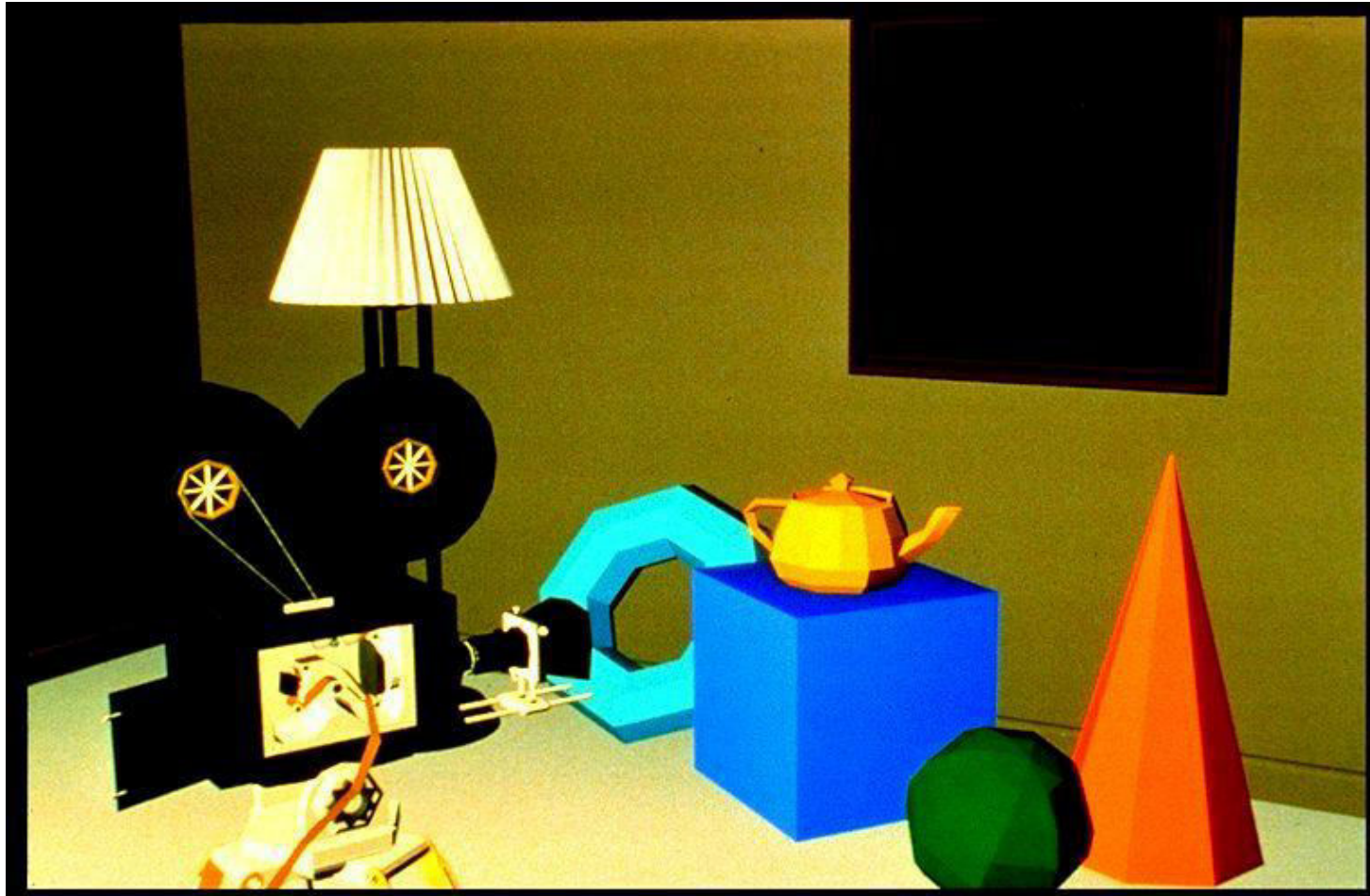
Shading

- ❑ Now we need to decide the colour of each pixels taking into account
 - the object's colour, lighting condition and the camera position



Shading – Flat Shading

- ❑ Objects coloured based on its own colour and the lighting condition
- ❑ One colour for one face



Gouraud shading, no specular highlights

- ❑ Lighting calculation per vertex

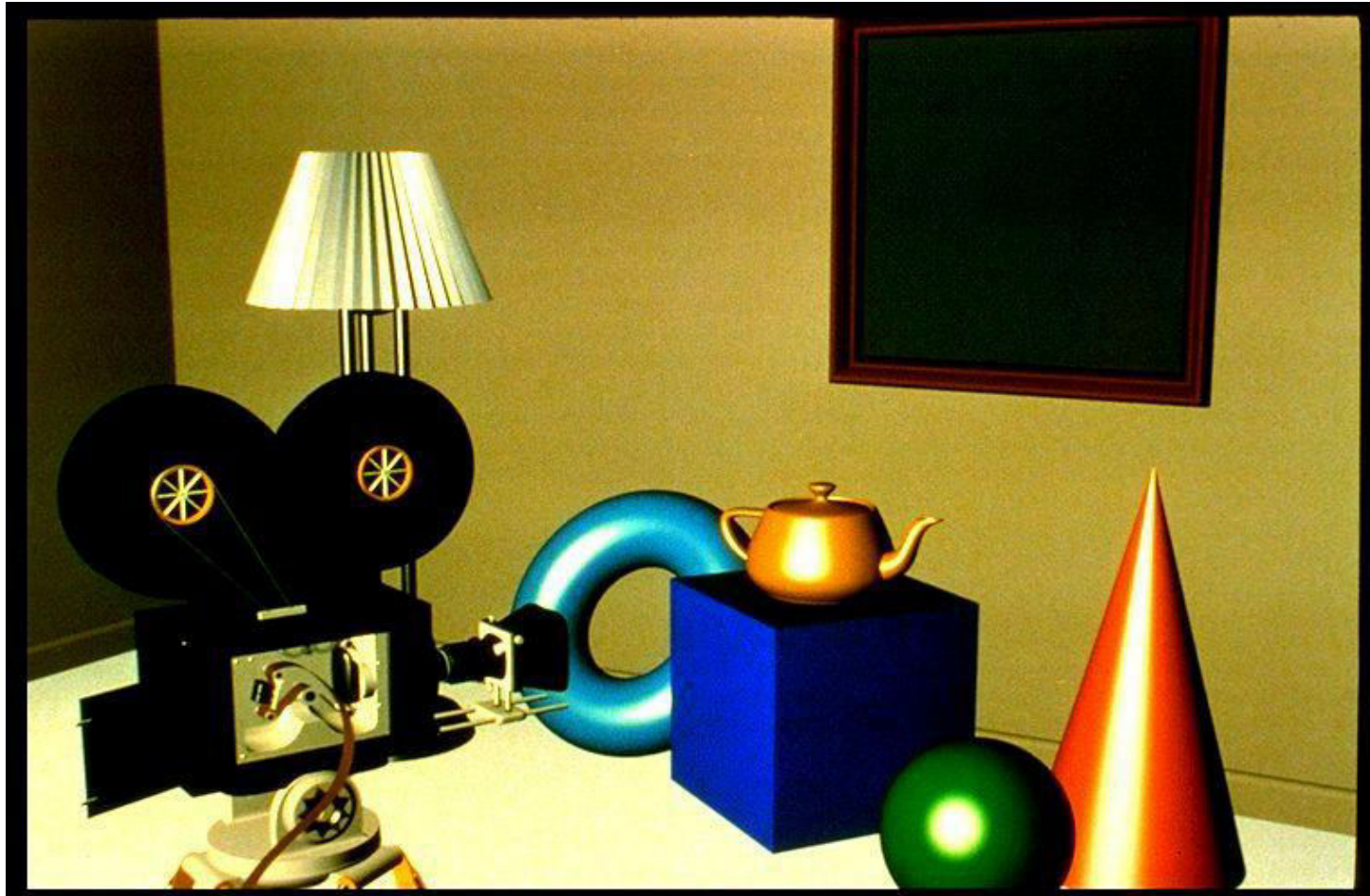


Specular highlights added

- ❑ Light perfectly reflected in a mirror-like way



Phong shading

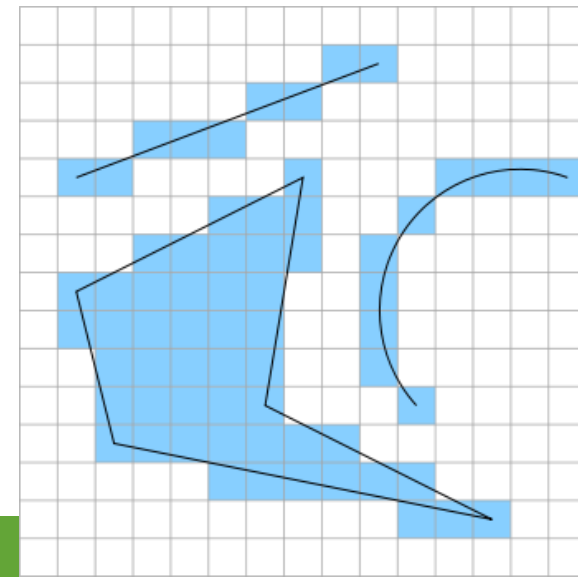


Rasterization

Converts the vertex information output by the geometry pipeline into pixel information needed by the video display

Aliasing: distortion artifacts produced when representing a high-resolution signal at a lower resolution.

Anti-aliasing : technique to remove aliasing



Texture mapping

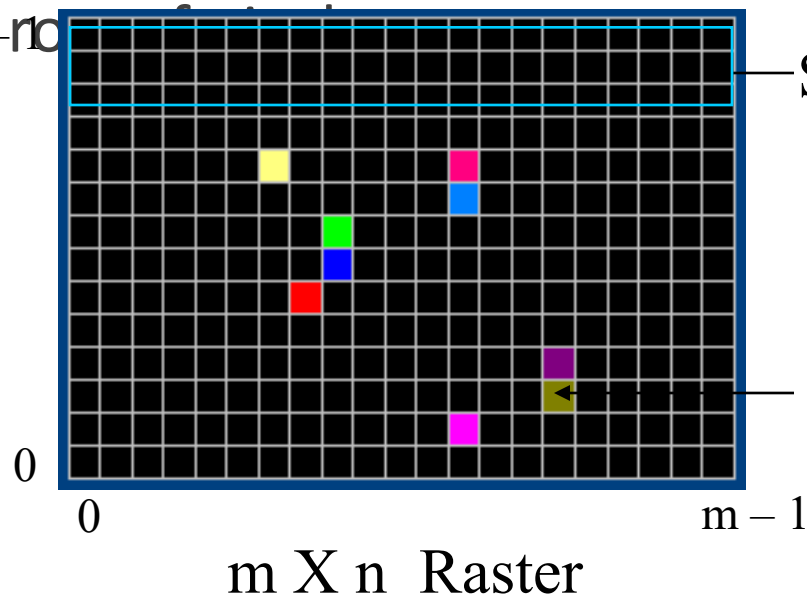


Reflections, shadows & Bump mapping



Graphics Definitions

- ❑ Most common display used for CG are Raster displays
 - CRT, Flat panel
- ❑ **Display surface** : On which image is presented
- ❑ **RASTER** : A rectangular array of points or dots on the surface
- ❑ **PIXEL** : One dot or picture element of the raster
- ❑ **SCAN LINE** : A row of pixels



We don't have any pixel like (1.2, 5.8). In raster device co-ordinates can take integer values only.

Graphics Definitions

Vertex

point in 3D

Edge

line in 3D connecting two vertices

Polygon/Face/Facet

arbitrary shape formed by connected vertices

fundamental unit of 3D computer graphics

Mesh

set of connected polygons forming a surface (or object)

:

Graphics Definitions

- ❑ The definition of image is stored in a 2 dimensional array in memory called **Frame Buffer**
- ❑ The display hardware produces the image line by line (**Raster Lines**)
- ❑ The high level process of taking a graphics model and producing an image is called **Rendering**
- ❑ The lower level process of drawing individual pixels or filling polygonal regions of the image by modifying the frame buffer is known as **Scan Conversion or Rasterization**

Frame Buffer

- ❑ a video output device that drives a video display from a memory containing the color for every pixel
- ❑ A frame buffer may be thought of as **computer memory** organized as a two-dimensional array with each (x,y) addressable location corresponding to one pixel.
- ❑ **Bit Planes** or **Bit Depth** is the number of bits corresponding to each pixel.
- ❑ A typical frame buffer resolution might be
 - 640 x 480 x 8 (x-resolution x y-resolution x color bit depth)
 - 800 x 600 x 16
 - 1024 x 768 x 24

Conclusion

- ❑ Computer graphics: generating 2D synthetic images of a 3D world represented in a computer.
- ❑ Main tasks:
 - *modeling*: (shape) creating and representing the geometry of objects in the 3D world
 - *rendering*: (light, perspective) generating 2D images of the objects

Recommended Texts

1. **Computer Graphics**

- Donald Hearn, M Pauline Baker

2. **Computer Graphics**

- R. Plastock, Zhigang Xiang
- (Schaum's Outline Series) McGraw Hill,

3. **Computer Graphics: Principles & Practice**

- J. D. Foley, A. van Dam, S. K. Feiner, J. F. Hughes

4. **Fundamentals of Computer Graphics**

- Steve Marschner, Peter Shirley

5. **Computer Graphics using OpenGL**

- F S Hill

☐ **Mathematics for Computer Graphics**

- John Vince

☐ **Mathematics for 3D Game Programming and Computer Graphics**

- Eric Lengyel

☐ **OpenGL Programming Guide**

- M. Woo, J. Neider, T. Davis, D. Shreiner

Modeling Transformations



2D TRANSFORMATIONS

Dr. Sk. Md. Masudul Ahsan

Professor

Modeling Transformations

- ❑ ~ places the objects defined by their own coordinate system into a common scene of world coordinate system.
- ❑ Moving or transforming an object can be achieved by
 - **Geometric transformation** – changes in orientation, size and shape of objects are accomplished by altering coordinate description of objects.
 - **Coordinate Transformation** – same effect is attained by holding the object stationary while coordinate system is transformed relative to object.

Use of Transformations

❑ Modeling

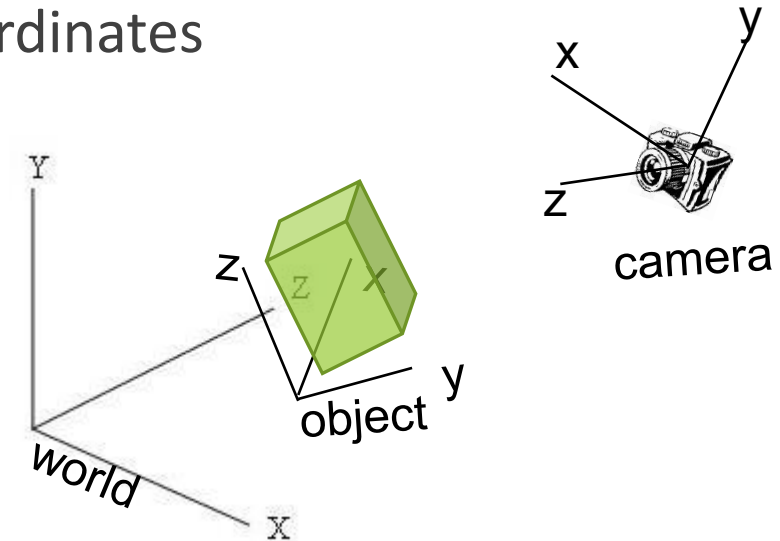
- Build complex models by positioning simple components
- Transform from object coordinates to world coordinates

❑ Viewing

- Placing the virtual camera in the world
- Specifying transformation from world to camera coordinates

❑ Animation

- Vary transformations over time to create motion



Translation – 2D

- If displacement is the vector $V = t_x I + t_y J$, new point $P'(x', y')$ can be found by adding translation distance (t_x, t_y) to the old point $P(x, y)$

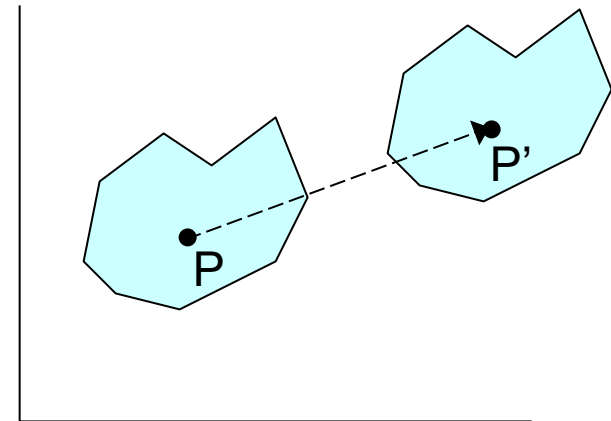
$$\text{i.e. } x' = x + t_x, \quad y' = y + t_y \quad \text{--- (i)}$$

Representing the points and displacement as a column vector

$$P = \begin{bmatrix} x \\ y \end{bmatrix} \quad P' = \begin{bmatrix} x' \\ y' \end{bmatrix} \quad T = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

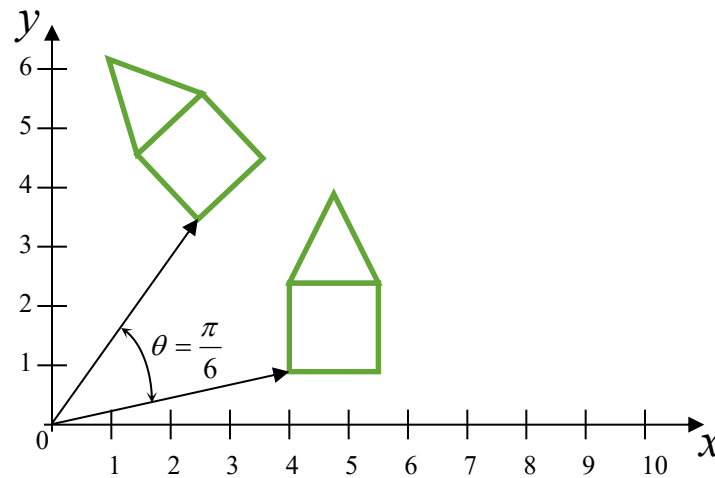
Eq. (i) becomes

$$P' = P + T$$



Rotation – 2D

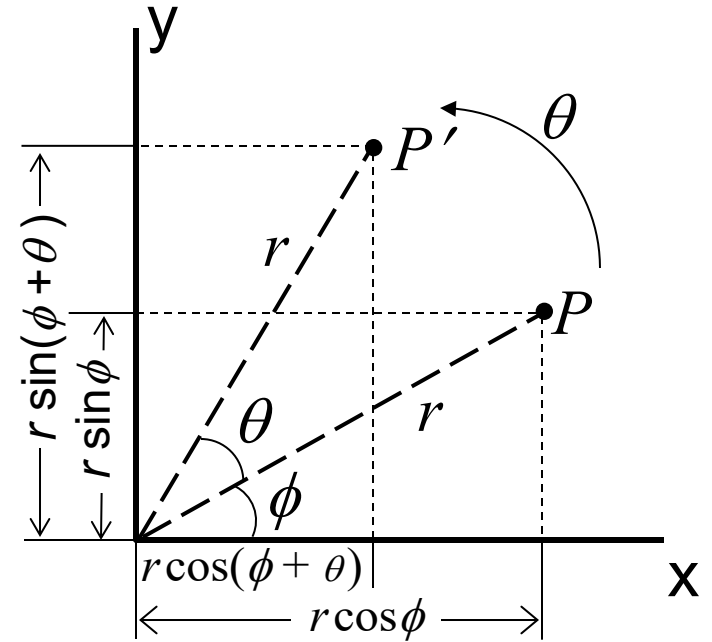
- ❑ In 2d rotation, object is repositioned along a circular path in xy plane.
 - Rotates all coordinates by a specified angle
- ❑ For rotation we specify
 - Rotation angle (θ)
 - Counter clockwise – positive angle
 - Clockwise – negative angle
 - Pivot point (x_r, y_r) – center of rotation
- ❑ What is the transformation eqn?



Rotation – 2D, origin as pivot point

$$P = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} r \cos \phi \\ r \sin \phi \end{bmatrix}$$

$$P' = \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} r \cos(\phi + \theta) \\ r \sin(\phi + \theta) \end{bmatrix}$$



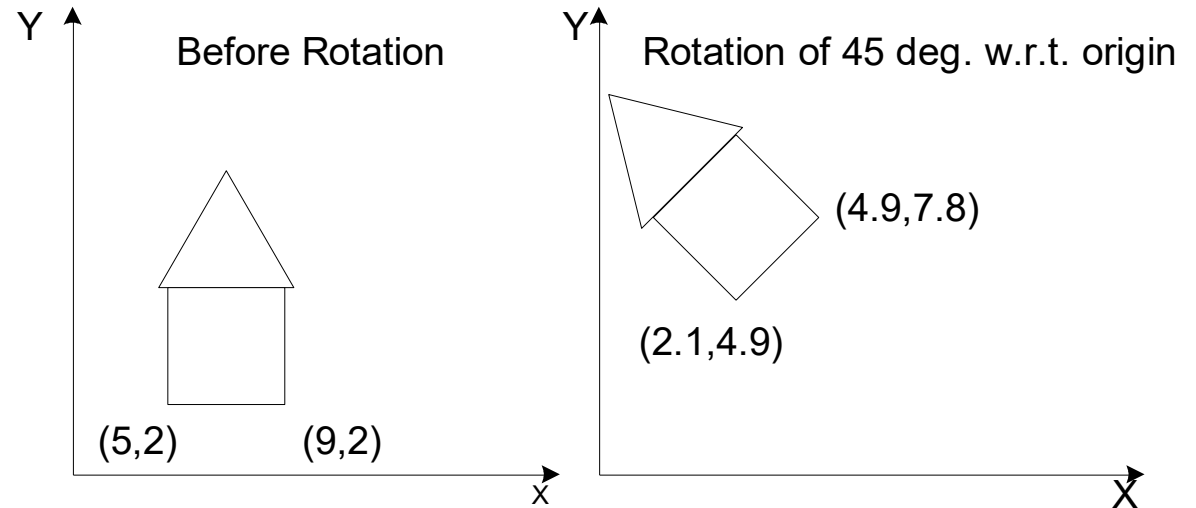
$$\text{expand } (\phi + \theta) \Rightarrow \begin{cases} x' = r \cos \phi \cos \theta - r \sin \phi \sin \theta \\ y' = r \cos \phi \sin \theta + r \sin \phi \cos \theta \end{cases}$$

$$\text{but } \begin{matrix} x = r \cos \phi \\ y = r \sin \phi \end{matrix} \Rightarrow \begin{matrix} x' = x \cos \theta - y \sin \theta \\ y' = x \sin \theta + y \cos \theta \end{matrix}$$

Rotation – 2D

$$x' = x \cdot \cos \theta - y \cdot \sin \theta$$

$$y' = x \cdot \sin \theta + y \cdot \cos \theta$$



This eq can be represented in matrix form as -

$$P' = R \cdot P$$
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \cdot \cos \theta - y \cdot \sin \theta \\ x \cdot \sin \theta + y \cdot \cos \theta \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

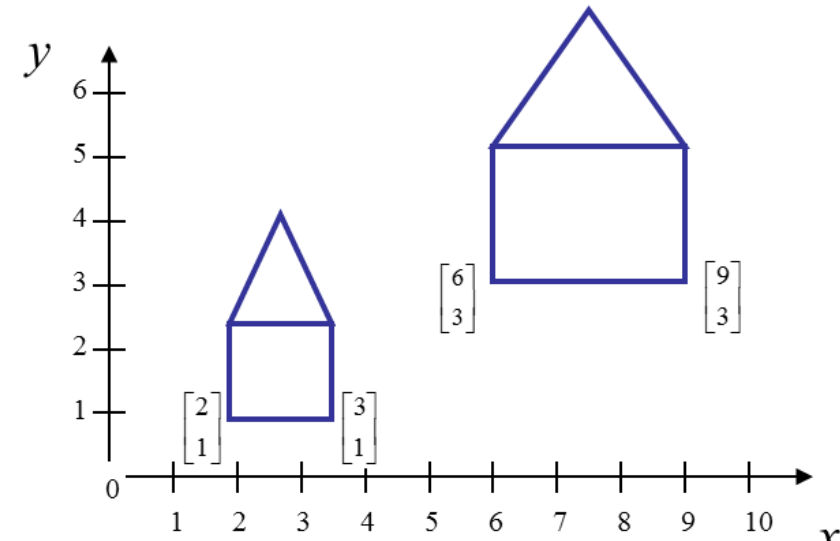
Scaling – 2D

- Scaling alters the size of an object
- Transformed coordinates $P'(x', y')$ are found by multiplying the coordinate values $P(x, y)$ of each vertex of object by **scaling factor** s_x and s_y .

ie.
$$x' = s_x \cdot x \quad , \quad y' = s_y \cdot y$$

- In matrix form

$$\begin{array}{ccc} P' & = & S \cdot P \\ \Downarrow & & \Downarrow \quad \Downarrow \\ \begin{bmatrix} x \cdot s_x \\ y \cdot s_y \end{bmatrix} & = & \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \end{array}$$



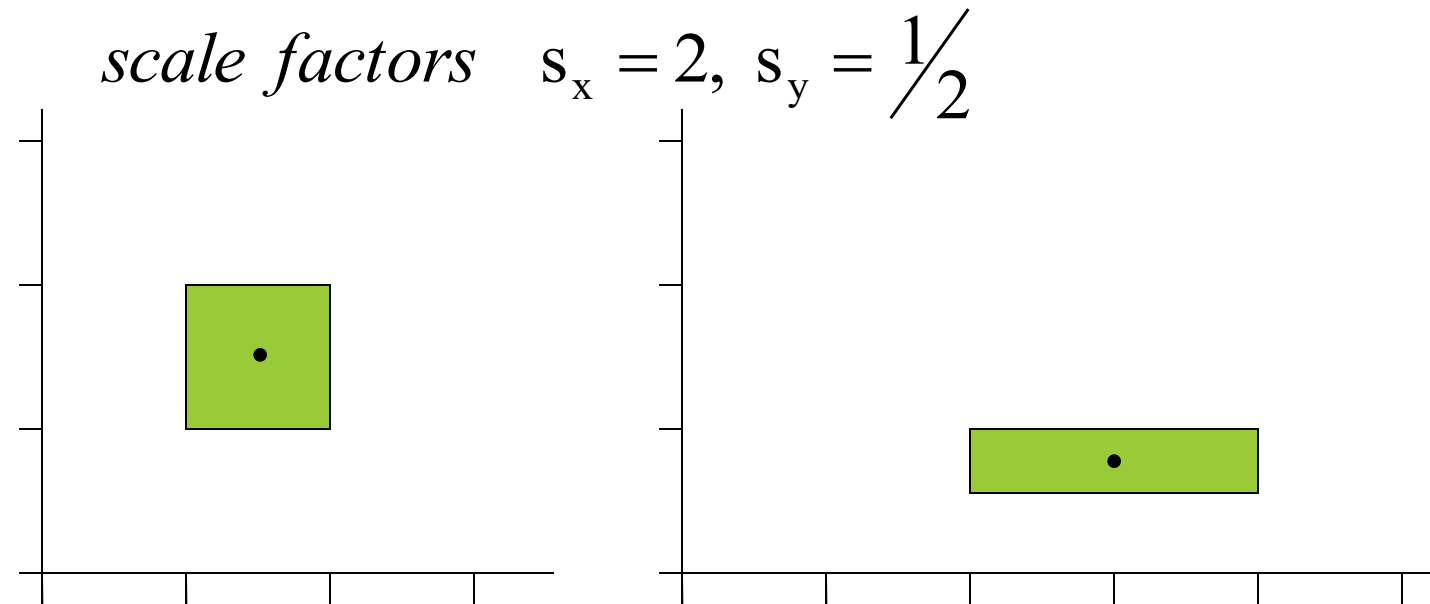
WATCH OUT: Objects grow and move!

Scaling – 2D

Types of Scaling:

- Differential ($s_x \neq s_y$)
- Uniform ($s_x = s_y$)

After scaling, new object is located at a different position relative to origin



Homogeneous Coordinates

- ❑ A point (x, y) can be re-written in **homogeneous coordinates** as (x_h, y_h, h)
- ❑ The **homogeneous parameter** h is a non-zero value such that:

$$x = \frac{x_h}{h} \qquad y = \frac{y_h}{h}$$

- ❑ We can then write any point (x, y) as (h_x, h_y, h)
- ❑ We can conveniently choose $h = 1$ so that (x, y) becomes $(x, y, 1)$

Why Homogeneous Coordinates?

- ❑ Mathematicians commonly use homogeneous coordinates as they allow scaling factors to be removed from equations
- ❑ Translation, scaling and rotation are expressed (non-homogeneously) as:
 - translation: $P' = P + T$
 - Scale: $P' = S \cdot P$
 - Rotate: $P' = R \cdot P$
- ❑ Composition is difficult to express, since translation not expressed as a matrix multiplication
- ❑ Using homogeneous coordinates allows us use matrix multiplication to calculate transformations – extremely efficient!

Homogeneous Translation

$$x' = x + t_x, \quad y' = y + t_y$$

□ Representing the point as a homogeneous column vector

➤ The translation of a point by (t_x, t_y) can be written in matrix form as:

$$\mathbf{x}' = \mathbf{T}_{(t_x, t_y)} \mathbf{x}$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Homogeneous Co-ordinates

➤ For rotation

$$x' = x \cdot \cos \theta - y \cdot \sin \theta \quad , \quad y' = x \cdot \sin \theta + y \cdot \cos \theta$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$R_{(\theta)}$$

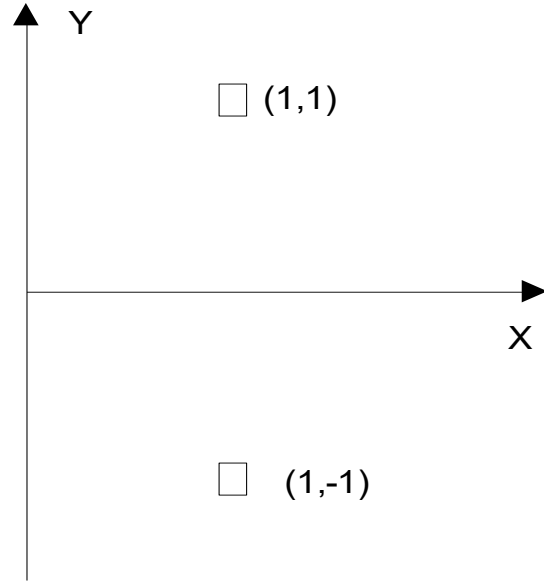
➤ For Scaling

$$x' = s_x \cdot x \quad , \quad y' = s_y \cdot y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$S_{(s_x, s_y)}$$

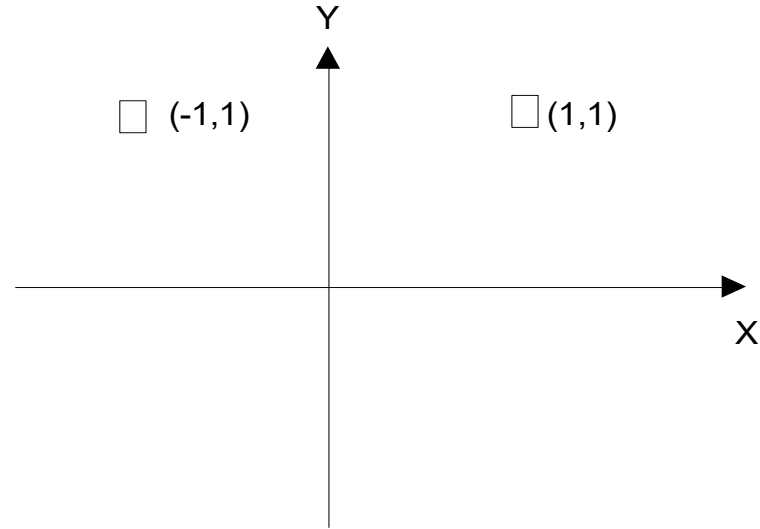
Mirror Reflection



Reflection about X - axis

$$x' = x \quad y' = -y$$

$$M_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Reflection about Y - axis

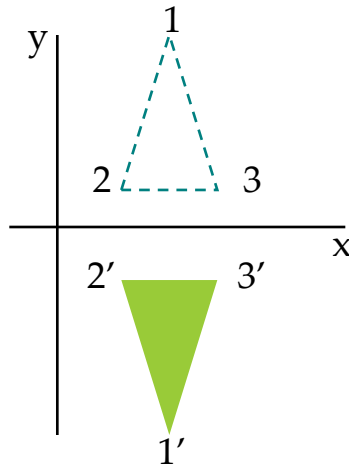
$$x' = -x \quad y' = y$$

$$M_y = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Reflection

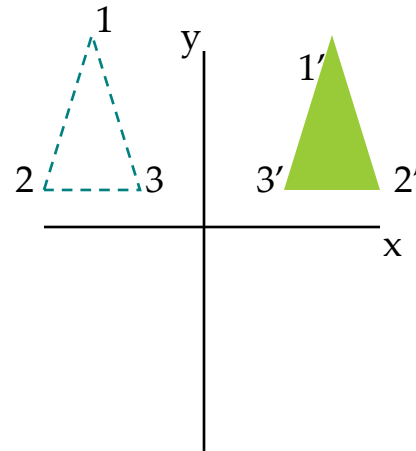
□ Reflection with respect to the axis

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



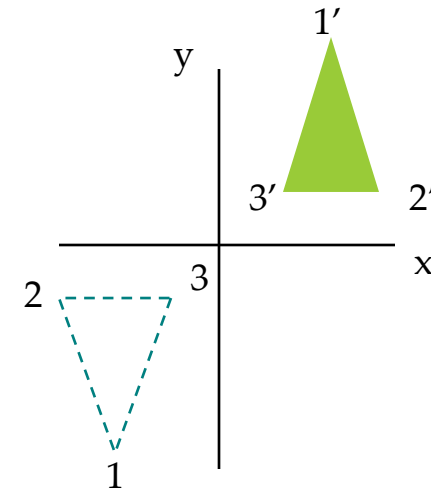
About x axis

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



About y axis

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



About origin

Shearing Transformation

$$x' = x + sh_x * y$$

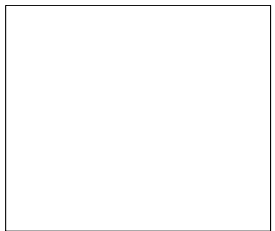
$$x' = x$$

$$x' = x + sh_x * y$$

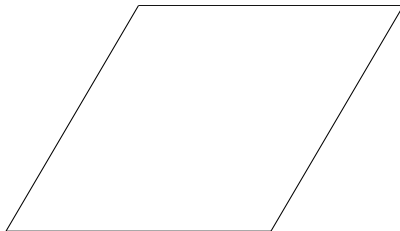
$$SH_{(sh_x)} = \begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$SH_{(sh_y)} = \begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

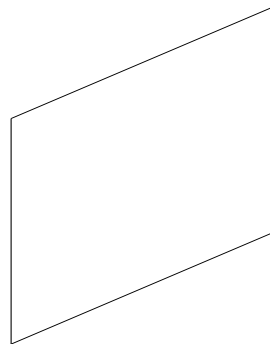
$$SH_{(sh_x, sh_y)} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



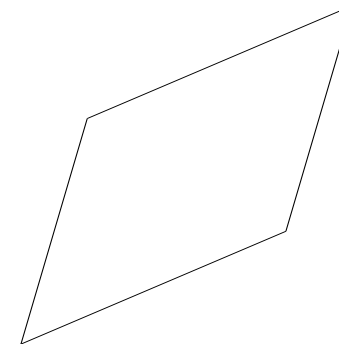
unit cube



Sheared in X
direction



Sheared in Y
direction



Sheared in both X
and Y direction

Inverse 2D - Transformations

☐ Transformations can easily be reversed using inverse transformations

$$\begin{aligned} \text{Translation: } T_{(tx,ty)}^{-1} &= \begin{bmatrix} 1 & 0 & -tx \\ 0 & 1 & -ty \\ 0 & 0 & 1 \end{bmatrix} \\ &= T_{(-tx,-ty)} \end{aligned}$$

$$\text{Rotation : } R_{(\theta)}^{-1} = R_{(-\theta)} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\text{Scaling : } S_{(sx,sy)}^{-1} = S_{(\frac{1}{sx},\frac{1}{sy})} = \begin{bmatrix} 1/sx & 0 & 0 \\ 0 & 1/sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\text{Mirror Ref: } M_x^{-1} = M_x$$

$$\mathbf{SH}_{(shx,shy)}^{-1} = \frac{1}{1 - shxshy} \begin{bmatrix} 1 & -shx & 0 \\ -shy & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Composite Transformation

❑ Transformations can be combined by matrix multiplication

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \left(\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$
$$p' = T(tx, ty) \quad R(\theta) \quad S(sx, sy) \quad p$$

❑ Efficiency with premultiplication

➤ Matrix multiplication is associative

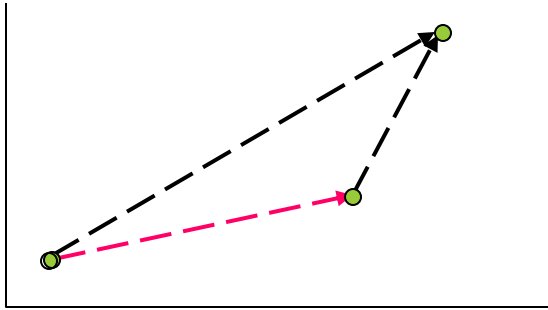
$$p' = (T \times (R \times (S \times p))) \quad \longrightarrow \quad p' = (T \times R \times S) \times p$$

Composite Transformation

- Translation: 2 successive ~ are applied to point P

$$P' = T(t_{x2}, t_{y2}) \cdot [T(t_{x1}, t_{y1}) \cdot P] = [T(t_{x2}, t_{y2}) \cdot T(t_{x1}, t_{y1})] \cdot P$$

Now, $T(t_{x2}, t_{y2}) \cdot T(t_{x1}, t_{y1}) =$

$$\begin{bmatrix} 1 & 0 & t_{x2} \\ 0 & 1 & t_{y2} \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & t_{x1} \\ 0 & 1 & t_{y1} \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & t_{x1} + t_{x2} \\ 0 & 1 & t_{y1} + t_{y2} \\ 0 & 0 & 1 \end{bmatrix} = T(t_{x1} + t_{x2}, t_{y1} + t_{y2})$$

- ie. Successive translations are additive

Composite Transformation - DIY

- ❑ Successive Rotation are additive, and
- ❑ Successive Scaling are multiplicative



Properties of rigid-body transformation

The following Matrix is Orthogonal if the upper left 2X2 matrix has the following properties

1.A) Each row are **unit vector**.

$$\text{sqrt}(r_{11}^* r_{11} + r_{12}^* r_{12}) = 1$$

B) Each column are unit vector.

$$\text{sqrt}(c_{11}^* c_{11} + c_{12}^* c_{12}) = 1$$

$$\begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

2.A) Rows will be **perpendicular** to each other

$$(r_{11}, r_{12}) \cdot (r_{21}, r_{22}) = 0$$

B) Columns will be perpendicular to each other

$$(c_{11}, c_{12}) \cdot (c_{21}, c_{22}) = 0$$

$$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

e.g. Rotation matrix is orthogonal

- **Orthogonal Transformation $\Rightarrow$ Rigid-Body Transformation**
- **For any orthogonal matrix $B \Rightarrow B^{-1} = B^T$**

Linear Transformations

□ Linear transformations are combinations of ...

- Scale
- Rotation
- Shear, and
- Mirror

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & 0 \\ c & d & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

□ Properties of linear transformations

- Satisfies:
- Origin maps to origin
- Lines map to lines
- Parallel lines remain parallel
- Ratios are preserved
- Closed under composition

$$T(s_1p_1 + s_2p_2) = s_1T(p_1) + s_2T(p_2)$$

Linear Transformations



Identity

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Affine Transformations

□ Affine transformations are combinations of

- Linear transformations, and
- Translations

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

□ Properties of affine transformations

- **Origin does not map to origin**
- Lines map to lines
- Parallel lines remain parallel (**Find how to prove it for exam**)
- Ratios are preserved
- Closed under composition



Projective Transformations

❑ Projective transformations...

- Affine transformations, and
- Projective warps

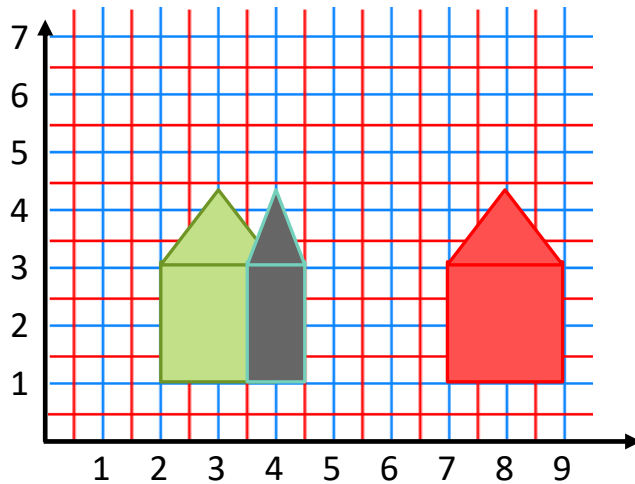
$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

❑ Properties of projective transformations

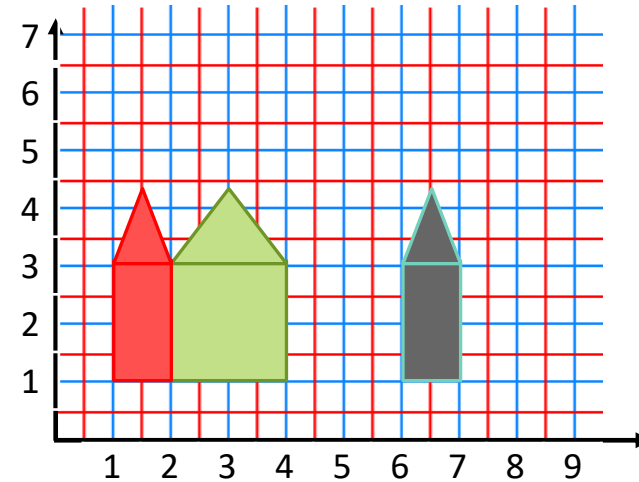
- Origin does not map to origin
- Lines map to lines
- Parallel lines do not necessarily remain parallel
- Ratios are not preserved
- Closed under composition

Commutativity of Transformation Matrices

- Matrix multiplication is not commutative
- Some non-commutative Compositions:
 - Non-uniform scale, Rotate
 - Translate, Scale
 - Rotate, Translate



Translate(5,0) -> Scale(0.5,1)



Scale(0.5,1) -> Translate(5,0)

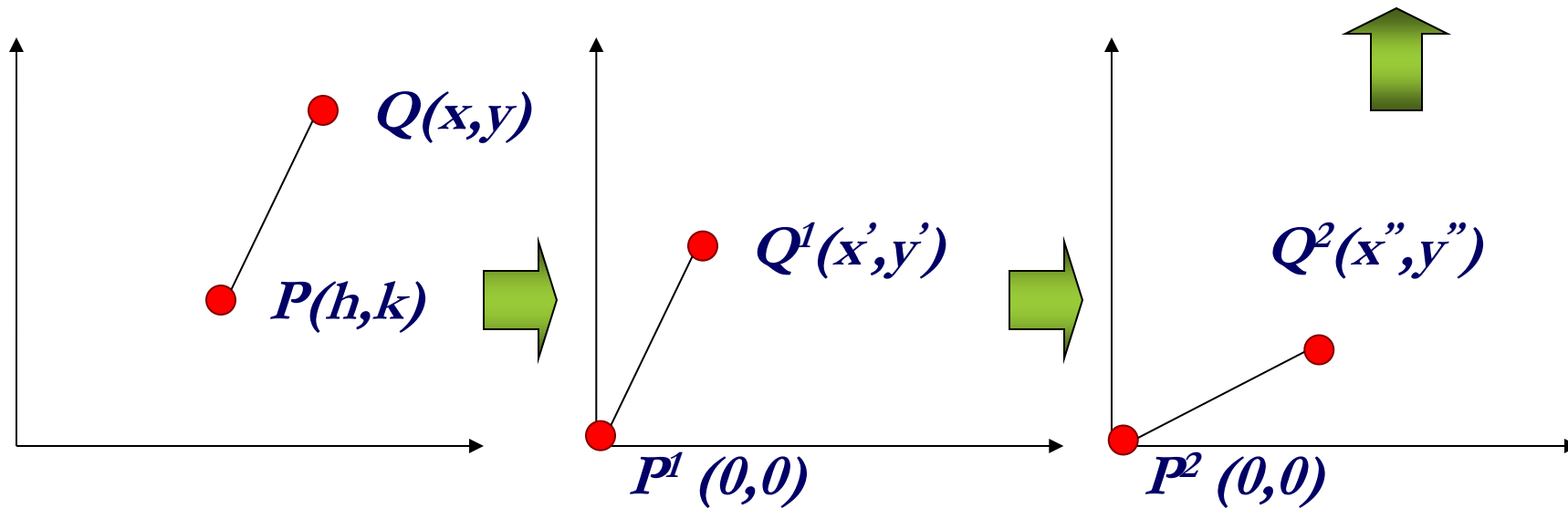
General Pivot point, $P(h,k)$, Rotation of θ : $R_{\theta,P}$

Step 1: Translate $P(h,k)$ to origin

Step 2: Rotate θ w.r.t to origin

Step 3: Translate $(0,0)$ to $P(h,k)$

$$R_{\theta,P} = T(h, k) * R_{\theta} * T(-h, -k)$$



General Pivot point, P(h,k), Rotation of θ : $R_{\theta,P}$

$$\begin{aligned} R_{\theta,p} &= T_{(h,k)} \bullet R_{\theta} \bullet T_{(-h,-k)} \\ R_{\theta,p} &= \begin{bmatrix} 1 & 0 & h \\ 0 & 1 & k \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -h \\ 0 & 1 & -k \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \cos \theta & -\sin \theta & h \\ \sin \theta & \cos \theta & k \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -h \\ 0 & 1 & -k \\ 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} \cos \theta & -\sin \theta & h(1-\cos \theta) + k \sin \theta \\ \sin \theta & \cos \theta & k(1-\cos \theta) - h \sin \theta \\ 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

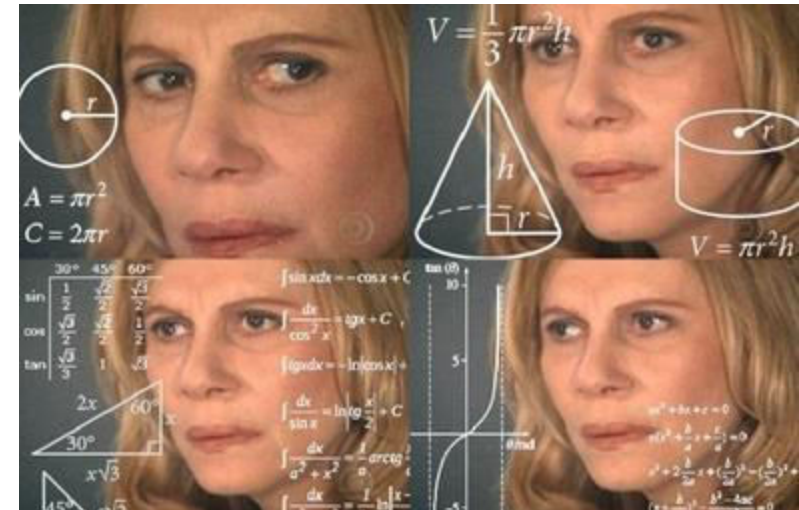
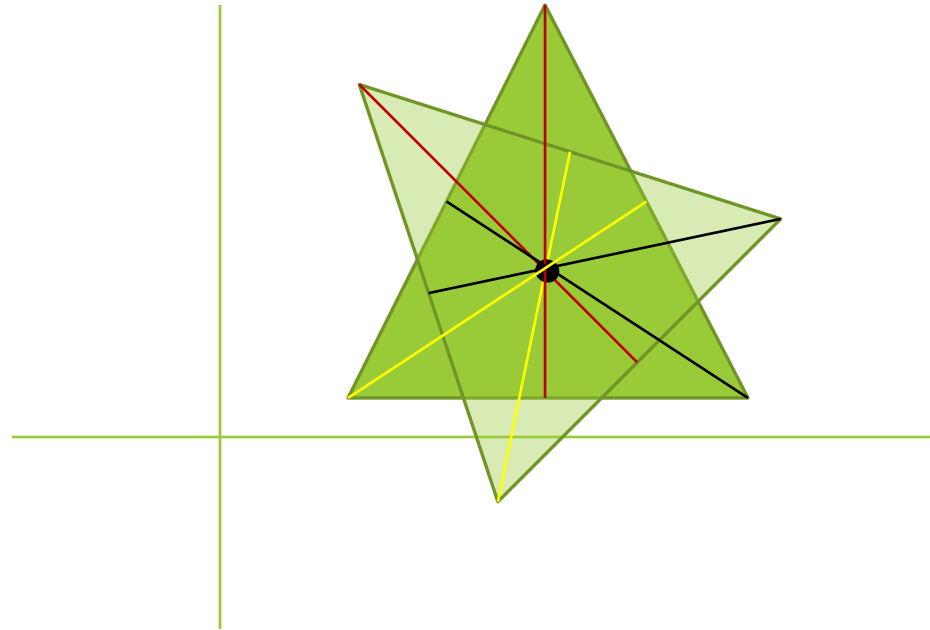
Example

□ Rotate the point P(4,5) 45° about the point A(1,1)

$$\begin{aligned}
 P' &= R_{45^\circ, A} \bullet P = T_{(1,1)} \bullet R_{45^\circ} \bullet T_{(-1,-1)} \bullet P \\
 &= \begin{bmatrix} 1 & 0 & h \\ 0 & 1 & k \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -h \\ 0 & 1 & -k \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & h(1-\cos \theta) + k \sin \theta \\ \sin \theta & \cos \theta & k(1-\cos \theta) - h \sin \theta \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix} \\
 &= \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 1(1-\frac{1}{\sqrt{2}})+1 \cdot \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 1(1-\frac{1}{\sqrt{2}})-1 \cdot \frac{1}{\sqrt{2}} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 1 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 1-\sqrt{2} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 5 \\ 1 \end{bmatrix} \\
 &= \begin{bmatrix} 1-\frac{1}{\sqrt{2}} \\ 1+\frac{7}{\sqrt{2}} \\ 1 \end{bmatrix}
 \end{aligned}$$

DIY

- ❑ Rotate the triangle keeping the center fixed



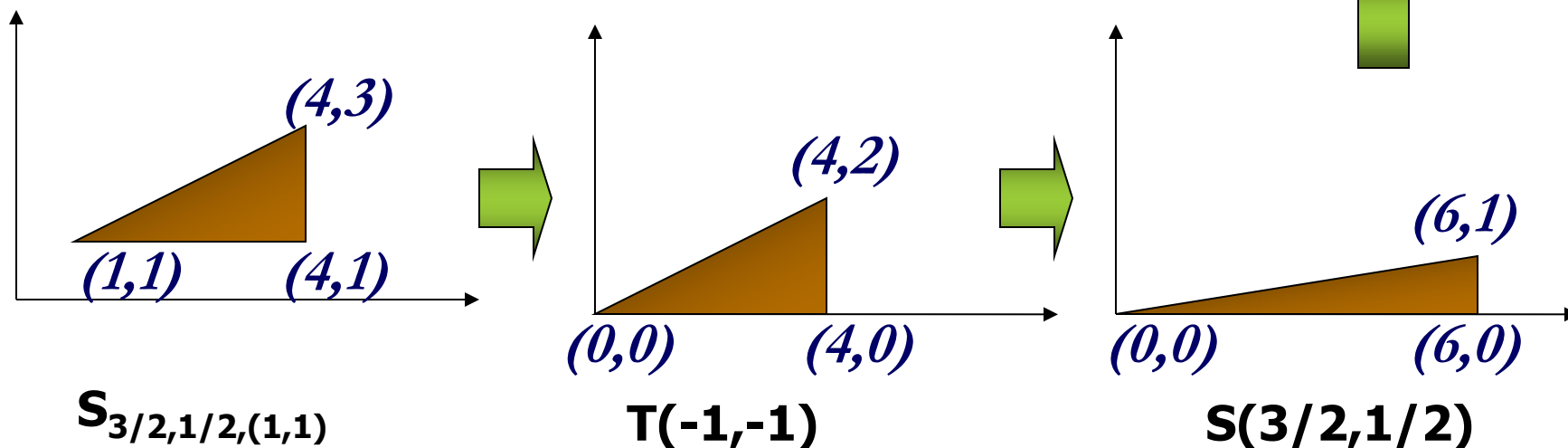
Fixed point P(h,k) Scaling : $S_{sx,sy,p}$

Step 1: Translate P(h,k) to origin

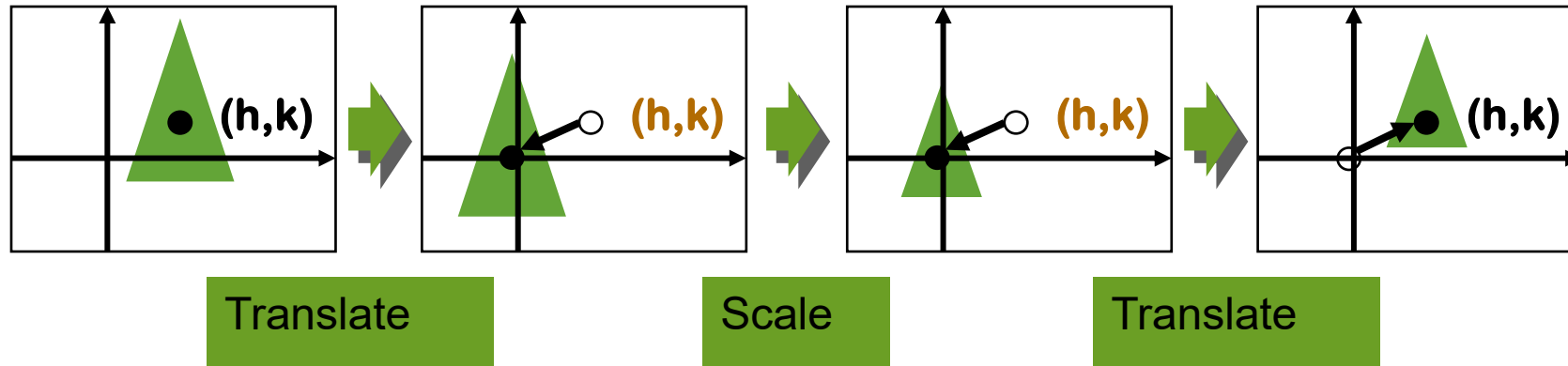
Step 2: Scale $S(s_x, s_y)$ w.r.t origin

Step 3: Translate (0,0) to P(h,k)

$$S_{sx,sy,p} = T(h, k) * S(s_x, s_y) * T(-h, -k)$$



Fixed point P(h,k) Scaling : $S_{s_x, s_y, p}$



$$T(h, k) \cdot S(s_x, s_y) \cdot T(-h, -k) = S(s_x, s_y, h, k)$$

$$\begin{bmatrix} 1 & 0 & h \\ 0 & 1 & k \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -h \\ 0 & 1 & -k \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & h(1-s_x) \\ 0 & s_y & k(1-s_y) \\ 0 & 0 & 1 \end{bmatrix}$$

Reflection about line L, M_L

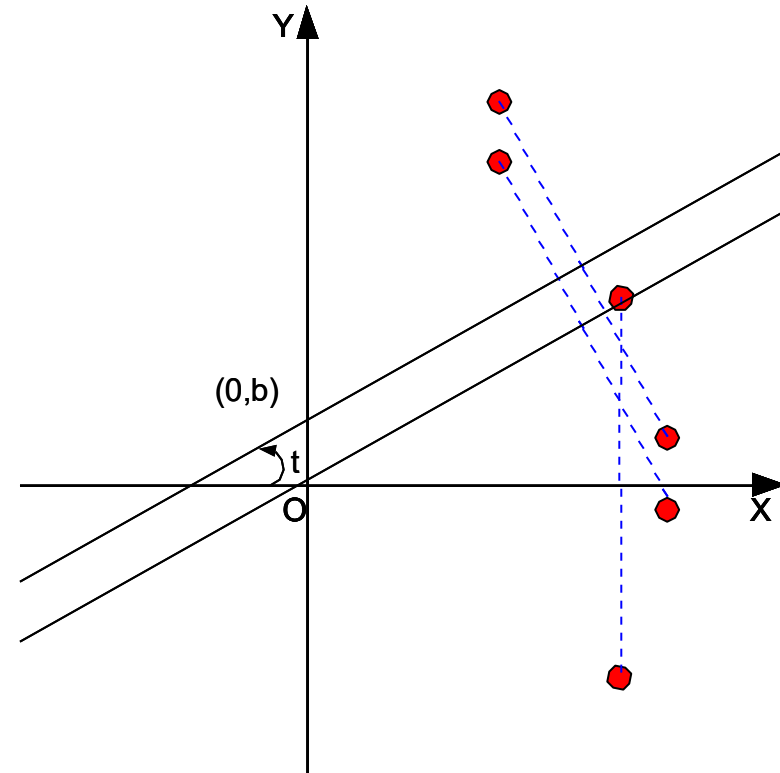
Step 1: Translate $(0,b)$ to origin

Step 2: Rotate $-\theta$ degrees

Step 3: Mirror reflect about X-axis

Step 4: Rotate θ degrees

Step 5: Translate origin to $(0,b)$



$$M_L = T(0, b) * R(\theta) * M_x * R(-\theta) * T(0, -b)$$

Reflection about line L, M_L

$$\mathbf{M}_L = \mathbf{T}(\mathbf{0}, \mathbf{b}) * \mathbf{R}(\theta) * \mathbf{M}_x * \mathbf{R}(-\theta) * \mathbf{T}(\mathbf{0}, -\mathbf{b})$$

$$M_L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & b \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -b \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & b \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ \sin \theta & -\cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -b \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos^2 \theta - \sin^2 \theta & 2 \sin \theta \cos \theta & 0 \\ 2 \sin \theta \cos \theta & -(\cos^2 \theta - \sin^2 \theta) & b \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -b \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \frac{1}{1 + \tan^2 \theta} \begin{bmatrix} 1 - \tan^2 \theta & 2 \tan \theta & 0 \\ 2 \tan \theta & -(1 - \tan^2 \theta) & b(1 + \tan^2 \theta) \\ 0 & 0 & 1 + \tan^2 \theta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & -b \\ 0 & 0 & 1 \end{bmatrix}$$

$$\sin(2\theta) = 2 \sin \theta \cos \theta = (\sin \theta + \cos \theta)^2 - 1 = \frac{2 \tan \theta}{1 + \tan^2 \theta}$$

$$\cos(2\theta) = \cos^2 \theta - \sin^2 \theta = 2 \cos^2 \theta - 1 = 1 - 2 \sin^2 \theta = \frac{1 - \tan^2 \theta}{1 + \tan^2 \theta}$$

$$M_L = \frac{1}{m^2 + 1} \begin{bmatrix} 1 - m^2 & 2m & -2bm \\ 2m & m^2 - 1 & 2b \\ 0 & 0 & m^2 + 1 \end{bmatrix}$$

Problem

Show that two successive reflections about any line (two lines) in the xy plane that intersects the coordinate origin is equivalent to a rotation in the xy plane about the origin

$$M_{L1} = \begin{bmatrix} \cos^2 \theta_1 - \sin^2 \theta_1 & 2 \sin \theta_1 \cos \theta_1 & 0 \\ 2 \sin \theta_1 \cos \theta_1 & -(\cos^2 \theta_1 - \sin^2 \theta_1) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos 2\theta_1 & \sin 2\theta_1 \\ \sin 2\theta_1 & -\cos 2\theta_1 \end{bmatrix} = \begin{bmatrix} \cos A & \sin A \\ \sin A & -\cos A \end{bmatrix}$$

$$M_{L2} = \begin{bmatrix} \cos B & \sin B \\ \sin B & -\cos B \end{bmatrix}$$

$$M_{L2}M_{L1} = \begin{bmatrix} \cos A \cos B + \sin A \sin B & \sin A \cos B - \cos A \sin B \\ \cos A \sin B - \sin A \cos B & \cos A \cos B + \sin A \sin B \end{bmatrix} = \begin{bmatrix} \cos(A-B) & \sin(A-B) \\ -\sin(A-B) & \cos(A-B) \end{bmatrix}$$

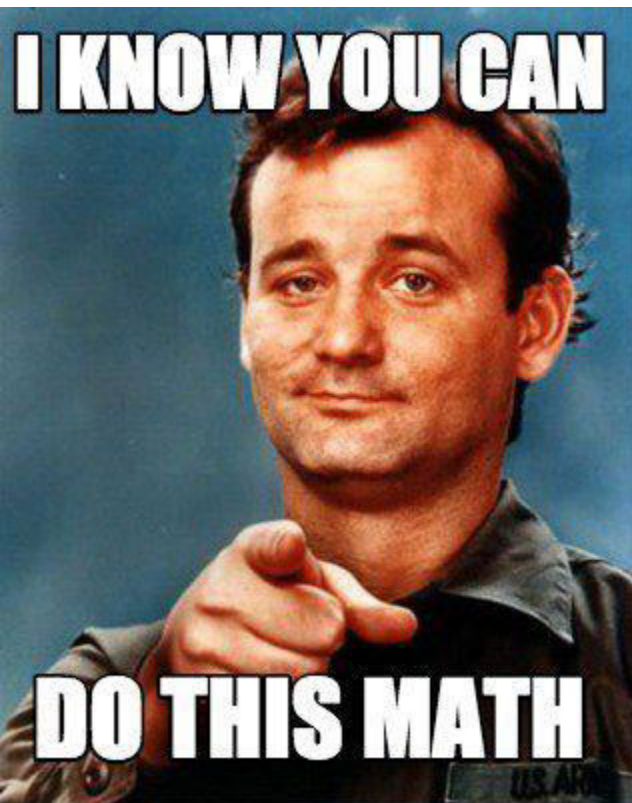


Problem

Reflecting a point $P = \langle x, y \rangle$ about an arbitrary axis passing through points $P_a = \langle x_a, y_a \rangle$ and $P_b = \langle x_b, y_b \rangle$, show that

$$x_5 = (x - x_a) \cdot \cos(2\theta) + (y - y_a) \cdot \sin(2\theta) + x_a$$

$$y_5 = (x - x_a) \cdot \sin(2\theta) - (y - y_a) \cdot \cos(2\theta) + y_a$$



Programming Techniques for Computer

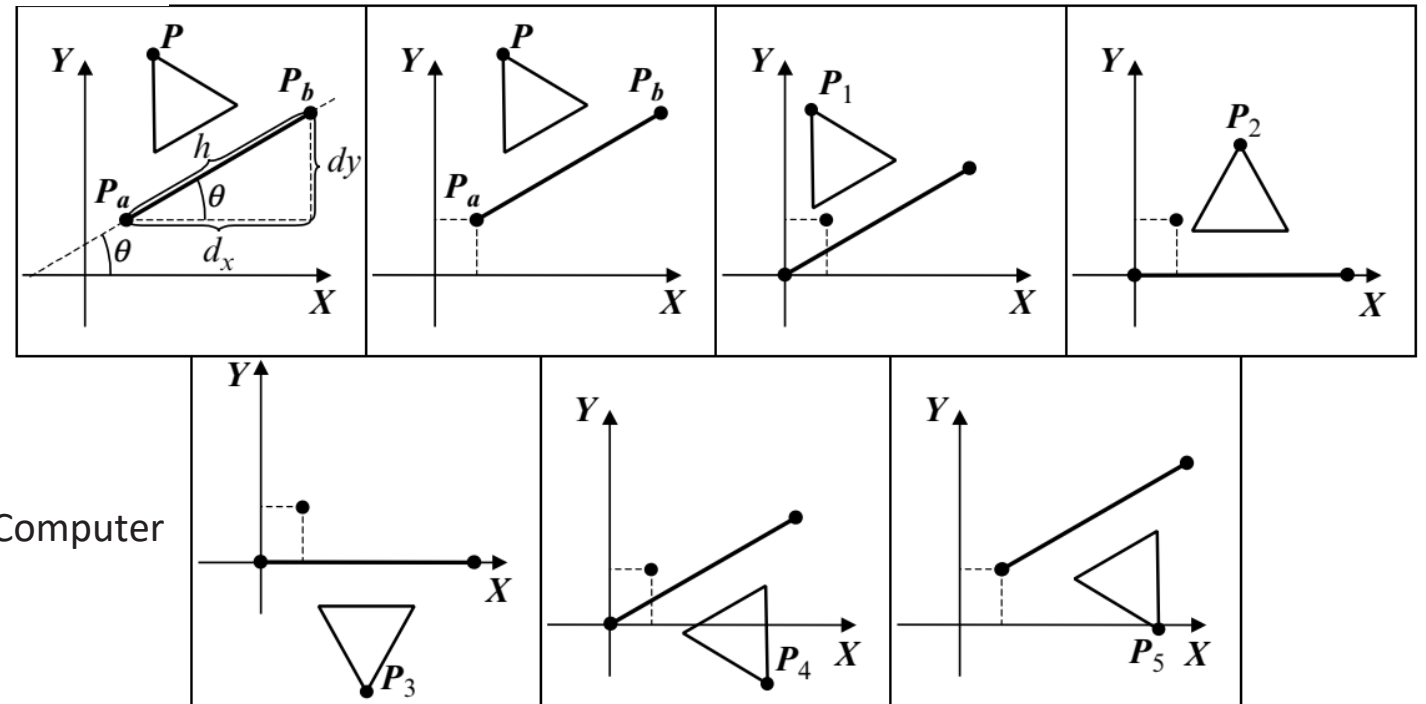
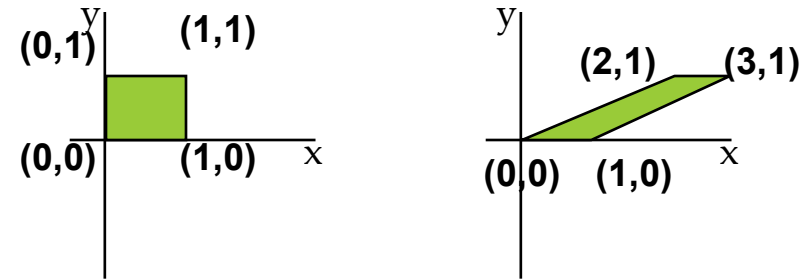


FIGURE 5.12. A reflection transformation about an arbitrary axis.

Shear – with ref line

❑ Converted to a parallelogram

$$x' = x + sh_x \cdot y, \quad y' = y \quad \begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

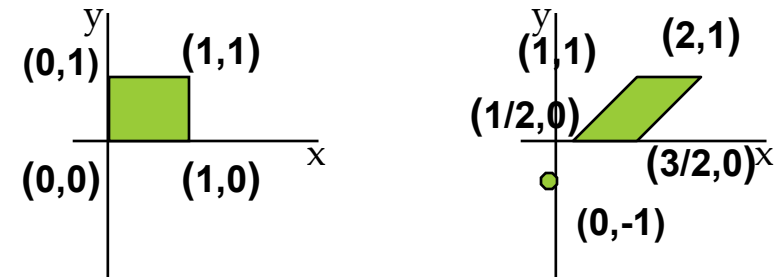


($Sh_x=2$)

❑ Transformed to a shifted parallelogram

($Y = Y_{ref}$)

$$x' = x + sh_x \cdot (y - y_{ref}), \quad y' = y \quad \begin{bmatrix} 1 & sh_x & -sh_x \cdot y_{ref} \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



($Sh_x=1/2, y_{ref}=-1$)

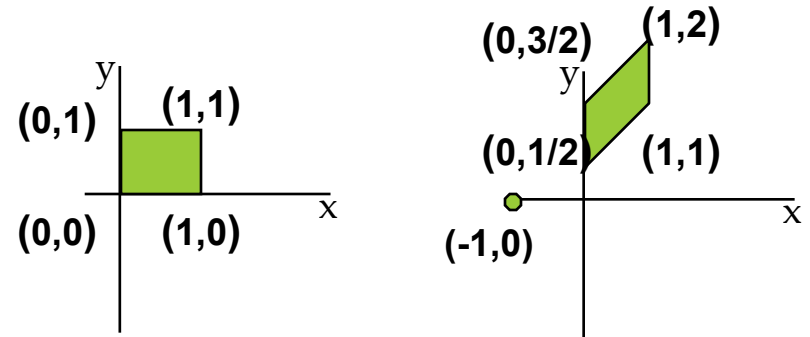
Shear – with ref line

❑ Transformed to a shifted parallelogram

($X = X_{ref}$)

$$\begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & -sh_y \cdot x_{ref} \\ 0 & 0 & 1 \end{bmatrix}$$

$$x' = x, \quad y' = sh_y \cdot (x - x_{ref}) + y$$



($Sh_y=1/2, x_{ref}=-1$)

Problems to be solved:

Schaum's outline series:

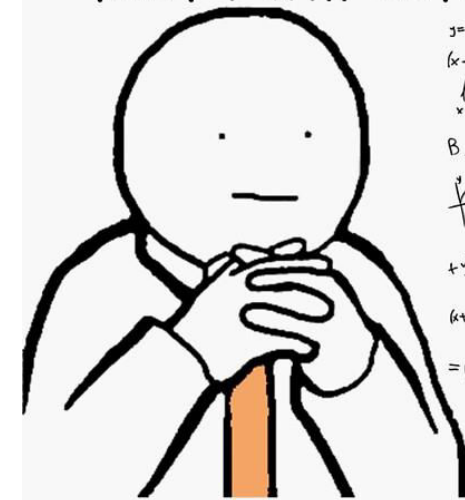
Problems:

- ❑ 4.2
- ❑ 4.3, 4.4, 4.5 $\Rightarrow R_{\theta,P}$
- ❑ 4.6, 4.7, 4.8 $\Rightarrow S_{sx,sy,,P}$
- ❑ 4.9, 4.10, 4.11, 4.21 $\Rightarrow M_L$
- ❑ 4.12 $\Rightarrow$ Shearing
- ❑ 4.16, 4.18, 4.29, 4.34

Hierarchical modeling

<https://www.youtube.com/watch?v=4iNJdWXsFQ4>

I'm still waiting for the day
that i will actually use this



$$\begin{aligned} y &= \frac{\Delta x}{\Delta z} \\ (x-y) &= \sqrt{\frac{\sum (x-m)^2}{n-1}} \quad Q = \sqrt{\frac{100}{100}} \approx 3.14 \\ \lim_{x \rightarrow 1} \frac{x^2-2}{x^2+3} &= \frac{1-2}{1+3} = \frac{-1}{4} = -0.25 \\ P &= r_{11} \quad \frac{1}{a} = \frac{1}{\tan \theta} \\ B &= n-1 \quad 4x = 8-3y^2 \quad e=2.79 \quad P = \sum_{i=0}^{\infty} x^i \\ y &= 2x^2+3x \quad \frac{1}{x} = \frac{1}{x} \\ (x+y)^2 &= \left(\frac{y}{2}\right)^2 = x^2+2ax+a^2 \quad \frac{1}{1-\tan \theta} \\ +y^2 &= 2 \quad \frac{\Delta x}{\Delta y} = \frac{1}{m} \frac{\Delta x}{\Delta y} \\ (u+a) &= \sin \theta \quad e = \cos \theta + i \sin \theta \quad \int \frac{1}{x+a} \\ &= (y-1)^2 \quad \sin \theta = \frac{1}{2} \quad \frac{1}{x^2} = \frac{1}{x^2} \end{aligned}$$

in real life

3D Modelling Transform



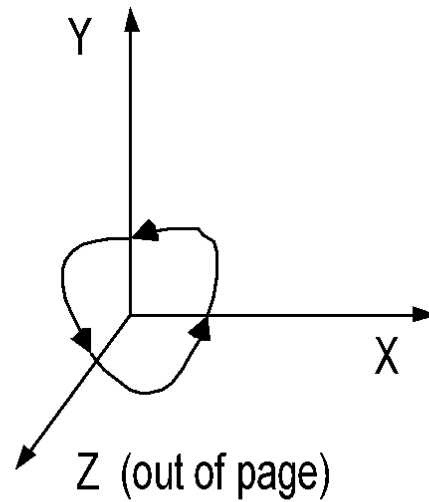
Dr. Sk. Md. Masudul Ahsan
Professor

3D Transformations

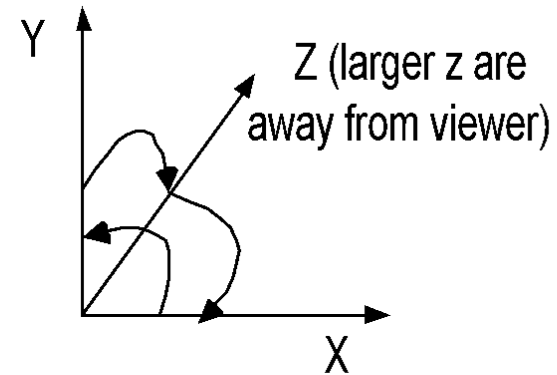
- ❑ Basics of 3D geometry
- ❑ Basic 3D Transformations
- ❑ Composite Transformations

Orientation

- ❑ Right-hand orientation
- ❑ Left-hand orientation



Right-handed orientation



Left-handed orientation

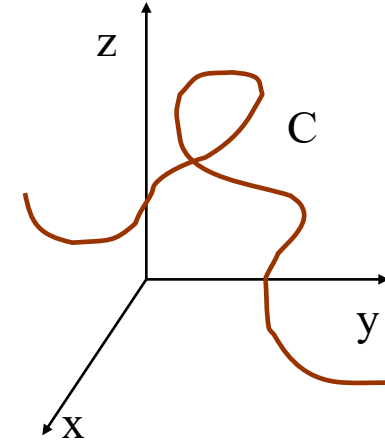
3D Equation of Curve & Line

□ Curve

➤ Non-Parametric form $C: y = f(x), z = g(x)$

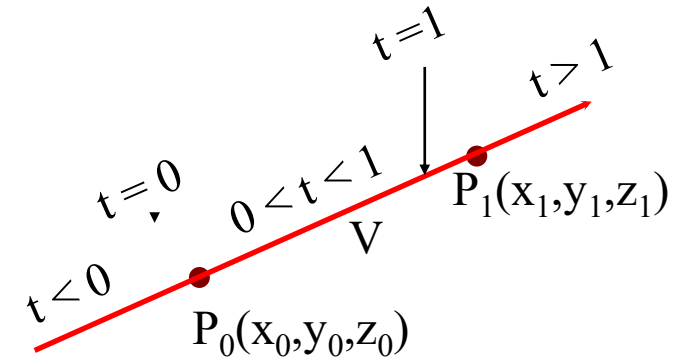
➤ Parametric form

$$\begin{aligned} x &= f(t) \\ C: \quad y &= g(t) \quad a \leq t \leq b \\ z &= h(t) \end{aligned}$$



□ Line

$$\begin{aligned} \overline{V} &= \overline{P_0 P_1} = P_1 - P_0 \\ x &= x_0 + (x_1 - x_0) \times t \\ L: \quad y &= y_0 + (y_1 - y_0) \times t \quad 0 \leq t \leq 1 \\ z &= z_0 + (z_1 - z_0) \times t \\ L &= P_0 + t(P_1 - P_0) = P_0 + t\overline{V} \end{aligned}$$



3D Equation of Surface & Plane

□ Parametric equations of Surface & Plane

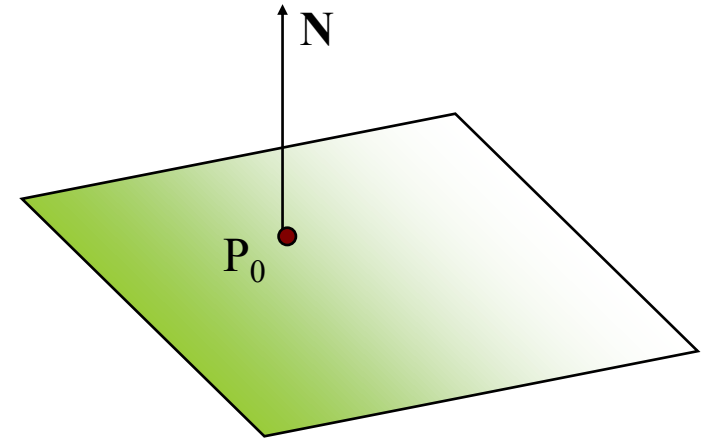
□ Surface

$$S: \begin{aligned} x &= f(s, t) \\ y &= g(s, t) \\ z &= h(s, t) \end{aligned} \quad \begin{aligned} a &\leq s \leq b \\ c &\leq t \leq d \end{aligned}$$

□ Plane : with Normal, **N**

$$Ax + By + Cz + D = 0$$

$$\overline{N} = A\hat{i} + B\hat{j} + C\hat{k}$$



3D Plane

□ Ways of defining a plane

1. 3 points P_0, P_1, P_2 on the plane
2. Plane Normal $\mathbf{N}$ & P_0 on plane
3. Plane Normal $\mathbf{N}$ & a vector $\mathbf{V}$ on the plane

Plane Passing through P_0, P_1, P_2

$$\overrightarrow{N} = \overrightarrow{P_0P_1} \times \overrightarrow{P_0P_2} = A\hat{i} + B\hat{j} + C\hat{k}$$

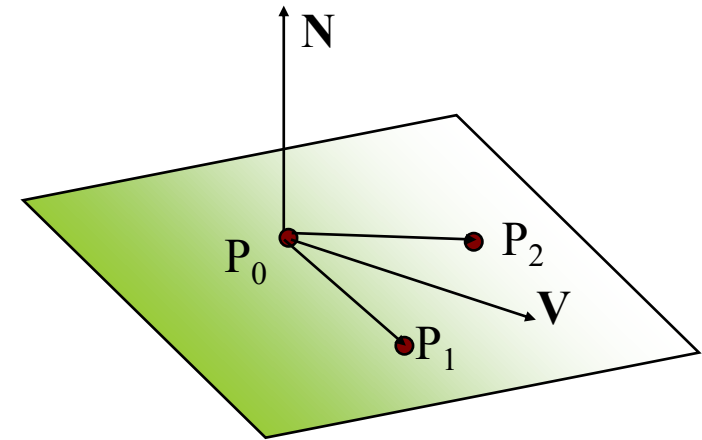
if $P(x, y, z)$ is on the plane

$$\overrightarrow{N} \bullet \overrightarrow{P_0P} = 0$$

$$\Rightarrow (A\hat{i} + B\hat{j} + C\hat{k}) \bullet [(x - x_0)\hat{i} + (y - y_0)\hat{j} + (z - z_0)\hat{k}] = 0$$

$$\Rightarrow Ax + By + Cz + D = 0$$

$$\text{where } D = -(Ax_0 + By_0 + Cz_0)$$



3D Plane

Equation of a plane in Cartesian coordinates

Example

Find a point P_0 and the perpendicular vector $\mathbf{n}$ to the plane $2x + 4y - z = 3$.

Solution: The equation of a plane is $n_x x + n_y y + n_z z = d$.

The components of the normal vector $\mathbf{n}$ are the coefficients that multiply the variables x , y and z . Hence,

$$\mathbf{n} = \langle 2, 4, -1 \rangle.$$

A point P_0 on the plane is simple to find. Just look for the intersection of the plane with one of the coordinate axis.

For example: set $y = 0$, $z = 0$ and find x from the equation of the plane: $2x = 3$, that is $x = 3/2$. Therefore, $P_0 = (3/2, 0, 0)$. $\triangleleft$

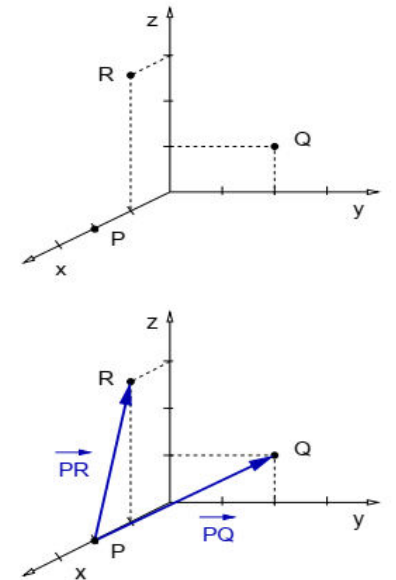
Equation of a plane in Cartesian coordinates

Example

Find the equation of the plane containing the points $P = (2, 0, 0)$, $Q = (0, 2, 1)$, $R = (1, 0, 3)$.

Solution:

Find two tangent vectors to the plane, for example, $\overrightarrow{PQ} = \langle -2, 2, 1 \rangle$ and $\overrightarrow{PR} = \langle -1, 0, 3 \rangle$.

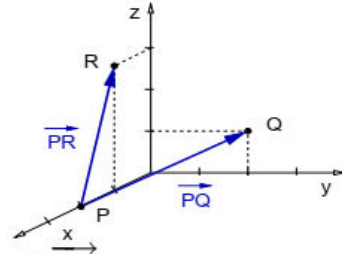


3D Plane

Equation of a plane in Cartesian coordinates

Solution:

Find two tangent vectors to the plane,
for example, $\vec{PQ} = \langle -2, 2, 1 \rangle$ and
 $\vec{PR} = \langle -1, 0, 3 \rangle$.



Find a vector $\mathbf{n}$ perpendicular to both $\vec{PQ}$ and $\vec{PR}$.

One way is using the cross product: $\mathbf{n} = \vec{PQ} \times \vec{PR}$. That is,

$$\mathbf{n} = \vec{PQ} \times \vec{PR} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ -2 & 2 & 1 \\ -1 & 0 & 3 \end{vmatrix} = (6 - 0)\mathbf{i} - (-6 + 1)\mathbf{j} + (0 + 2)\mathbf{k}.$$

The result is: $\mathbf{n} = \langle 6, 5, 2 \rangle$. Choose any point on the plane,
say $P = (2, 0, 0)$. Then, the equation of the plane is:

$$6(x - 2) + 5y + 2z = 0.$$



Vectors in 3D

Have length and direction

$$\mathbf{V} = [a, b, c]$$

Length is given by the Euclidean Norm

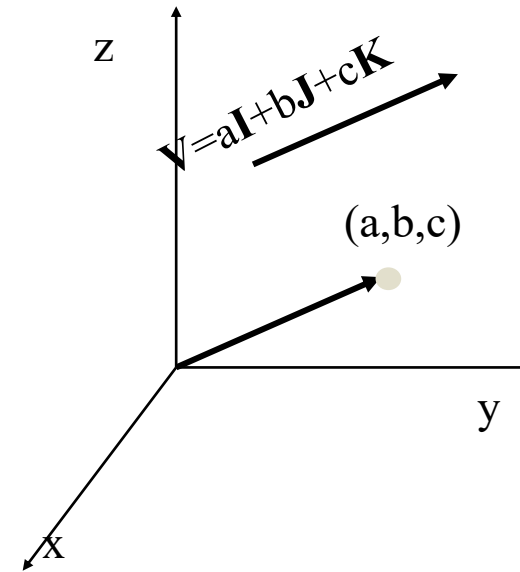
$$|\mathbf{V}| = \sqrt{a^2 + b^2 + c^2}$$

Dot Product

$$\begin{aligned}\mathbf{V} \cdot \mathbf{U} &= [x_v, y_v, z_v] \cdot [x_u, y_u, z_u] \\ &= x_v * x_u + y_v * y_u + z_v * z_u \\ &= |\mathbf{V}| |\mathbf{U}| \cos \beta\end{aligned}$$

Cross Product

$$\begin{aligned}\mathbf{V} \times \mathbf{U} &= [y_v * z_u - z_v * y_u, -x_v * z_u + z_v * x_u, x_v * y_u - y_v * x_u] \\ &= \eta |\mathbf{V}| |\mathbf{U}| \sin \beta \\ \mathbf{V} \times \mathbf{U} &= -(\mathbf{U} \times \mathbf{V})\end{aligned}$$

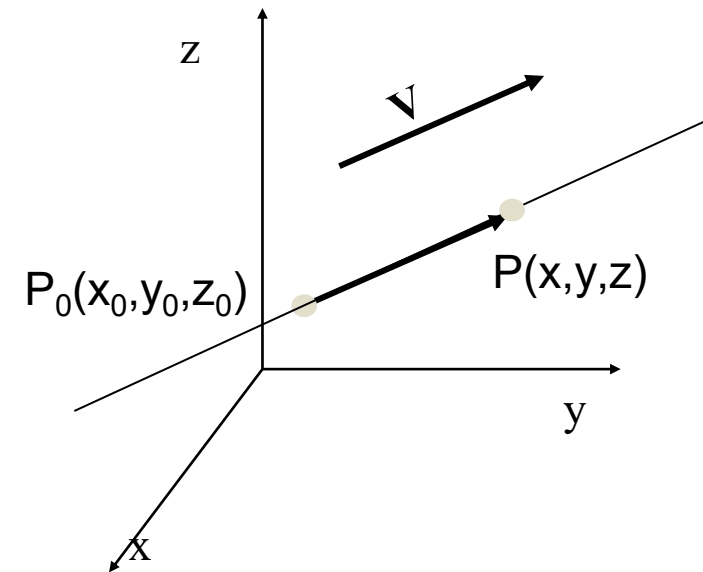


Vector Equation of Line

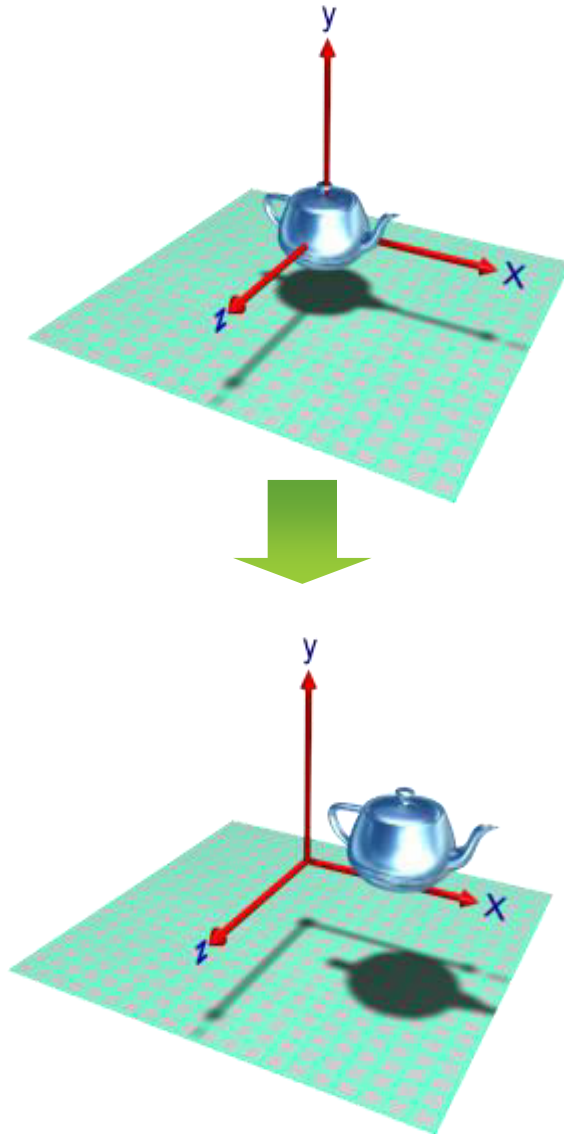
□ A line is defined by

- A direction vector V
- A point $P_0(x_0, y_0, z_0)$

$$\begin{aligned} L &= P_0 + t(P_1 - P_0) = P_0 + t\overline{V} \\ &= (1-t)P_0 + tP_1 \end{aligned}$$



Translation – 3D



$$x' = x + d_x$$

$$y' = y + d_y$$

$$z' = z + d_z$$



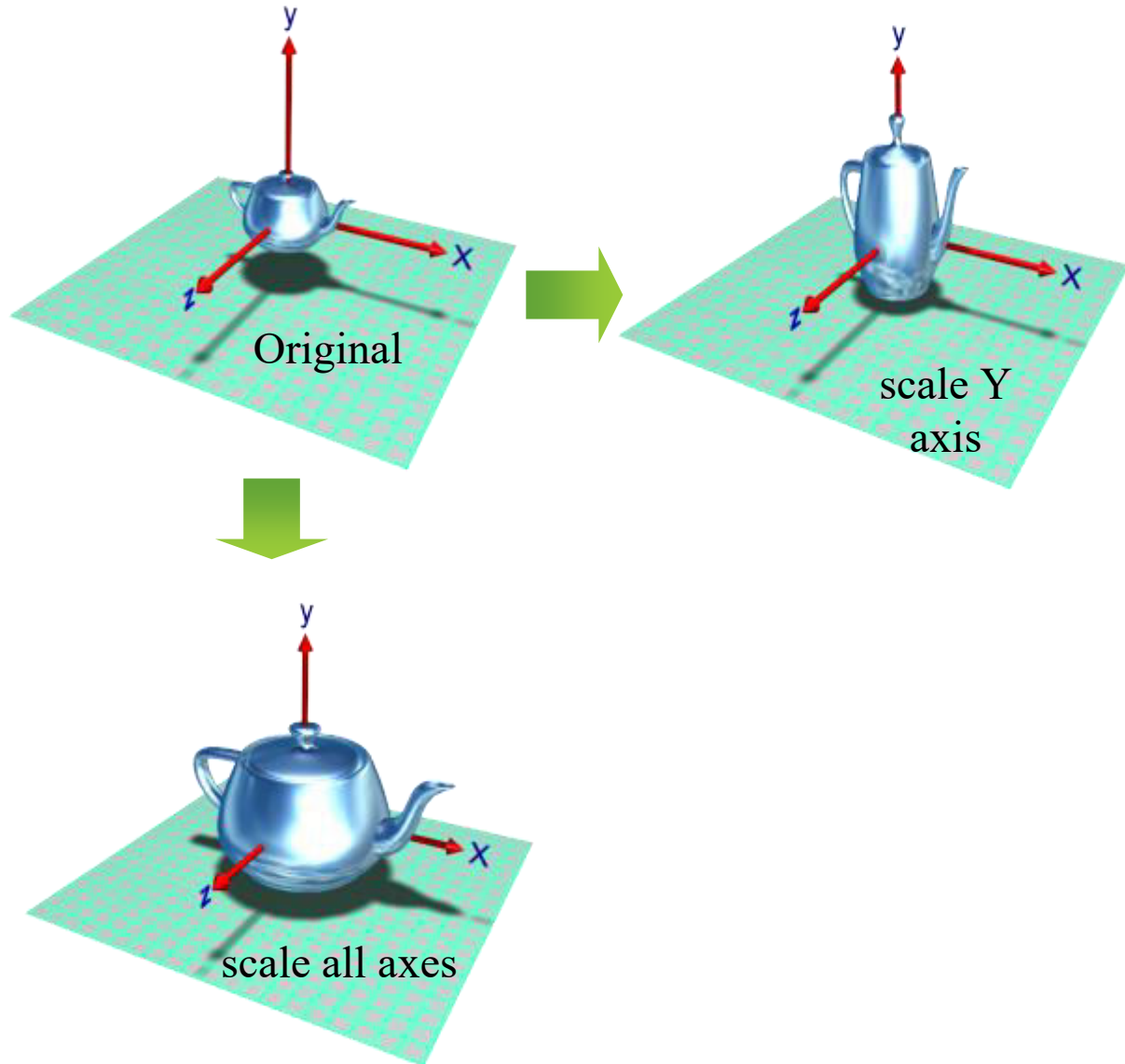
$$\begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x + d_x \\ y + d_y \\ z + d_z \\ 1 \end{bmatrix}$$



$$T(d_x, d_y, d_z) * P = P'$$



Scaling – 3D



$$x' = s_x * x$$

$$y' = s_y * y$$

$$z' = s_z * z$$



$$S(s_x, s_y, s_z) * P = P'$$



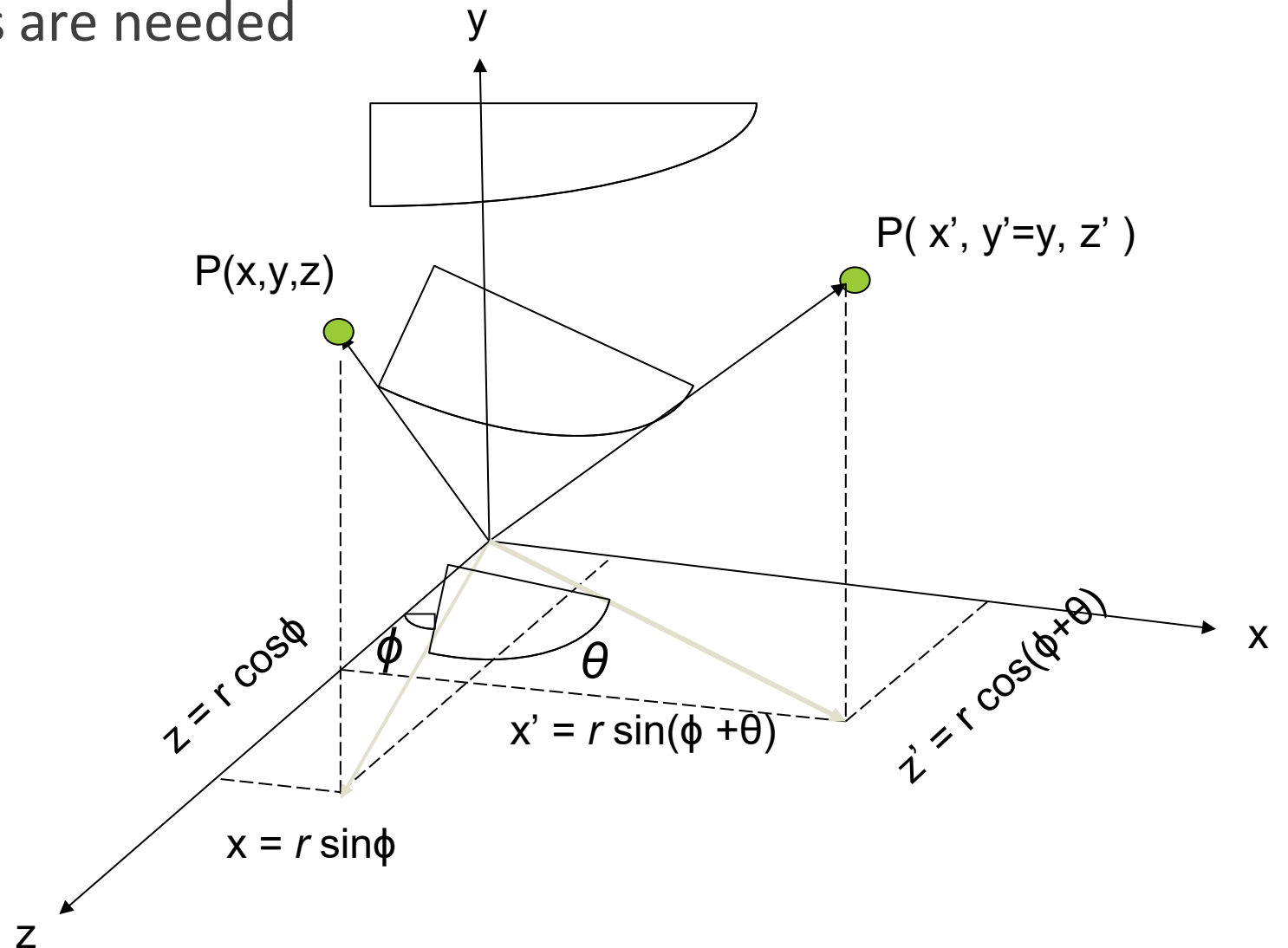
$$\begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x * s_x \\ y * s_y \\ z * s_z \\ 1 \end{bmatrix}$$

Rotation in 3D

❑ For 3D-Rotation 2 parameters are needed

- Angle of rotation
- Axis of rotation

❑ Rotation about y axis



Rotation in 3D

$$x = r \sin \varphi$$

$$z = r \cos \varphi$$

$$\begin{aligned} x' &= r \sin(\varphi + \theta) = r \sin \varphi \cos \theta + r \cos \varphi \sin \theta \\ &= x \cos \theta + z \sin \theta \end{aligned}$$

$$y' = y$$

$$\begin{aligned} z' &= r \cos(\varphi + \theta) = r \cos \varphi \cos \theta - r \sin \varphi \sin \theta \\ &= z \cos \theta - x \sin \theta \\ &= -x \sin \theta + z \cos \theta \end{aligned}$$

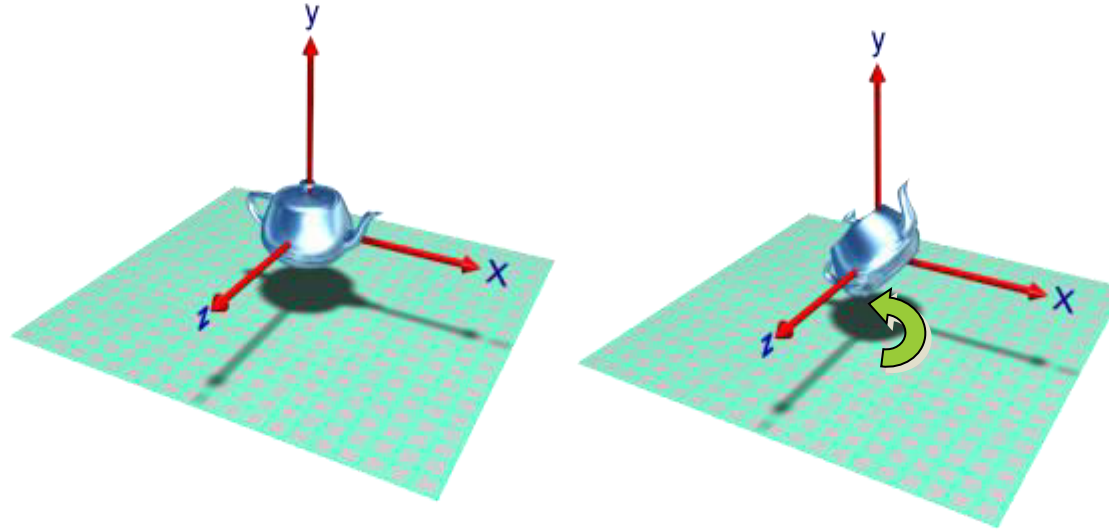
Rotation in 3D

About y-axis

$$\begin{array}{ccc} R_{\theta, y} & * & P = P' \\ \Downarrow & & \Downarrow \\ \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & * & \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x * \cos \theta + z * \sin \theta \\ y \\ -x * \sin \theta + z * \cos \theta \\ 1 \end{bmatrix} \end{array}$$

Rotation – 3D

□ Rotation about z-axis:



$$\begin{array}{ccc} R_{\theta,k} & * & P = P' \\ \Downarrow & & \Downarrow \\ \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & * & \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x * \cos\theta - y * \sin\theta \\ x * \sin\theta + y * \cos\theta \\ z \\ 1 \end{bmatrix} \end{array}$$

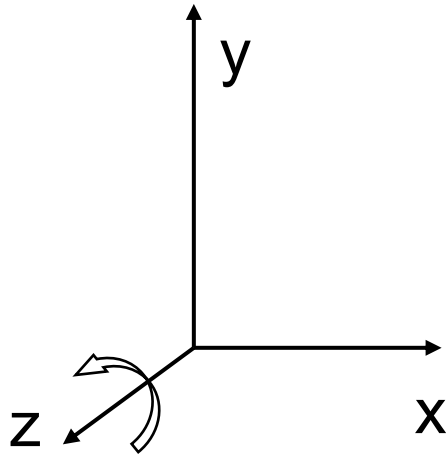
Rotation about X-axis

About x-axis

$$\begin{array}{ccc} R_{\theta,i} & * P & = P' \\ \Downarrow & \Downarrow & \Downarrow \\ \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} & = \begin{bmatrix} x \\ y * \cos\theta - z * \sin\theta \\ y * \sin\theta + z * \cos\theta \\ 1 \end{bmatrix} \end{array}$$

Coordinate-Axes Rotations

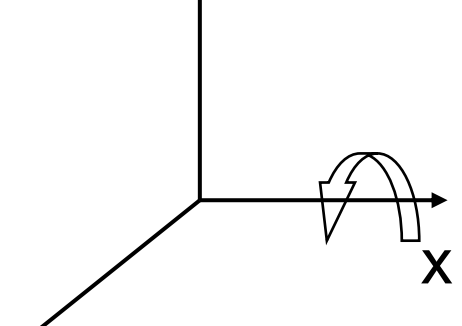
■ Z-Axis Rotation



A 3D coordinate system with x, y, and z axes. A curved arrow around the z-axis indicates a rotation by an angle θ in the xy-plane.

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

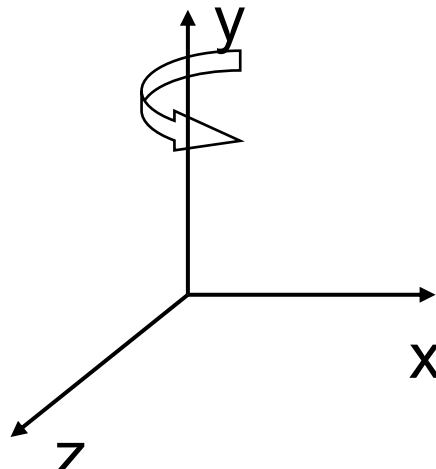
■ X-Axis Rotation



A 3D coordinate system with x, y, and z axes. A curved arrow around the x-axis indicates a rotation by an angle θ in the yz-plane.

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

■ Y-Axis Rotation

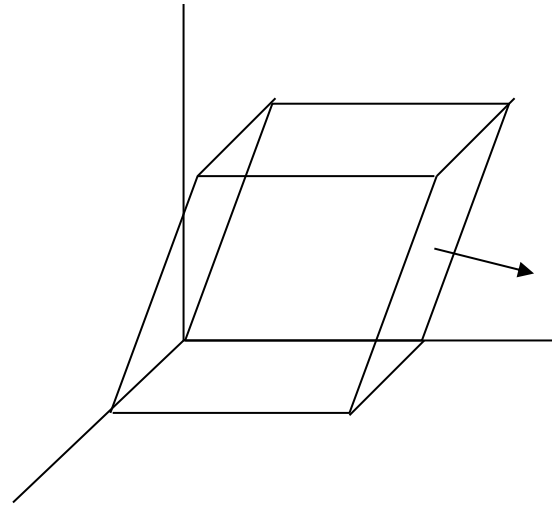
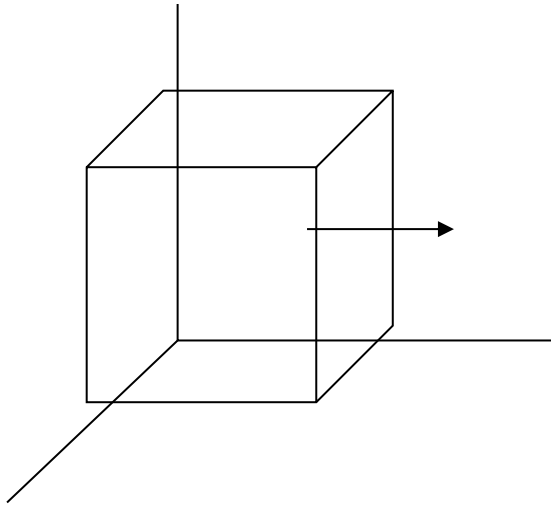


A 3D coordinate system with x, y, and z axes. A curved arrow around the y-axis indicates a rotation by an angle θ in the xz-plane.

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

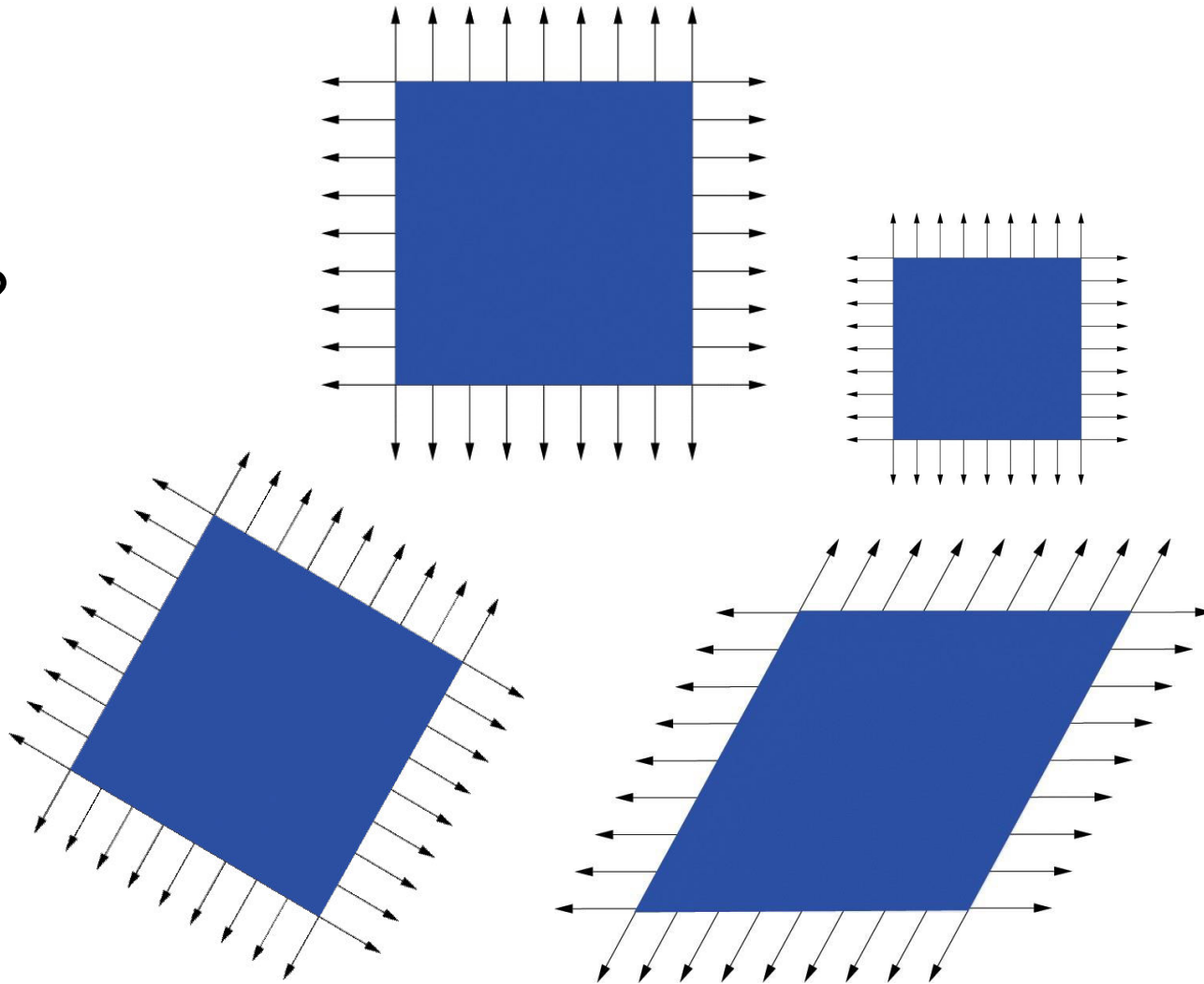
Object Transformation

- ❑ **Line:** Can be transformed by transforming the end points
- ❑ **Plane:(described by 3-points)** Can be transformed by transforming the 3-points
- ❑ **Plane:(described by a point and Normal)** Point is transformed as usual. **Special treatment** is needed for transforming **Normal**



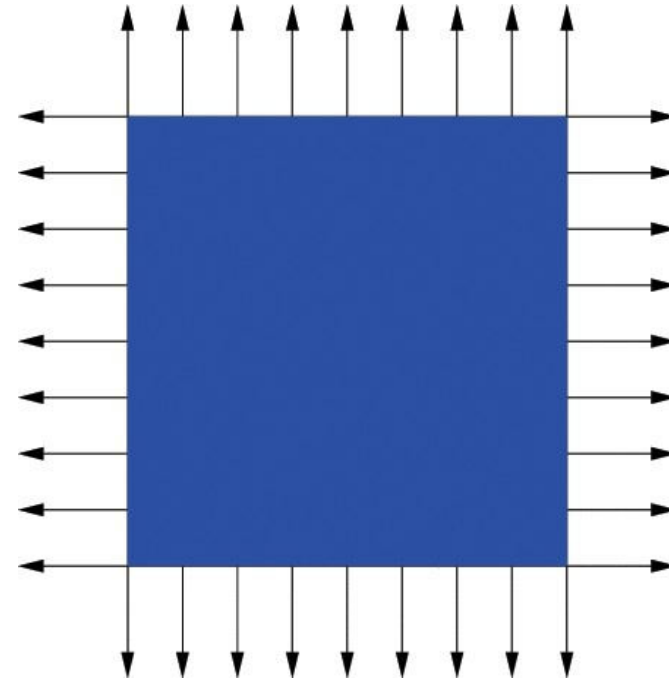
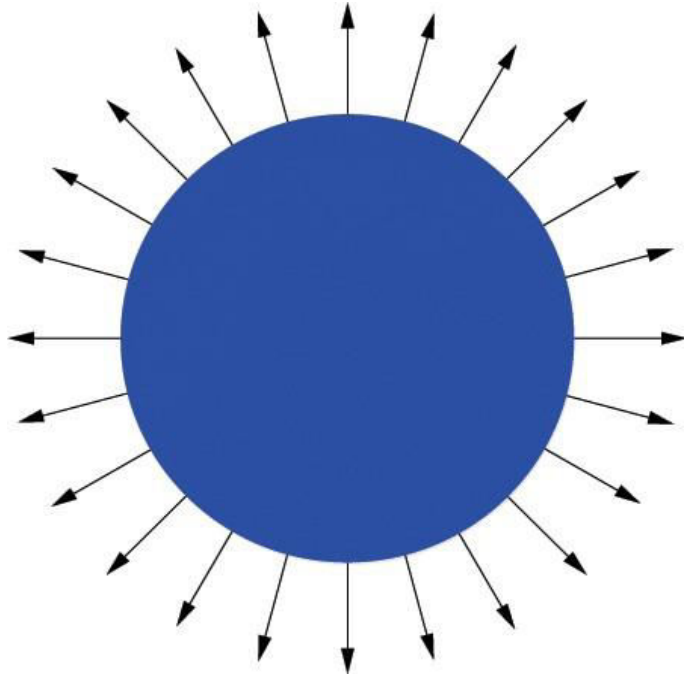
Transform Normal like Object?

- translation?
- rotation?
- isotropic scale?
- scale?
- reflection?
- shear?
- perspective?



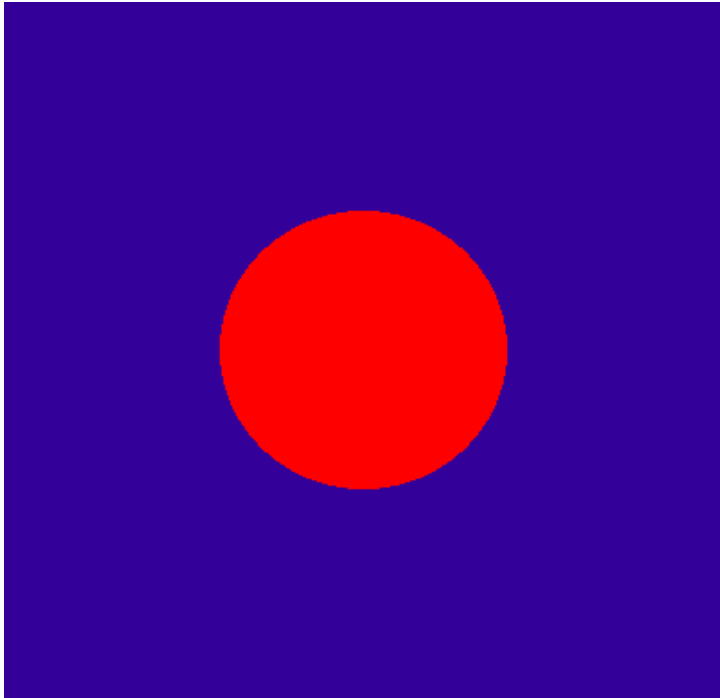
Normal

□ Surface Normal: unit vector that is locally perpendicular to the surface

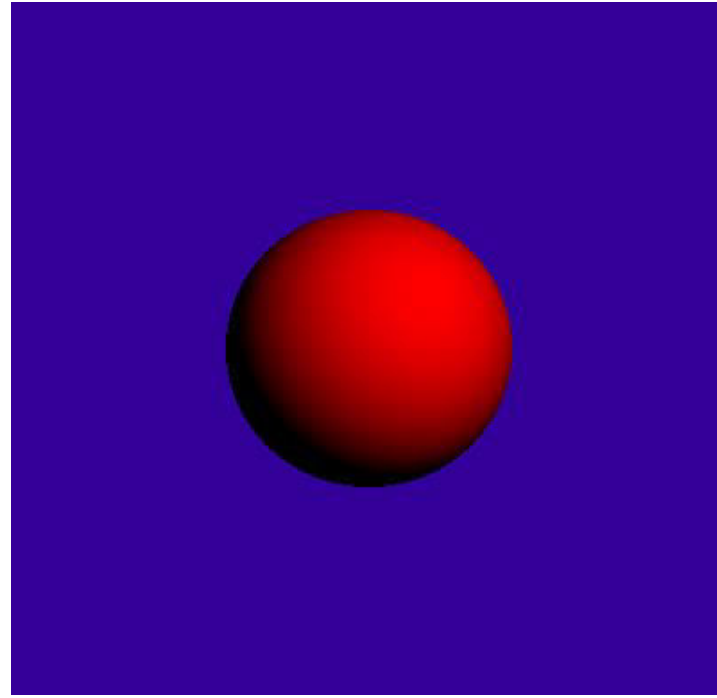


Why is the Normal important?

- It's used for shading — makes things look 3D!

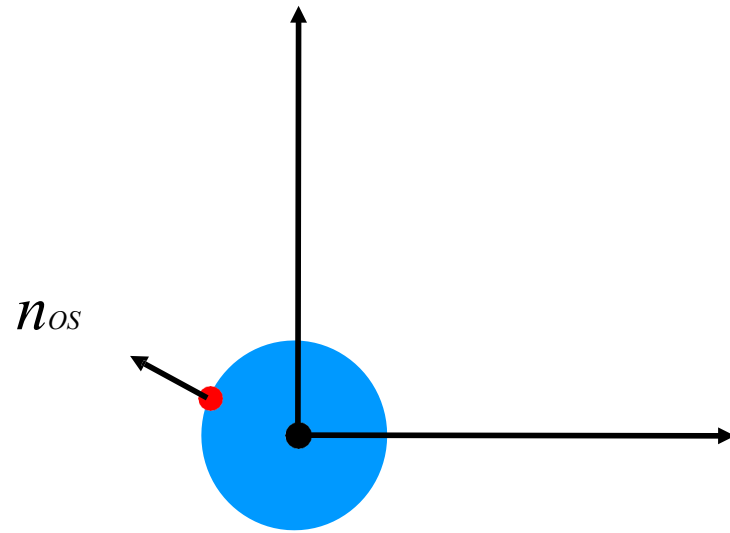


object color only

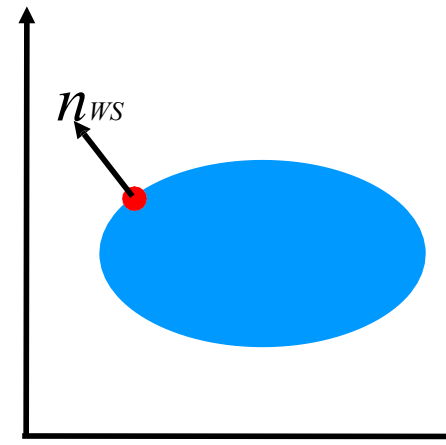


Diffuse Shading

How do we transform normals?



Object Space



World Space

Transforming Normal Vector

Let the plan be represented by point P and normal N

$$N = [A \ B \ C \ D]^T \quad \text{and} \quad P = [x \ y \ z \ 1]^T$$

$$\text{such that } N^T * P = Ax + By + Cz + D = 0$$

Let M be the transformation to be applied on P and

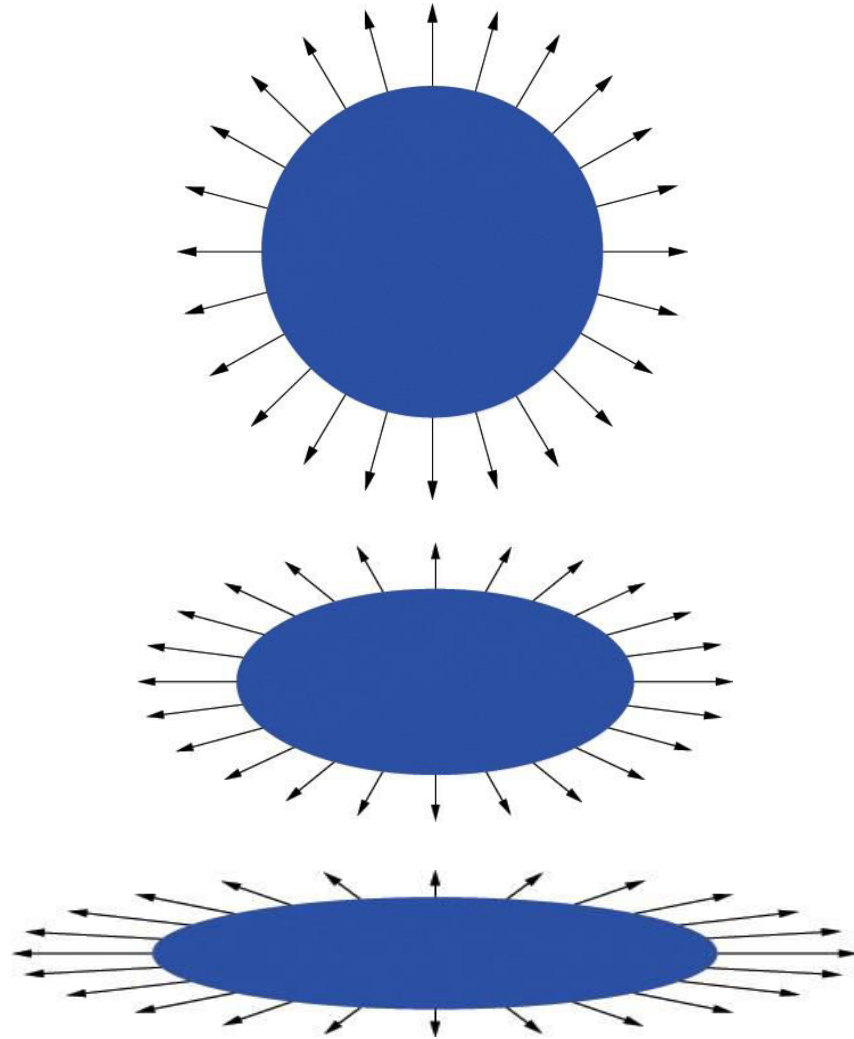
Q be a transformation such that

$$\left. \begin{array}{l} (Q^T * N)^T * M * P = 0 \\ N^T * Q^T * M * P = 0 \end{array} \right\} \text{ which leads to } \left\| \begin{array}{l} Q^T * M = I \\ \therefore Q^T = M^{-1} \end{array} \right.$$

$$\therefore N' = (M^{-1})^T * N$$

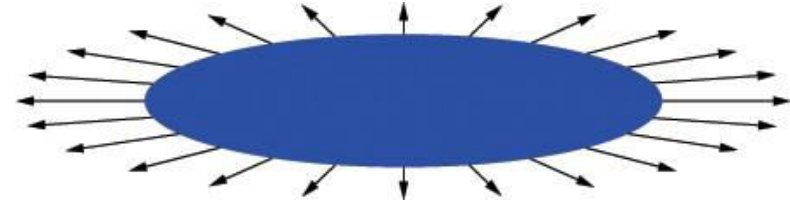
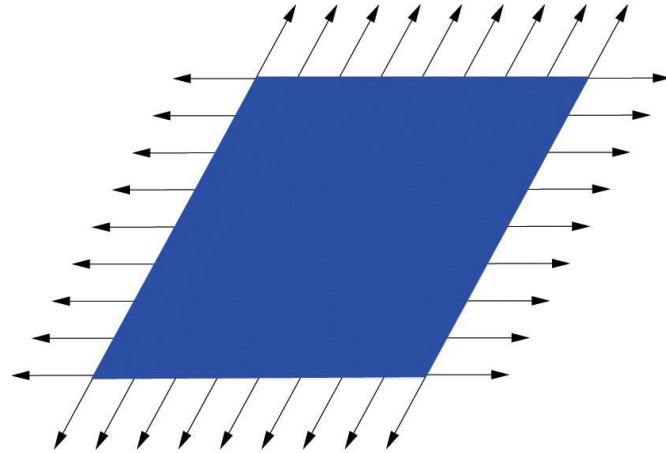
Transform Normal like Object?

- translation?
- rotation?
- isotropic scale?
- scale?
- reflection?
- shear?
- perspective?

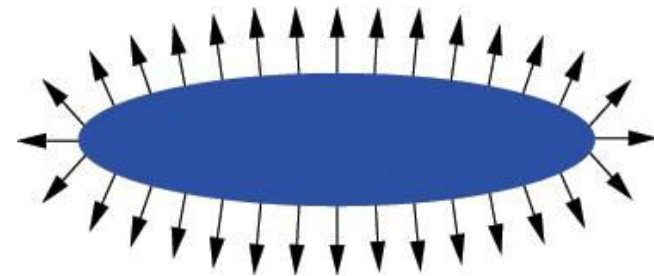
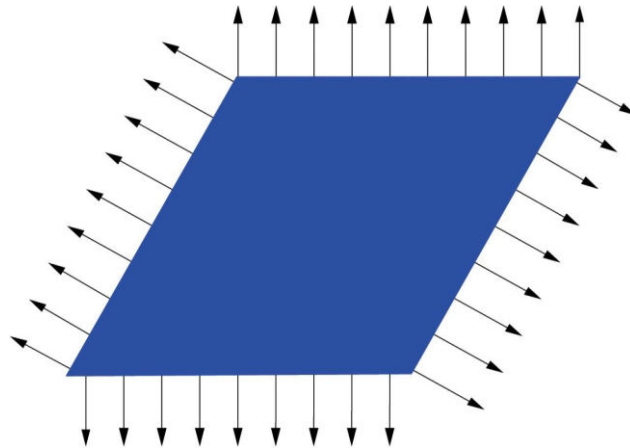


Transformation for shear and scale

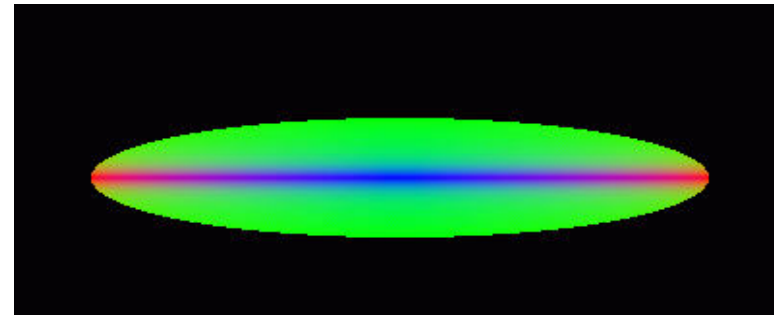
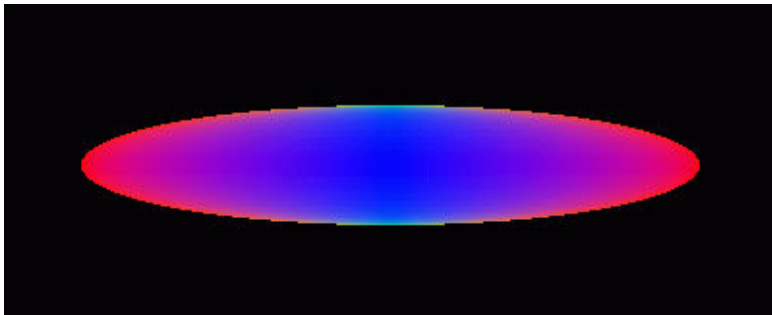
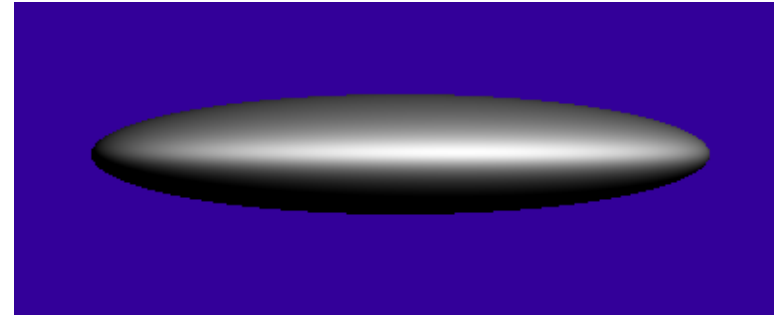
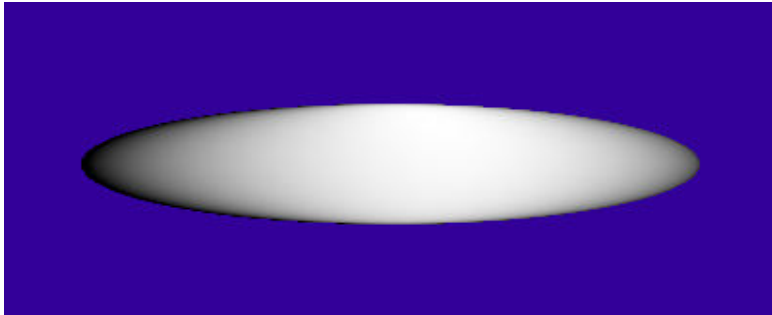
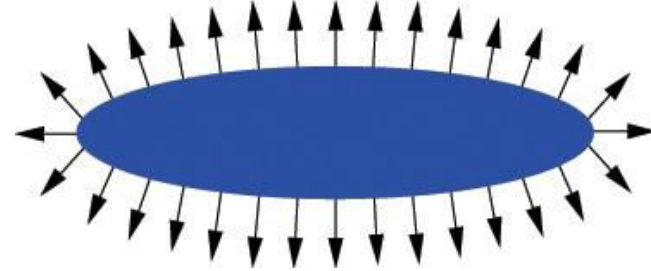
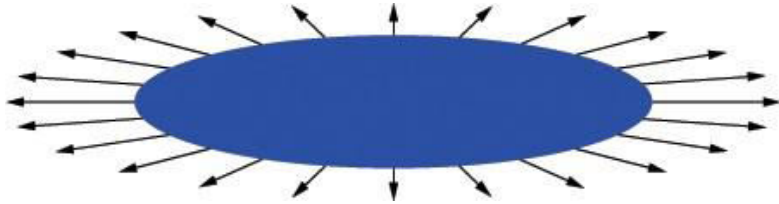
Incorrect
Normal
Transformation



Correct
Normal
Transformation



More Normal Visualizations



Incorrect Normal
Transformation

Correct Normal
Transformation

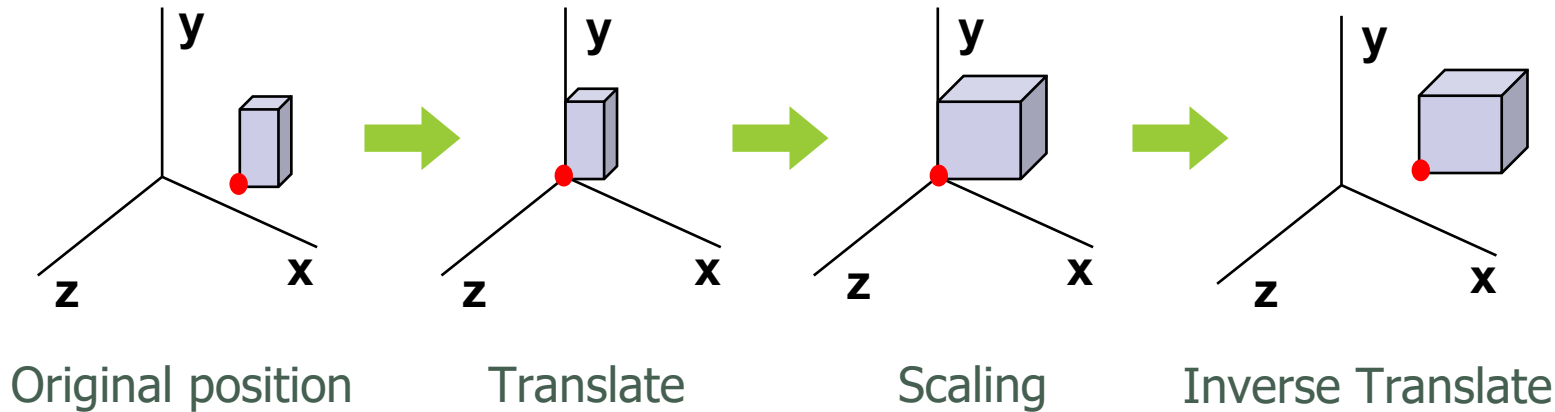
Composite Transformations – 3D

❑ Some of the composite transformations to be studied are:

- Relative Scaling
- $A_{V,N}$ = aligning a vector V with a vector N
- $R_{\theta,L}$ = rotation about an axis $L(V, P)$
- $S_{sx,sy,P}$ = scaling w.r.t. point P

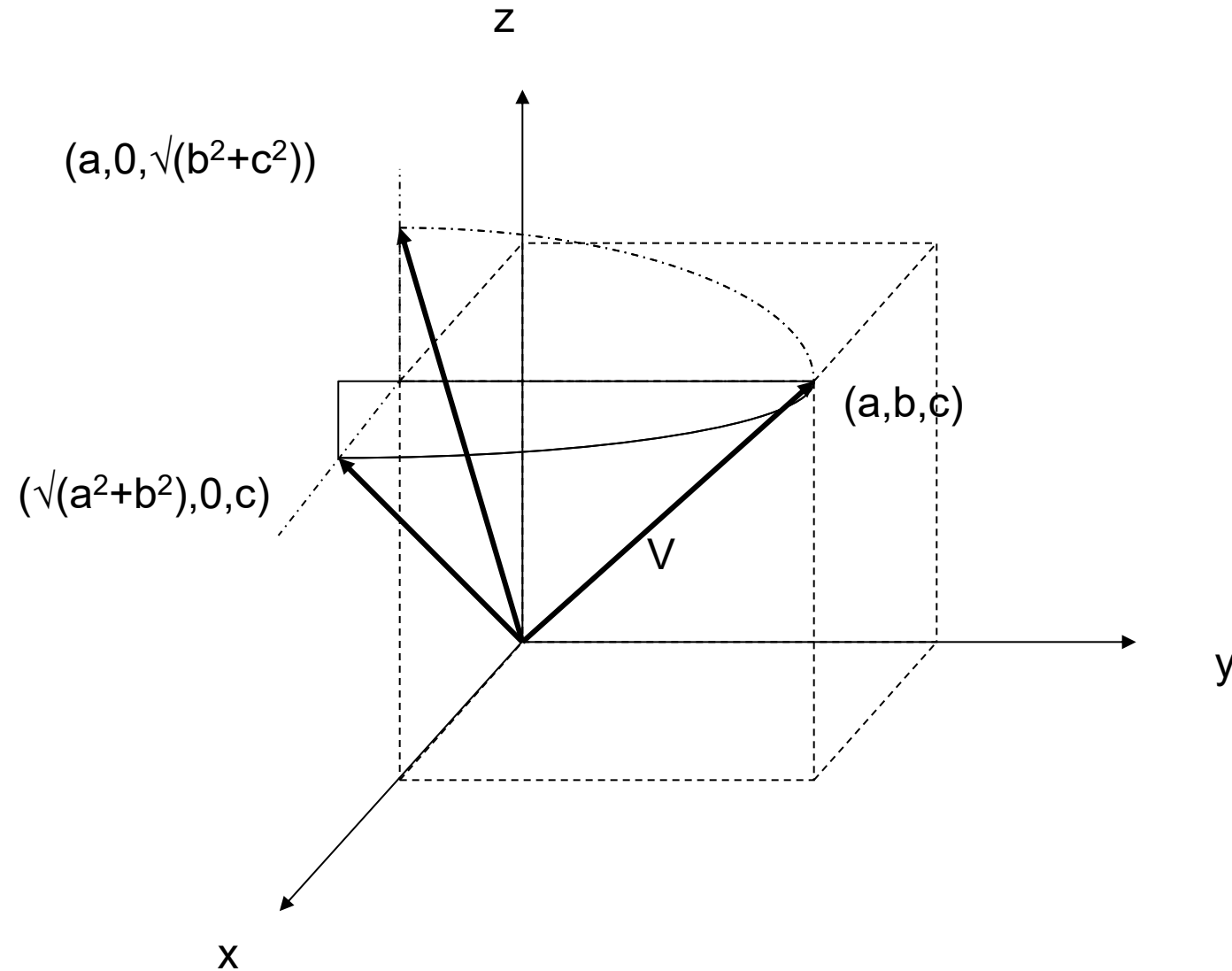
Relative Scaling

Scaling with a Selected Fixed Position

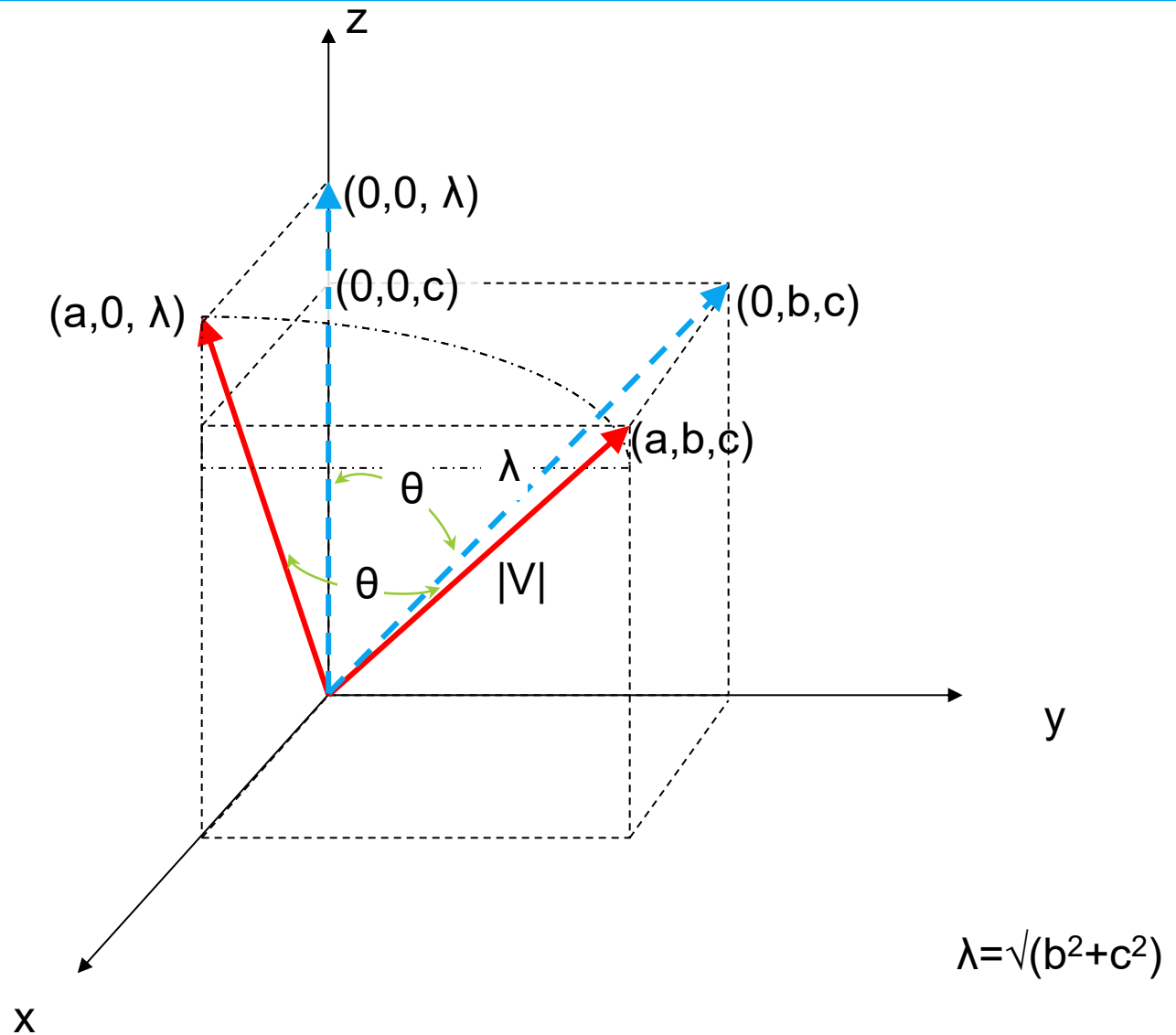


$$T(x_f, y_f, z_f) \cdot S(s_x, s_y, s_z) \cdot T(-x_f, -y_f, -z_f) = \begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & x_f \\ 0 & 1 & 0 & y_f \\ 0 & 0 & 1 & z_f \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_f \\ 0 & 1 & 0 & -y_f \\ 0 & 0 & 1 & -z_f \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Av: Aligning vector V with k



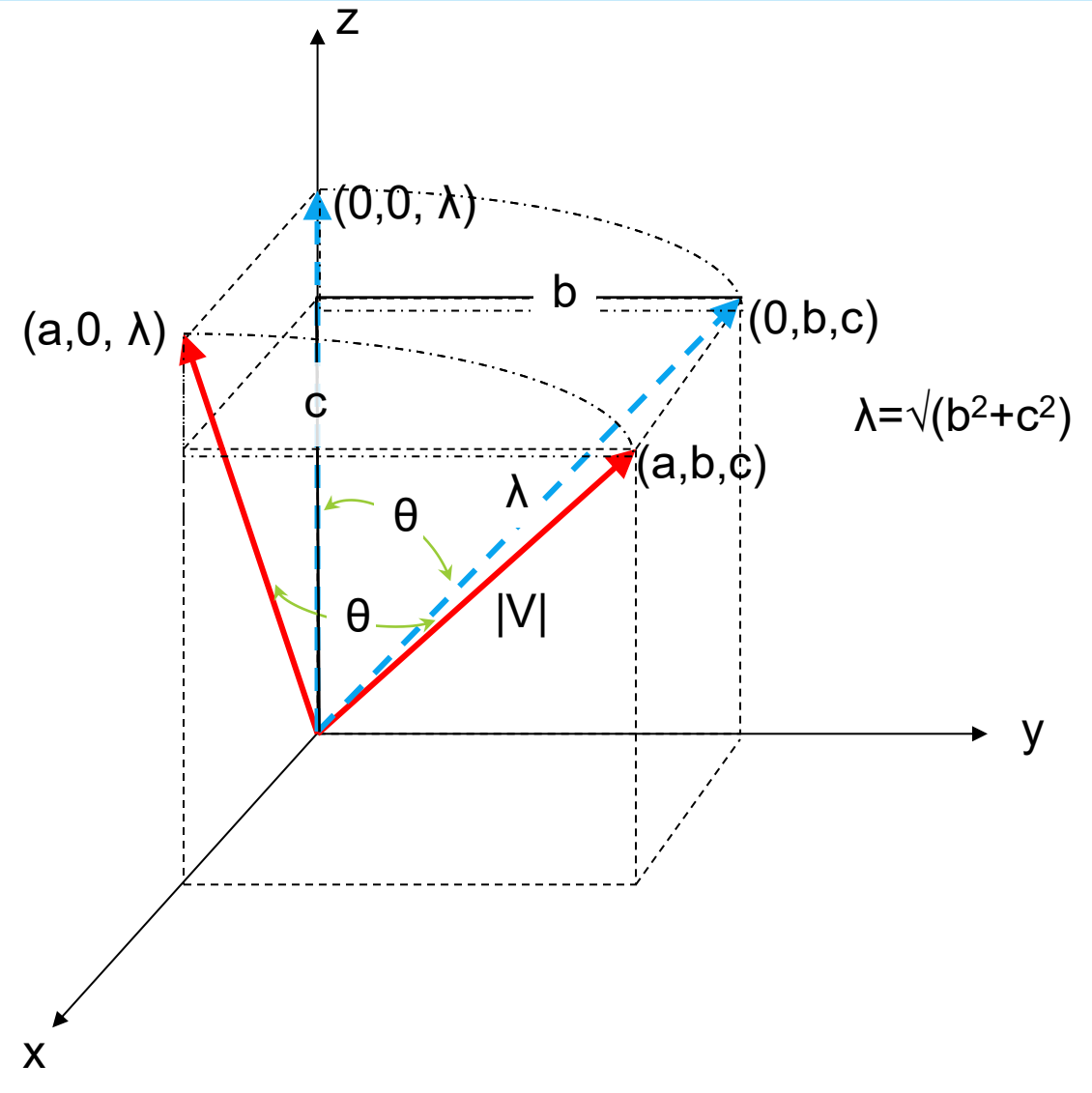
Av: Aligning vector V with k



A_V : aligning vector V with k

Step 1: Rotate about x-axis by θ

$$\left. \begin{aligned} \sin \theta &= \frac{b}{\lambda} \\ \cos \theta &= \frac{c}{\lambda} \end{aligned} \right\} \lambda = \sqrt{b^2 + c^2}$$



A_V

$R_{\theta, i}$

A_V : aligning vector V with k

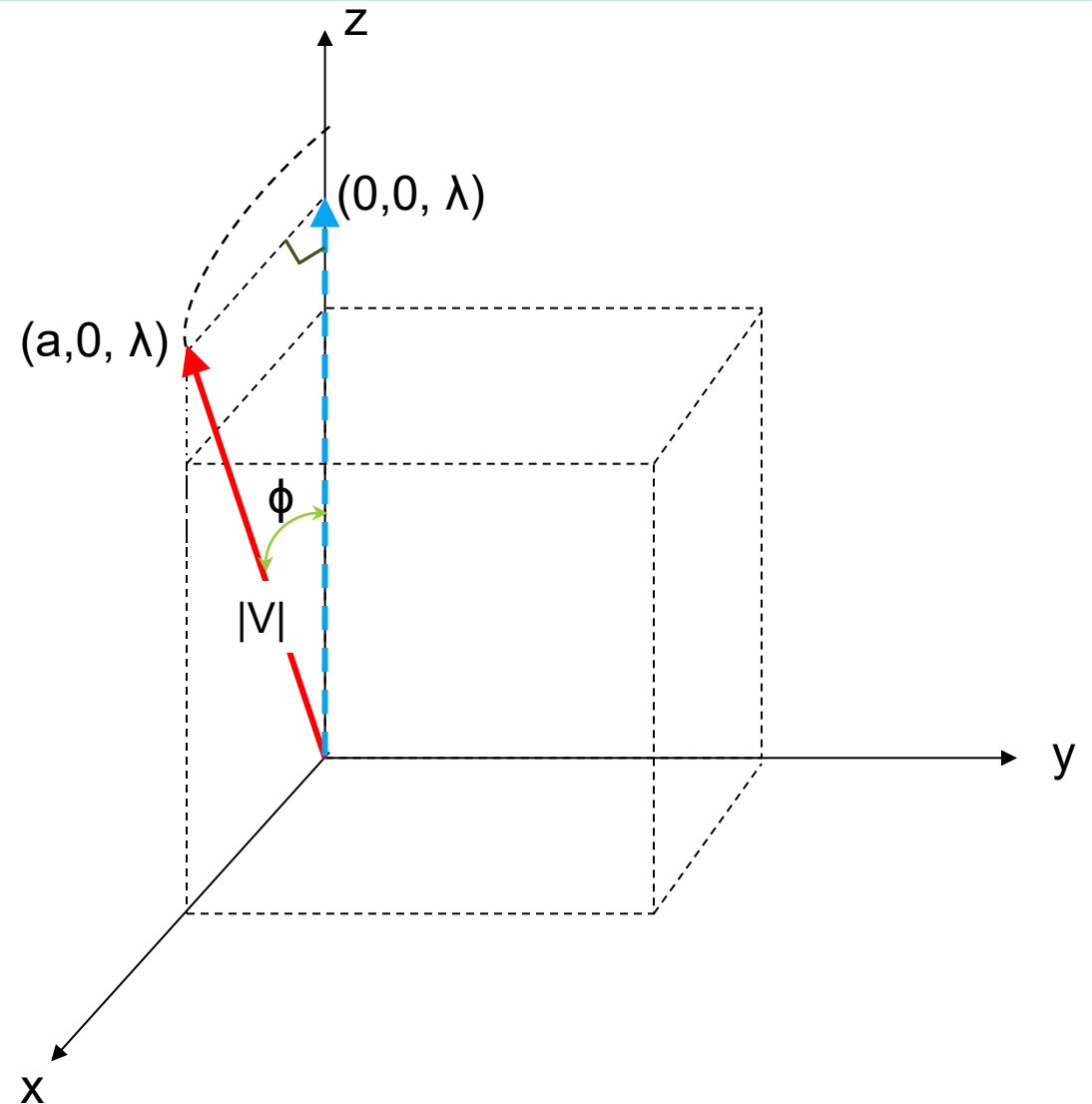
Step 1: Rotate V about x-axis by θ

$$\left. \begin{aligned} \sin \theta &= \frac{b}{\lambda} \\ \cos \theta &= \frac{c}{\lambda} \end{aligned} \right\} \lambda = \sqrt{b^2 + c^2}$$

Step 2: Rotate V about y-axis by $-\phi$

$$\left. \begin{aligned} \sin(-\phi) &= \frac{-a}{|V|} \\ \cos(-\phi) &= \frac{\lambda}{|V|} \end{aligned} \right\} |V| = \sqrt{a^2 + b^2 + c^2}$$

$$A_V = R_{-\phi,j} * R_{\theta,i}$$



Av: Alignment transformation

$$A_v = \begin{bmatrix} \cos(-\phi) & 0 & \sin(-\phi) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(-\phi) & 0 & \cos(-\phi) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_v = \begin{bmatrix} \frac{\lambda}{|V|} & 0 & \frac{-a}{|V|} & 0 \\ 0 & 1 & 0 & 0 \\ \frac{a}{|V|} & 0 & \frac{\lambda}{|V|} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{c}{\lambda} & \frac{-b}{\lambda} & 0 \\ 0 & \frac{b}{\lambda} & \frac{c}{\lambda} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A_v = \begin{bmatrix} \frac{\lambda}{|V|} & \frac{-ab}{\lambda|V|} & \frac{-ac}{\lambda|V|} & 0 \\ 0 & \frac{c}{\lambda} & \frac{-b}{\lambda} & 0 \\ \frac{a}{|V|} & \frac{b}{|V|} & \frac{c}{|V|} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Av: Alignment transformation

□ If we normalize $\mathbf{V} = a\mathbf{I} + b\mathbf{J} + c\mathbf{K}$ to a unit vector $\mathbf{v} = a\mathbf{I} + b\mathbf{J} + c\mathbf{K}$ before rotation the matrix become simple enough

□ Where $a = a / |\mathbf{V}|$ $b = b / |\mathbf{V}|$ $c = c / |\mathbf{V}|$

$$A_v = \begin{bmatrix} \lambda & \frac{-ab}{\lambda} & \frac{-ac}{\lambda} & 0 \\ 0 & \frac{c}{\lambda} & \frac{-b}{\lambda} & 0 \\ a & b & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Inverse Alignment

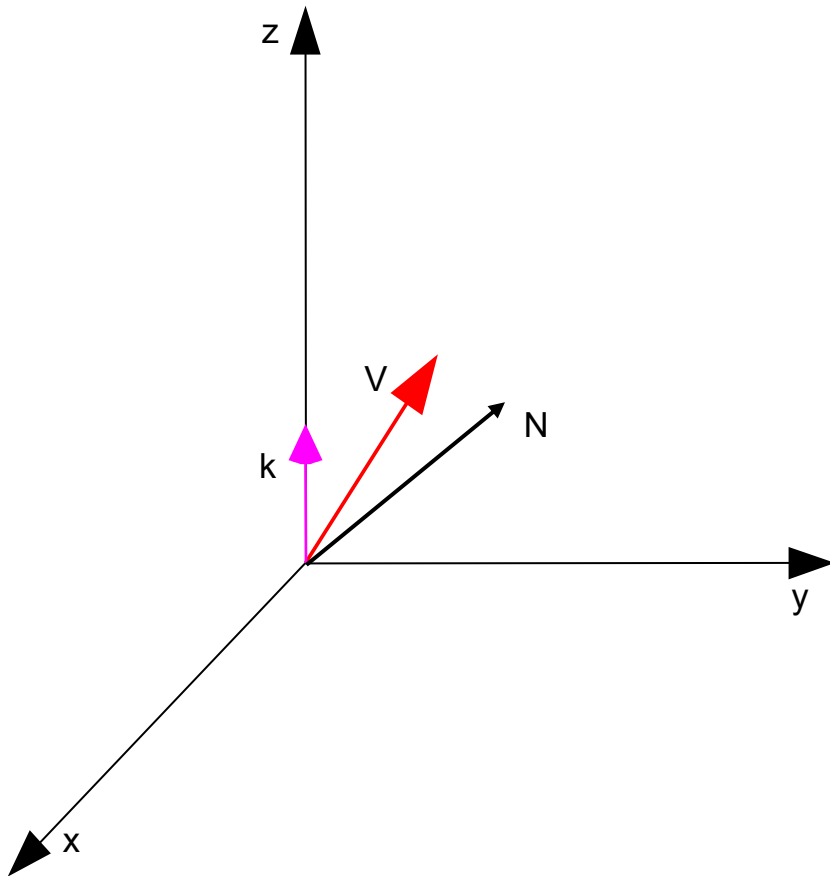
- Aligns the vector k with vector v

$$A_v^{-1} = (R_{-\phi,j} \cdot R_{\theta,i})^{-1} = R_{\theta,i}^{-1} \cdot R_{-\phi,j}^{-1} = R_{-\theta,i} \cdot R_{\phi,j} = A_v^T$$

A_V : aligning vector v (V) with n (N)

□ $A_V^{-1} = A_V^T$

□ $A_{V,N} = A_N^{-1} * A_V$

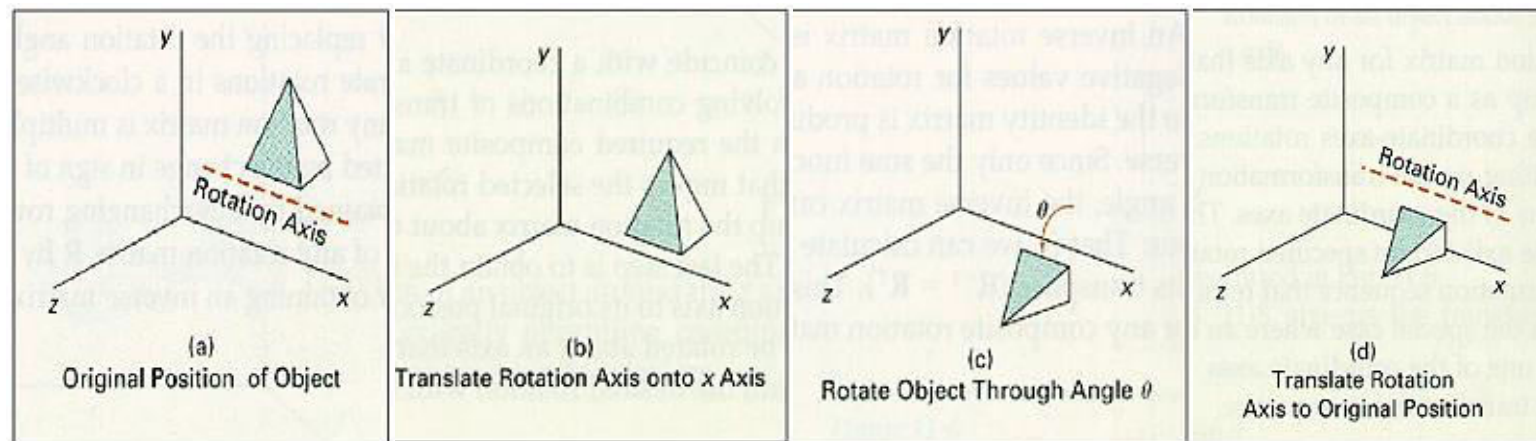


$$A_V = \begin{bmatrix} \frac{\lambda}{|V|} & \frac{-ab}{\lambda|V|} & \frac{-ac}{\lambda|V|} & 0 \\ 0 & \frac{c}{\lambda} & \frac{-b}{\lambda} & 0 \\ \frac{a}{|V|} & \frac{b}{|V|} & \frac{c}{|V|} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

General 3D Rotations

□ Rotation about an Axis that is Parallel to One of the Coordinate Axes

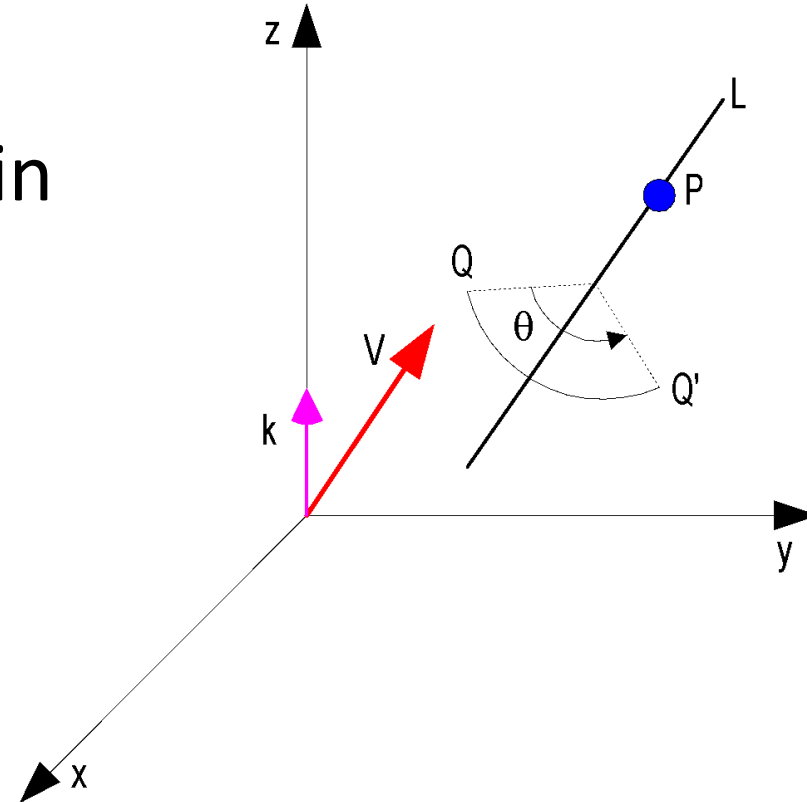
- **Translate** the object so that the rotation axis coincides with the parallel coordinate axis
- Perform the specified **rotation** about that axis
- **Translate** the object so that the rotation axis is moved back to its original position



$R_{\theta,L}$: Rotation about an axis defined by line L

□ Let L be represented by vector V and passing through point P

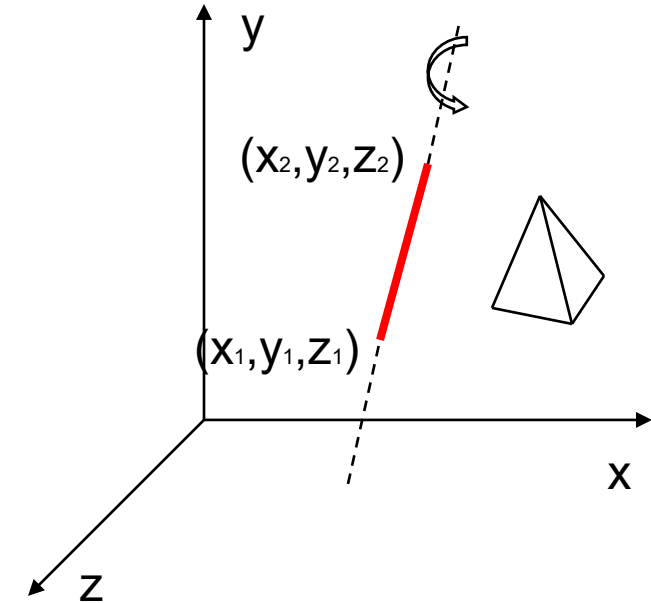
1. Translate P to the origin
2. Align V with vector k
3. Rotate θ° about k
4. Reverse step 2
5. Reverse step 1



$$R_{\theta,L} = T_{-P}^{-1} * A_V^{-1} * R_{\theta,k} * A_V * T_{-P}$$

$R_{\theta,L}$: Rotation about an axis defined by line L

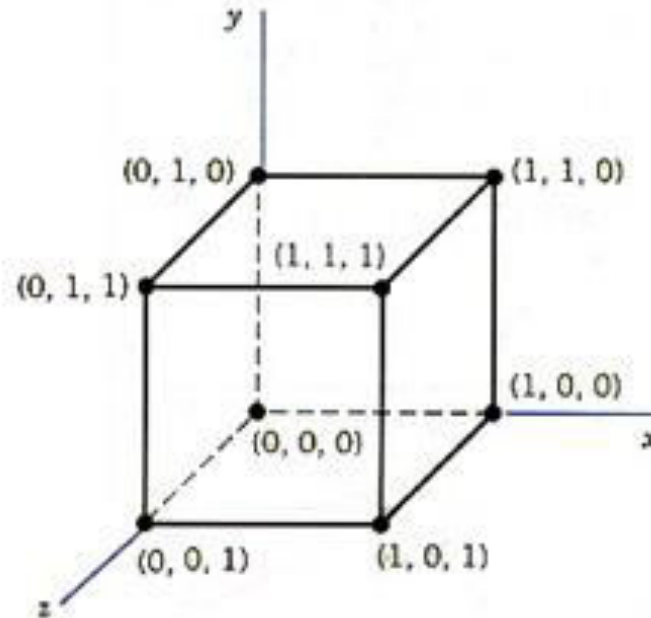
- ❑ Let, L be represented by two points P1 and P2
- ❑ Then find $V = P2 - P1$, and use any one point as P



$$\begin{aligned} R_{\theta,L} &= T_{-P}^{-1} * A_V^{-1} * R_{\theta,k} * A_V * T_{-P} \\ &= T_P * R_{\alpha,i} * R_{\beta,j} * R_{\theta,k} * R_{-\beta,i} * R_{-\alpha,i} * T_{-P} \end{aligned}$$

Example

Find the new coordinates of a unit cube 90°-rotated about an axis defined by its endpoints $A(2,1,0)$ and $B(3,3,1)$.



A Unit Cube

Example

$$\begin{aligned}
 \mathbf{R}(\theta) &= \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{\sqrt{5}}{5} & \frac{2\sqrt{5}}{5} & 0 \\ 0 & -\frac{2\sqrt{5}}{5} & \frac{\sqrt{5}}{5} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{30}}{6} & 0 & \frac{\sqrt{6}}{6} & 0 \\ 0 & 1 & 0 & 0 \\ -\frac{\sqrt{6}}{6} & 0 & \frac{\sqrt{30}}{6} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} \frac{\sqrt{30}}{6} & 0 & -\frac{\sqrt{6}}{6} & 0 \\ 0 & 1 & 0 & 0 \\ \frac{\sqrt{6}}{6} & 0 & \frac{\sqrt{30}}{6} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{\sqrt{5}}{5} & -\frac{2\sqrt{5}}{5} & 0 \\ 0 & \frac{2\sqrt{5}}{5} & \frac{\sqrt{5}}{5} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -2 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} 0.166 & -0.075 & 0.983 & 1.742 \\ 0.742 & 0.667 & 0.075 & -1.151 \\ -0.650 & 0.741 & 0.167 & 0.560 \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$

Example

- ❑ Multiplying $\mathbf{R}(\theta)$ by the point matrix of the original cube

$$[\mathbf{P}'] = \mathbf{R}(\theta) \cdot [\mathbf{P}]$$

$$[\mathbf{P}'] = \begin{bmatrix} 0.166 & -0.075 & 0.983 & 1.742 \\ 0.742 & 0.667 & 0.075 & -1.151 \\ -0.650 & 0.741 & 0.167 & 0.560 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$
$$= \begin{bmatrix} 2.650 & 1.667 & 1.834 & 2.816 & 2.725 & 1.742 & 1.909 & 2.891 \\ -0.558 & -0.484 & 0.258 & 0.184 & -1.225 & -1.151 & -0.409 & -0.483 \\ 1.467 & 1.301 & 0.650 & 0.817 & 0.726 & 0.560 & -0.091 & 0.076 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Rotation using Quaternion

Quaternions

- ❑ One Real Part + Three Imaginary Part

$$q = s + ia + jb + kc$$

- ❑ Properties:

$$i^2 = j^2 = k^2 = -1$$

$$ij = -ji = k$$

$$jk = -kj = i$$

$$ki = -ik = j$$

- ❑ Addition and Scalar Multiplication

$$q_1 + q_2 = (s_1 + s_2) + i(a_1 + a_2) + j(b_1 + b_2) + k(c_1 + c_2)$$

$$dq_1 = s_1 d + ia_1 d + jb_1 d + kc_1 d$$

Quaternions

□ Scalar 's' + Vector " $\mathbf{v} = (x, y, z)$ "

$$\mathbf{q} = (s, \mathbf{v})$$

or

$$\mathbf{q} = [s + \mathbf{v}]$$

□ Addition: $\mathbf{q}_1 \pm \mathbf{q}_2 = (s_1 \pm s_2, \mathbf{v}_1 \pm \mathbf{v}_2)$

$$q^{-1} = \frac{1}{|q|^2} (s, -\mathbf{v})$$

□ Multiplication $\mathbf{q}_1 = [s_1 + \mathbf{v}_1] = [s_1 + x_1\mathbf{i} + y_1\mathbf{j} + z_1\mathbf{k}]$
 $\mathbf{q}_2 = [s_2 + \mathbf{v}_2] = [s_2 + x_2\mathbf{i} + y_2\mathbf{j} + z_2\mathbf{k}]$

$$qq^{-1} = q^{-1}q = (1, 0)$$

$$\begin{aligned} \mathbf{q}_1\mathbf{q}_2 = & [(s_1s_2 - x_1x_2 - y_1y_2 - z_1z_2) + (s_1x_2 + s_2x_1 + y_1z_2 - y_2z_1)\mathbf{i} \\ & + (s_1y_2 + s_2y_1 + z_1x_2 - z_2x_1)\mathbf{j} + (s_1z_2 + s_2z_1 + x_1y_2 - x_2y_1)\mathbf{k}] \end{aligned}$$



$$\mathbf{q}_1\mathbf{q}_2 = (s_1s_2 - \mathbf{v}_1 \cdot \mathbf{v}_2, s_1\mathbf{v}_2 + s_2\mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2)$$

Quaternions

□ Pure quaternion

$$\mathbf{q} = [0 + \mathbf{v}] = [\mathbf{v}]$$

$$\mathbf{q}_1 = [\mathbf{v}_1] = [x_1\mathbf{i} + y_1\mathbf{j} + z_1\mathbf{k}]$$

$$\mathbf{q}_2 = [\mathbf{v}_2] = [x_2\mathbf{i} + y_2\mathbf{j} + z_2\mathbf{k}]$$

$$\mathbf{q}_1\mathbf{q}_2 = (\mathbf{v}_1 \times \mathbf{v}_2)$$

□ Conjugate

$$\mathbf{q}^* = [s - \mathbf{v}]$$

$$\mathbf{q}\mathbf{q}^* = \mathbf{q}^*\mathbf{q} = \mathbf{q} \cdot \mathbf{q} = |\mathbf{q}|^2 = q^2$$

□ Magnitude $|\mathbf{q}| = \sqrt{s^2 + x^2 + y^2 + z^2}$

$$|\mathbf{q}|^2 = s^2 + \mathbf{v} \cdot \mathbf{v}$$

□ Inverse

$$\mathbf{q}^{-1} = \frac{\mathbf{q}^*}{|\mathbf{q}|^2} = \frac{s - x\mathbf{i} - y\mathbf{j} - z\mathbf{k}}{|\mathbf{q}|^2}$$

$$\mathbf{q}\mathbf{q}^{-1} = \mathbf{q}^{-1}\mathbf{q} = (1, \mathbf{0})$$

A unit quaternion has a magnitude equal to 1

3D Rotation with Quaternion

□ point $P(x, y, z)$ is represented in quaternion form by

$$\mathbf{P} = [0 + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}]$$

□ It can be shown that a position vector P is rotated through an angle θ about an axis using the following operation:

$$\mathbf{P}' = \mathbf{q}\mathbf{P}\mathbf{q}^{-1}$$

➤ where the axis and angle of rotation are encoded within the unit quaternion $\mathbf{q}$, and $\mathbf{P}'$ is the rotated vector.

3D Rotation with Quaternion

❑ To rotate a point $P(x, y, z)$ an angle θ about any axis $\mathbf{u}$

❑ Use the following steps:

1. Convert the point $P(x, y, z)$ to a pure quaternion $\mathbf{P} = [0 + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}]$ or $\mathbf{P} = (0, \mathbf{p})$

1. Define the axis of rotation as a unit vector $\hat{\mathbf{u}} = u_x\mathbf{i} + u_y\mathbf{j} + u_z\mathbf{k}$; $\hat{\mathbf{u}} = \frac{\mathbf{u}}{|\mathbf{u}|}$

1. Define the transforming quaternions $\mathbf{q} = [s + \mathbf{v}]$ and $\mathbf{q}^{-1} = [s - \mathbf{v}]$

$$\mathbf{q} = \left[\cos \frac{\theta}{2} + \sin \frac{\theta}{2} \hat{\mathbf{u}} \right] \quad \mathbf{q}^{-1} = \left[\cos \frac{\theta}{2} - \sin \frac{\theta}{2} \hat{\mathbf{u}} \right]$$

1. Compute $\mathbf{P}' = \mathbf{q}\mathbf{P}\mathbf{q}^{-1}$ and unpack (x', y', z')

$$P'(x', y', z') \Leftarrow \mathbf{P}' = [0 + x'\mathbf{i} + y'\mathbf{j} + z'\mathbf{k}]$$

Rotations with Quaternions

□ Carry out the quaternion operation $\mathbf{P}' = q\mathbf{P}q^{-1}$

$$q\mathbf{P}q^{-1} = (s, \mathbf{v})(0, \mathbf{p})(s, -\mathbf{v})$$

$$= (s \cdot 0 - \mathbf{v} \cdot \mathbf{p}, s\mathbf{p} + 0\mathbf{v} + \mathbf{v} \times \mathbf{p})(s, -\mathbf{v})$$

$$= (-\mathbf{v} \cdot \mathbf{p}, s\mathbf{p} + \mathbf{v} \times \mathbf{p})(s, -\mathbf{v})$$

$$= [-s\mathbf{v} \cdot \mathbf{p} + (s\mathbf{p} + \mathbf{v} \times \mathbf{p}) \cdot \mathbf{v}, (\mathbf{v} \cdot \mathbf{p})\mathbf{v} + s(s\mathbf{p} + \mathbf{v} \times \mathbf{p}) - (s\mathbf{p} + \mathbf{v} \times \mathbf{p}) \times \mathbf{v}]$$

$$= [(\mathbf{v} \times \mathbf{p}) \cdot \mathbf{v}, (\mathbf{v} \cdot \mathbf{p})\mathbf{v} + s^2\mathbf{p} + s(\mathbf{v} \times \mathbf{p}) - s(\mathbf{p} \times \mathbf{v}) - \mathbf{v} \times \mathbf{p} \times \mathbf{v}]$$

$$= [0, (\mathbf{v} \cdot \mathbf{p})\mathbf{v} + s^2\mathbf{p} + 2s(\mathbf{v} \times \mathbf{p}) + \mathbf{v} \times \mathbf{v} \times \mathbf{p}]$$

$$= (\mathbf{v} \cdot \mathbf{p})\mathbf{v} + s^2\mathbf{p} + 2s(\mathbf{v} \times \mathbf{p}) + (\mathbf{v} \cdot \mathbf{p})\mathbf{v} - \mathbf{v}^2\mathbf{p}$$

$$q\mathbf{P}q^{-1} = (s^2 - \mathbf{v}^2)\mathbf{P} + 2s\mathbf{v} \times \mathbf{P} + 2(\mathbf{v} \cdot \mathbf{P})\mathbf{v}$$

Since, $\mathbf{A} \cdot (\mathbf{A} \times \mathbf{B}) = 0$, $\mathbf{A} \times \mathbf{B}$ is perpendicular to $\mathbf{A}$

since Since $\mathbf{A} \times (\mathbf{B} \times \mathbf{C}) = (\mathbf{A} \cdot \mathbf{C})\mathbf{B} - (\mathbf{A} \cdot \mathbf{B})\mathbf{C}$

Rotations with Quaternions: Matrix form

□ Carry out the quaternion operation $\mathbf{P}' = q\mathbf{P}q^{-1}$

$$q\mathbf{P}q^{-1} = (s + \mathbf{v})\mathbf{P}(s - \mathbf{v})$$

$$q\mathbf{P}q^{-1} = (s^2 - \mathbf{v}^2)\mathbf{P} + 2s\mathbf{v} \times \mathbf{P} + 2(\mathbf{v} \cdot \mathbf{P})\mathbf{v}$$

$$= (s + ai + bj + ck)(xi + yj + zk)(s - ai - bj - ck)$$

$$= [x(s^2 + a^2 - b^2 - c^2) + 2y(ab - sc) + 2z(ac + sb)]\mathbf{i} +$$

$$[2x(ab + sc) + y(s^2 - a^2 + b^2 - c^2) + 2z(bc - sa)]\mathbf{j} +$$

$$[2x(ac - sb) + 2y(bc + sa) + z(s^2 - a^2 - b^2 + c^2)]\mathbf{k}$$

$$= \begin{bmatrix} i & j & k \end{bmatrix} \begin{bmatrix} s^2 + a^2 - b^2 - c^2 & 2(ab - sc) & 2(ac + sb) \\ 2(ab + sc) & (s^2 - a^2 + b^2 - c^2) & 2(bc - sa) \\ 2(ac - sb) & 2(bc + sa) & (s^2 - a^2 - b^2 + c^2) \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$\mathbf{P}' = \mathbf{M}_{R(\theta)} \cdot \mathbf{P}$$

Rotations with Quaternions : Matrix form

□ Since $|q|^2 = s^2 + a^2 + b^2 + c^2 = 1$

$$\mathbf{M}_{R(\theta)} = \begin{bmatrix} 1 - 2b^2 - 2c^2 & 2ab - 2sc & 2ac + 2sb \\ 2ab + 2sc & 1 - 2a^2 - 2c^2 & 2bc - 2sa \\ 2ac - 2sb & 2bc + 2sa & 1 - 2a^2 - 2b^2 \end{bmatrix}$$
$$= \begin{bmatrix} u_x^2(1 - \cos \theta) + \cos \theta & u_x u_y(1 - \cos \theta) - u_z \sin \theta & u_x u_z(1 - \cos \theta) + u_y \sin \theta \\ u_y u_x(1 - \cos \theta) + u_z \sin \theta & u_y^2(1 - \cos \theta) + \cos \theta & u_y u_z(1 - \cos \theta) - u_x \sin \theta \\ u_z u_x(1 - \cos \theta) - u_y \sin \theta & u_z u_y(1 - \cos \theta) + u_x \sin \theta & u_z^2(1 - \cos \theta) + \cos \theta \end{bmatrix}$$

Putting explicit values of $q = (s, a, b, c)$, and apply following trigonometry

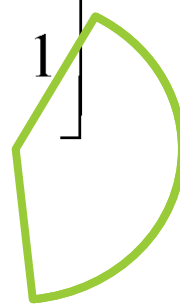
$$\cos^2 \frac{\theta}{2} - \sin^2 \frac{\theta}{2} = 1 - 2 \sin^2 \frac{\theta}{2} = \cos \theta, \quad 2 \cos \frac{\theta}{2} \sin \frac{\theta}{2} = \sin \theta$$

Example

□ Rotation about z axis

➤ Set the unit quaternion: $s = \cos \frac{\theta}{2}, \mathbf{v} = (0, 0, 1) \sin \frac{\theta}{2}$

➤ Substitute $a=b=0, c=\sin(\theta/2)$ into the matrix:

$$\mathbf{M}_R(\theta) = \begin{bmatrix} 1 - 2\sin^2 \frac{\theta}{2} & -2\cos \frac{\theta}{2} \sin \frac{\theta}{2} & 0 \\ 2\cos \frac{\theta}{2} \sin \frac{\theta}{2} & 1 - 2\sin^2 \frac{\theta}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$1 - 2\sin^2 \frac{\theta}{2} = \cos \theta$
 $2\cos \frac{\theta}{2} \sin \frac{\theta}{2} = \sin \theta$

Rotation about a Line using Quaternion

□ Let L be represented by vector V and passing through point P

1. Translate P to the origin
2. Use $M_{R(\theta)}$ for rotation
3. Reverse step 1

$$R_{\theta,L} = T_P * M_{R(\theta)} * T_{-P}$$

$$\begin{aligned} \text{i.e.} \quad M_{R(\theta)} &= A_V^{-1} * R_{\theta,k} * A_V \\ &= R_{\alpha,i} * R_{\beta,j} * R_{\theta,k} * R_{-\beta,i} * R_{\alpha,i} \end{aligned}$$

Rodrigues' rotation formula

$$\mathbf{M}_{R(\theta)} = \begin{bmatrix} u_x^2(1 - \cos \theta) + \cos \theta & u_x u_y(1 - \cos \theta) - u_z \sin \theta & u_x u_z(1 - \cos \theta) + u_y \sin \theta \\ u_y u_x(1 - \cos \theta) + u_z \sin \theta & u_y^2(1 - \cos \theta) + \cos \theta & u_y u_z(1 - \cos \theta) - u_x \sin \theta \\ u_z u_x(1 - \cos \theta) - u_y \sin \theta & u_z u_y(1 - \cos \theta) + u_x \sin \theta & u_z^2(1 - \cos \theta) + \cos \theta \end{bmatrix}$$

$$= \sin \theta \begin{bmatrix} 0 & -u_z & u_y \\ u_z & 0 & -u_x \\ -u_y & u_x & 0 \end{bmatrix} + (1 - \cos \theta) \begin{bmatrix} -u_y^2 - u_z^2 & u_x u_y & u_x u_z \\ u_x u_y & -u_x^2 - u_z^2 & u_y u_z \\ u_x u_z & u_y u_z & -u_x^2 - u_y^2 \end{bmatrix} + \mathbf{I}$$

$$= \sin \theta \mathbf{K} + (1 - \cos \theta) \mathbf{K}^2 + \mathbf{I}$$

K is linear a transformation matrix that maps vector $\mathbf{v} \xrightarrow{\mathbf{K}} \mathbf{u} \times \mathbf{v}$
 K is formed from another vector $\mathbf{u}$
Here, $\mathbf{u}$ is the axis of rotation

Finding an Axis and Angle from a Rotation Matrix

$$\begin{aligned} M_{R(\theta)} - M_{R(\theta)}^T &= \sin \theta \mathbf{K} + (1 - \cos \theta) \mathbf{K}^2 + \mathbf{I} - \\ &\quad (\sin \theta \mathbf{K}^T + (1 - \cos \theta)(\mathbf{K}^2)^T + \mathbf{I}^T) \\ &= 2 \sin \theta \mathbf{K} \quad \text{Since } \mathbf{K}^T = -\mathbf{K} \\ &\quad (\mathbf{K}^2)^T = \mathbf{K}^2 \end{aligned}$$

Trace (rotation matrix) = $1 + 2\cos\theta$

when $\sin \theta \neq 0$, We can get axis of rotation

DIY

- ❑ Exhibit a unit quaternion that performs a rotation of 60 degrees about the axis 0,3,4 .
- ❑ Consider the following matrix R that represents a 3D rotation. Determine the axis and the angle of the rotation that the above matrix represents

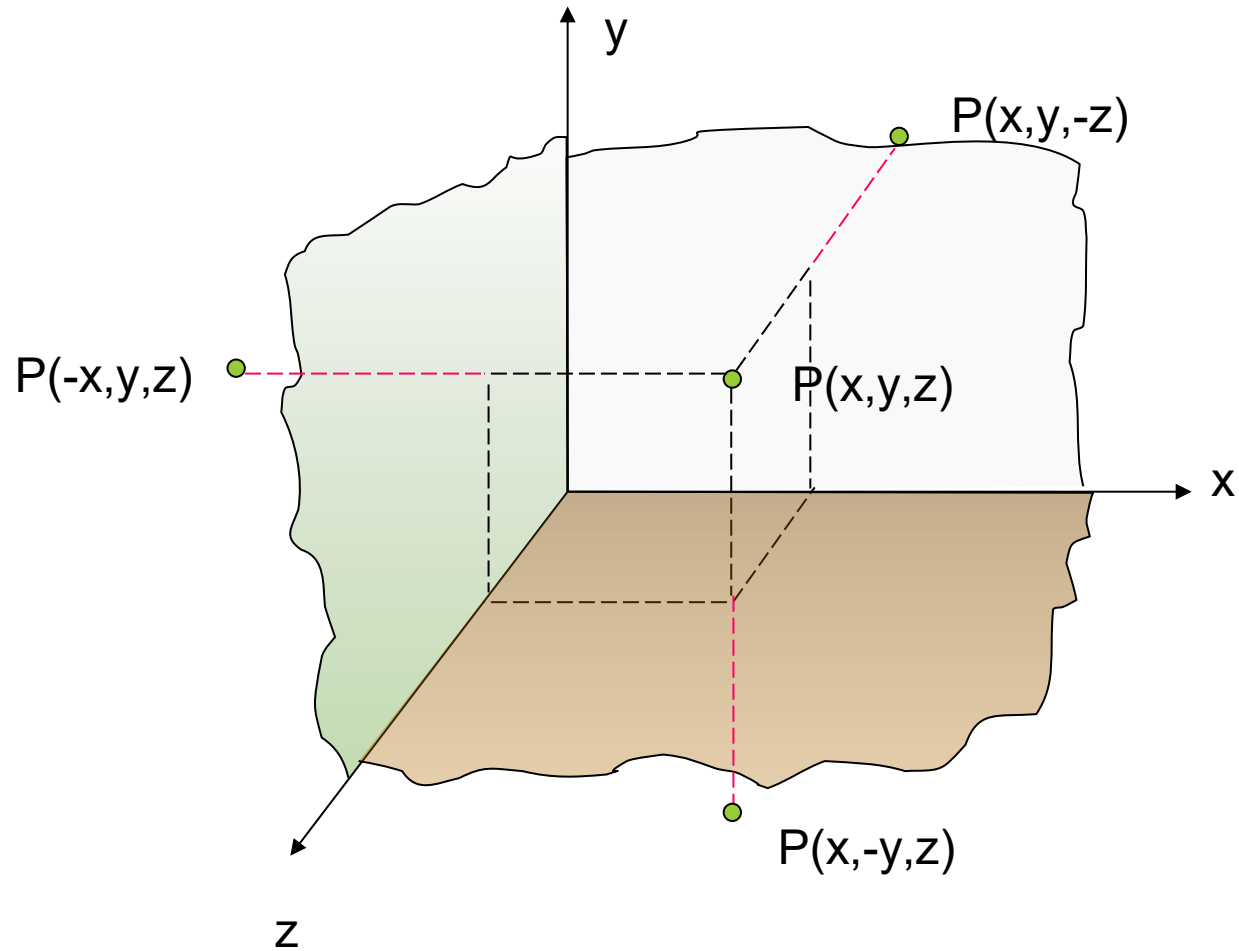
$$R = \begin{bmatrix} \frac{2}{3} & -\frac{1}{3} & \frac{2}{3} \\ \frac{2}{3} & \frac{2}{3} & -\frac{1}{3} \\ -\frac{1}{3} & \frac{2}{3} & \frac{2}{3} \end{bmatrix}$$

Use trace of a rotation matrix, $1+2\cos\theta$

Further Study

- Mathematics for Computer Graphics
John Vince
- Mathematics for 3D Game Programming and
Computer Graphics
Eric Lengyel

Mirror reflection

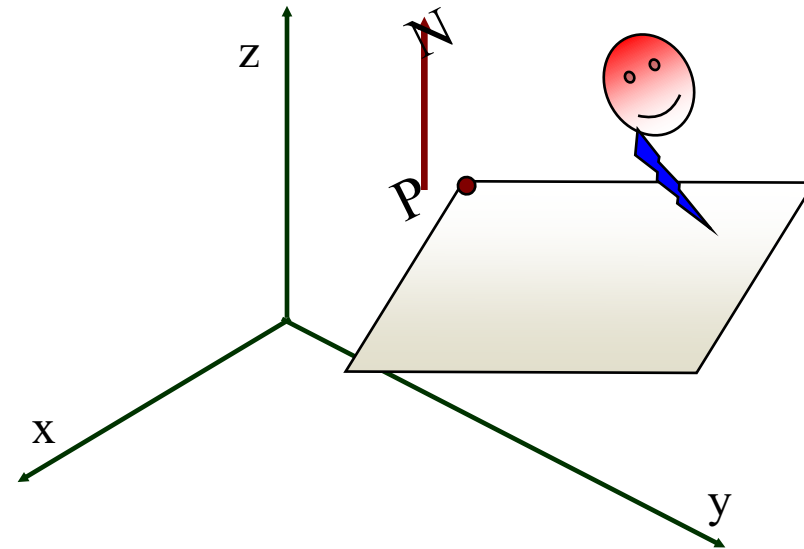


Mirror reflection

$$M_{xy} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad M_{yz} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$
$$M_{zx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$M_{N,P}$: Mirror reflection

- Let the plane be represented by plane normal N and a point P in that plane

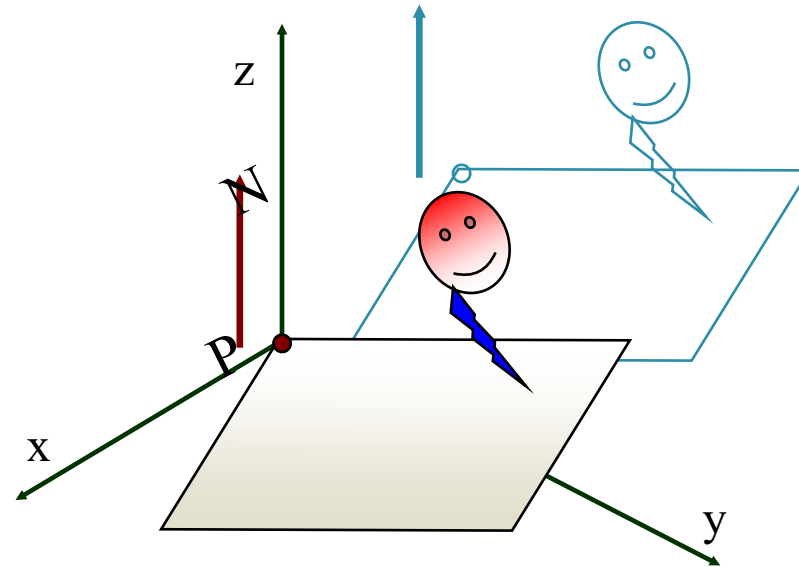


$$M_{N,P} =$$

$M_{N,P}$: Mirror reflection

- Let the plane be represented by plane normal N and a point P in that plane

1. Translate P to the origin



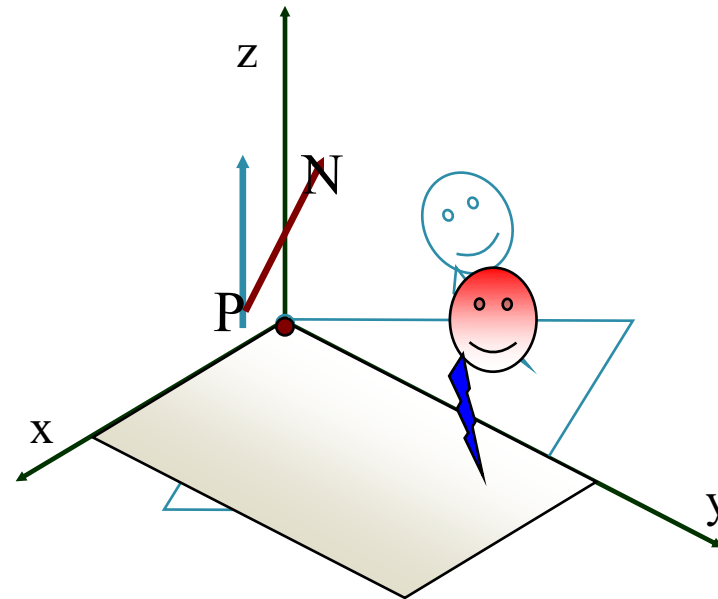
$$M_{N,P} =$$

$$T_{-P}$$

$M_{N,P}$: Mirror reflection

□ Let the plane be represented by plane normal N and a point P in that plane

1. Translate P to the origin
2. Align N with vector k



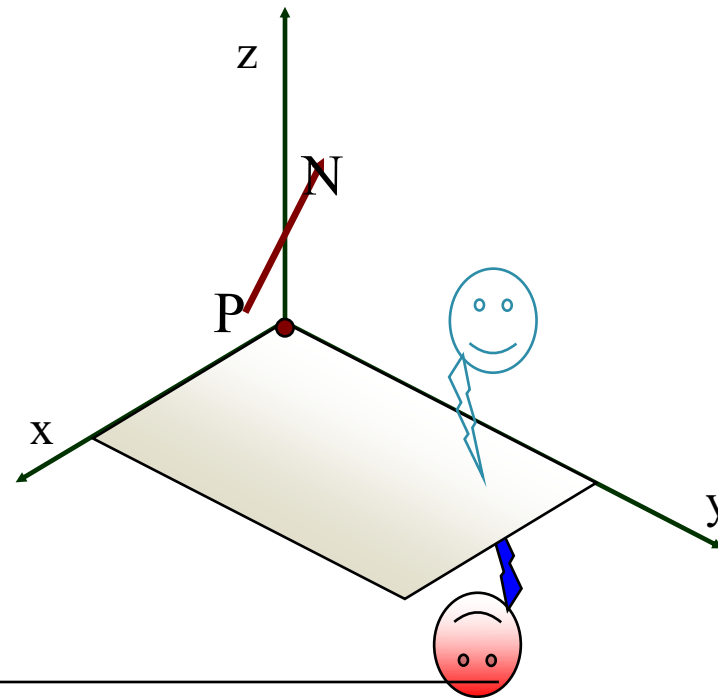
$$M_{N,P} =$$

$$A_N * T_{-P}$$

$M_{N,P}$: Mirror reflection

□ Let the plane be represented by plane normal N and a point P in that plane

1. Translate P to the origin
2. Align N with vector k
3. Reflect w.r.t xy -plane

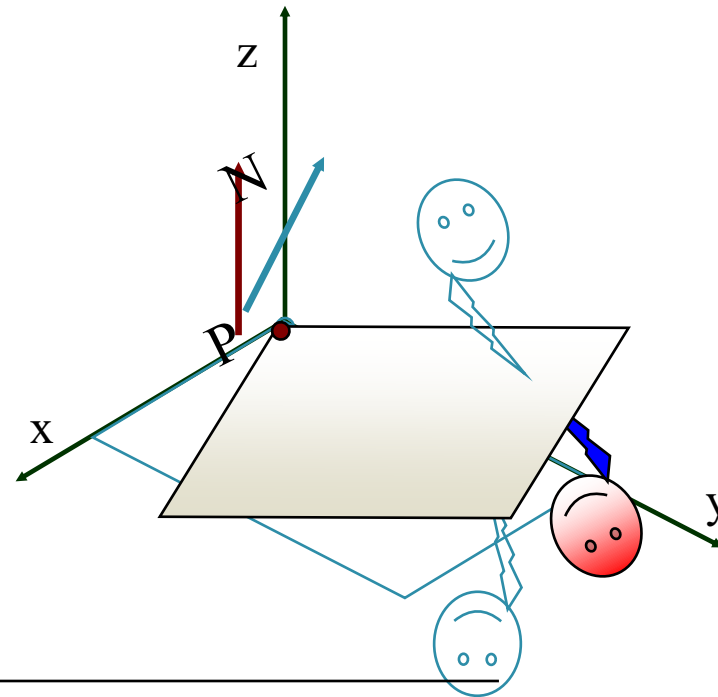


$$M_{N,P} = M_{xy} * A_N * T_{-P}$$

$M_{N,P}$: Mirror reflection

□ Let the plane be represented by plane normal N and a point P in that plane

1. Translate P to the origin
2. Align N with vector k
3. Reflect w.r.t xy -plane
4. Reverse step 2

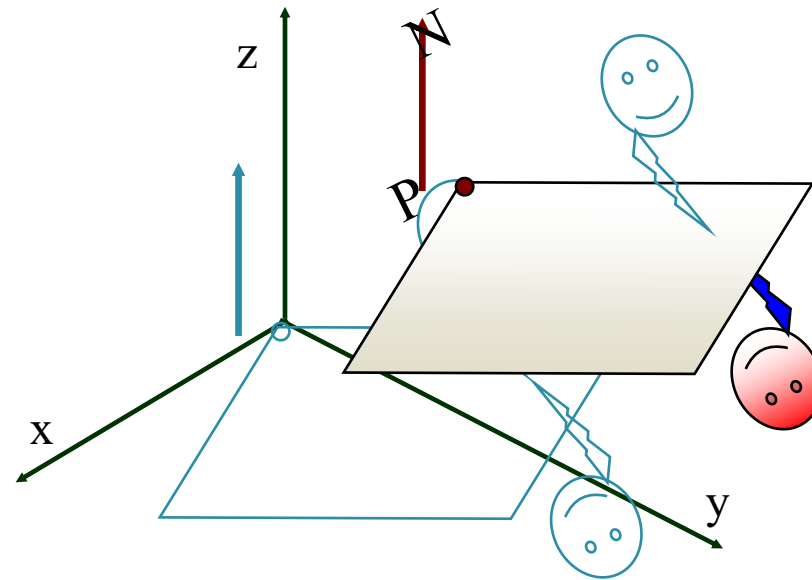


$$M_{N,P} = A_N^{-1} * M_{xy} * A_N * T_{-P}$$

$M_{N,P}$: Mirror reflection

□ Let the plane be represented by plane normal N and a point P in that plane

1. Translate P to the origin
2. Align N with vector k
3. Reflect w.r.t xy -plane
4. Reverse step 2
5. Reverse step 1

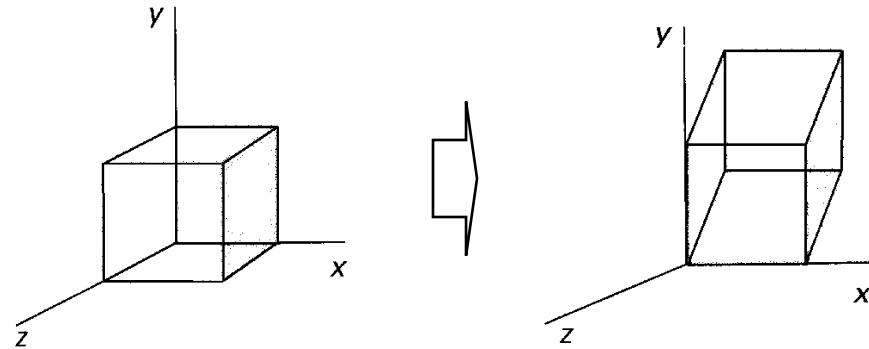


$$M_{N,P} = T_{-P}^{-1} * A_N^{-1} * M_{xy} * A_N * T_{-P}$$

Other Transformations

□ Z-axis Shear

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$



Solve it

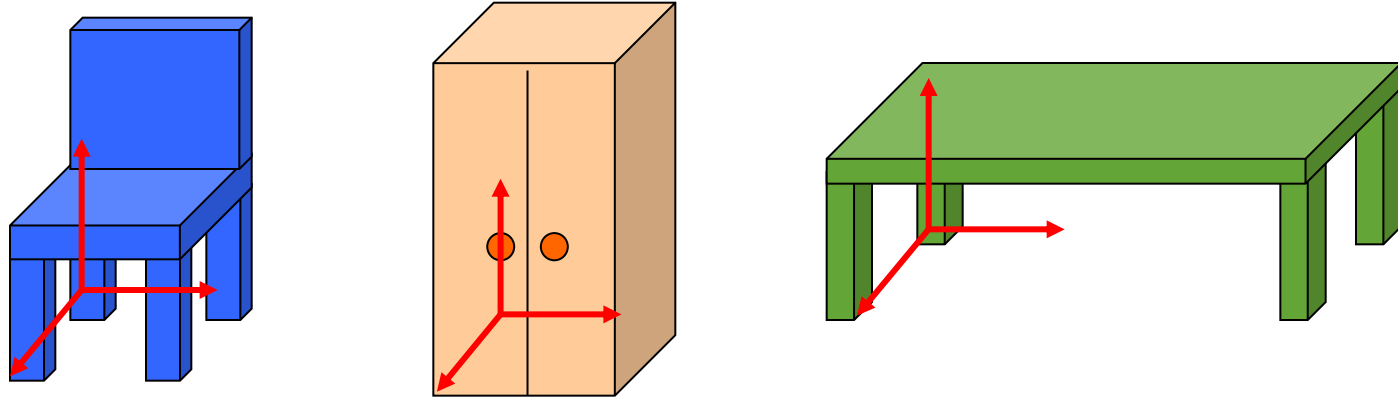
Consider a polygon with points $P_1 (2,1,1)$, $P_2(4,2,3)$ and $P_3 (3,3,1)$. Find a composite transformation matrix to

1. Place the polygon in positive yz plane
2. Place P_1 at origin and P_1P_2 along positive z axis

Coordinate Transformations

❑ Multiple Coordinate System

- Local (modeling) coordinate system
- World coordinate scene

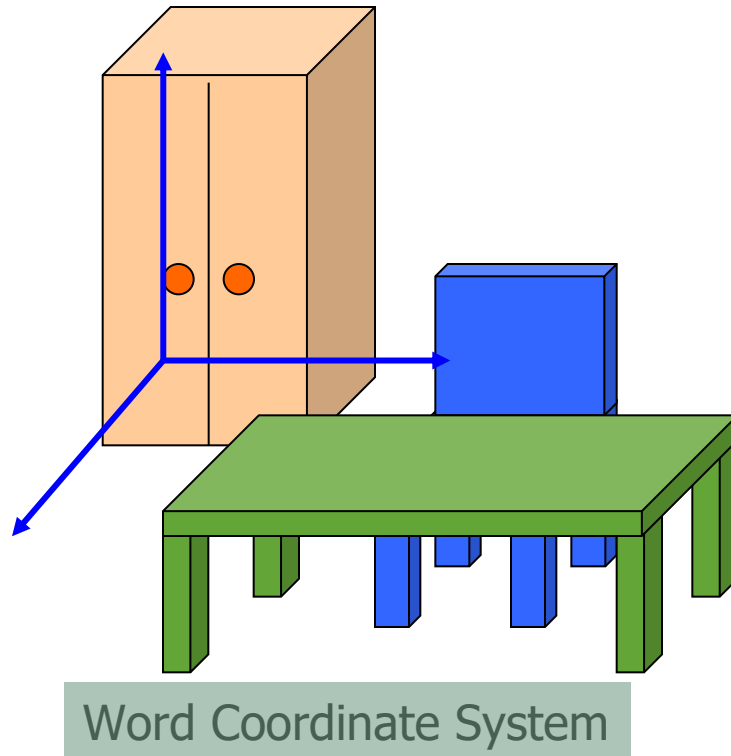


Local Coordinate System

Coordinate Transformations

❑ Multiple Coordinate System

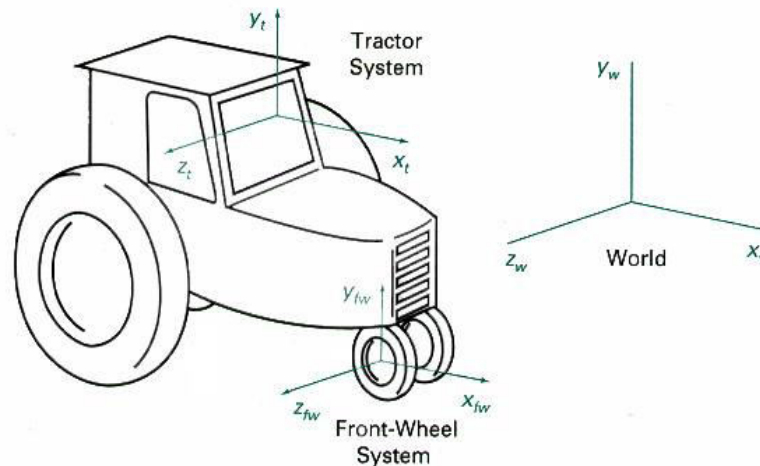
- Local (modeling) coordinate system
- World coordinate scene



Coordinate Transformations

□ Example – Simulation of Tractor movement

- As tractor moves, **tractor coordinate system** and **front-wheel coordinate system** move in world coordinate system
- **Front wheels** rotate in wheel coordinate system



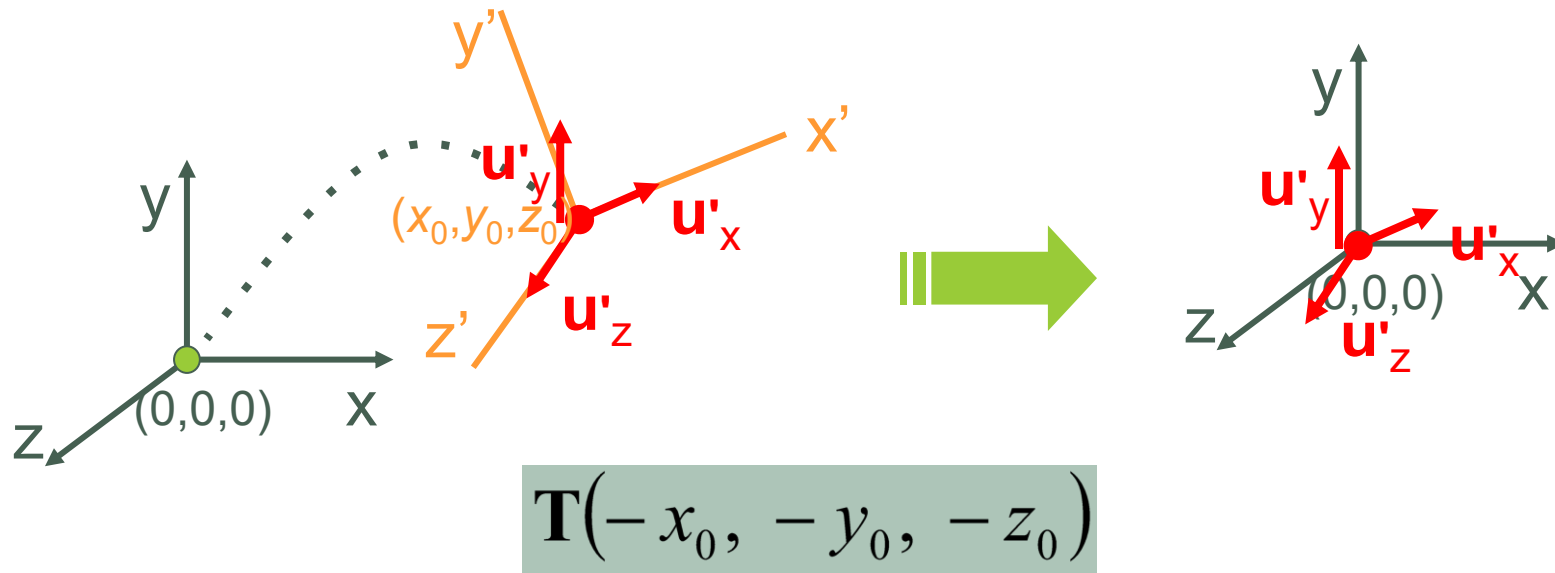
Coordinate Transformations

❑ Transformation of an Object Description from One Coordinate System to Another

❑ Transformation Matrix

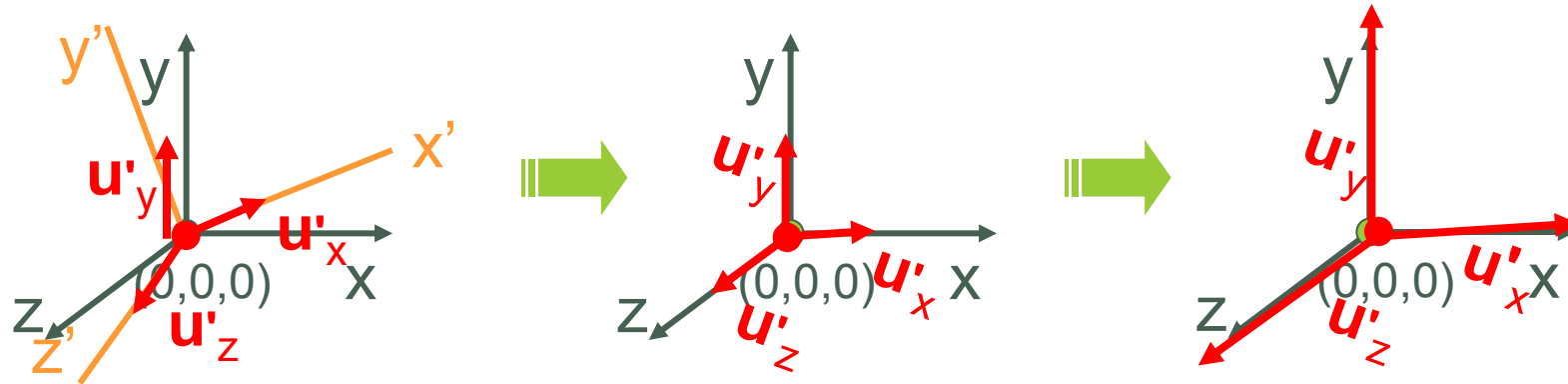
➤ Bring the two coordinates systems into alignment

1. Translation



Coordinate Transformations

2. Rotation



$$\mathbf{R} = \begin{bmatrix} u'_{x_1} & u'_{x_2} & u'_{x_3} & 0 \\ u'_{y_1} & u'_{y_2} & u'_{y_3} & 0 \\ u'_{z_1} & u'_{z_2} & u'_{z_3} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Problems to be solved:

Schaum's outline series:

Problems:

□ 6.1

□ 6.2, 6.5, 6.9, 6.10, 6.11, 6.12 $\Rightarrow A_v$

□ 6.3, 6.4 $\Rightarrow R_{\theta,L}$

□ 6.6, 6.7, 6.8 $\Rightarrow M_{N,P}$

Acknowledgement
cgvr.korea.ac.kr

Problems to be solved:

3. Consider a 2D rectangle $ABCD$ where $A = (0, 0)$, $B = (2, 0)$, $C = (2, 1)$ and $D = (0, 1)$. We want to apply a 2D transformation to this rectangle which makes it a parallelopiped $ABEF$ where $E = (4, 1)$ and $F = (2, 1)$. a. What kind of transformation is this? What is the 3x3 matrix M achieving this transformation? c. What additional transformation N would we need to apply to $ABEF$ to get the parallelopiped $A'B'E'F'$ where $A' = (1, 2)$, $B' = (3, 2)$, $E' = (5, 3)$, and $F' = (3, 3)$? d. What is the final concatenated matrix in terms of M and N that will transform $ABCD$ to $A'B'E'F'$?
4. Derive the scaling matrix for scaling an object by a scale factor 3 along an arbitrary direction given by vector $u = (1, 2, 1)$ rooted at $(5, 5, 5)$.
5. Explain what transformation is produced by each of the following matrices when applied on a 4x1 homogeneous coordinate.

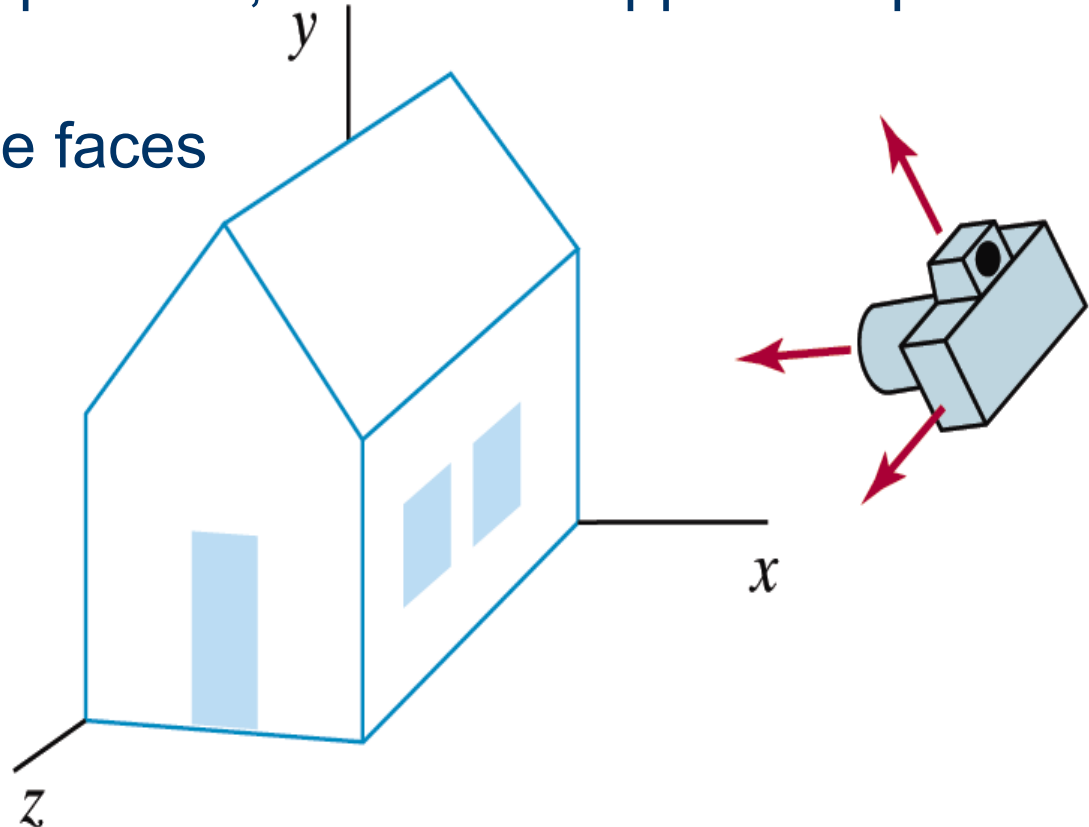
$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad \begin{pmatrix} 1 & 0 & p & -p(1+r) \\ 0 & 1 & q & -q(1+r) \\ 0 & 0 & 1+r & -r(1+r) \\ 0 & 0 & 1 & -r \end{pmatrix} \quad (6.59)$$

3D Viewing Transform

Dr. Sk. Md. Masudul Ahsan
Professor
Dept. of CSE, KUET

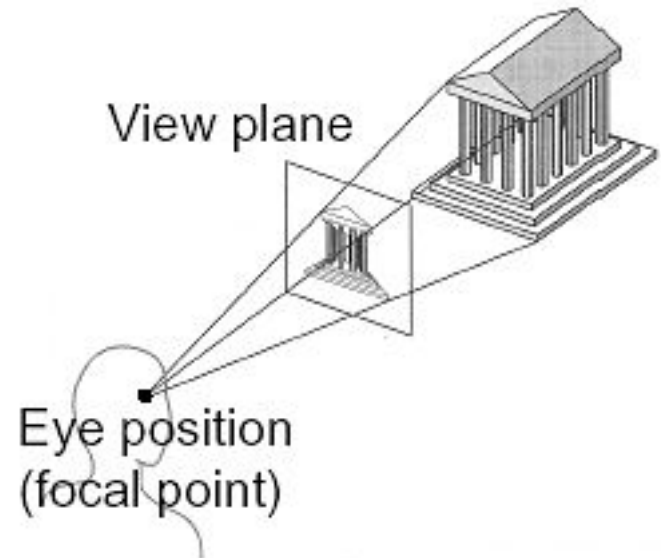
Remember the big thing – Camera Analogy

- 3D view of a scene is analogous to photographing
 - Need to position the camera
 - Need to decide camera orientation
 - When shutter is pressed, scene is cropped to aperture (window) size
 - Light from visible faces projected into film



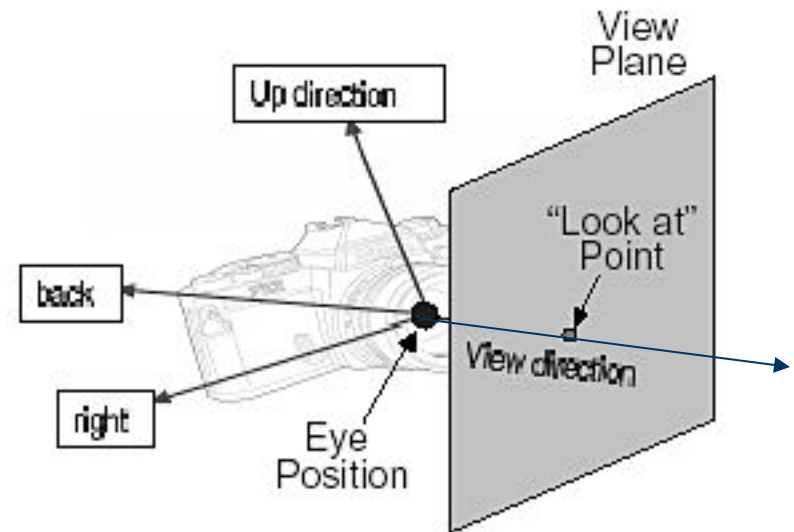
Camera Models

- The most common model is **pin-hole camera**
 - All captured light rays arrive along paths toward focal point without lens distortion (everything is in focus)
 - Sensor response proportional to radiance
- Other models consider...
 - Depth of field
 - Motion blur
 - Lens distortion

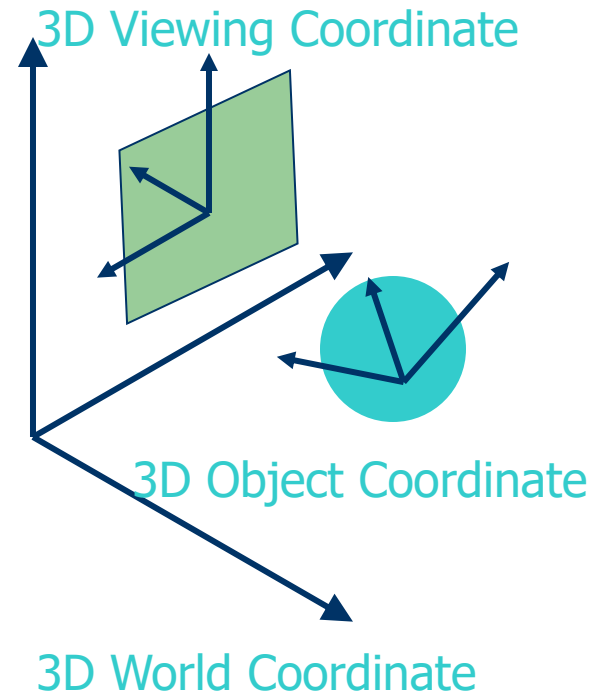
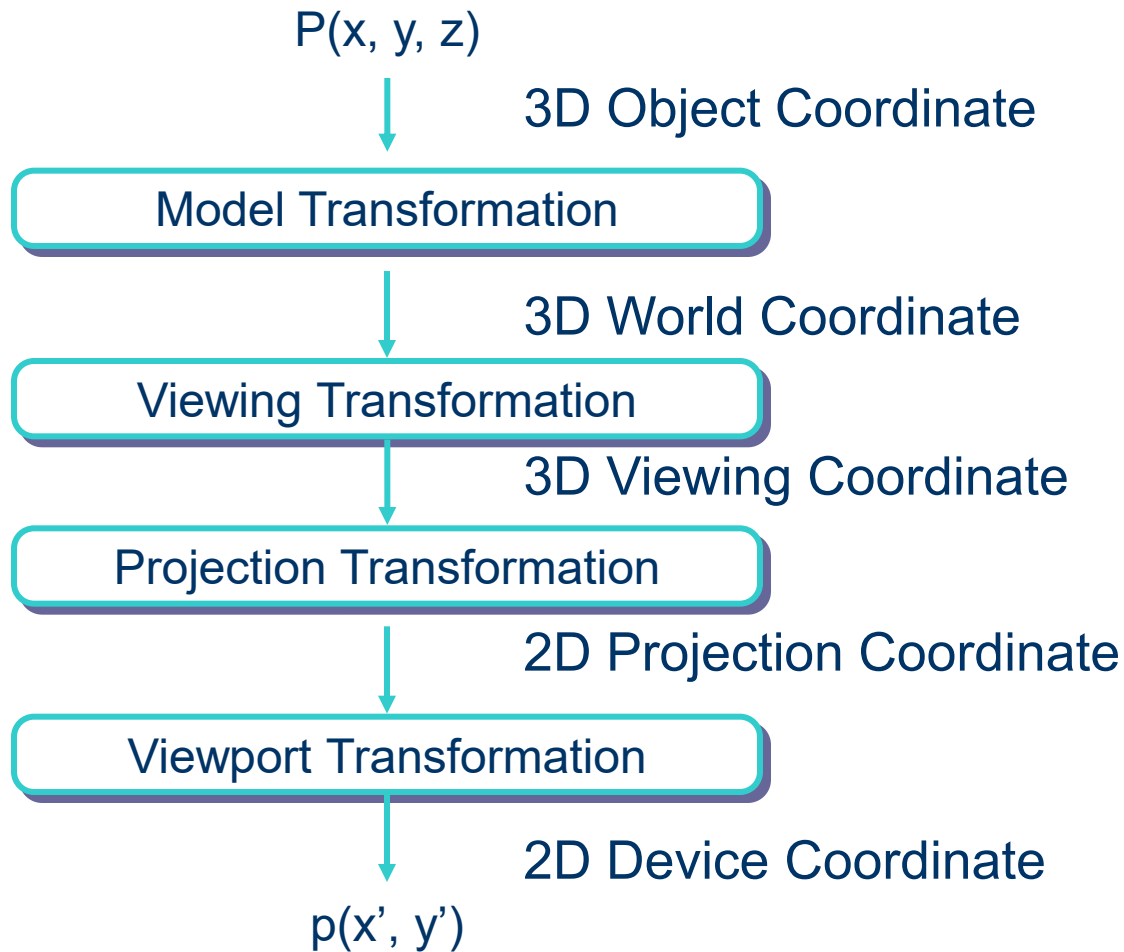


Viewing Parameters

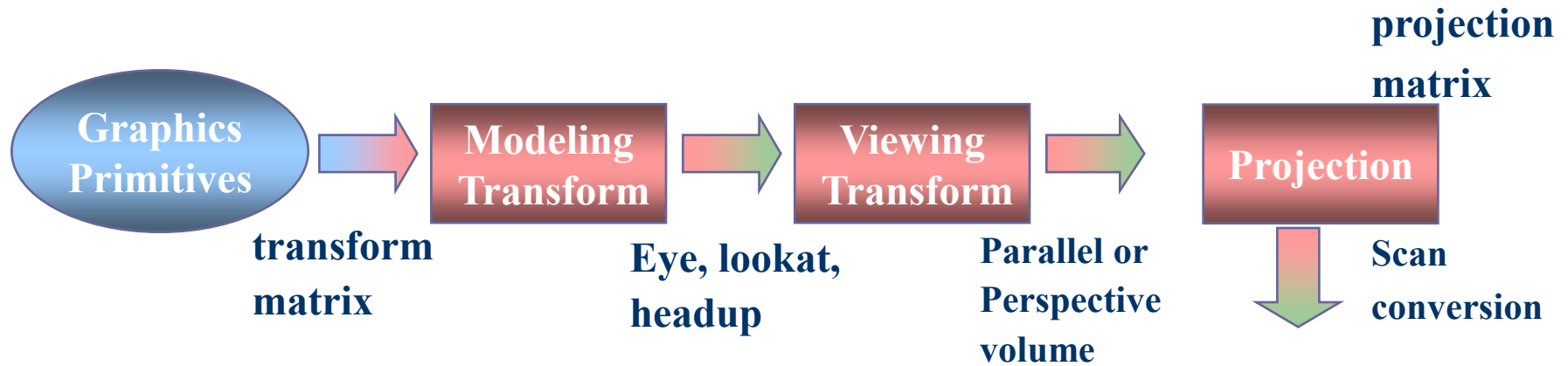
- Position
 - Eye position(p_x , p_y , p_z)
- Orientation
 - View direction(d_x , d_y , d_z)
 - Up direction(u_x , u_y , u_z)
- Aperture
 - Field of view($xfov$, $yfov$)
- Film plane
 - “look at” point
 - View plane normal



Transformations

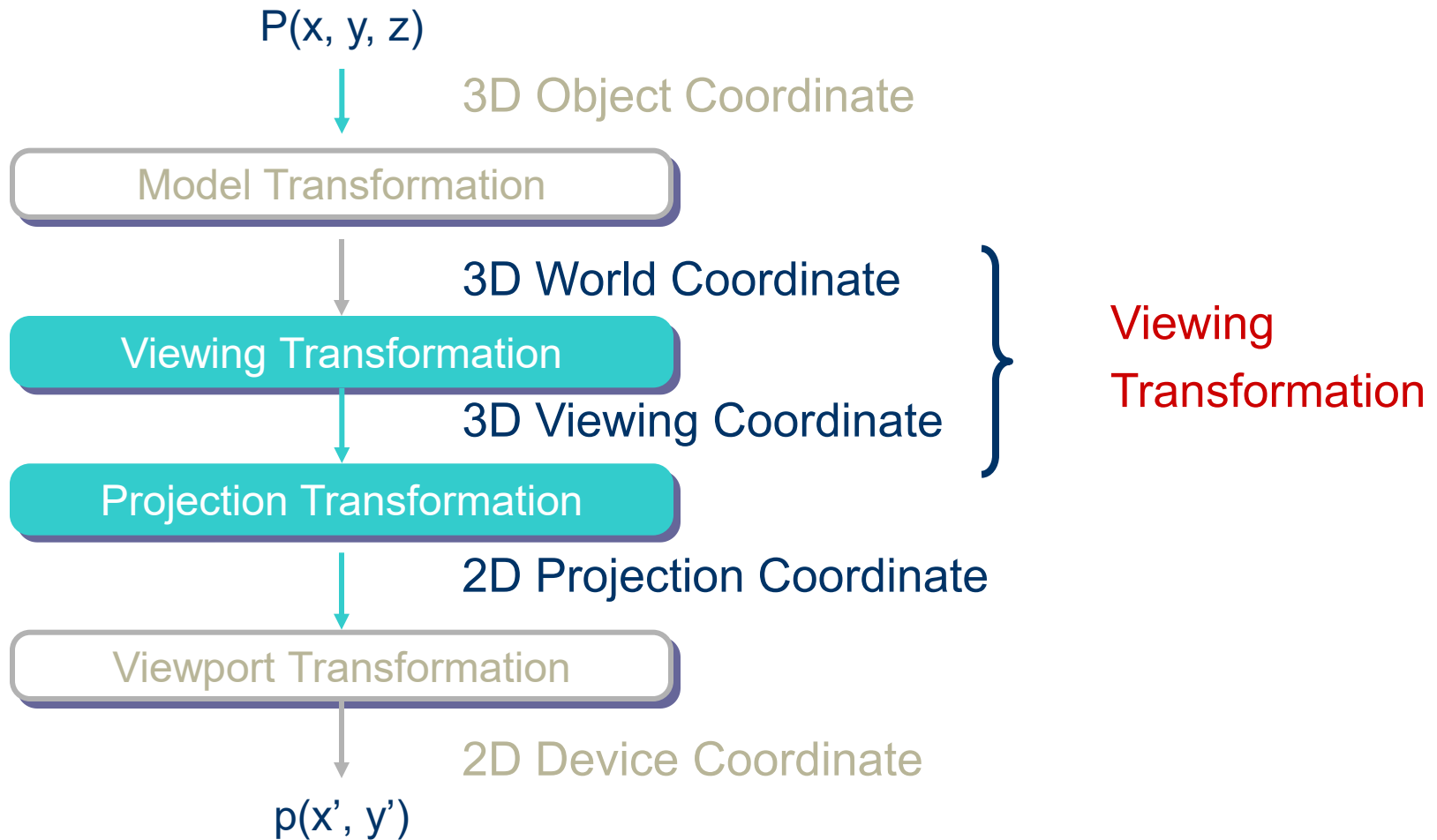


Graphics Pipeline



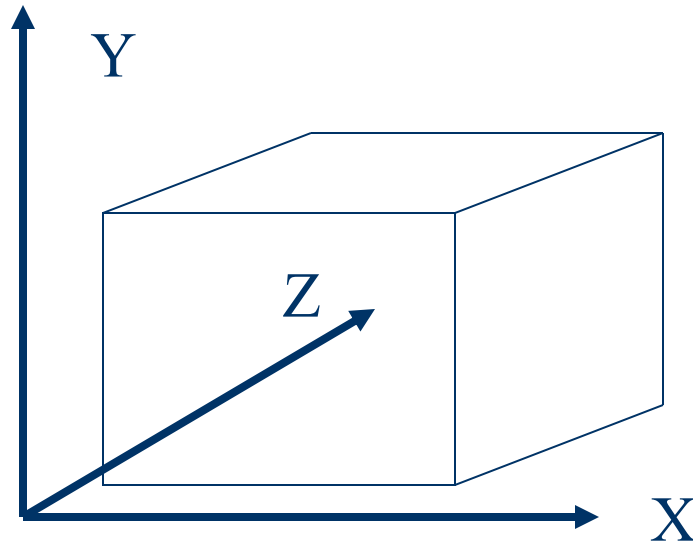
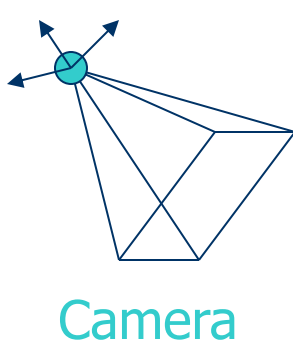
- Once scene has been modeled, world coordinates are converted to viewing coordinates
- Viewing coordinate system is used in graphics package for specifying viewer position and position of projection plane (film)

Viewing Transformation



Viewing Transformation

- Mapping from world to Viewing coordinates
 - Origin moves to eye position
 - Up vector maps to Y axis
 - Right vector maps to X axis



Viewing Coordinate system

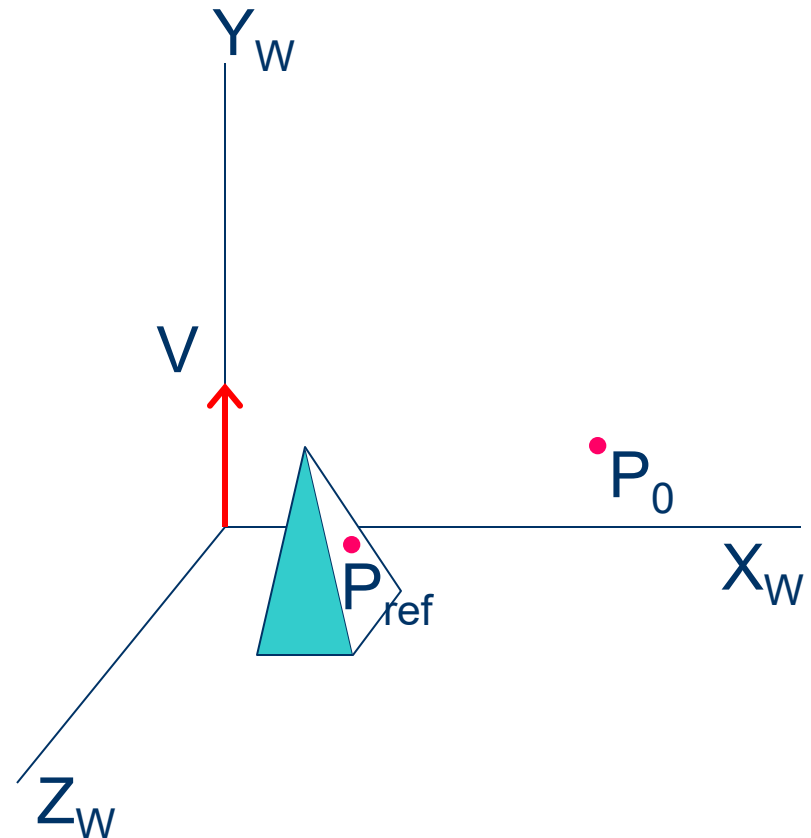
- Particular view of a scene is chosen by
 - Establishing Viewing Coordinate system
 - And then, a View plane (projection plane) is setup perpendicular to the viewing z_v axis.
- To establish Viewing Coordinate system (VCS)
 - First, a world coordinate position (P_0) is picked as **view reference point**, it serves as **origin** of VCS.
 - Then, positive direction of z_v axis is specified by specifying **view plane normal vector (N)**
 - Another world position (P) is picked – *look at*

Viewing Coordinate system

- And finally, up direction of view is specified by **view up vector (V)**.
 - Generally specified by a world coordinate vector
 - It gives an appx. direction of y_v axis
 - Exact value of y_v axis then calculated by graphics package
- Using N and V, graphics package compute third vector U perpendicular to both N and V, to denote x_v axis
 - $U = V \times N$
- Vector denoting y_v axis can be computed by $N \times U$

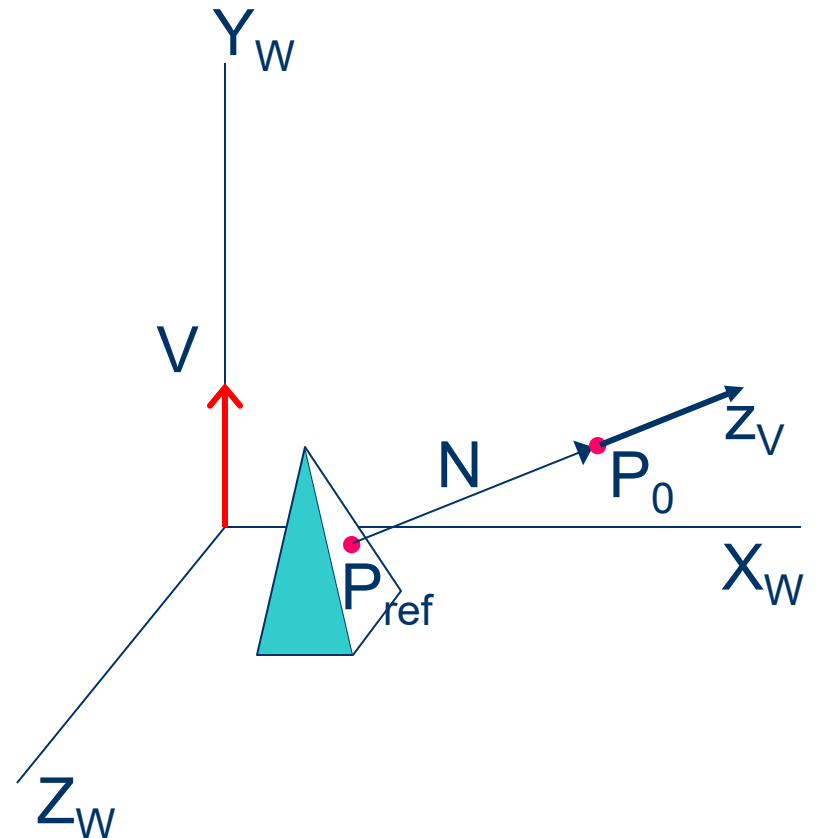
Viewing Coordinate system

- Specification of VCS :
 - P_0 : *View or eye point*
 - P_{ref} : *Center or look-at point*
 - V : *View-up vector*



Viewing Coordinate system

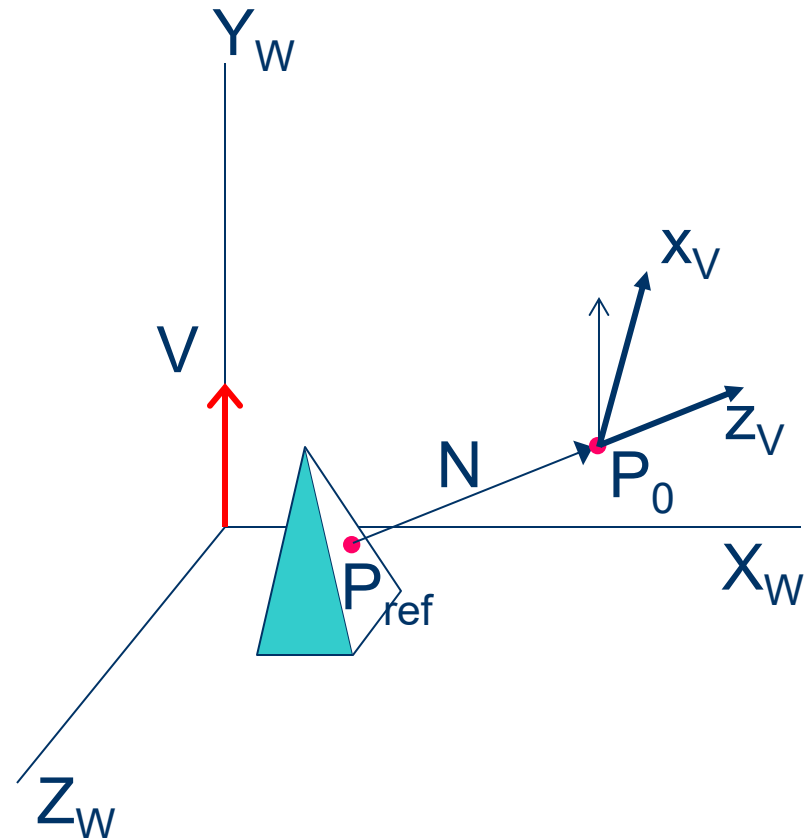
- computation of VCS :
 - $z_v = N = P_0 - P_{ref}$



Viewing Coordinate system

- computation of VCS :

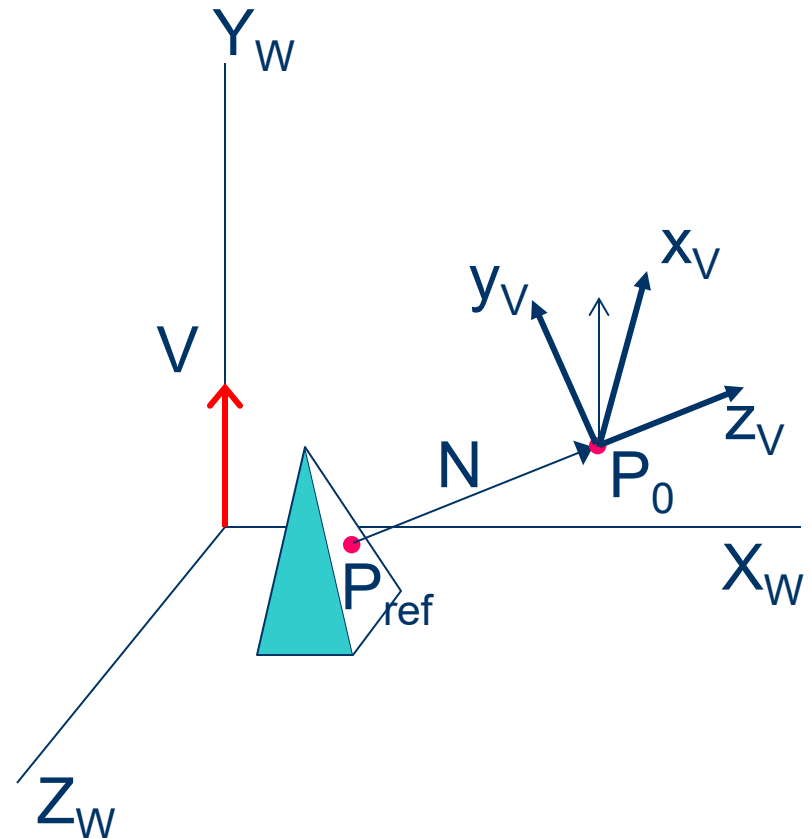
- $z_v = N = P_0 - P$
- $x_v = V \times N = U$



Viewing Coordinate system

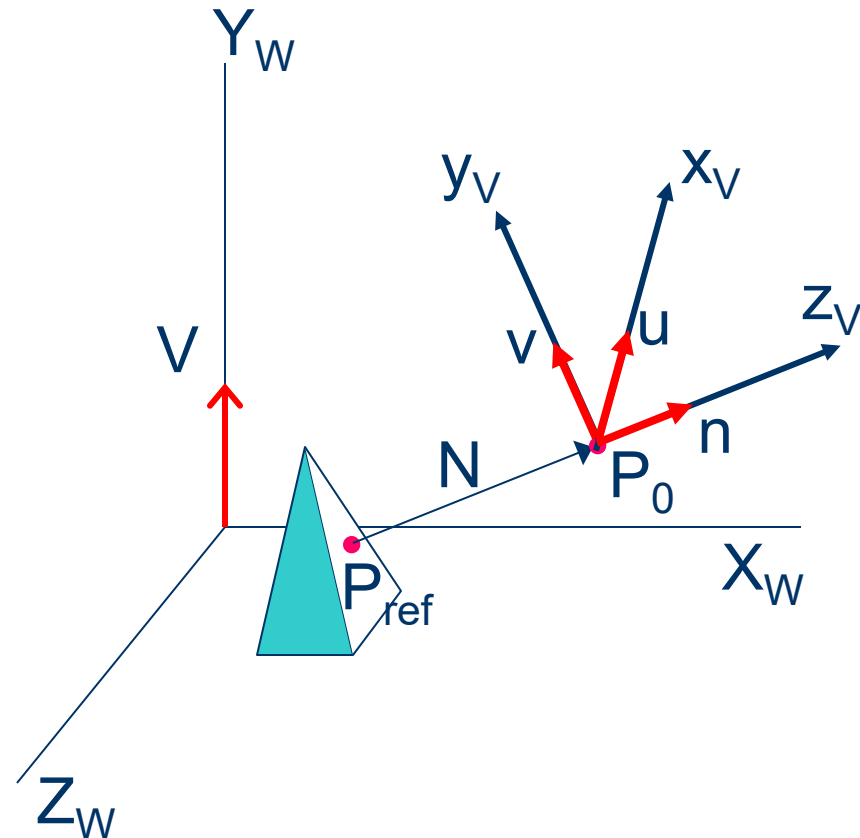
- computation of VCS :

- $z_v = N = P_0 - P$
- $x_v = V \times N = U$
- $y_v = N \times U$

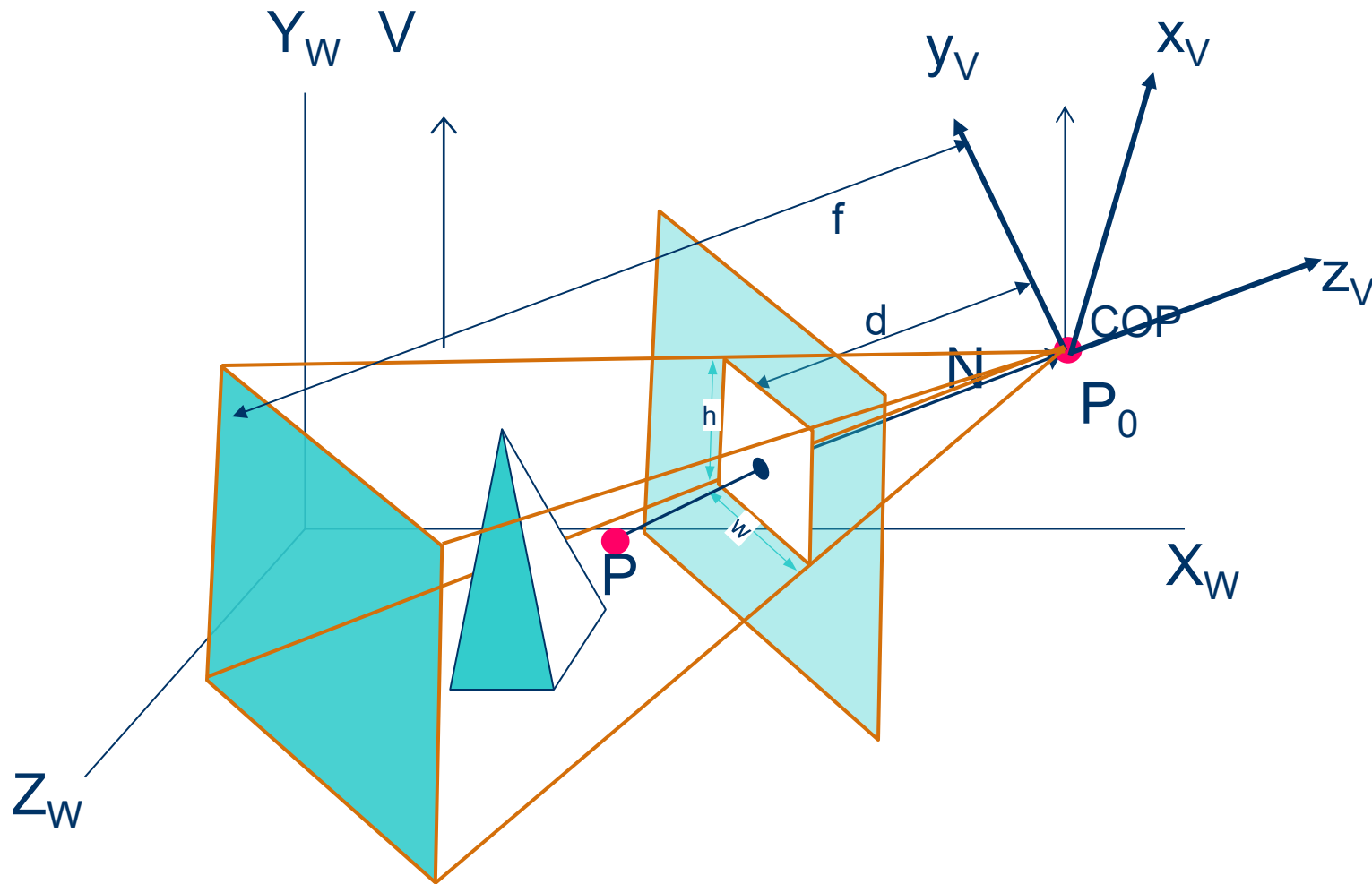


Viewing Coordinate system

- $z_v = N = P_0 - P$
- $x_v = V \times N = U$
- $y_v = N \times U$
- All these computations are actually carried out with unit vectors **u**, **v**, **n**
- And the system is called **uvn** system
- Graphics package allow user to choose view plane position along **n** axis
 - By specifying **view plane distance** from **viewing origin**



WCS, VCS and projection



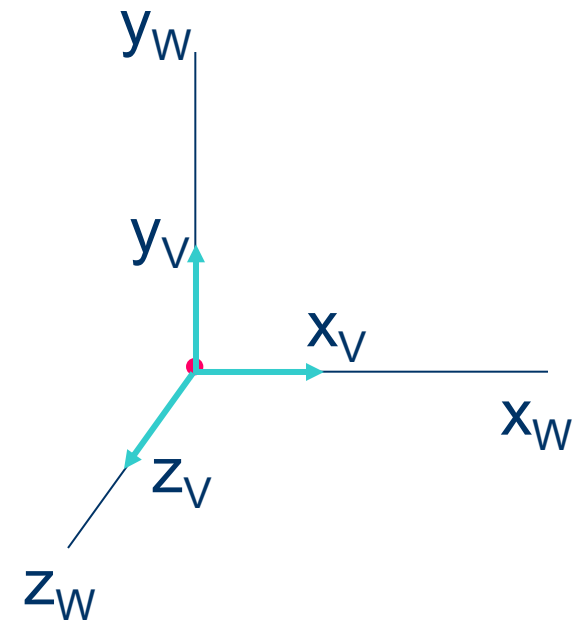
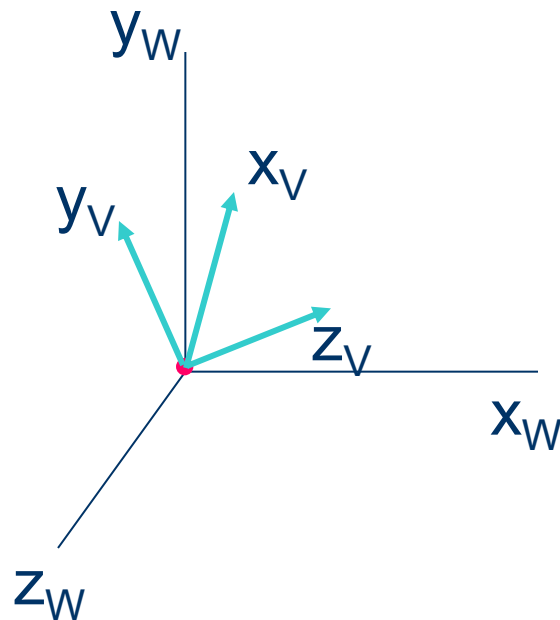
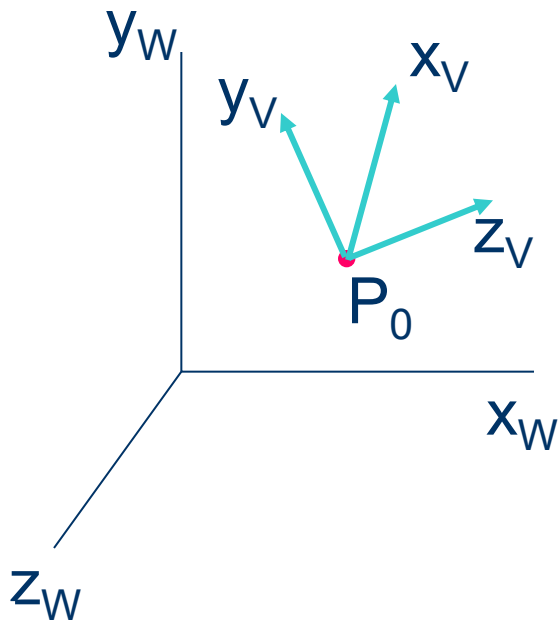
WC to VC transformation

- Before projection, object description must be converted from world to viewing system
- Conversion is eqv. to a transformation that superimposes the viewing system to world system
- This transformation can be achieved by basic transformations
 1. Translate view reference point $P_0(x_0, y_0, z_0)$ to world origin
 2. Apply rotation to align x_V, y_V, z_V axes with x_W, y_W, z_W axes, respectively

WC to VC transformation

- The translation matrix is

$$T = \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Translation from world to viewing coordinates

- Rotation matrix $\mathbf{R}$ can be found by up to three world coordinate axis rotations R_z , R_y , $R_x = \mathbf{R}$
 - R_x places N or z_v to $x_w z_w$ plane
 - R_y aligns z_v to z_w axis
 - R_z aligns y_v to y_w axis
- $M_{WC,VC} = \mathbf{R} \cdot \mathbf{T}$

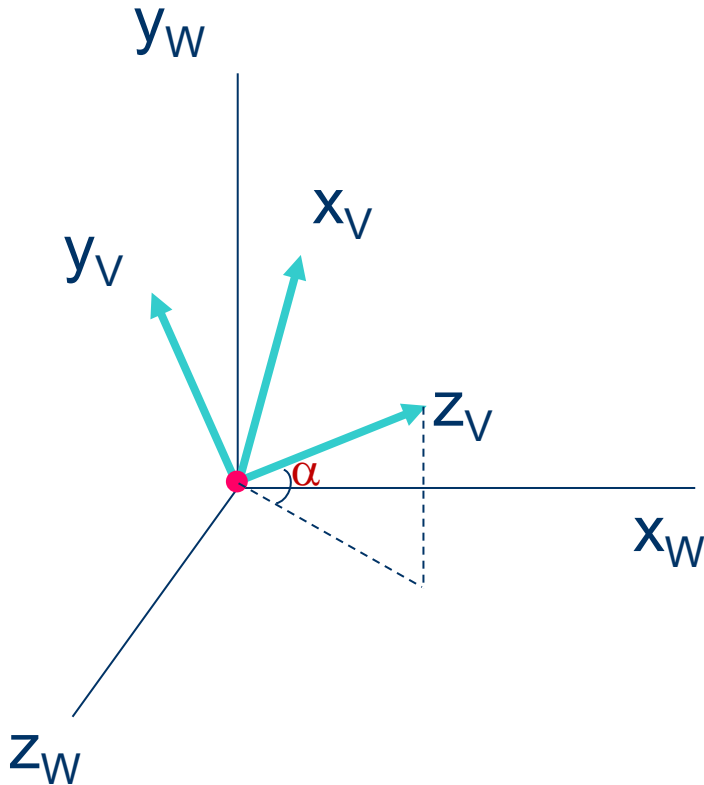
$$n = \frac{N}{|N|} = (n_x, n_y, n_z)$$

$$u = \frac{V \times N}{|V \times N|} = (u_x, u_y, u_z)$$

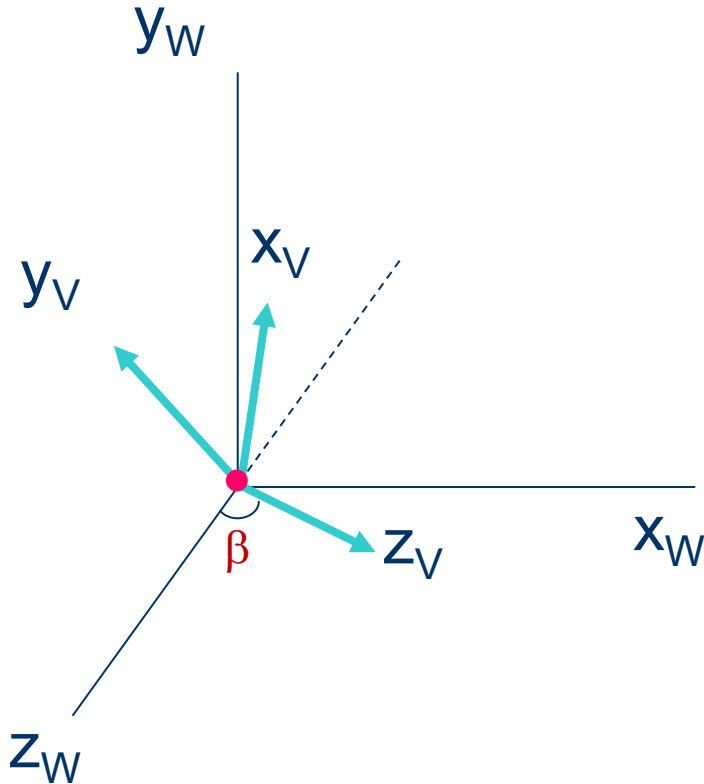
$$v = n \times u = (v_x, v_y, v_z)$$

Composite R

- Rotate α about X_W , R_i

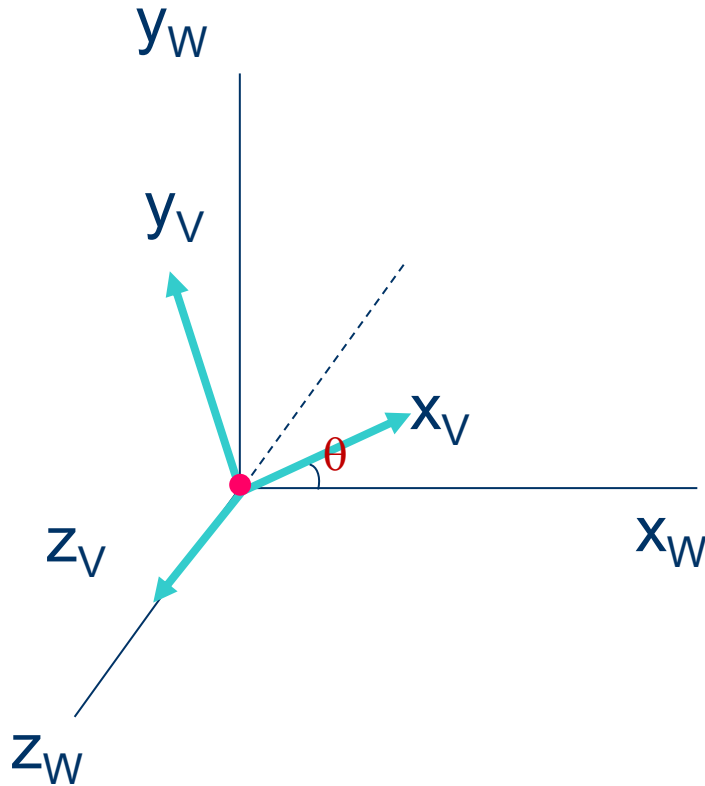


Composite R



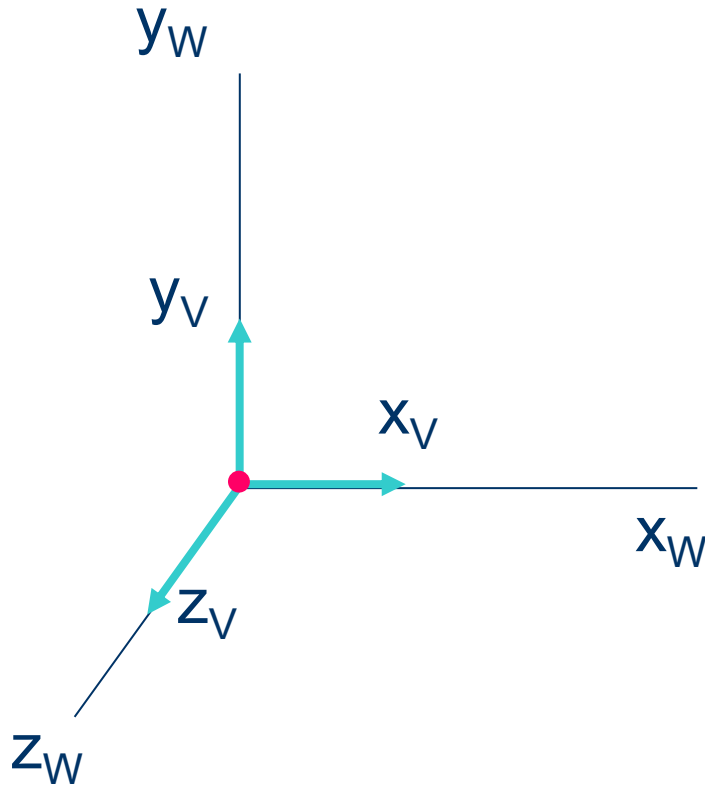
- Rotate α about X_W , R_i
 - Places Z_V in XZ plane
- Rotate β about Y_W , R_j

Composite R



- Rotate α about X_W , R_i
 - Places Z_V in XZ plane
- Rotate β about Y_W , R_j
 - Aligns Z_V with Z_W
 - X_V, Y_V are in XY plane
- Rotate θ about Z_W , R_k

Composite R



- Rotate α about X_W , R_i
 - Places Z_v in XZ plane
- Rotate β about Y_W , R_j
 - Aligns Z_v with Z_w
 - X_v, Y_v are in XY plane
- Rotate θ about Z_W , R_k

Finding composite R

$$\begin{aligned}
 R = R_k R_j R_i &= \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \beta & 0 & \sin \beta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \beta & 0 & \cos \beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta \cos \beta & -\sin \theta & \cos \theta \sin \beta & 0 \\ \sin \theta \cos \beta & \cos \theta & \sin \theta \sin \beta & 0 \\ -\sin \beta & 0 & \cos \beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 &= \begin{bmatrix} \cos \theta \cos \beta & -\sin \theta \cos \alpha + \sin \alpha \cos \theta \sin \beta & \sin \alpha \sin \theta + \cos \alpha \cos \theta \sin \beta & 0 \\ \sin \theta \cos \beta & \cos \theta \cos \alpha + \sin \alpha \sin \theta \sin \beta & -\sin \alpha \cos \theta + \cos \alpha \sin \theta \sin \beta & 0 \\ -\sin \beta & \sin \alpha \cos \beta & \cos \alpha \cos \beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$

R is Rigid-body Transform – i.e. orthogonal matrix

- i) *rows and columns are unit vectors*
- ii) *vectors are perpendicular to each other*

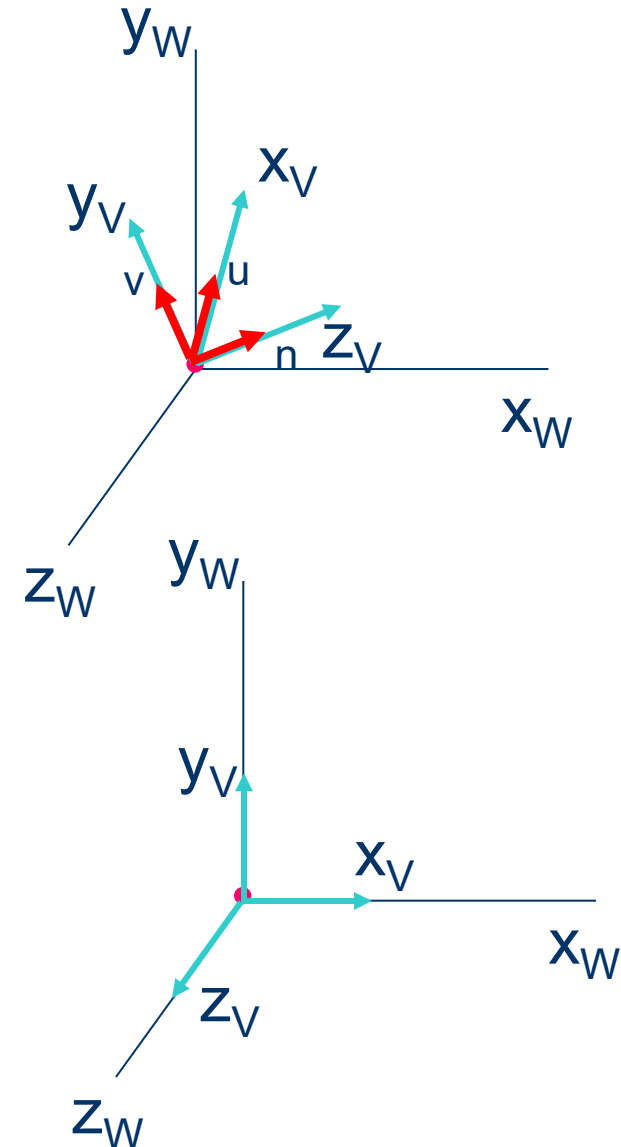
Finding composite R (another way)

Let R be

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} = \begin{bmatrix} R_{1x} & R_{1y} & R_{1z} \\ R_{2x} & R_{2y} & R_{2z} \\ R_{3x} & R_{3y} & R_{3z} \end{bmatrix}$$

Note: $R_{1x} \Rightarrow x$ component of vector $\vec{R}_1$

R aligns unit vectors $\mathbf{u}, \mathbf{v}, \mathbf{n}$ to $\mathbf{i}, \mathbf{j}, \mathbf{k}$ respectively



Finding R

we can write

$$\begin{aligned}\Rightarrow R \cdot u &= \hat{i} & \Rightarrow R^T \cdot \hat{i} &= u \quad \left[\because R^{-1} = R^T \right] \\ & & \Rightarrow \begin{bmatrix} R_{1x} & R_{2x} & R_{3x} \\ R_{1y} & R_{2y} & R_{3y} \\ R_{1z} & R_{2z} & R_{3z} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} &= u \\ & & \Rightarrow \begin{bmatrix} R_{1x} \\ R_{1y} \\ R_{1z} \end{bmatrix} &= u = \overrightarrow{R_1}\end{aligned}$$

similarly we can find that

$$\overrightarrow{R_2} = \hat{v} \text{ and } \overrightarrow{R_3} = n$$

WC to VC transformation

- Rotation matrix **R** can be found by

- Calculate unit vectors u, v, n
- Form composite R directly

$$R = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$n = \frac{N}{|N|} = (n_x, n_y, n_z)$$

$$u = \frac{V \times N}{|V \times N|} = (u_x, u_y, u_z)$$

$$v = n \times u = (v_x, v_y, v_z)$$

- $M_{WC,VC} = R.T = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -x_0 \\ 0 & 1 & 0 & -y_0 \\ 0 & 0 & 1 & -z_0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} u_x & u_y & u_z & dx \\ v_x & v_y & v_z & dy \\ n_x & n_y & n_z & dz \\ 0 & 0 & 0 & 1 \end{bmatrix}$

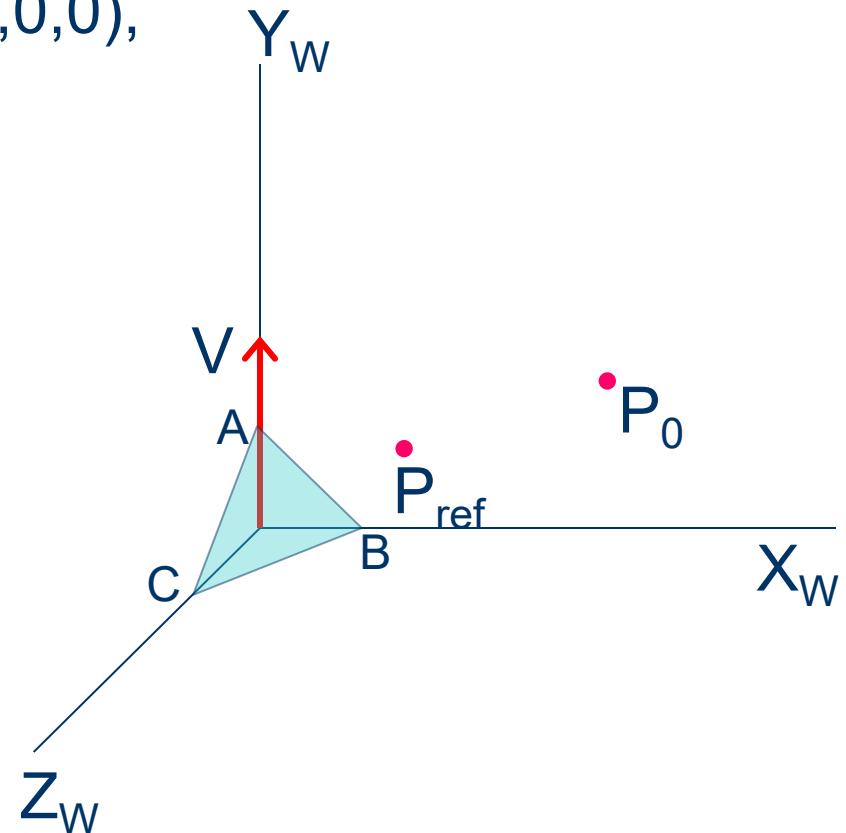
$$dx = -P_0 \cdot u$$

$$dy = -P_0 \cdot v$$

$$dz = -P_0 \cdot n$$

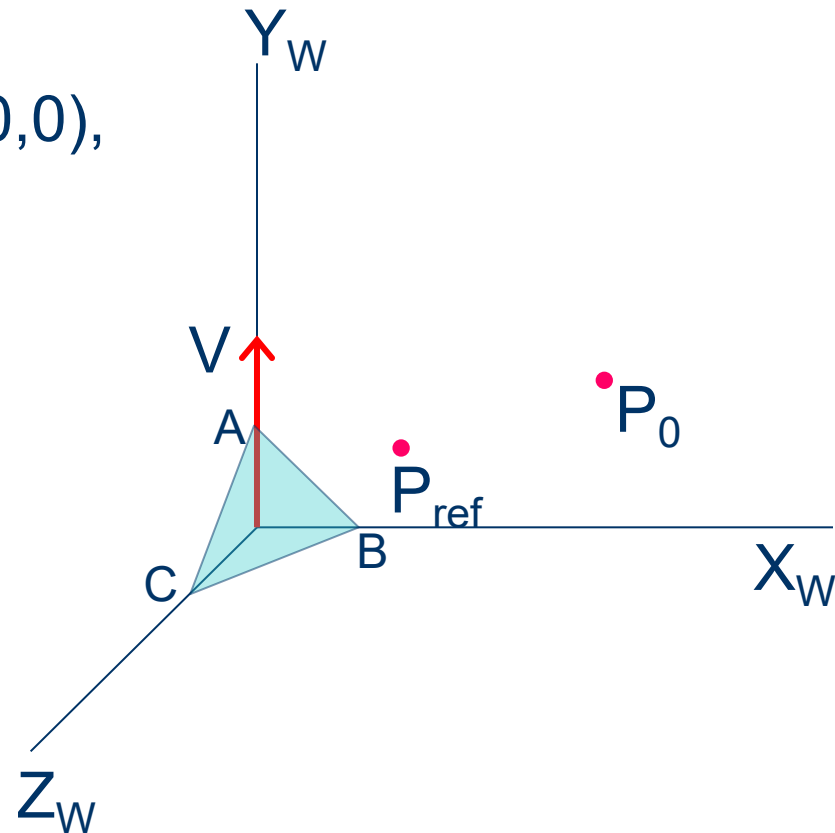
example

- Find the VCS values for the
 - Object: triangle $A(0,1,0)$, $B(1,0,0)$, $C(0,0,1)$
- when
 - Eye: $(3,2,3)$
 - Look at point: $(2,1,2)$
 - Up vector $(0,1,0)$



example

- Find the VCS values for the
 - Object: triangle $A(0,1,0)$, $B(1,0,0)$, $C(0,0,1)$
- when
 - Eye: $(3,2,3)$
 - Look at point: $(2,1,2)$
 - Up vector $(0,1,0)$



$$N = P_0 - P_{\text{ref}} = (1, 1, 1)$$

$$n = \left(\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}} \right)$$

$$V \times N = (1, 0, -1)$$

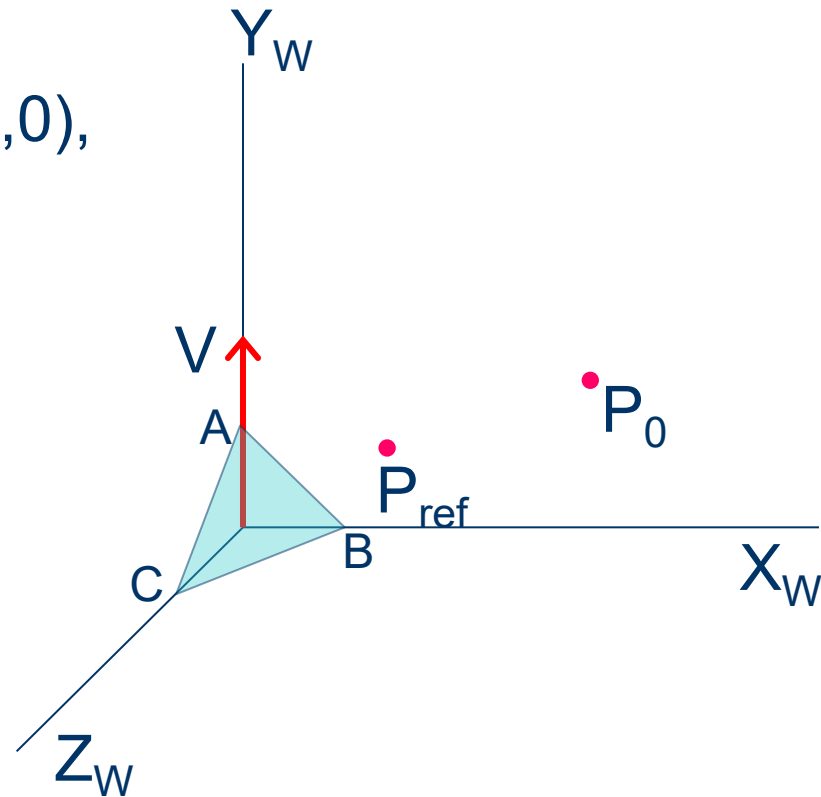
$$u = \left(\frac{1}{\sqrt{2}}, 0, -\frac{1}{\sqrt{2}} \right)$$

$$v = n \times u$$

$$= \left(-\frac{1}{\sqrt{6}}, \frac{2}{\sqrt{6}}, -\frac{1}{\sqrt{6}} \right)$$

example

- Find the VCS values for the
 - Object: triangle $A(0,1,0)$, $B(1,0,0)$, $C(0,0,1)$
- when
 - Eye: $(3,2,3)$
 - Look at point: $(2,1,2)$
 - Up vector $(0,1,0)$



$$u = \left(\frac{1}{\sqrt{2}}, 0, -\frac{1}{\sqrt{2}}\right)$$

$$v = \left(-\frac{1}{\sqrt{6}}, \frac{2}{\sqrt{6}}, -\frac{1}{\sqrt{6}}\right)$$

$$n = \left(\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}\right)$$

$$dx = -P_0 \cdot u = 0$$

$$dy = -P_0 \cdot v = \frac{2}{\sqrt{6}}$$

$$dz = -P_0 \cdot n = \frac{-8}{\sqrt{3}}$$

DIY

- Find the VCS values for the
 - Point: $A(1, 0.5, 1)$
- when
 - Eye: $(2, 2, 3)$ + last 3 digit of your roll
 - Look at point: $(2, 1, 1)$
 - Up vector $(0, 1, 0)$

Acknowledgements
cgvr.korea.ac.kr

10. Write down the 4×4 rotation matrix that takes the orthonormal 3D vectors $\mathbf{u} = (x_u, y_u, z_u)$, $\mathbf{v} = (x_v, y_v, z_v)$, and $\mathbf{w} = (x_w, y_w, z_w)$, to orthonormal 3D vectors $\mathbf{a} = (x_a, y_a, z_a)$, $\mathbf{b} = (x_b, y_b, z_b)$, and $\mathbf{c} = (x_c, y_c, z_c)$. So $M\mathbf{u} = \mathbf{a}$, $M\mathbf{v} = \mathbf{b}$, and $M\mathbf{w} = \mathbf{c}$.

The camera's current position is at (14,25-16), the camera is looking at the point (37,42,20), and the up vector is along $(3i + 5j - 4k)$. Find the transformation matrix that would align the direction that the camera is looking at along the negative Y axis, and align the direction that points to the right of the camera along the negative X axis.



Projection

Dr. Sk. Md. Masudul Ahsan

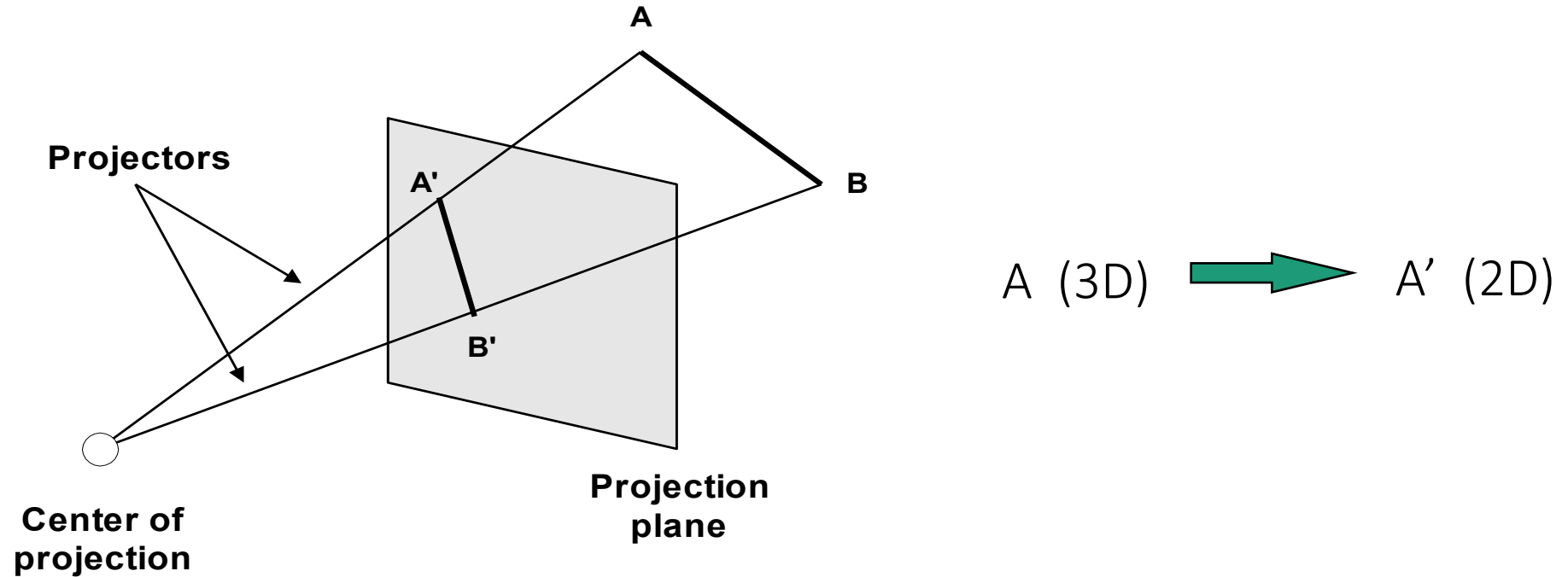
Professor

Dept. of CSE, KUET

Projection

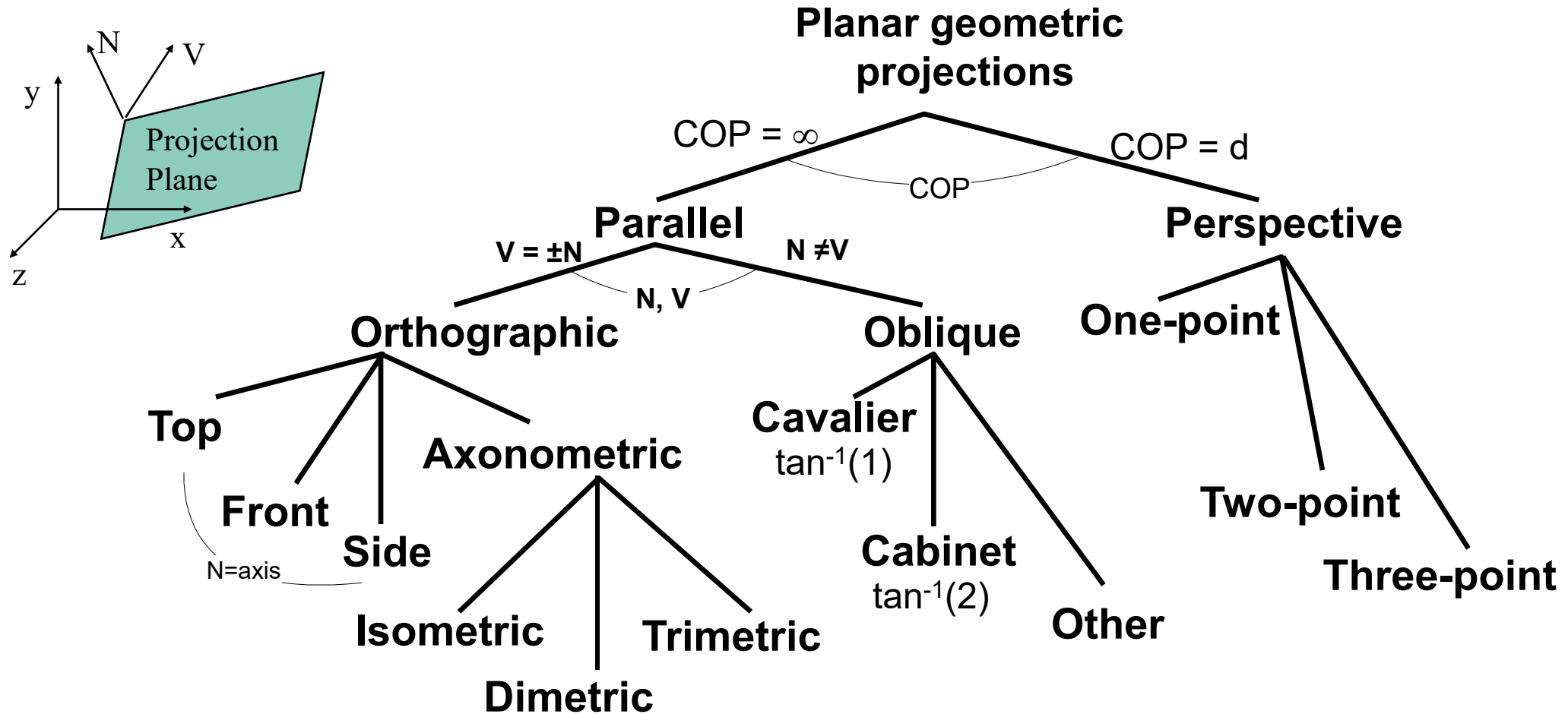
- ❑ In general, **projections** transform points in a coordinate system of dimension n into points in a coordinate system of dimension less than n .
- ❑ We shall limit ourselves to the projection from 3D to 2D.
- ❑ In computer graphics
 - Map viewing coordinates to 2D screen coordinates
- ❑ We will deal with **planar geometric projections** where:
 - The projection is onto a plane rather than a curved surface
 - The projectors are straight lines rather than curves

Projections – key terms



- ❑ The **projection** of a 3D object is defined by straight projection rays (called **projectors**) emanating from a **center of projection**, passing through each point of the object, and intersecting a **projection plane** to form the projection.

Planner Geometric Projections taxonomy

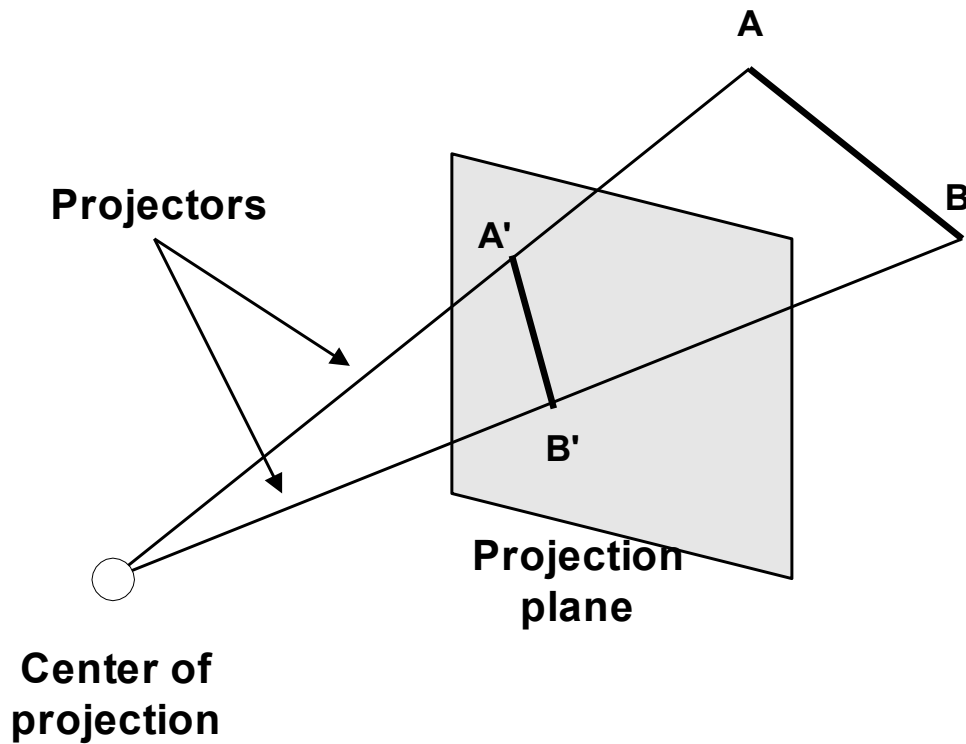


Subclasses of planar geometric projections

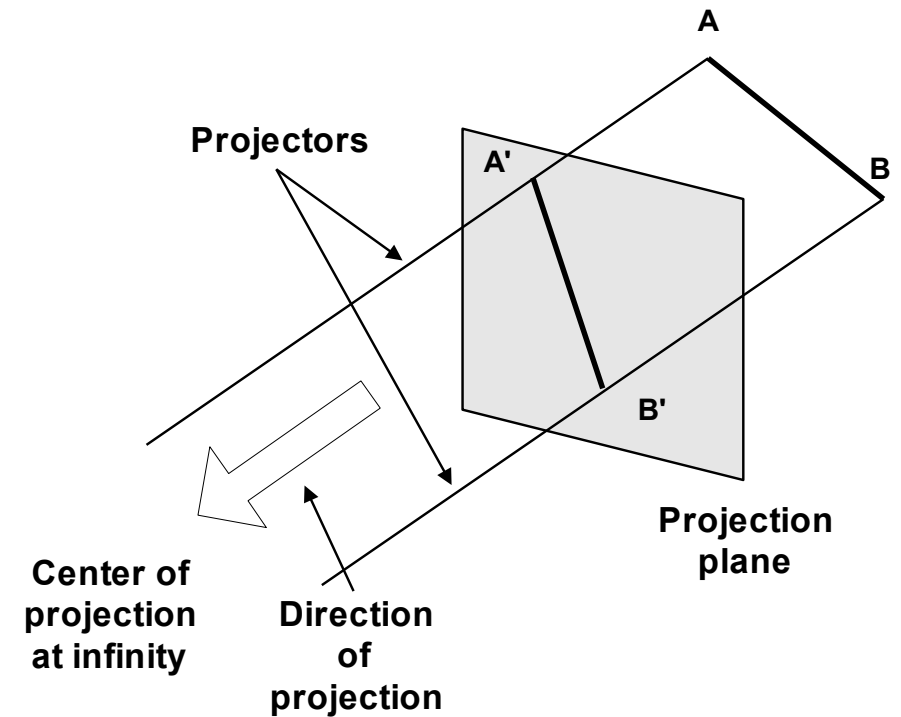
Planer Projection – Major Types

□ Key factor is the *center of projection, COP*.

- if distance to center of projection is finite : **Perspective**
- if infinite : **Parallel** -> so needs direction of projection vector, DOP



Perspective projection

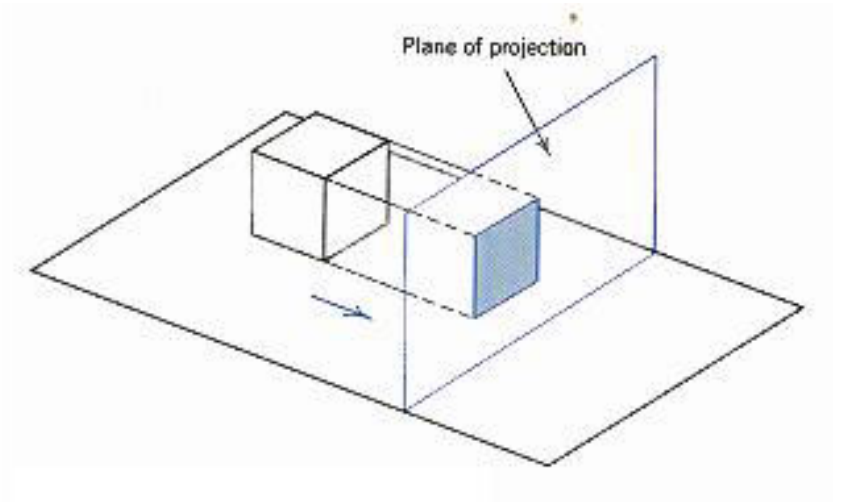
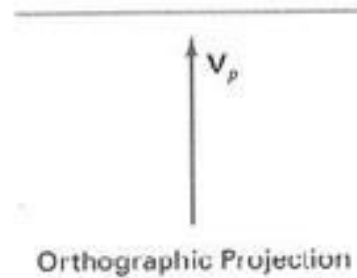


Parallel projection

Parallel projection - Types

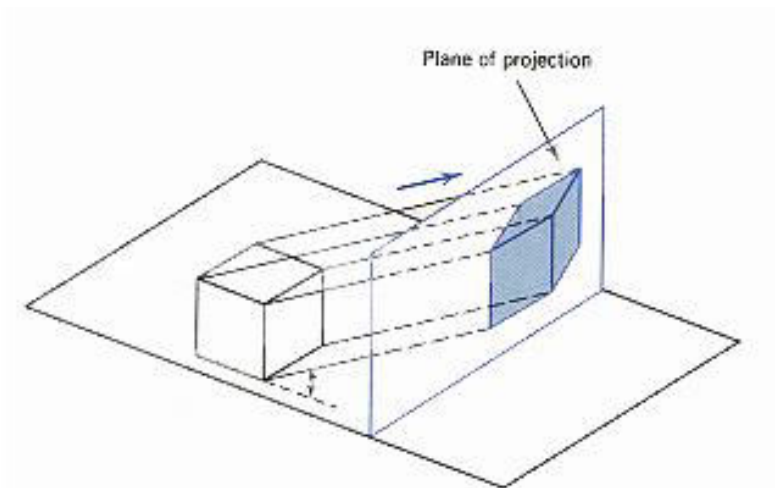
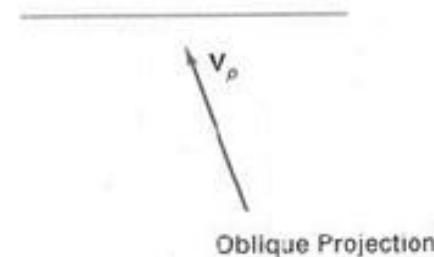
❑ Orthographic projection

- the projection is **perpendicular** to the view plane



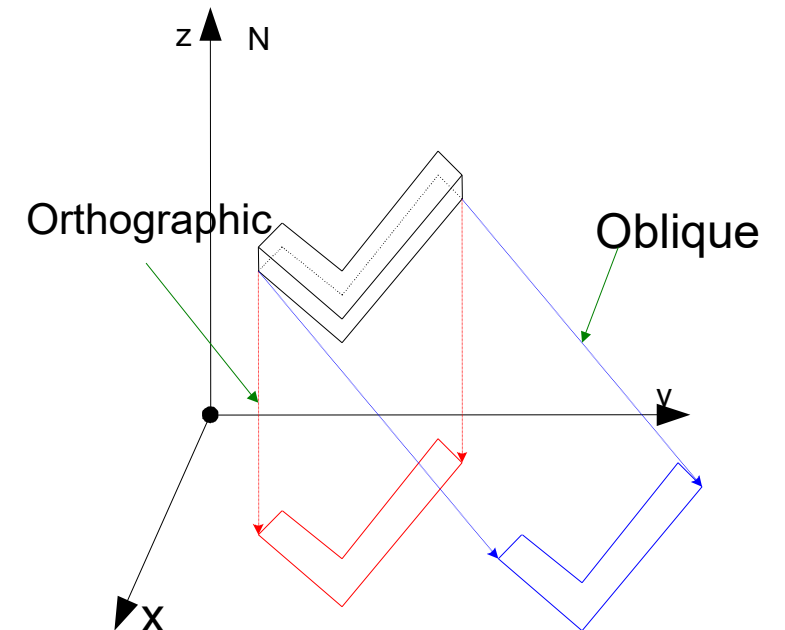
❑ Oblique projection

- The projectors are **inclined** with respect to the view plane



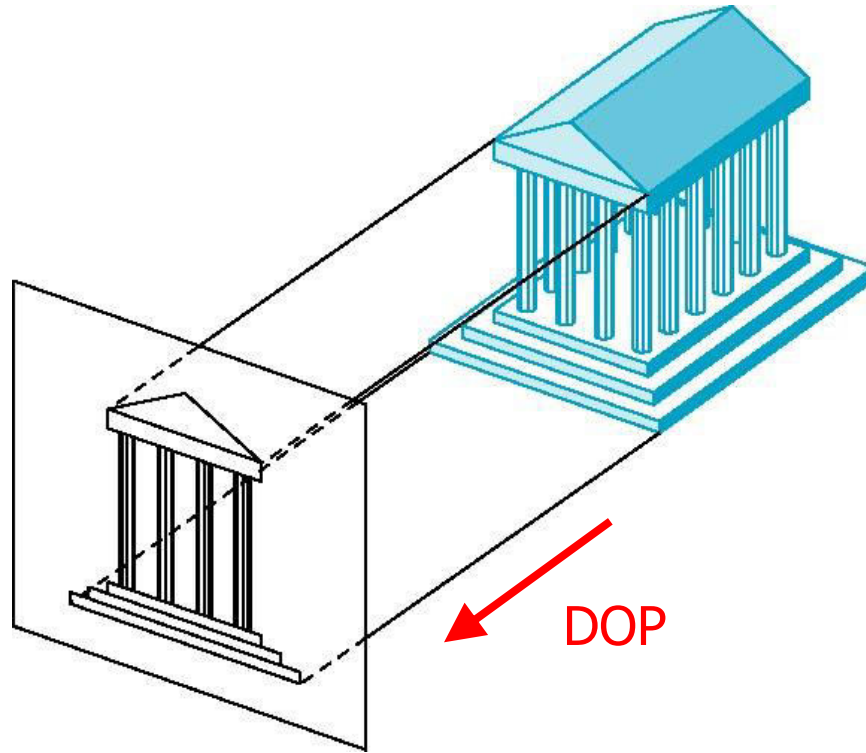
Parallel projection - Types

- ❑ 2 principle types:
 - on the basis of DOP, V and projection plane normal N)
- ❑ Orthographic :
 - V and N are the same or the reverse of each other, i.e. V is perpendicular to view plane
- ❑ Oblique :
 - direction of projection $\neq$ the projection plane normal.



Orthographic Projection

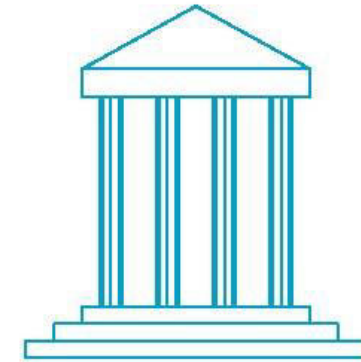
- ❑ DOP or all Projectors are orthogonal (perpendicular) to projection surface



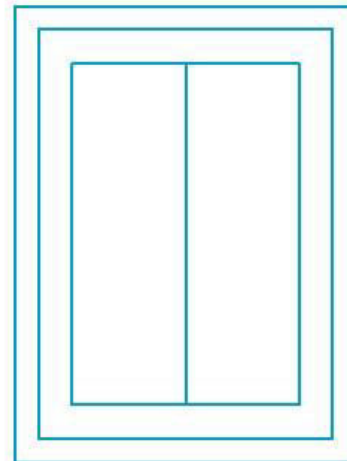
Multiview Orthographic Projection

- ❑ Projection plane parallel to principal plane
- ❑ Usually form front, top, side view

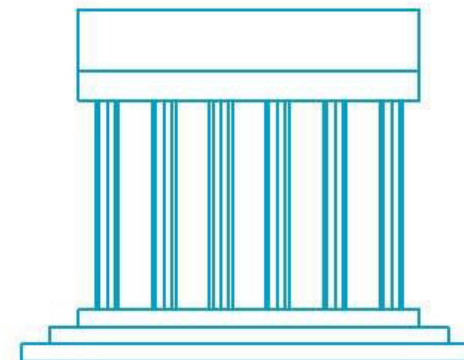
isometric (not multiview
orthographic view)



front



top



side

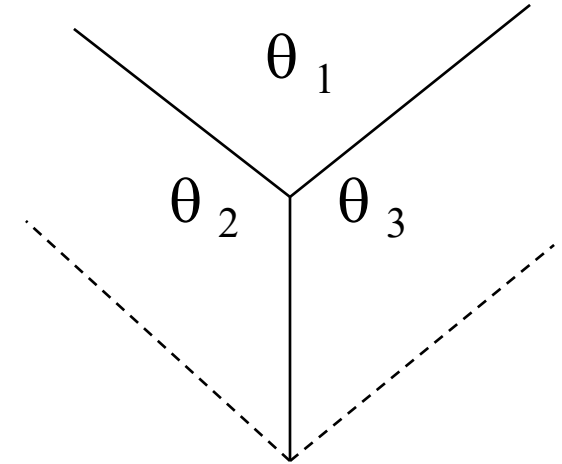
In CAD and architecture,
we often display three
multiviews plus isometric

Multiview Orthographic projection

- ❑ Orthographic (or orthogonal) projections:
 - front elevation, top-elevation and side-elevation.
 - all have projection plane perpendicular to a principle axes.
- ❑ Useful because angle and distance measurements can be made...
- ❑ However, As only one face of an object is shown, it can be hard to create a mental image of the object, even when several view are available

Axonometric projection

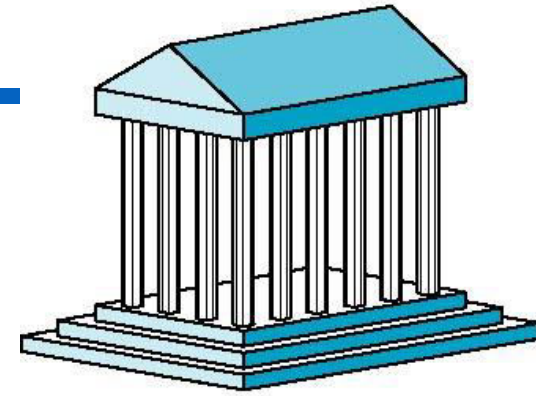
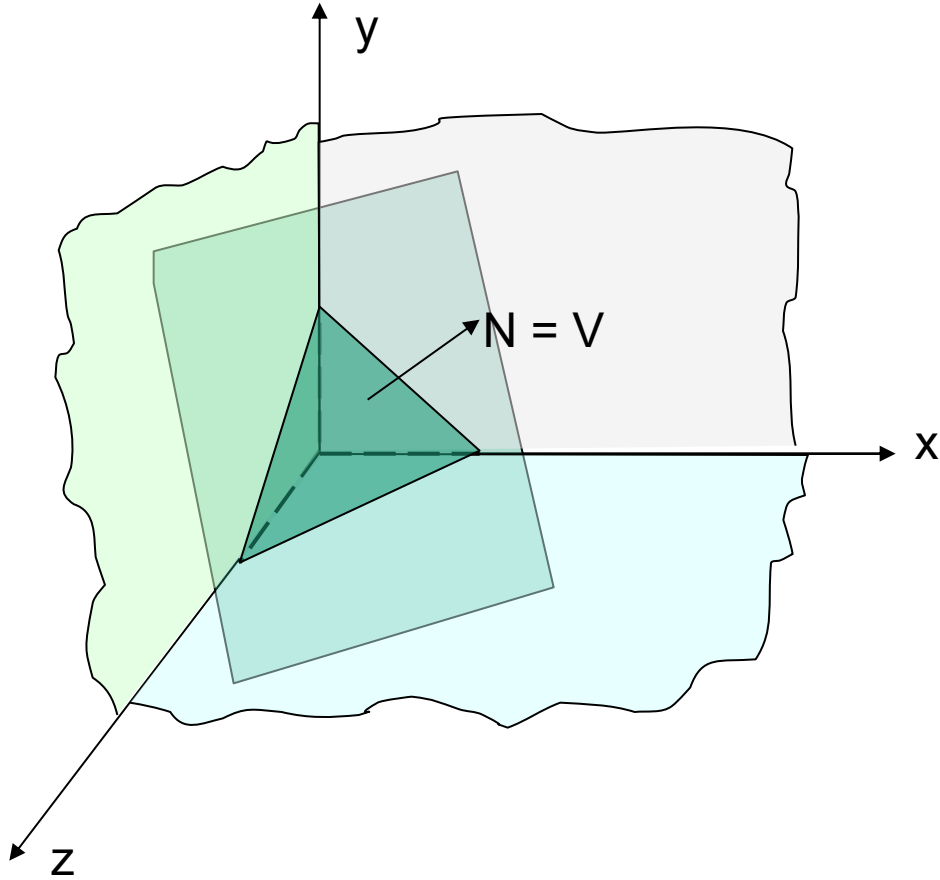
- ❑ A type of parallel projection
 - Uses projection planes that are not normal to any principal axis.
- ❑ On the basis of *projection plane normal* $\mathbf{N} = (dx, dy, dz)$ subclasses are:
 - *Isometric* : $|dx| = |dy| = |dz|$ i.e. $\mathbf{N}$ makes equal angles with all principal axes.
 - *Dimetric* : $|dx| = |dy|$
 - *Trimetric* : $|dx| \neq |dy| \neq |dz|$



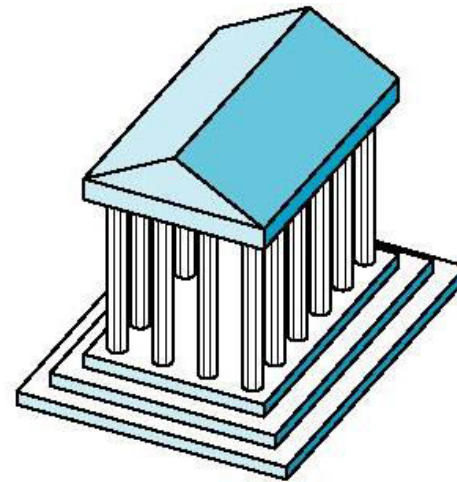
classify by how many angles of a corner of a projected cube are the same

none: trimetric
two: dimetric
three: isometric

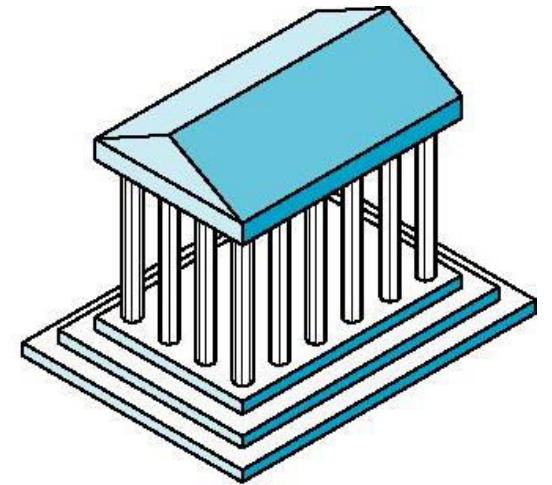
Axonometric projection



Dimetric



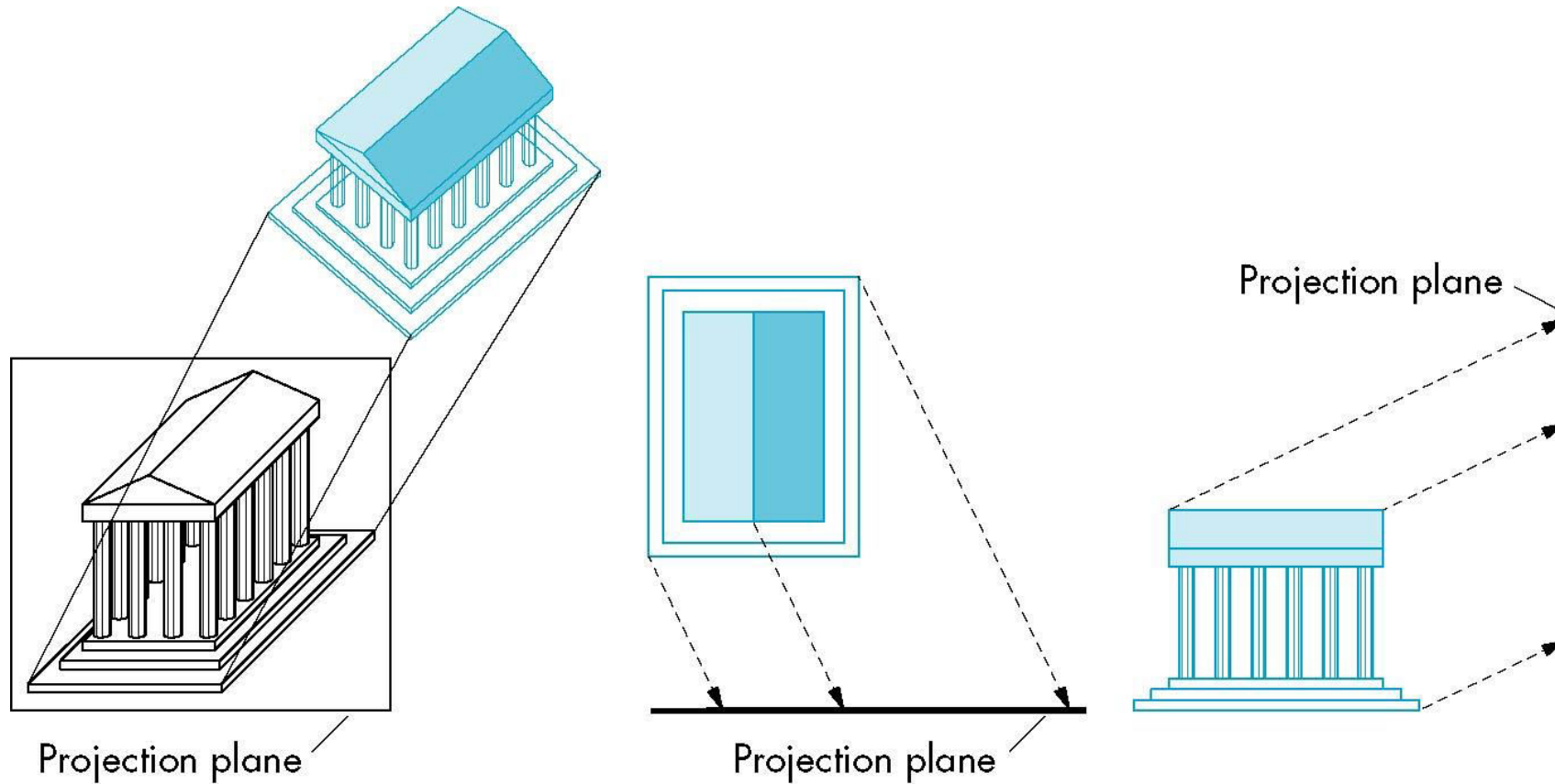
Trimetric



Isometric

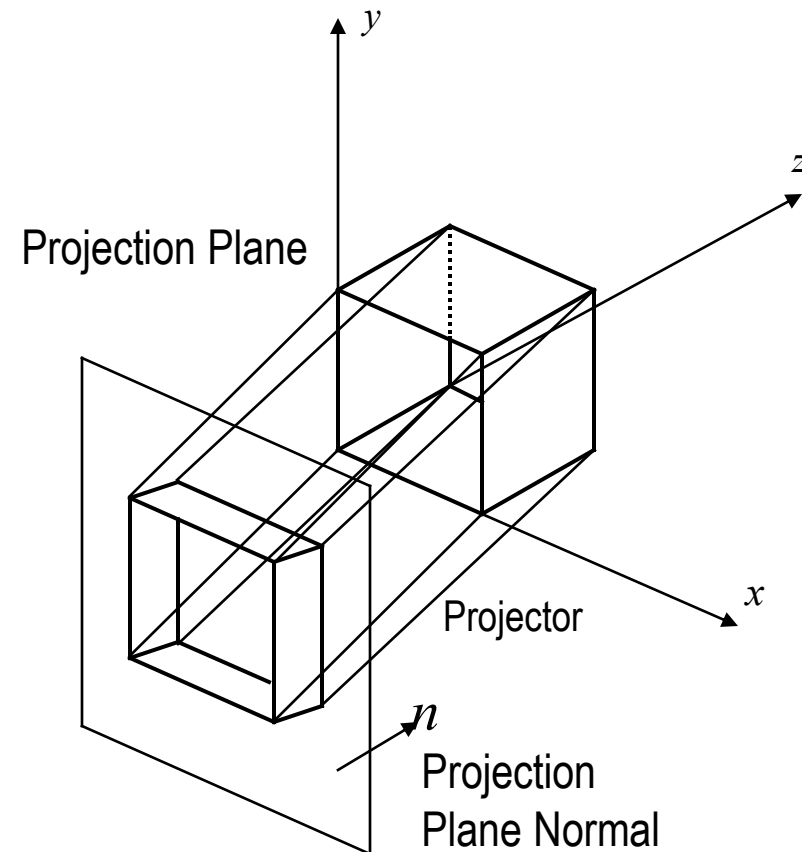
Oblique Projection

- ❑ Arbitrary relationship between projectors and projection plane



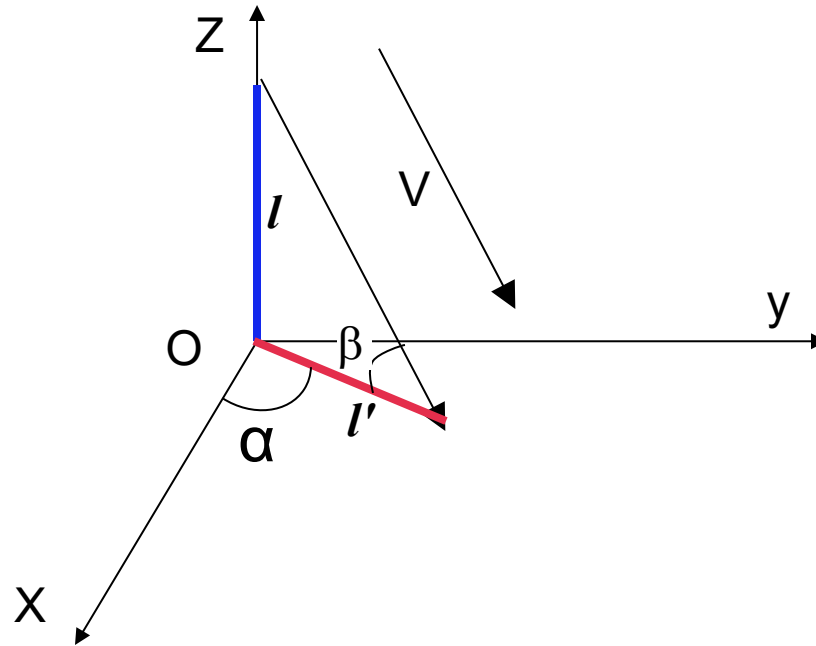
Oblique parallel projection

- ❑ Objects can be visualized better than orthographic projections
- ❑ Can measure distances, but not angles
 - Can only measure angles for faces of objects parallel to the plane
- ❑ Two types
 - *Cavalier* and *Cabinet*



Oblique Projections

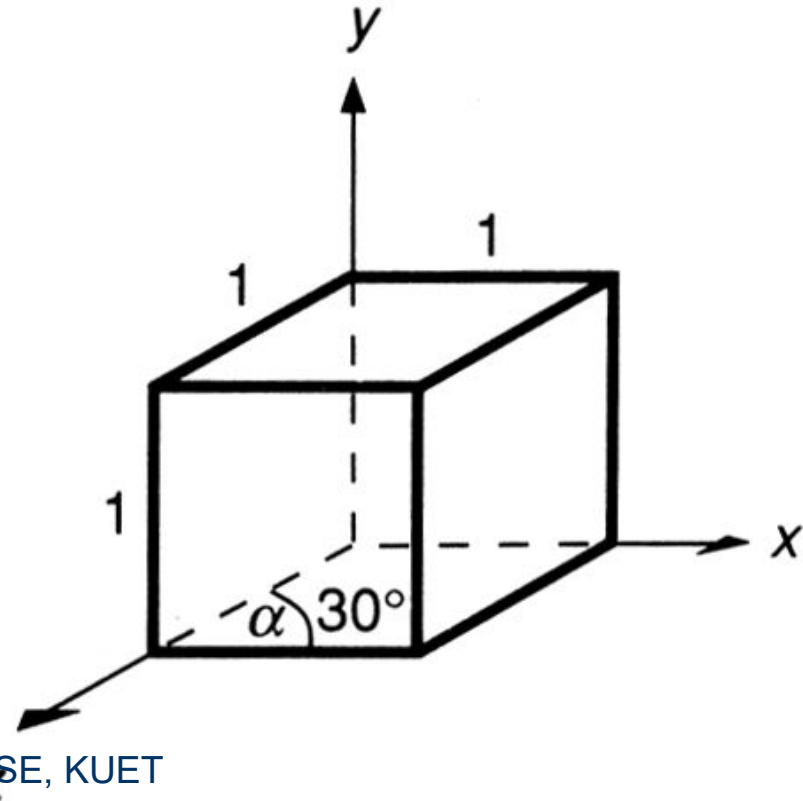
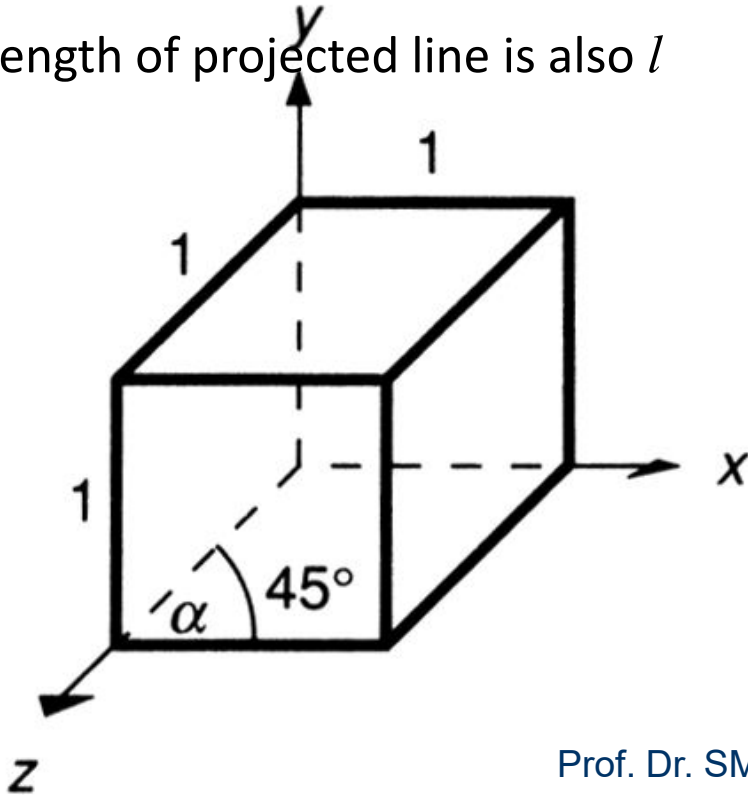
- ❑ Azimuth, α : is the angle the projection makes with x-axis
- ❑ Elevation, β : angle between view plane and direction of projection
- ❑ l : original length of a line perpendicular to view plane
- ❑ l' : projected length of a line perpendicular to view plane



Oblique parallel projection

▣ Cavalier:

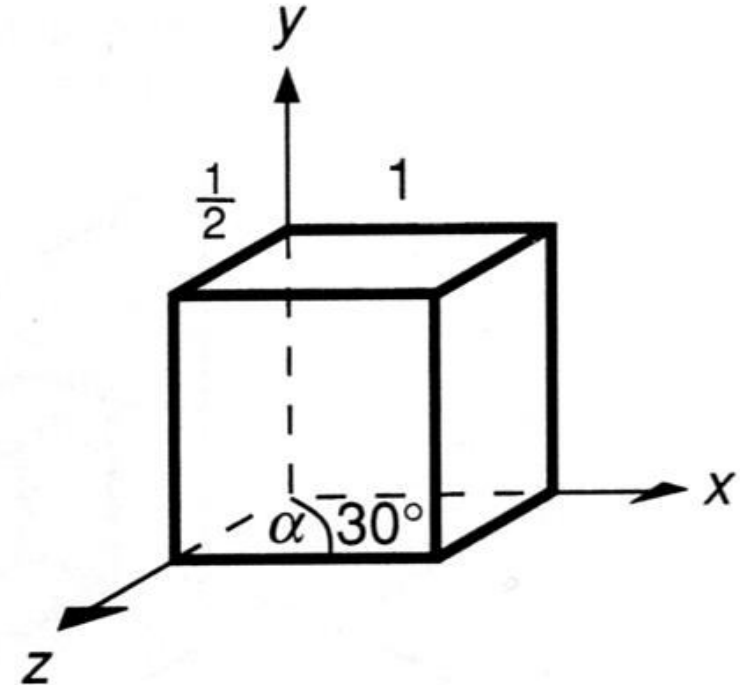
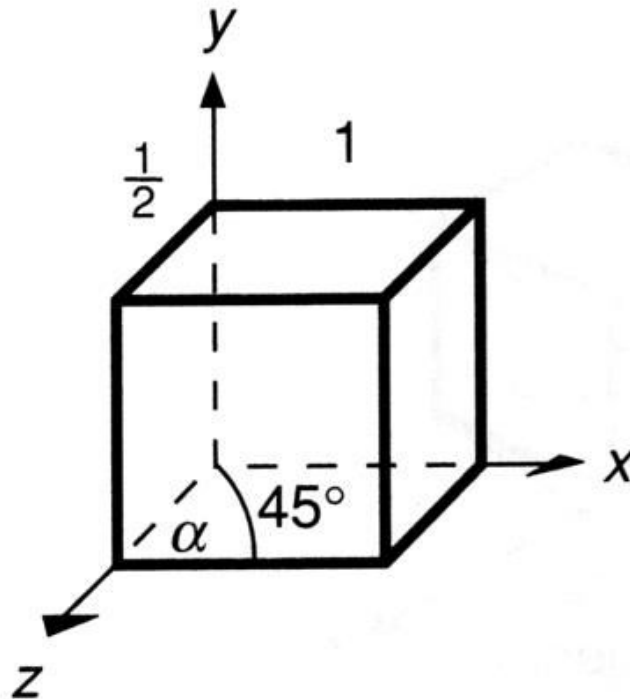
- $l' = l; \beta = 45^\circ$
- The DOP makes a 45 degree angle with the projection plane.
- There is no foreshortening
 - Length of any line perpendicular to view plane is l
 - The Length of projected line is also l



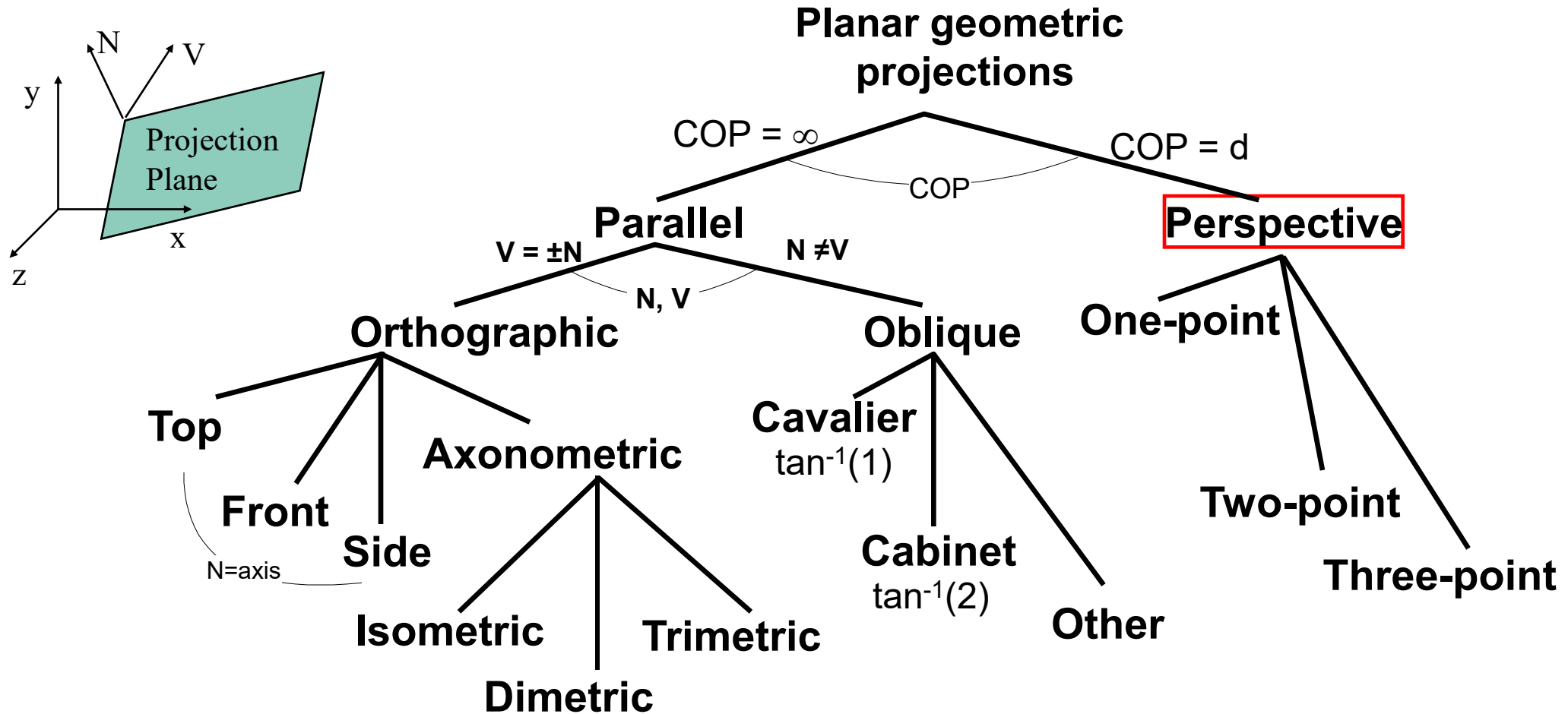
Oblique parallel projection

□ Cabinet:

- $l' = l/2$; $\beta = 63.4^\circ$
- The DOP makes a 63.4 degree angle with the projection plane.
- This results in foreshortening of the z axis, and provides a more “realistic” view

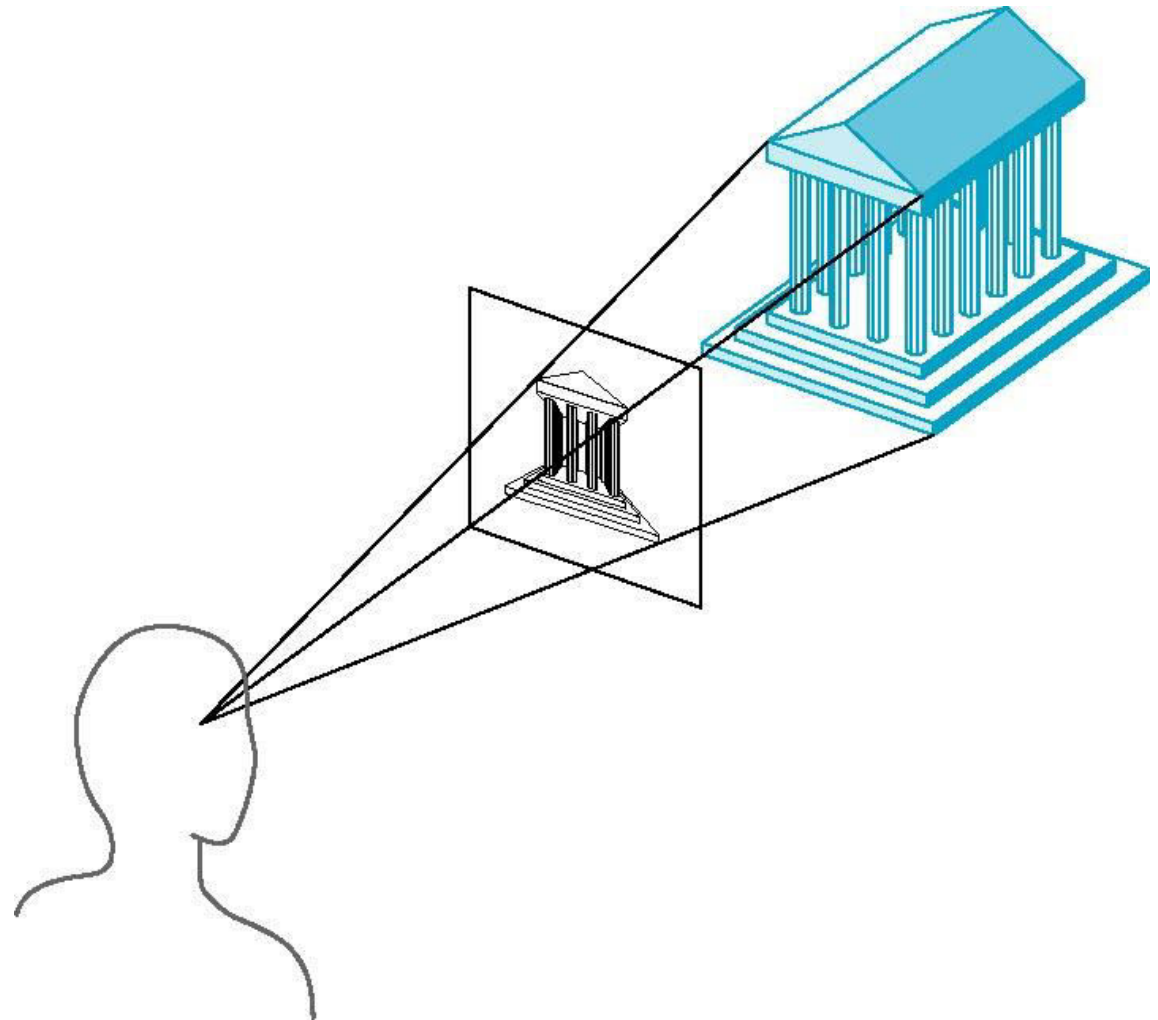


Planner Geometric Projections taxonomy



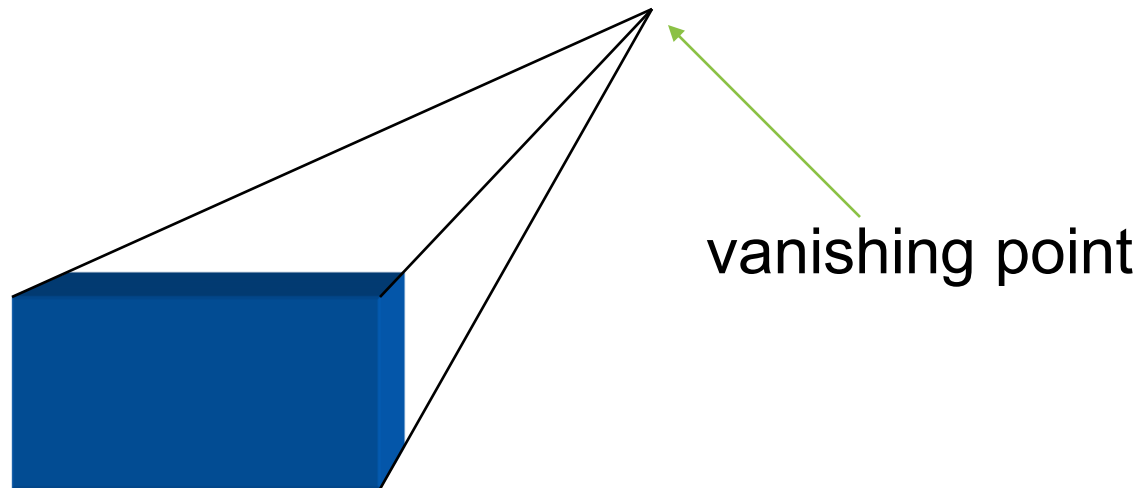
Perspective Projection

- ❑ Map points onto “view plane” along “projectors” emanating from “center of projection”(cop)
- ❑ Projectors converge at COP



Vanishing Points

- ❑ Parallel lines (not parallel to the projection plan) on the object converge at a single point in the projection (the *vanishing point*)
- ❑ Drawing simple perspectives by hand uses these vanishing point(s)

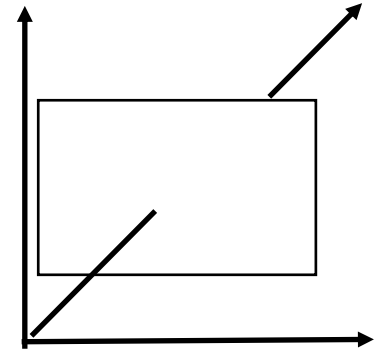
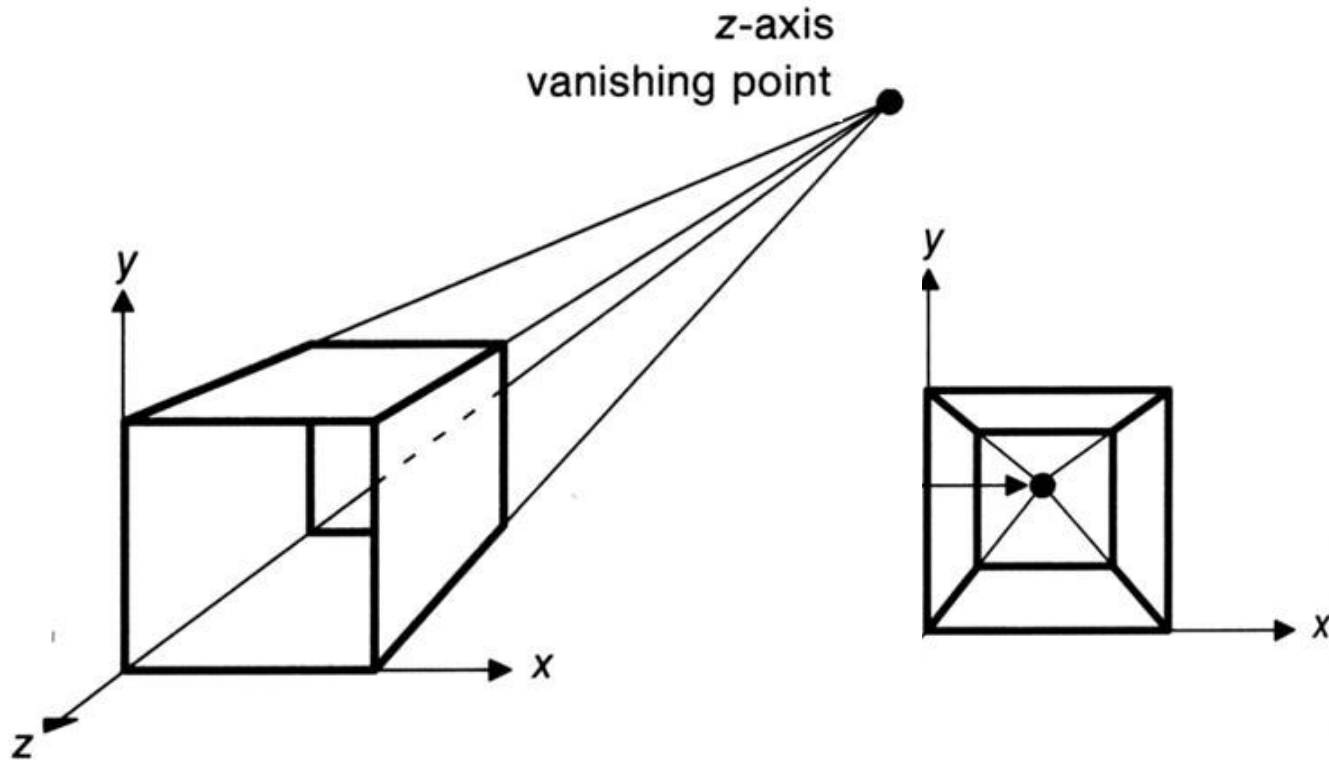


Perspective Projection - types

- ❑ Based on vanishing points
- ❑ If a set of lines are parallel to one of the three axes, the vanishing point is called an **axis vanishing point** (Principal Vanishing Point).
- ❑ There are at most 3 such points, corresponding to the number of axes cut by the projection plane
- ❑ One-point / two-point / three-point perspective:
 - One / two / three principle axis cut by projection plane

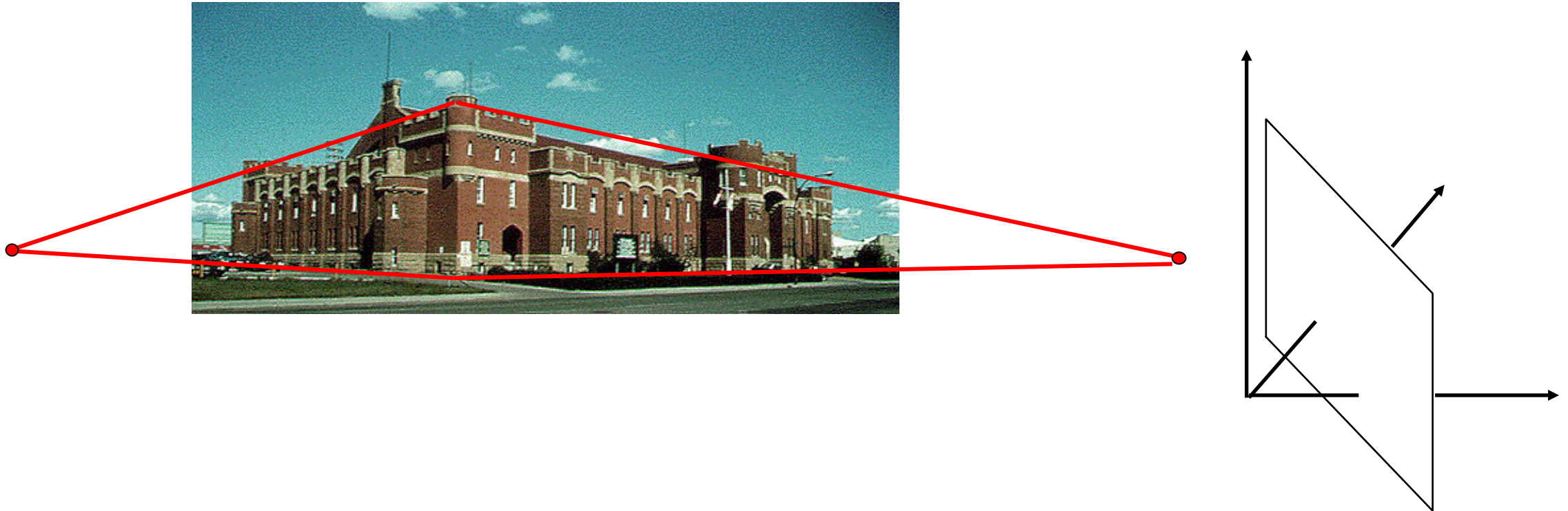
Perspective Projection

- One point perspective projection of a cube
 - X and Y parallel lines do not converge



Perspective Projection

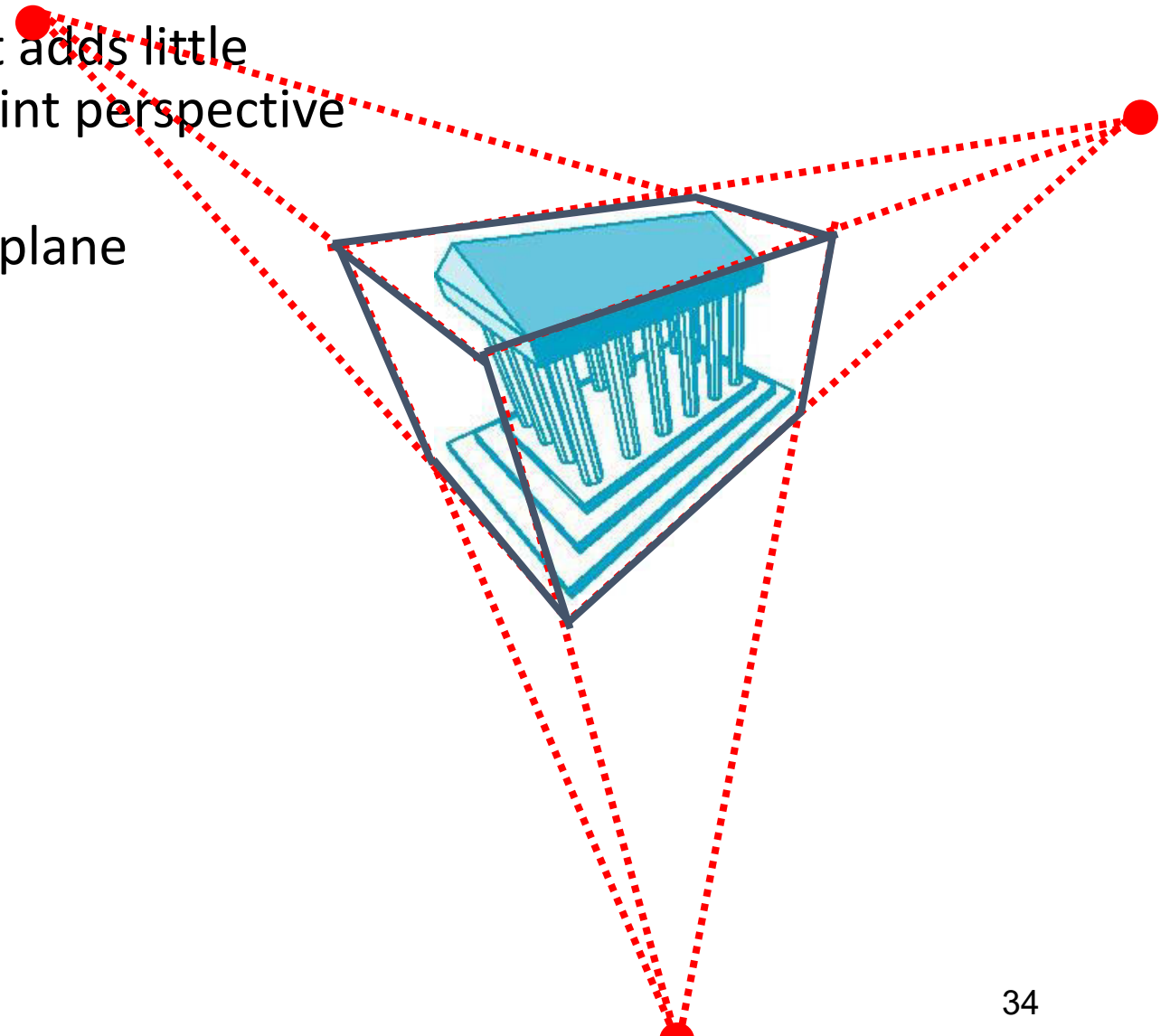
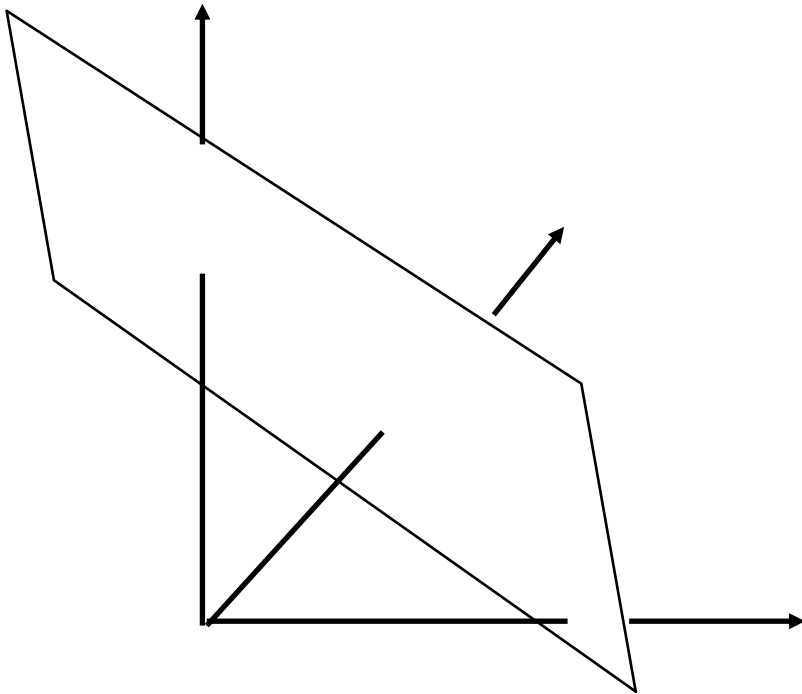
- ❑ Two-point perspective:
 - often used in architectural, engineering and industrial design drawings.



Perspective Projection

❑ Three-Point Perspective

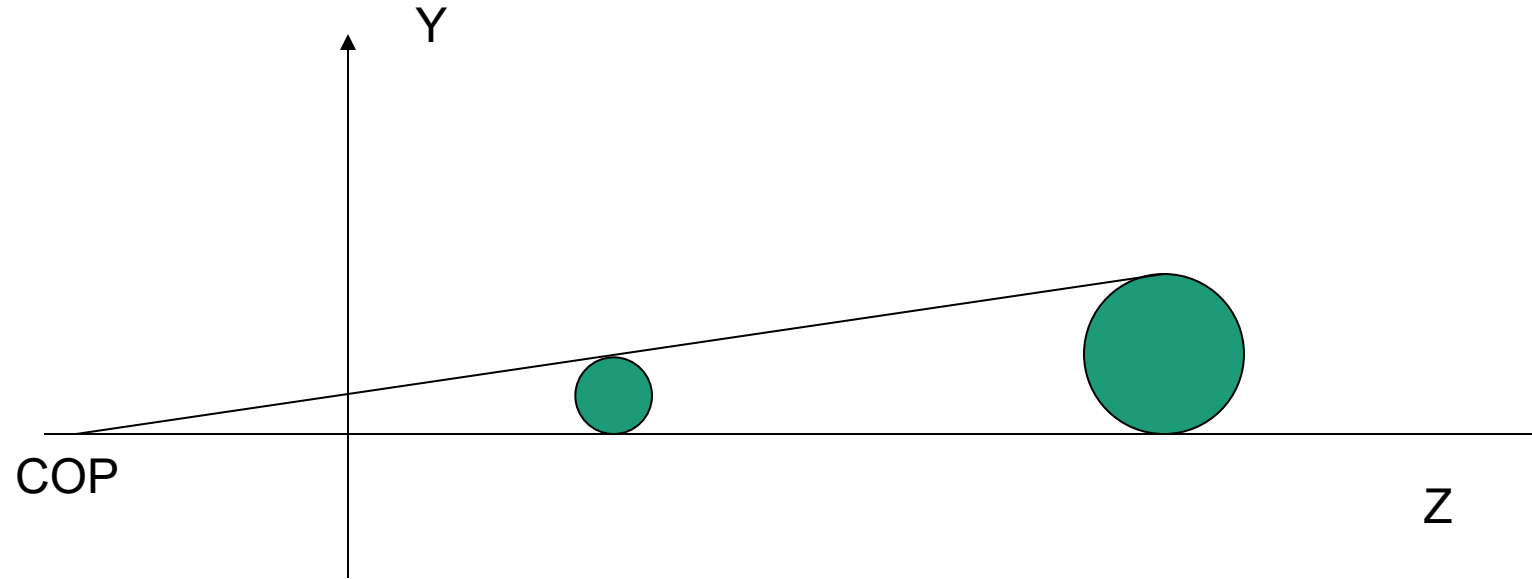
- Three-point is used less frequently as it adds little extra realism to that offered by two-point perspective projection.
- No principal face parallel to projection plane



Perspective Projection-Anomalies

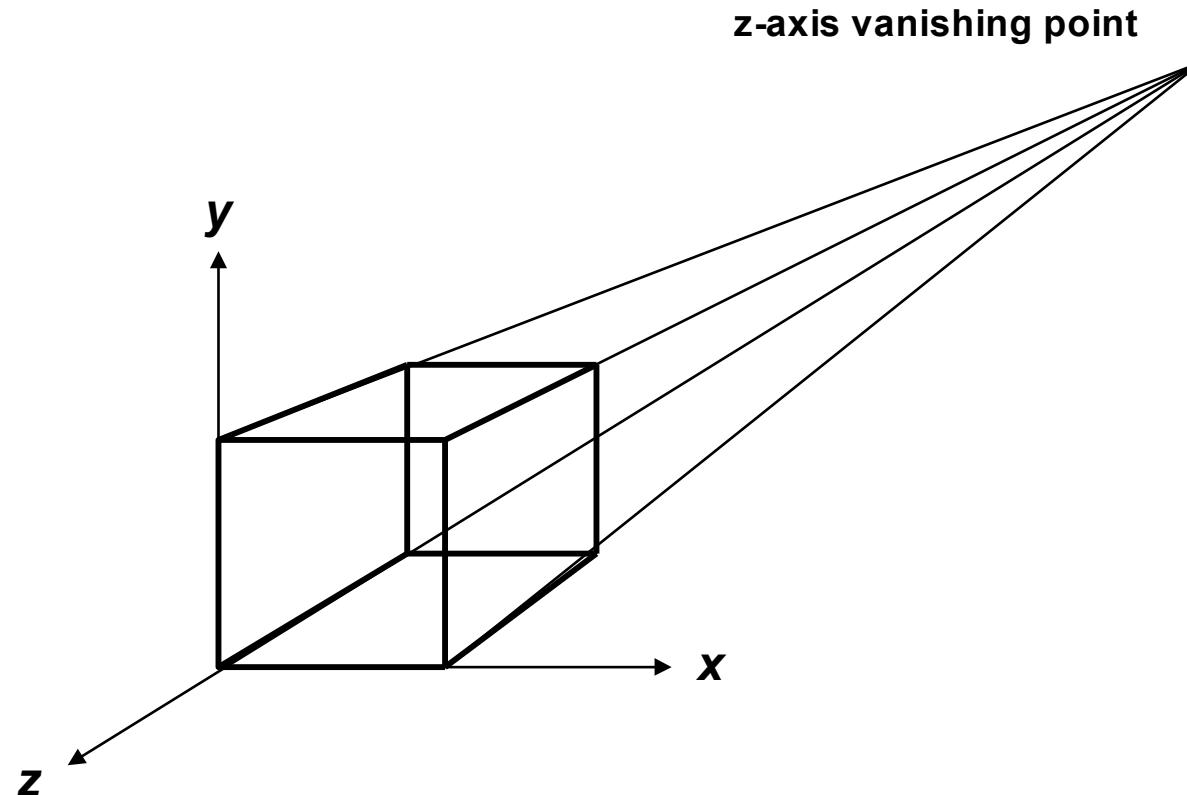
- ❑ Enhances realism in terms of depth cue but distorts sizes and shapes

- ❑ **Perspective foreshortening** The farther an object is from COP the smaller it appears



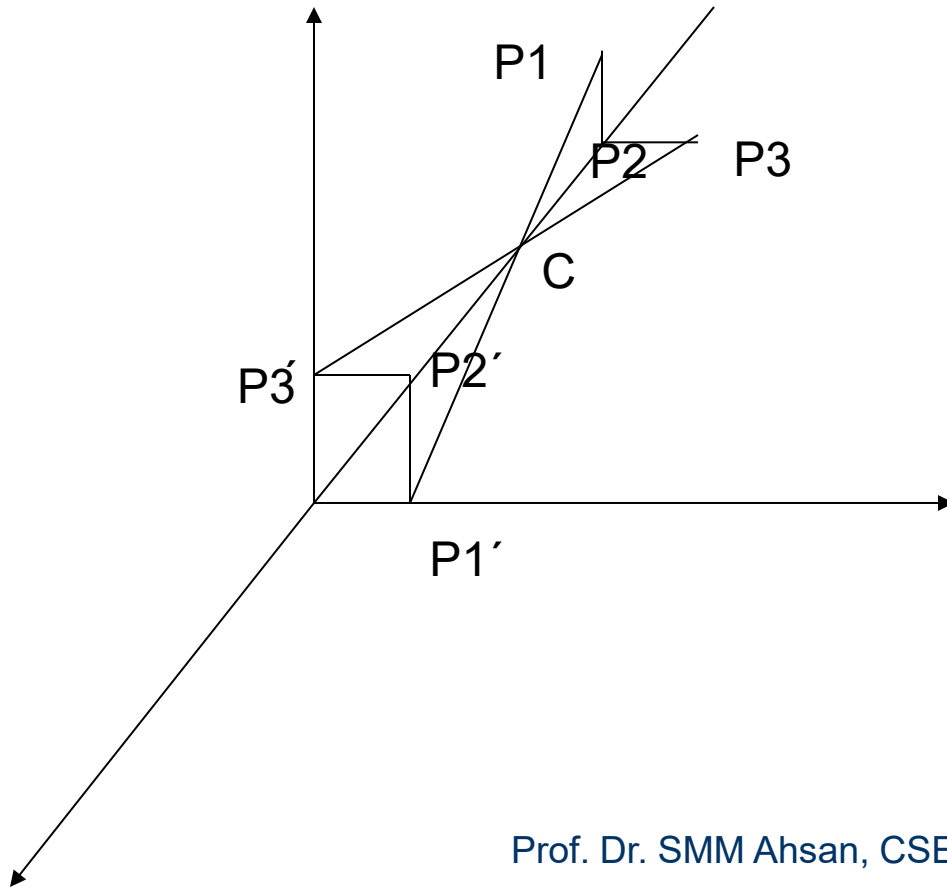
Perspective Projection-Anomalies

- ❑ **Vanishing Points:** Any set of parallel lines not parallel to view plane (or not perpendicular to view plane normal) appear to meet at some point.



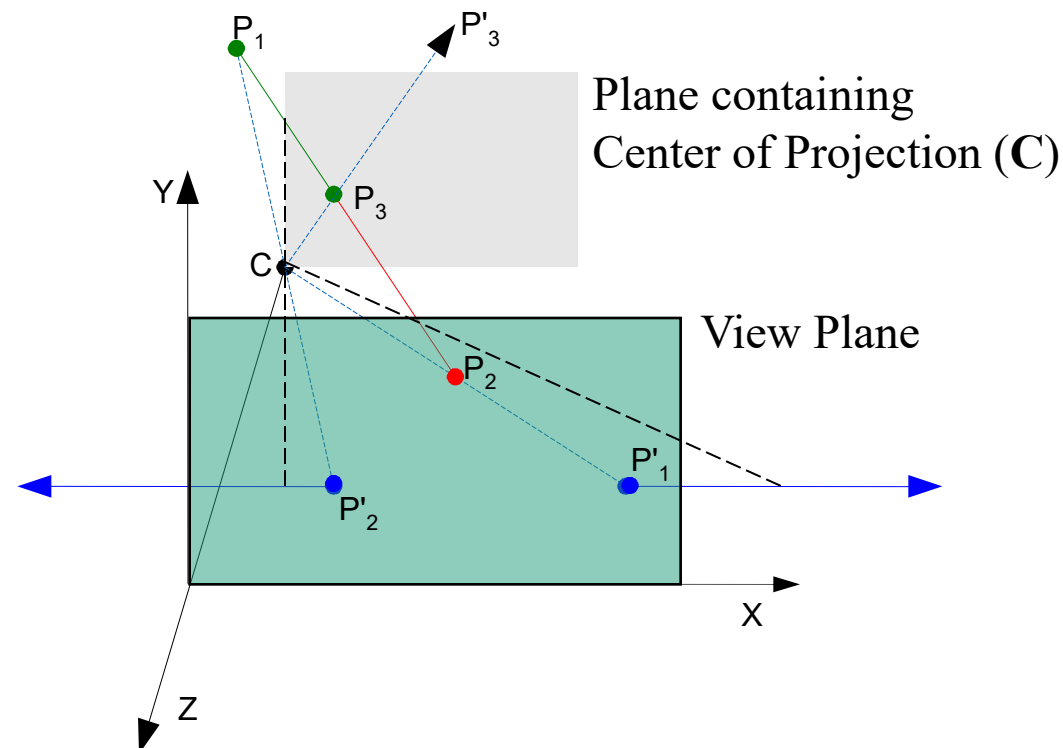
Perspective Projection-Anomalies

- ❑ **View Confusion:** Objects behind the center of projection are projected upside down and backward onto the view-plane.



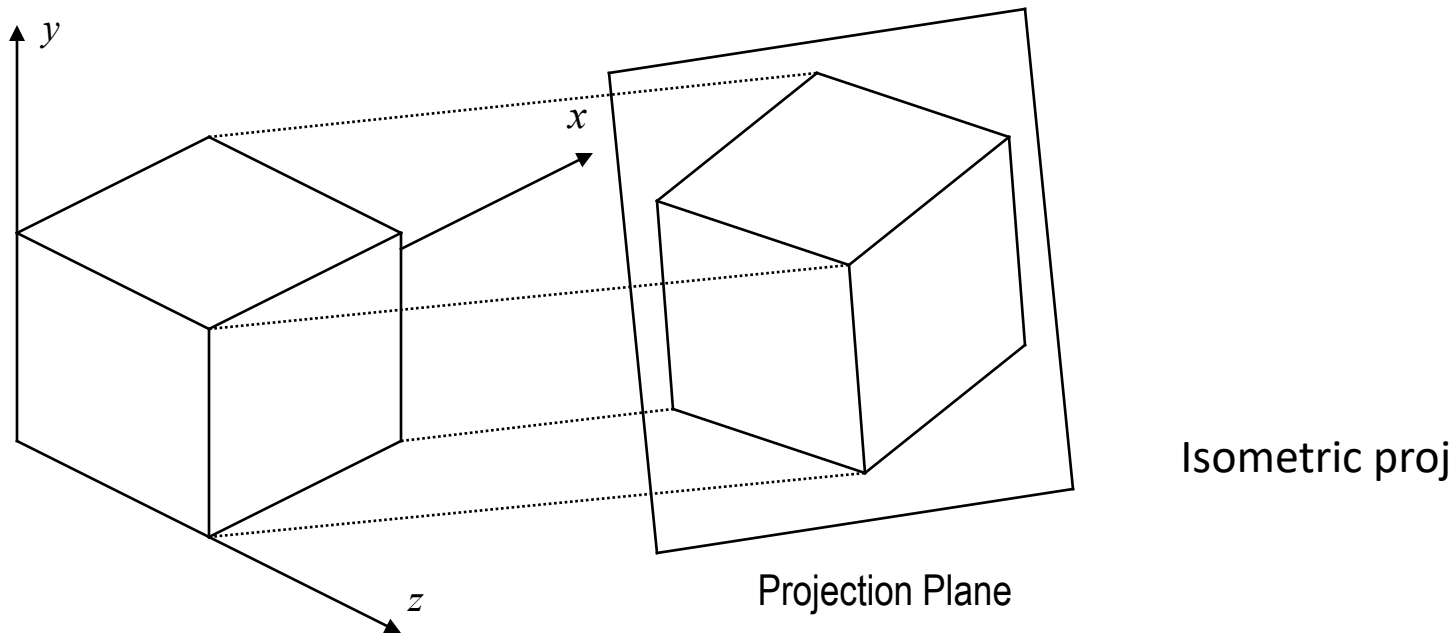
Perspective Projection-Anomalies

- ❑ **Topological distortion:** A line segment joining a point which lies in front of the viewer to a point in back of the viewer is projected to a broken line of infinite extent.



Axonometric vs Perspective

- ❑ Axonometric projection shows several faces of an object at once like perspective projection.
- ❑ But the foreshortening is uniform rather than being related to the distance from the COP.



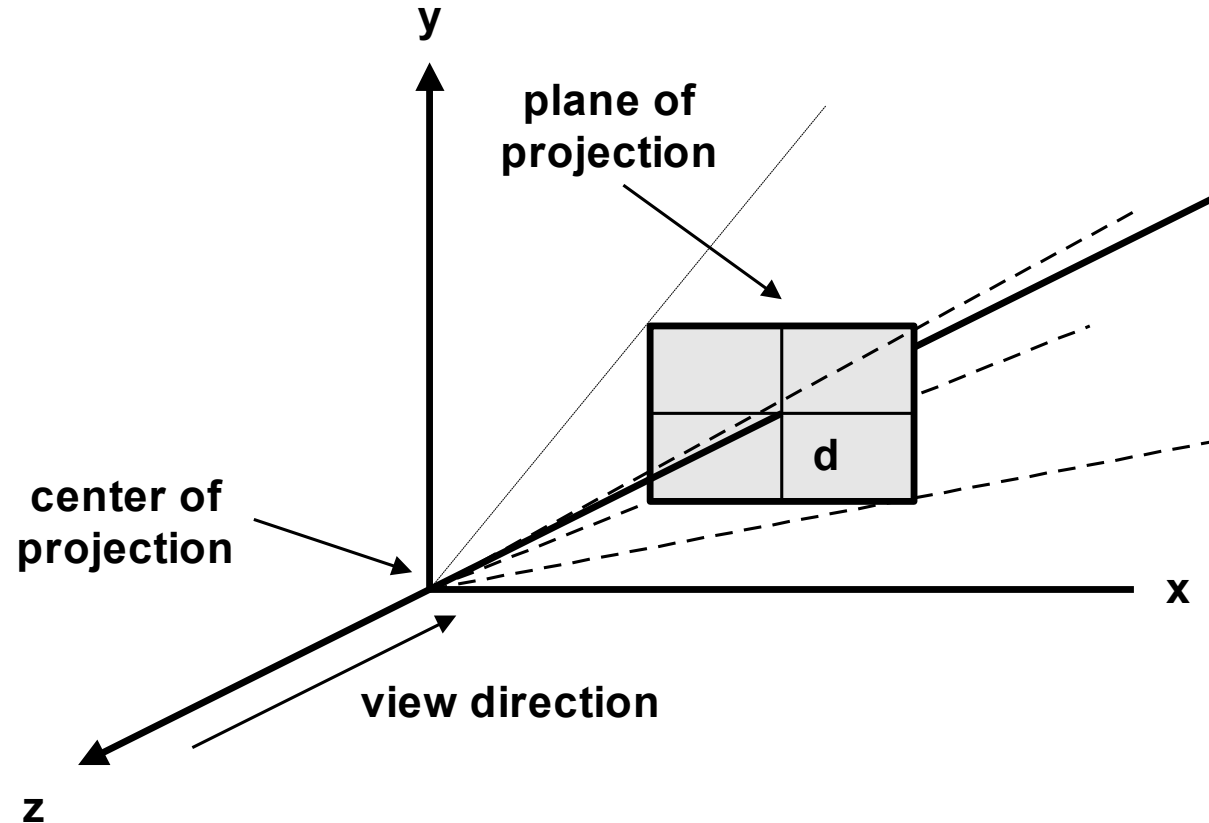
Projection Mathematics



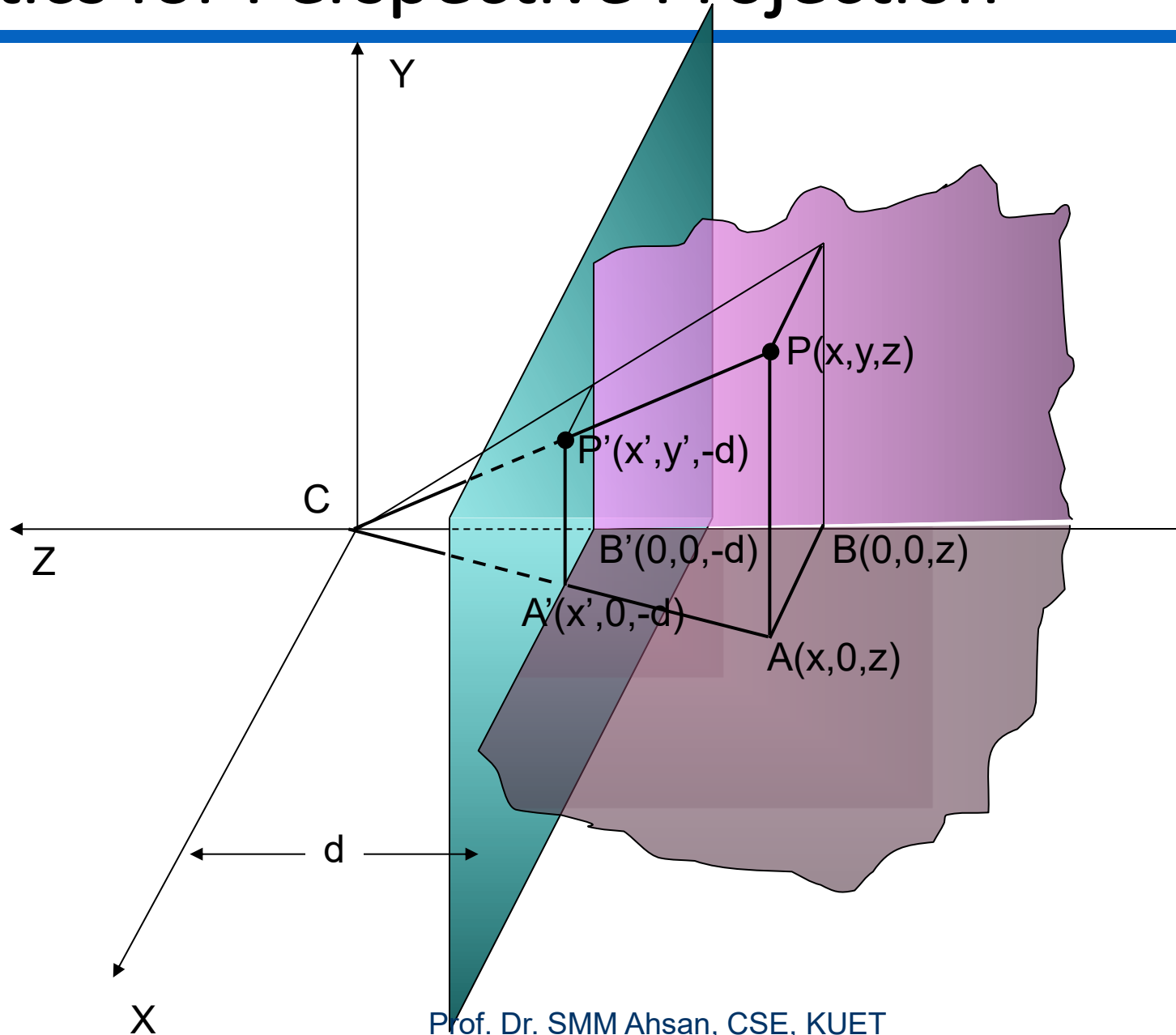
Projective Transformations

- ❑ Projection plane, COP, etc. all are defined in VCS

Settings for
perspective
projection



Mathematics for Perspective Projection



Mathematics for Perspective Projection

From triangle ABC and A'B'C

$$\frac{AB}{BC} = \frac{A'B'}{B'C}$$

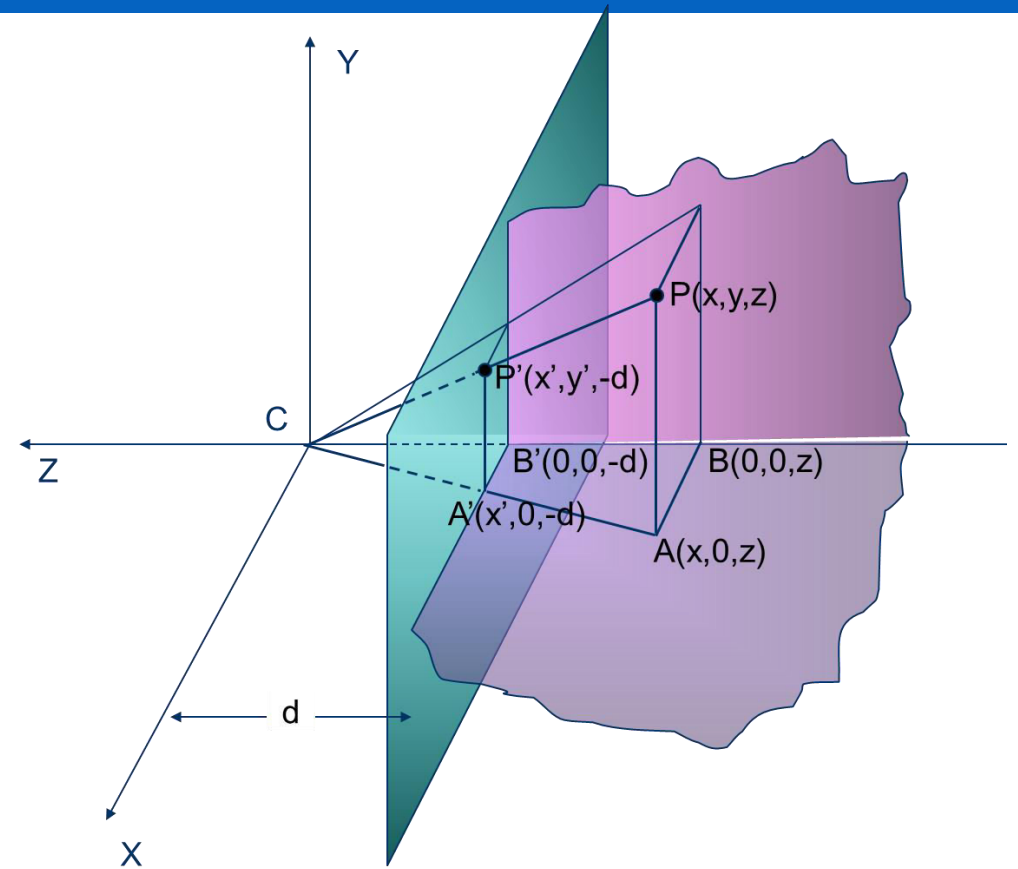
$$\frac{x}{z} = \frac{x'}{-d} \Rightarrow x' = \frac{x}{-(z/d)}$$

similarly, $y' = \frac{y}{-(z/d)}$ and,

$$z' = -d$$

$$(x', y', z', 1) \Rightarrow \left(\frac{x}{-(z/d)}, \frac{y}{-(z/d)}, -d, 1 \right) \equiv (x, y, z, -(z/d))$$

Some text book doesn't use the minus sign here, both are OK. So, be careful which convention you are using



Matrices for Projective Trans.

$$\begin{array}{c}
 \begin{pmatrix} ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{x}{-(z/d)} \\ y \\ \frac{y}{-(z/d)} \\ -d \\ 1 \end{pmatrix} \xrightarrow{\quad} \begin{pmatrix} ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \\ ? & ? & ? & ? \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ -\frac{z}{d} \end{pmatrix} \\
 \downarrow \\
 \begin{pmatrix} \frac{x}{-(z/d)} \\ y \\ \frac{y}{-(z/d)} \\ -d \\ 1 \end{pmatrix} \xleftarrow{\text{homogenization}} \begin{pmatrix} x \\ y \\ z \\ -\frac{z}{d} \end{pmatrix} \xleftarrow{\quad} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -\frac{1}{d} & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ -\frac{z}{d} \end{pmatrix}
 \end{array}$$

Matrices for Projective Trans.

Projection plane cuts the x axis

$$\mathbf{M}_{PER} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -\frac{1}{d_x} & 0 & 0 & 0 \end{bmatrix}$$

Projection plane cuts the y axis

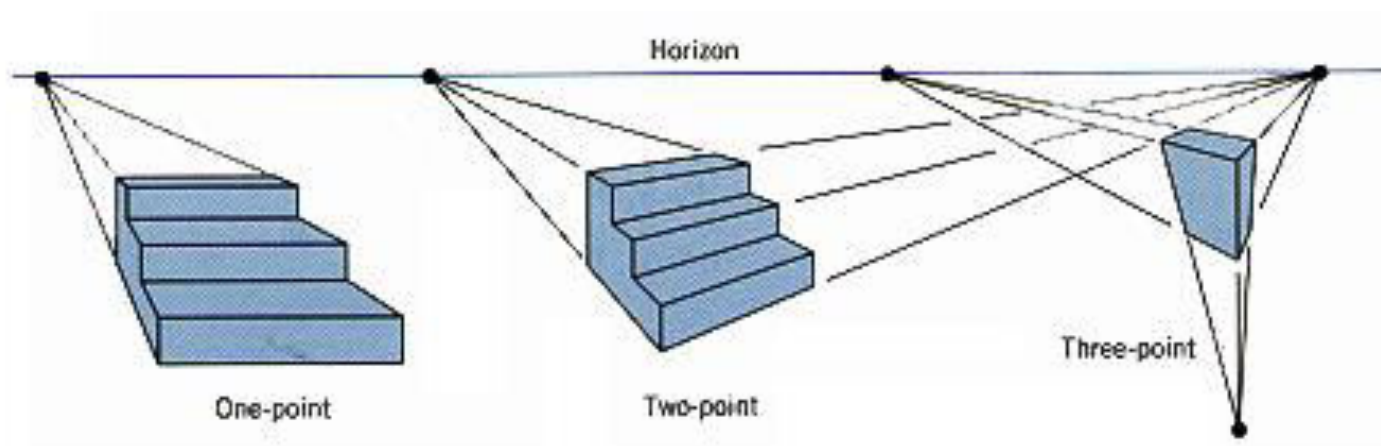
$$\mathbf{M}_{PER} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -\frac{1}{d_y} & 0 & 0 \end{bmatrix}$$

Perspective Projection Matrix

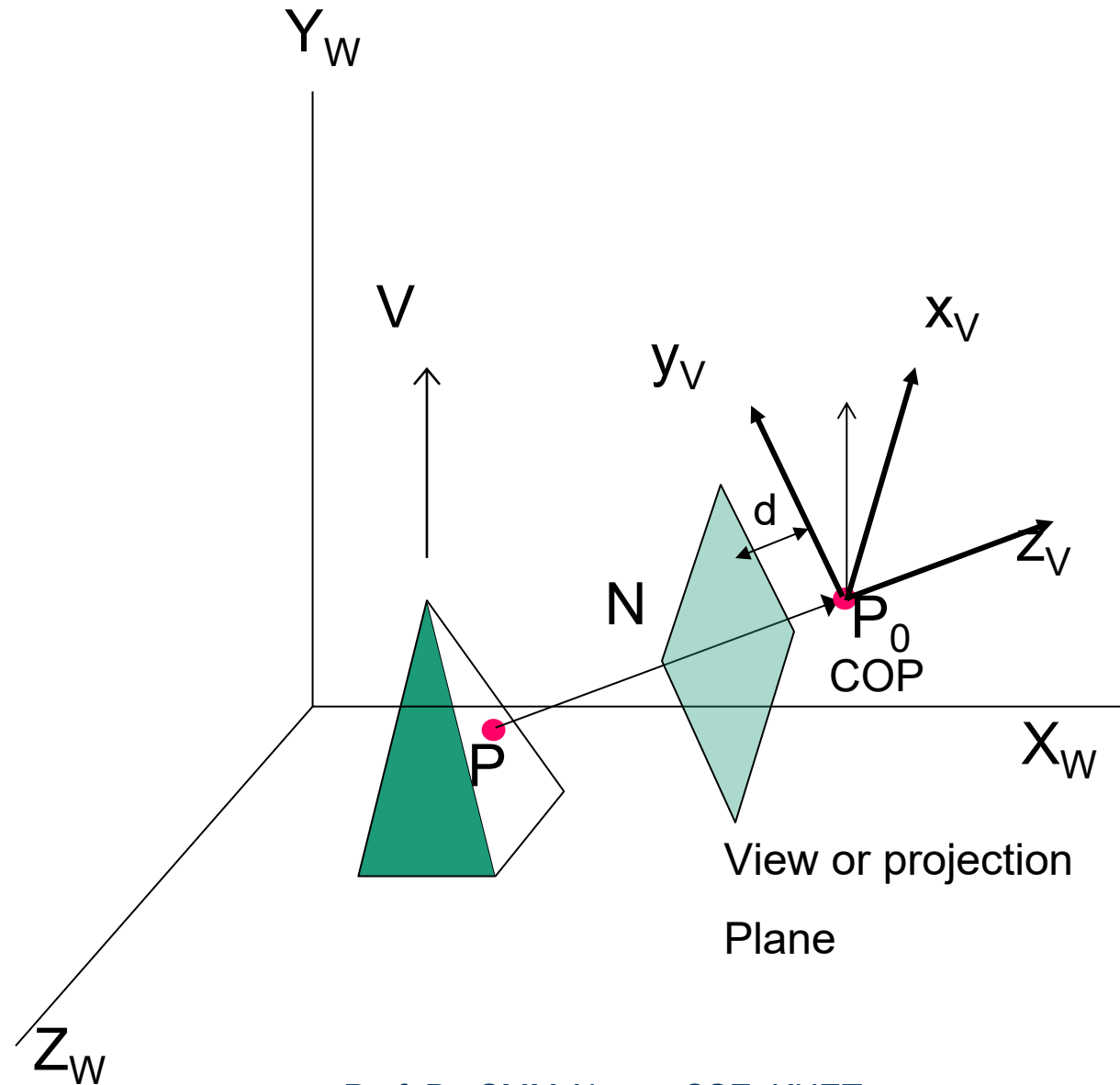
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ r & s & 0 & 0 \end{bmatrix} \quad \text{or} \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & s & t & 0 \end{bmatrix} \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ r & s & t & 0 \end{bmatrix}$$

2-point perspectives

3-point perspectives

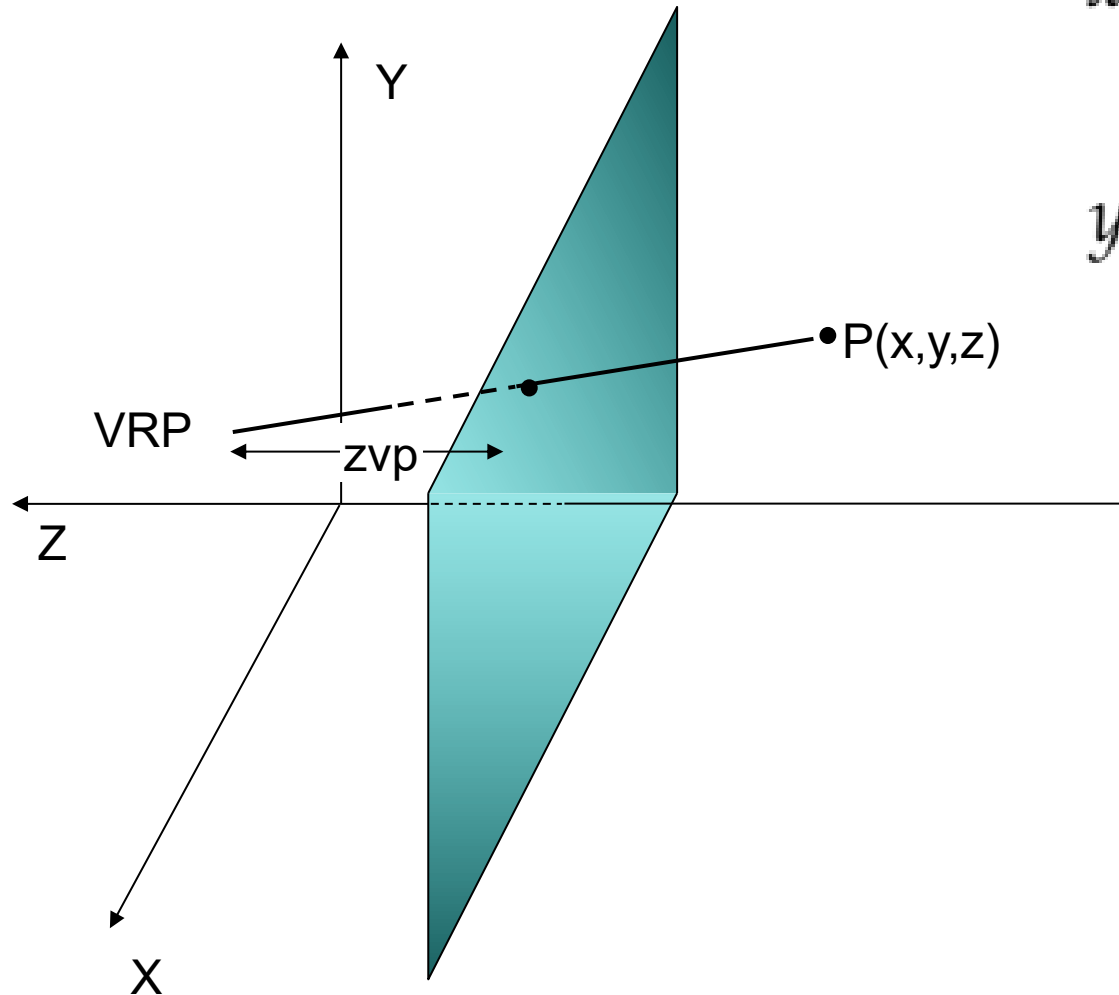


Relⁿ with VCS and projection



Perspective Projection

❑ Arbitrary VRP/COP

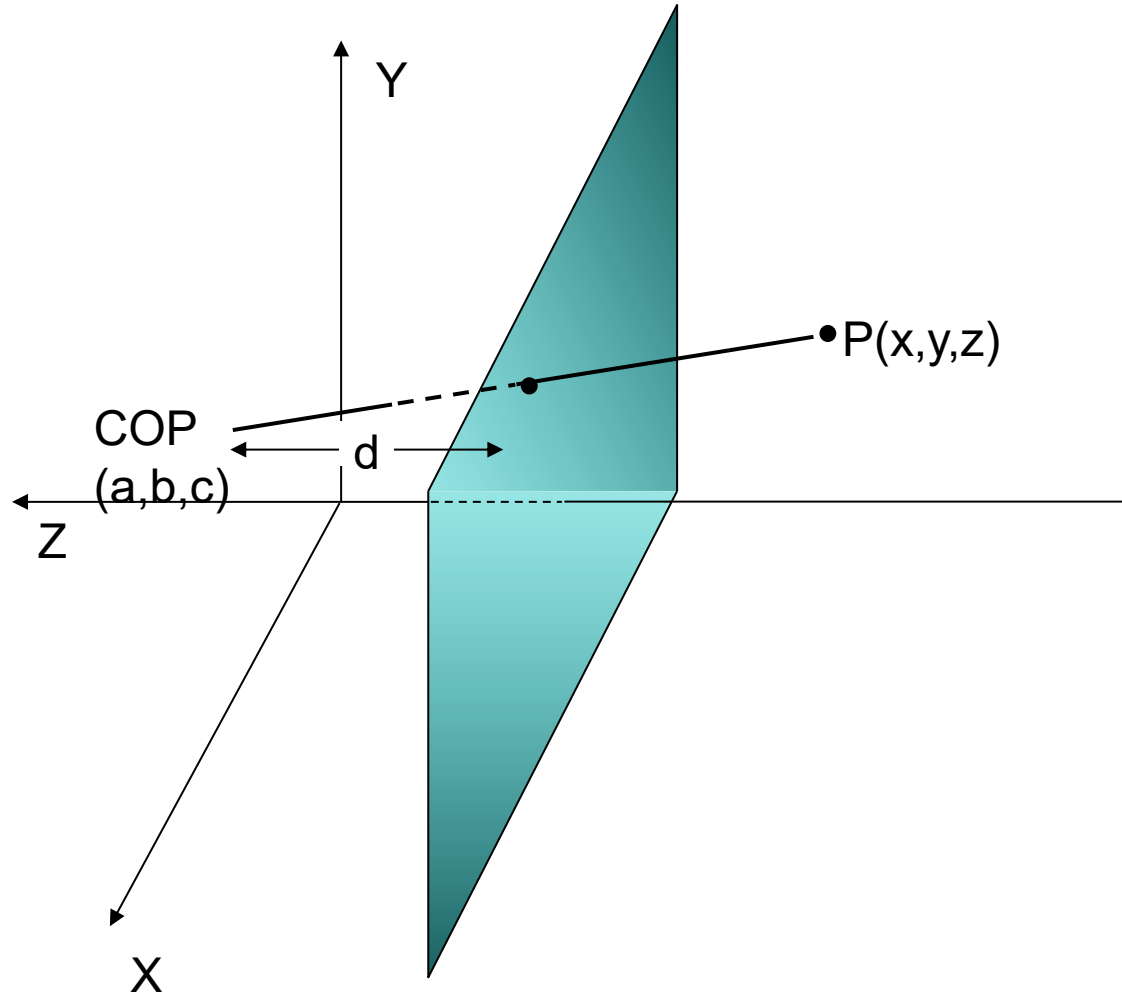


$$x_p = x \left(\frac{z_{prp} - z_{vp}}{z_{prp} - z} \right) + x_{prp} \left(\frac{z_{vp} - z}{z_{prp} - z} \right)$$

$$y_p = y \left(\frac{z_{prp} - z_{vp}}{z_{prp} - z} \right) + y_{prp} \left(\frac{z_{vp} - z}{z_{prp} - z} \right)$$

Perspective Projection

Arbitrary VRP/COP

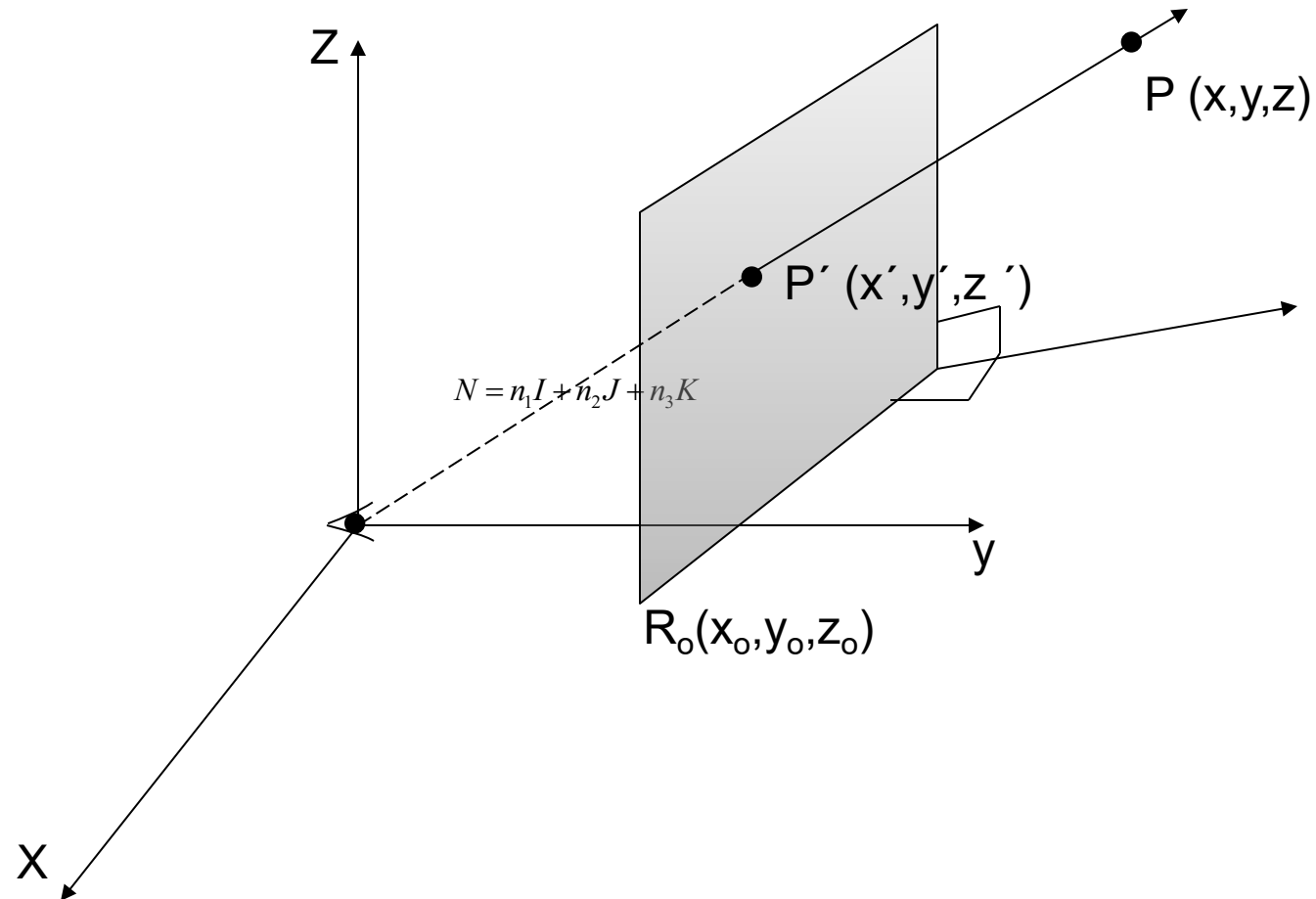


$$M_{Per,Vrp} = \begin{pmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{d-c} & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -a \\ 0 & 1 & 0 & -b \\ 0 & 0 & 1 & -c \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 & \frac{a}{d-c} & 0 \\ 0 & 1 & \frac{b}{d-c} & 0 \\ 0 & 0 & 1 + \frac{c}{d-c} & 0 \\ 0 & 0 & \frac{1}{d-c} & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -a \\ 0 & 1 & 0 & -b \\ 0 & 0 & 1 & -c \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & \frac{a}{d-c} & \frac{-ad}{d-c} \\ 0 & 1 & \frac{b}{d-c} & \frac{-bd}{d-c} \\ 0 & 0 & \frac{d}{d-c} & \frac{-cd}{d-c} \\ 0 & 0 & \frac{1}{d-c} & \frac{-c}{d-c} \end{pmatrix}$$

Perspective Projection – arbitrary plane

Using origin as COP, projection plane is a plane with normal N passing through point R_0



Perspective Projection – arbitrary plane

$\overline{OP}$ and $\overline{OP'}$ are same direction

$$\overline{OP'} = \alpha \overline{OP}$$

$$x' = \alpha x, \quad y' = \alpha y \quad z' = \alpha z$$

$$N \cdot \overline{P'R_0} = 0$$

$$n_1(x' - x_0) + n_2(y' - y_0) + n_3(z' - z_0) = 0$$

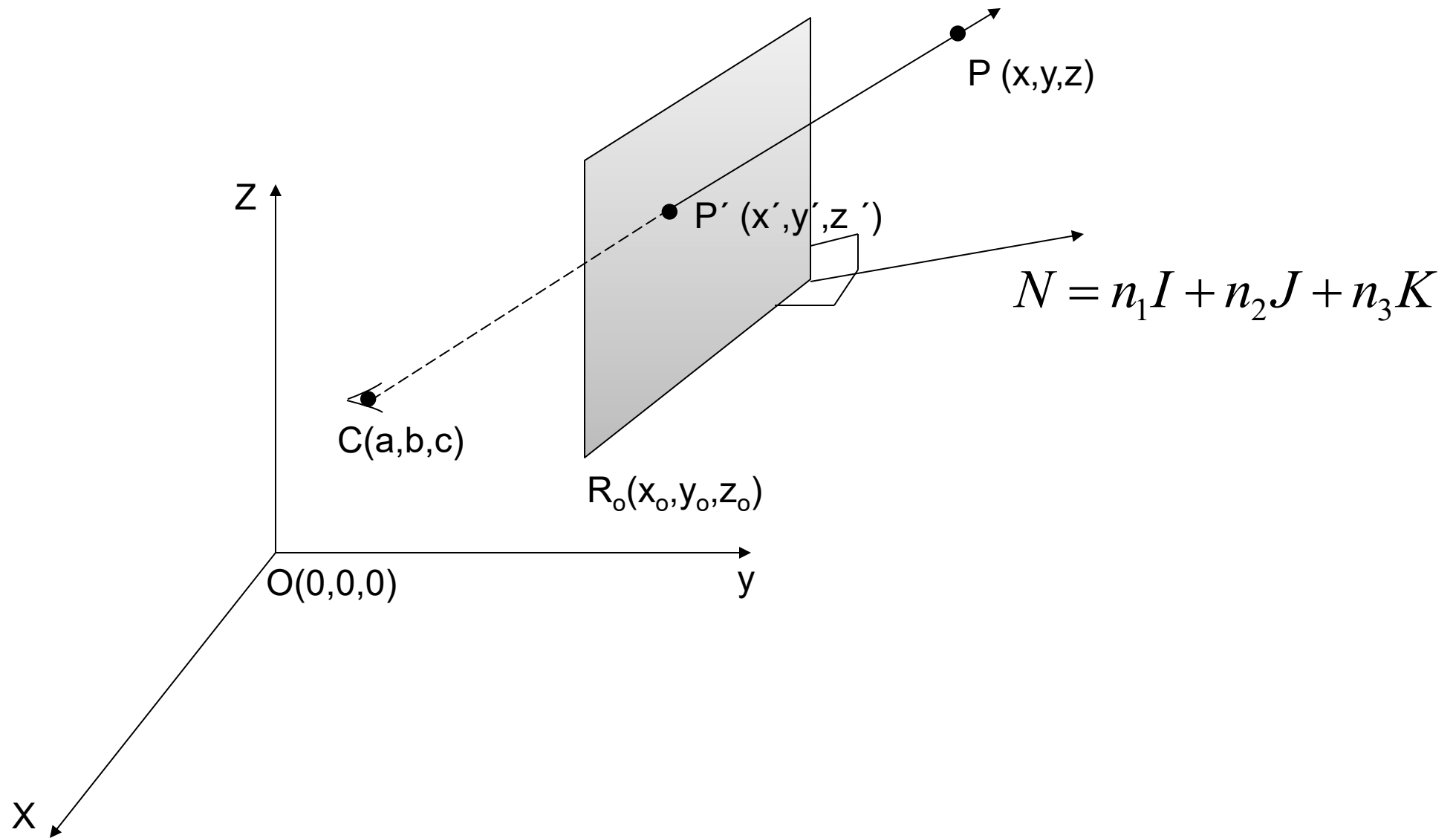
$$n_1x' + n_2y' + n_3z' = d_0 \quad , \quad \text{where } d_0 = n_1x_0 + n_2y_0 + n_3z_0$$

$$\alpha = \frac{d_0}{n_1x + n_2y + n_3z}$$

Perspective Projection – arbitrary plane

$$P' = Per_{N,R_0} \cdot P$$
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} d_0.x \\ d_0.y \\ d_0.z \\ n_1x + n_2y + n_3z \end{bmatrix} = \begin{bmatrix} d_0 & 0 & 0 & 0 \\ 0 & d_0 & 0 & 0 \\ 0 & 0 & d_0 & 0 \\ n_1 & n_2 & n_3 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Perspective Projection



Perspective Projection

$$P'C = \alpha PC$$

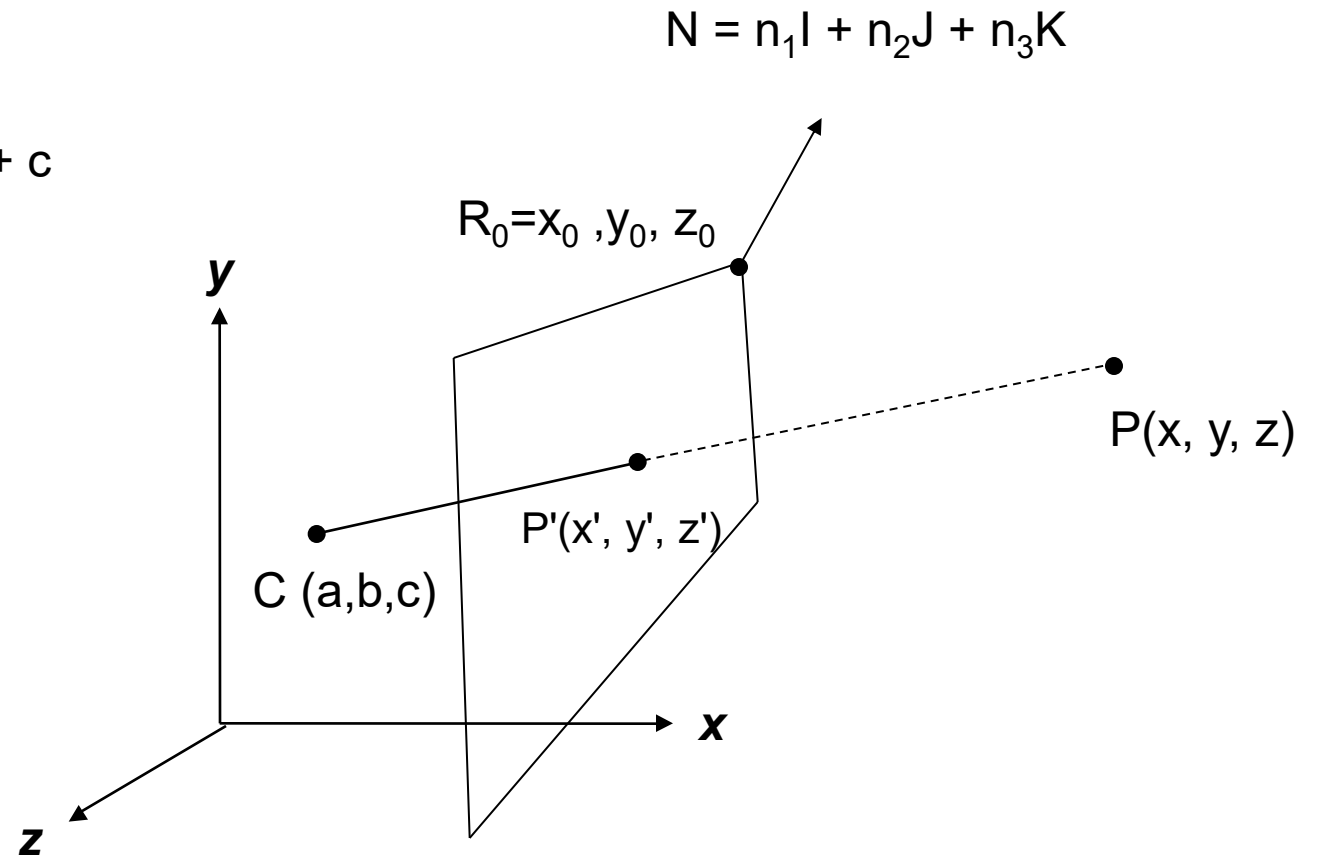
$$x' = \alpha(x-a) + a; \quad y' = \alpha(y-b) + b; \quad z' = \alpha(z-c) + c$$

$$n_1(x' - x_0) + n_2(y' - y_0) + n_3(z' - z_0) = 0$$

$$n_1x' + n_2y' + n_3z' = d_0$$

$$\alpha = \frac{d}{n_1(x-a) + n_2(y-b) + n_3(z-c)}$$

$$\begin{aligned} d &= (n_1x_0 + n_2y_0 + n_3z_0) - (n_1a + n_2b + n_3c) \\ &= d_0 - d_1 \end{aligned}$$



Perspective Projection - DIY

□ Follow the steps –

- Translate so that C lies at the origin, hence, $R_0 = (x_0 - a, y_0 - b, z_0 - c)$

▪ Per

- Translate back

$$\begin{bmatrix} 1 & 0 & 0 & a \\ 0 & 1 & 0 & b \\ 0 & 0 & 1 & c \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} d & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & d & 0 \\ n_1 & n_2 & n_3 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & -a \\ 0 & 1 & 0 & -b \\ 0 & 0 & 1 & -c \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

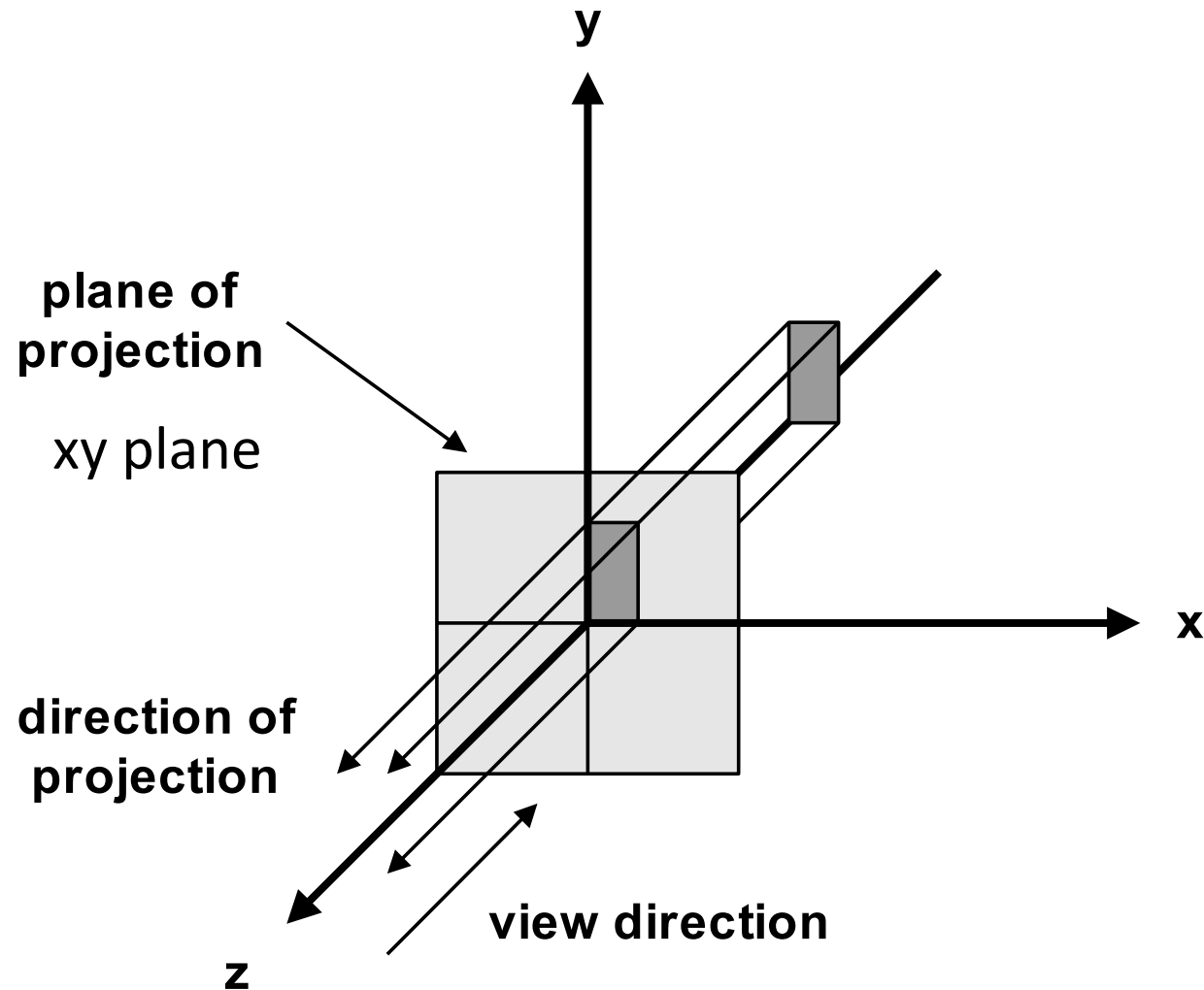
$$\begin{aligned} d &= (n_1 x_0 + n_2 y_0 + n_3 z_0) - (n_1 a + n_2 b + n_3 c) \\ &= d_0 - d_1 \end{aligned}$$

$$\begin{pmatrix} d + an_1 & an_2 & an_3 & -ad_0 \\ bn_1 & d + bn_2 & bn_3 & -bd_0 \\ cn_1 & cn_2 & d + cn_3 & -cd_0 \\ n_1 & n_2 & n_3 & -d_1 \end{pmatrix}$$

Finding Vanishing Point

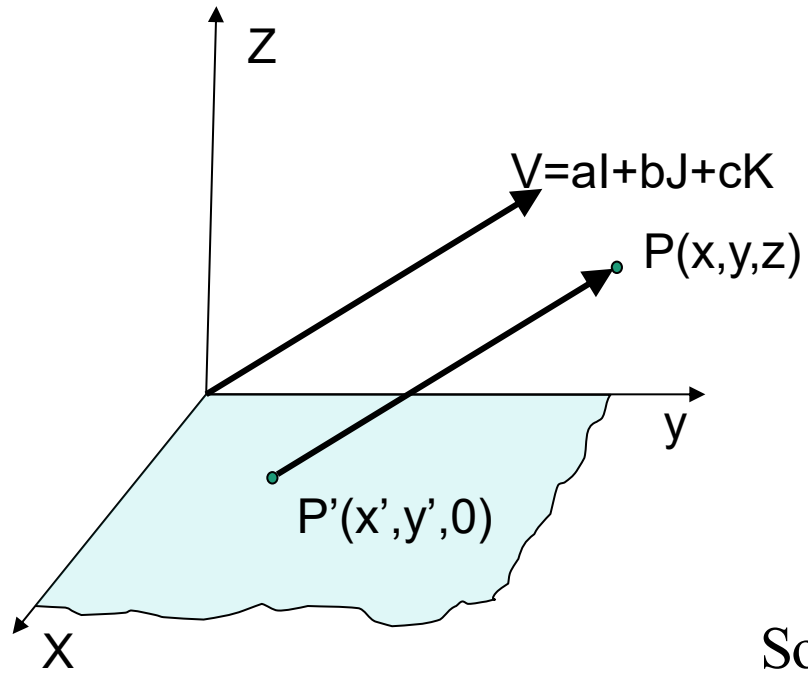
- ❑ Find (a) the vanishing points for a given perspective transformation in the direction given by a vector U , (b) principal vanishing point.
- ❑ Family of parallel lines having the direction $U(u_1, u_2, u_3)$ can be written in parametric form as
 - $x = u_1t + p, y = u_2t + q, z = u_3t + r$
 - here $P(p, q, r)$ is any point on the line
- ❑ Let, $\text{proj}(x, y, z, 1) = (x', y', z', h)$
 - $x' = (d + an_1)(u_1t + p) + an_2(u_2t + q) + an_3(u_3t + r) - ad_0$
 - $y' = bn_1(u_1t + p) + (d + bn_2)(u_2t + q) + bn_3(u_3t + r) - bd_0$
 - $z' = cn_1(u_1t + p) + cn_2(u_2t + q) + (d + cn_3)(u_3t + r) - cd_0$
 - $h = n_1(u_1t + p) + n_2(u_2t + q) + n_3(u_3t + r) - d_1$

Orthographic Projection Matrix



$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ 0 \\ 1 \end{pmatrix}$$

Parallel projection on xy plane with DOP $\mathbf{V} = a\mathbf{I} + b\mathbf{J} + c\mathbf{K}$



V and $\overline{P'P}$ has same direction,

$$\text{so } \overline{P'P} = kV$$

Comparing components

$$x - x' = ka \quad y - y' = kb \quad z - z' = kc$$

Since, projection on xy plane, $z' = 0$

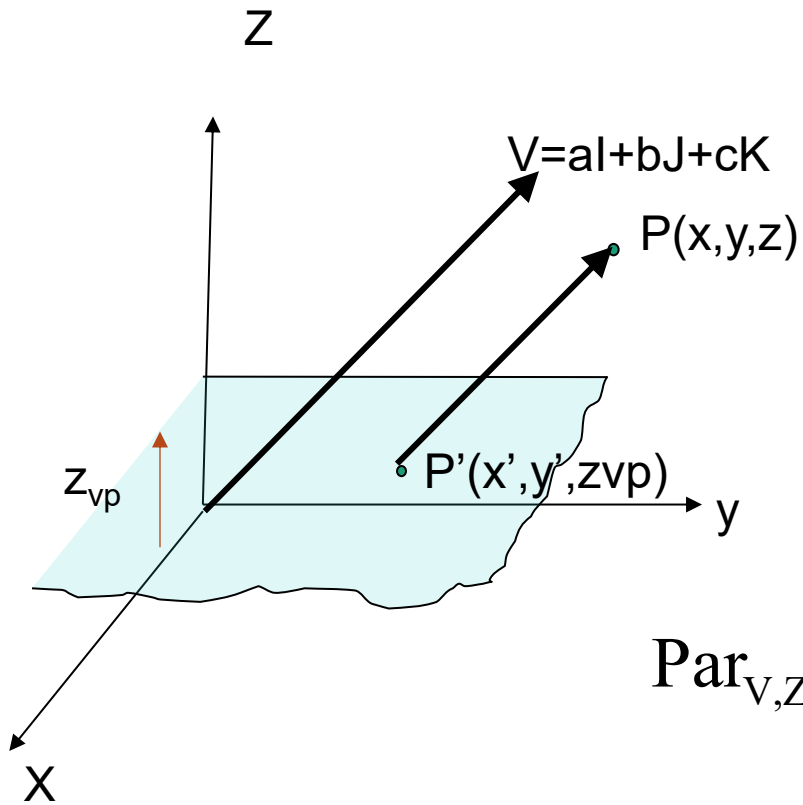
$$\text{So, } k = \frac{z}{c} \quad x' = x - \frac{a}{c}z \quad y' = y - \frac{b}{c}z$$

$$\text{So, } P' = \text{Par}_V \bullet P$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} x - \frac{a}{c}z \\ y - \frac{b}{c}z \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & -\frac{a}{c} & 0 \\ 0 & 1 & -\frac{b}{c} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Parallel projection with DOP $\mathbf{V} = a\mathbf{I} + b\mathbf{J} + c\mathbf{K}$

▣ projection on $z=z_{vp}$



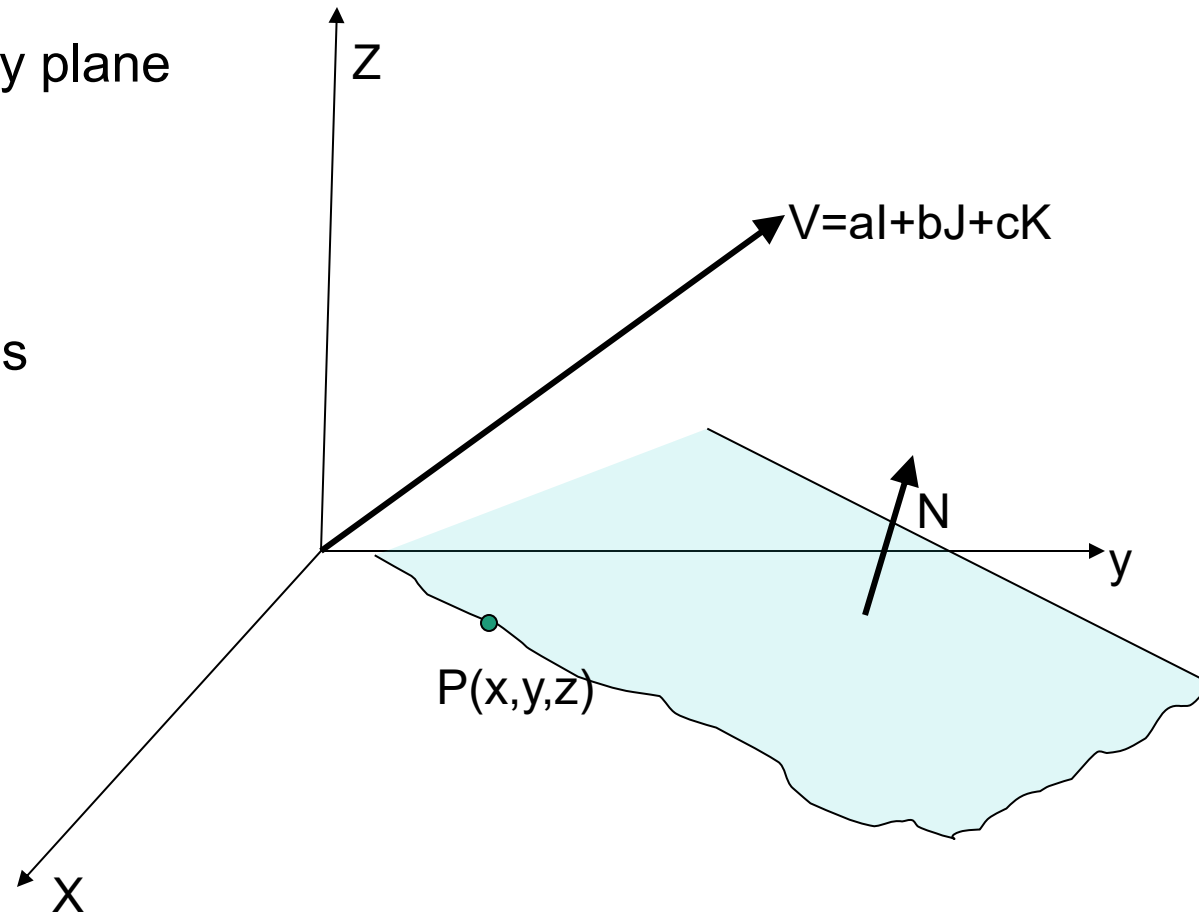
$$\text{Par}_{V, Z_{vp}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & z_{vp} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -a/c & 0 \\ 0 & 1 & -b/c & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -z_{vp} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Derive eq of parallel projection with DOP $\mathbf{V} = a\mathbf{I} + b\mathbf{J} + c\mathbf{K}$ on plane with plane normal $\mathbf{N}$, passing through P_0

Do by yourself

Hint:

- Align the plane with xy plane
- Adjust the DOP
- Do the projection
- Do the reverse actions



Derive eq of parallel projection with DOP $\mathbf{V} = a\mathbf{I} + b\mathbf{J} + c\mathbf{K}$ on plane with plane normal $\mathbf{N}$, passing through P_0

Do by yourself

Hint:

- Align the plane with xy plane
- Adjust the DOP
- Do the projection
- Do the reverse actions

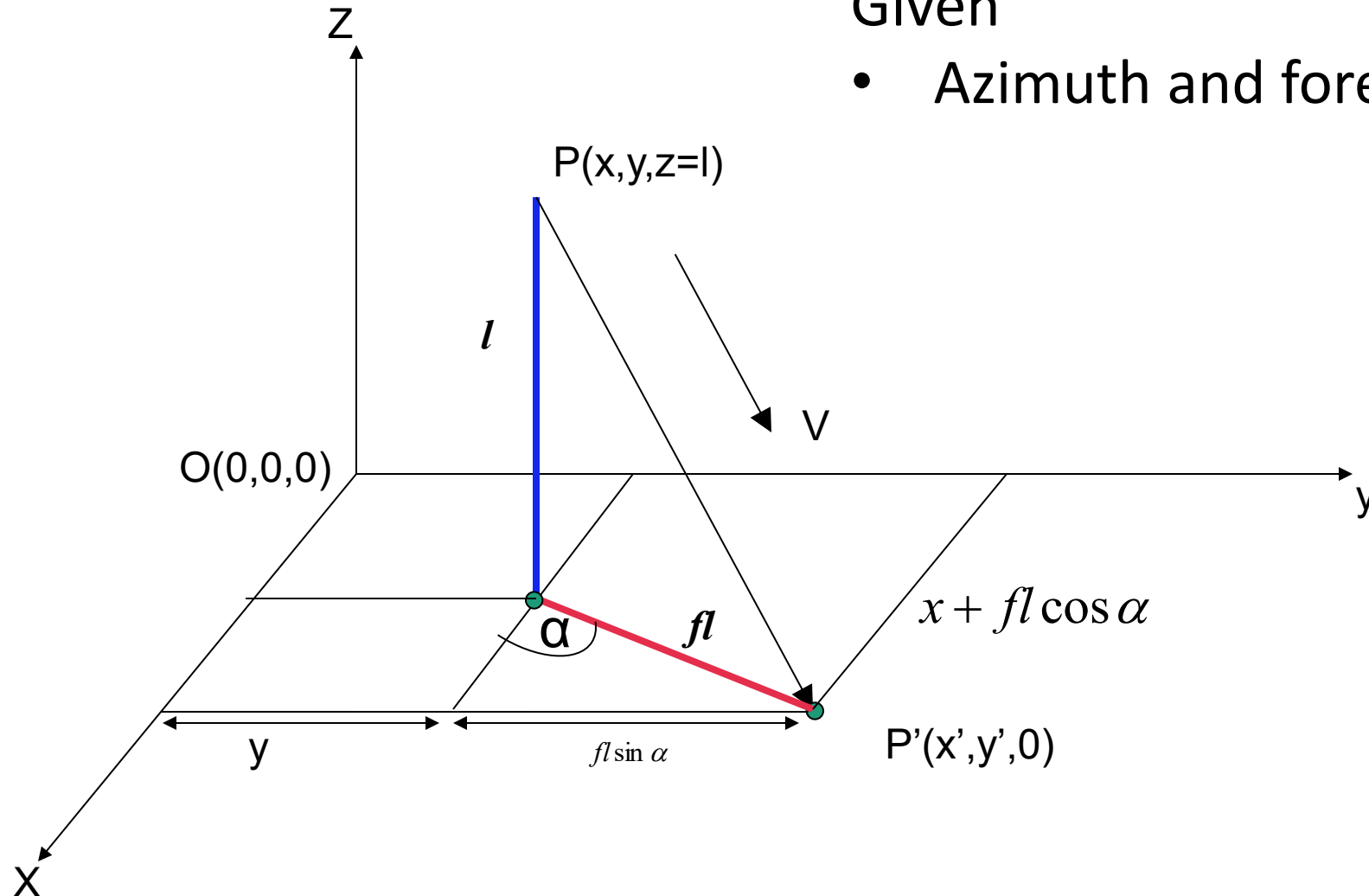
$$Par_{\mathbf{V}, \mathbf{N}, P_0} = \begin{pmatrix} d_1 - an_1 & -an_2 & -an_3 & ad_0 \\ -bn_1 & d_1 - bn_2 & -bn_3 & bd_0 \\ -cn_1 & -cn_2 & d_1 - cn_3 & cd_0 \\ 0 & 0 & 0 & d_1 \end{pmatrix}$$

Here $d_0 = n_1x_0 + n_2y_0 + n_3z_0$ and $d_1 = n_1a + n_2b + n_3c$.

Math of Oblique Projection

Given

- Azimuth and foreshortening factor



Solution

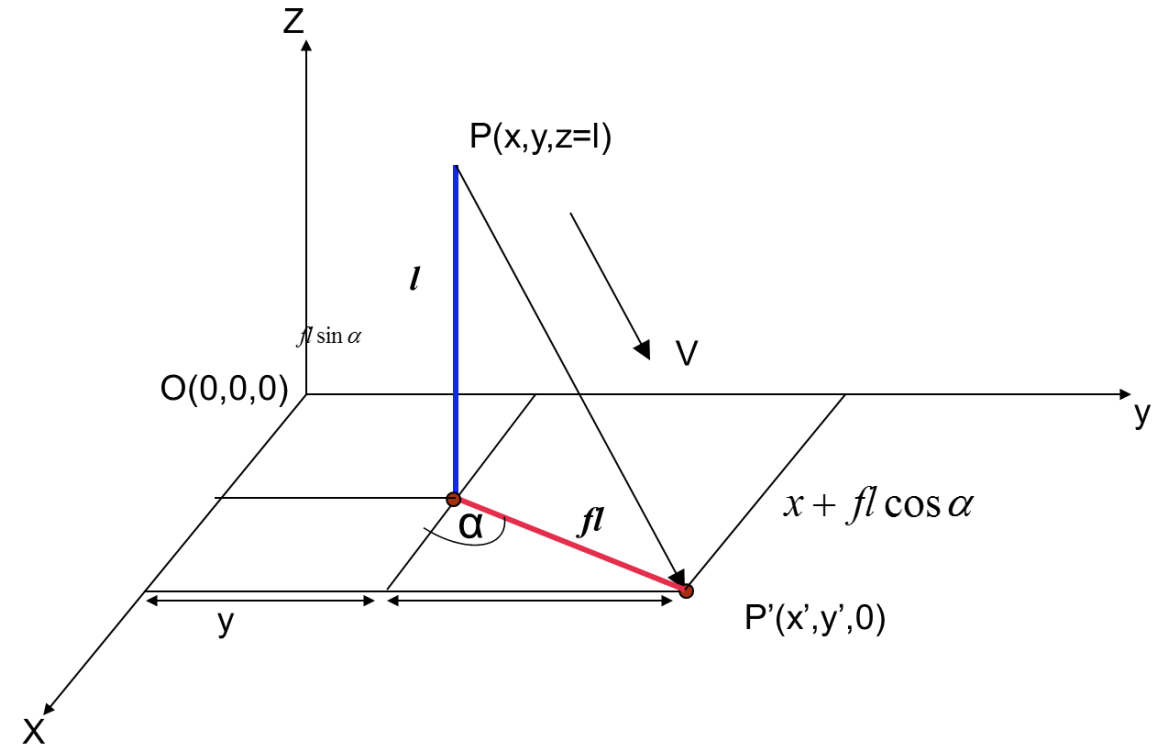
$$x' = x + fl \cos \alpha = x + zf \cos \alpha$$

$$y' = y + fl \sin \alpha = y + zf \sin \alpha$$

$$z' = 0$$

In Matix form

$$\text{Par}_v = \begin{bmatrix} 1 & 0 & f \cos \alpha & 0 \\ 0 & 1 & f \sin \alpha & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Solution

Given

- Azimuth and elevation

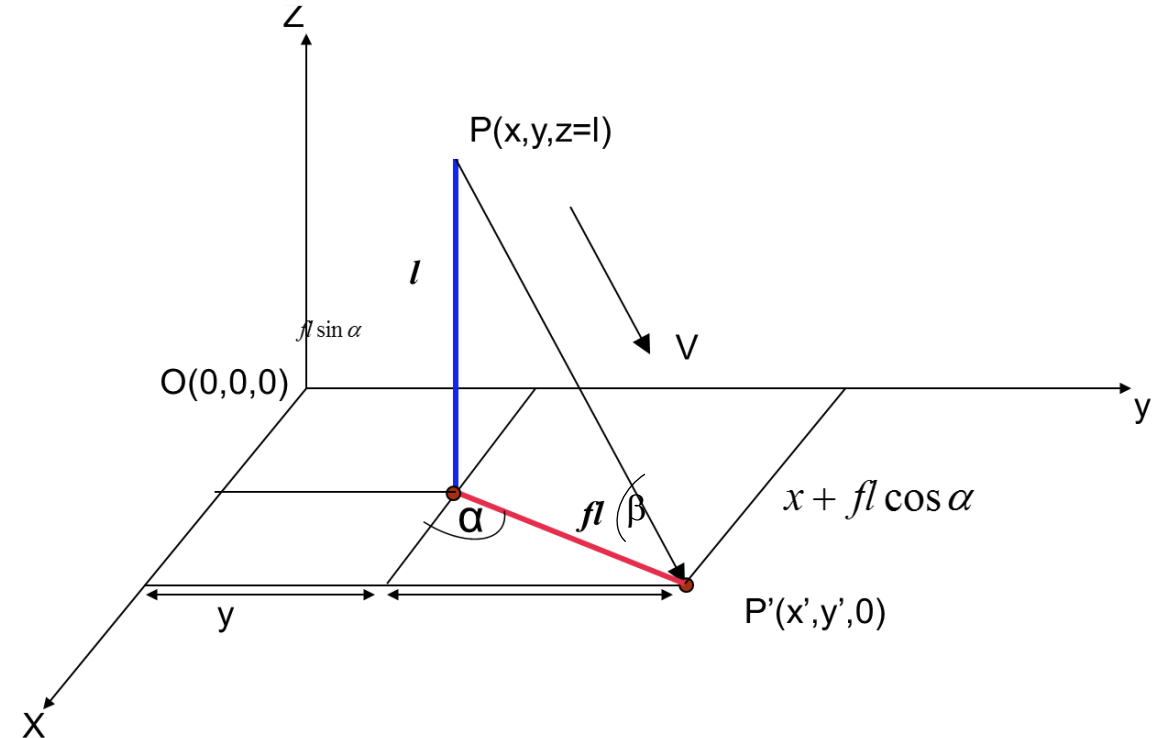
$$x' = x + fl \cos \alpha = x + zf \cos \alpha$$

$$y' = y + fl \sin \alpha = y + zf \sin \alpha$$

$$z' = 0$$

In Matix form

$$\text{Par}_v = \begin{bmatrix} 1 & 0 & f \cos \alpha & 0 \\ 0 & 1 & f \sin \alpha & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

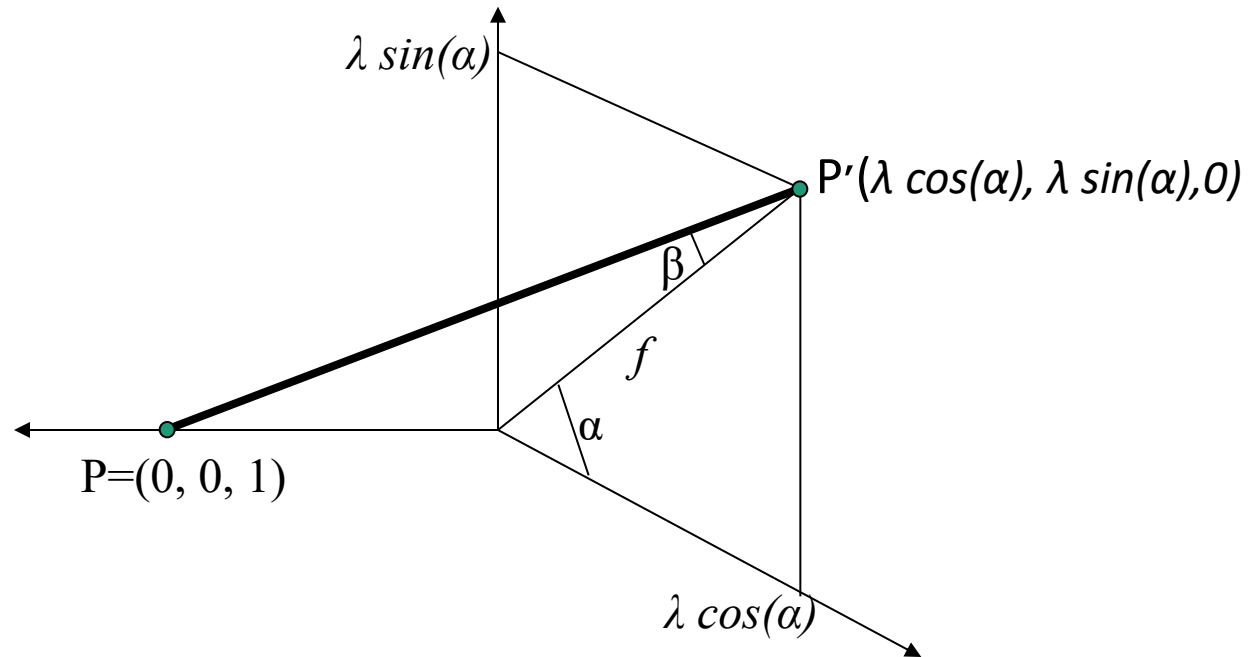


$$\tan \beta = \frac{1}{f}$$

Parallel projection

□ Cavalier, cabinet and orthographic projections can all be specified in terms of (α, β) or (α, f) since

- $\tan(\beta) = 1/f$



Parallel projection

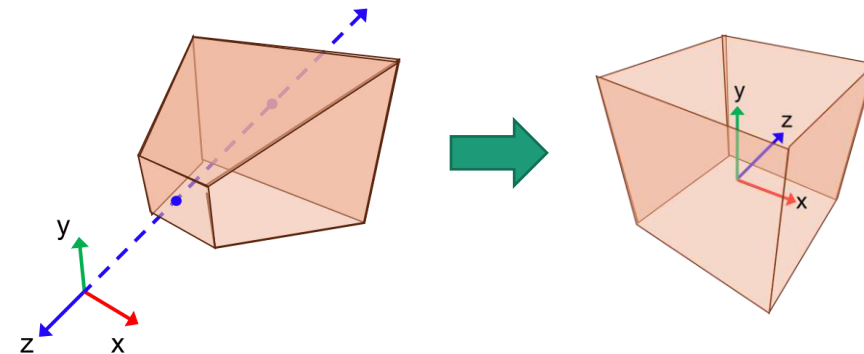
$\lambda=1$	$\beta = 45$	Cavalier projection	$\alpha = 0 - 360$
$\lambda=0.5$	$\beta = 63.4$	Cabinet projection	$\alpha = 0 - 360$
$\lambda=0$	$\beta = 90$	Orthogonal projection	$\alpha = 0 - 360$

Problems

❑ Schaums Series

- 7.2
- 7.6

- A2.14



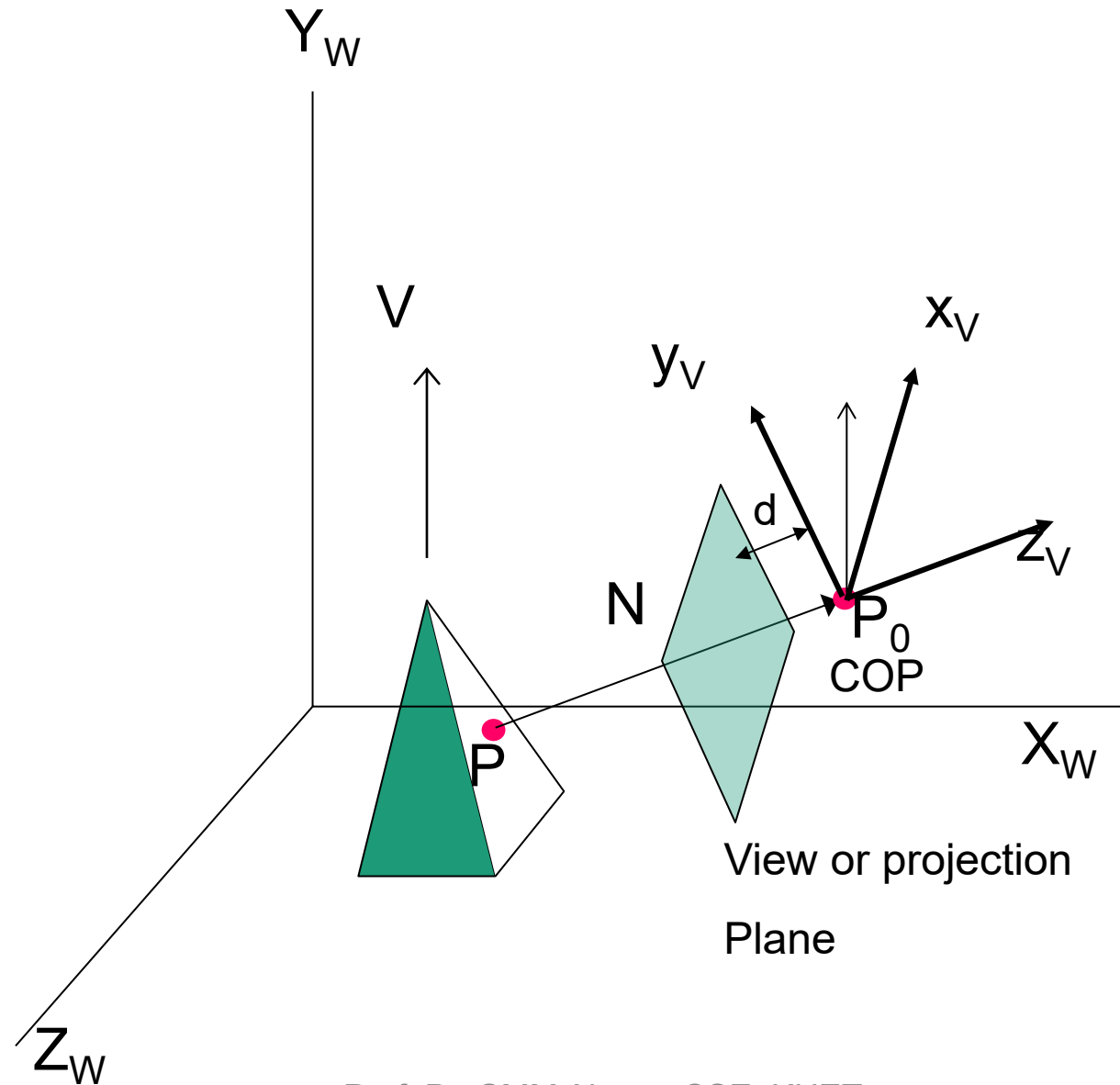
Projective Transformation

Dr. Sk. Md. Masudul Ahsan

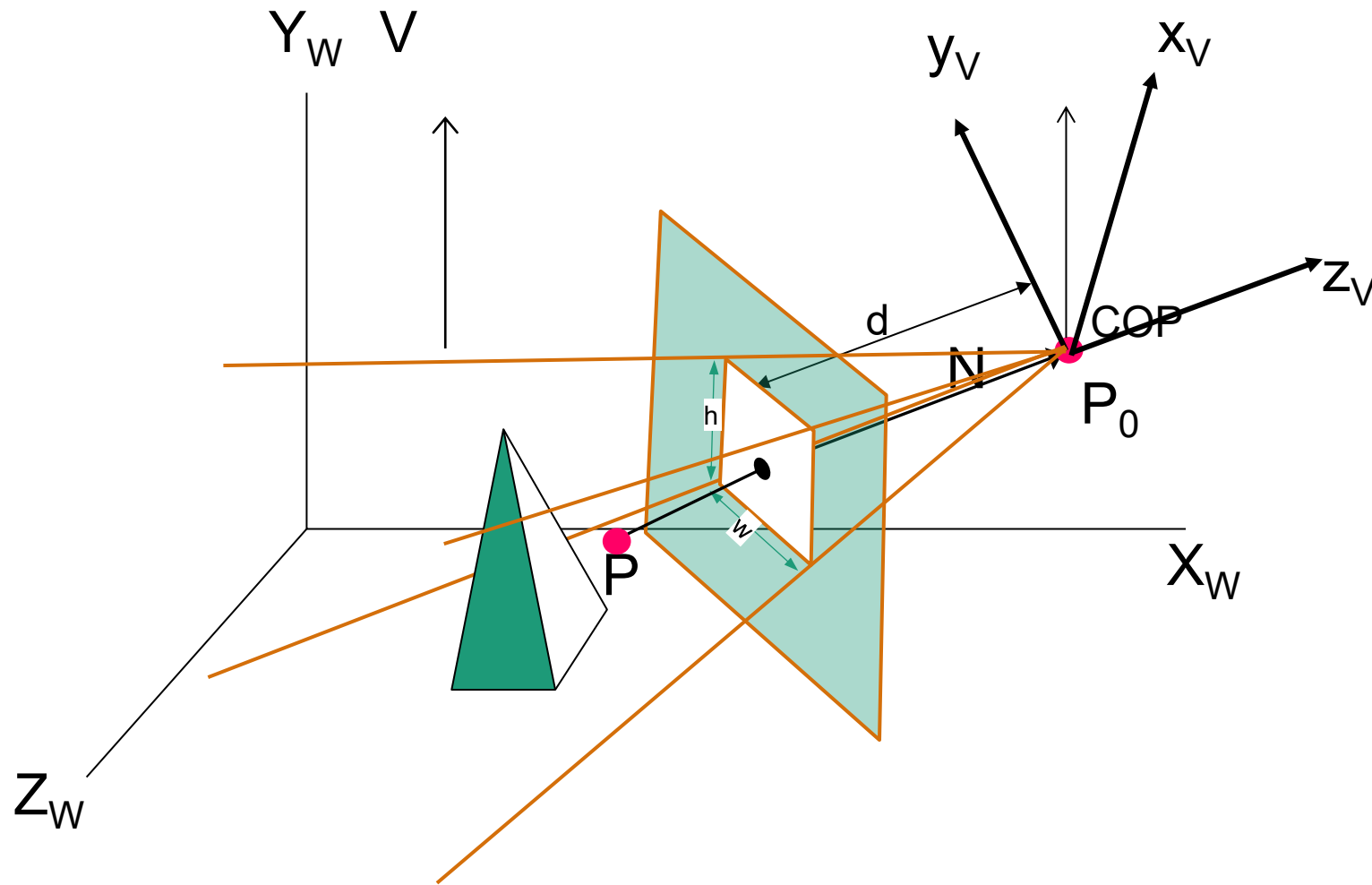
Professor

Dept. of CSE, KUET

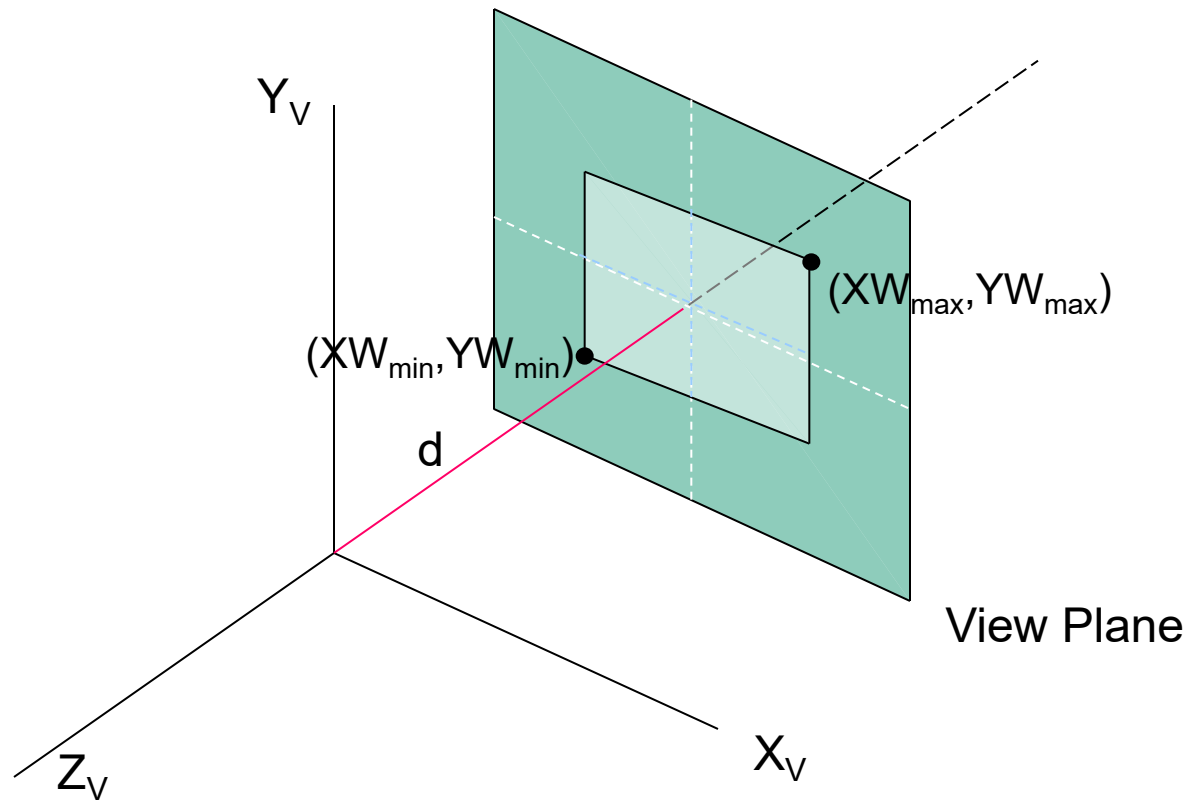
WCS, VCS and projection



View Window



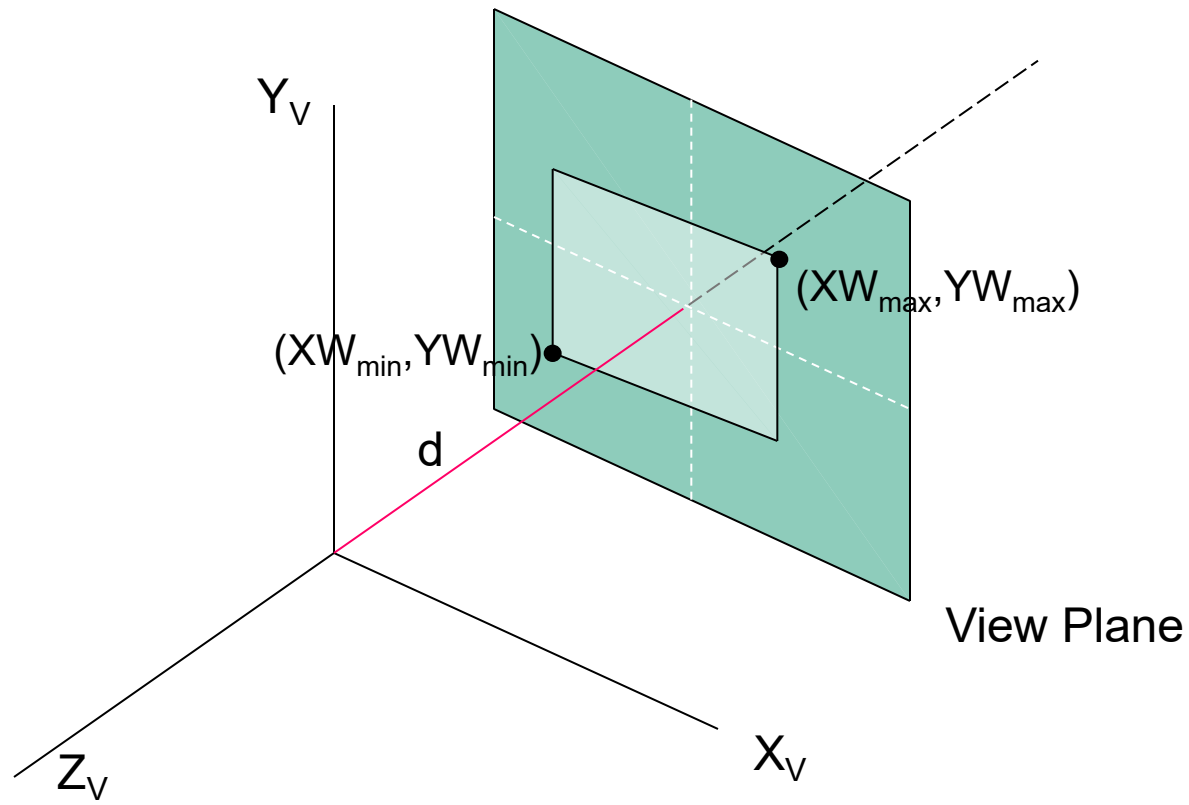
View Window - symmetric



$$XW_{max} = -XW_{min},$$

$$YW_{max} = -YW_{min}$$

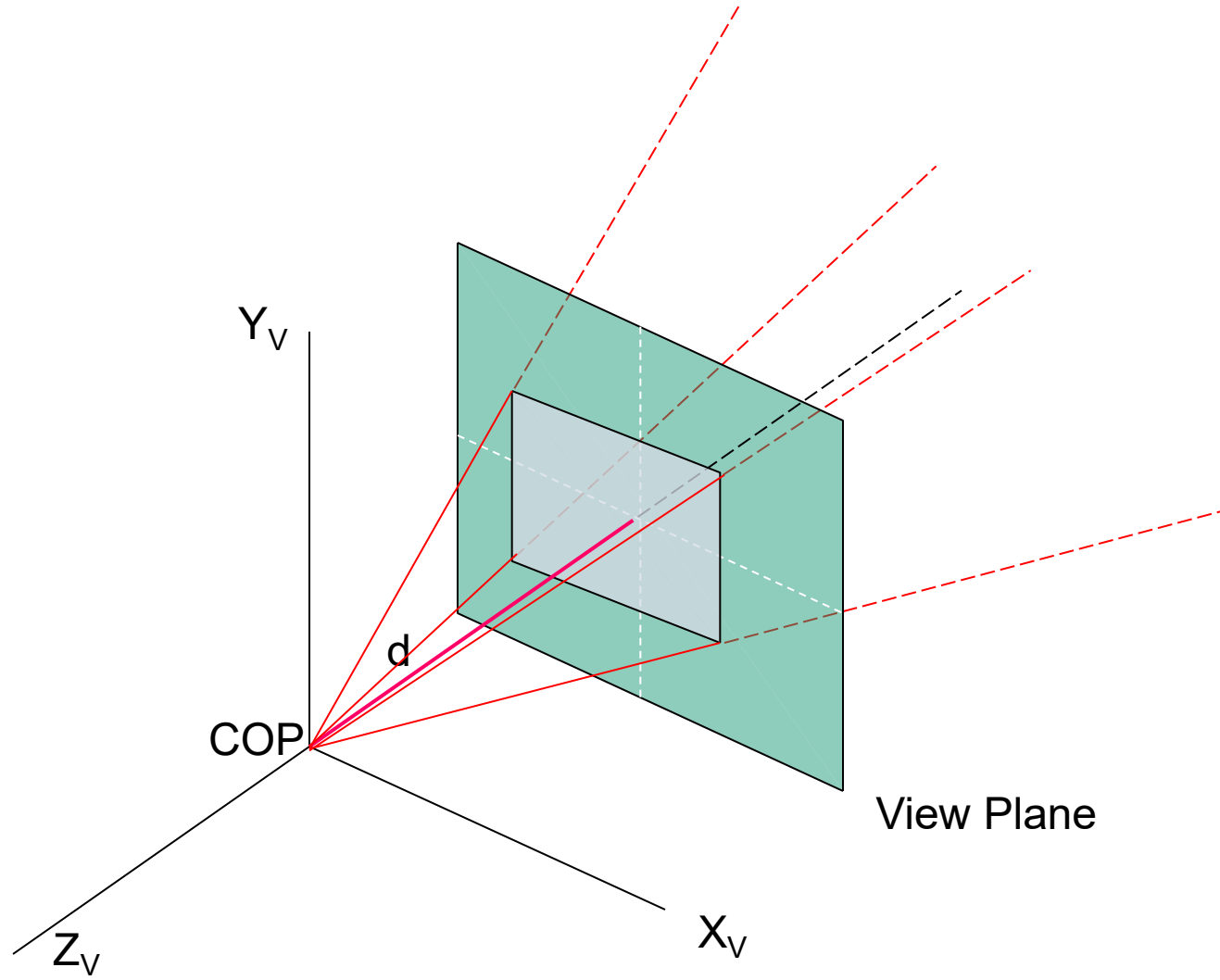
View Window - asymmetric



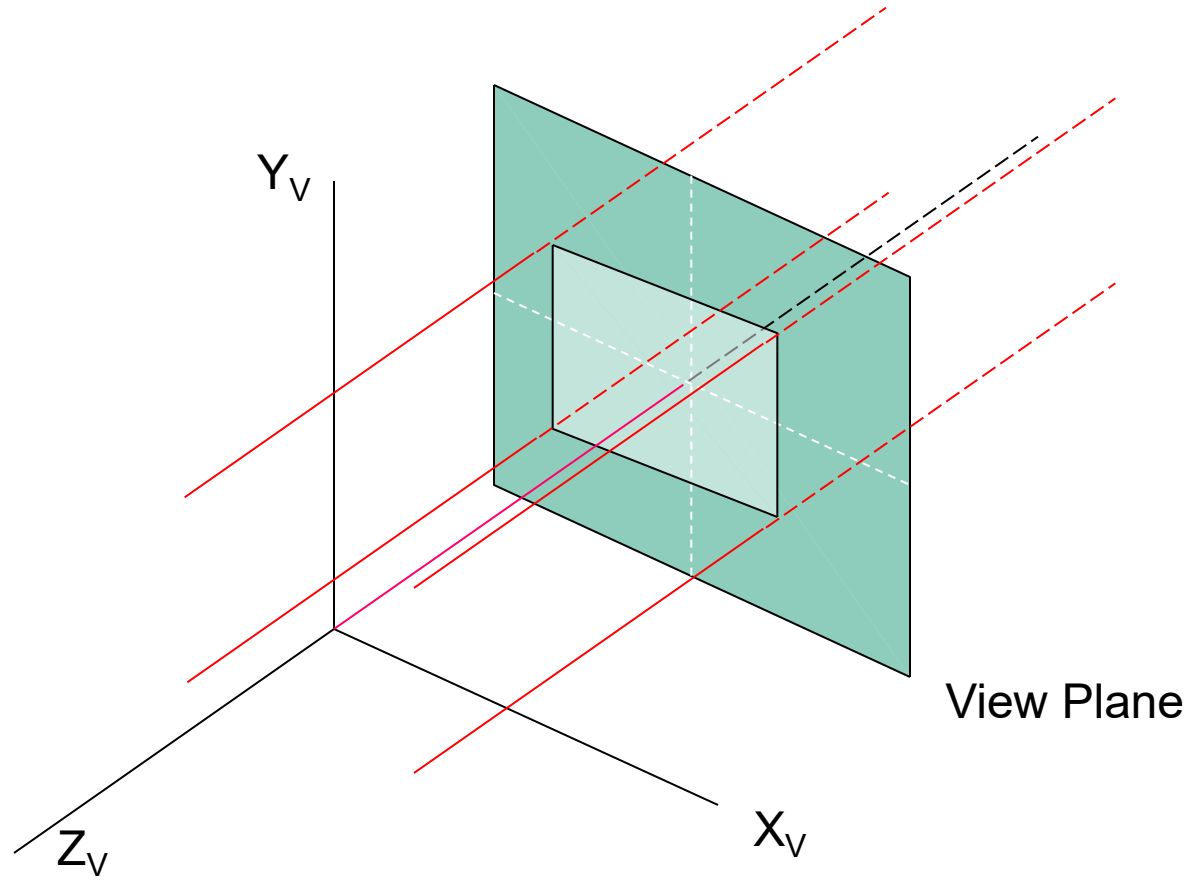
$$XW_{max} \neq -XW_{min},$$

$$YW_{max} \neq -YW_{min}$$

View Volume - Perspective



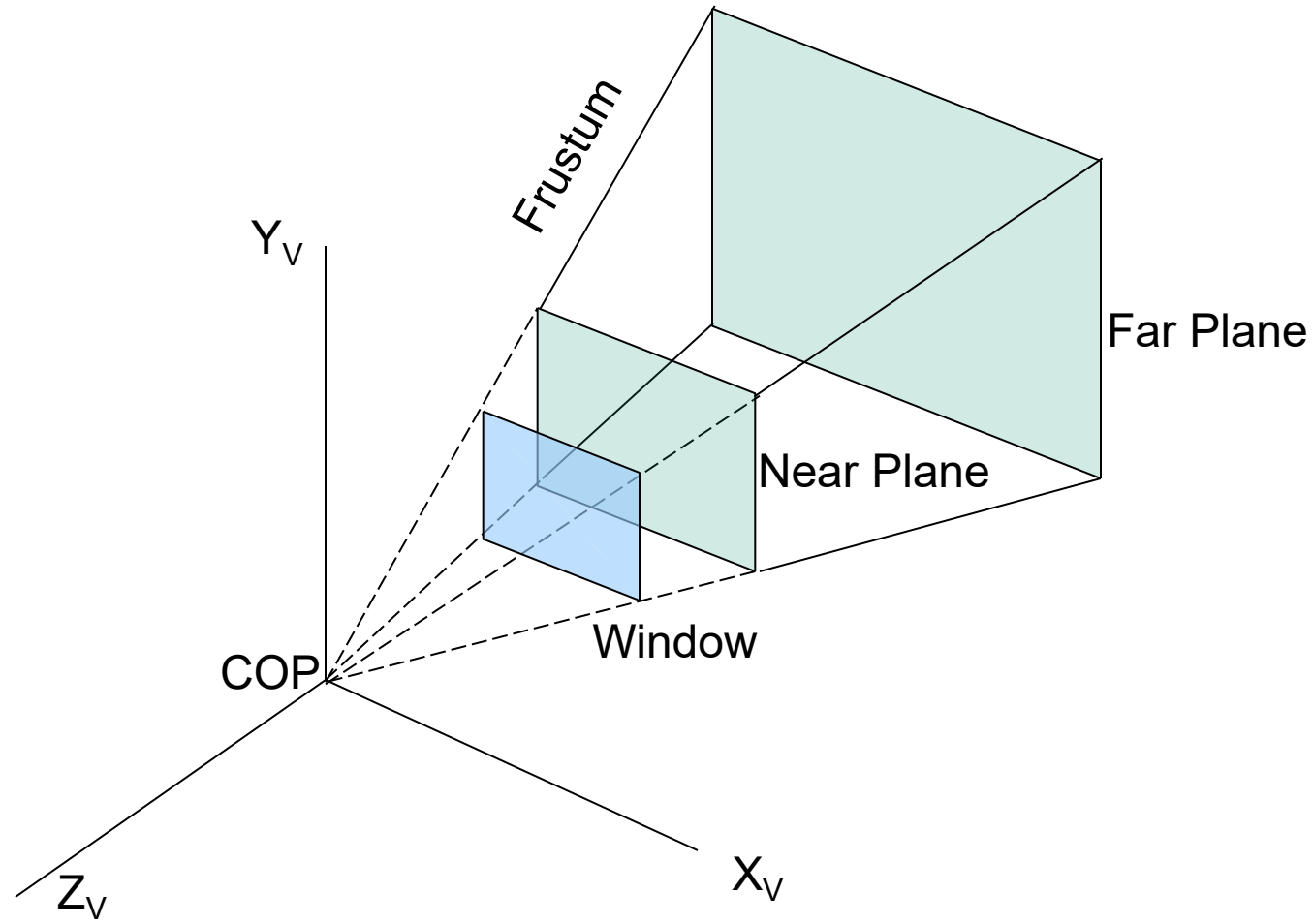
View Volume – Orthographic Projection



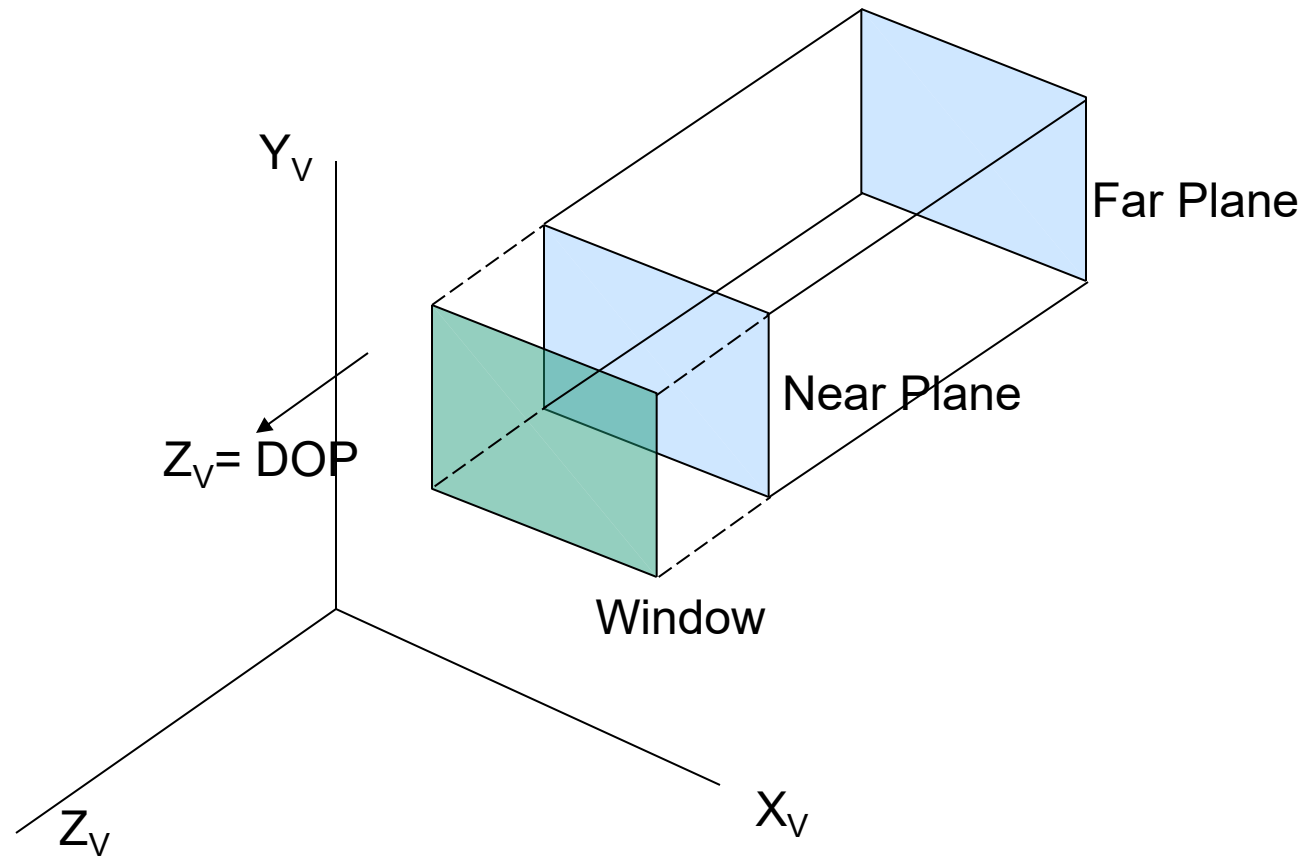
View Volume

- ❑ Finite $\sim$ is obtained by limiting the extent of volume in Z_v direction
- ❑ Two plane parallel to view plane are used for this purpose, planes are –
 - **Near / Front plane** and **Far / Back plane**
 - Both must be same side of COP
- ❑ Adv. of using **Near, Far plane**
 - These planes eliminates parts of the scene from viewing operation based on depth.
 - Special adv. in case of perspective projection

Finite View Volume - Perspective

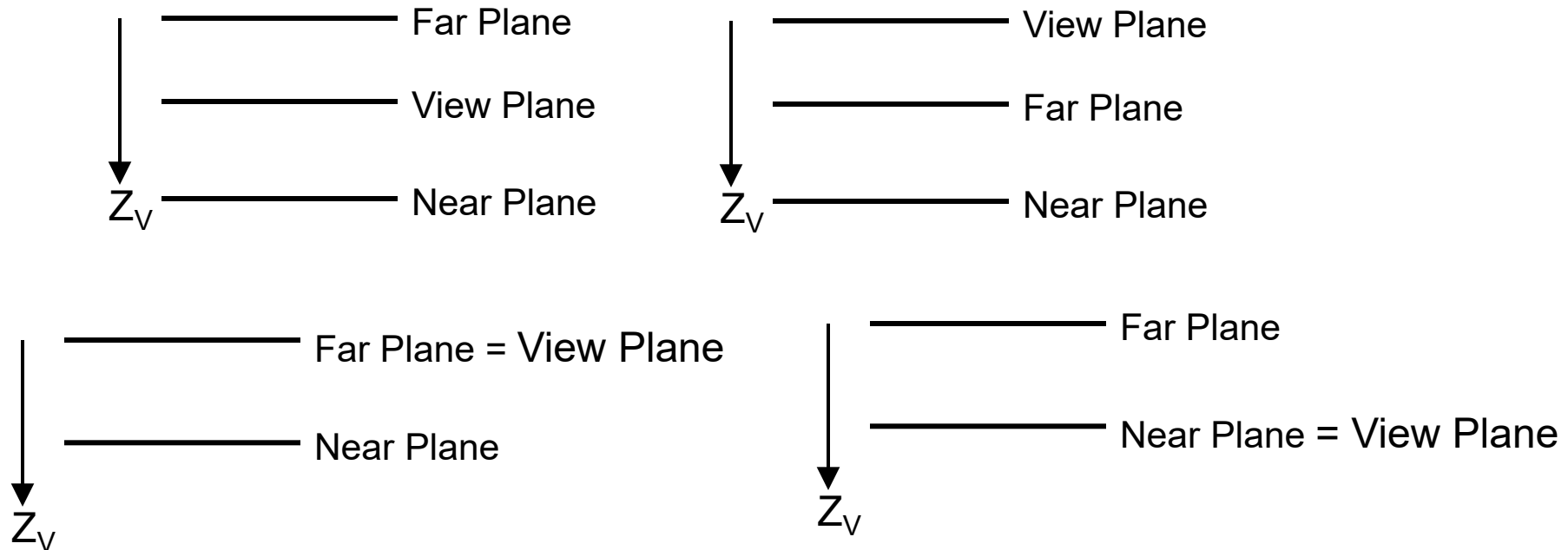


Finite View Volume – Orthographic Projection



Relative placement

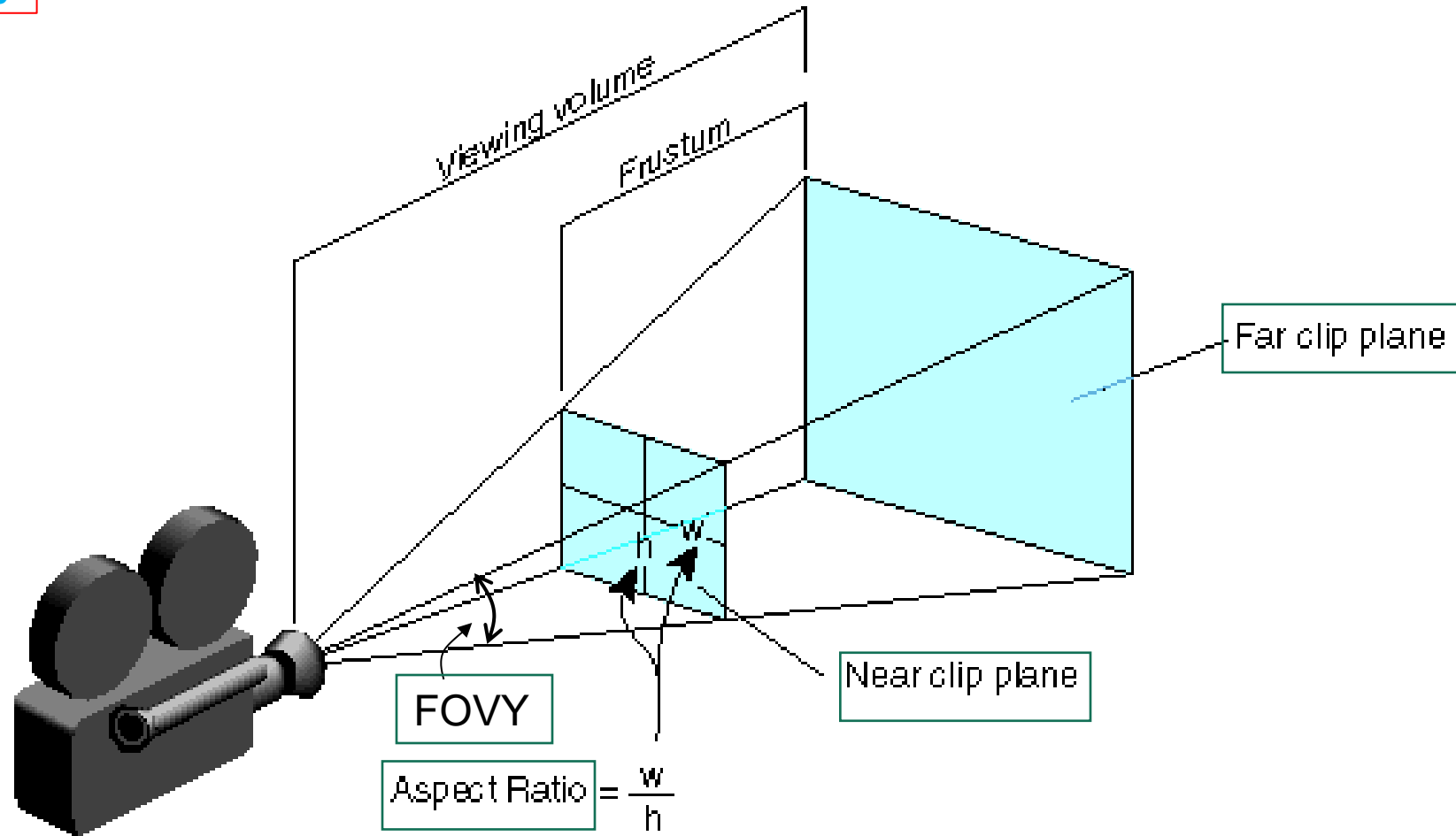
- ~ of **View, Near, Far planes** depend on
 - type of view we want &
 - limitation of Graphics package



OpenGL Uses it

Symmetric view volume - Perspective

Another way



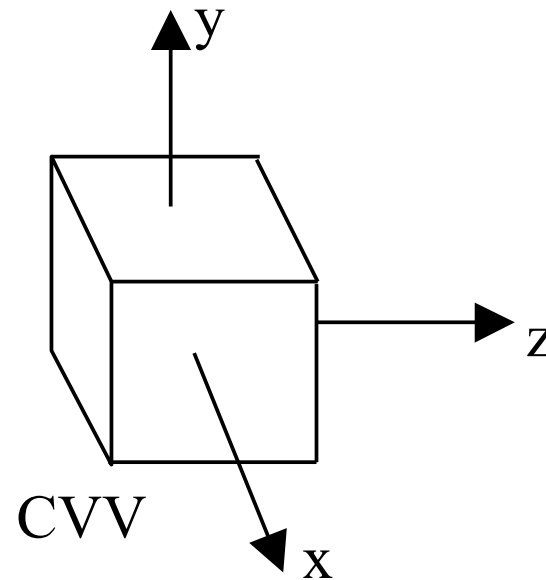
$$h = 2d * \tan(\text{fovy}/2)$$

Normalization Transformation

- ❑ Transforms VCS values to NDCS one

- ❑ Canonical View Volumes

- The 3D object model is not actually clipped inside the projection view volume.
- The view volume is instead mapped to a **canonical view volume** (CVV) which is a cube that extends from -1 to $+1$ in each dimension, having center at the origin.
- The dimensions of the CVV facilitates a fast and efficient clipping.



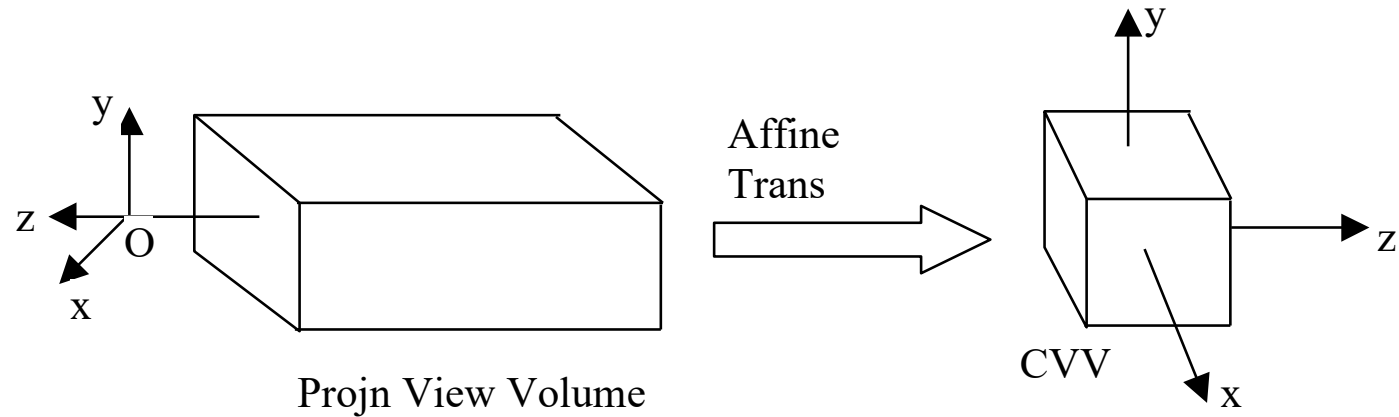
$$-1 \leq x \leq 1$$

$$-1 \leq y \leq 1$$

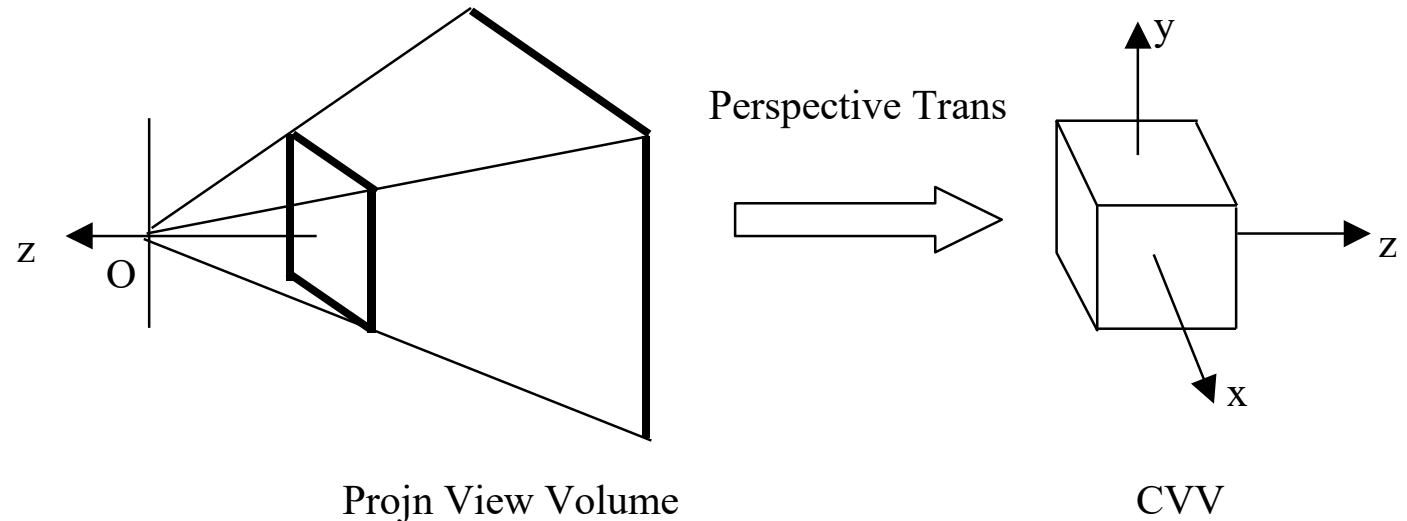
$$-1 \leq z \leq 1$$

Mapping to CVV

Orthographic:



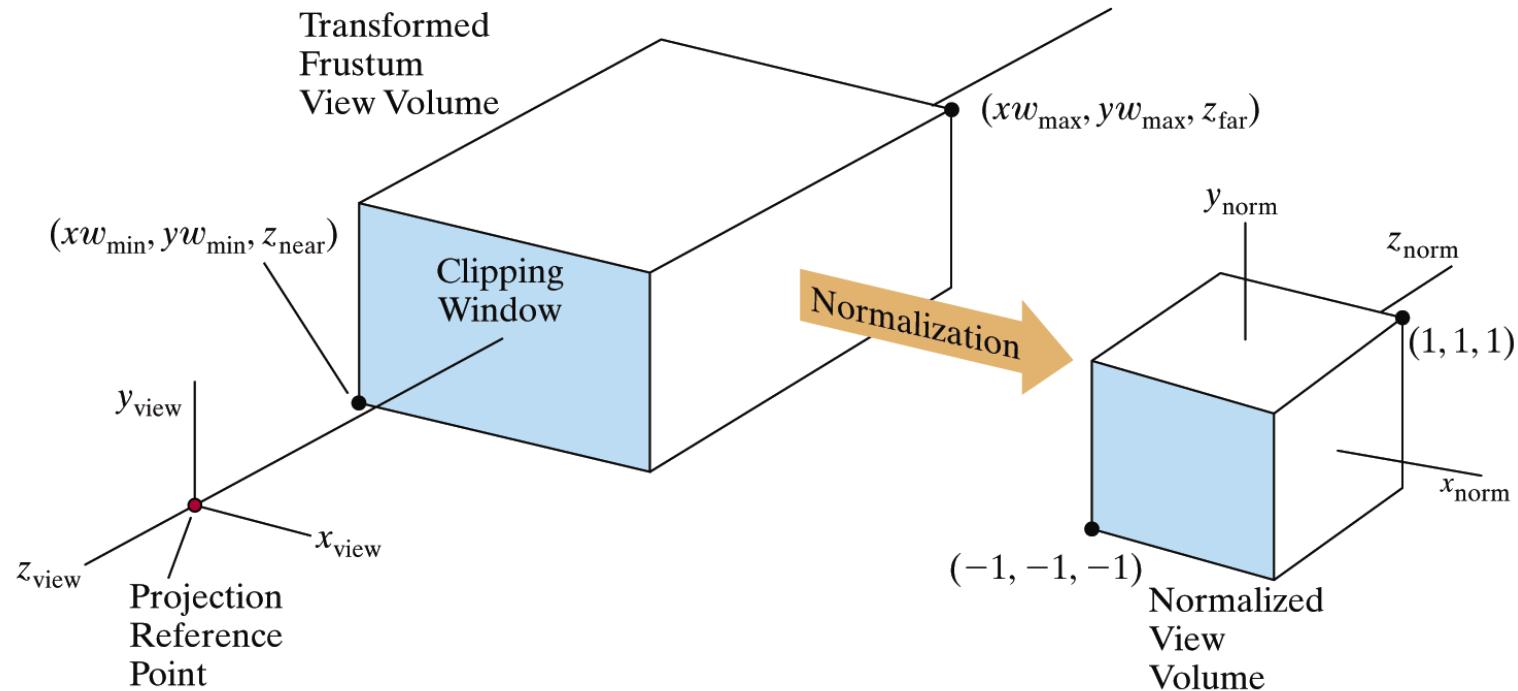
Perspective:



Normalization lets us clip against simple cube regardless of type of projection

Orthographic Transform

- ❑ The parallelepiped view volume is squeezed (scaled) around position (0, 0, 0) and the z axis is reversed

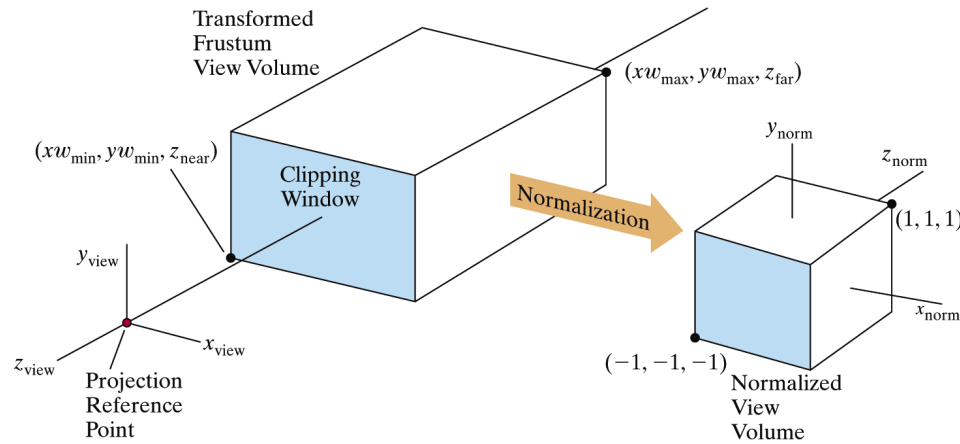


Orthographic Transform

$$T\left(-\frac{r+l}{2}, -\frac{t+b}{2}, \frac{|n|+|f|}{2}\right)$$

$$S\left(\frac{2}{r-l}, \frac{2}{t-b}, \frac{-2}{|f|-|n|}\right)$$

$$M_{orth,norm} = S.T$$



$$M_{orth,norm} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{|n|-|f|} & \frac{|n|+|f|}{|n|-|f|} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Here,

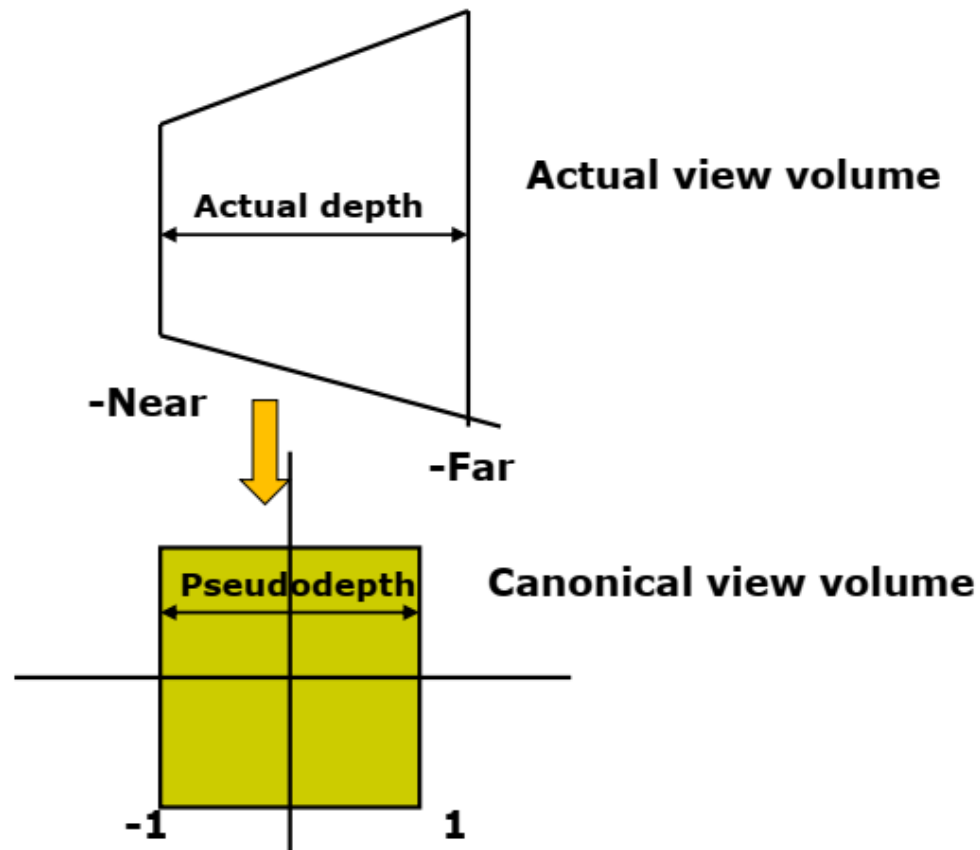
$$f < n < 0$$

Or

$$0 < |n| < |f|$$

Perspective Transform

- **Perspective transformation** maps actual z distance of perspective view volume to range $[-1 \text{ to } 1]$ (**Pseudodepth**) for canonical view volume

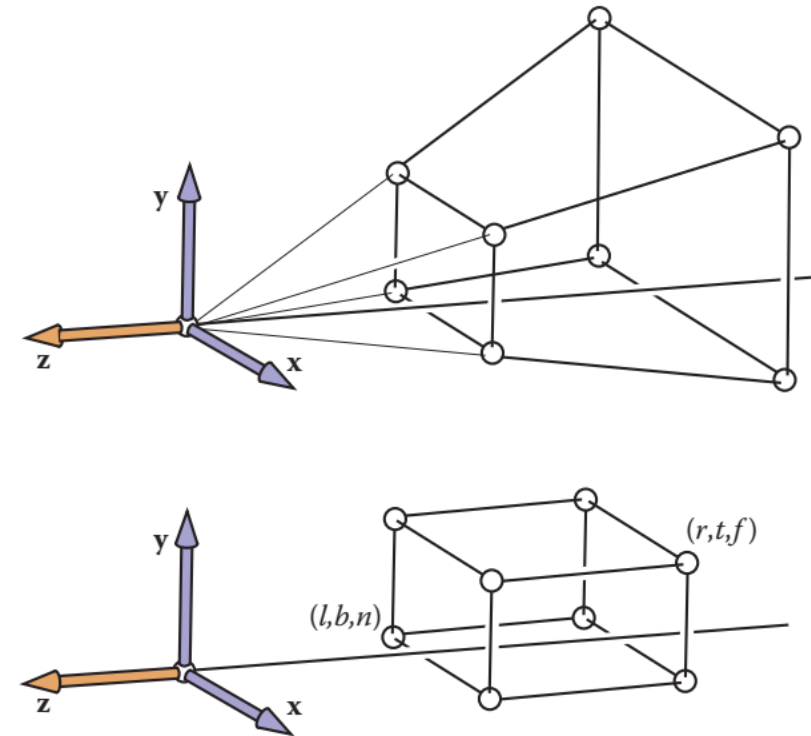
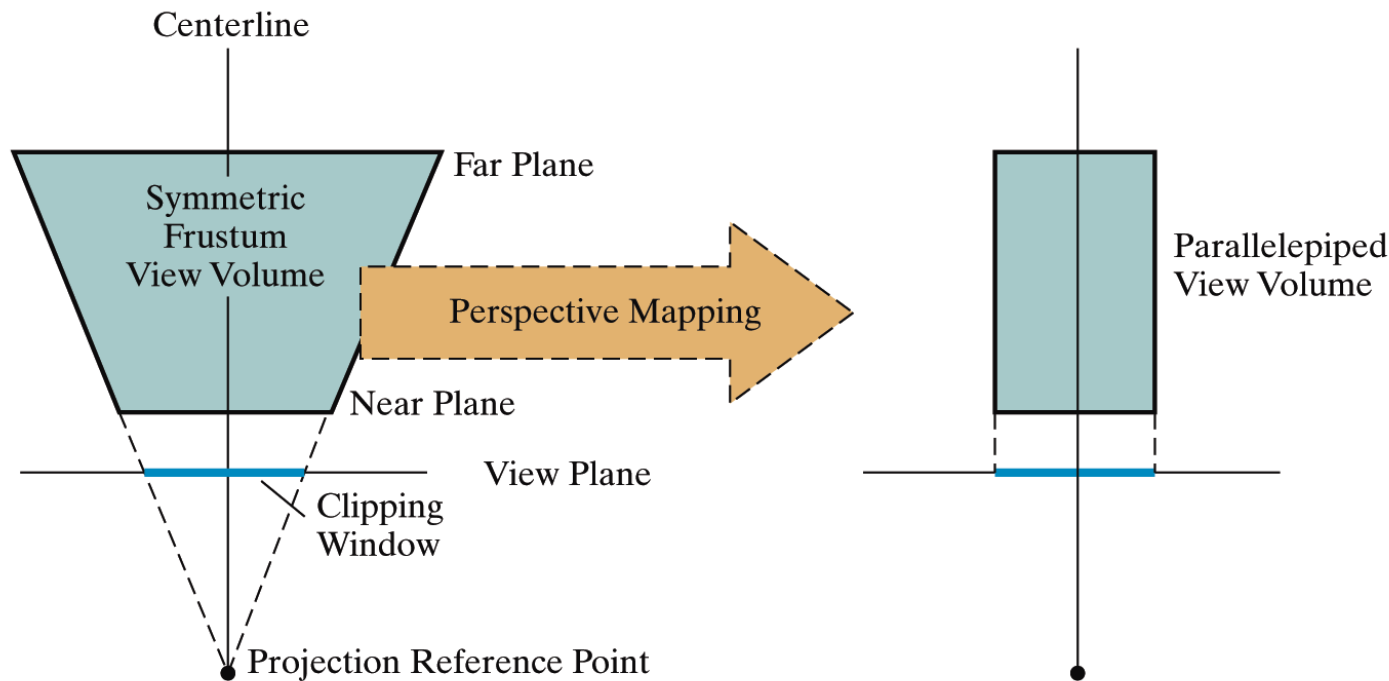


We want **perspective Transformation** and NOT classical projection!!

Set scaling z
 $\text{Pseudodepth} = az + b$
Next solve for a and b

Perspective Transform

- ❑ The frustum shaped viewing volume has been converted to a parallelepiped
 - remember we preserved all z coordinate depth information
 - How?
- ❑ And then parallelepiped volume is mapped to CVV as before



Perspective without Depth

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -\frac{1}{d} & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ -\frac{z}{d} \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \\ z \\ -\frac{z}{d} \end{pmatrix} \xrightarrow{\text{perspective division}} \begin{pmatrix} \frac{x}{-\left(z/d\right)} \\ \frac{y}{-\left(z/d\right)} \\ -d \\ 1 \end{pmatrix}$$

- The depth information is lost as the last two components are same
- But depth information of the projected points is essential for hidden surface removal and other purposes like blending, shading etc.

Perspective with Depth

❑ Sufficient to use **pseudodepth**

- what is a good choice? Can we use z as pseudodepth? (farther point has more negative z value)
- $z' = \alpha z + \beta$ which is a linear function before perspective division

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -\frac{1}{d} & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ \alpha z + \beta \\ -\frac{z}{d} \end{pmatrix} \xrightarrow{\text{perspective division}} \begin{pmatrix} \frac{x}{-(z/d)} \\ \frac{y}{-(z/d)} \\ \frac{d(\alpha z + \beta)}{-z} \\ 1 \end{pmatrix}$$

$$z' = -d \cdot \alpha - \frac{d \cdot \beta}{z}$$

For $\beta < 0$, z' is a monotonically increasing function of *depth*.

Perspective Transform - Frustum to Parallelepiped

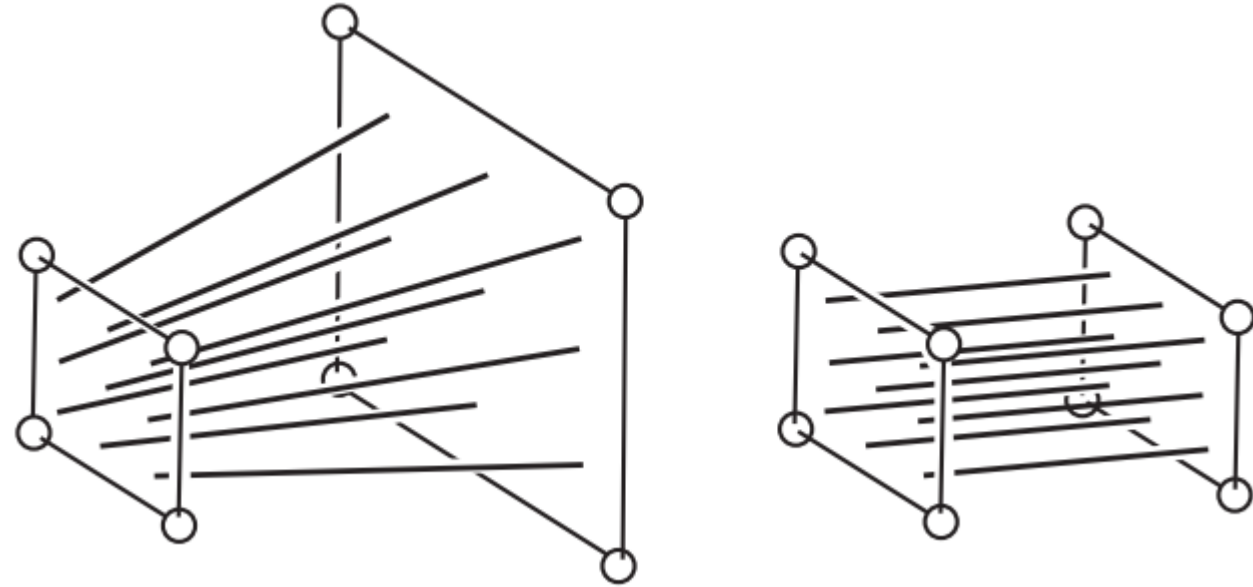
Rearrange the perspective transformation matrix

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -\frac{1}{d} & 0 \end{pmatrix} \cong \begin{pmatrix} |n| & 0 & 0 & 0 \\ 0 & |n| & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Perspective Transform - Frustum to Parallelepiped

Chose α, β , such a way that a point in near plane / far plane perspective projects on near / far plane

The perspective projection maps any line through the origin/eye to a line parallel to the z-axis and without moving the point on the line at $z = n$.



$$\begin{pmatrix} |n| & 0 & 0 & 0 \\ 0 & |n| & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} r & rf/n \\ t & tf/n \\ -|n| & -|f| \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} |n|r & rf \\ |n|t & tf \\ -\alpha|n| + \beta & -\alpha|f| + \beta \\ |n| & |f| \end{pmatrix} = \begin{pmatrix} r & r \\ t & t \\ \frac{-\alpha|n| + \beta}{|n|} & \frac{-\alpha|f| + \beta}{|f|} \\ 1 & 1 \end{pmatrix}$$

Perspective Transform - Frustum to Parallelepiped

Chose α, β , such a way that a point in near plane / far plane perspective projects on near / far plane

$$\begin{pmatrix} |n| & 0 & 0 & 0 \\ 0 & |n| & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} r & rf/n \\ t & tf/n \\ -|n| & -|f| \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} |n|r & rf \\ |n|t & tf \\ -\alpha|n| + \beta & -\alpha|f| + \beta \\ |n| & |f| \end{pmatrix} = \begin{pmatrix} r & r \\ t & t \\ \frac{-\alpha|n| + \beta}{|n|} & \frac{-\alpha|f| + \beta}{|f|} \\ 1 & 1 \end{pmatrix}$$

Apply Boundary Condition

$$\frac{-\alpha|n| + \beta}{|n|} = -|n|$$

$$\Rightarrow -\alpha|n| + \beta = -|n|^2$$

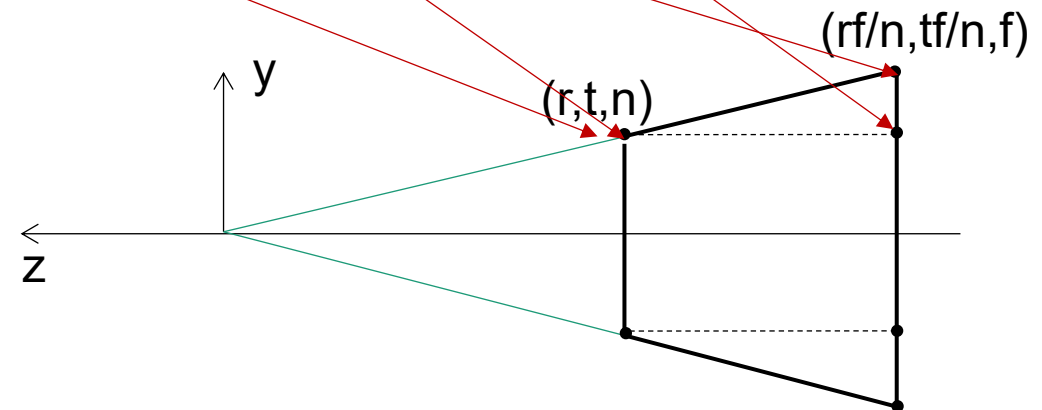
Similarly

$$-\alpha|f| + \beta = -|f|^2$$

Solving

$$\alpha = |n| + |f|$$

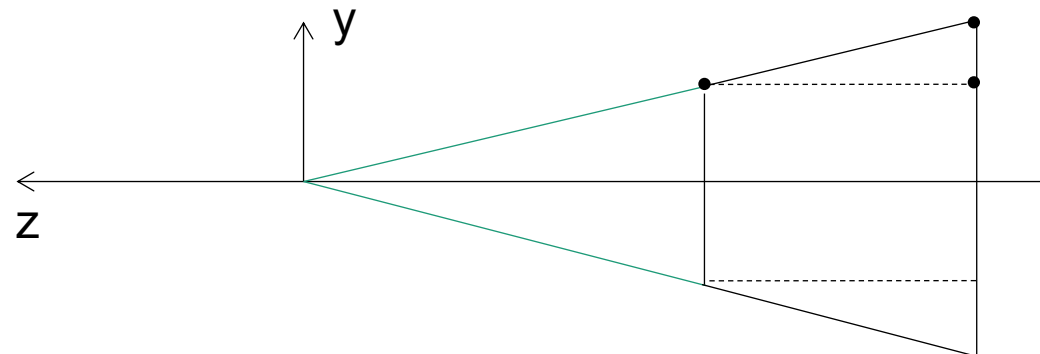
$$\beta = |n||f|$$



Perspective Transform - Parallelepiped to CVV

$$M_{per,norm} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{|n|-|f|} & \frac{|n|+|f|}{|n|-|f|} \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} |n| & 0 & 0 & 0 \\ 0 & |n| & 0 & 0 \\ 0 & 0 & |n|+|f| & |n||f| \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

$$= \begin{bmatrix} \frac{2|n|}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2|n|}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{|n|+|f|}{|n|-|f|} & \frac{2|f||n|}{|n|-|f|} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$



Perspective transform

If window is symmetric, $r = -l$ and $t = -b$, then

$$M_{per,norm} = \begin{bmatrix} \frac{2|n|}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2|n|}{t-b} & 0 & 0 \\ 0 & 0 & \frac{|n|+|f|}{|n|-|f|} & \frac{2|f||n|}{|n|-|f|} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

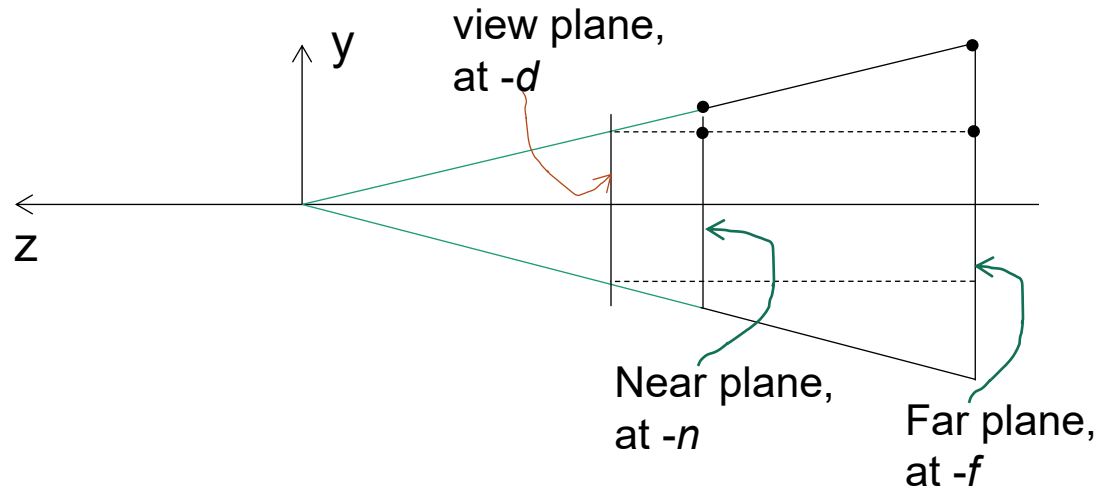
symmetric

$$M_{per,norm} = \begin{bmatrix} \frac{2|n|}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2|n|}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{|n|+|f|}{|n|-|f|} & \frac{2|f||n|}{|n|-|f|} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Asymmetric / general

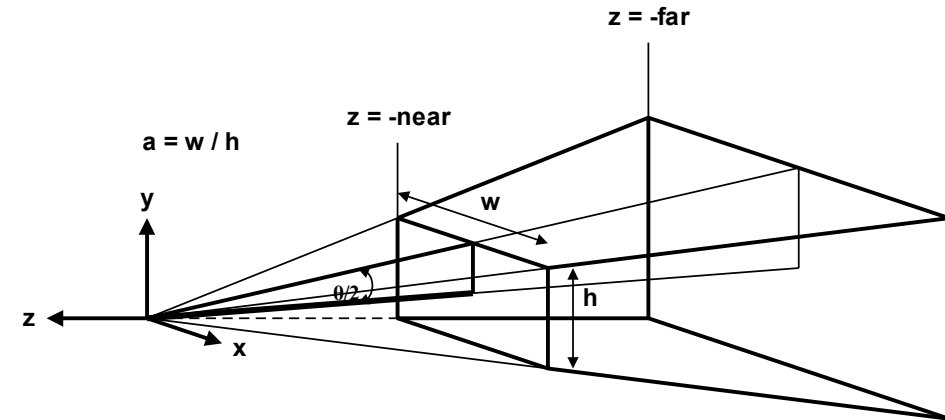
Perspective Transform - Parallelepiped to CVV

What if, view plane $\neq$ near plane



Perspective transform

$$M_{per,norm} = \begin{bmatrix} \frac{2|n|}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2|n|}{t-b} & 0 & 0 \\ 0 & 0 & \frac{|n|+|f|}{|n|-|f|} & \frac{2|f||n|}{|n|-|f|} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$



Window is symmetric, but defined by aspect ratio a , fovy θ , then

$$M_{per,norm} = \begin{bmatrix} \frac{c}{a} & 0 & 0 & 0 \\ 0 & c & 0 & 0 \\ 0 & 0 & \frac{|n|+|f|}{|n|-|f|} & \frac{2|f||n|}{|n|-|f|} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

$$c = \cot\left(\frac{\theta}{2}\right) = \frac{|n|}{t} \Rightarrow t = \frac{|n|}{c}; \quad b = -\frac{|n|}{c}$$

$$a = \frac{w}{h} = \frac{r}{t} \Rightarrow r = at = \frac{a|n|}{c}; \quad l = -\frac{a|n|}{c}$$

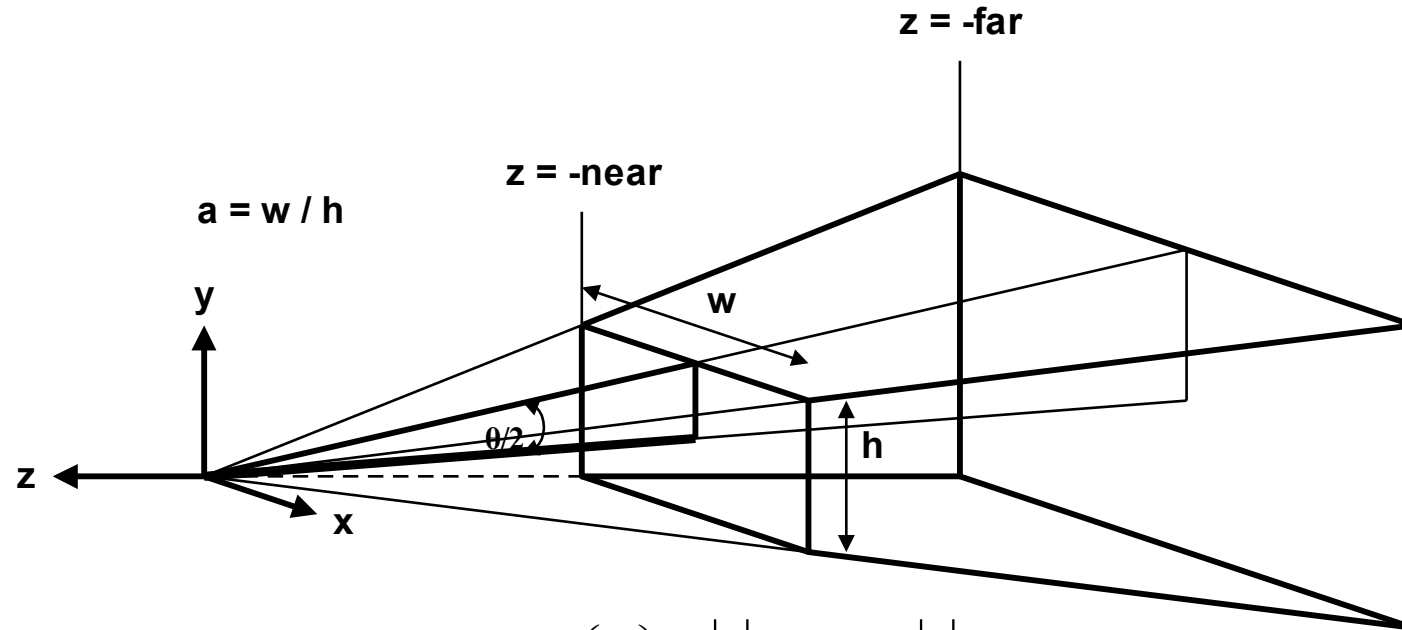
Perspective Transformation Matrix

Another way

The matrix to perform ***perspective transformation***:

$$\begin{pmatrix} \gamma & 0 & 0 & 0 \\ 0 & \delta & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} \gamma \cdot x \\ \delta \cdot y \\ \alpha \cdot z + \beta \\ -z \end{pmatrix}$$
$$= \begin{pmatrix} \gamma \cdot x / (-z) \\ \delta \cdot y / (-z) \\ (\alpha \cdot z + \beta) / (-z) \\ 1 \end{pmatrix}$$

Perspective Transformation Matrix - Symmetric

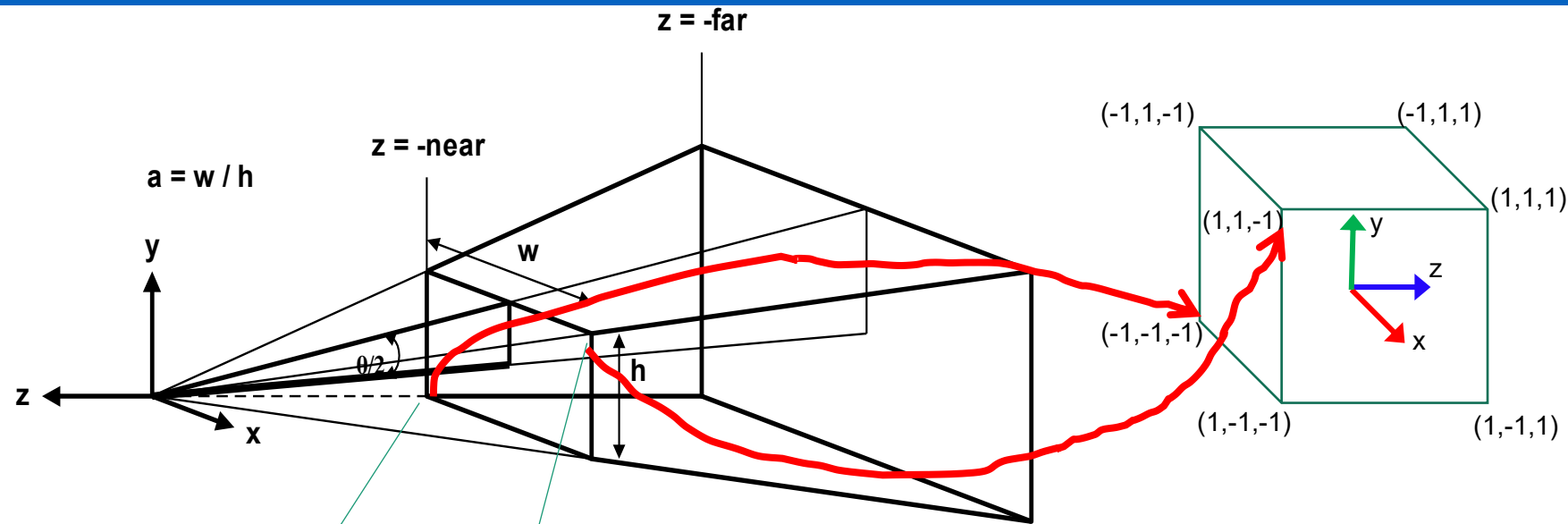


$$c = \cot\left(\frac{\theta}{2}\right) = \frac{|z|}{|y|} \Rightarrow |y| = \frac{|z|}{c}$$

$$a = \frac{w}{h} = \frac{|x|}{|y|} \Rightarrow |x| = a|y| = \frac{a|z|}{c}$$

$$\therefore (x, y, z) \Rightarrow \left(\pm \frac{a|z|}{c}, \pm \frac{|z|}{c}, z \right)$$

Perspective Transformation Matrix - Symmetric



$$\left(-\frac{a|n|}{c}, -\frac{|n|}{c}, -|n| \right) \rightarrow (-1, -1, -1)$$

$$\left(-\frac{a|n|}{c}, \frac{|n|}{c}, -|n| \right) \rightarrow (-1, 1, -1)$$

$$\left(\frac{a|n|}{c}, -\frac{|n|}{c}, -|n| \right) \rightarrow (1, -1, -1)$$

$$\left(\frac{a|n|}{c}, \frac{|n|}{c}, -|n| \right) \rightarrow (1, 1, -1)$$

$$\left(-\frac{af}{c}, -\frac{f}{c}, -f \right) \rightarrow (-1, -1, 1)$$

$$\left(-\frac{af}{c}, \frac{f}{c}, -f \right) \rightarrow (-1, 1, 1)$$

$$\left(\frac{af}{c}, -\frac{f}{c}, -f \right) \rightarrow (1, -1, 1)$$

$$\left(\frac{af}{c}, \frac{f}{c}, -f \right) \rightarrow (1, 1, 1)$$

all f are $|f|$

Perspective Transformation Matrix - Symmetric

$$\left(-\frac{an}{c}, -\frac{n}{c}, -n\right) \rightarrow (-1, -1, -1)$$

$$\begin{pmatrix} \gamma & 0 & 0 & 0 \\ 0 & \delta & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix} \cdot \begin{pmatrix} -\frac{an}{c} \\ -\frac{n}{c} \\ -n \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ -1 \\ 1 \end{pmatrix}$$

$$\Leftrightarrow \begin{pmatrix} -\frac{\gamma an}{c} \\ -\frac{\delta n}{c} \\ -\alpha n + \beta \\ n \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ -1 \\ 1 \end{pmatrix}$$

$$\Leftrightarrow \begin{pmatrix} -\frac{\gamma a}{c} \\ -\frac{\delta}{c} \\ -\alpha n + \beta \\ n \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ -1 \\ 1 \end{pmatrix}$$

$$\Rightarrow \gamma = \frac{c}{a}$$

$$\Rightarrow \delta = c$$

$$\Rightarrow -\alpha n + \beta = -n$$

Perspective Transformation Matrix - Symmetric

$$\left(-\frac{af}{c}, -\frac{f}{c}, -f\right) \rightarrow (-1, -1, 1)$$

$$\begin{pmatrix} \gamma & 0 & 0 & 0 \\ 0 & \delta & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{pmatrix} \cdot \begin{pmatrix} -\frac{af}{c} \\ -\frac{f}{c} \\ c \\ -f \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ 1 \\ 1 \end{pmatrix}$$

$$\Leftrightarrow \begin{pmatrix} -\frac{\gamma af}{c} \\ -\frac{\delta f}{c} \\ -\alpha f + \beta \\ f \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ -1 \\ 1 \end{pmatrix}$$

$$\Leftrightarrow \begin{pmatrix} -\frac{\gamma a}{c} \\ -\frac{\delta}{c} \\ -\alpha f + \beta \\ f \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ 1 \\ 1 \end{pmatrix}$$

$$\Rightarrow -\alpha f + \beta = f$$

$$\because -\alpha n + \beta = -n$$

$$\therefore \alpha = \frac{f + n}{n - f}$$

$$\therefore \beta = \frac{2fn}{n - f}$$

Perspective Transformation Matrix - Symmetric

The matrix to perform *perspective transformation*:

$$\begin{pmatrix} \frac{c}{a} & 0 & 0 & 0 \\ 0 & c & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Perspective projection

- ❑ Using the Frustum parameters, and considering it a symmetric one
- ❑ Final matrix

$$\begin{bmatrix} \frac{2n}{r-l} & 0 & 0 & 0 \\ 0 & \frac{2n}{t-b} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

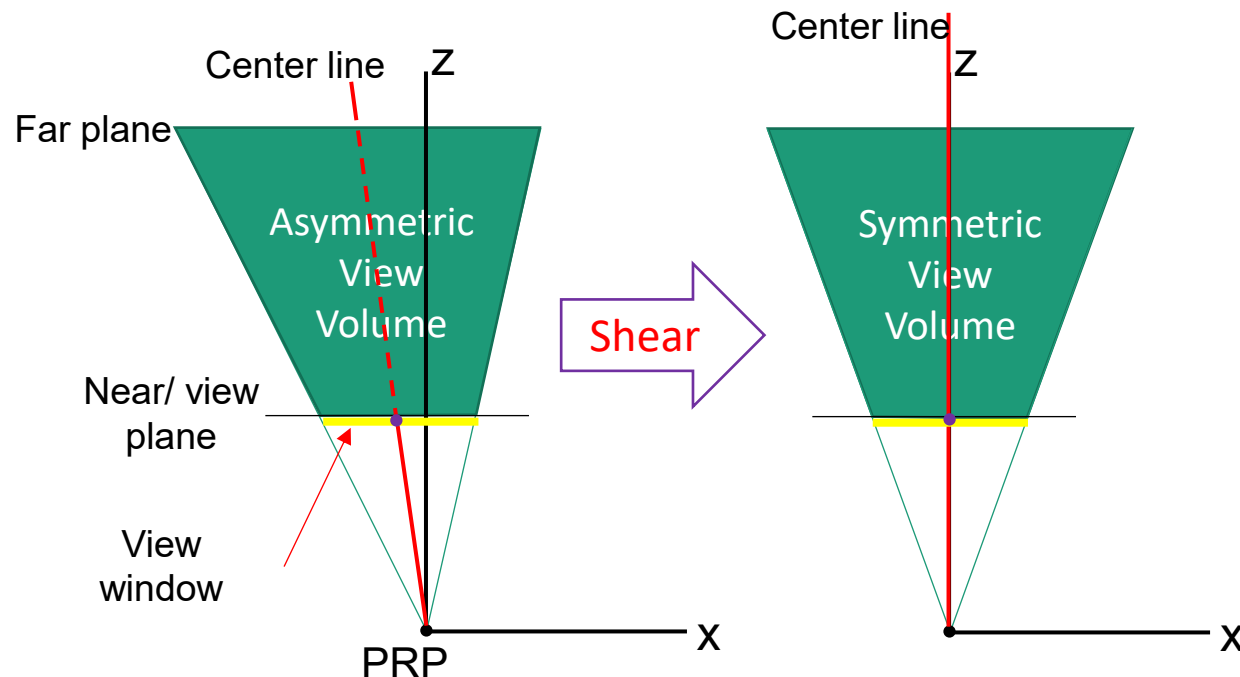
$$\begin{bmatrix} \frac{-2n}{r-l} & 0 & 0 & 0 \\ 0 & \frac{-2n}{t-b} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{-2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Baker Book

This is also true, here all n and f are absolute values

Perspective projection

- ❑ The view frustum may not be centered along the view vector, so it needs one more step, first shear the window.



$$\begin{pmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Shear matrix

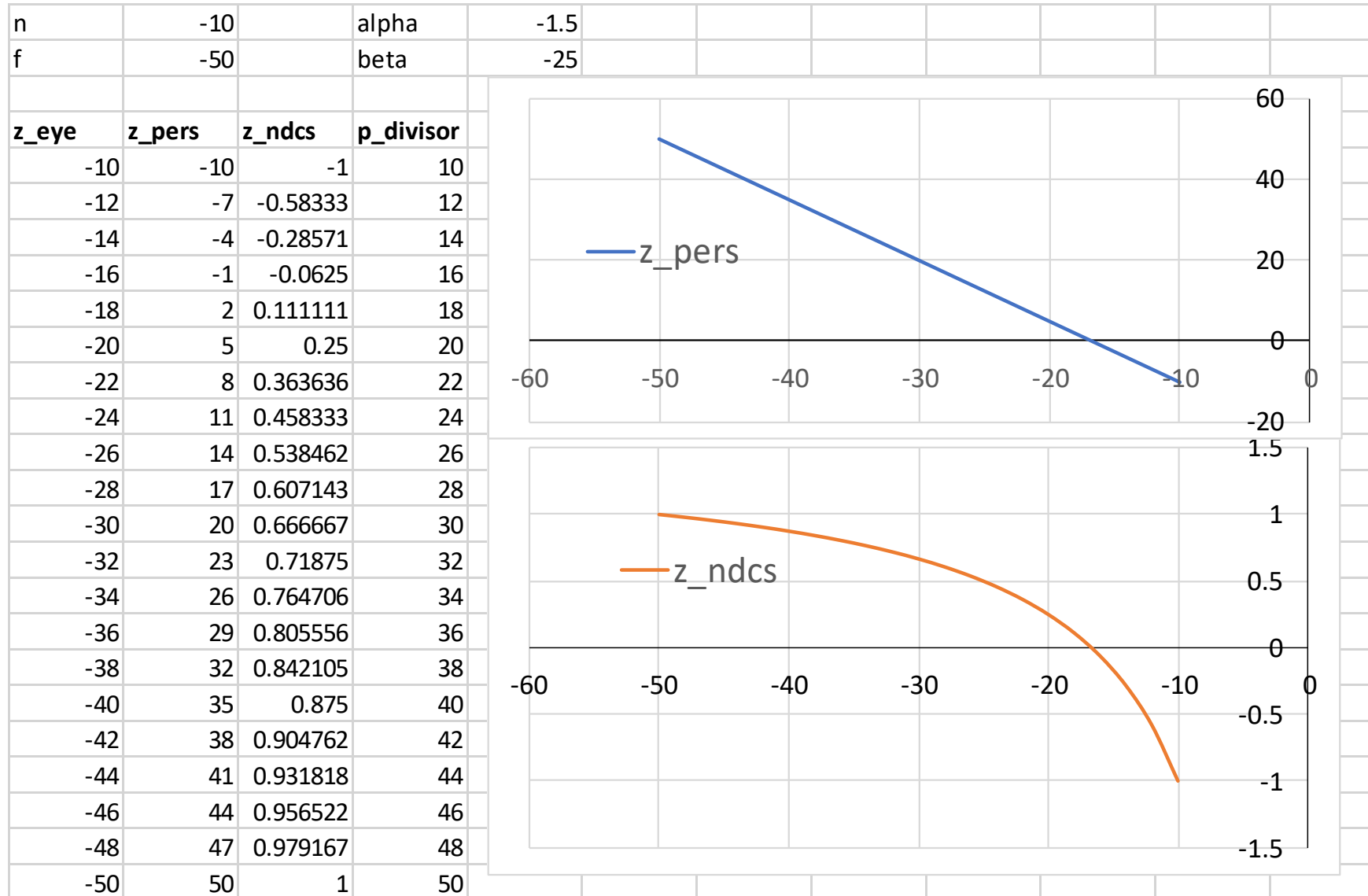
Shear

$$\begin{aligned} x' &= x + az \\ y' &= y + bz \end{aligned}$$

$$((r+l)/2, (t+b)/2, -n) \rightarrow (0, 0, -n)$$

$$\begin{aligned} a &= (r+l)/2n \\ b &= (t+b)/2n \end{aligned}$$

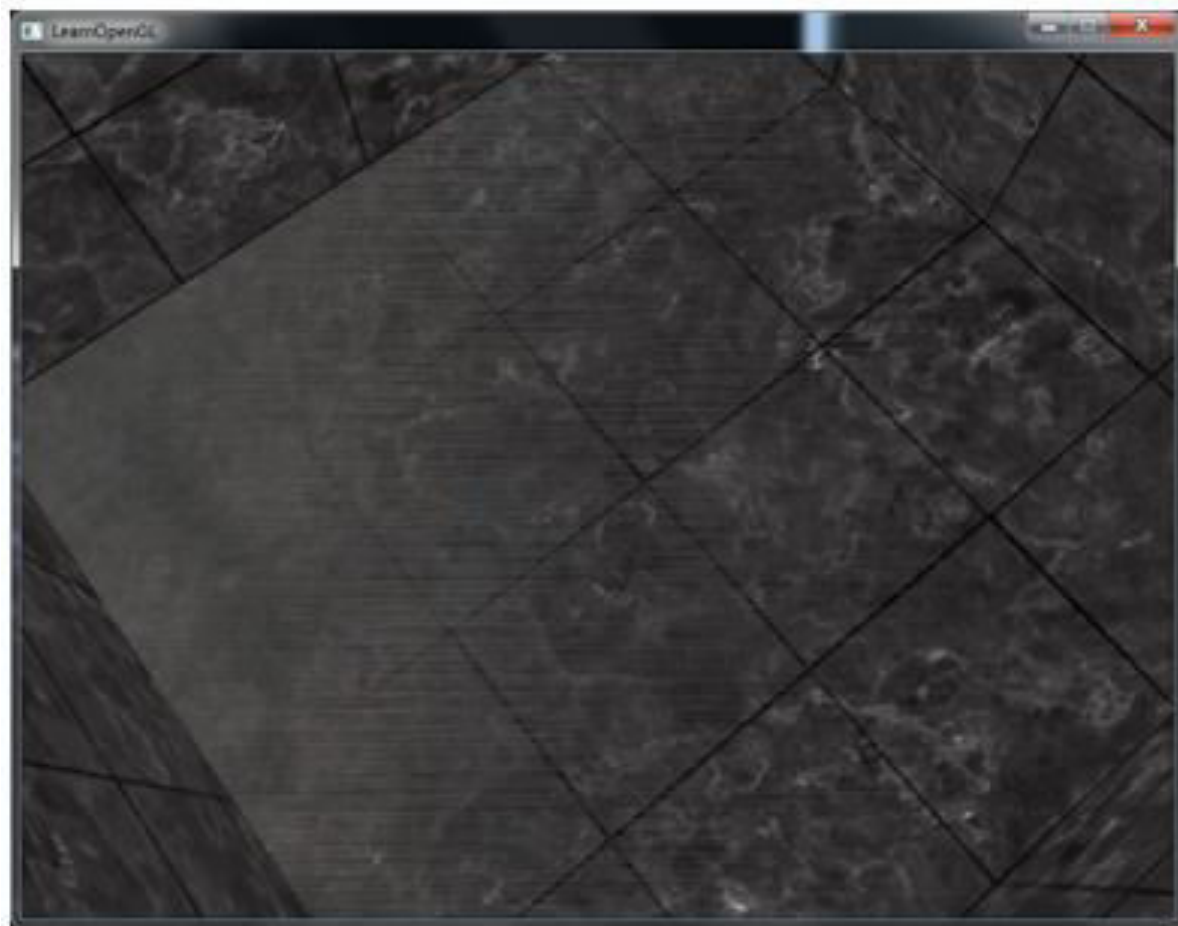
Perspective projection



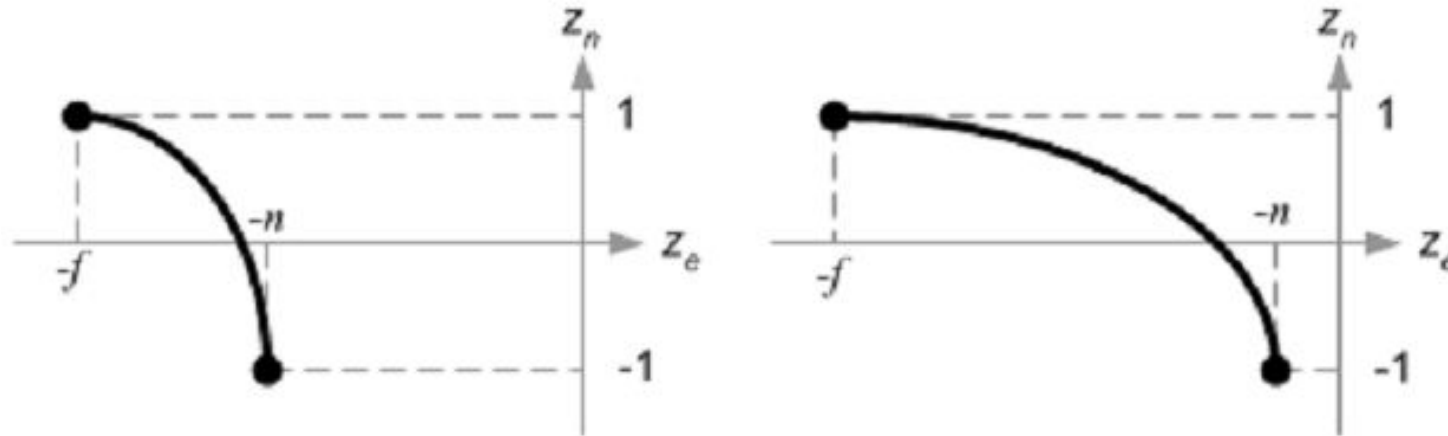
Z-Fighting Problem

- ❑ A common visual artifact might occur when two planes are closely aligned to each
 - The depth buffer does not have enough precision to figure out which one is in front of the other.

$$z' = -d \cdot \alpha - \frac{d \cdot \beta}{z}$$



Z-Fighting Problem



Comparison of Depth Buffer Precisions

$$z' = -d \cdot \alpha - \frac{d \cdot \beta}{z}$$

❑ N=1, F=100, z' ranges from -1 to +1

❑ Prevention

- *set the near plane as far as possible*
- *never place objects too close to each other in a way that some of them closely overlap*
- *use a higher precision depth buffer*

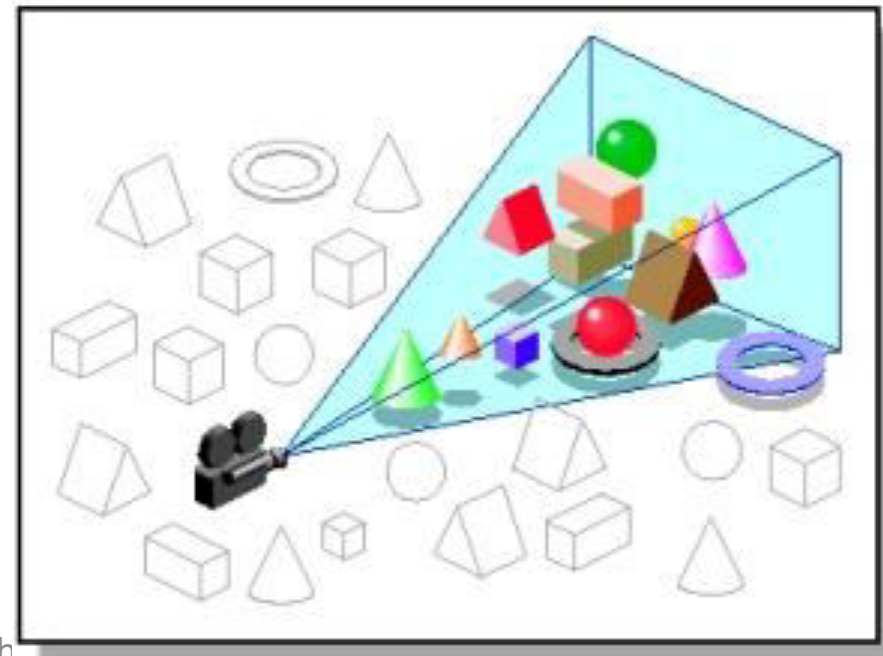
z	z'
97	0.9993752
98	0.9995877
99	0.9997959

WCS, VCS, NDCS and projection

- ❑ You are given info about
 - P_0, P, V in WCS
 - $d, f, a=h/w, \text{fovy}$
- ❑ Let the apex of the pyramidal object in the scene $(1,2,1)_{\text{WCS}}$
- ❑ Find the NDCS value of the apex

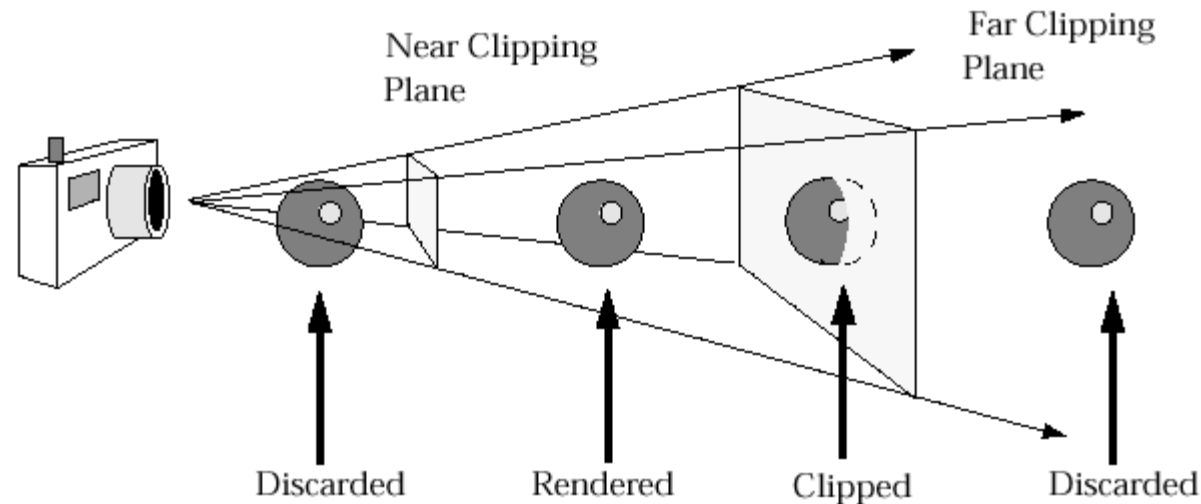
3-D Clipping

- ❑ Clipping removes objects that will not be visible
- ❑ The point of this is to remove computational effort
- ❑ Achieved in two basic steps
 - Discard objects that can't be viewed
 - i.e. objects that are behind the camera, outside the field of view, or too far away
 - Clip objects that intersect with any clipping plane



3-D Clipping

- ❑ Discarding objects that cannot possibly be seen involves comparing an objects bounding box/sphere against the dimensions of the view volume
 - Can be done before or after projection



When Do We Clip?

- ❑ We perform clipping after normalisation are done
- ❑ So, we have the following:

$$\begin{bmatrix} x_h \\ y_h \\ z_h \\ h \end{bmatrix} = M_{Norm} \cdot \begin{bmatrix} x_e \\ y_e \\ z_e \\ w_e \end{bmatrix}$$

- ❑ We apply all clipping to these homogeneous coordinates

How to Clip

- Because we have a normalised clipping volume we can test for these regions as follows:

$$-1 \leq \frac{x_h}{h} \leq 1 \quad -1 \leq \frac{y_h}{h} \leq 1 \quad -1 \leq \frac{z_h}{h} \leq 1$$

- Rearranging these we get:

$$-h \leq x_h \leq h \quad -h \leq y_h \leq h \quad -h \leq z_h \leq h \quad \text{if } h > 0$$

$$h \leq x_h \leq -h \quad h \leq y_h \leq -h \quad h \leq z_h \leq -h \quad \text{if } h < 0$$

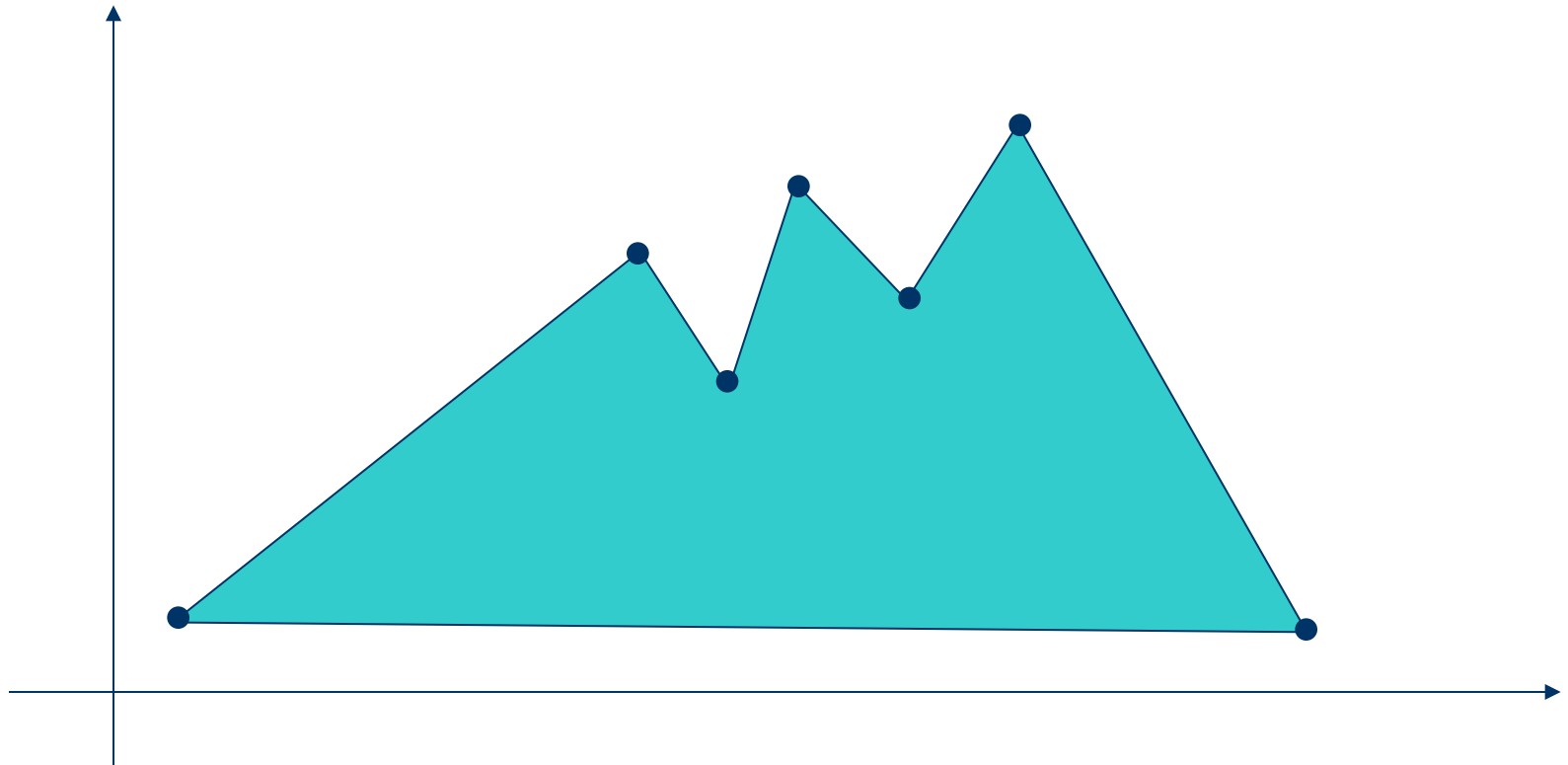
- Objects that are partially within the viewing volume need to be clipped – just like the 2D case

Clipping

Dr. Sk. Md. Masudul Ahsan
Professor
Dept. of CSE, KUET

Windowing

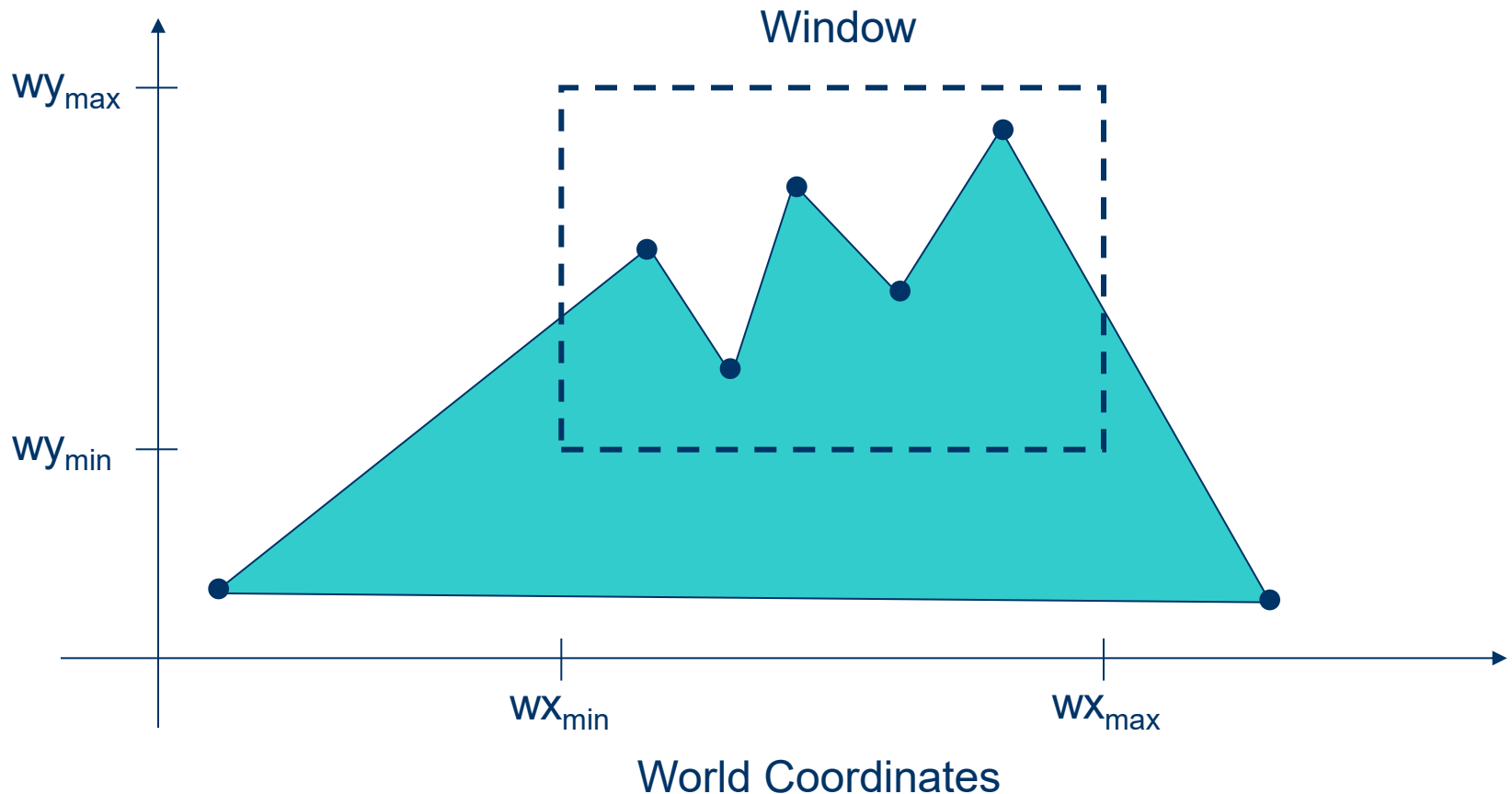
- A scene is made up of a collection of objects specified in world coordinates



World Coordinates

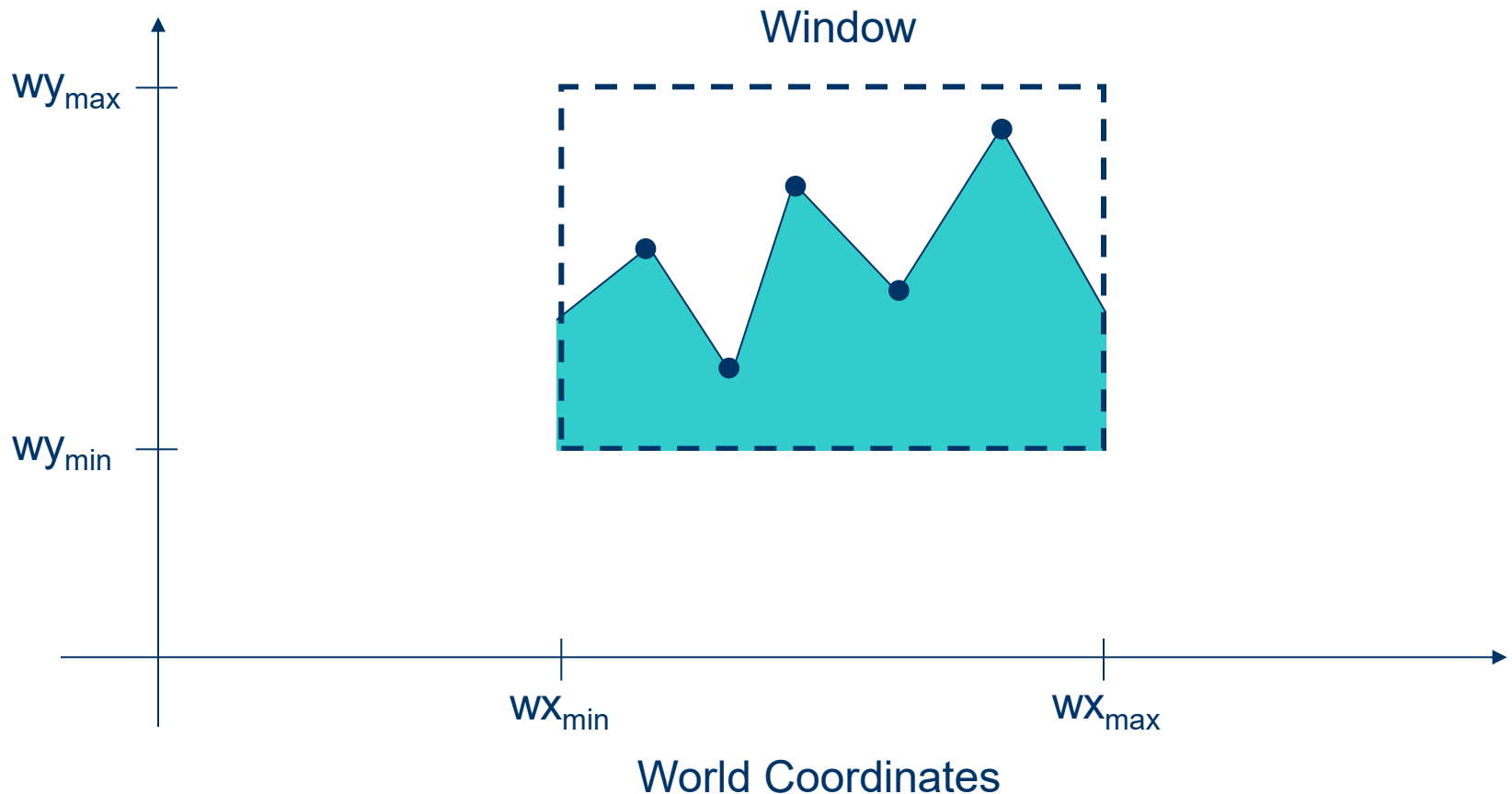
Windowing

- When we display a scene only those objects within a particular window are displayed



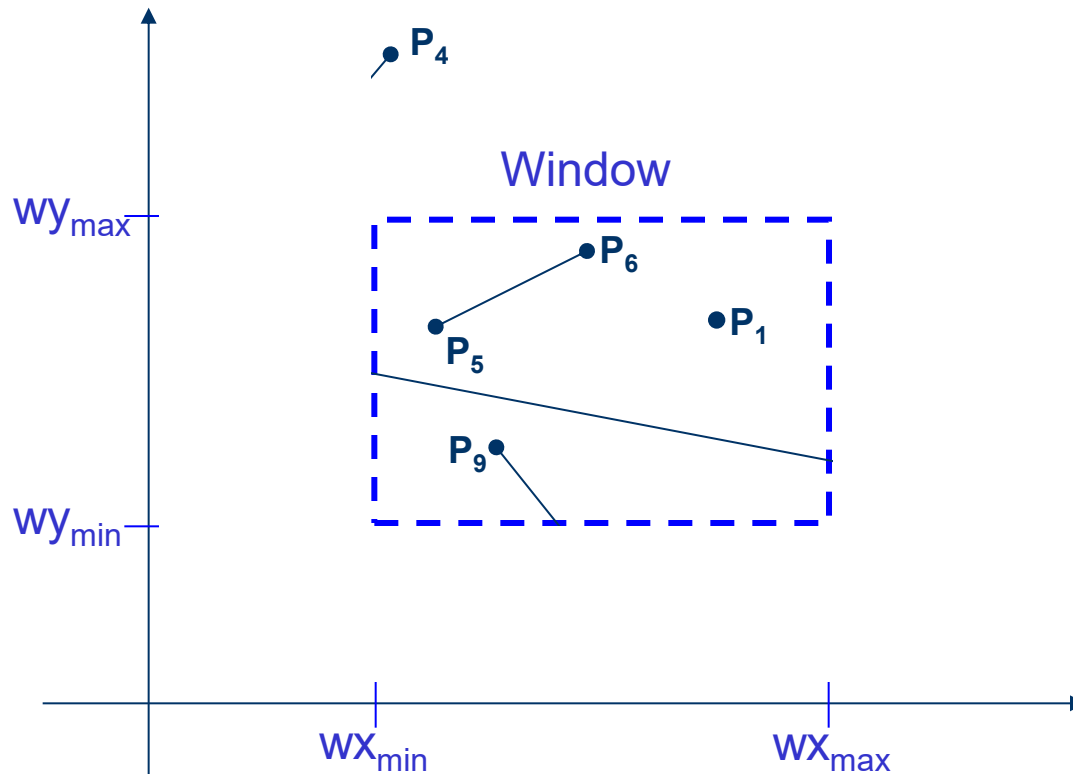
Windowing

- Because drawing things to a display takes time we *clip* everything outside the window



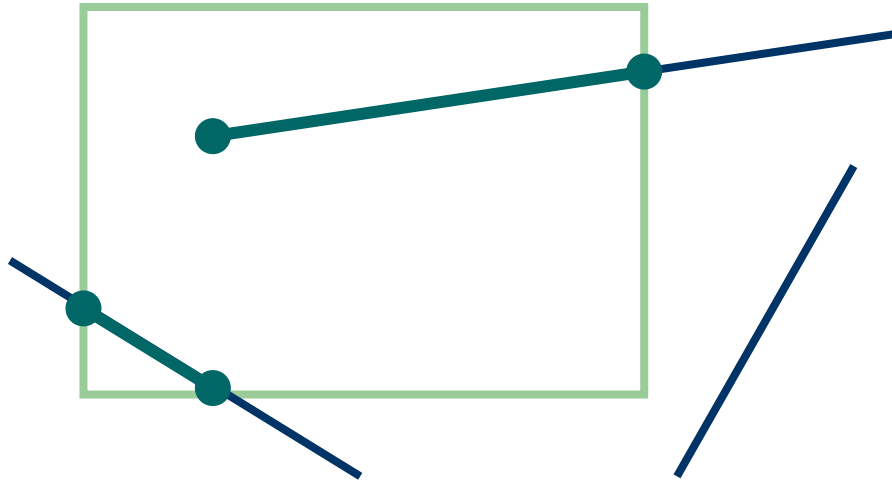
Clipping

- For the image below consider which lines and points should be kept and which ones should be clipped



Clipping

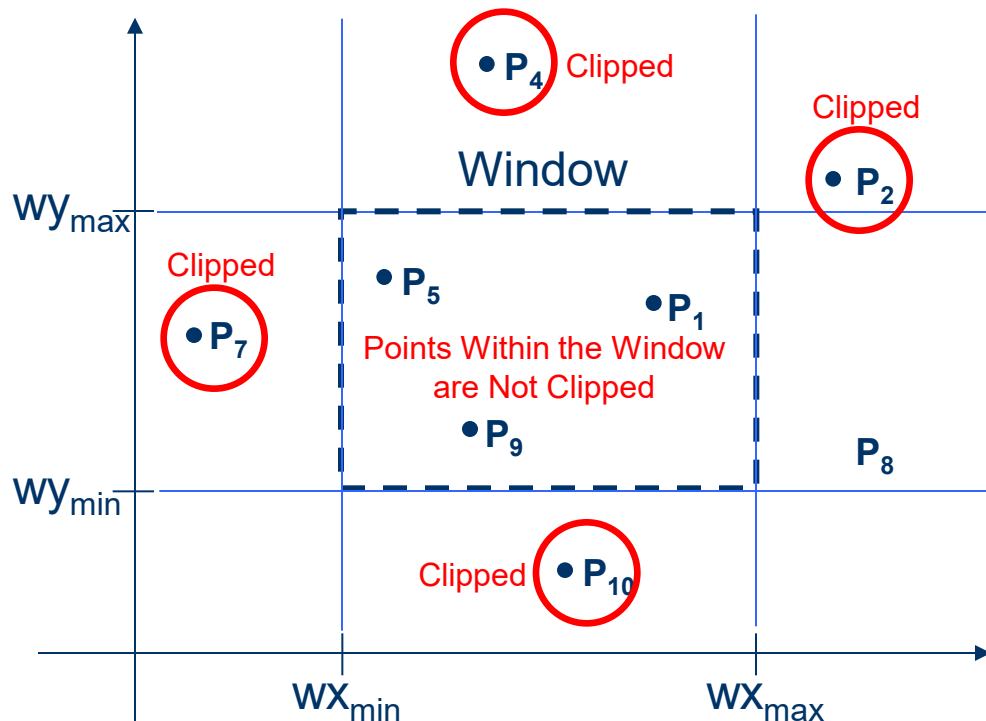
- Analytically calculating the portions of primitives (lines, triangles, polygons) within the clipping window



- Why
 - Bad idea to rasterize outside of framebuffer bounds
 - Also, don't waste time scan converting pixels outside window

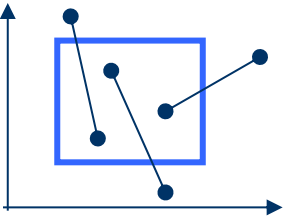
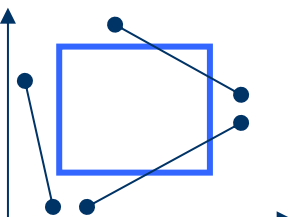
Point Clipping

- Easy - a point (x,y) is not clipped if:
- $wx_{min} \leq x \leq wx_{max}$ AND $wy_{min} \leq y \leq wy_{max}$
- otherwise it is clipped



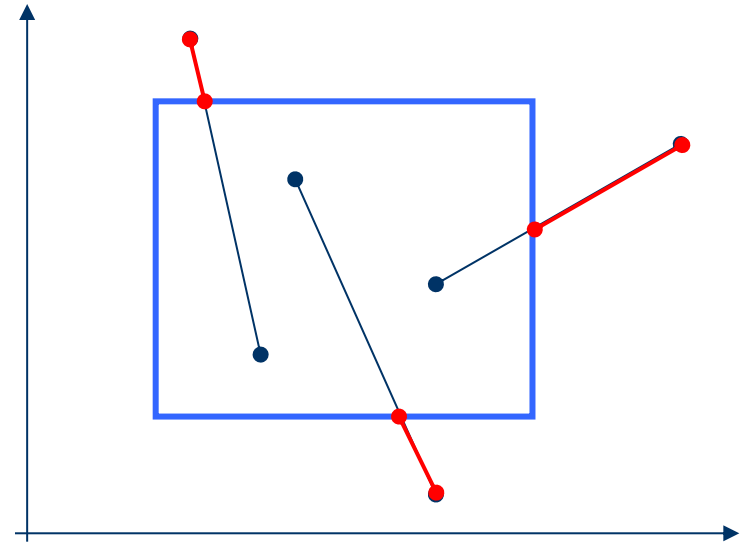
Line Clipping

- Harder - examine the end-points of each line to see if they are in the window or not

Situation	Solution	Example
Both end-points inside the window	Don't clip	
One end-point inside the window, one outside	Must clip	
Both end-points outside the window	Don't know!	

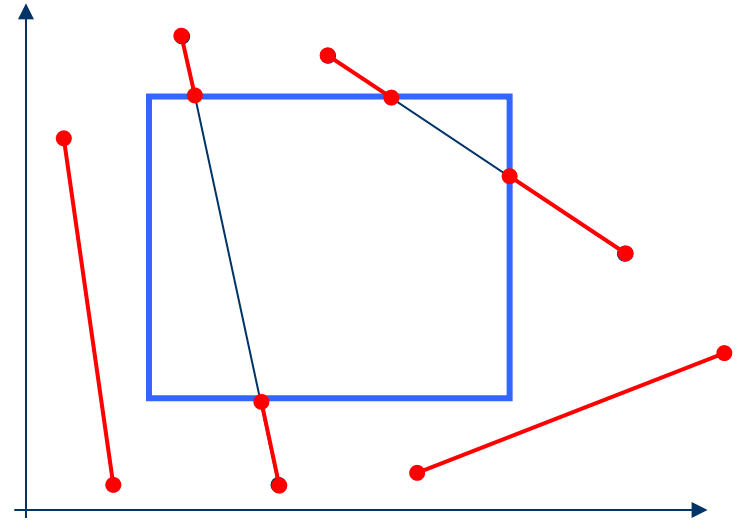
Brute Force Line Clipping

- Brute force line clipping can be performed as follows:
 - Don't clip lines with both end-points within the window
 - For lines with one end-point inside the window and one end-point outside, calculate the intersection point (using the equation of the line) and clip from this point out



Brute Force Line Clipping (cont...)

- For lines with both end-points outside the window test the line for intersection with all of the window boundaries, and clip appropriately



However, calculating line intersections is computationally expensive

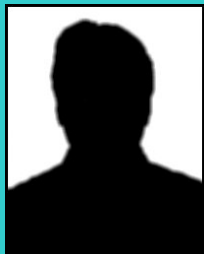
Because a scene can contain so many lines, the brute force approach to clipping is much too slow

Cohen-Sutherland Clipping Algorithm

- An efficient line clipping algorithm
- The key advantage of the algorithm is that it vastly reduces the number of line intersections that must be calculated



Dr. Ivan E. Sutherland co-developed the Cohen-Sutherland clipping algorithm. Sutherland is a graphics giant and includes amongst his achievements the invention of the head mounted display.

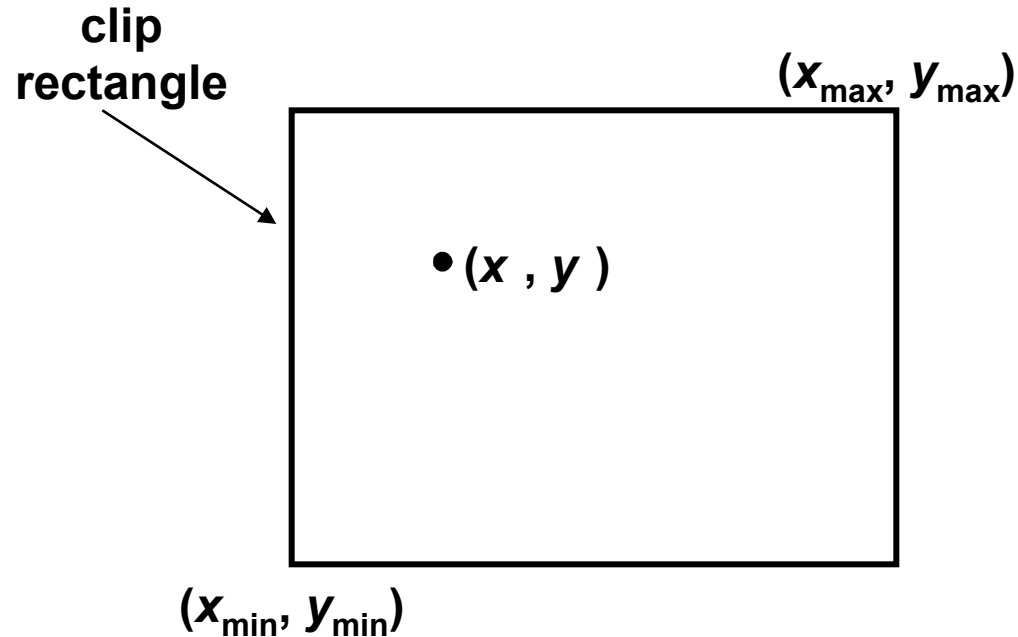


Cohen is something of a mystery – can anybody find out who he was?

2D Clipping algorithms

- **Point Clipping**
- **Cohen-Sutherland Algorithm (Line)**
- **Liang-Barsky algorithm (Line)**
- **Sutherland-Hodgeman Algorithm (Polygon)**

Point Clipping



For a point (x, y) to be inside the clip rectangle:

$$x_{\min} \leq x \leq x_{\max}$$

$$y_{\min} \leq y \leq y_{\max}$$

Line Clipping

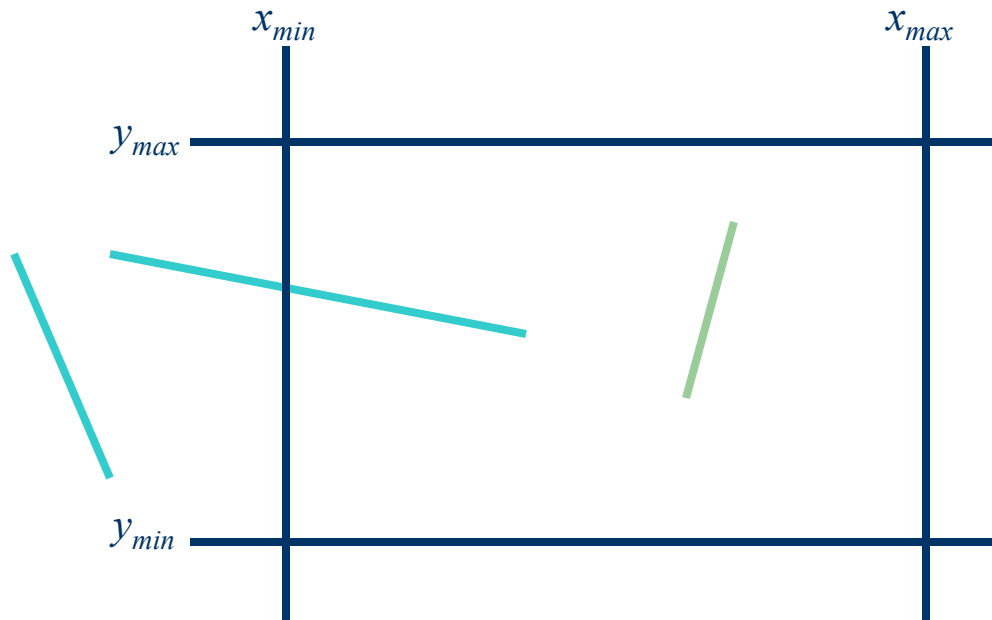
- The naïve approach to clipping lines:

```
for each line segment
  for each edge of viewport
    find intersection points
    pick "nearest" point
  if anything is left, draw it
```

- *What do we mean by “nearest”?*
- *How can we optimize this?*

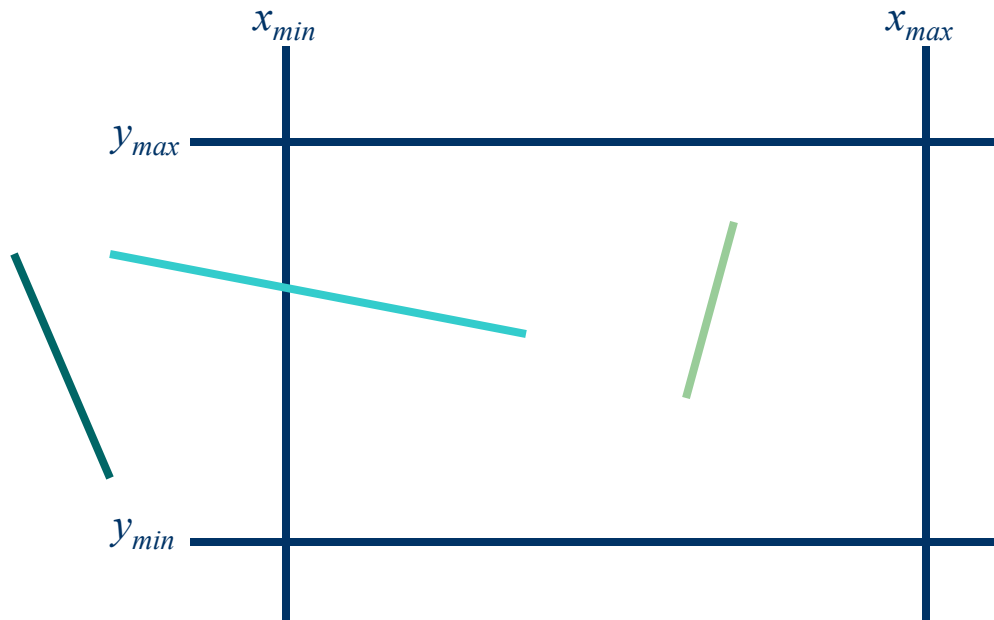
Trivial Accepts

- Big optimization: trivial accept/rejects
- *How can we quickly determine whether a line segment is entirely inside the viewport?*
- A: test both endpoints.



Trivial Rejects

- *How can we know a line is outside viewport?*
- A: if both endpoints on wrong side of same edge, can trivially reject line



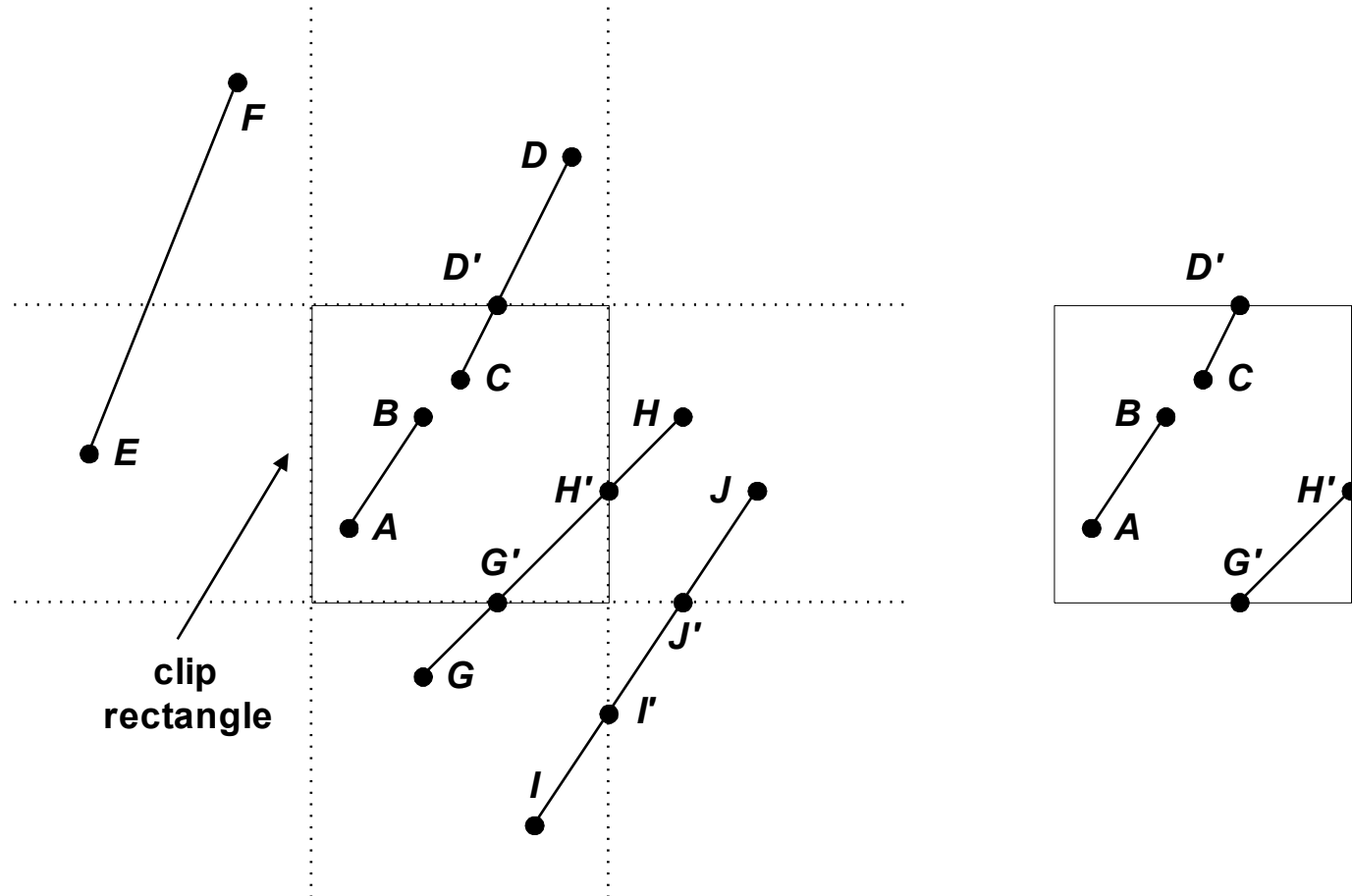
Line Clipping

- Cases for clipping lines
 - **Visible::** both end points are within the window
 - **Not visible::** both end points are outside the window
 - (x_1, y_1) and (x_2, y_2) defines the line
 - Any of these are true

$$\begin{array}{ll} x_1, x_2 > x_{\max} & x_1, x_2 < x_{\min} \\ y_1, y_2 > y_{\max} & y_1, y_2 < y_{\min} \end{array}$$

- **Clipping candidate::** neither category 1 nor 2

Line Clipping



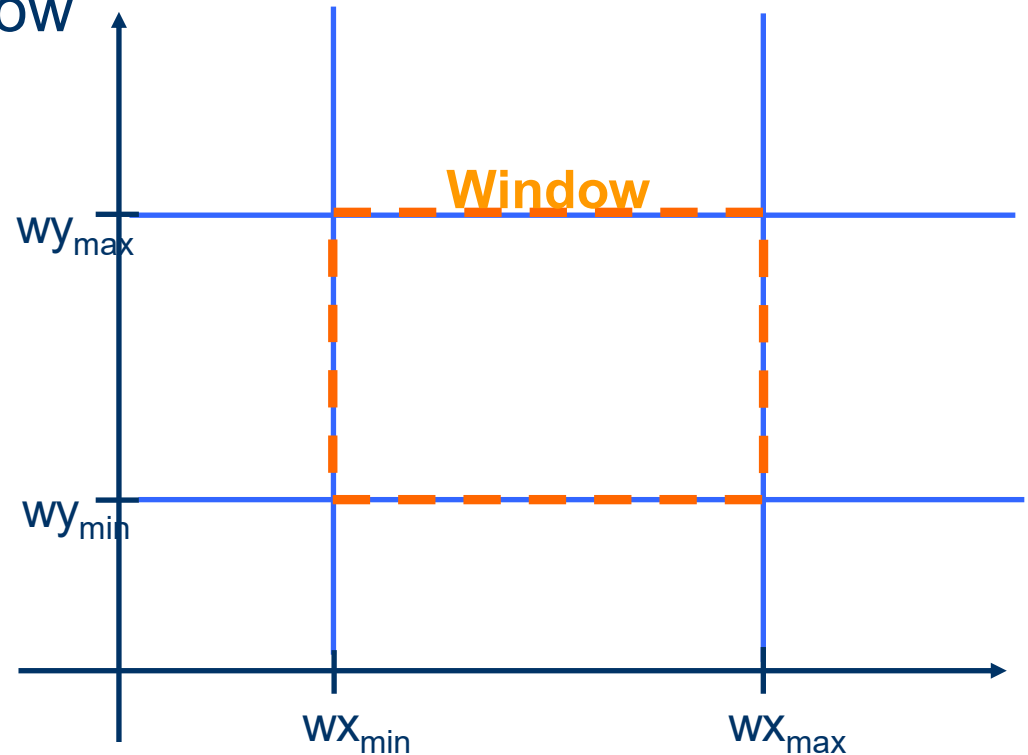
Cases for clipping lines

Cohen-Sutherland Algorithm

- The Cohen-Sutherland Line-Clipping Algorithm performs initial tests on a line to determine whether intersection calculations can be avoided.
 1. First, end-point pairs are checked for Trivial Acceptance.
 2. If the line cannot be trivially accepted, region checks are done for Trivial Rejection.
 3. If the line segment can be neither trivially accepted nor rejected, it is divided into two segments at a clip edge, so that one segment can be trivially rejected.
 - These three steps are performed iteratively until what remains can be trivially accepted or rejected.

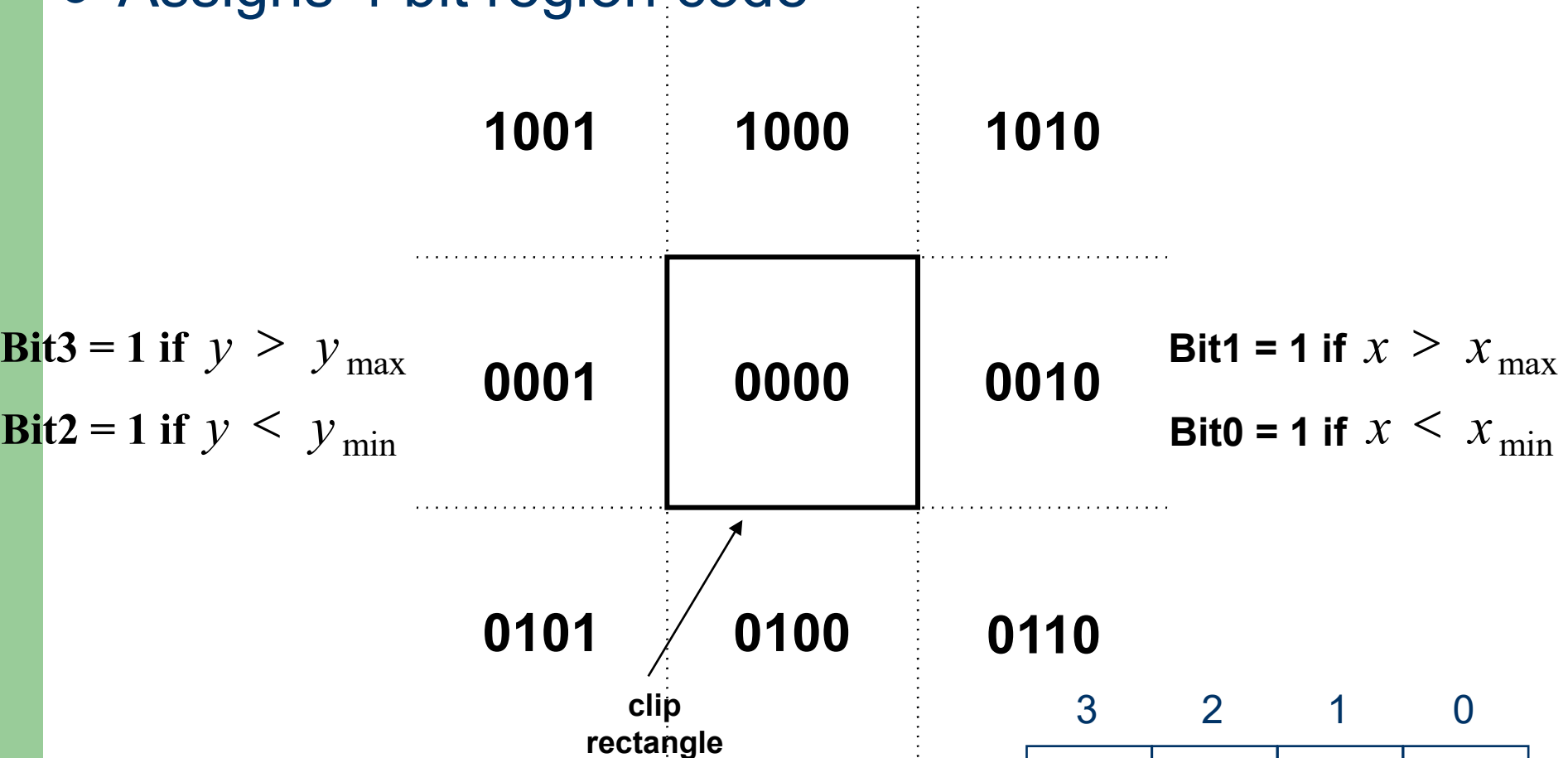
Cohen-Sutherland: World Division

- World space is divided into regions based on the window boundaries
 - Each region has a unique four bit region code
 - Region codes indicate the position of the regions with respect to the window



Cohen-Sutherland Algorithm

- Assigns 4 bit region code



Region outcodes

3	2	1	0
above	below	right	left

Region Code Legend

Cohen-Sutherland Algorithm

- A line segment can be trivially accepted (visible) if the outcodes of both the endpoints are zero.
- A line segment can be trivially rejected (not visible) if the logical AND of the outcodes of the endpoints is not zero.
- A line segment is clipping candidate if the logical AND of the outcodes of the endpoints is zero.

Intersect point

- If bit 3 is 1, intersect with line $y=y_{\max}$
- If bit 2 is 1, intersect with line $y=y_{\min}$
- If bit 1 is 1, intersect with line $x=x_{\max}$
- If bit 0 is 1, intersect with line $x=x_{\min}$

Intersect Point (x_i, y_i)

$$\left. \begin{aligned} x_i &= x_{\min} \text{ or } x_{\max} \\ y_i &= y_1 + m(x_i - x_1) \end{aligned} \right\} \text{ If edge line is vertical}$$

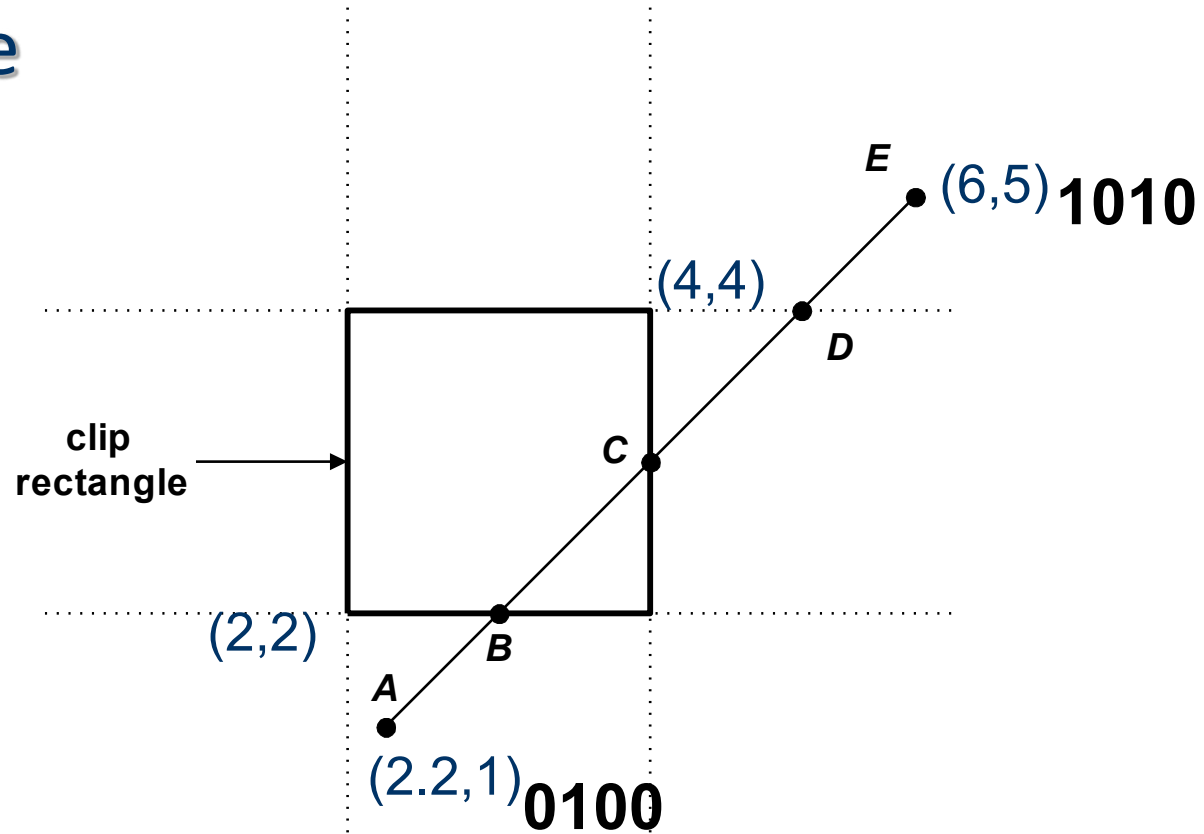
or

$$\left. \begin{aligned} x_i &= x_1 + (y_i - y_1)/m \\ y_i &= y_{\min} \text{ or } y_{\max} \end{aligned} \right\} \text{ If edge line is horizontal}$$

where, $m = (y_2 - y_1)/(x_2 - x_1)$

Cohen-Sutherland Algorithm

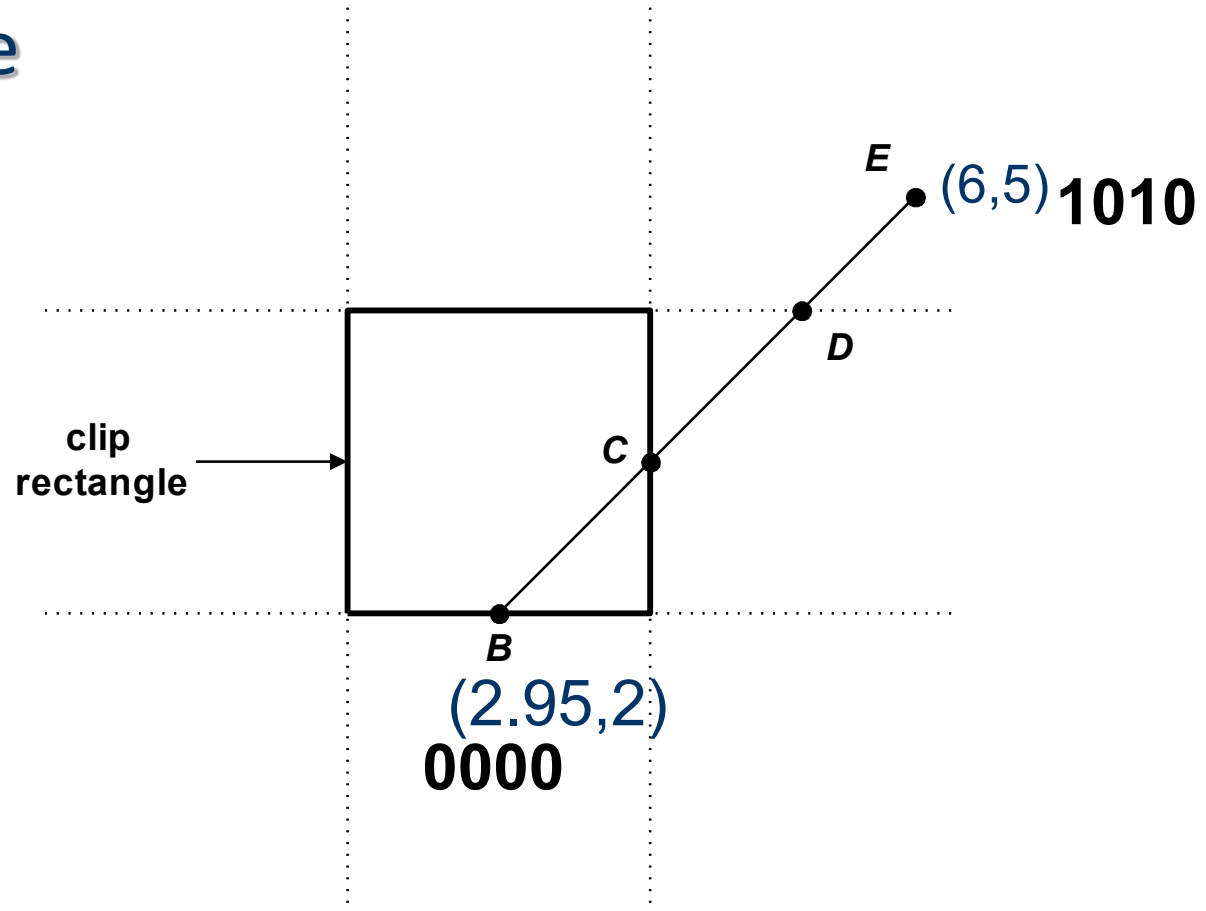
An Example



$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{5 - 1}{6 - 2.2} = \frac{4}{3.8} = 1.05$$

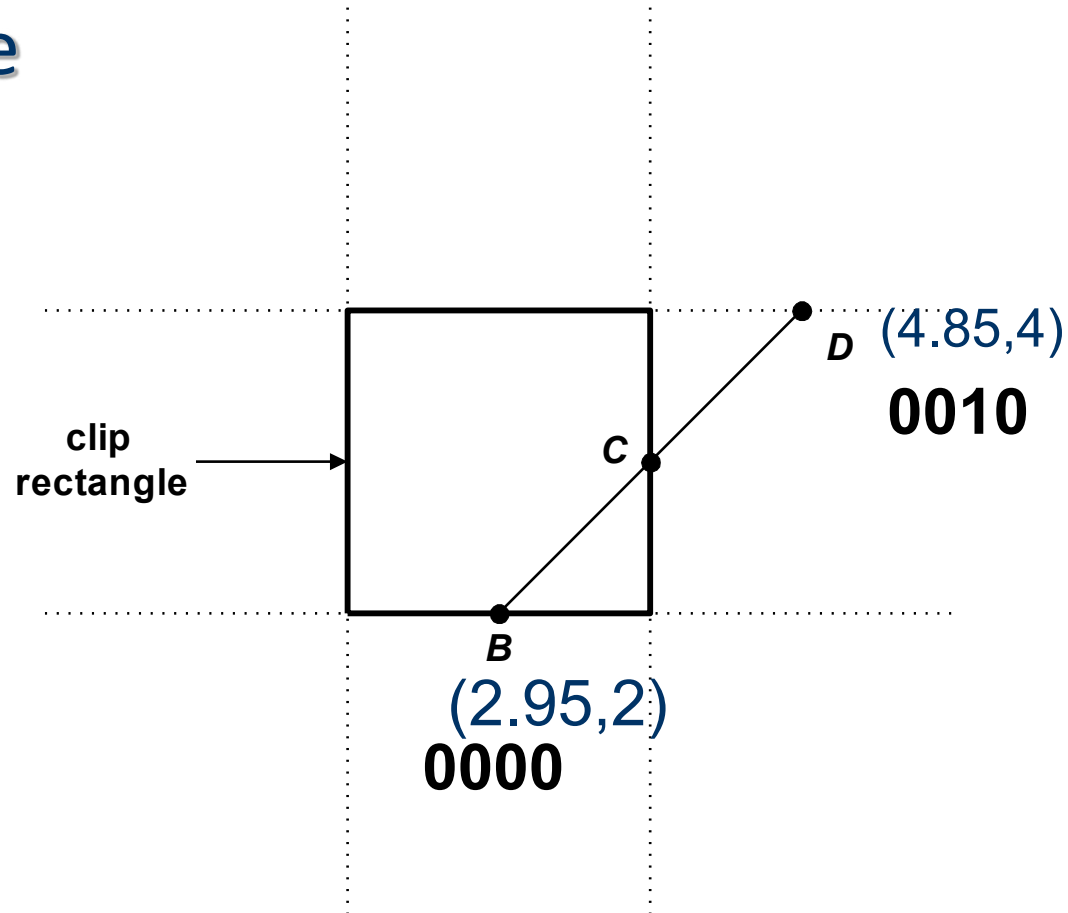
Cohen-Sutherland Algorithm

An Example



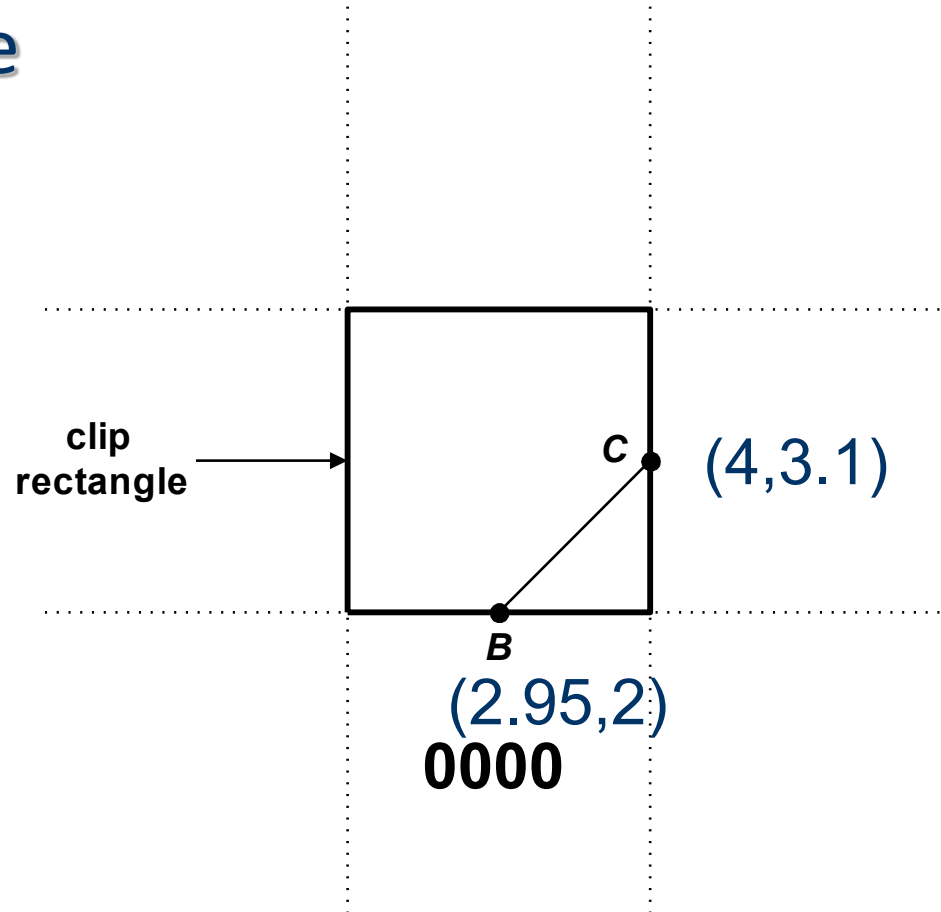
Cohen-Sutherland Algorithm

An Example

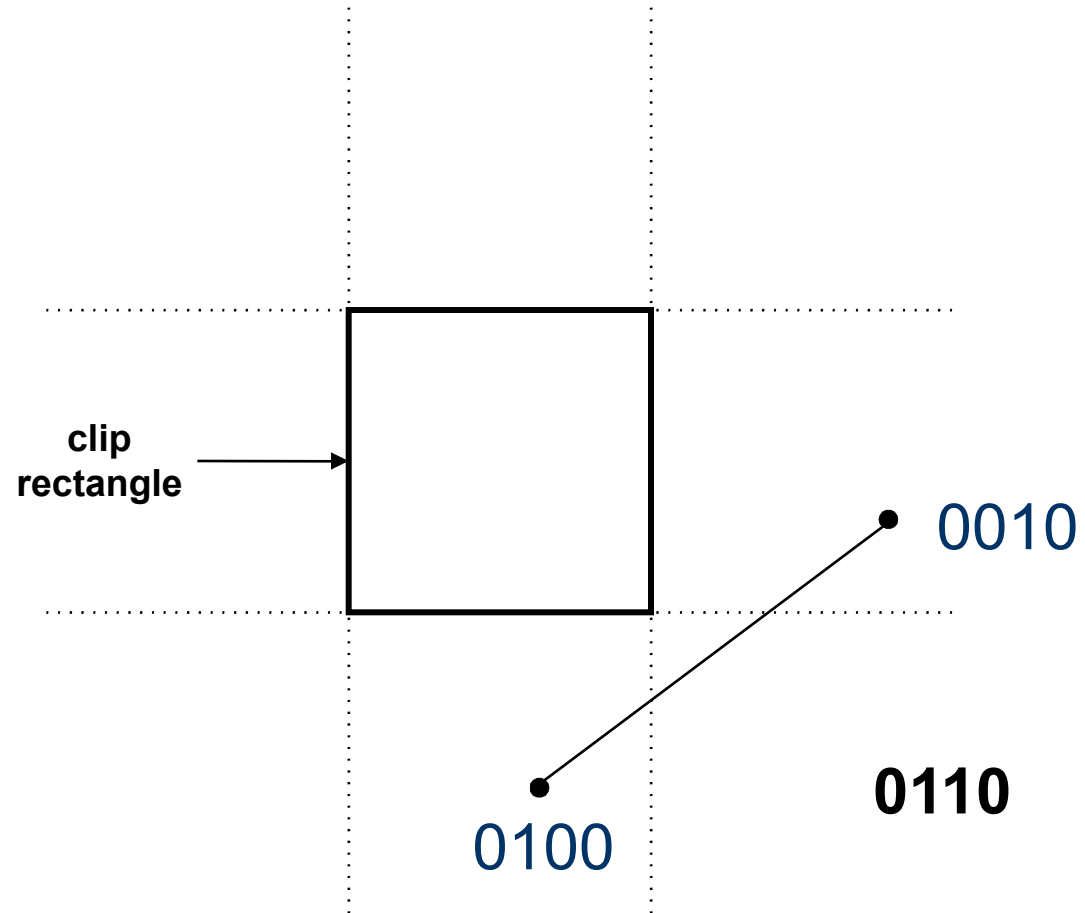


Cohen-Sutherland Algorithm

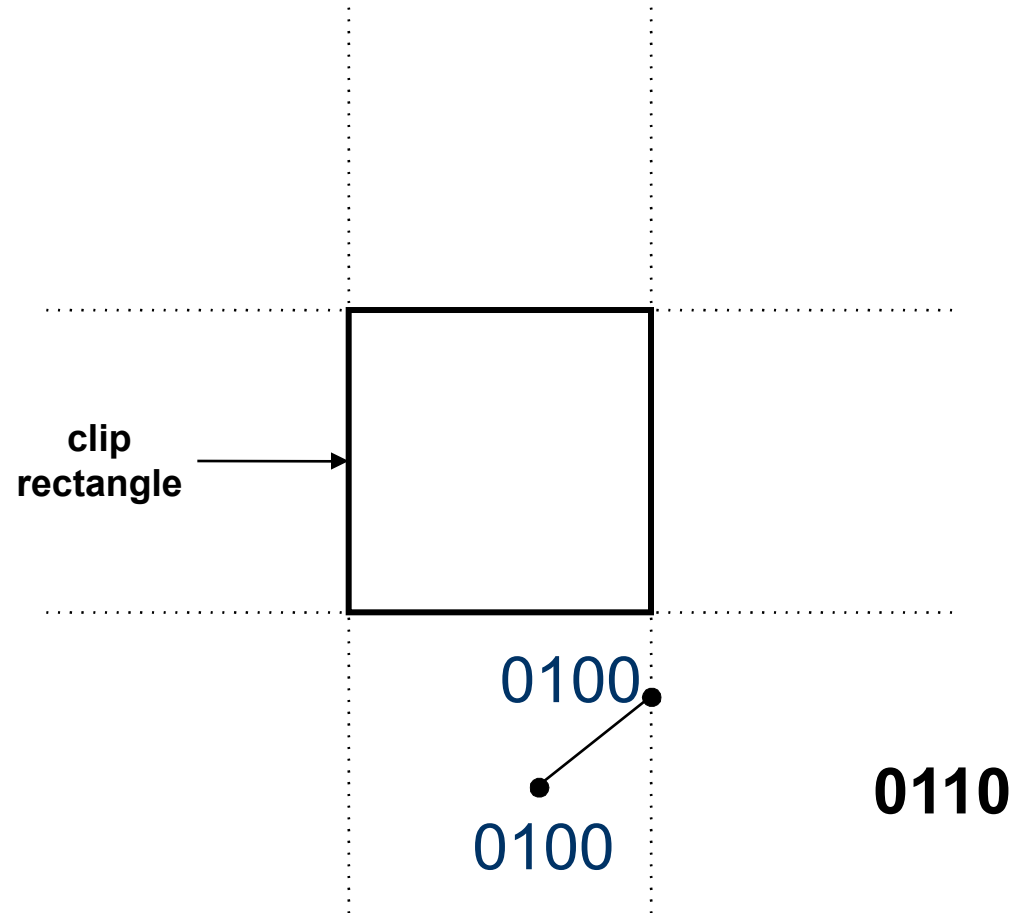
An Example



Another Example

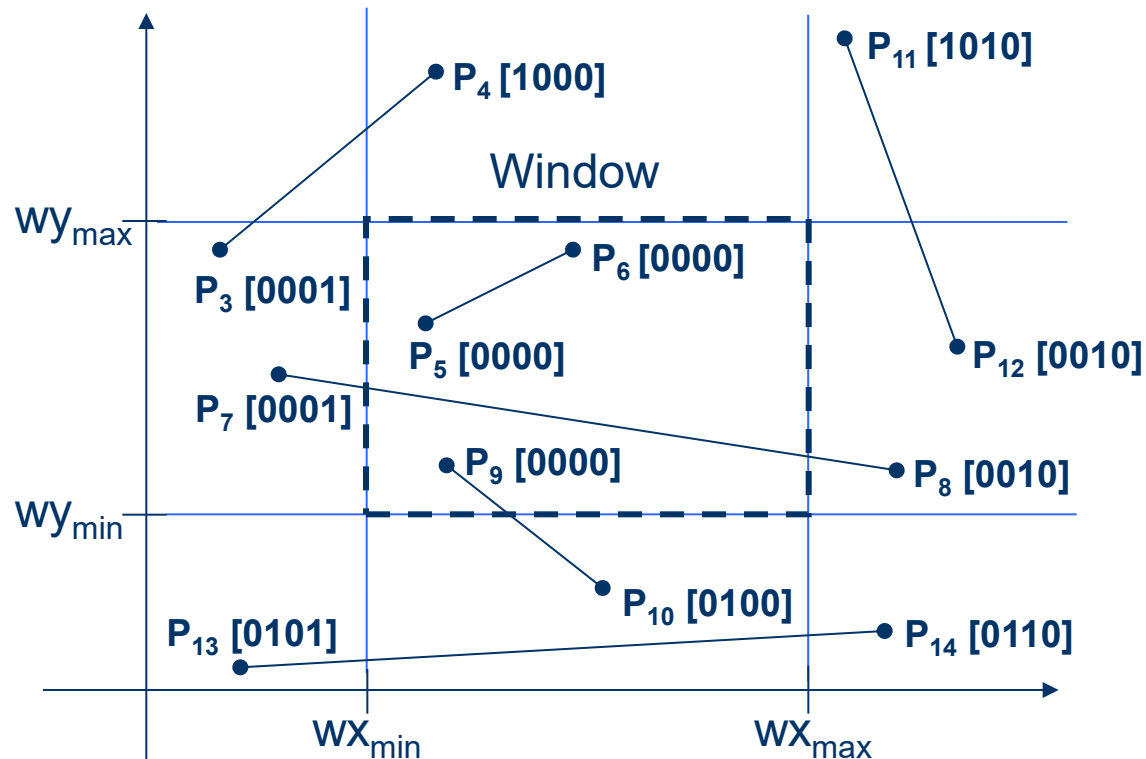


Another Example



Cohen-Sutherland

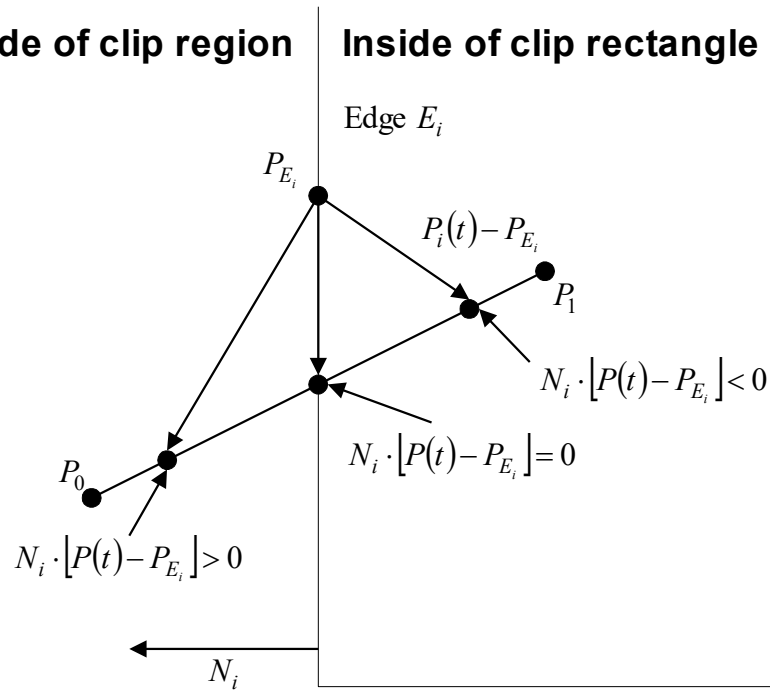
- Every end-point is labelled with the appropriate region code



Parametric Line-Clipping

- (1) This fundamentally different (from Cohen-Sutherland algorithm) and generally more efficient algorithm was originally published by Cyrus and Beck.
- (2) Liang and Barsky later independently developed a more efficient algorithm that is especially fast in the special cases of upright 2D and 3D clipping regions. They also introduced more efficient trivial rejection tests for general clip regions.

The Cyrus-Beck Algorithm



$$\text{Line } P_0P_1 : P(t) = P_0 + (P_1 - P_0)t$$

$$N_i \cdot [P(t) - P_{E_i}] = 0$$

$$\Rightarrow N_i \cdot [P_0 + (P_1 - P_0)t - P_{E_i}] = 0$$

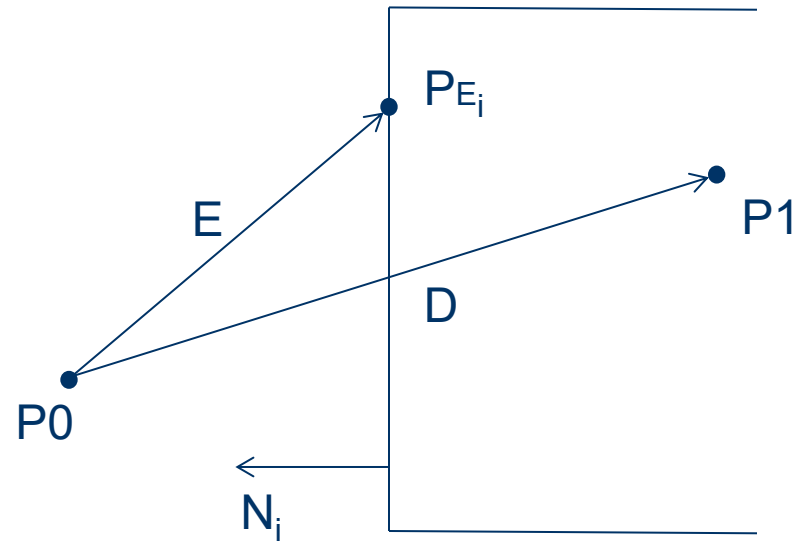
$$\Rightarrow t = -\frac{N_i \cdot [P_0 - P_{E_i}]}{N_i \cdot (P_1 - P_0)}$$

$$\Rightarrow t = \frac{N_i \cdot [P_{E_i} - P_0]}{N_i \cdot D}, \quad D = P_1 - P_0$$

The Cyrus-Beck Algorithm

here

$$N_i \cdot D < 0$$



$$t = \frac{N_i \cdot [P_{E_i} - P_0]}{N_i \cdot D} = \frac{N_i \cdot E}{N_i \cdot D}$$

t exists when

- (1) $N_i \neq 0$
- (2) $D \neq 0 \Rightarrow P_0 \neq P_1$
- (3) $N_i \cdot D \neq 0$

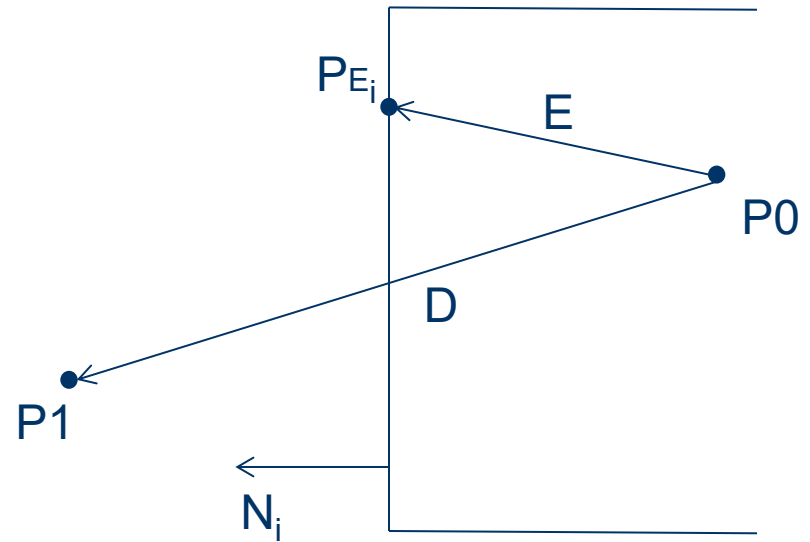
PE = Potentially Entering

$$N_i \cdot D < 0 \Rightarrow PE \\ \Rightarrow Angle > 90^\circ$$

The Cyrus-Beck Algorithm

here

$$N_i \cdot D > 0$$



$$t = \frac{N_i \cdot [P_{E_i} - P_0]}{N_i \cdot D} = \frac{N_i \cdot E}{N_i \cdot D}$$

t exists when

- (1) $N_i \neq 0$
- (2) $D \neq 0 \Rightarrow P_0 \neq P_1$
- (3) $N_i \cdot D \neq 0$

PL = Potentially Leaving

$$N_i \cdot D > 0 \Rightarrow PL \\ \Rightarrow Angle < 90^\circ$$

The Cyrus-Beck Algorithm

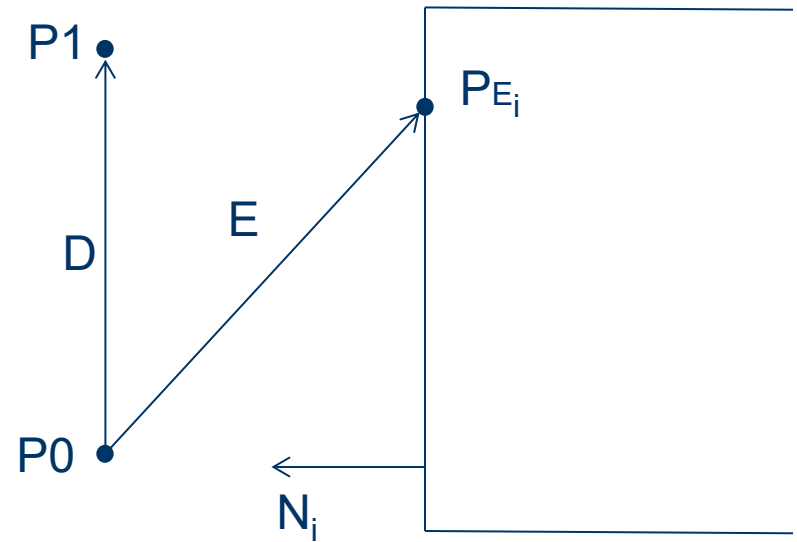
here

$$N_i \cdot D = 0$$

$$t = \frac{N_i \cdot [P_{E_i} - P_0]}{N_i \cdot D} = \frac{N_i \cdot E}{N_i \cdot D}$$

t exists when

- (1) $N_i \neq 0$
- (2) $D \neq 0 \Rightarrow P_0 \neq P_1$
- (3) $N_i \cdot D \neq 0$



The Cyrus-Beck Algorithm

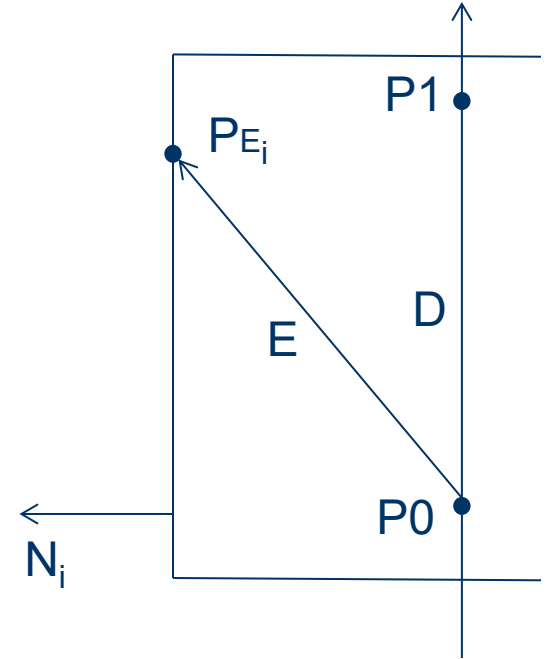
here

$$N_i \cdot D = 0$$

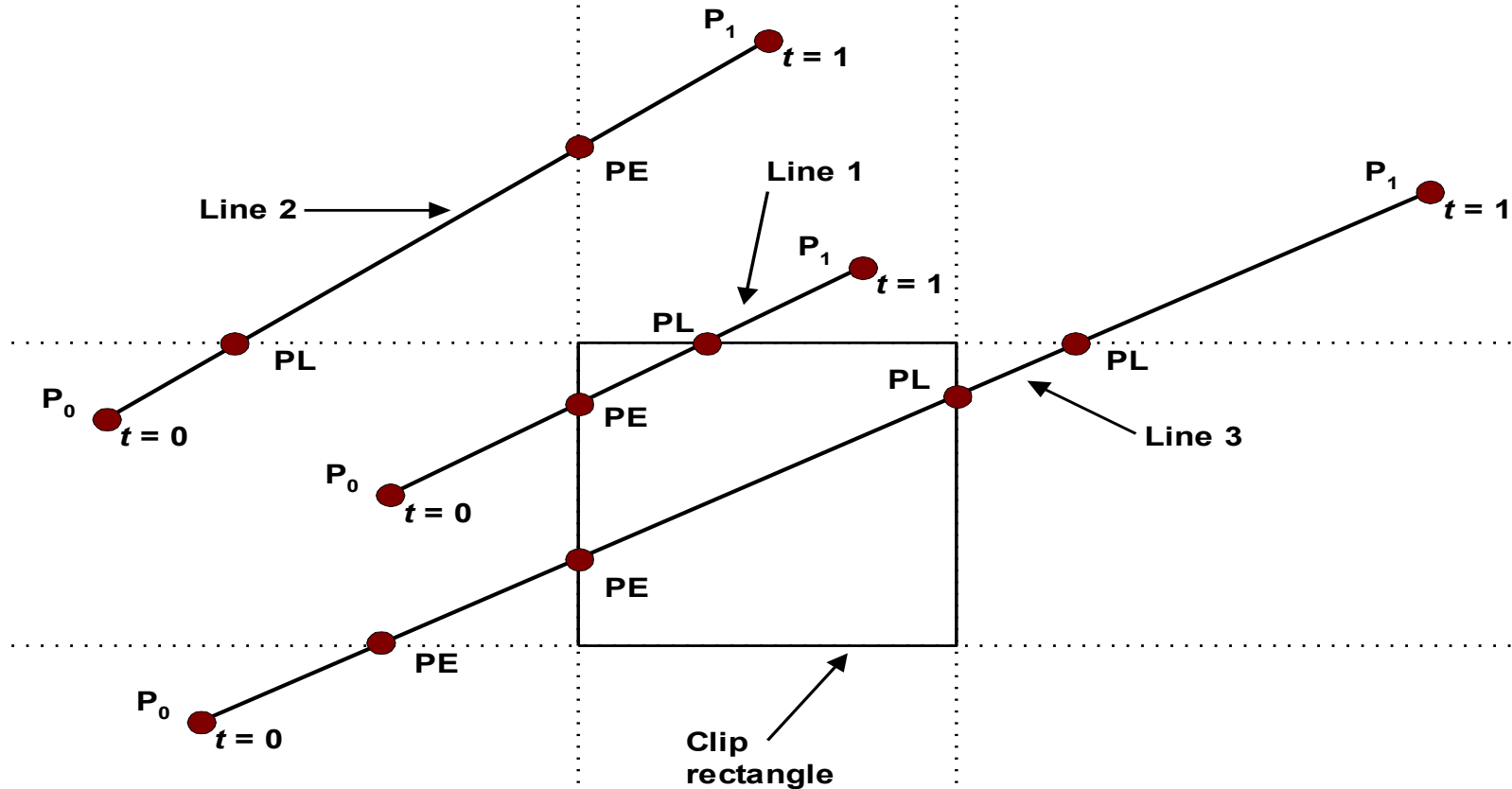
$$t = \frac{N_i \cdot [P_{E_i} - P_0]}{N_i \cdot D} = \frac{N_i \cdot E}{N_i \cdot D}$$

t exists when

- (1) $N_i \neq 0$
- (2) $D \neq 0 \Rightarrow P_0 \neq P_1$
- (3) $N_i \cdot D \neq 0$



The Cyrus-Beck Algorithm



PE = Potentially Entering

$$N_i \cdot D < 0 \Rightarrow PE$$

$$\Rightarrow Angle > 90^\circ$$

PL = Potentially Leaving

$$N_i \cdot D > 0 \Rightarrow PL$$

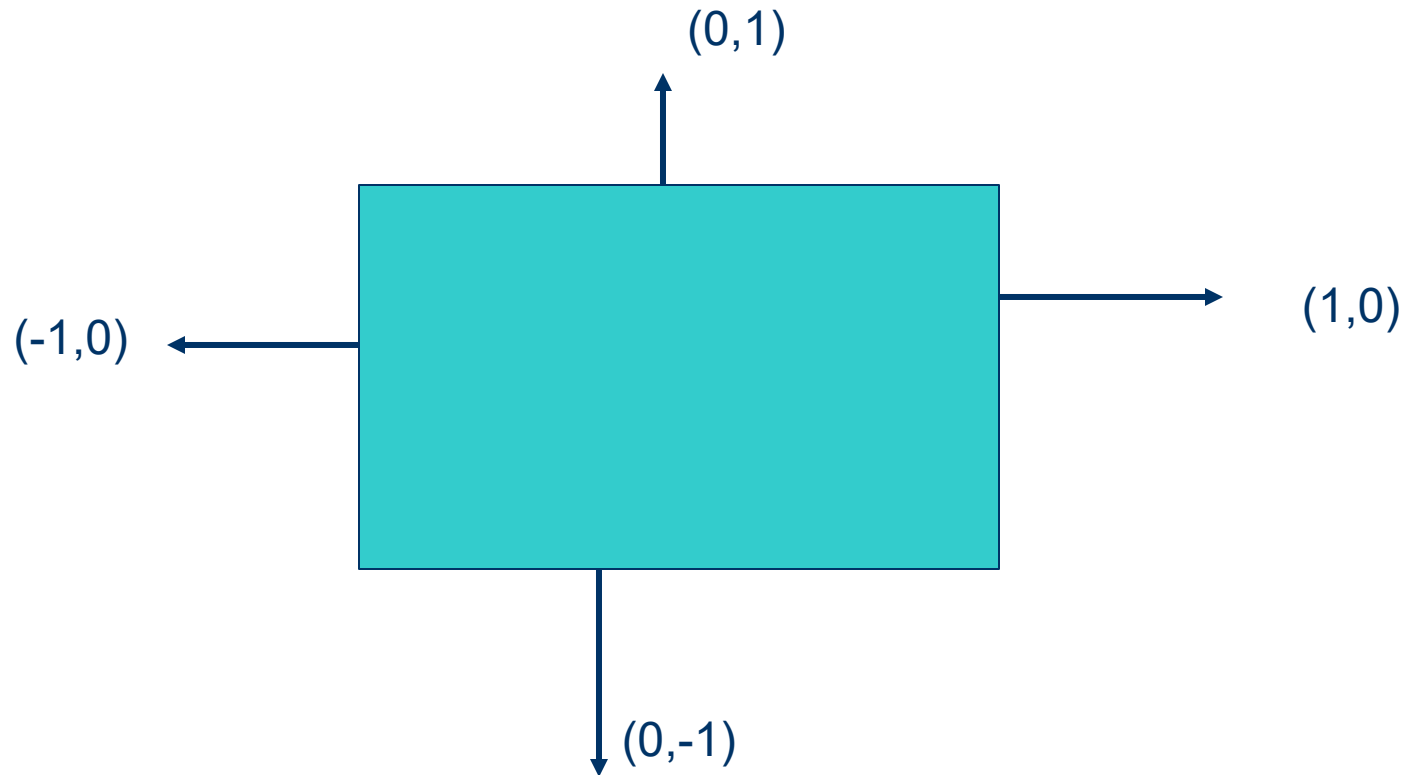
$$\Rightarrow Angle < 90^\circ$$

The Cyrus-Beck Algorithm

```
Precalculate  $N_i$  and  $P_{Ei}$  for each edge
for (each line segment to be clipped) {
    if ( $P_1 == P_0$ )
        line is degenerated, so clip as a point;
    else {
         $t_E = 0$ ;  $t_L = 1$ ;
        for (each candidate intersection with a clip edge) {
            if ( $N_i \cdot D \neq 0$ ) { /* Ignore edges parallel to line */
                calculate  $t$ ;
                use sign of  $N_i \cdot D$  to categorize as PE or PL;
                if (PE)  $t_E = \max(t_E, t)$ ;
                if (PL)  $t_L = \min(t_L, t)$ ;
            }
        }
        if ( $t_E > t_L$ ) return NULL;
        else return  $P(t_E)$  and  $P(t_L)$  as true clip intersection;
    }
}
```

Liang-Barsky Improvement

- Considered up right rectangular window



Liang-Barsky Improvement

<u>Clip edge_i</u>	<u>Normal N_i</u>	<u>P_{E_i}</u>	<u>$P_{E_i} - P_0$</u>	<u>$t = \frac{N_i \cdot (P_{E_i} - P_0)}{N_i \cdot D}$</u>	<u>$N_i \cdot D$</u>
left: $x = x_{\min}$	$(-1, 0)$	$(x_{\min}, y)$	$(x_{\min} - x_0, y - y_0)$	$\frac{(x_{\min} - x_0)}{(x_1 - x_0)}$	$-(x_1 - x_0)$
right: $x = x_{\max}$	$(1, 0)$	$(x_{\max}, y)$	$(x_{\max} - x_0, y - y_0)$	$\frac{(x_{\max} - x_0)}{(x_1 - x_0)}$	$(x_1 - x_0)$
bottom: $y = y_{\min}$	$(0, -1)$	$(x, y_{\min})$	$(x - x_0, y_{\min} - y_0)$	$\frac{(y_{\min} - y_0)}{(y_1 - y_0)}$	$-(y_1 - y_0)$
top: $y = y_{\max}$	$(0, 1)$	$(x, y_{\max})$	$(x - x_0, y_{\max} - y_0)$	$\frac{(y_{\max} - y_0)}{(y_1 - y_0)}$	$(y_1 - y_0)$

Liang-Barsky Improvement

here

$$N_L \cdot D = 0$$

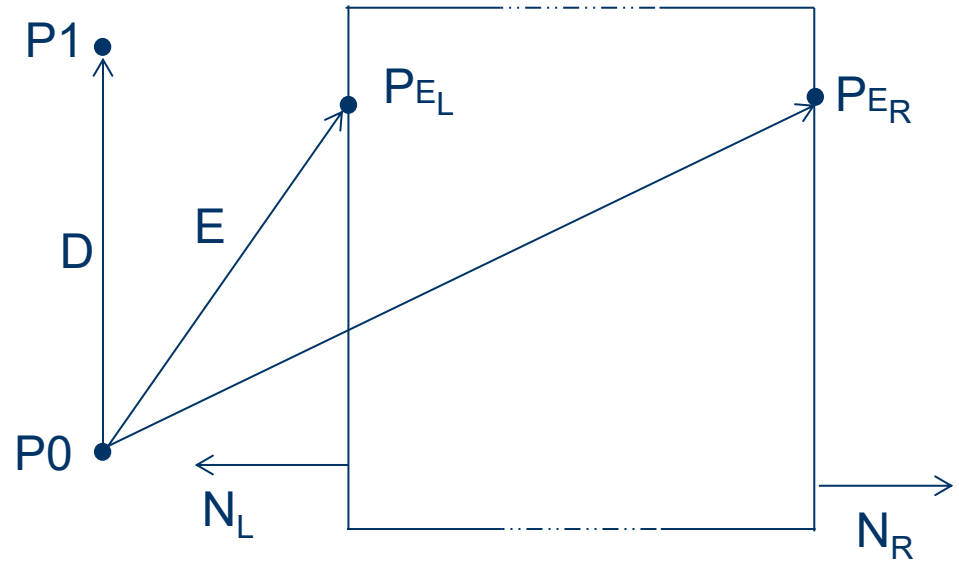
$$N_R \cdot D = 0$$

If $(N_L \cdot E_L)(N_R \cdot E_R) < 0$ reject

$$t = \frac{N_i \cdot [P_{E_i} - P_0]}{N_i \cdot D} = \frac{N_i \cdot E}{N_i \cdot D}$$

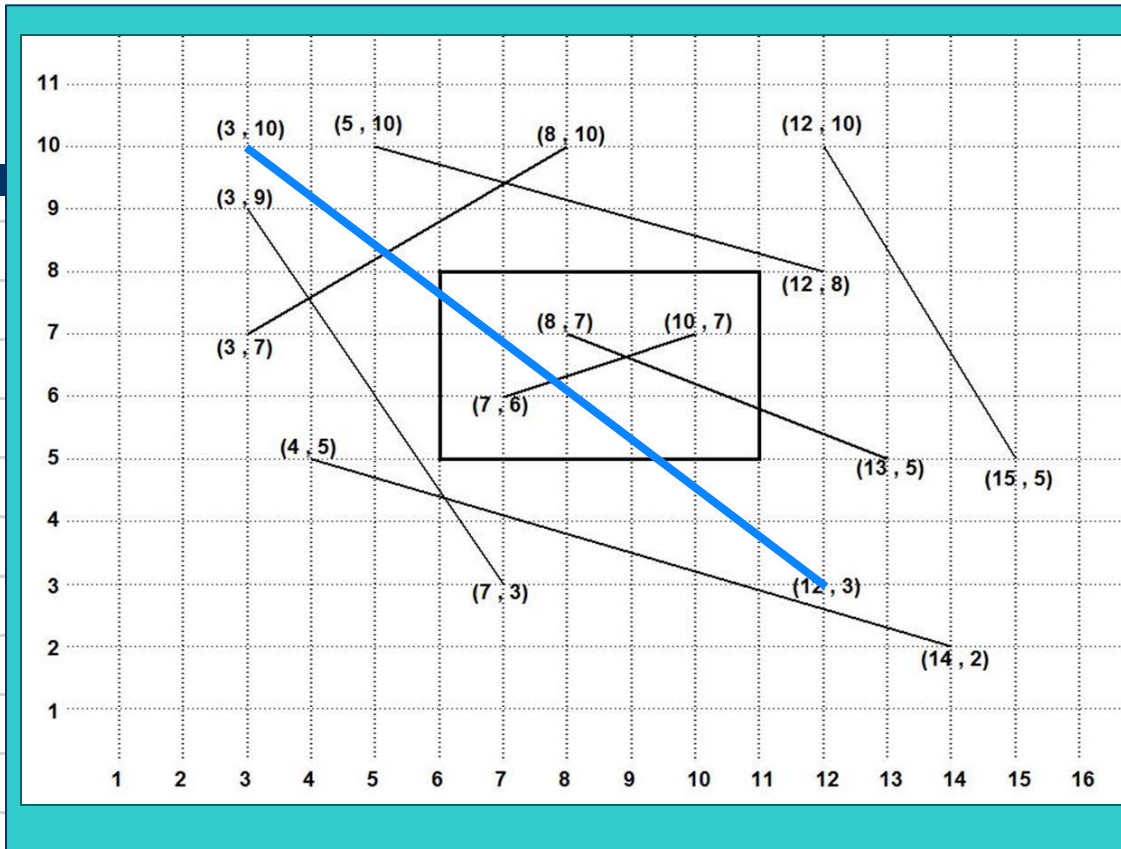
t exists when

- (1) $N_i \neq 0$
- (2) $D \neq 0 \Rightarrow P_0 \neq P_1$
- (3) $N_i \cdot D \neq 0$



Example

Pmin	6,5		
pmax	11,8		
	L1		
P0	3,10		
P1	12,3		
$x1-x0=dx$	9		
$y1-y0=dy$	-7		
$tl=(xmin-x0)/dx$	$3/9 =$	0.333	PE
N.D=-dx for left	-9 (-ve)		
$tr=(xmax-x0)/dx$	$8/9 =$	0.889	PL
N.D=dx for right	9 (+ve)		
$tb=(ymin-y0)/dy$	$-5/(-7) =$	0.714	PL
N.D=-dy for bottom	7 (+ve)		
$tt=(ymax-y0)/dy$	$-2/(-7) =$	0.286	PE
N.D=dy for top	-7 (-ve)		
$te = \max(0, \text{all PE})$	0.333		
$tl = \min(1, \text{all PL})$	0.714		



5.997 7.669

9.426 5.002

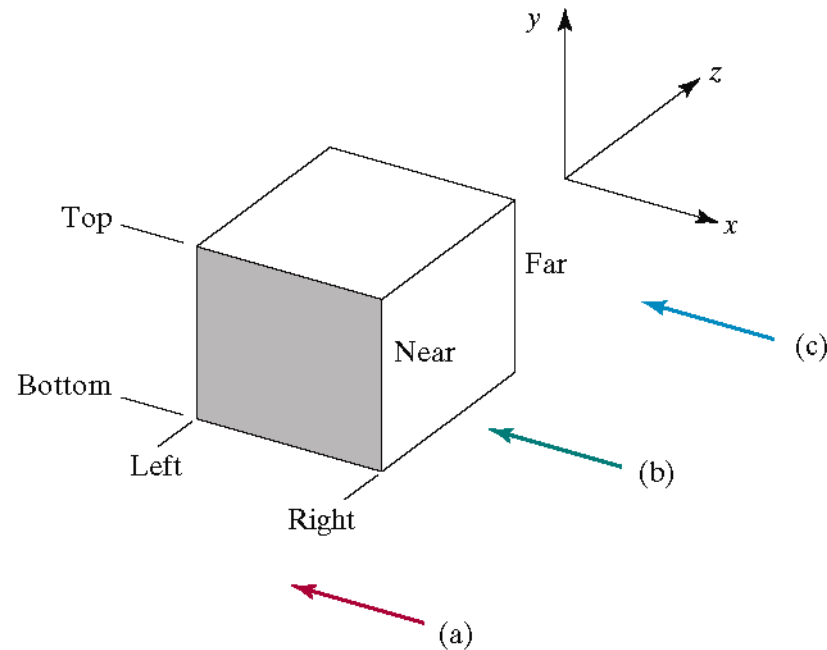
Comparison

- Cohen-Sutherland:
 - Repeated clipping is expensive
 - Best used when trivial acceptance and rejection is possible for most lines
- Liang-Barsky:
 - Computation of t -intersections is cheap
 - Computation of (x,y) clip points is only done once
 - Algorithm doesn't consider trivial accepts/rejects
 - Best when many lines must be clipped

Extension to 3d

- Liang Barsky
- Cohen Sutherland

Region Codes



011001	011000	011010
010001	010000	010010
010101	010100	010110

Region Codes
In Front of Near Plane
(a)

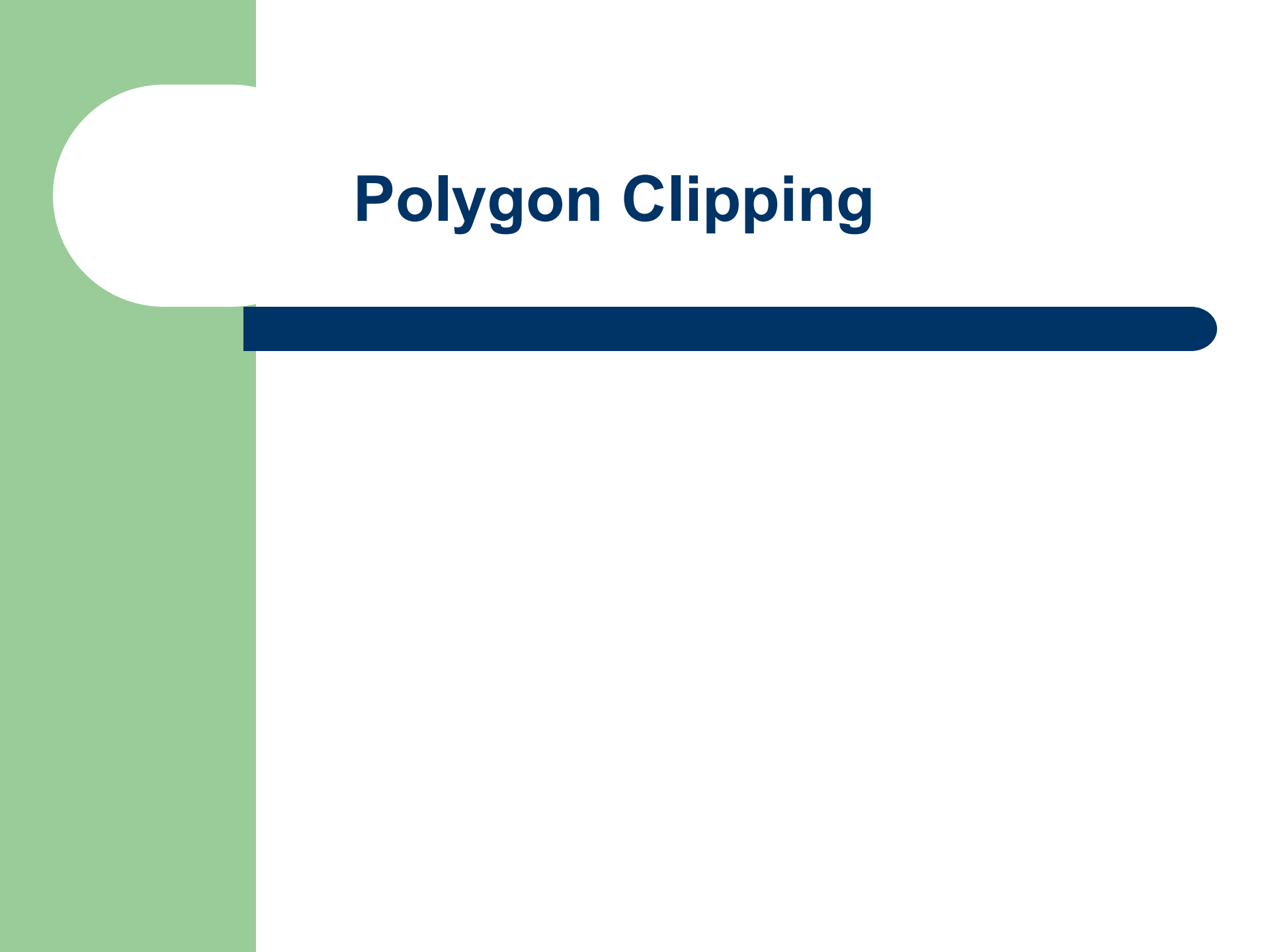
001001	001000	001010
000001	000000	000010
000101	000100	000110

Region Codes
Between Near and Far Planes
(b)

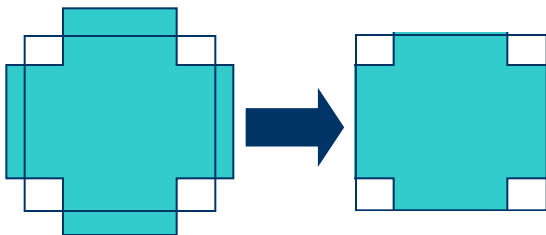
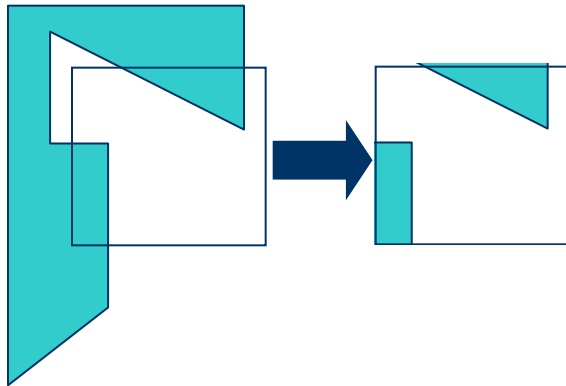
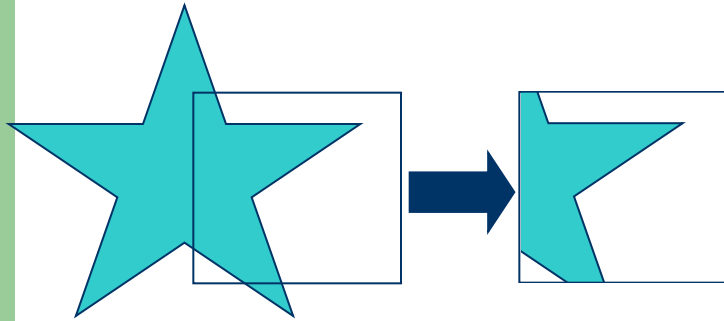
101001	101000	101010
100001	100000	100010
100101	100100	100110

Region Codes
Behind Far Plane
(c)

Polygon Clipping



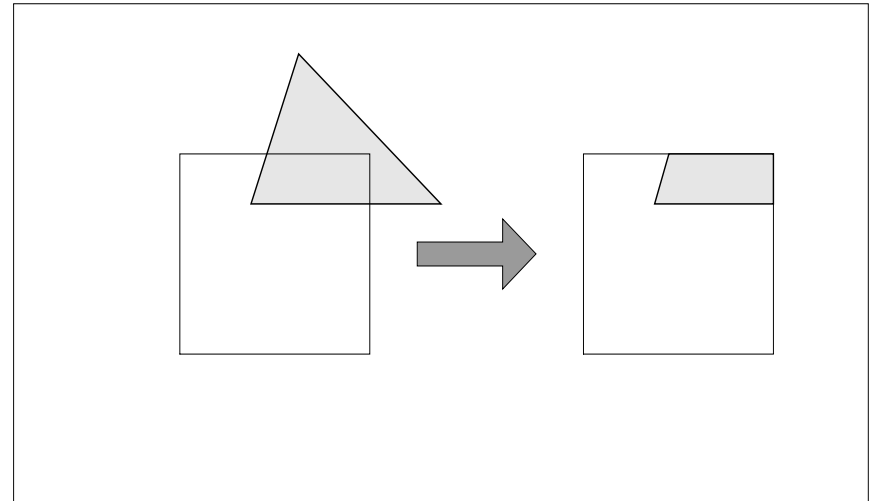
Polygon / Area Clipping



- Similarly to lines, areas must be clipped to a window boundary
- Consideration must be taken as to which portions of the area must be clipped

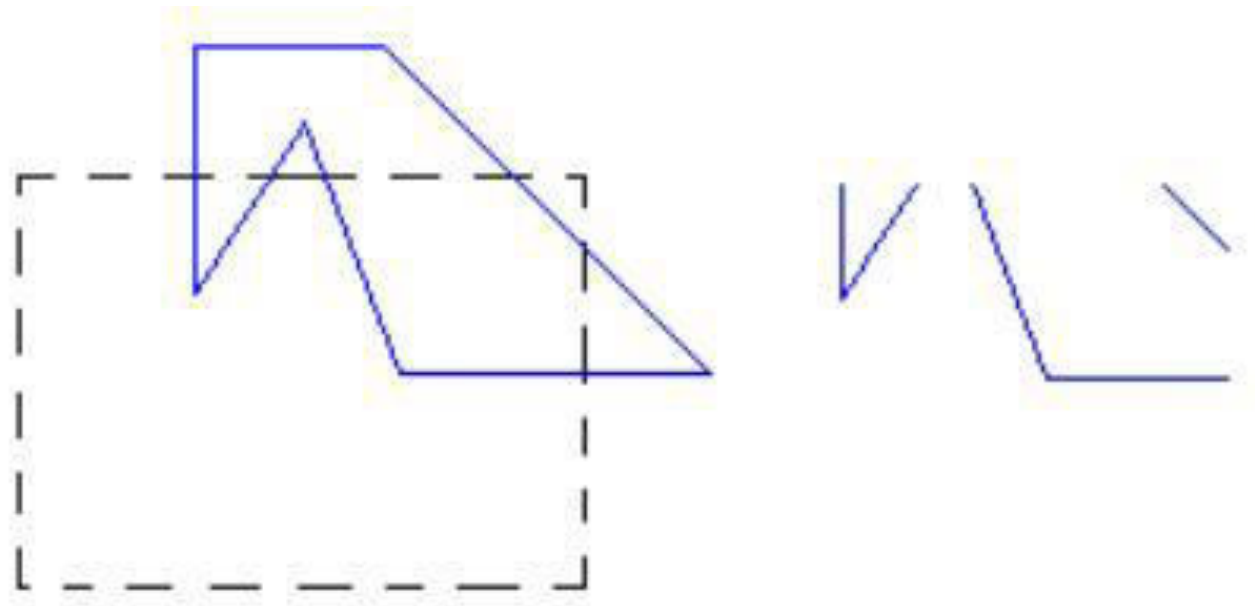
Polygon Clipping

- We know how to clip a single line segment
 - How about a polygon in 2D?
 - How about in 3D?
- Clipping polygons is more complex than clipping the individual lines
 - Input: polygon
 - Output: polygon, or nothing



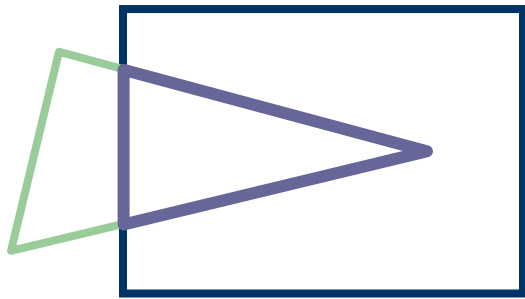
Polygon Clipping

- To clip a **polygon**, we cannot directly apply a line-clipping method to the individual polygon edges because this approach would produce a series of **unconnected line segments** as shown in figure .

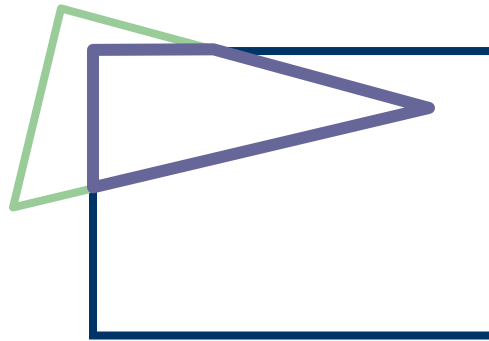


Why Is Clipping Hard?

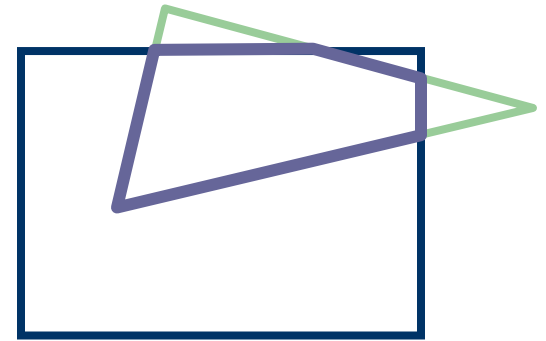
- What happens to a triangle during clipping?
- Possible outcomes:



triangle $\square$ triangle



triangle $\square$ quad

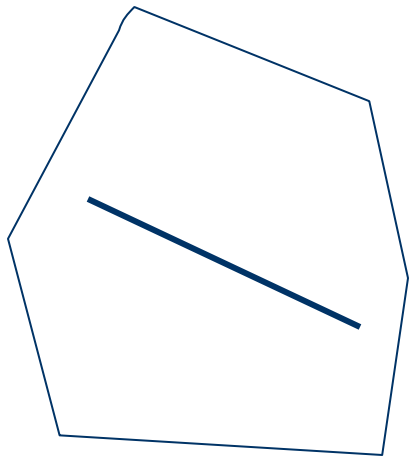


triangle $\square$ 5-gon

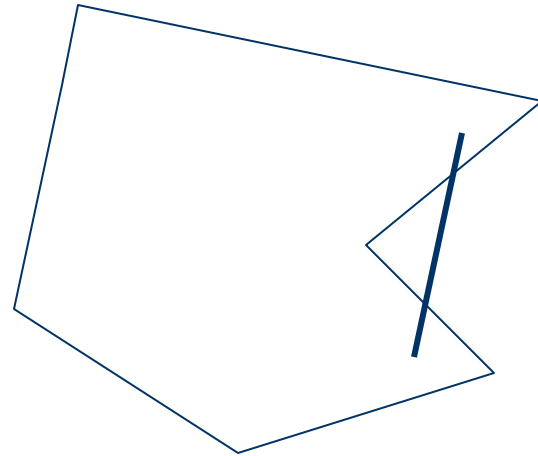
How many sides can a clipped triangle have?

Polygon Clipping

- Convex polygonal clipping window
 - **Convex polygon:** if the line joining two interior points lies completely inside the polygon
 - Otherwise it is called **concave polygon**

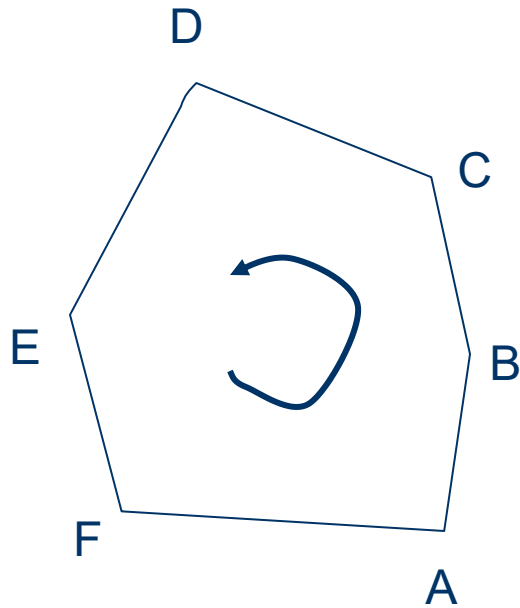


Convex

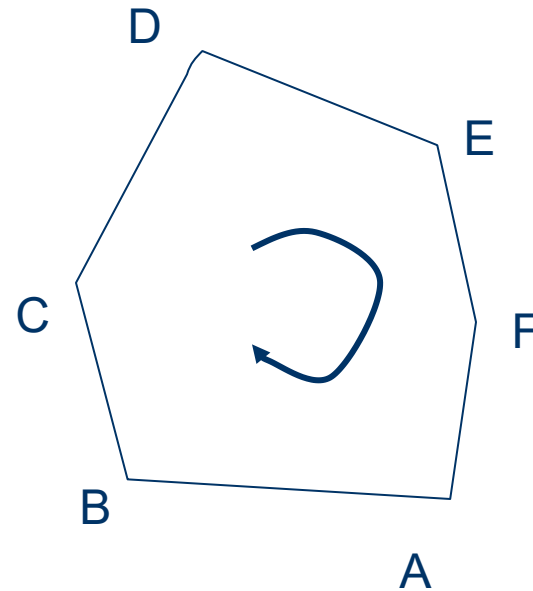


Concave

Polygon Clipping



Positive
Orientation



Negative
Orientation

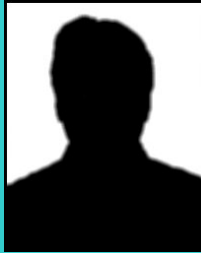
Polygon Clipping

- Left hand side of any directed edge $\overline{P_{i-1}P_i}$ or $\overline{P_N P_1}$ points inside the polygon
- Let, a point $P(x,y)$. If it is to the left of every edge of the polygon, it is inside the polygon

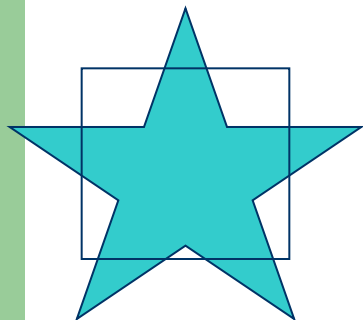
Sutherland-Hodgman Algorithm

- A technique for clipping areas developed by Sutherland & Hodgman
- Put simply the polygon is clipped by comparing it against each boundary in turn

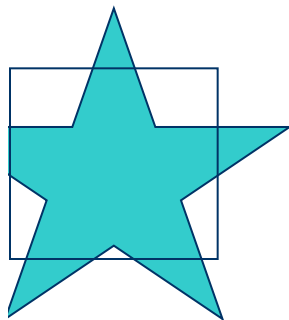
Sutherland turns up again. This time with



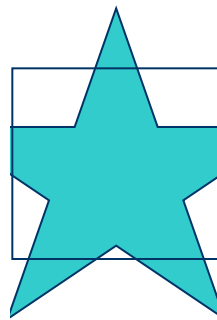
Gary Hodgman with whom he worked at the first ever graphics company Evans & Sutherland



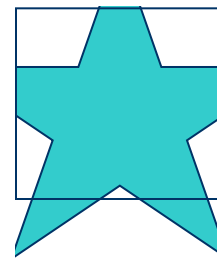
Original Area



Clip Left



Clip Right

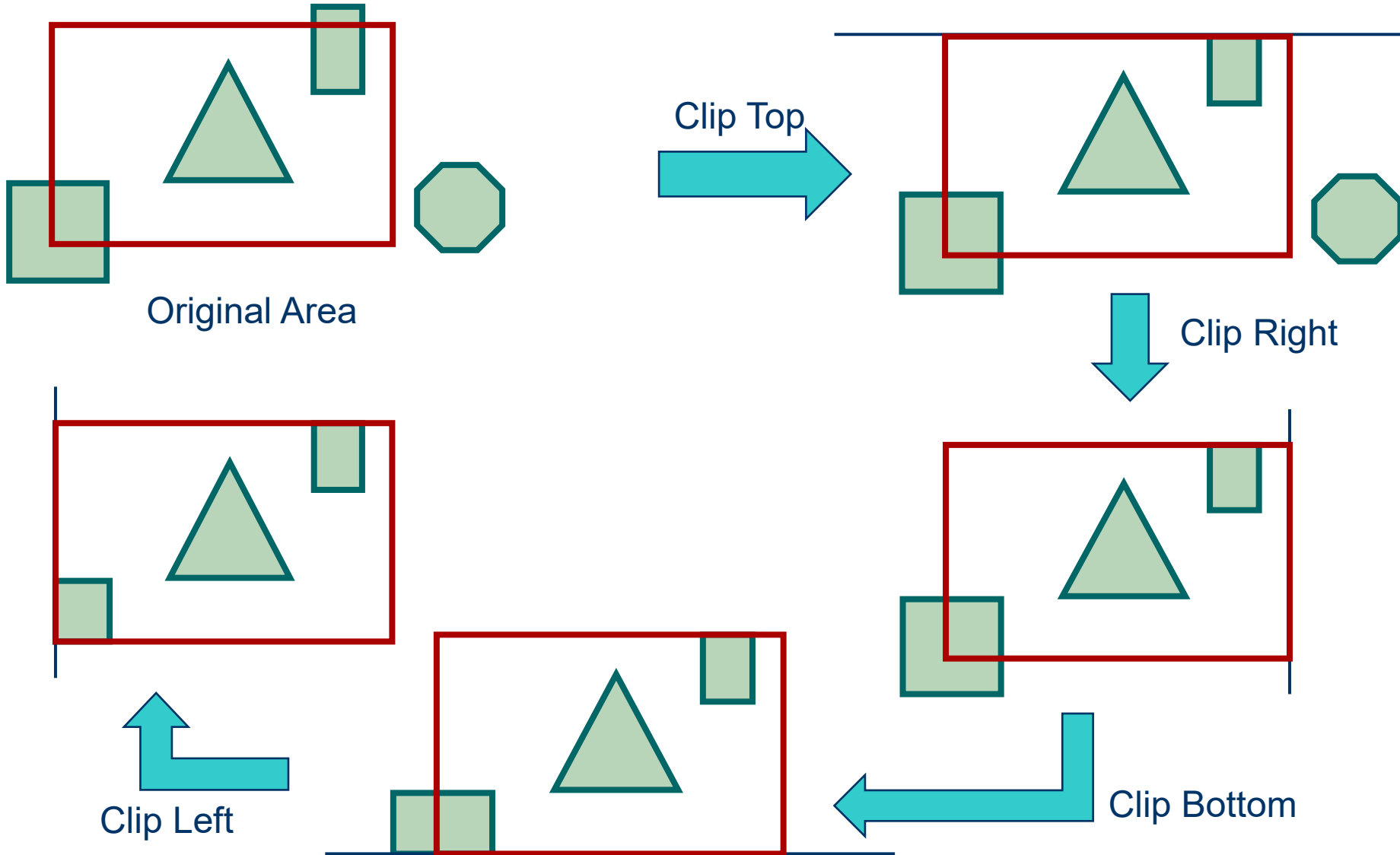


Clip Top

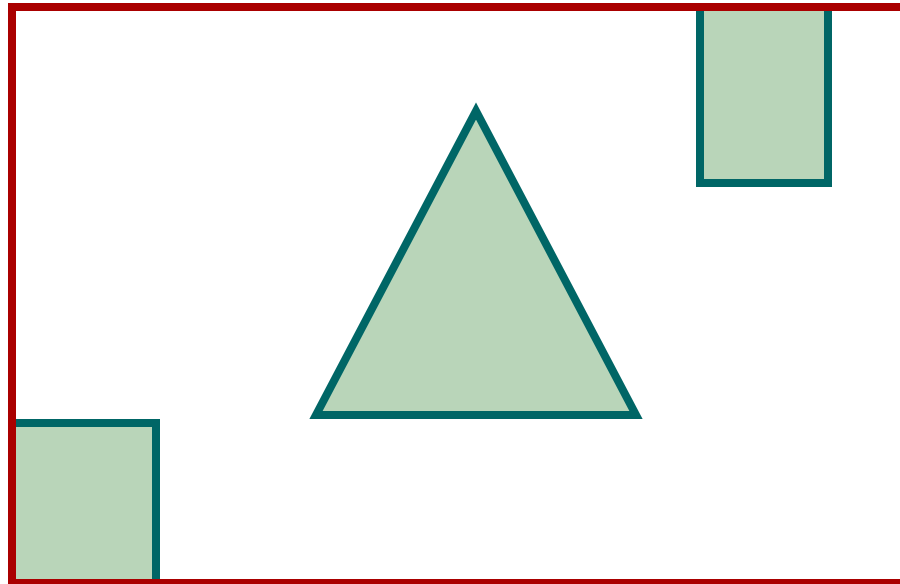


Clip Bottom

Sutherland-Hodgeman Polygon Clipping



Polygon Clipping



After Clipping

Sutherland Hodgman Algorithm

- $P_1, P_2, \dots, P_N$ be vertex list of the polygon to be clipped (**subject polygon**).
- Edge E, defined by points A and B, be any edge of the positively oriented, convex **clipping polygon**
- Vertex output list – a list containing the vertices that to be displayed after clipping

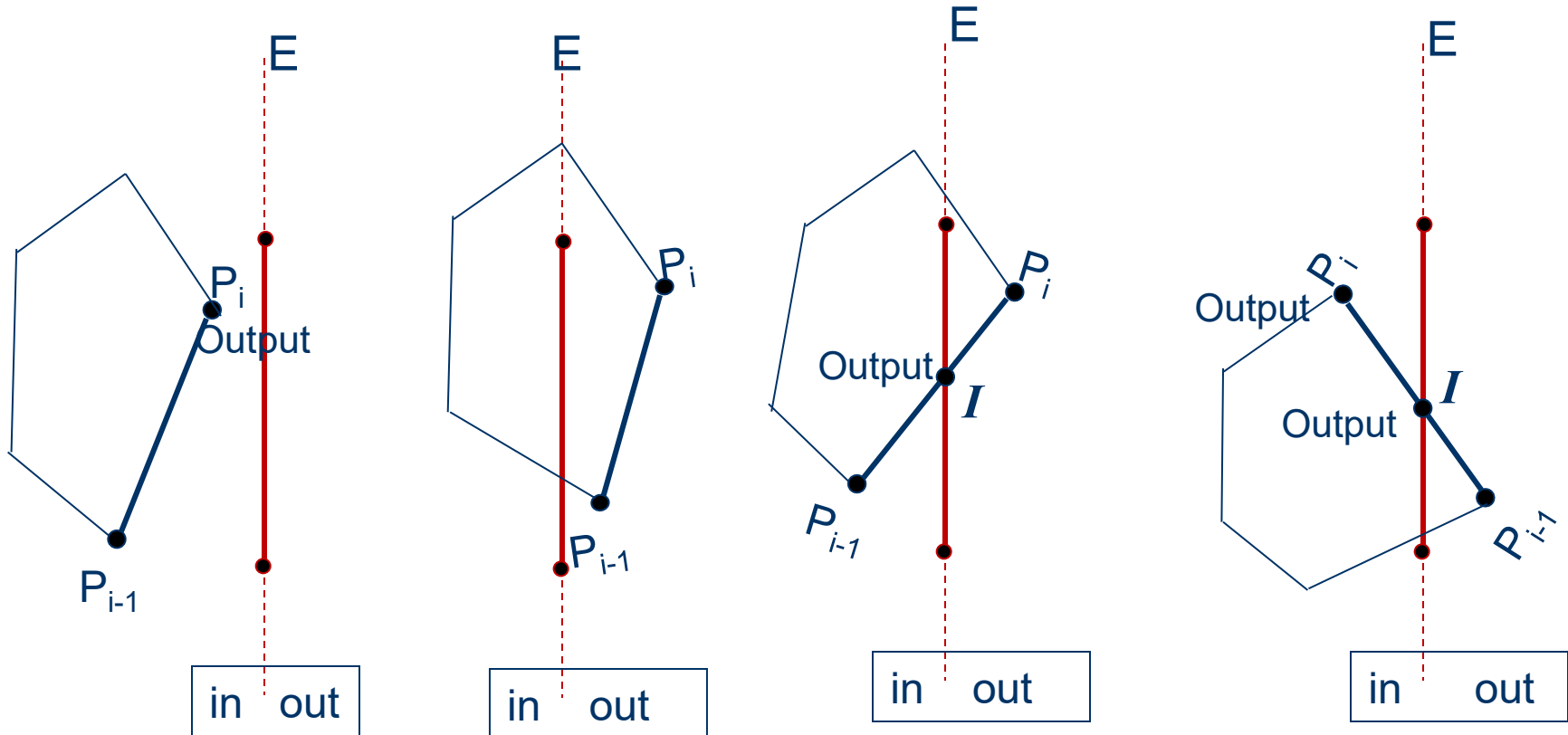
Sutherland Hodgman Algorithm

Consider edge $\overline{P_{i-1}P_i}$

- If both P_{i-1} and P_i are left of the edge E , place P_i to vertex output list
 - If both P_{i-1} and P_i are right of the edge, place nothing to vertex output list
 - If P_{i-1} is left and P_i is right of the edge, find intersect point I and place I to vertex output list
 - If P_{i-1} is right and P_i is left of the edge, find intersect point I and place both I and P_i to vertex output list
- Algorithm proceeds by passing each clipped polygon to the next edge of the window.

Sutherland Hodgman Algorithm

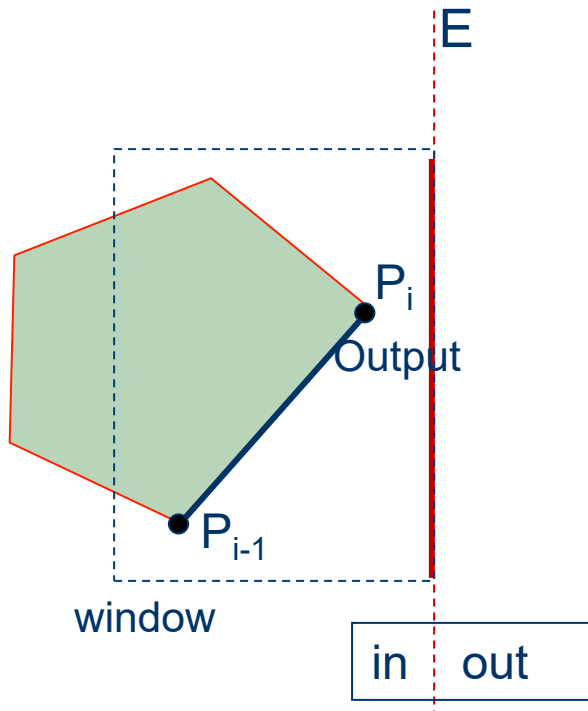
- 4 cases



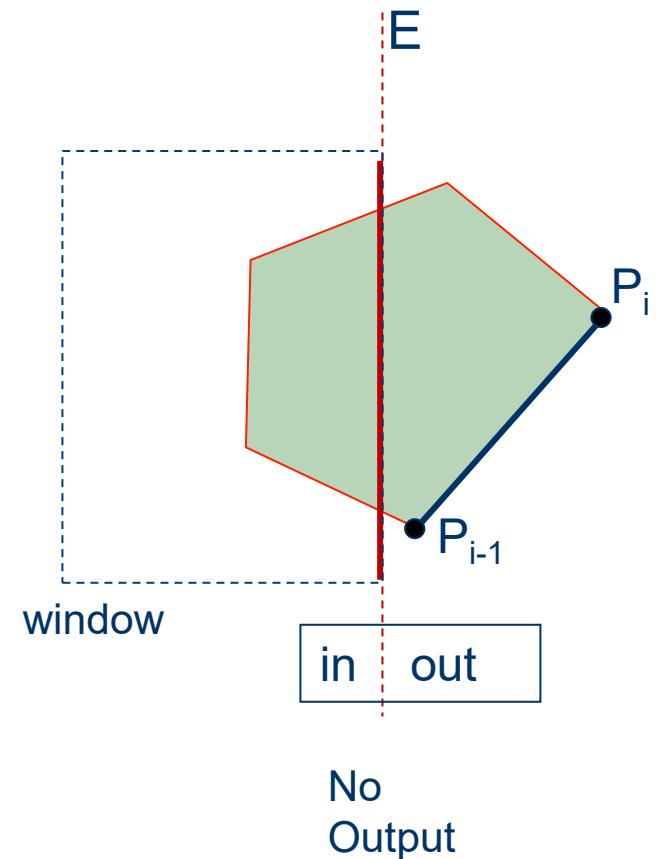
Sutherland Hodgman Algorithm

- 4 cases

both P_{i-1} and P_i are left / inside



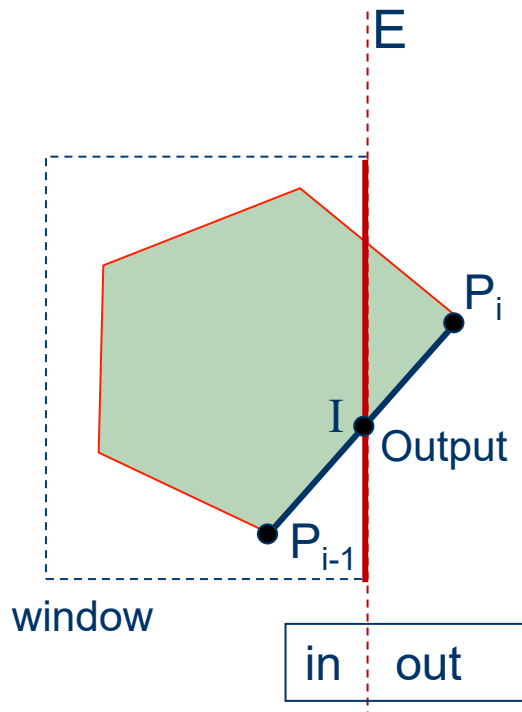
both P_{i-1} and P_i are right / outside



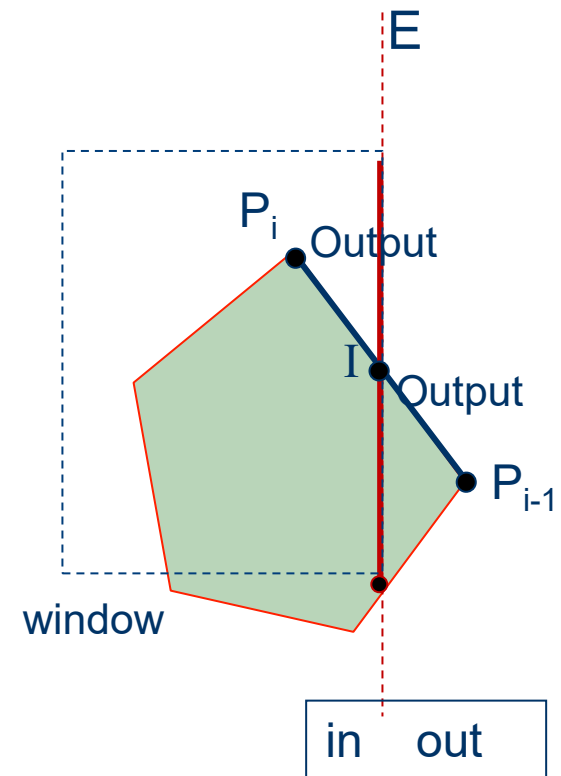
Sutherland Hodgman Algorithm

- 4 cases

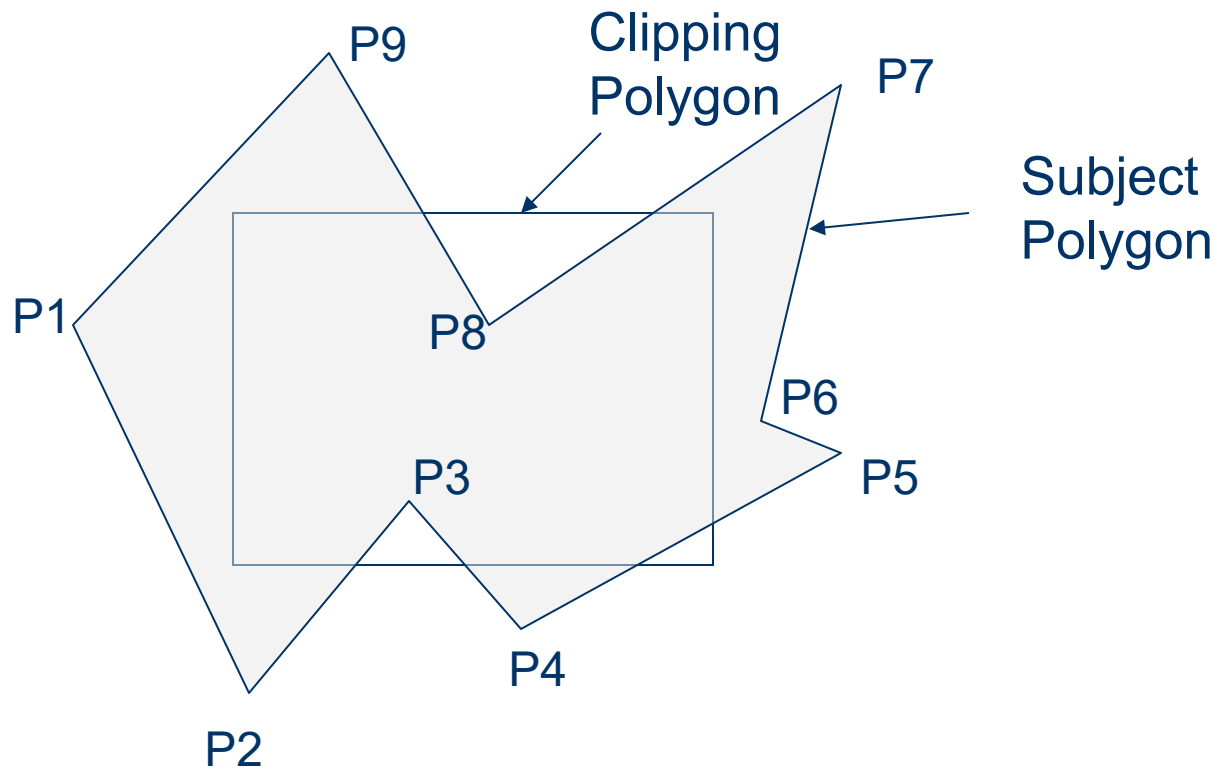
P_{i-1} is left and P_i is right



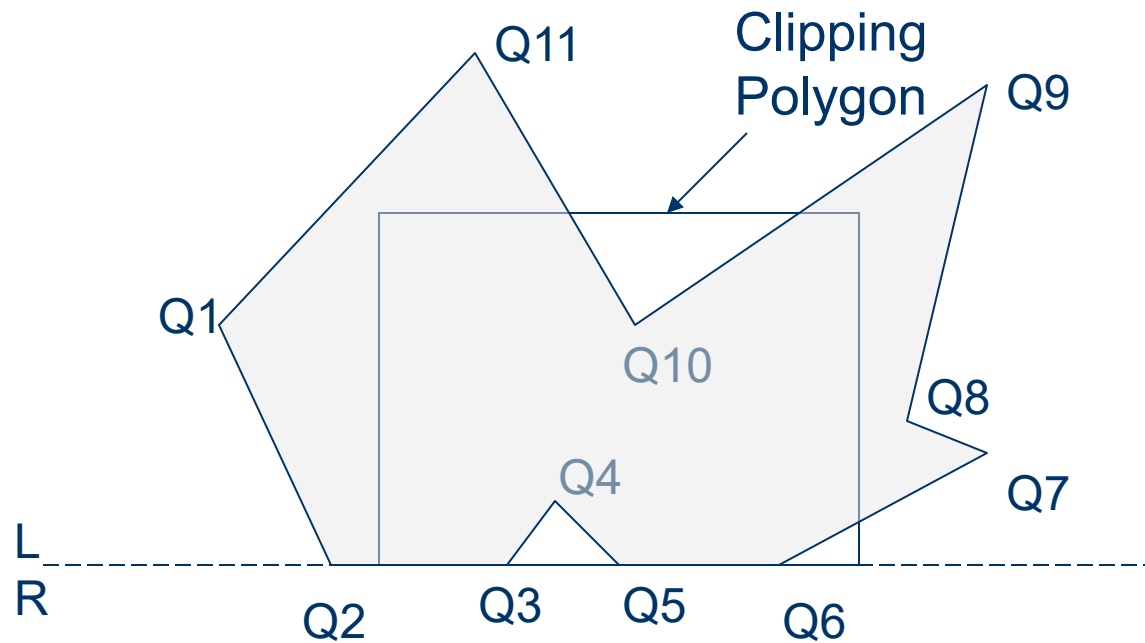
P_{i-1} is right and P_i is left



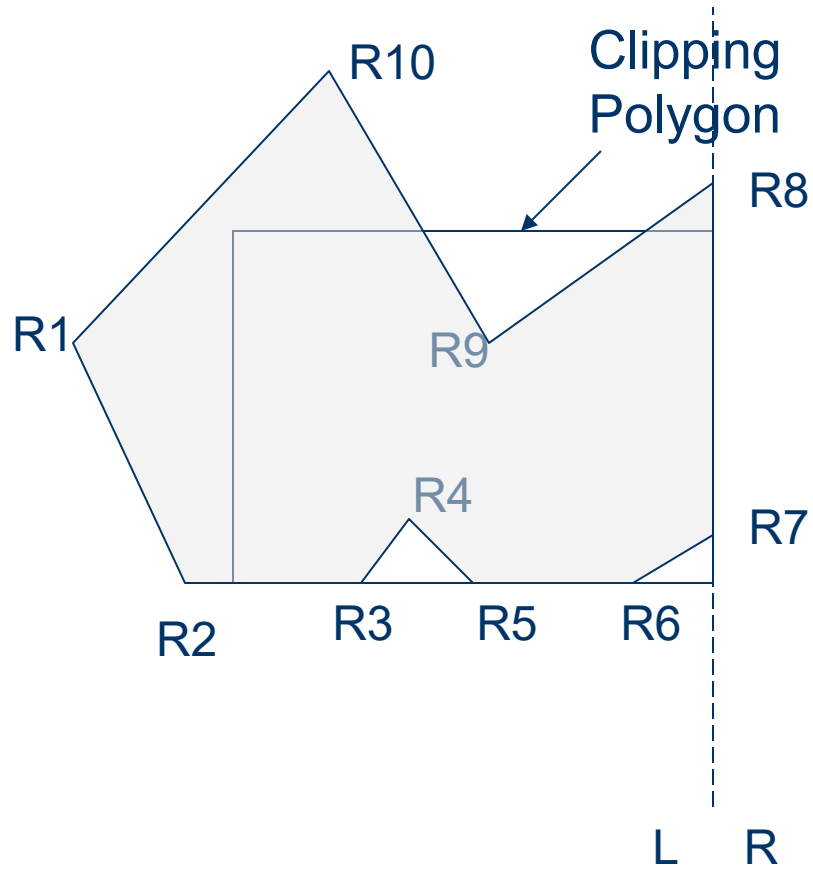
Example



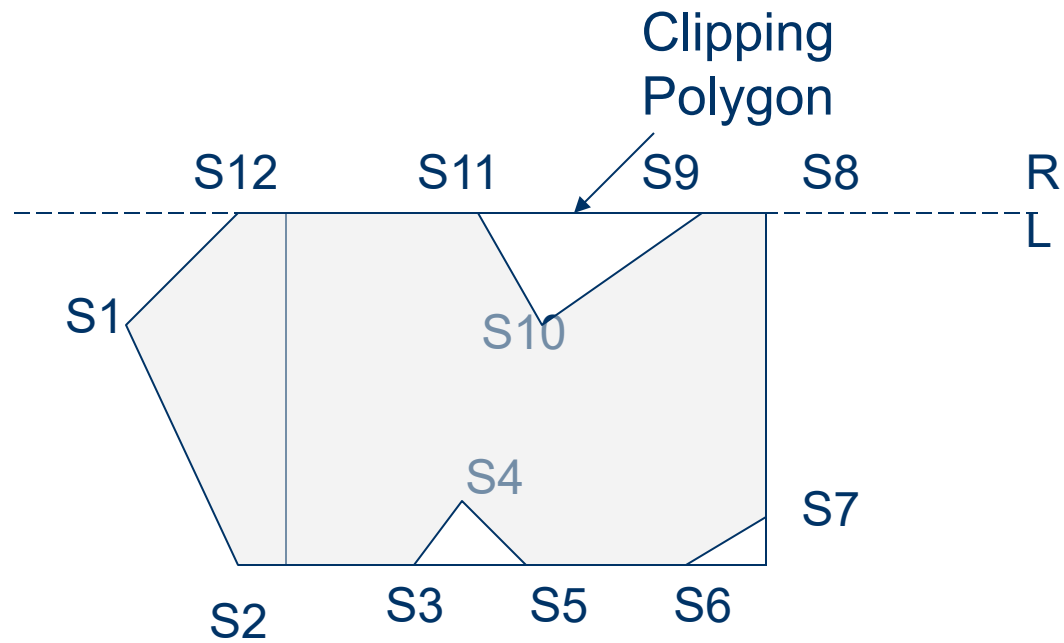
Example



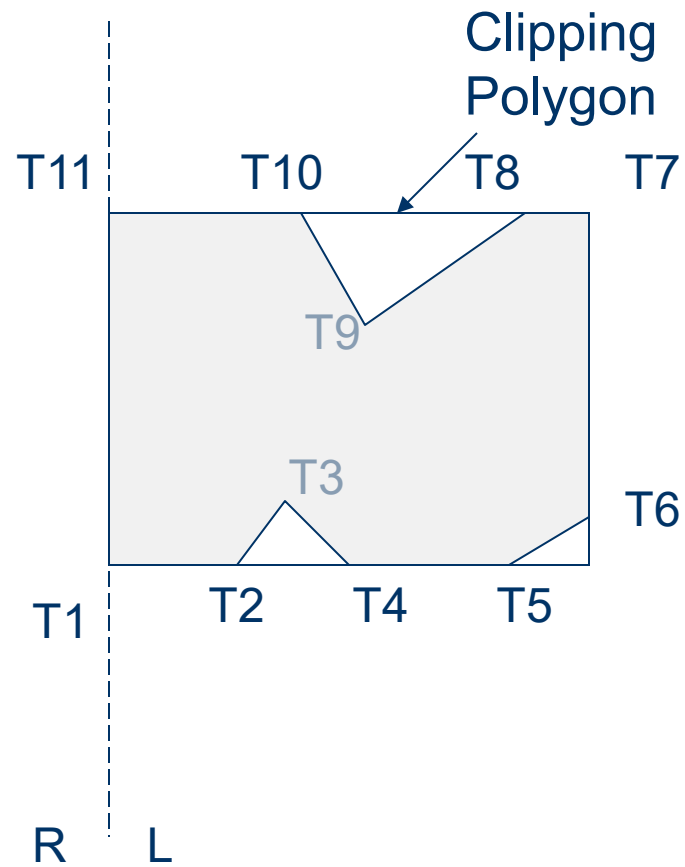
Example



Example



Example



Point-to-line test

- A very general test to determine if a point p is “inside” a line L or plane L for 3D, defined by q and n :

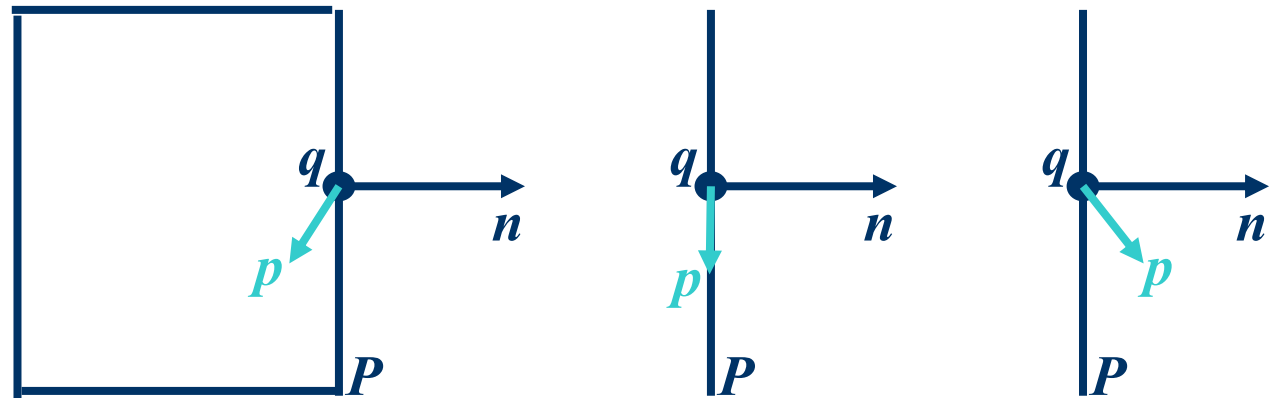
$(p - q) \cdot n < 0$: p inside L

$(p - q) \cdot n = 0$: p on L

$(p - q) \cdot n > 0$: p outside L

Remember: $A \cdot B = |A| |B| \cos(\theta) = A_x \cdot B_x + A_y \cdot B_y$

θ = angle between A and B



Finding Line- edge Intersections

- Use parametric definition of Line and edge:

$$L(t) = P_0 + t(P_1 - P_0)$$

- Line intersects an clipping edge where $L(t)$ is on E

- q is a point on E
- n is normal to E

$$(L(t) - q) \cdot n = 0$$

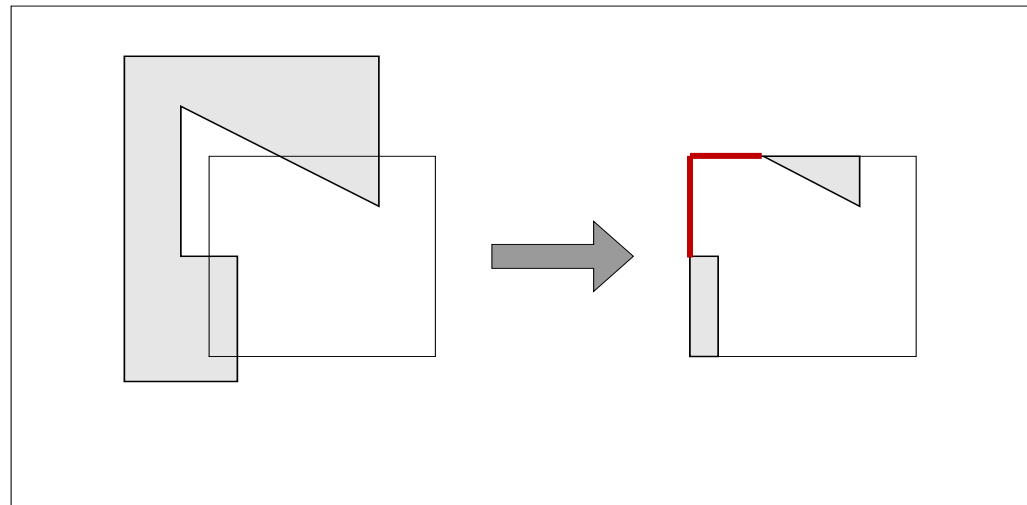
$$(P_0 + t(P_1 - P_0) - q) \cdot n = 0$$

$$t = [(q - P_0) \cdot n] / [(P_1 - P_0) \cdot n]$$

- The intersection point $I = L(t)$ for this value of t

Unwanted effect

- The Sutherland-Hodgman algorithm correctly clips **convex** polygons, but **concave** polygons may be displayed with extraneous lines
- Since there is only one output vertex list, the **last vertex** in the list is always **joined** to the **first vertex**.



Weiler-Atherton Polygon Clipping

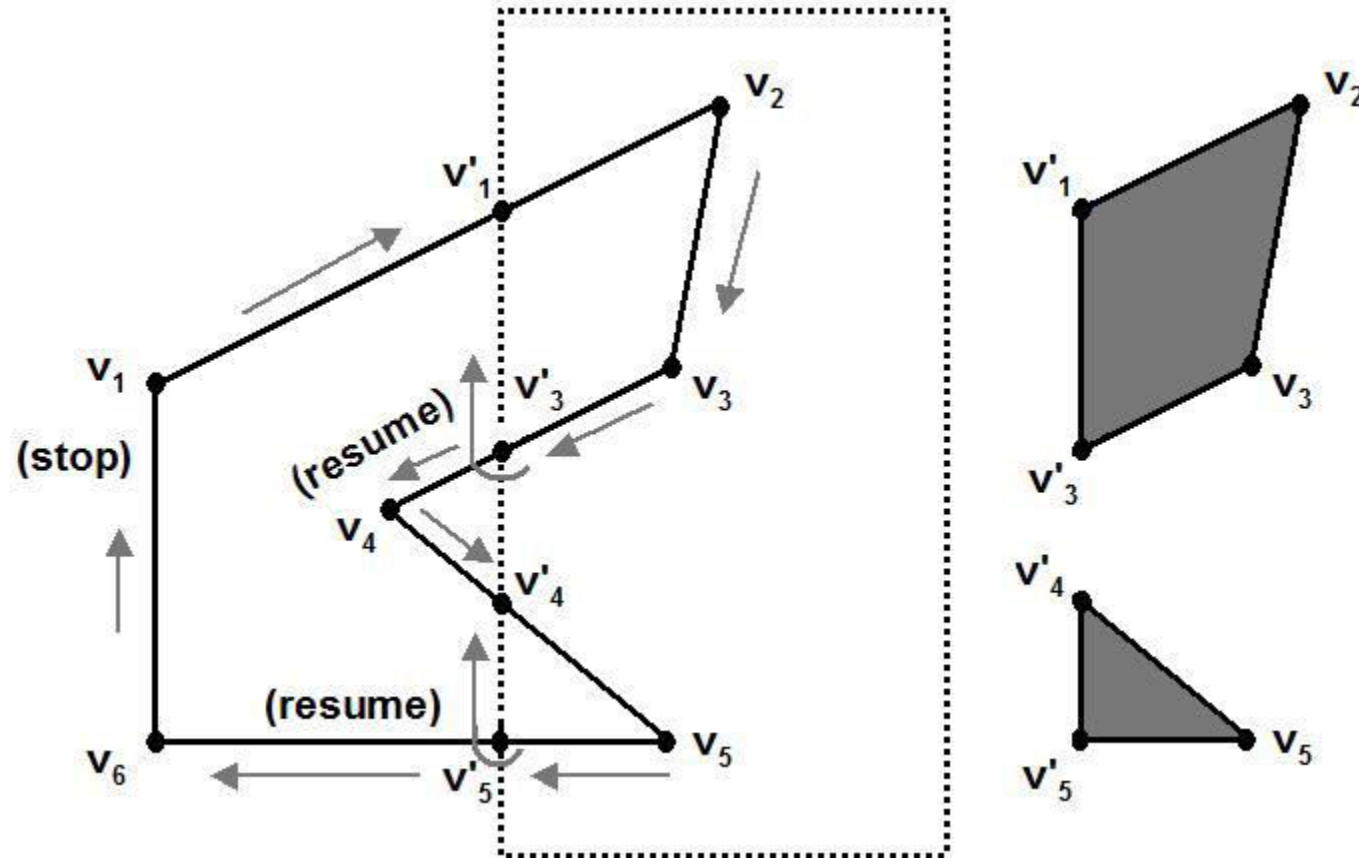
- can be used to clip either a **convex** or a **concave** polygon.
- The **basic idea** of this algorithm is that instead of proceeding around the polygon edges as vertices are processed, we will **follow** the **window boundaries**.
- The path we follow depends on:
 - polygon-processing direction (**clockwise** or **counterclockwise**)
 - The pair of polygon vertices
 - **outside-to-inside** or an **inside-to-outside**.

Weiler-Atherton Polygon Clipping

- For clockwise processing of polygon vertices, we use the following rules:
- For an **outside-to-inside** pair of vertices, **follow polygon boundaries**.
- For an **inside-to-outside** pair of vertices, **follow window boundaries** in a **clockwise** direction.

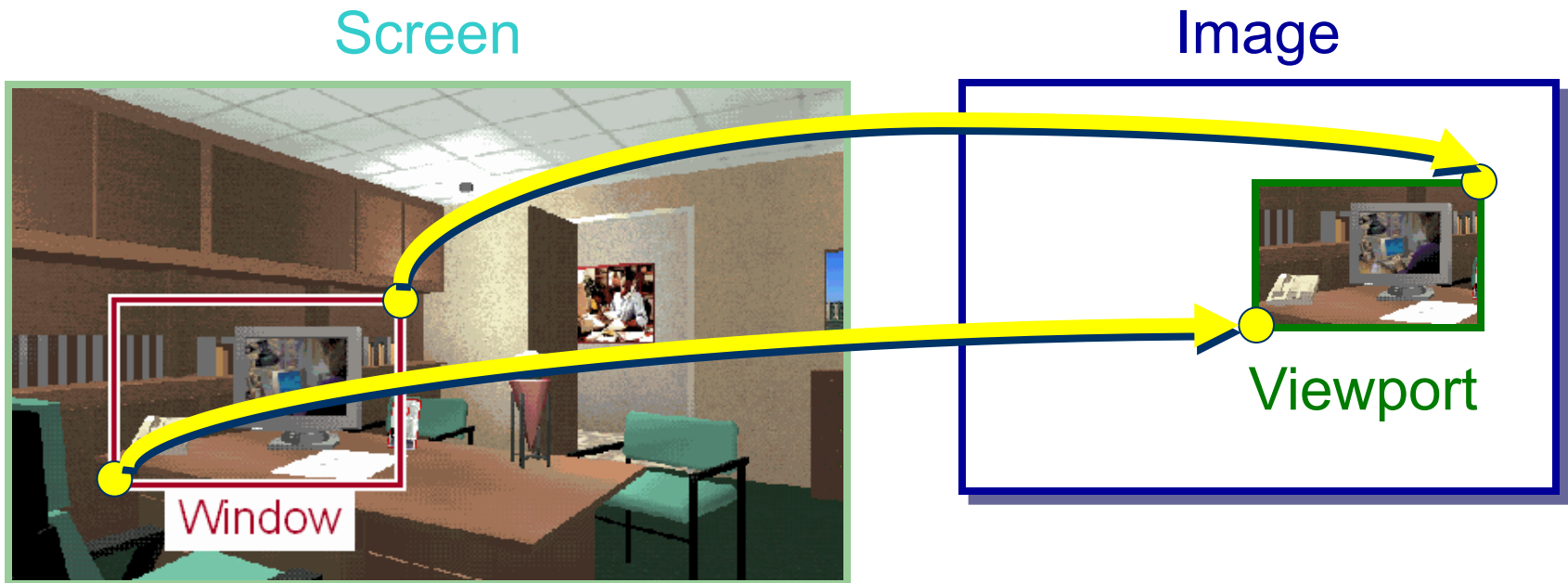
Weiler-Atherton Polygon Clipping

- Details is SS



Viewport Transformation

- Transform 2D Geometric Primitives from Screen Coordinate System (Projection Coordinates) to Image Coordinate System (Device Coordinates)

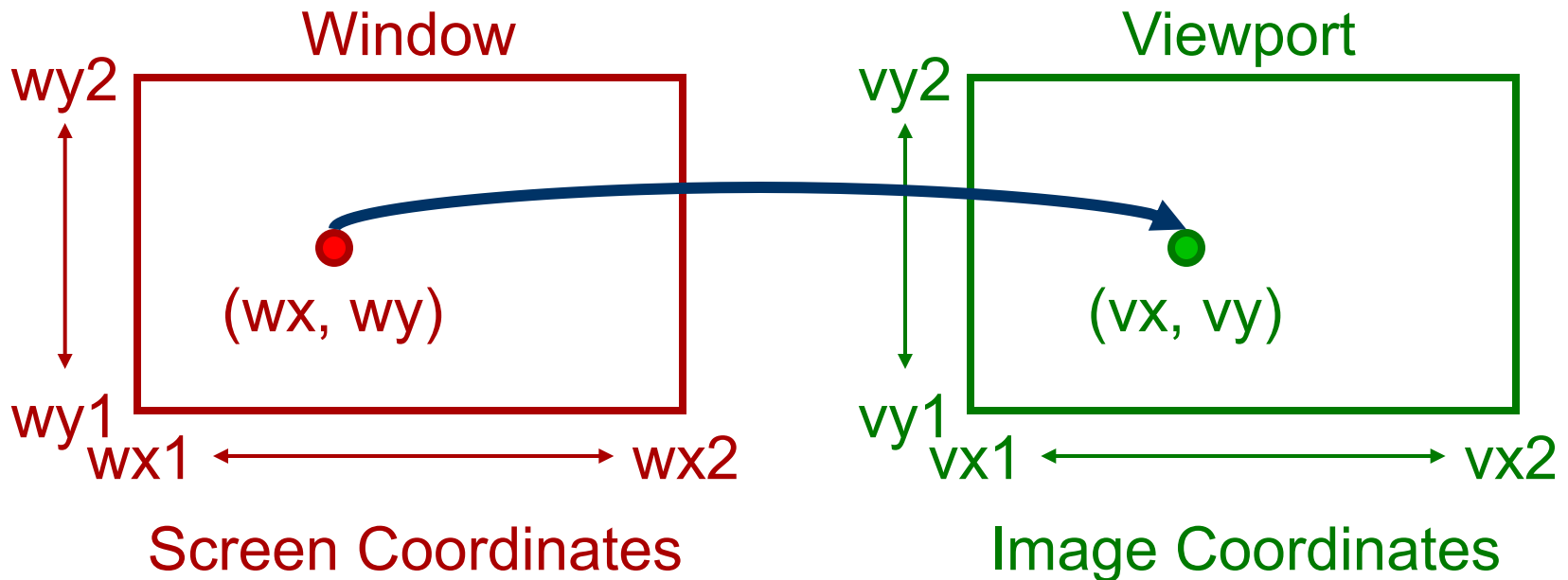


Window vs. Viewport

- Window
 - World-coordinate area selected for display
 - What is to be viewed
- Viewport
 - Area on the display device to which a window is mapped
 - Where it is to be displayed

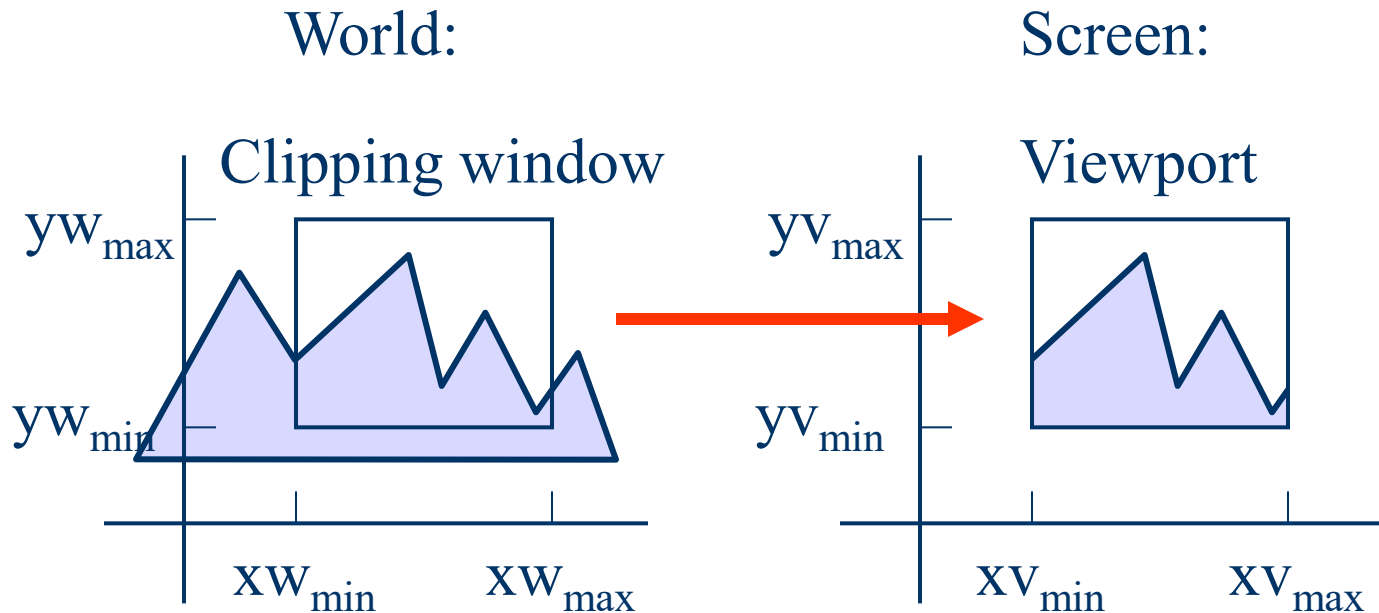
Viewport Transformation

- Window-to-Viewport Mapping



$$\begin{aligned} vx &= vx1 + (wx - wx1) * (vx2 - vx1) / (wx2 - wx1); \\ vy &= vy1 + (wy - wy1) * (vy2 - vy1) / (wy2 - wy1); \end{aligned}$$

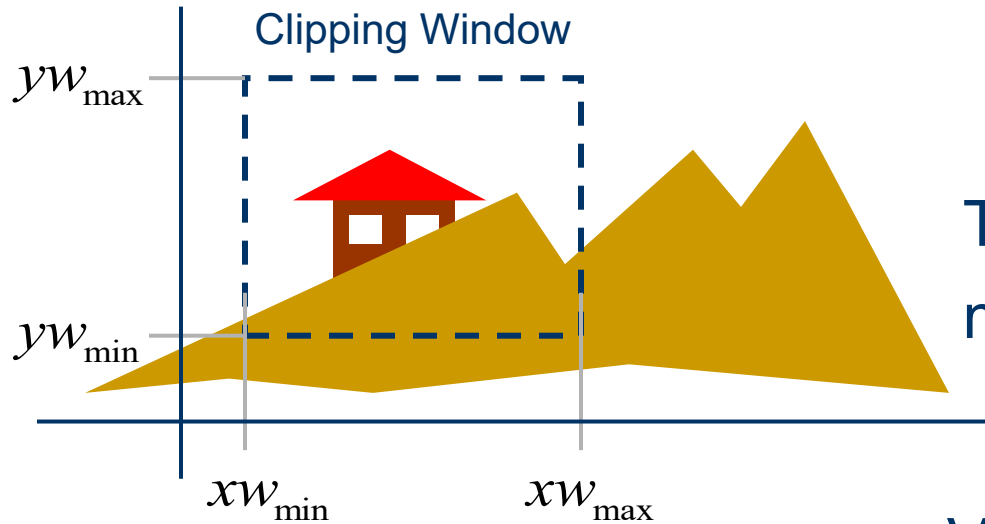
Viewport Transformation



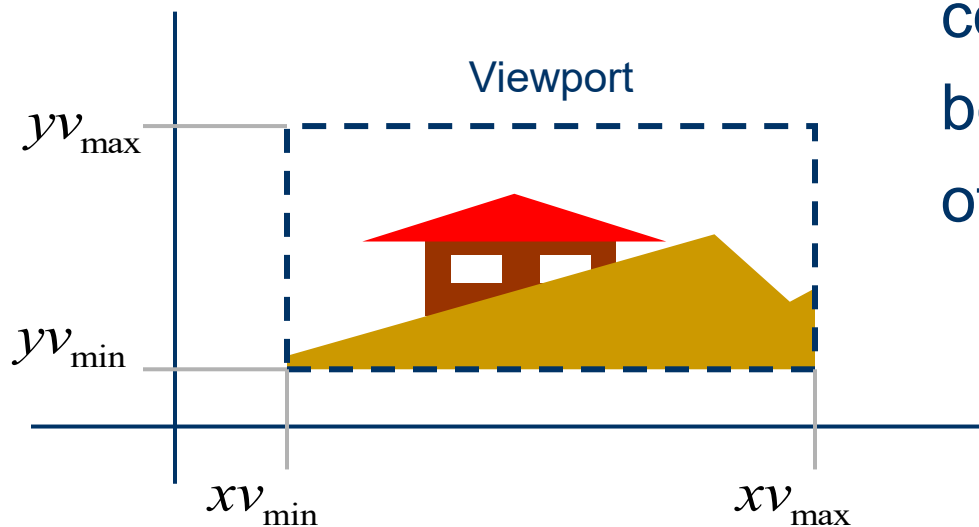
Clipping window:
What do we want to
see?

Viewport:
Where do we want to see it?

Viewport Transformation



The clipping window is mapped into a viewport.



Viewing world has its own coordinates, which may be a non-uniform scaling of world coordinates.

Viewport Coordinates

Summary

- Objects within a scene must be clipped to display the scene in a window
- Because there are can be so many objects clipping must be extremely efficient
- The Cohen-Sutherland algorithm can be used for line clipping
- The Sutherland-Hodgman algorithm can be used for area clipping

Illumination Model

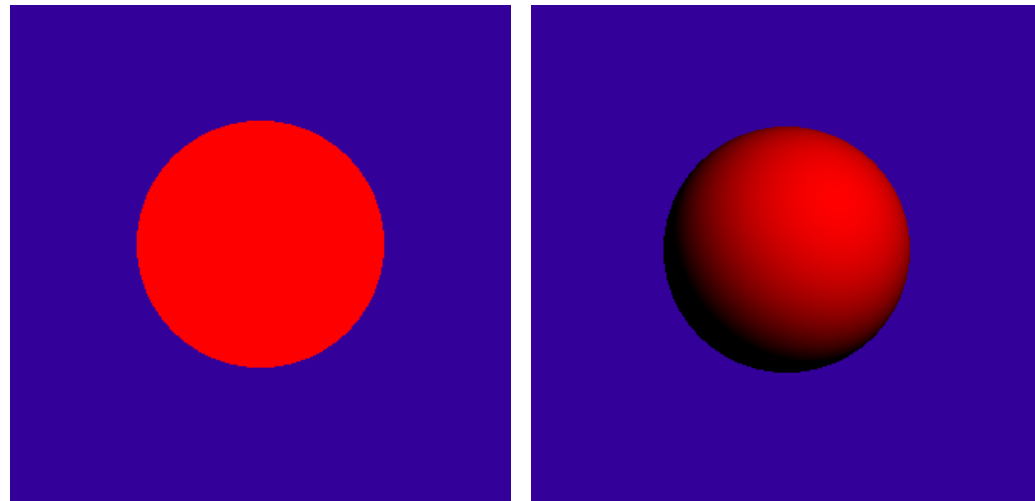
PROF. DR. SK. MD. MASUDUL AHSAN

CSE, KUET

A solid green horizontal bar at the bottom of the slide.

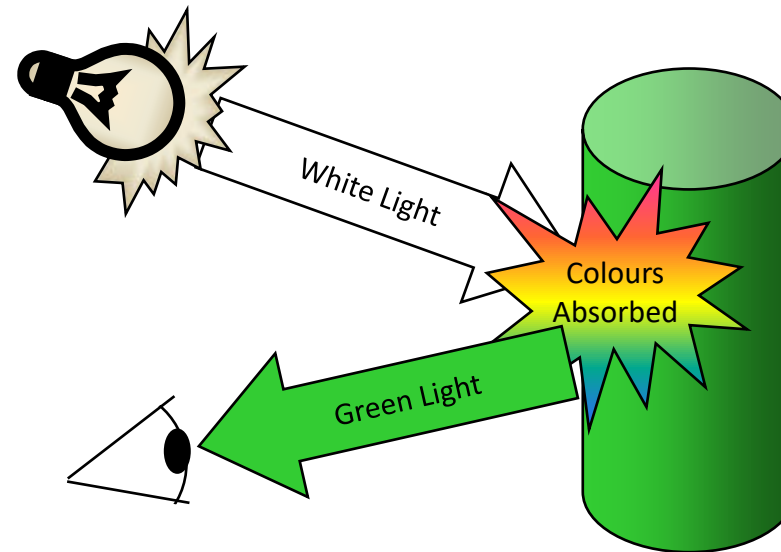
Why Lighting?

- ❑ Color is a fundamental attribute of our viewing experience. The perception of color arises from light energy entering our visual system.
- ❑ This complex process is relevant to computer graphics because a realistic image is one that seems indistinguishable from the light energy coming from a real scene.
- ❑ If we don't have lighting effects nothing looks three dimensional!

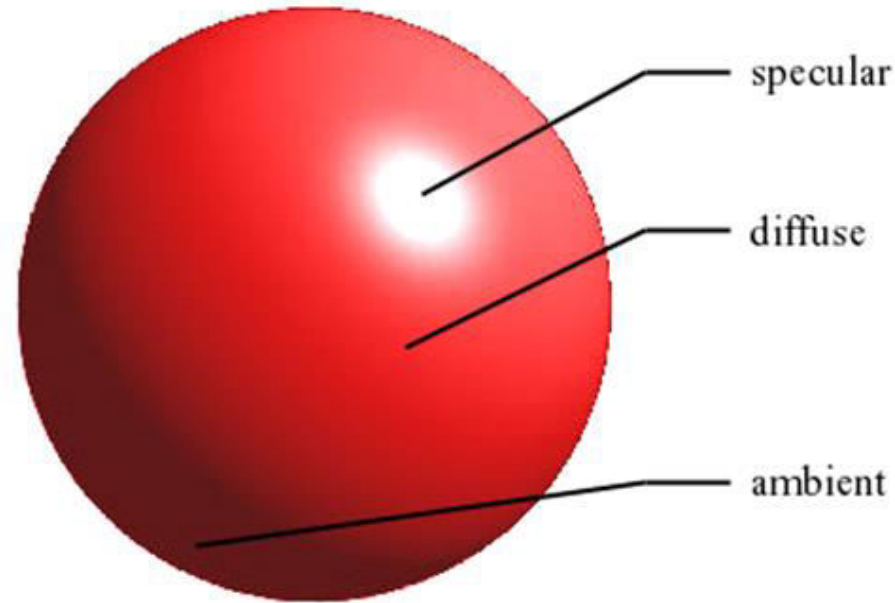


Basics of Reflected Light

- ❑ The colours that we perceive are determined by the nature of the light reflected from an object
- ❑ For example, if white light is shone onto a green object most wavelengths are absorbed, while green light is reflected from the object



Components of Reflections



Ambient – surface exposed to **indirect light** reflected from nearby objects.

Diffuse – reflection from **incident** light with equal intensity in all directions.
Depends on surface properties.

Specular – near **total** of the incident light around reflection angle.

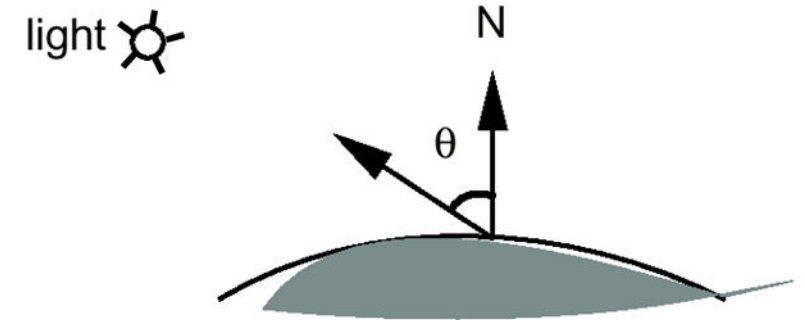
Illumination Model

- ❑ An illumination model, also called a lighting model is used to calculate the **intensity of light** that we should see at a given point on the surface of an object.
 - Done by simulating some light attributes
- ❑ Can be
 - Local illumination
 - Global illumination

Illumination Model

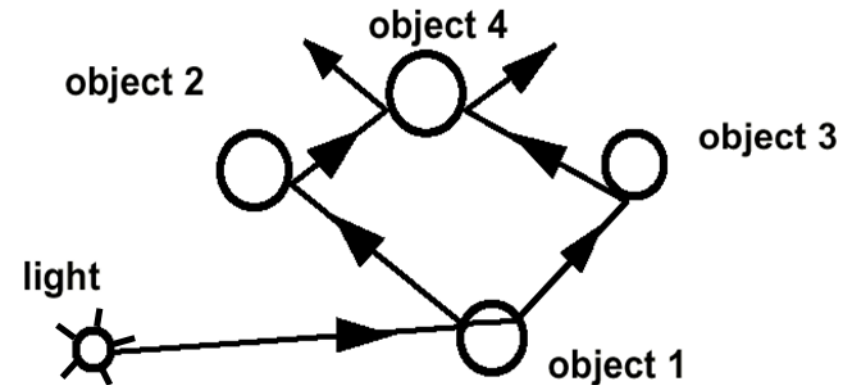
□ Local illumination

- Models only the direct illumination from the source
 - Does not consider light reaching after bouncing off other objects
- Only consider the light, the observer position, and the object material properties



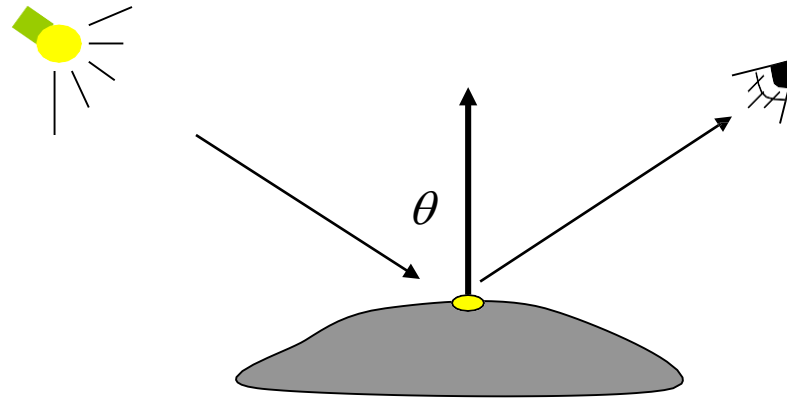
□ Global illumination

- take into account the interaction of light from all the surfaces in the scene



Local Illumination (Lighting) Model

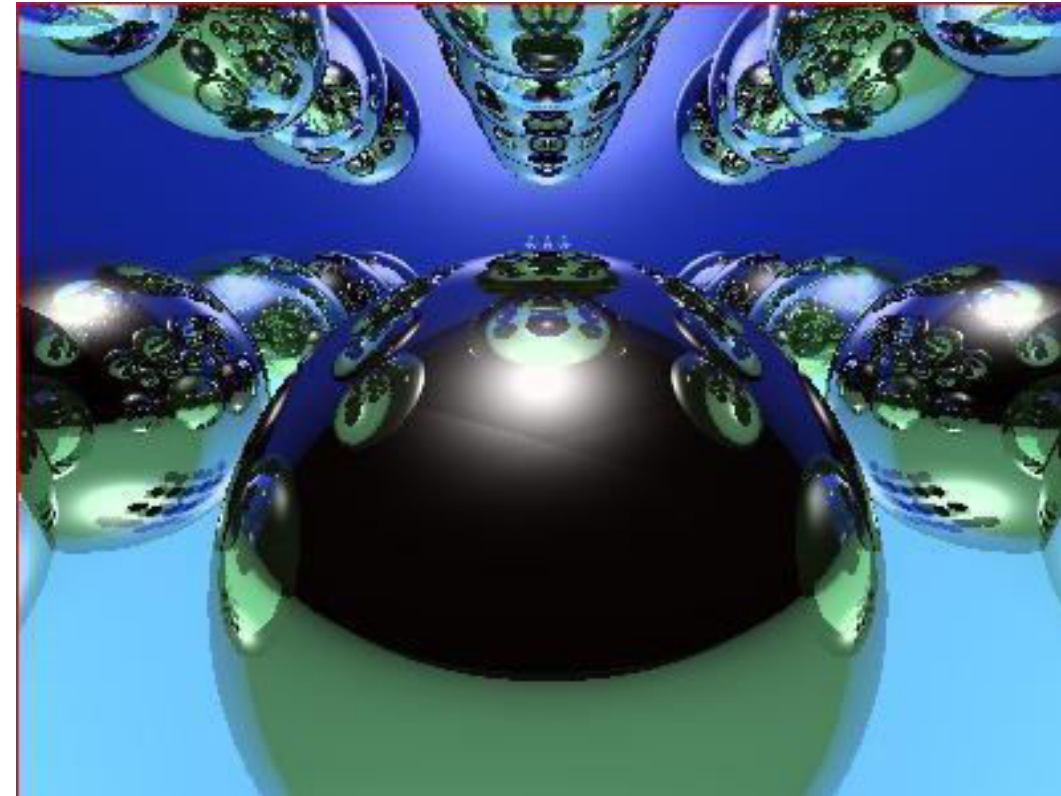
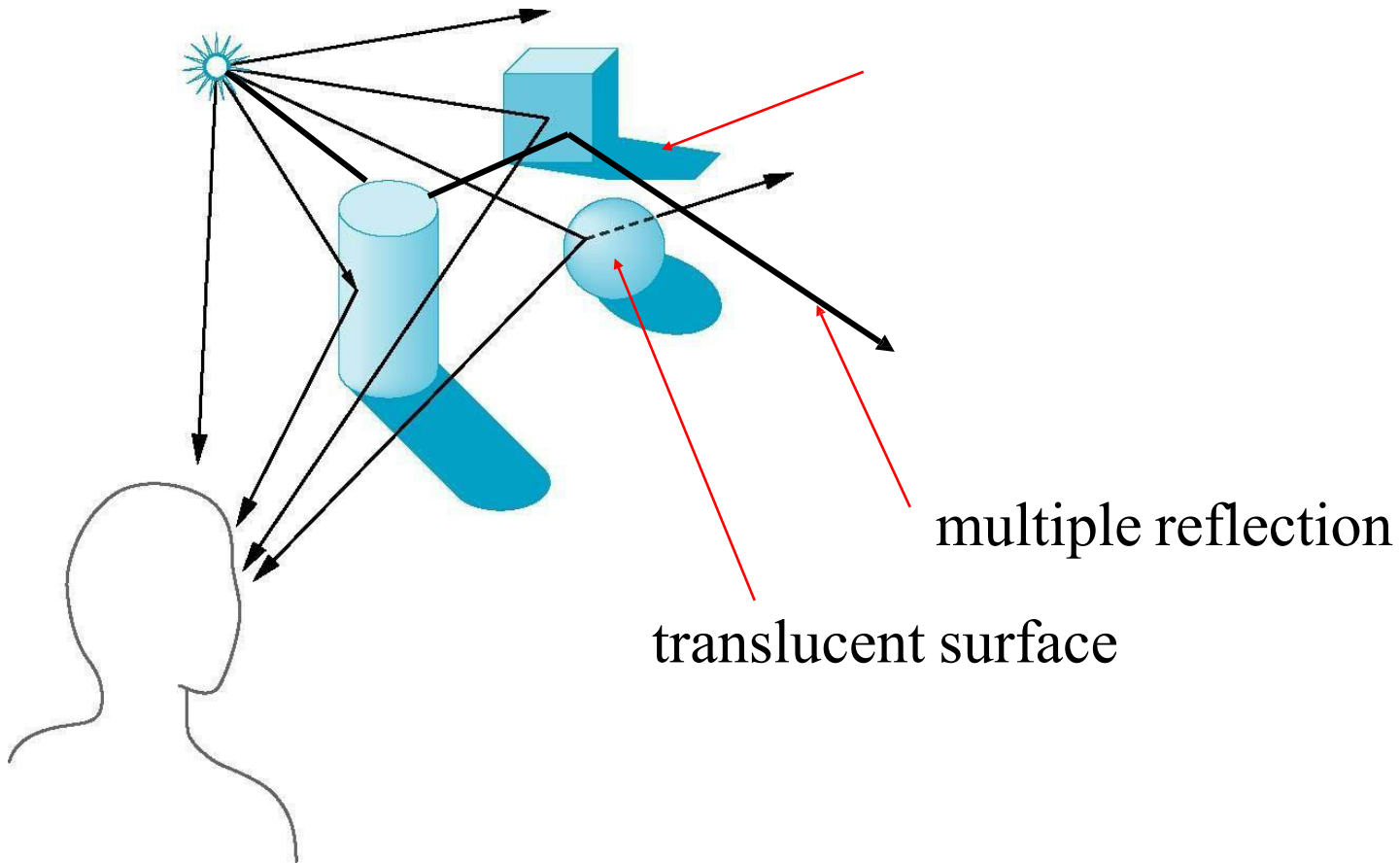
- ❑ One bounce!
 - Doesn't track inter-reflections, transmissions (e.g. OpenGL)



- Global Illumination (GI) is accurate, looks real
 - But raster graphics pipeline (e.g. OpenGL) renders each polygon independently (local rendering), no GI

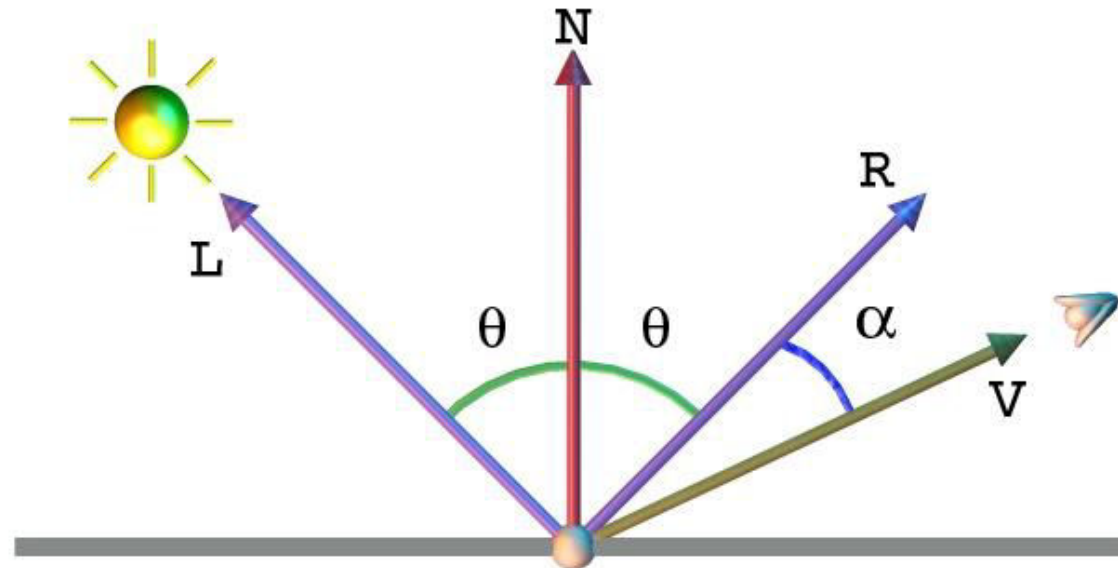
Global Illumination (Lighting) Model

- ❑ interaction of light from all surfaces in the scene (track multiple bounces)



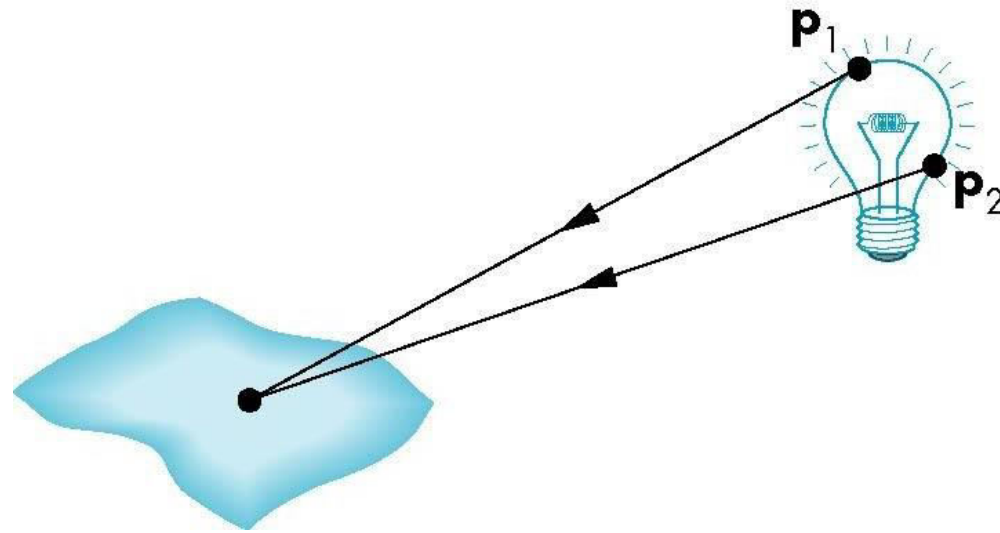
Things to Model

- ❑ Light sources
 - What color, intensity, lines through space
- ❑ Reflection of light off surfaces
 - How much light reflected in each direction
 - How are color and intensity changed
- ❑ Life on a surface
 - L: direction to light
 - N: normal vector
 - R: reflection of light about normal
 - V: direction to viewer (i.e. reflection direction of interest)



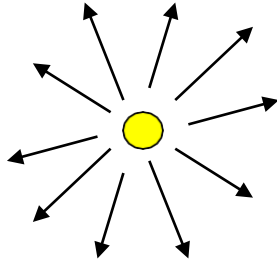
Light Sources

- ❑ General light sources are difficult to model (e.g. light bulb)
- ❑ Why? We must compute effect of light coming from all points on light source

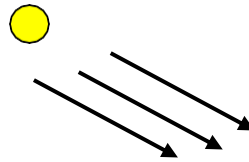


Light Sources Abstractions

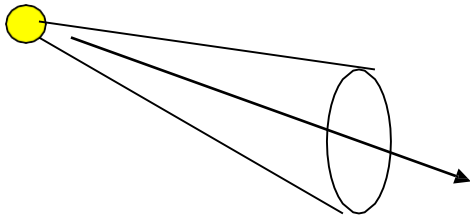
- ❑ We generally use simpler light sources
- ❑ Abstractions that are easier to model



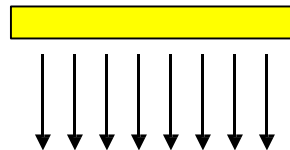
Point light



Directional light



Spot light



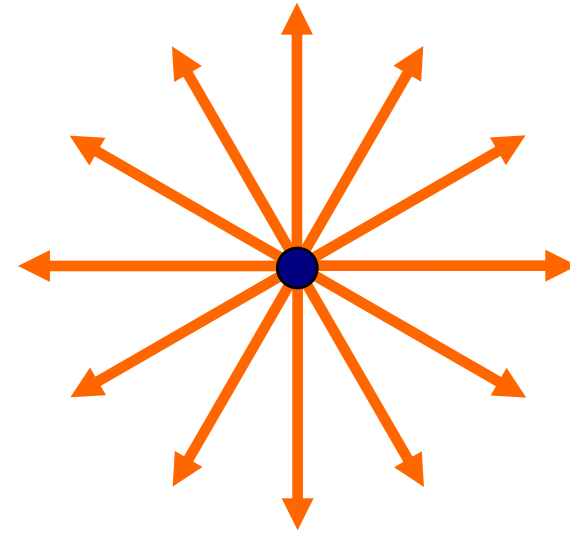
Area light

Light intensity can be **independent** or **dependent** of the distance between object and the light source

Point Light Sources

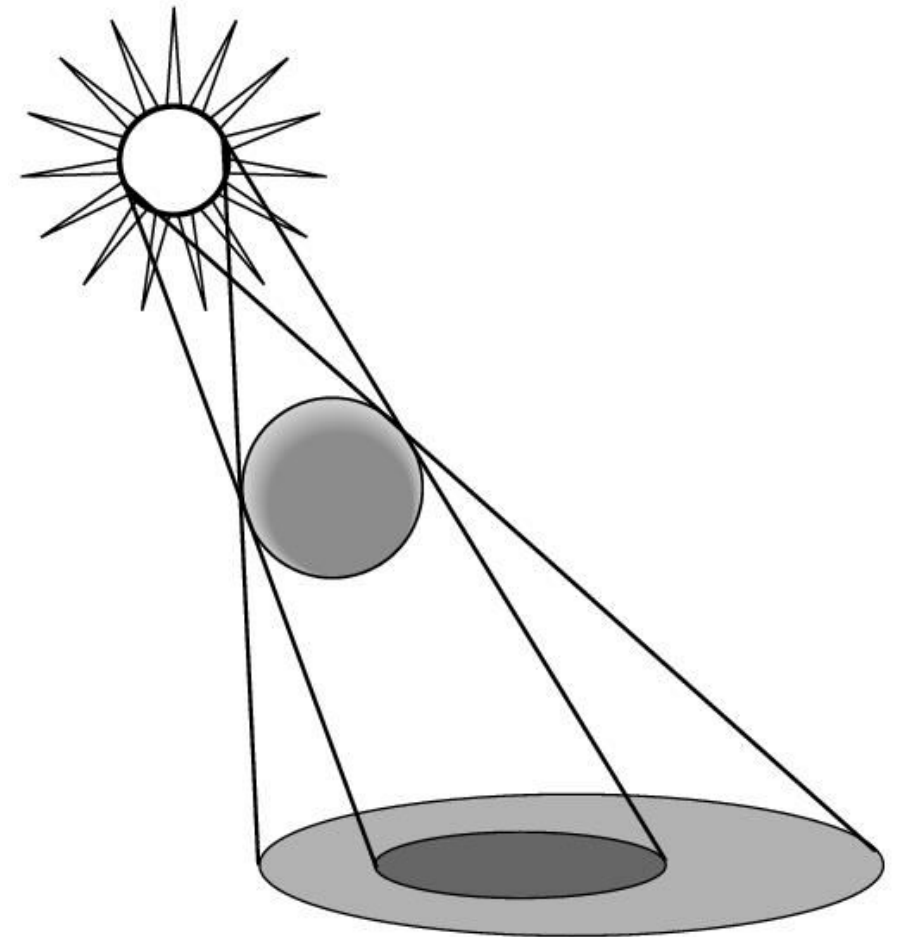
- ❑ Simplest model we can use for a light source
- ❑ Defined by:
 - The position (x,y,z) of the light
 - The RGB values for the colour of the light
- ❑ Radiates equal intensity in all direction

$$L = P_{\text{light}} - P_{\text{surface}}$$



Limitations of point lights

- ❑ Usually result in artificially high-contrast images
- ❑ Can generate umbra (full shadow) but not penumbra (partial shadow)
- ❑ *Area lights* generate softer shadows, but are usually used only in non-interactive graphics (e.g. raytracing or radiosity)



Directional Light Sources

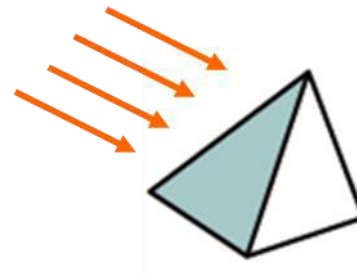
- ❑ A large light source, like the sun, can be modelled as a point light source
- ❑ However, it will have very little directional effect
 - i.e. All light rays are considered parallel



Specified by:

- direction (x,y,z)
- intensity (r,g,b)

$L = -\text{direction}$



Intensity Attenuation

- ❑ As light moves from a light source its intensity diminished
- ❑ At any distance d away from the light source the intensity diminishes by a factor of $\frac{1}{d^2}$
- ❑ This means that
 - a surface close to the light source (small d) **receives a higher incident** intensity from the source than a distant surface (large d).

Intensity Attenuation

- ❑ However, using the factor $1/d^2$ does not produce very good results so we use something different
- ❑ We use instead in inverse quadratic function of the form:

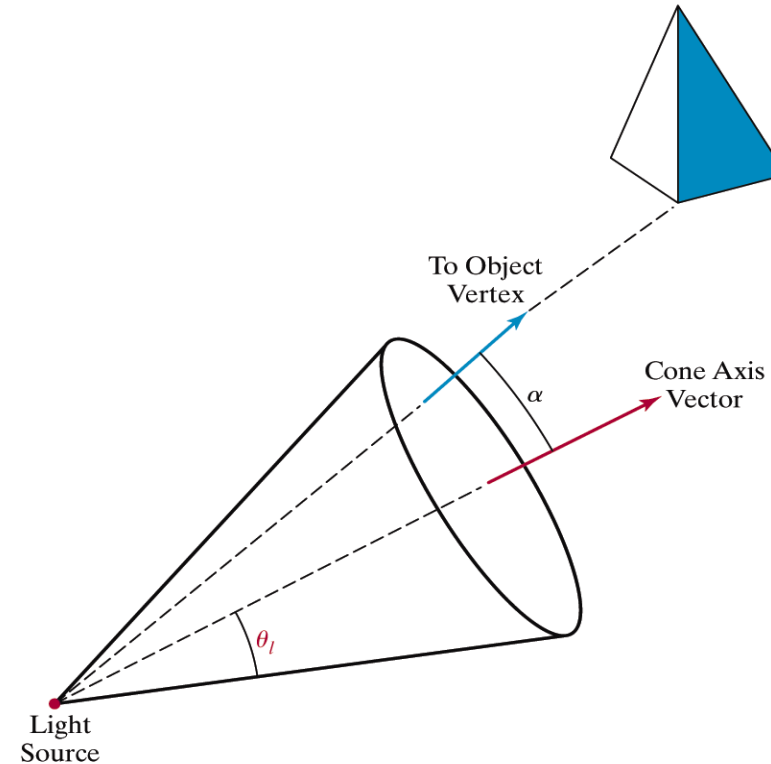
$$f_{\text{radial attenuation}}(d) = \begin{cases} 1.0, & \text{source at infinity} \\ \frac{1}{a_0 + a_1d + a_2d^2}, & \text{local source} \end{cases}.$$

- ❑ A user can then fiddle with the coefficients a_0 , a_1 , and a_2 , to obtain a variety of lighting effects
- ❑ The value of the constant term a_0 can be adjusted to prevent **f(d) from becoming too large when d is very small.**

Spotlights

- ❑ To turn a point light source into a spotlight we simply add a vector direction and an angular limit θ_l
- ❑ If V_{light} and V_{obj} are the unit vector
- ❑ Then

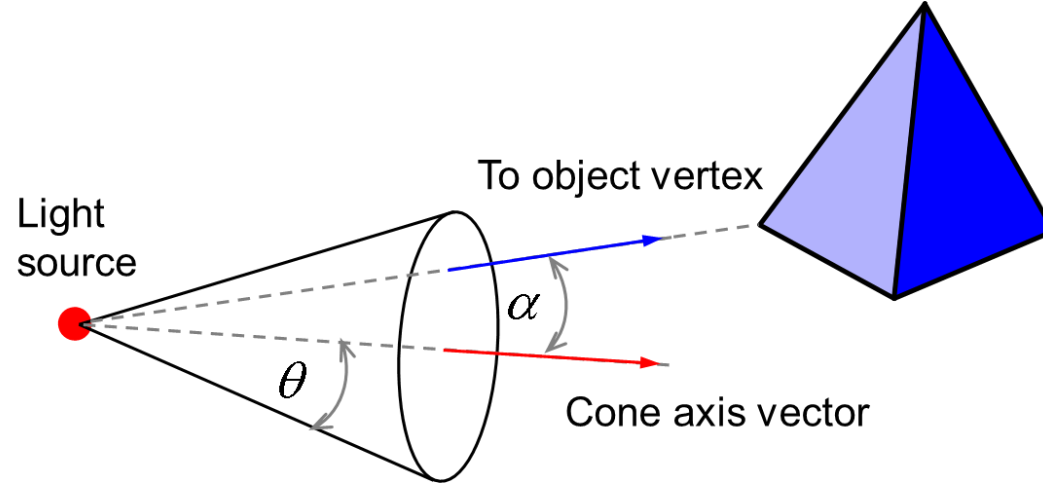
$$V_{obj} \cdot V_{light} = \cos \alpha$$



- ❑ If this angle is inside the light's angular limit then the object is within the spotlight

Spotlights - Angular Intensity Attenuation

- Points outside the cone are not illuminated (stay in dark).



- Angular attenuation is:

$$f_{\text{angular attenuation}}(\alpha) = \begin{cases} 1.0, & \text{source not a spotlight} \\ 0.0, & \mathbf{V}_{\text{obj}} \cdot \mathbf{V}_{\text{light}} = \cos \alpha < \cos \theta \\ \left(\mathbf{V}_{\text{obj}} \cdot \mathbf{V}_{\text{light}} \right)^a, & \text{otherwise} \end{cases}$$

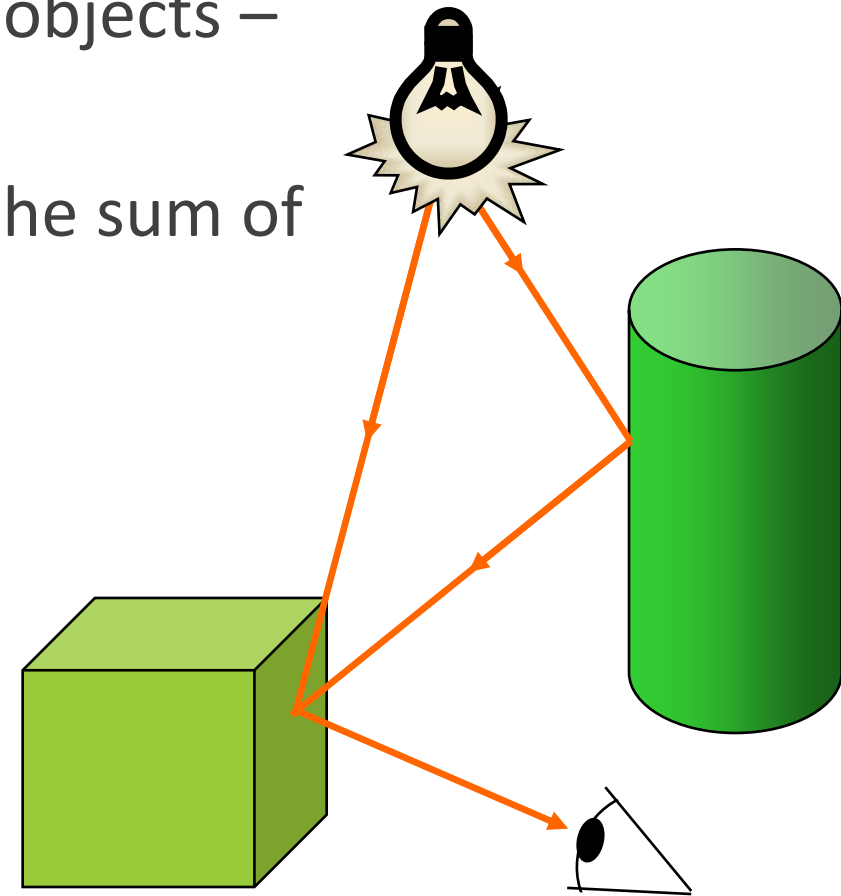
a is a constant

Phong Model

- ❑ **Phong model** is an empirical model developed in 1973 by Phong Bui Tuong
- ❑ Simple and fast method for calculating surface intensity at a given point
- ❑ Gives reasonably good results and is used in most graphics systems
- ❑ The important components are:
 - Ambient light
 - Diffuse reflection
 - Specular reflection
 - Emissive light
- ❑ For the most part we will consider only monochromatic light sources

Ambient Light

- ❑ A surface that is not exposed to direct light may still be lit up by reflections from other nearby objects – **ambient light**
- ❑ The total reflected light from a surface is the sum of the contributions from light sources and reflected light

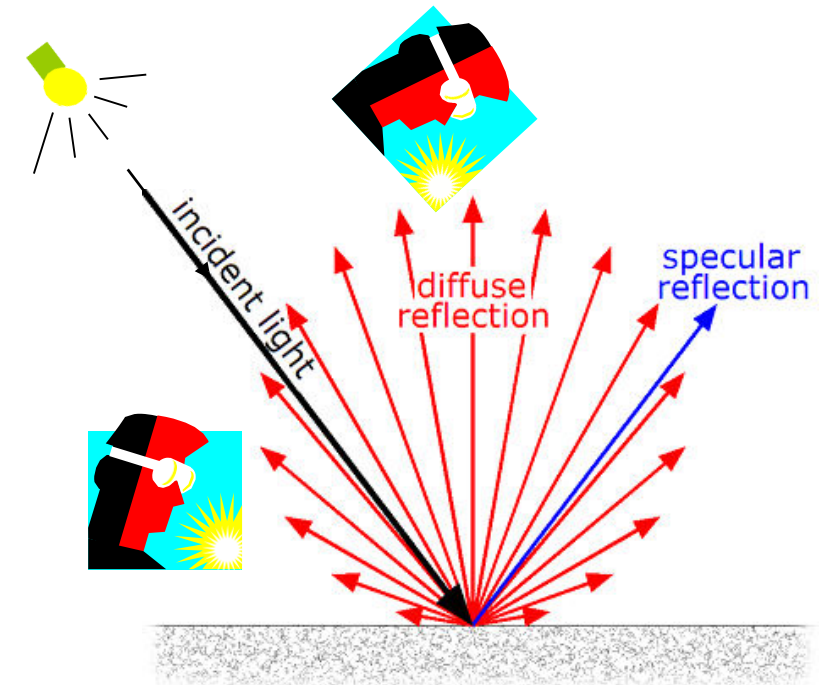


Ambient Light

- ❑ To incorporate background light Phong model simply set a general brightness level for a scene
- ❑ This approximates the global diffuse reflections (inter-reflections) from various surfaces within the scene
- ❑ This light intensity is simply denoted by I_a
- ❑ It is
 - Independent of the light positions or surface directions
 - Independent of location of viewer
 - Has no direction

Diffuse Reflection

- ❑ Surfaces that are rough or grainy tend to reflect light in all directions
- ❑ This scattered light is called **diffuse reflection**
- ❑ This type of reflection is called Lambertian Reflection (thus, Lambertian surfaces)
- ❑ The brightness of the surface is independent of the observer position (since the light is reflected in all directions equally)



Eye position does not matter

Diffuse Reflection (cont...)

- ❑ Surface's material property:
 - A parameter k_d is set for each surface
 - k_d determines the fraction of light that is to be scattered as diffuse reflections from that surface
- ❑ This parameter is known as the **diffuse-reflection coefficient** or the **diffuse reflectivity**
- ❑ k_d is assigned a value between 0.0 and 1.0
 - 0.0: dull surface that absorbs almost all light
 - 1.0: shiny surface that reflects almost all light

Diffuse Reflection – Ambient Light

❑ For background lighting effects we can assume that every surface is fully illuminated by the scene's ambient light I_a

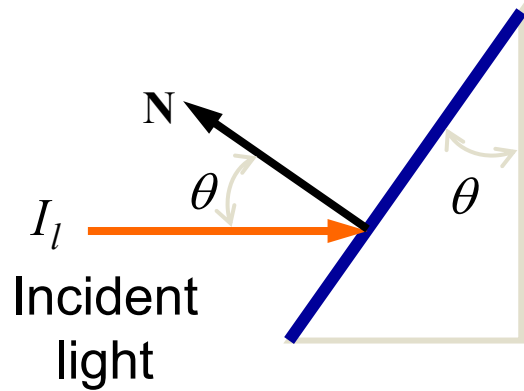
❑ Therefore the ambient contribution to the diffuse reflection is given as:

$$I_{ambdiff} = k_d I_a$$

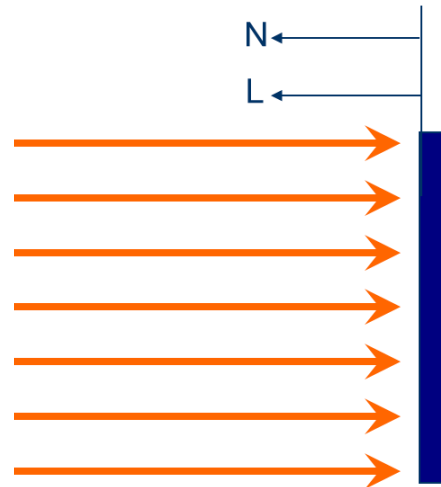
❑ Ambient light alone is not enough to render a scene or very uninteresting so we need some other lights in a scene as well

Diffuse Reflection (cont...)

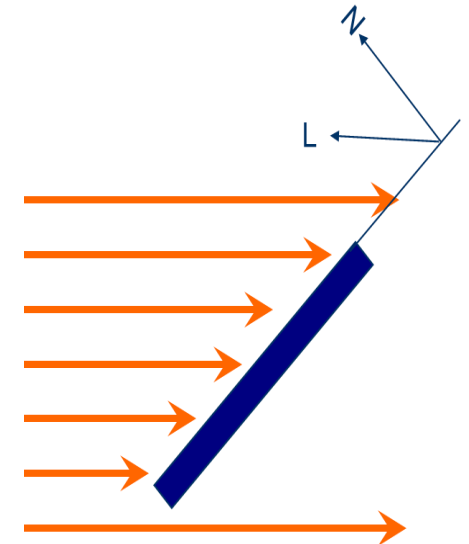
- When a surface is illuminated by a light source, the amount of incident light depends on the orientation of the surface relative to the light source direction



θ = angle of incidence



Receive more light



Receive less light

Diffuse Reflection (cont...)

- So the amount of incident light on a surface is given as:

$$I_{l,incident} = I_l \cos \theta$$

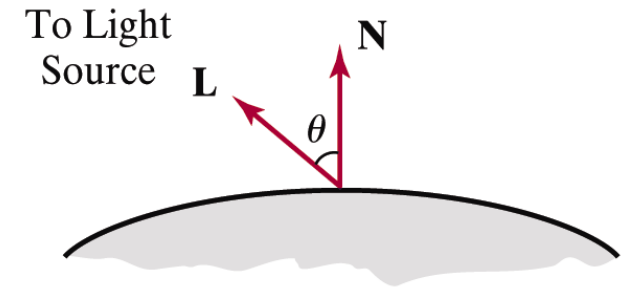
- So we can model the diffuse reflections as:

$$\begin{aligned} I_{l,diff} &= k_d I_{l,incident} \\ &= k_d I_l \cos \theta \end{aligned}$$

- if N and L are unit vector,

$$N \cdot L = \cos \theta$$

$$I_{l,diff} = \begin{cases} k_d I_l (N \cdot L) & \text{if } N \cdot L > 0 \\ 0 & \text{if } N \cdot L \leq 0 \end{cases}$$



Combining Ambient And Diffuse Reflections

- ❑ To combine the diffuse reflections arising from ambient and incident light most graphics packages use two separate diffuse-reflection coefficients:
 - k_a for ambient light
 - k_d for incident light
- ❑ The total diffuse reflection equation for a single point source can then be given as:

$$I_{diff} = \begin{cases} k_a I_a + k_d I_l (N \cdot L) & \text{if } N \cdot L > 0 \\ k_a I_a & \text{if } N \cdot L \leq 0 \end{cases}$$

Ambient Light Example

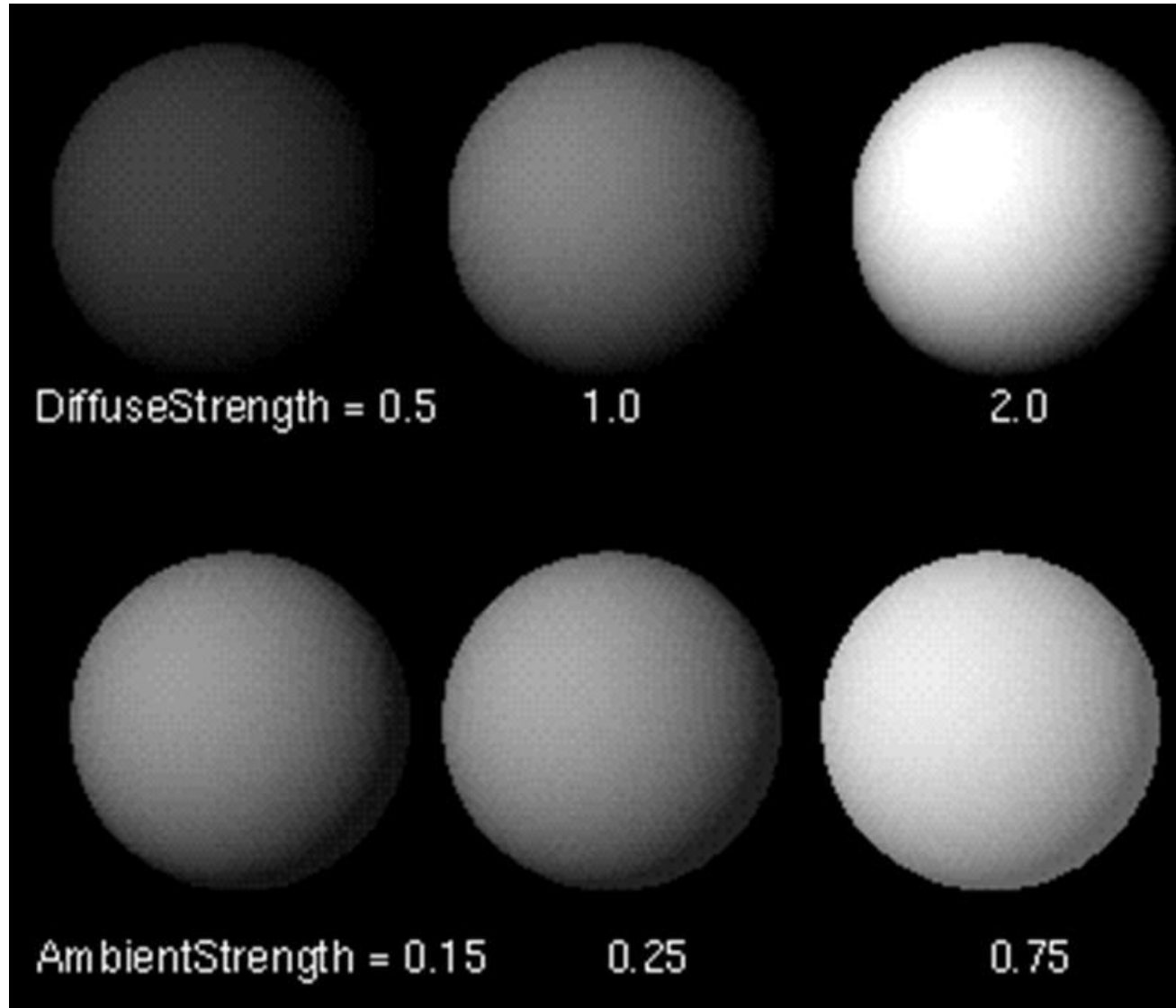


Ambient: background light,
scattered by environment

Diffuse Lighting Example



Examples

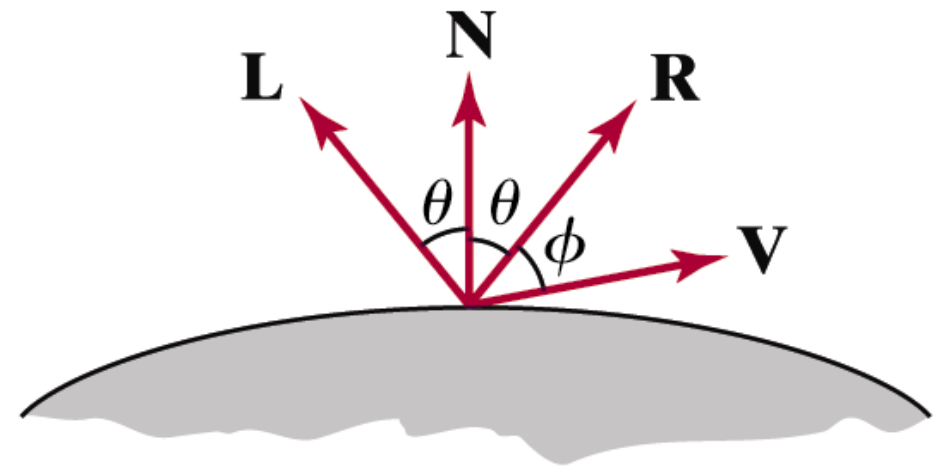


Ambient = 0

Ambient = 1

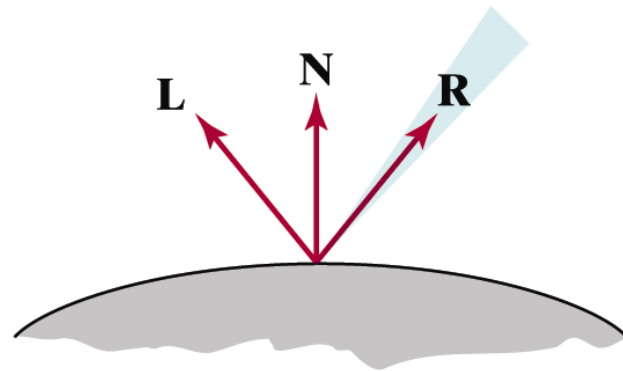
Specular Reflection

- Additionally to diffuse reflection we see a **highlight, or bright spot**, on objects (such as polished metal, apple ...) at **certain viewing directions**.
- This phenomenon, called *specular reflection*,
- It is the result of total, or near total reflection of the incident light in a concentrated region around the specular reflection angle.

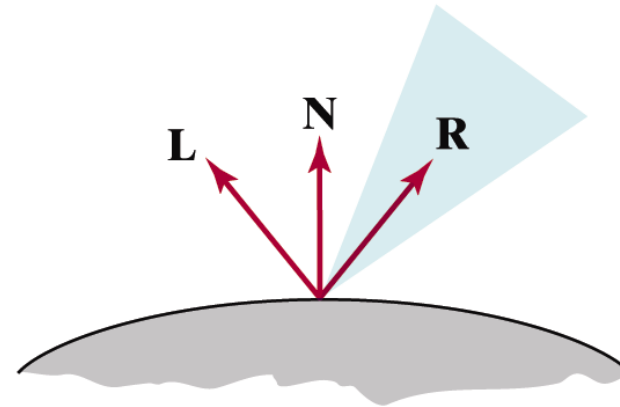


Specular Reflection (cont...)

- ❑ A perfect mirror reflects light only in the specular-reflection direction
- ❑ Other objects exhibit specular reflections over a finite range of viewing positions around vector R



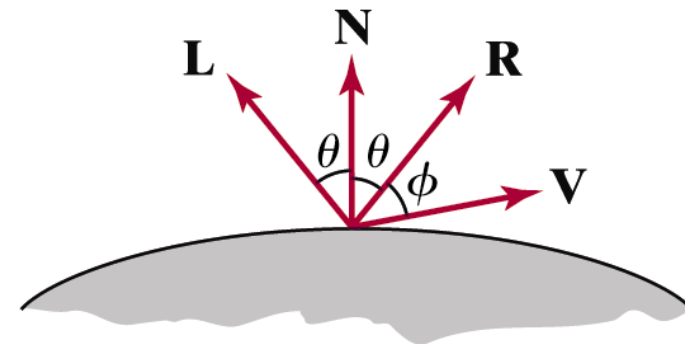
Shiny Surface
(Large n_s)



Dull Surface
(Small n_s)

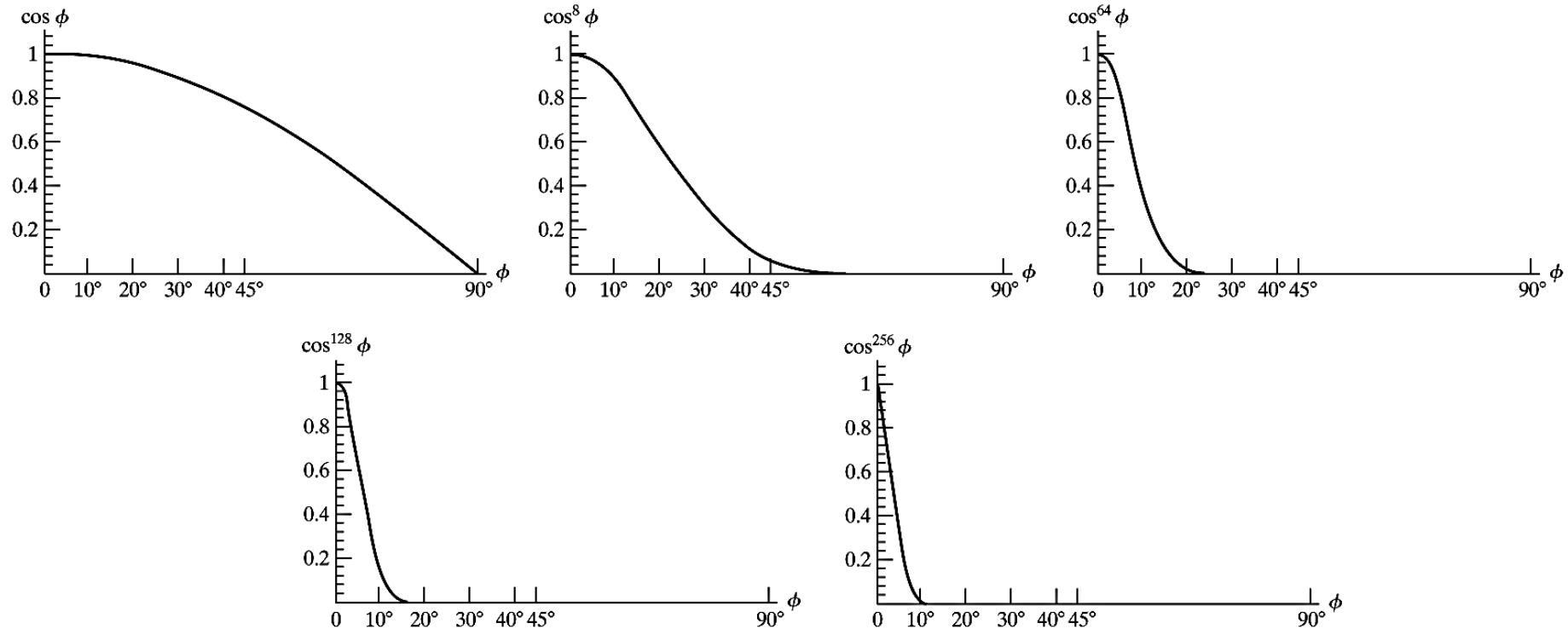
Specular Reflection Model

- ❑ The Phong model sets the intensity of specular reflection as proportional to the angle between the viewing vector and the specular reflection vector
- So, the specular reflection intensity is proportional to
 - The angle ϕ can be varied between 0° and 90° so that $\cos\phi$ varies from 1.0 to 0.0
- The **specular-reflection exponent**, n_s is determined by the type of surface we want to display
 - Shiny surfaces have a very large value (>100)
 - Rough surfaces would have a value near 1



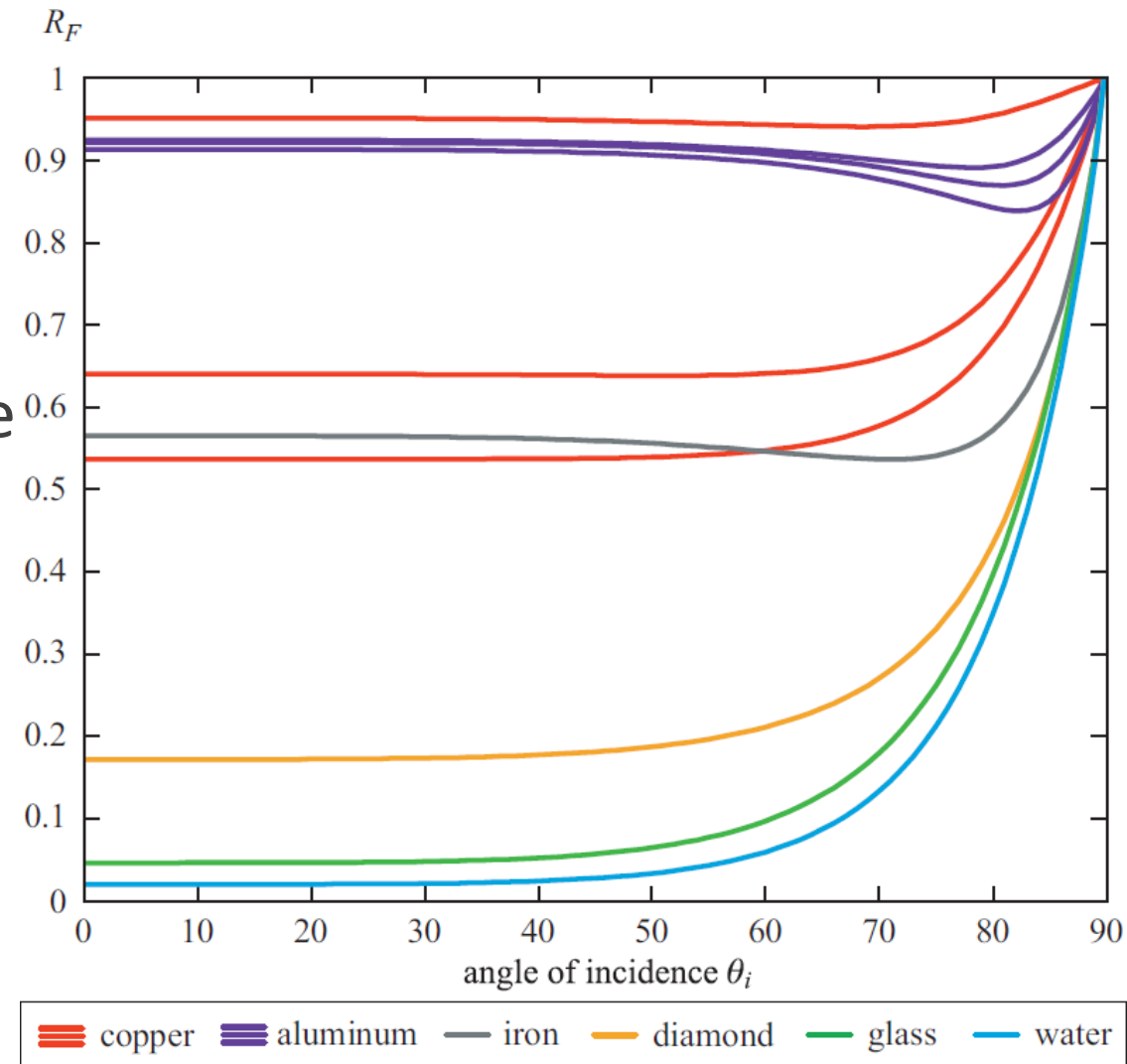
Specular Reflection Model

- The effect of n_s on the angular range in which we can expect to see specular reflections



Specular Reflection Model

- ❑ For some materials the amount of specular reflection depends heavily on the angle of the incident light
- ❑ *Fresnel's Laws of Reflection* describe in great detail how specular reflections behave



Specular Reflection Model

- ❑ *Fresnel's Laws of Reflection* describe in great detail how specular reflections behave
 - if the material is a dielectric, in other words, if it is not a conductor/metal. A specular reflection is only a reflection of a light source off of the surface of an object.



- Some metals have a color (bronze, gold, copper) and are purely reflective. Metals change the color of the specular reflections with their own colors

Specular Reflection Model

- ❑ Phong's original formula for the specular term:
 - $W(i)[\cos \phi]^n$
 - ϕ is the angle between the view and specular reflection directions.
 - “ $W(i)$ is a function which gives the ratio of the specular reflected light and the incident light as a function of the incident angle i .”
 - Ranges from 10 to 80 percent.
- ❑ However, we don't need to worry about this and instead approximate the specular effects with a constant specular reflection coefficient k_s

Specular Reflection Model

- So the specular reflection intensity is given as:

$$I_{l,spec} = k_s I_l \cos^{n_s} \phi$$

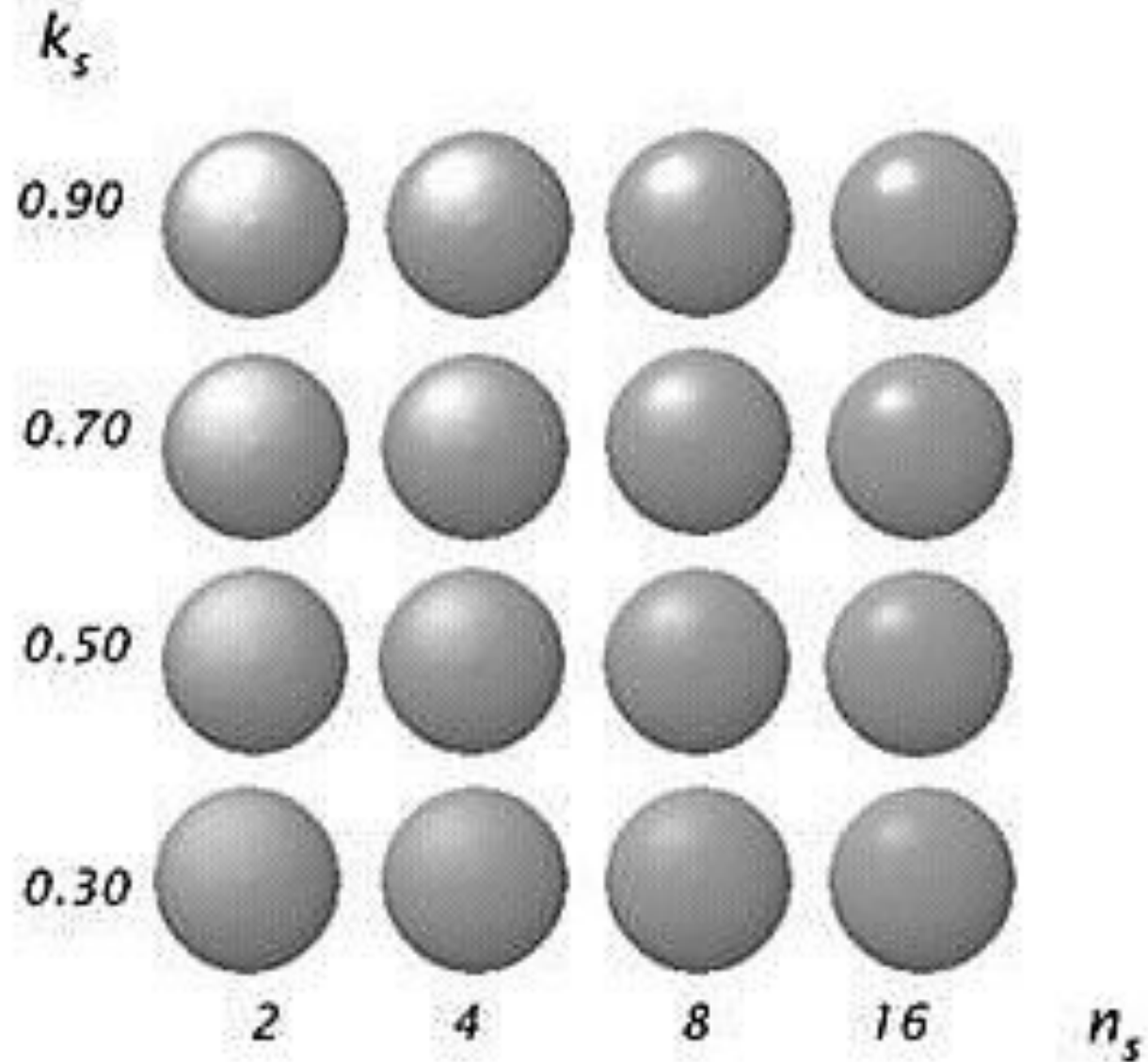
$$V \cdot R = \cos \phi$$

- Remembering that

We can say:

$$I_{l,spec} = \begin{cases} k_s I_l (V \cdot R)^{n_s} & \text{if } V \cdot R > 0 \text{ and } N \cdot L > 0 \\ 0.0 & \text{if } V \cdot R < 0 \text{ or } N \cdot L \leq 0 \end{cases}$$

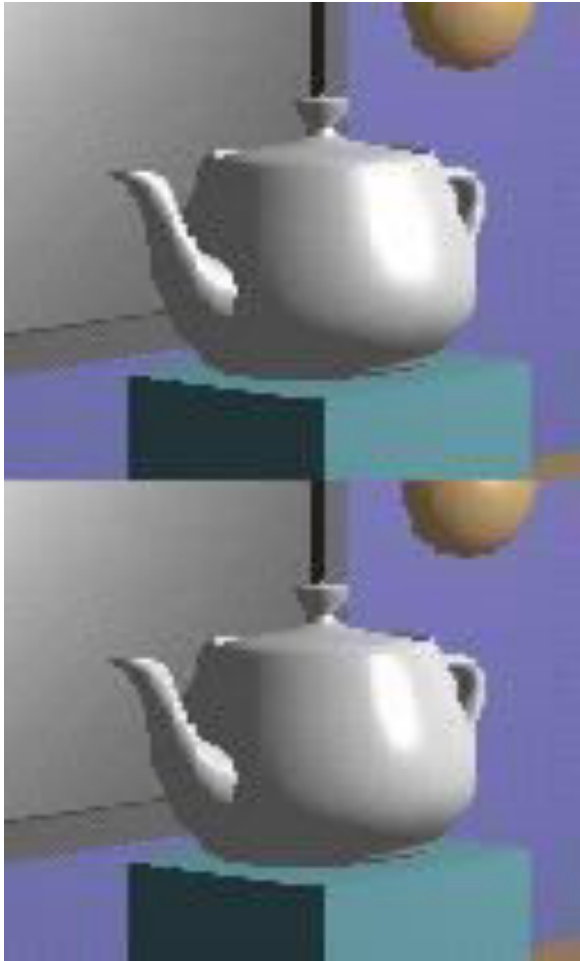
Example



Specular reflection: Effect of ' n_s '

$$\mathbf{I}_s = k_s \mathbf{I} \cos^{n_s} \phi$$

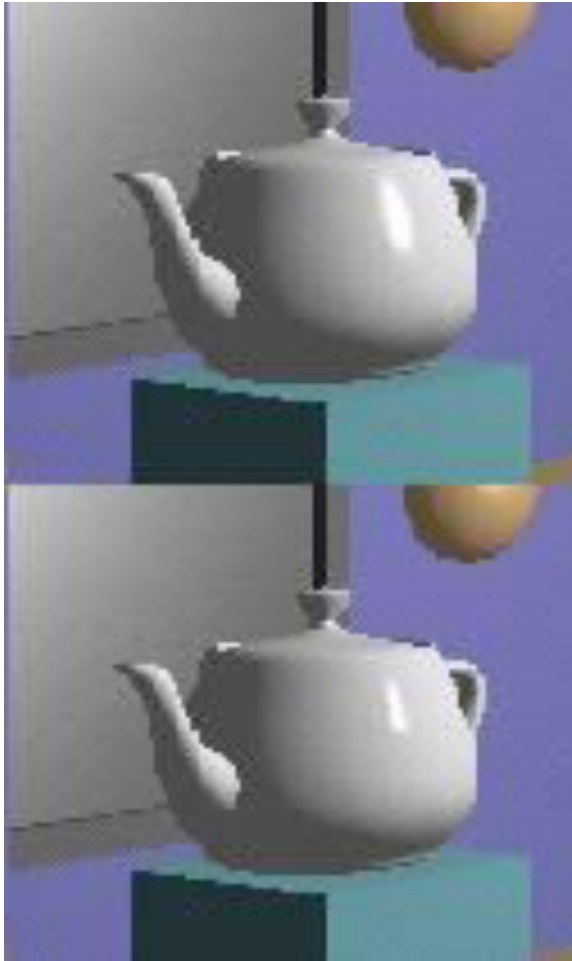
$n_s = 10$



$n_s = 30$



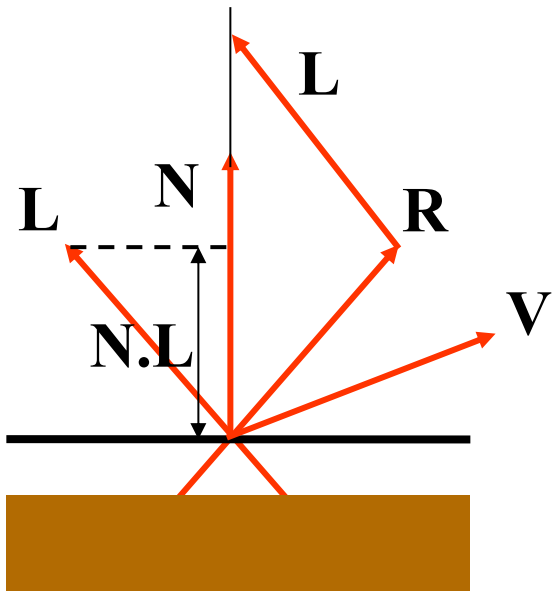
$n_s = 90$



$n_s = 270$

Calculating Reflection Vector

Phong model: calculating the vectors



$$\mathbf{R} + \mathbf{L} = (2\mathbf{N} \cdot \mathbf{L})\mathbf{N}$$

hence

$$\mathbf{R} = (2\mathbf{N} \cdot \mathbf{L})\mathbf{N} - \mathbf{L}$$

$$\mathbf{V} = \frac{\mathbf{P}_{\text{view}} - \mathbf{P}_{\text{surf}}}{|\mathbf{P}_{\text{view}} - \mathbf{P}_{\text{surf}}|}$$

Half way Vector - Blinn-Phong Model

- An alternative way of computing Phong specular lighting is:

$$I_{l,spec} = k_s I_l (N \cdot H)^{n_s}$$

- H (halfway vector): halfway between V and L

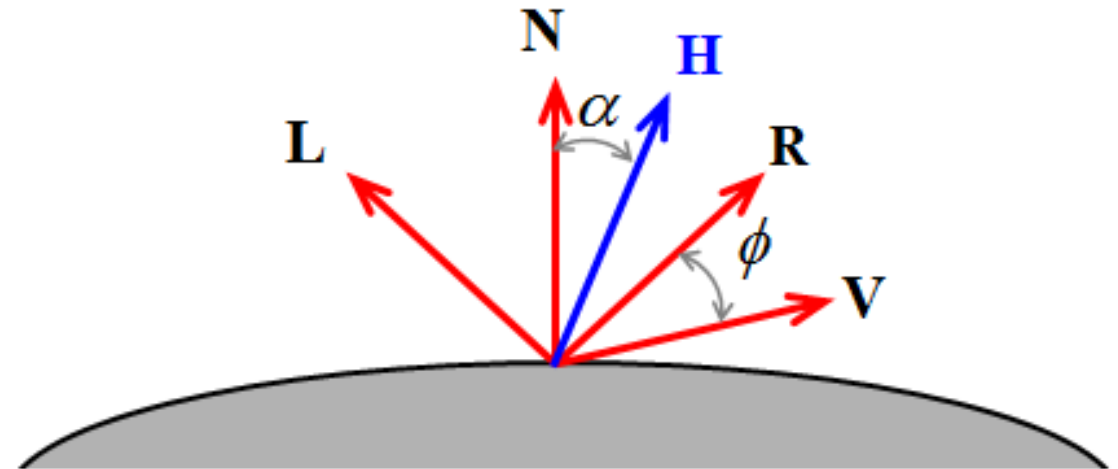
$$\mathbf{H} = \frac{\mathbf{L} + \mathbf{V}}{|\mathbf{L} + \mathbf{V}|}$$

- Why?

- Efficient computations
- Still view-dependent since H depends on V .

If light source and viewer far away:

$\mathbf{H} \pm \text{constant}$.



Combining Diffuse & Specular Reflections

- For a single light source we can combine the effects of diffuse and specular reflections simply as follows:

$$\begin{aligned} I &= I_{diff} + I_{spec} \\ &= k_a I_a + k_d I_l (N \cdot L) + k_s I_l (V \cdot R)^{n_s} \end{aligned}$$

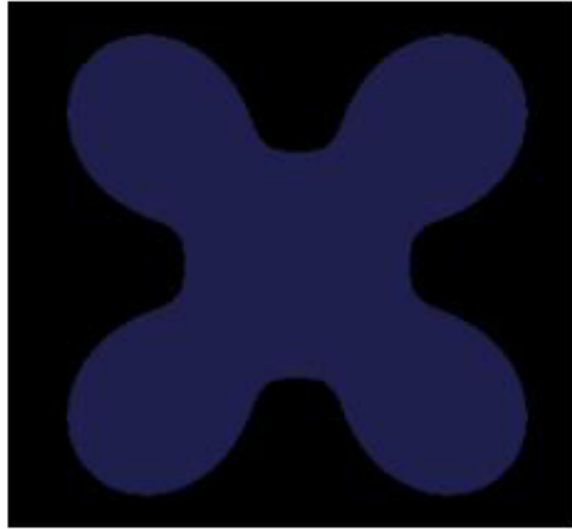
Specular light example



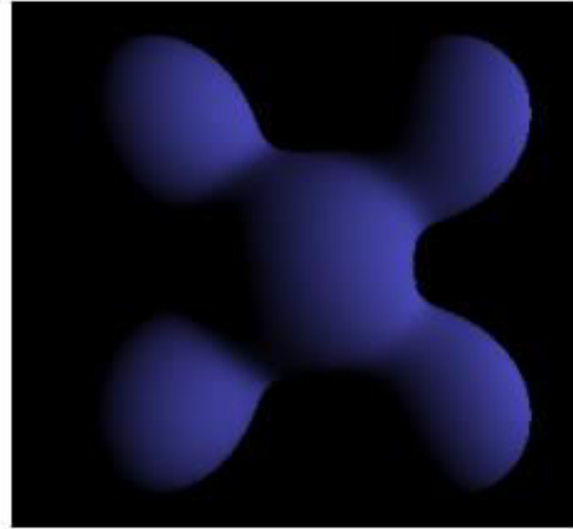
**Specular?
Bright spot
on object**

Example

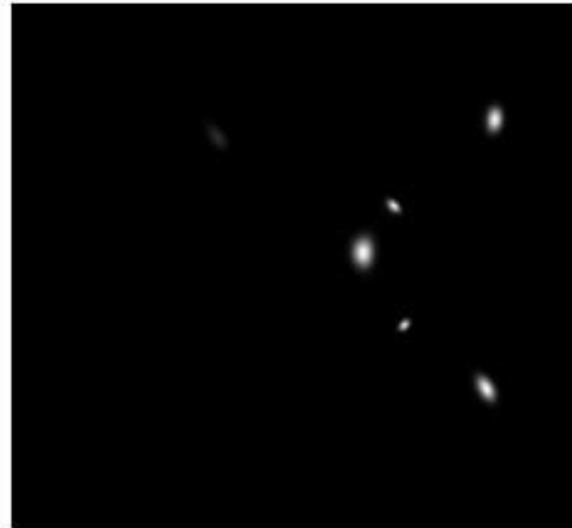
Ambient



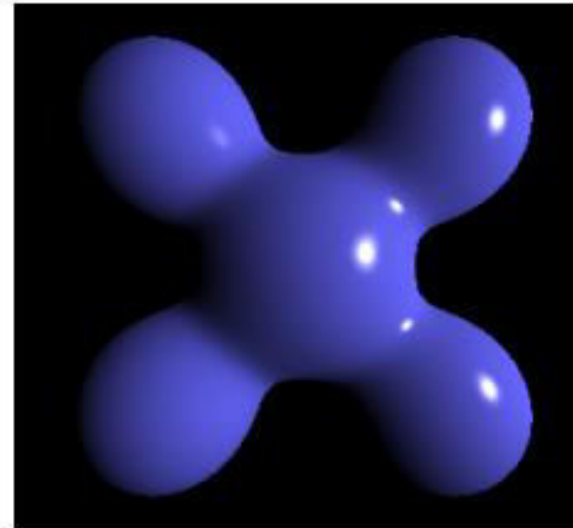
Diffuse



Specular



Final Image



Emission

- ❑ Some objects are self-luminous
- ❑ Light intensity due to emission is measured as –

$$I_e = k_i$$

k_i = coefficient of the object's intrinsic intensity

- ❑ Therefore, $I = I_{diff} + I_{spec} + I_e$

$$= k_a I_a + k_d I_l (N \cdot L) + k_s I_l (V \cdot R)^{n_s} + I_e$$

$$I^\lambda = \rho_a^\lambda I_a^{\lambda, \text{in}} + \rho_d^\lambda I_d^{\lambda, \text{in}} (\ell \cdot \mathbf{n}) + \rho_s^\lambda I_s^{\lambda, \text{in}} (\mathbf{r} \cdot \mathbf{v})^f + I_e^\lambda.$$

Multiple Light Sources

- ❑ We can place any number of light sources in a scene
- ❑ We compute the diffuse and specular reflections as sums of the contributions from the various sources

$$\begin{aligned} I &= I_{ambdiff} + \sum_{l=1}^n \left[I_{l,diff} + I_{l,spec} \right] \\ &= k_a I_a + \sum_{l=1}^n I_l \left[k_d (N \cdot L_l) + k_s (V \cdot R_l)^{n_s} \right] \end{aligned}$$

$$\mathbf{I} = \rho_a * \mathbf{I}_a^{\text{in}} + \rho_d * \sum_{i=1}^k \mathbf{I}_d^{\text{in},i} (\ell_i \cdot \mathbf{n}) + \rho_s * \sum_{i=1}^k \mathbf{I}_s^{\text{in},i} (\mathbf{r}_i \cdot \mathbf{v})^f + \mathbf{I}_e$$

It is common to specify a global value, $\mathbf{I}_a^{\text{in,global}}$, for global ambient light and to have each light source contribute some additional ambient light to the scene. Then

$$\mathbf{I}_a^{\text{in}} = \mathbf{I}_a^{\text{in,global}} + \sum_{i=1}^k \mathbf{I}_a^{\text{in},i}.$$

Exercise III.1 *Why is it customary to use the same specular exponent for all wavelengths? What would a specular highlight look like if different wavelengths had different specular exponents?*

Adding Intensity Attenuation

- ❑ To incorporate radial and angular intensity attenuation into our model we simply adjust our equation to take these into account
- ❑ So, light intensity is now given as:

$$I = I_{ambdiff} + \sum_{l=1}^n \left[f_{l,radatten} f_{l,angatten} (I_{l,diff} + I_{l,spec}) \right]$$

- ❑ where $f_{radatten}$ and $f_{angatten}$ are as discussed previously

RGB Colour Considerations

- ❑ For RGB colour description each intensity specification is a three element vector
- ❑ For each light source: $I_l = (I_{lR}, I_{lG}, I_{lB})$
- ❑ Similarly all parameters are given as : $k_a = (k_{aR}, k_{aG}, k_{aB})$

$$k_d = (k_{dR}, k_{dG}, k_{dB}) \quad k_s = (k_{sR}, k_{sG}, k_{sB})$$

- ❑ Each component of the surface colour is then calculated with a separate expression

$$I_{lR,diff} = k_{dR} I_{lR} (N \cdot L)$$

$$I_{lB,diff} = k_{dB} I_{lB} (N \cdot L)$$

$$I_{lG,diff} = k_{dG} I_{lG} (N \cdot L)$$

Coefficients for Real Materials

Material	Ambient <i>Kar, Kag,kab</i>	Diffuse <i>Kdr, Kdg,kdb</i>	Specular <i>Ksr, Ksg,ksb</i>	Exponent α
Black plastic	0.0 0.0 0.0	0.01 0.01 0.01	0.5 0.5 0.5	32
Brass	0.329412 0.223529 0.027451	0.780392 0.568627 0.113725	0.992157 0.941176 0.807843	27.8974
Polished Silver	0.23125 0.23125 0.23125	0.2775 0.2775 0.2775	0.773911 0.773911 0.773911	89.6

Figure 8.17, F S Hill, courtesy of McReynolds and Blythe

Exercise

Exercise 11.5. There are two light sources, L^0 and L^1 , the respective values of whose lighting properties matrices are

$$\begin{bmatrix} 0.0 & 0.0 & 0.0 \\ 0.7 & 0.1 & 0.1 \\ 0.7 & 0.1 & 0.1 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 0.0 & 0.0 & 0.0 \\ 0.1 & 0.7 & 0.1 \\ 0.1 & 0.7 & 0.1 \end{bmatrix}$$

The material properties matrix at a vertex V is

$$\begin{bmatrix} 0.1 & 0.8 & 0.9 \\ 0.1 & 0.8 & 0.9 \\ 1.0 & 1.0 & 1.0 \end{bmatrix}$$

Furthermore, the shininess exponent of the surface at V is 2.0, the unit outward normal vector at V is

$$[0.0 \ 1.0 \ 0.0]^T$$

and the emissive light vector at V is

$$[0.01 \ 0.0 \ 0.0]^T$$

Computer Graphics Through OpenGL®
From Theory to Experiments, Third Edition
Sumanta Guha

The position vectors of L^0 , L^1 , V and the eye are, respectively,

$$[0.0 \ 5.0 \ 5.0]^T, \ [5.0 \ 5.0 \ 5.0]^T, \ [0.0 \ 0.0 \ 5.0]^T \text{ and } [0.0 \ 0.0 \ 0.0]^T$$

The global ambient light vector is

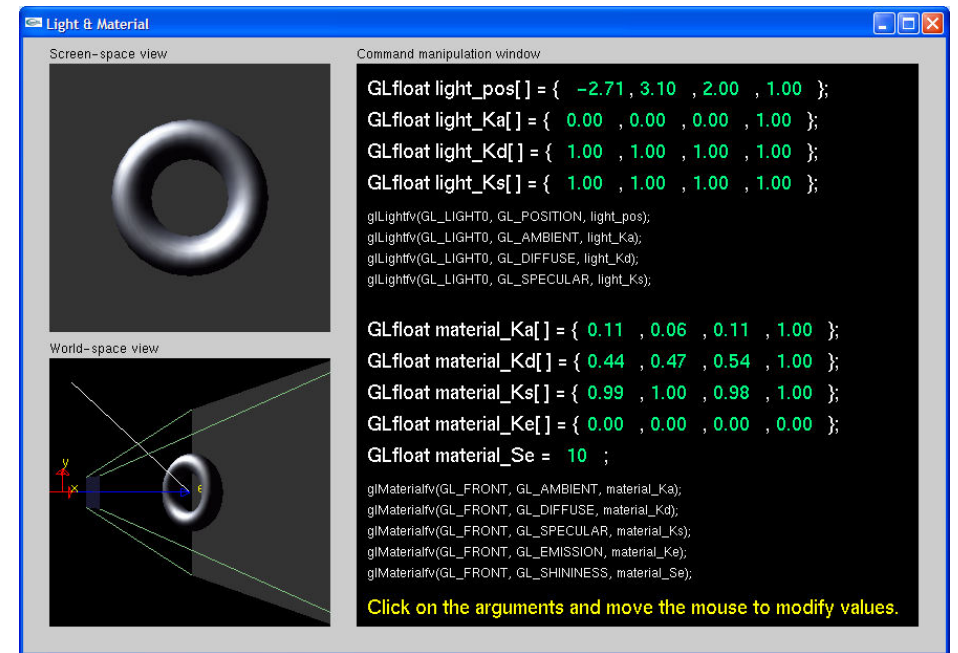
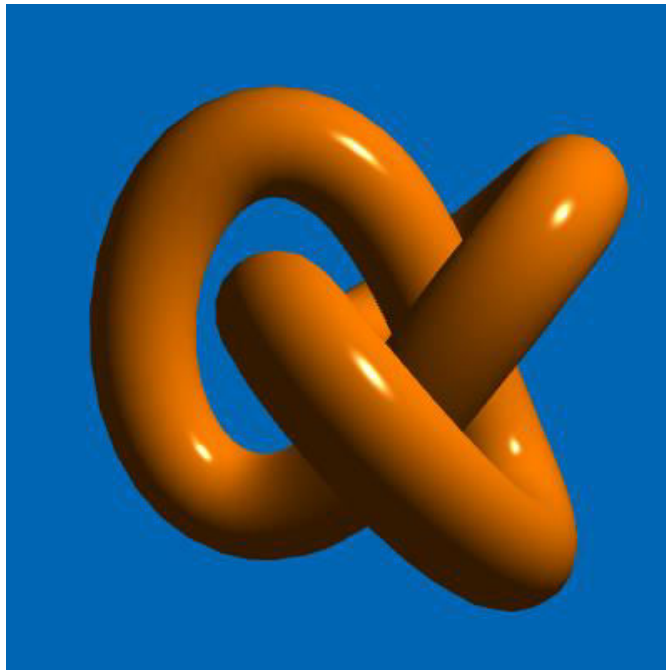
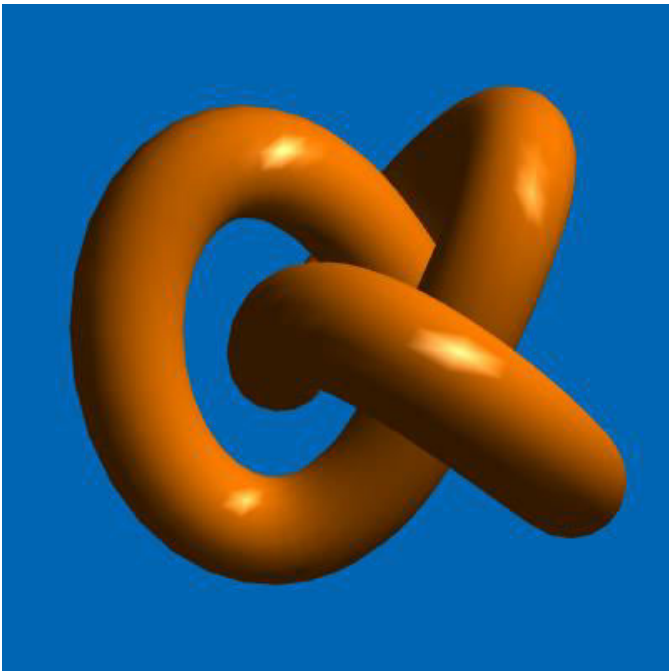
$$[0.1 \ 0.1 \ 0.1]^T$$

Fill out color values at V in the table below.

	R	G	B
Emission			
Global Ambient			
Ambient			
Diffuse			
Specular			
Total (color vector at V)			

Tutorials/Demo

- ❑ <http://www.cs.toronto.edu/~jacobson/phong-demo/>
- ❑ <https://user.xmission.com/~nate/tutors.html>



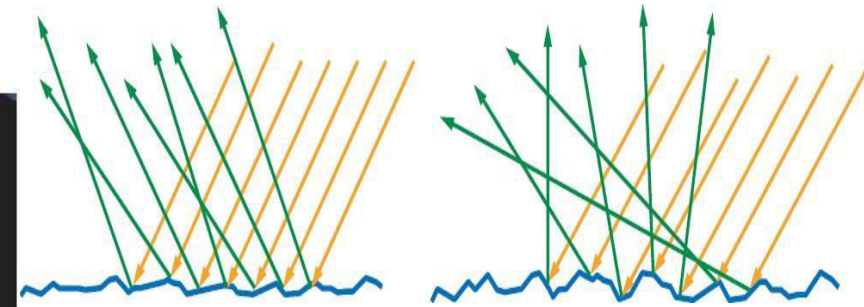
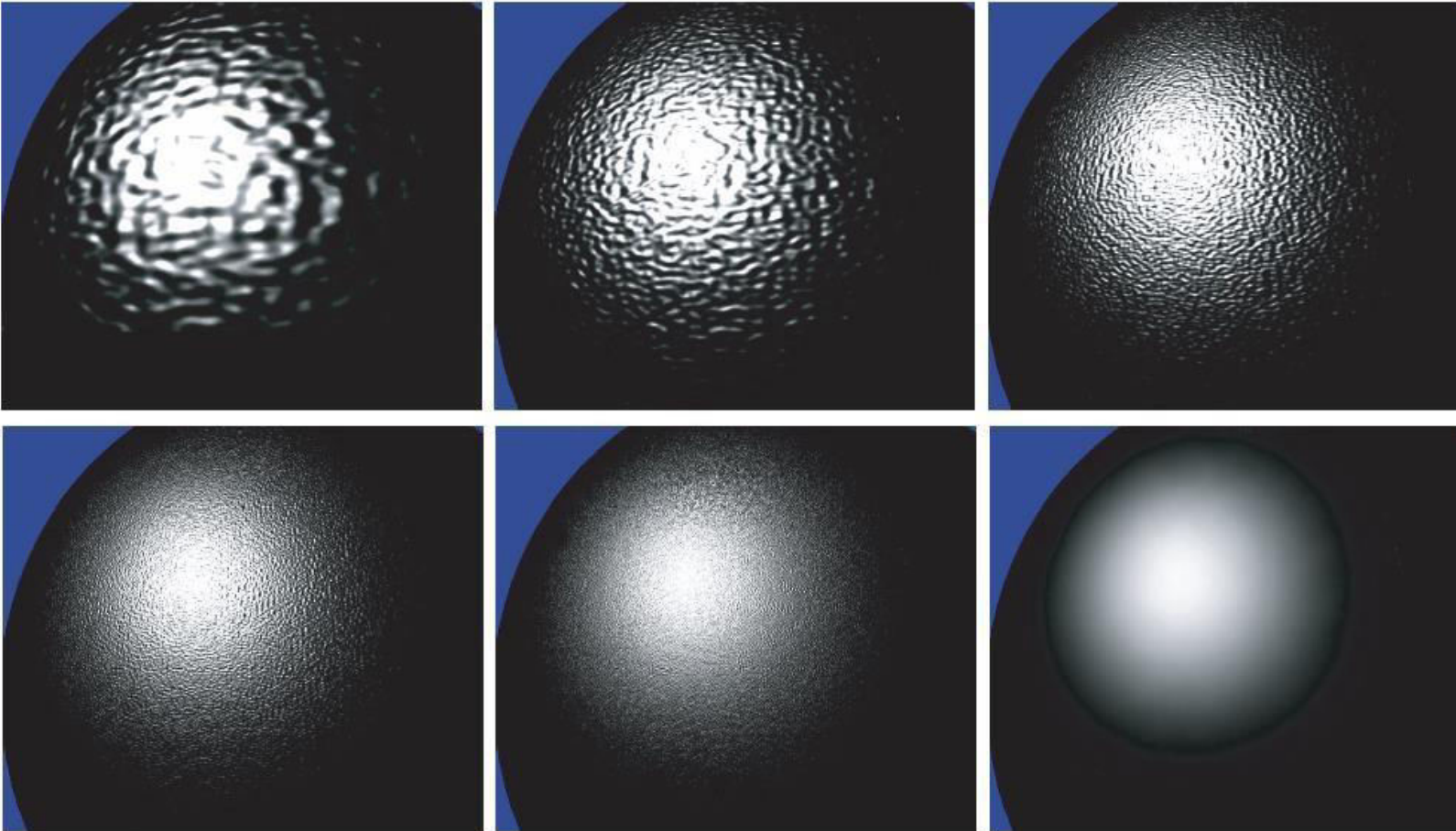
Physically-Based Lighting Models

Physically-Based Lighting Models

- ❑ Phong model produces pretty pictures
- ❑ Cons: empirical (fudged?) ($\cos^\alpha \phi$), plastic look
- ❑ Shaders can implement better lighting/shading models
- ❑ Big trend towards Physically-based lighting models
- ❑ Physically-based?
- ❑ Based on physics of how light interacts with actual surface
- ❑ Apply Optics/Physics theories
- ❑ Classic: Cook-Torrance shading model (TOGS 1982)

Microgeometry

- ❑ Rougher surfaces bounce light all over the place



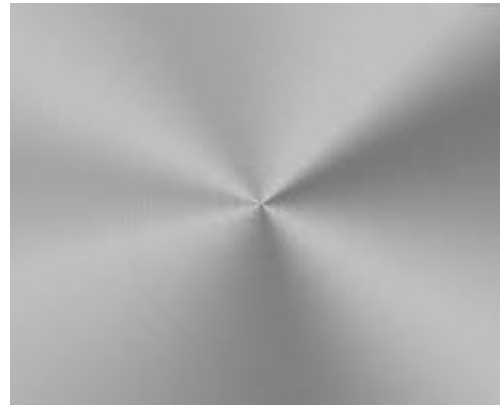
Increasing roughness

Isotropic Vs Anisotropic Surfaces

- **Isotropic:** light bounced equally in all directions
- **Anisotropic:**
 - Surface has grooves with directions. E.g. Brushed steel
 - Light bounced differently along vs across the grain.



Isotropic



Anisotropic (brushed steel)



Cook-Torrance Shading Model

- Same ambient and diffuse terms as Phong
- New, better specular component than $(\cos^\alpha \phi)$,

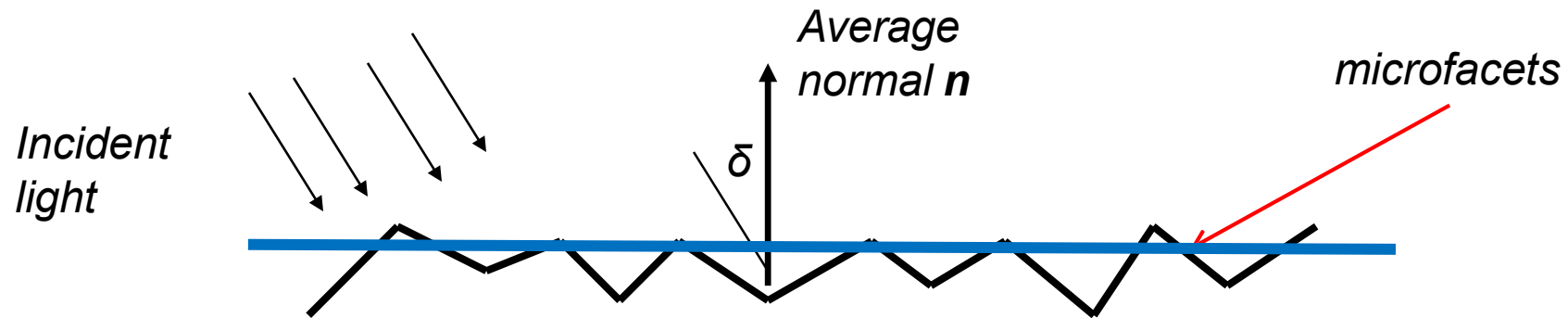
$$\cos^\alpha \phi \rightarrow \frac{F(\phi, \eta) D G}{(\mathbf{n} \cdot \mathbf{v})}$$

- Where
 - D - Distribution term
 - G - Geometric term
 - F - Fresnel term

$$I_s = \frac{(\mathbf{n} \cdot \boldsymbol{\ell})}{(\mathbf{n} \cdot \mathbf{v})} s F G D \cdot I_s^{\text{in}},$$

Distribution Term, D

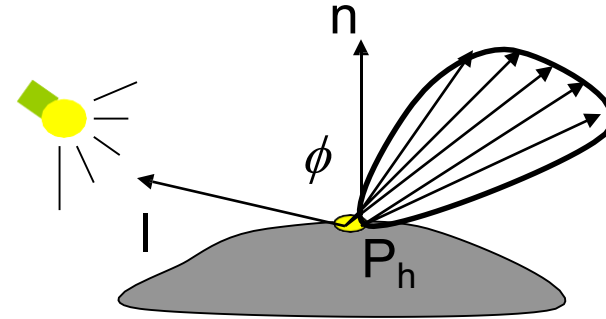
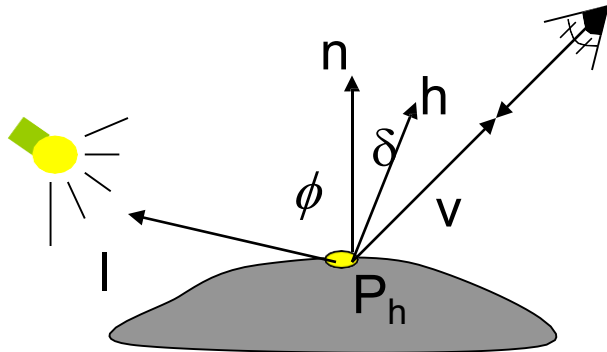
- **Idea:** surfaces consist of small V-shaped **microfacets (grooves)**



- Many grooves at each surface point
- Grooves facing a direction contribute
- $D(\delta)$ term: what fraction of grooves facing each angle δ
- E.g. half of grooves at hit point face 30 degrees, etc

Cook-Torrance Shading Model

- Define angle δ as deviation of $\mathbf{h}$ from surface normal
- Only microfacets with pointing along halfway vector, $\mathbf{h} = \mathbf{l} + \mathbf{v}$, contributes



- Can use old Phong cosine ($\cos^n \phi$), as D
- Use Beckmann distribution instead

$$D(\delta) = \frac{1}{4\mathbf{m}^2 \cos^4(\delta)} e^{-\left(\frac{\tan(\delta)}{\mathbf{m}}\right)^2}$$

- $\mathbf{m}$ expresses roughness of surface. How?

Cook-Torrance Shading Model

- **Isotropic Distributions** $D(\omega) \Rightarrow D(\alpha)$ $\alpha = \angle N, H$
 - α : angle to average normal of surface
 - m : average slope of the microfacets

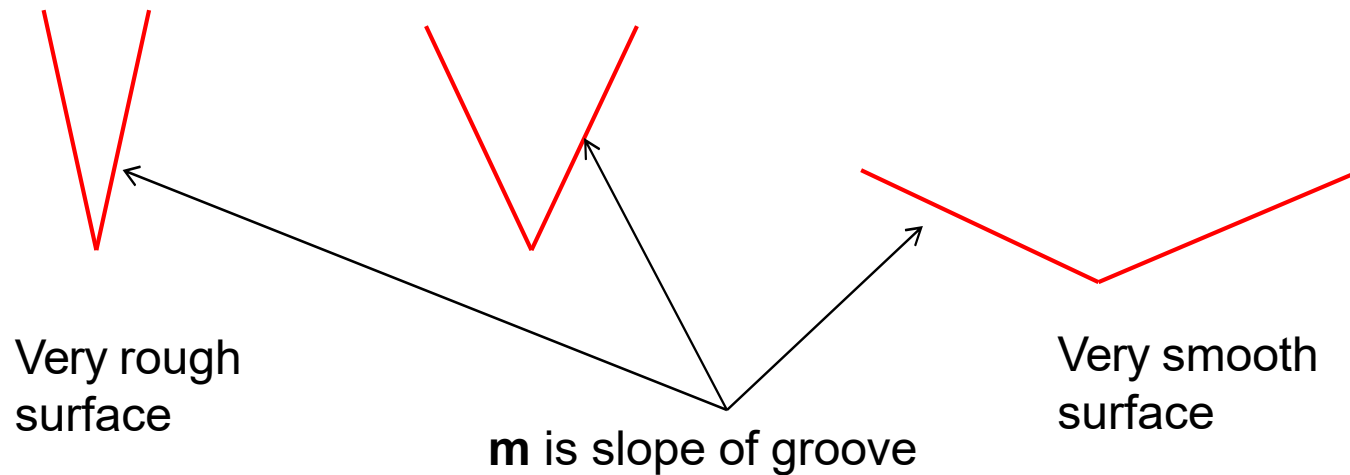
- **Blinn:**
$$D(\alpha) = \cos^{\frac{\ln 2}{\cos m \alpha}} \alpha$$

- **Torrance-Sparrow**
$$D(\alpha) = e^{-\left(\frac{\alpha}{m}\right)^2}$$
 - Gaussian

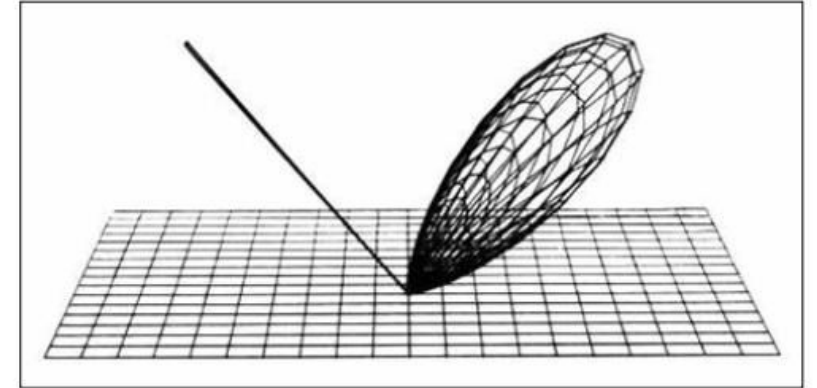
- **Beckmann**
$$D(\alpha) = \frac{1}{\pi m^2 \cos^4 \alpha} e^{-\left(\frac{\tan \alpha}{m}\right)^2}$$
 - Used by Cook-Torrance

Cook-Torrance Shading Model

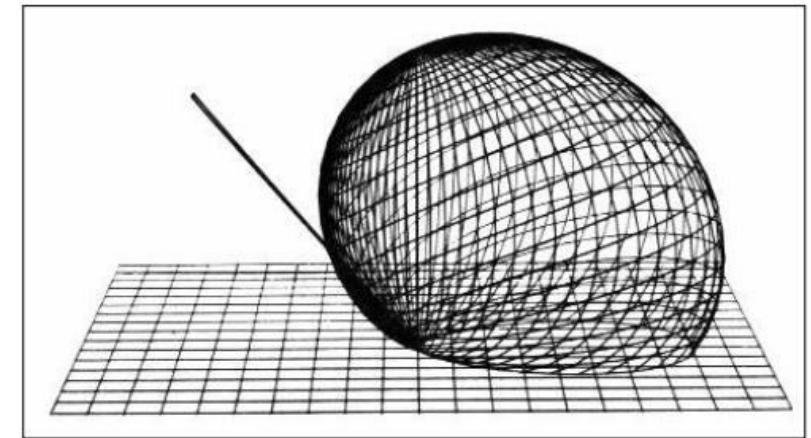
- m is Root-mean-square (RMS) of **slope** of V-groove
- $m = 0.2$ for nearly smooth
- $m = 0.6$ for very rough



$m=0.2$



$m=0.6$



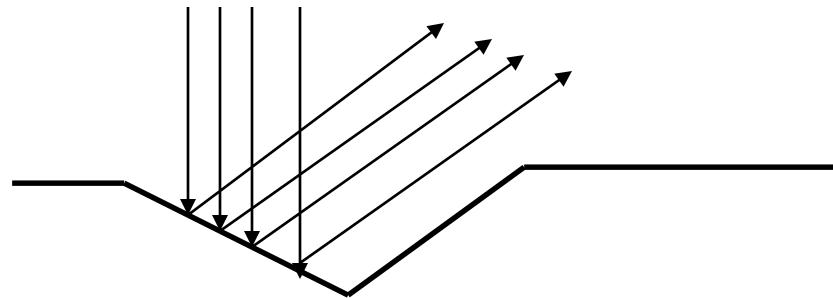
Self-Shadowing (G Term)

- ❑ Some grooves on extremely rough surface may block other grooves



Geometric Term, G

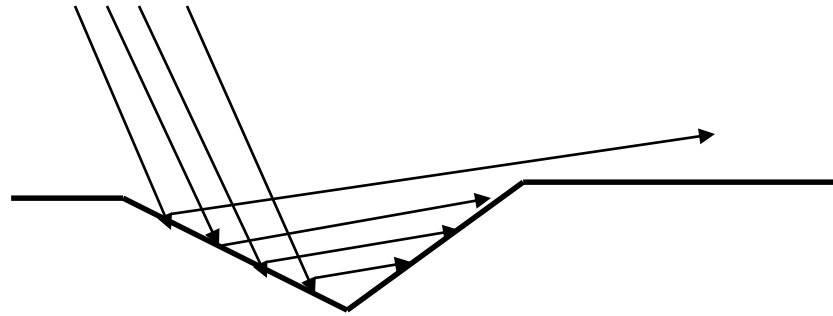
- Surface may be so rough that interior of grooves is blocked from light by edges
- Self blocking known as **shadowing** or **masking**
- Geometric term G accounts for this
- Break G into 3 cases:
- G, case a: No self-shadowing (light in-out unobstructed)



- Mathematically, $G = 1$

Geometric Term, G

- G_m , case b: No blocking on entry, blocking of exiting light (**masking**)



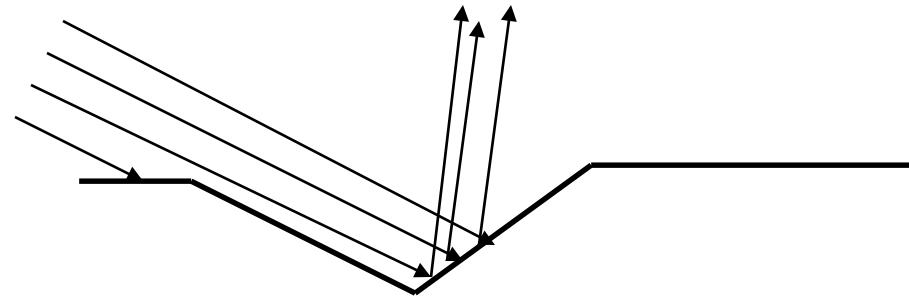
- Mathematically,

$$G_m = \frac{2(\mathbf{n} \cdot \mathbf{h})(\mathbf{n} \cdot \mathbf{v})}{\mathbf{h} \cdot \mathbf{v}}$$

Geometric Term, G

- G_s , case c: blocking of incident light, no blocking of exiting light (**shadowing**)
- Mathematically,

$$G_s = \frac{2(\mathbf{n} \cdot \mathbf{h})(\mathbf{n} \cdot \mathbf{l})}{\mathbf{h} \cdot \mathbf{l}}$$



- G term is minimum of 3 cases, hence

$$G = (1, G_m, G_s)$$

Fresnel Term, F

- F term, $F(\phi, \eta)$ gives fraction of incident light reflected

$$\mathcal{F} = \frac{1}{2} \left(\frac{\sin^2(\varphi - \theta)}{\sin^2(\varphi + \theta)} + \frac{\tan^2(\varphi - \theta)}{\tan^2(\varphi + \theta)} \right)$$

where φ is the angle of incidence and θ is the angle of refraction

$$F = \frac{1}{2} \frac{(g - c)^2}{(g + c)^2} \left\{ \left| 1 + \left(\frac{c(g + c) - 1}{c(g - c) - 1} \right)^2 \right| \right\}$$

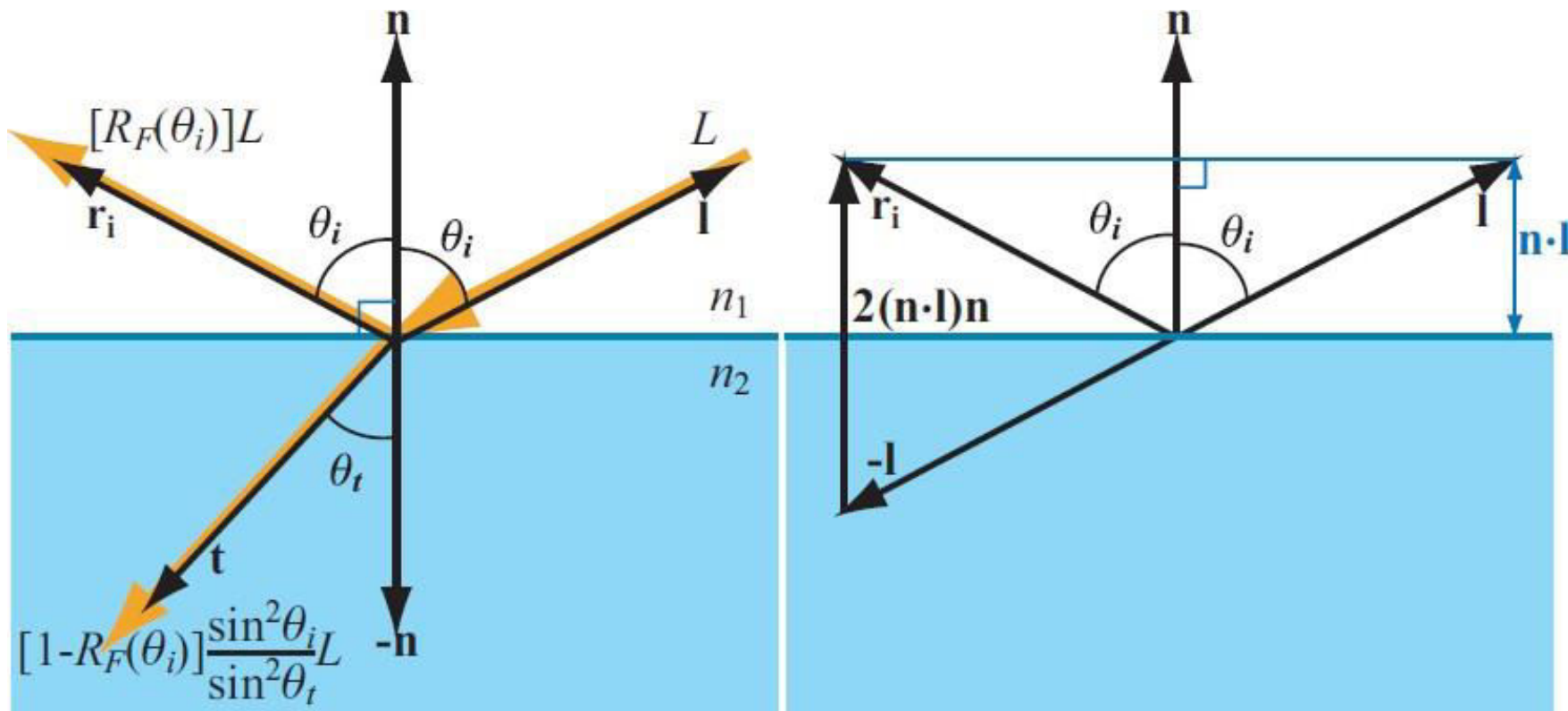
F is function of material
and incident angle

- where $c = \cos(\phi) = \mathbf{n} \cdot \mathbf{s}$ and $g^2 = \eta^2 + c^2 + 1$
- ϕ is incident angle, η is refractive index of material

$$\frac{\sin \varphi}{\sin \theta} = \eta.$$

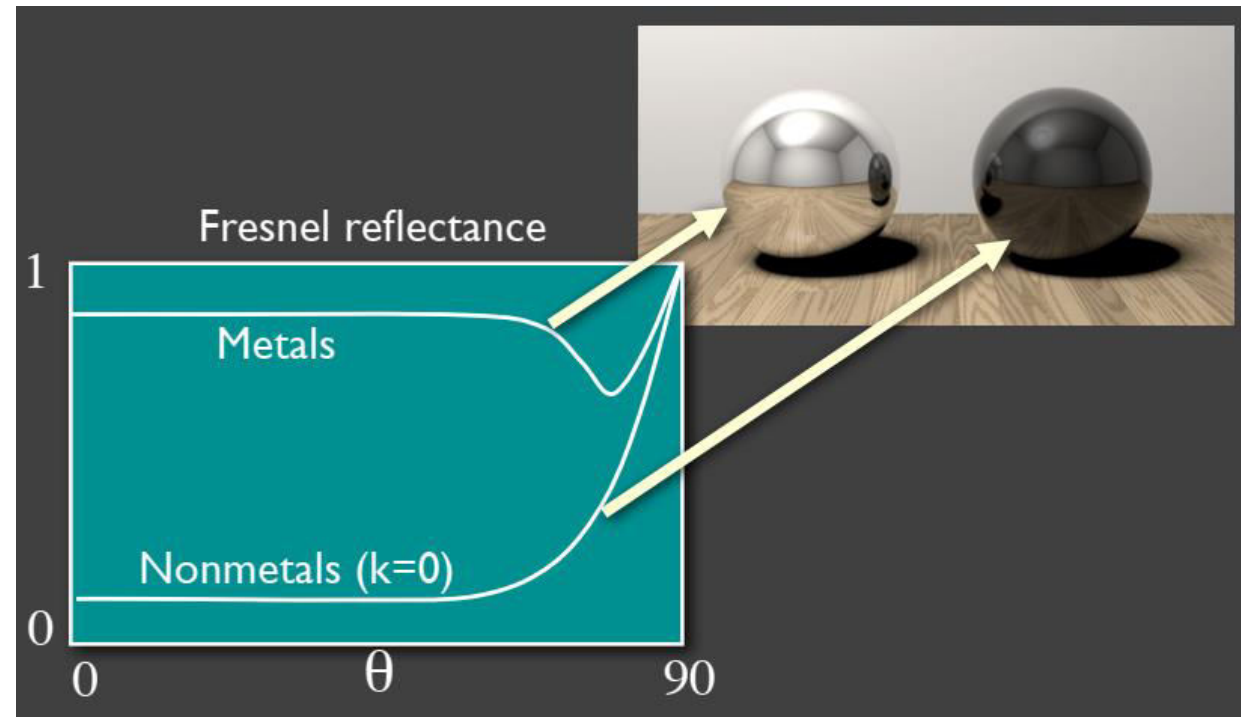
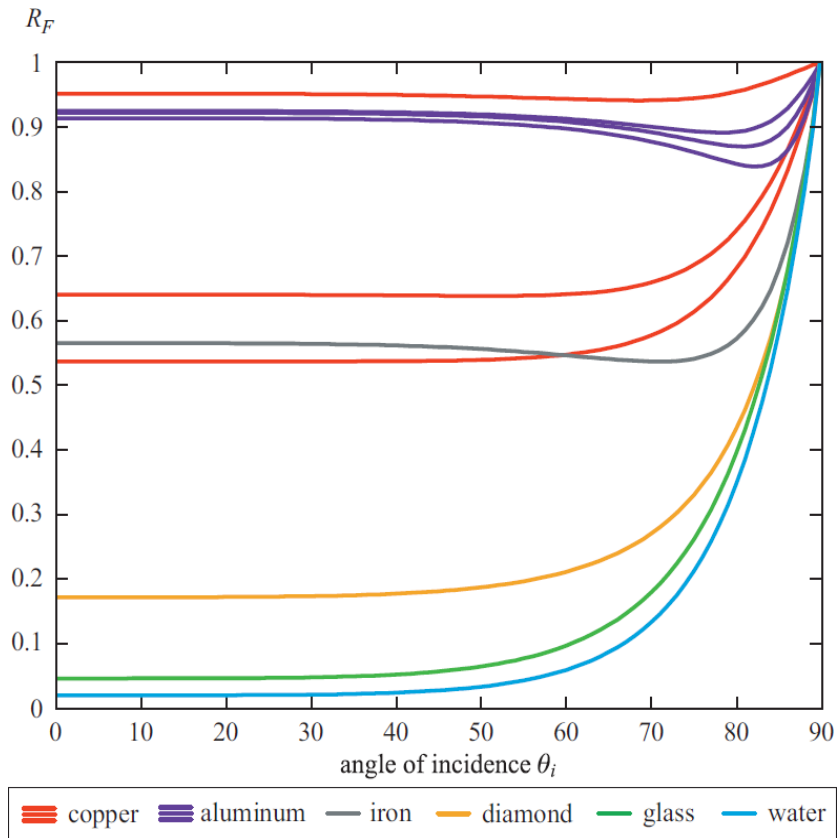
Fresnel Reflectance

- ❑ Equation that determines what fraction of incident light is reflected (and what fraction is transmitted)



Fresnel Reflectance

- ❑ Depends on angle of incidence and material



Metal vs. Nonmetal

Fresnel Reflectance

- Usually, physics table for each material's fresnel reflectance at zero degrees of incidence

Material	Fresnel Value (R,G,B)
Water	0.02, 0.02, 0.02
Plastic	0.05, 0.05, 0.05
Glass	0.08, 0.08, 0.08
Diamond	0.17, 0.17, 0.17
Copper	0.95, 0.64, 0.54
Aluminum	0.91, 0.92, 0.92

	Red	Green	Blue
Gold:	0.93	0.88	0.38
Iridium:	0.26	0.28	0.26
Iron:	0.44	0.435	0.43
Nickel:	0.50	0.47	0.36
Copper:	0.93	0.80	0.46
Platinum:	0.63	0.62	0.57
Silver:	0.97	0.97	0.96

- Schlick approximation to get arbitrary F

$$F(\theta) = F(0) + (1 - F(0))(1 - \cos \theta)^5$$

Fresnel Reflectance

- ❑ Schlick's approximation of Fresnel

$$R_F(\theta) = R_F(0) + (1 - R_F(0))(1 - \cos(\theta))^5$$

- ❑ For dielectric

$$R_F(0) = \left(\frac{\eta - 1}{\eta + 1}\right)^2$$

Exercise III.8★ *Prove that the reflectance $\mathcal{F}$ can also be computed by the formula*

$$\mathcal{F} = \frac{1}{2} \left[\left(\frac{\eta \cos \theta - \cos \varphi}{\eta \cos \theta + \cos \varphi} \right)^2 + \left(\frac{\eta \cos \varphi - \cos \theta}{\eta \cos \varphi + \cos \theta} \right)^2 \right]. \quad \text{III.21}$$

[Hint: Use Equation III.20 and use trigonometry identities to show

Rob Cook's vases



Source: Cook, Torrance 1981

Sub-Surface Scattering

Crysis skin [demo](#)



Marble



Human Skin

More Examples...



Leaves



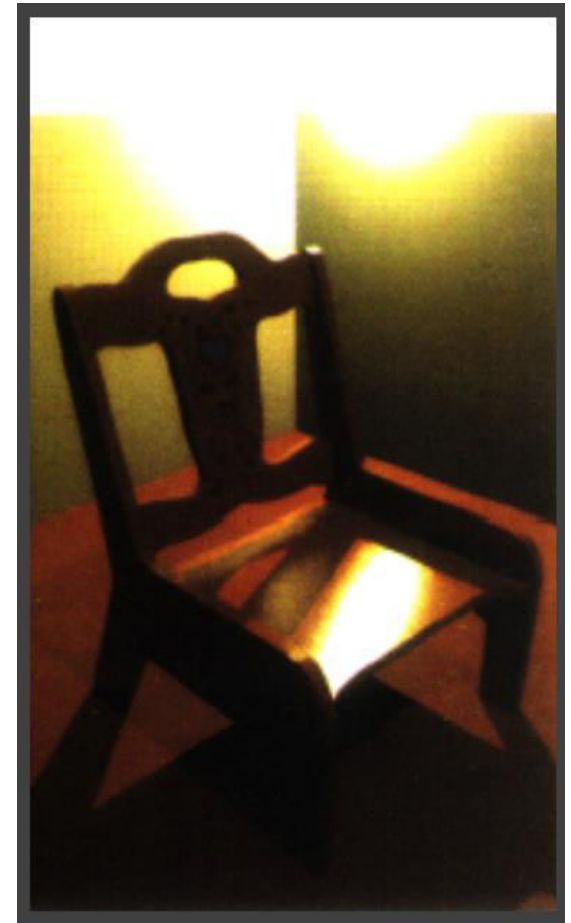
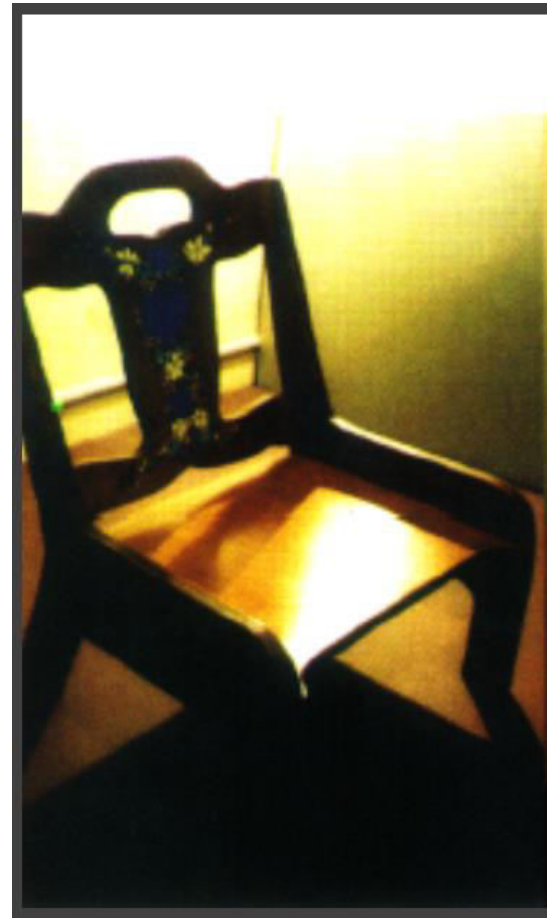
Hair



Milk

Ward Model

- ❑ Physically valid
 - Energy conserving
 - Satisfies reciprocity
 - Easy to integrate
- ❑ Based on empirical data
- ❑ Isotropic and anisotropic materials



Ward Model

□ Isotropic

$$f_s = \rho_s \frac{1}{4\pi m^2} \frac{1}{\sqrt{N.LN.V}} e^{-\frac{\tan^2 \theta_h}{m^2}}$$

where,

— m (usually α) is surface roughness

Ward Model

□ anisotropic

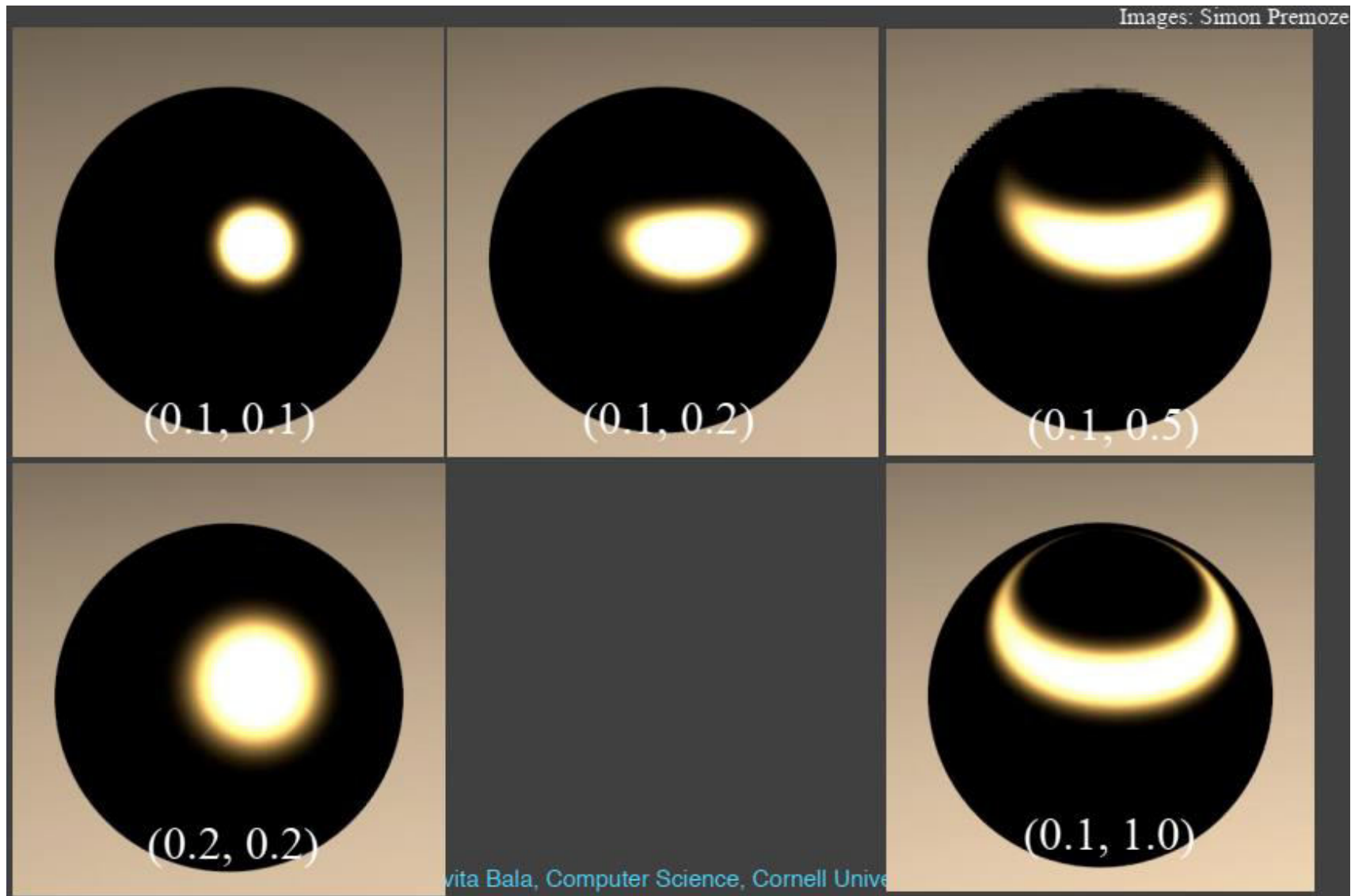
$$f_s = \rho_s \frac{1}{4\pi m_x m_y} \frac{1}{\sqrt{N.LN.V}} e^{-\tan^2 \theta_h \left(\frac{\cos^2 \phi_h}{m_x^2} + \frac{\sin^2 \phi_h}{m_y^2} \right)}$$

$$f_s = \rho_s \frac{1}{4\pi m_x m_y} \frac{1}{\sqrt{N.LN.V}} e^{-2 \frac{\left(\frac{H.\hat{x}}{m_x} \right)^2 + \left(\frac{H.\hat{y}}{m_y} \right)^2}{1+N.H}}$$

where,

- m_x, m_y are surface roughness in $\hat{x}, \hat{y}$
- $\hat{x}, \hat{y}$ are mutually perpendicular to the normal

Ward Model - Example



Ward Model - Example



Summary

- ❑ To create realistic (or even semi-realistic) looking scenes we must model light correctly
- ❑ To successfully model lighting effects we need to consider:
 - Ambient light
 - Diffuse reflections
 - Specular reflections

References

- ❑ 3-D Computer Graphics A Mathematical Introduction with OpenGL S R Buss
- ❑ Interactive Computer Graphics (6th edition), Angel and Shreiner
- ❑ Computer Graphics using OpenGL (3rd edition), Hill and Kelley
- ❑ procedural-elements-of-computer-graphics C Rogers
- ❑ Peter Shirley book

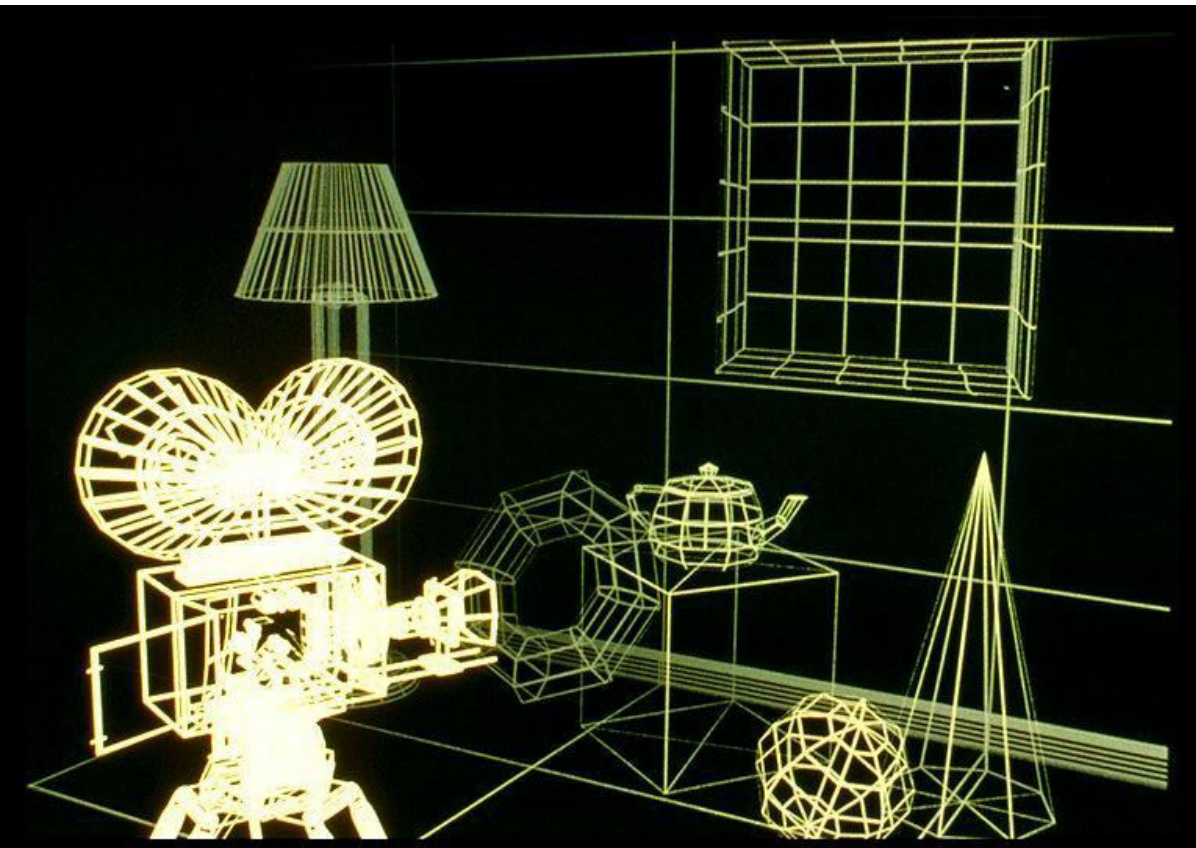
Prof Emmanuel Agu

Computer Science Dept.

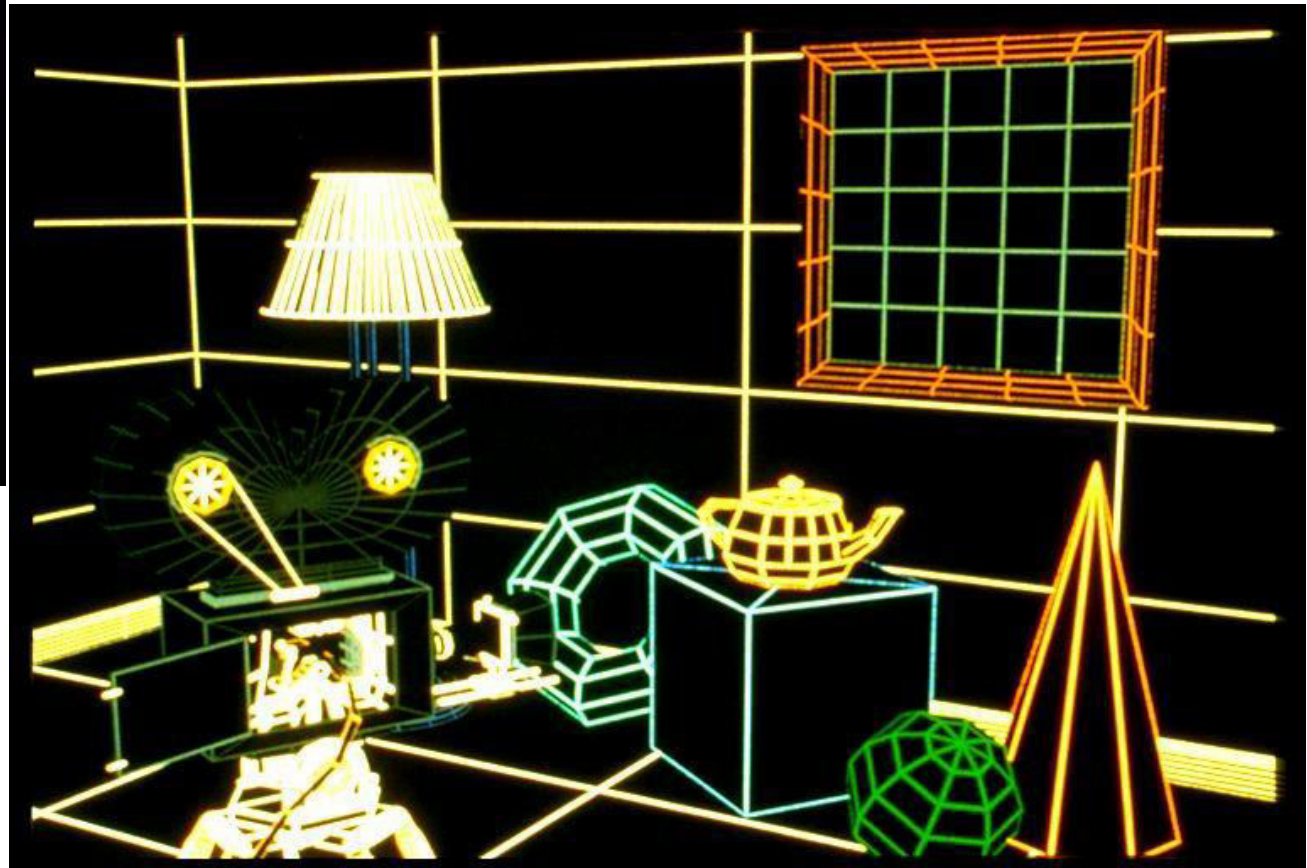
Worcester Polytechnic Institute (WPI)

Shading

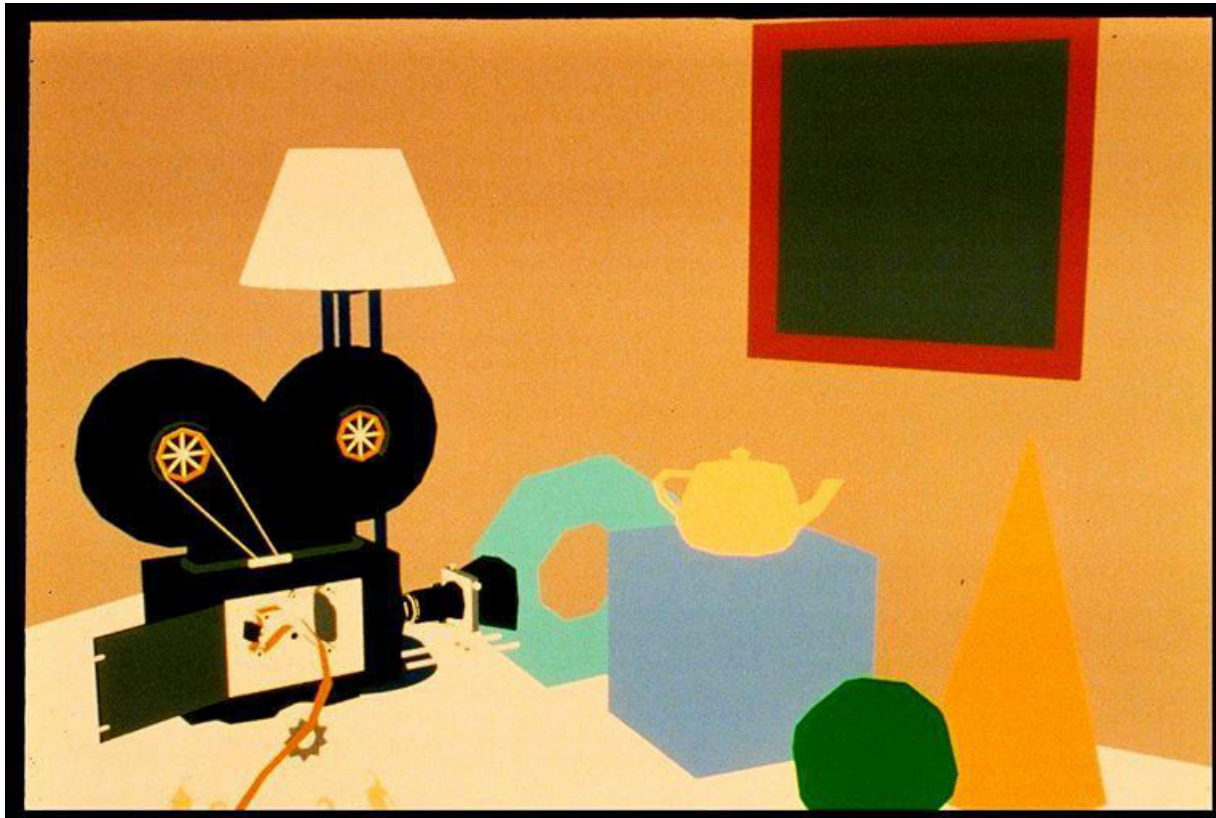
SURFACE RENDERING
METHODS



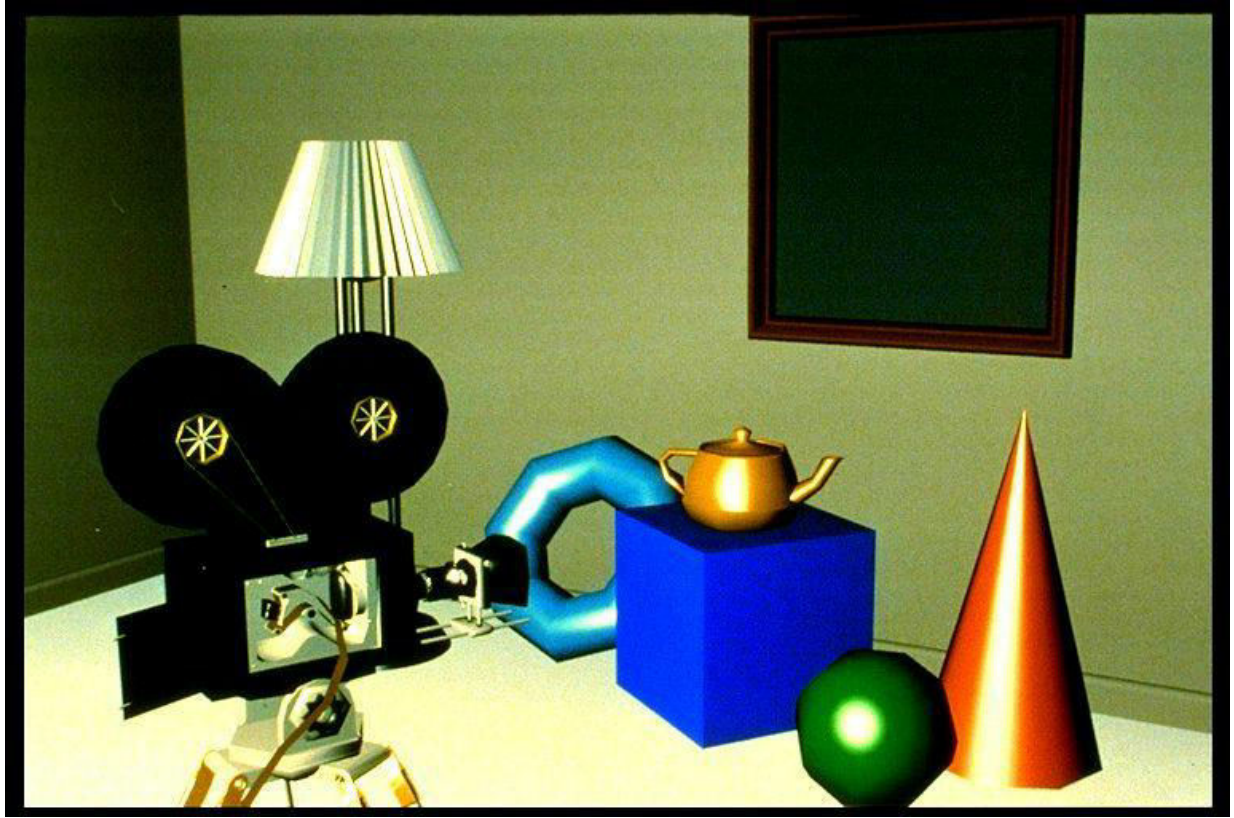
Wireframe model of a scene



No Surface Rendering Vs Surface Rendering



Object Rendering



With Surface Rendering

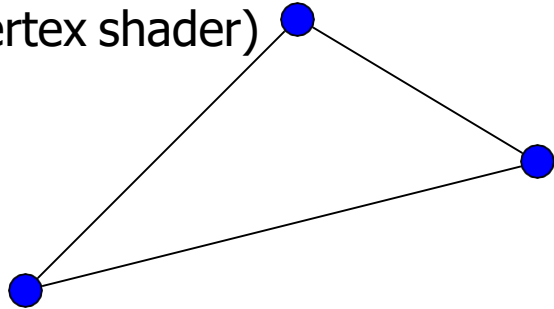
Shading

- ❑ Surface rendering means ***a procedure for applying a lighting model to obtain*** pixel intensities for all the projected surface positions in a scene.
- ❑ A surface-rendering algorithm uses the intensity calculations from an illumination model to determine the light intensity for all projected pixel positions for the various surfaces in a scene.
- ❑ Surface rendering can be performed by applying the illumination model to every visible surface point

Shading?

- After triangle is rasterized (converted to pixels)
 - Per-vertex lighting calculation means color at vertices is accurate, known (red dots)
- Shading: Graphics hardware figures out color of interior pixels (blue dots)
- How? Assume linear change => interpolate

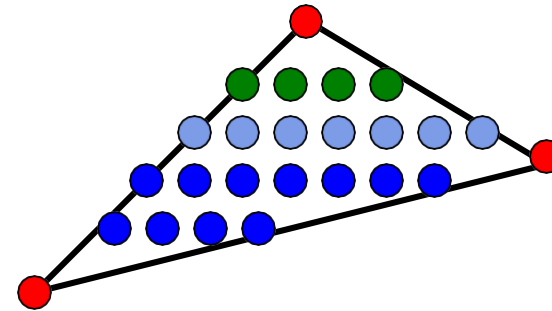
Lighting
(calc at vertices
in vertex shader)



Rasterization
Find pixels belonging
to each object



Shading
(done in hardware
during rasterization)

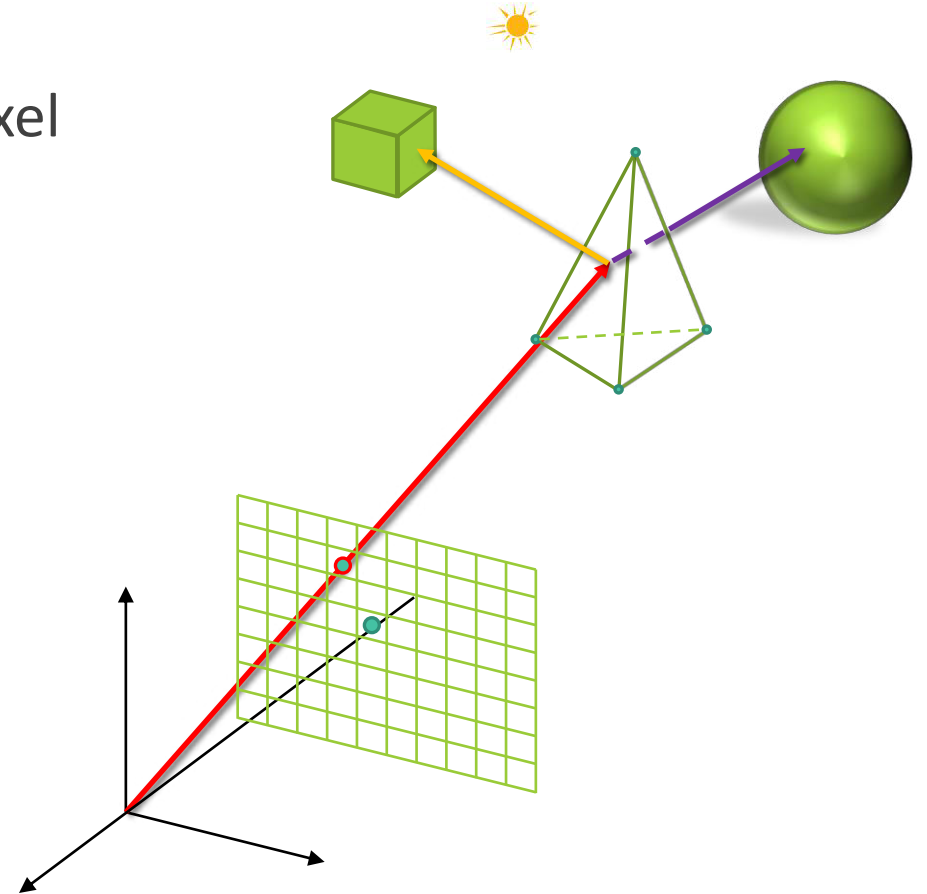
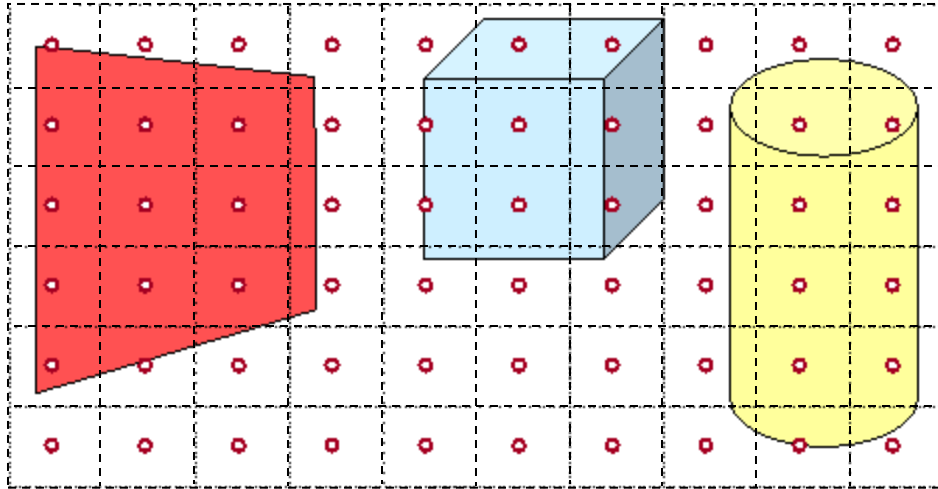


Shading Methods

- ❑ Ray Casting
 - Polygon Shading
- ❑ Ray Tracing
- ❑ Radiosity

Ray Casting

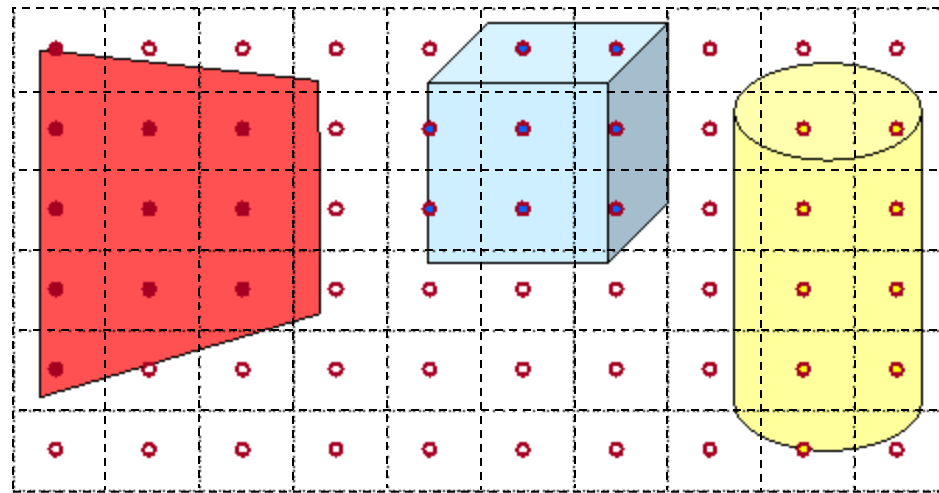
- Simplest Shading Approach
 - Perform independent lighting calculation for every pixel



$$I = I_E + K_A I_{AL} + \sum_i (K_D (N \cdot L_i) I_i + K_S (V \cdot R_i)^n I_i)$$

Polygon Shading

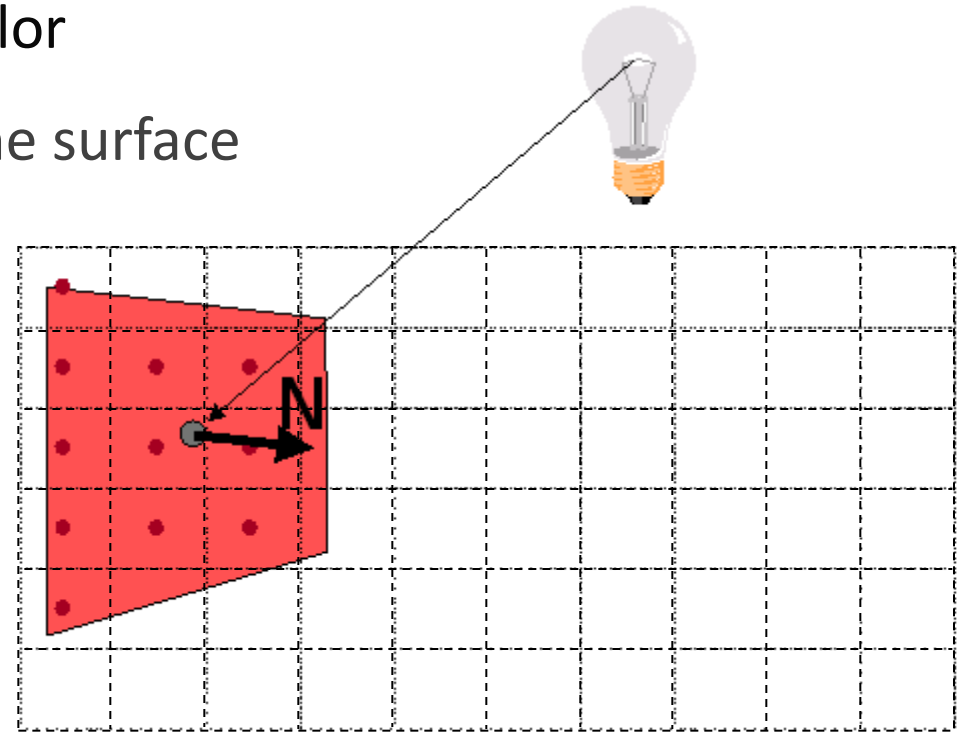
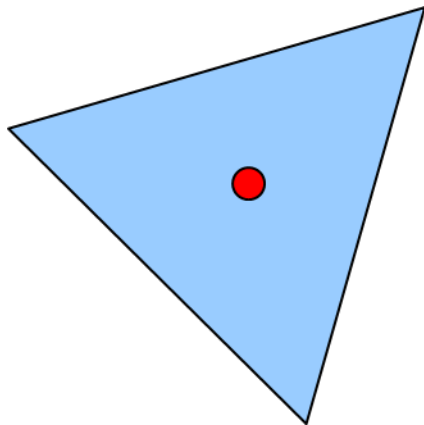
- ❑ Can Take Advantage of Spatial Coherence
 - Illumination calculations for pixels covered by same primitive are related to each other



$$I = I_E + K_A I_{AL} + \sum_i (K_D (N \cdot L_i) I_i + K_S (V \cdot R_i)^n I_i)$$

Flat Shading

- ❑ Simplest method, same color is assigned to all surface positions
- ❑ One Illumination Calculation per Polygon
 - Assign all pixels inside each polygon the same color
- ❑ Illumination at a single point (usually center) on the surface is calculated and used for the entire surface

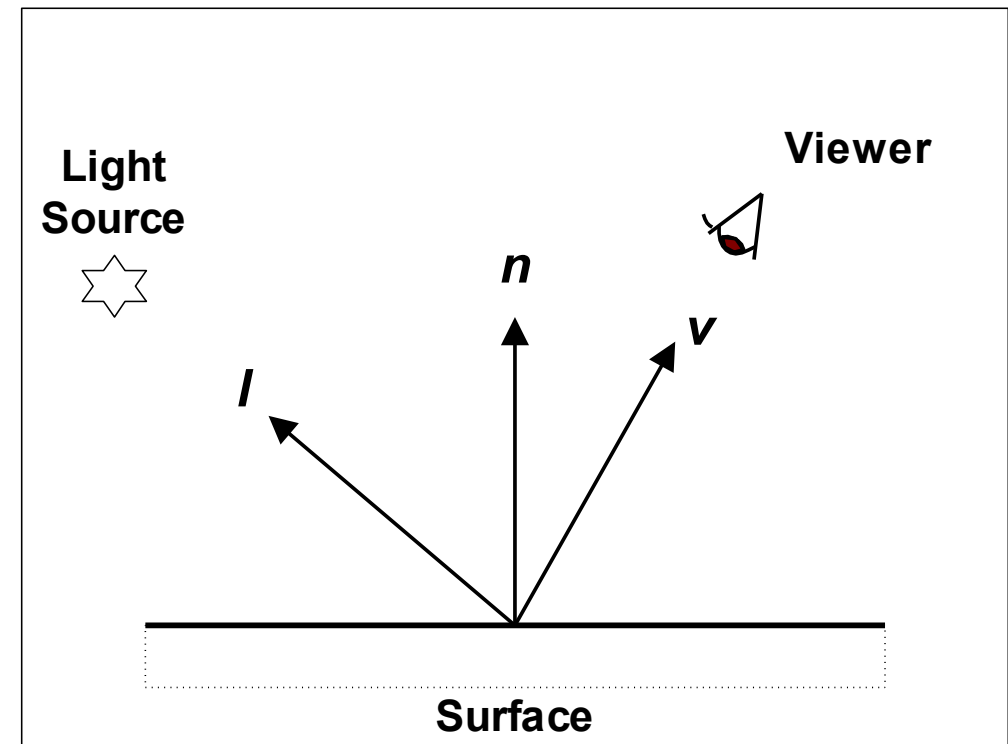


Flat Surface Rendering

□ Assumptions - For each surface

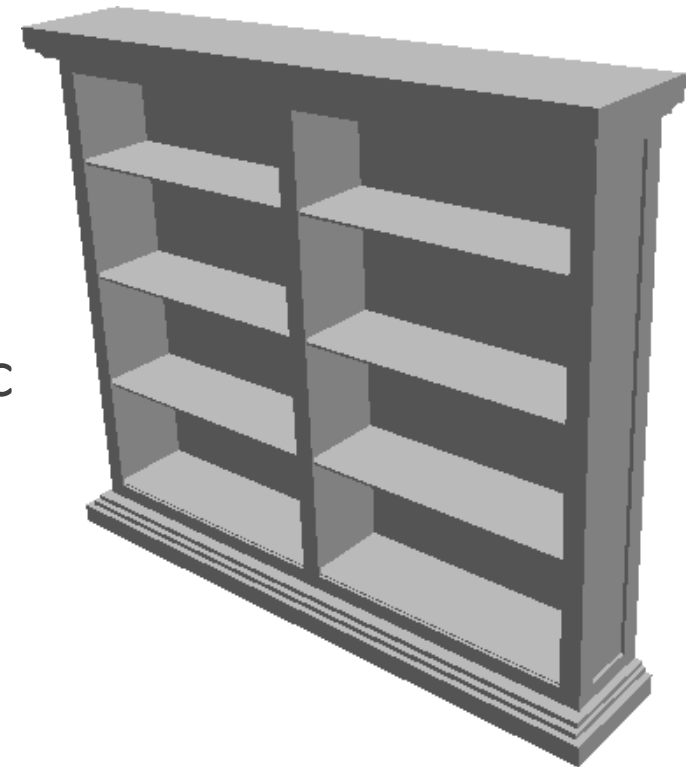
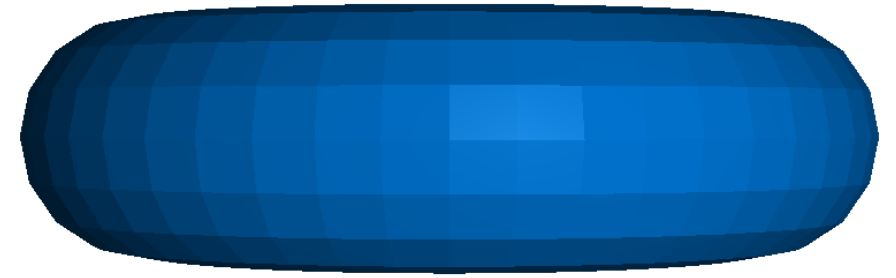
- Light source at infinity $\vec{n} \cdot \vec{l}$ is constant
- Viewer at infinity $\vec{n} \cdot \vec{v}$ is constant
- The polygon represents the actual surface being modeled

$$I = I_E + K_A I_{AL} + \sum_i (K_D (N \cdot L_i) I_i + K_S (V \cdot R_i)^n I_i)$$

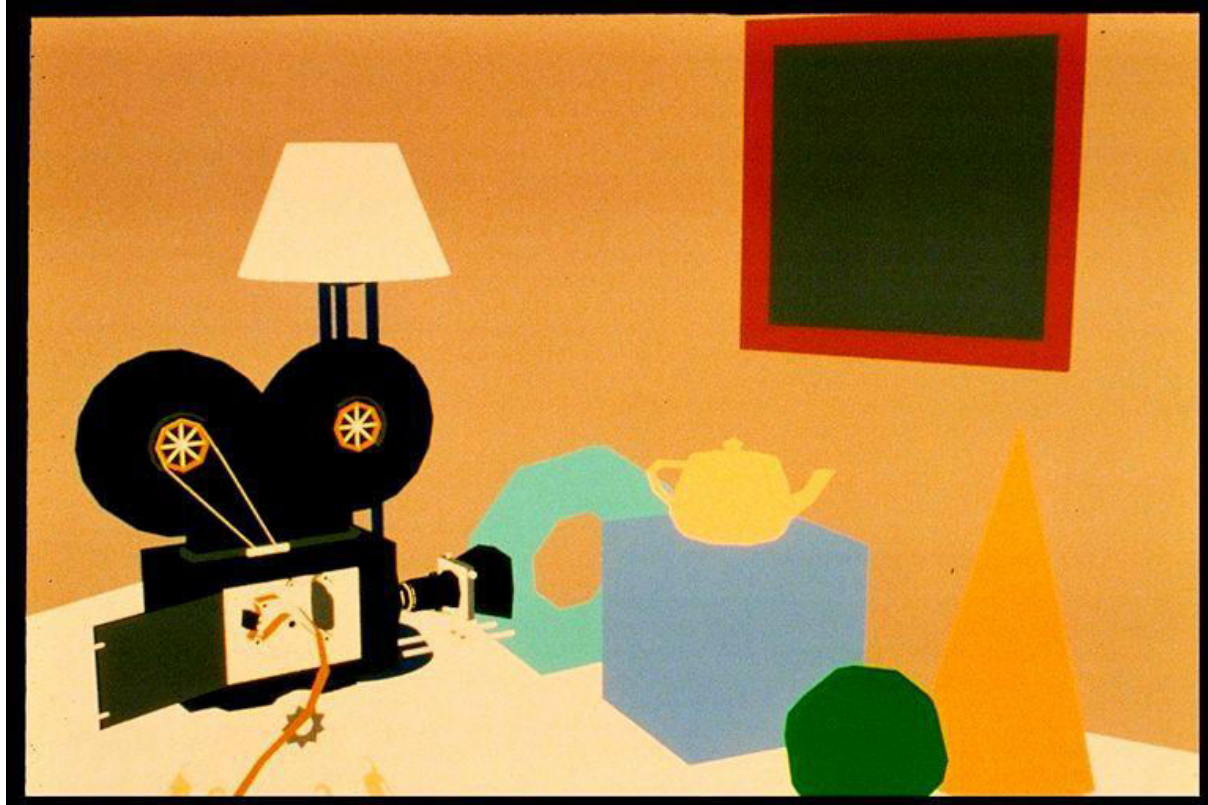


Flat Surface Rendering

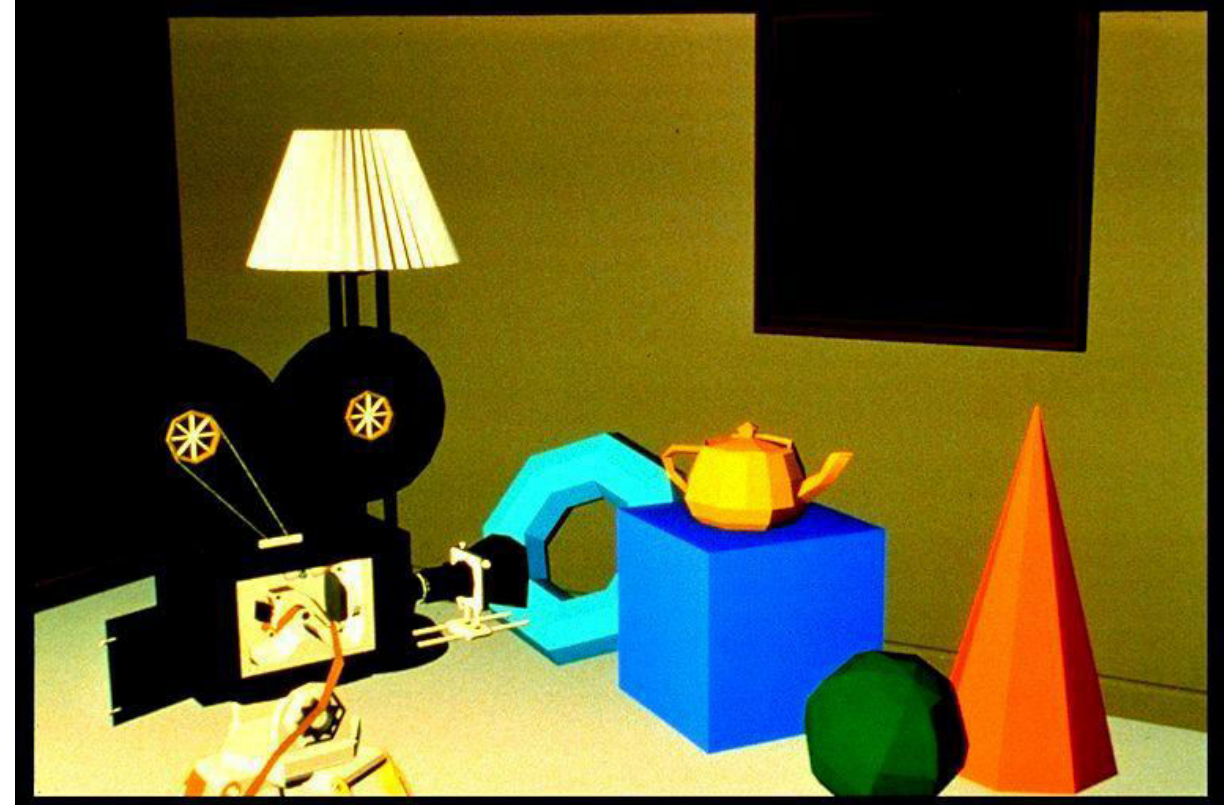
- ❑ Ok if:
 - Object consists of planar faces, and
 - Light sources are far away, and
 - Eye point is far away,
- ❑ or
 - Polygons are about a pixel in size.
- ❑ Surface rendering is extremely fast, but can be unrealistic
 - Highlights not visible,
 - Facetted appearance, increased by Mach banding effect.



No Surface Rendering Vs Flat Surface Rendering



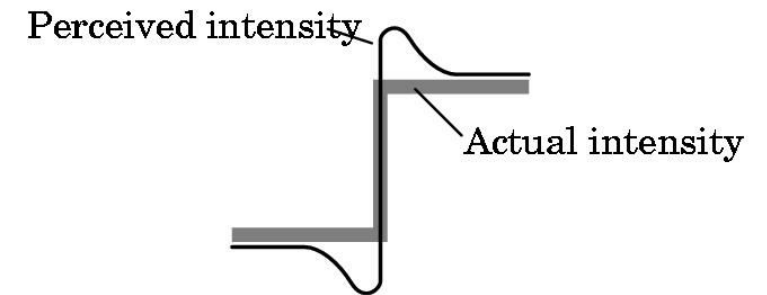
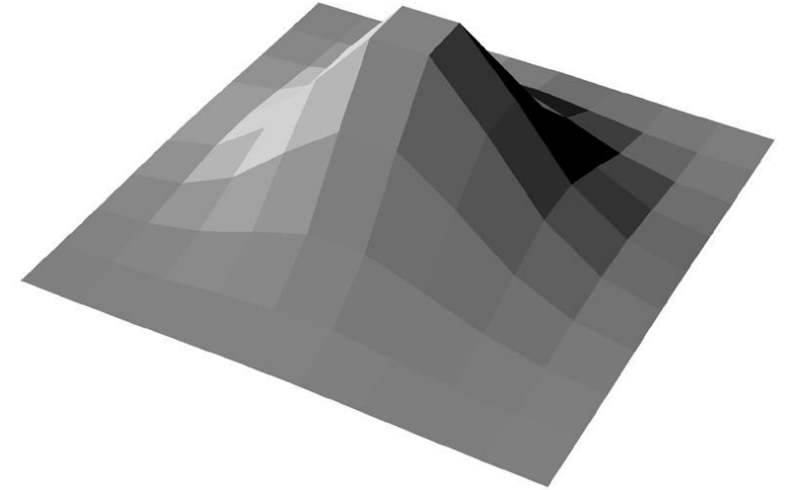
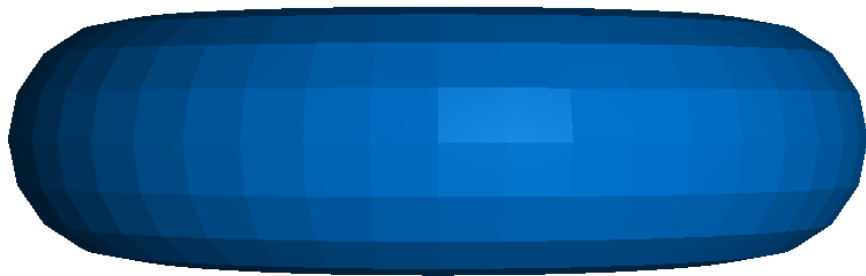
No Surface Rendering



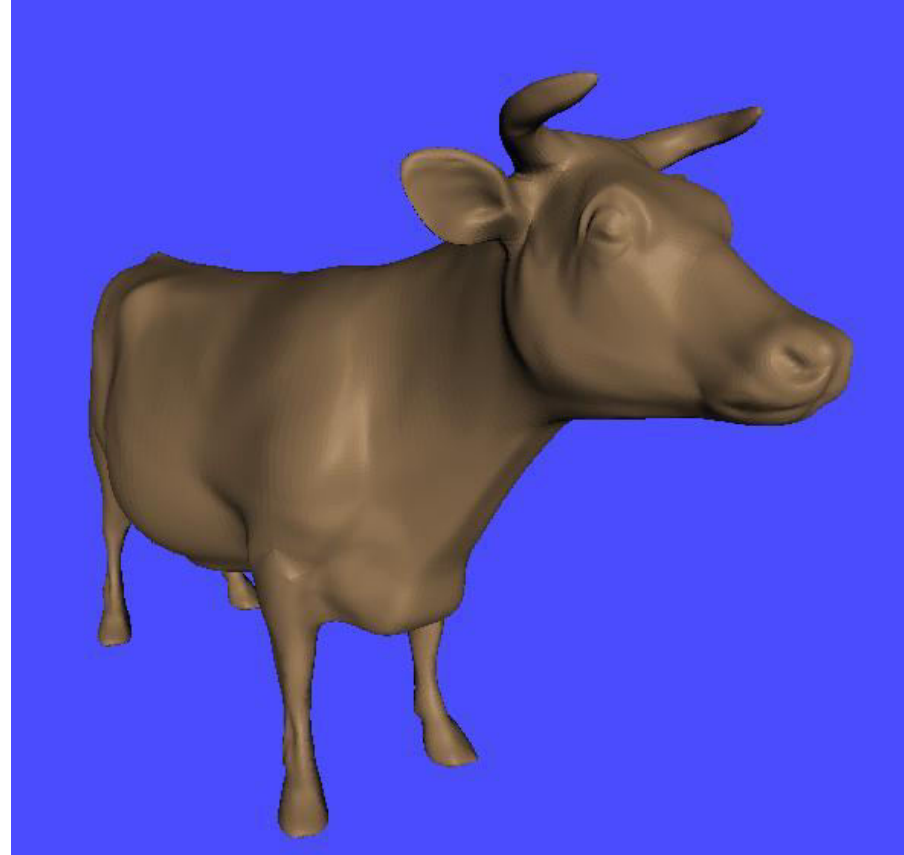
Flat Surface Rendering

Flat shading drawbacks

- ❑ The human visual system enhances edges
- ❑ We see stripes (known as **Mach Bands**) along edges
- ❑ Much like a sharpening convolution!
- ❑ How to avoid?



Overcoming Flat Shading Limitations



- ❑ Just add lots and lots of polygons – however, this is SLOW!

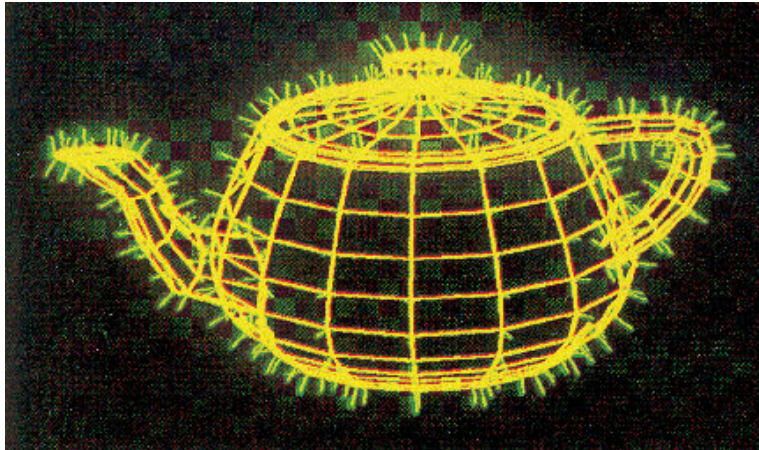
Gouraud Shading

- ❑ Developed in the 1970s by Henri Gouraud
- ❑ Worked at the University of Utah along with Ivan Sutherland and David Evans
- ❑ Often also called **intensity- interpolation surface rendering**
- ❑ Intensity levels are calculated at each vertex and interpolated across the surface



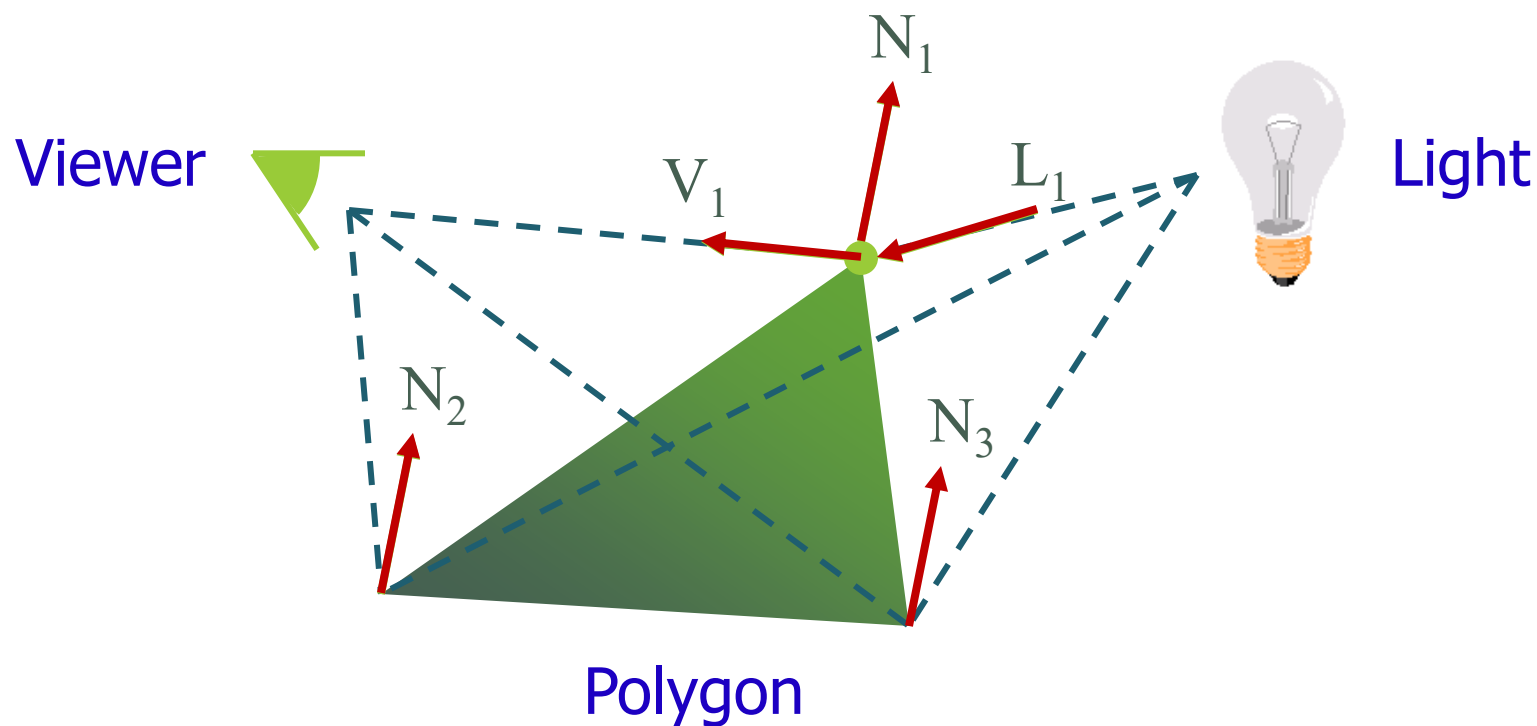
Gouraud Shading

- ❑ Smooth Surface are
 - Represented by polygonal mesh with a normal at each vertex



Gouraud Shading

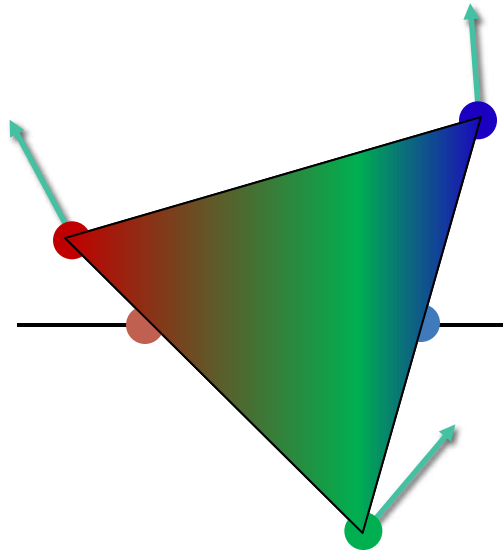
- ❑ One Lighting Calculation per Vertex
 - Assign pixels inside polygon by interpolating/ lerping colors computed at vertices



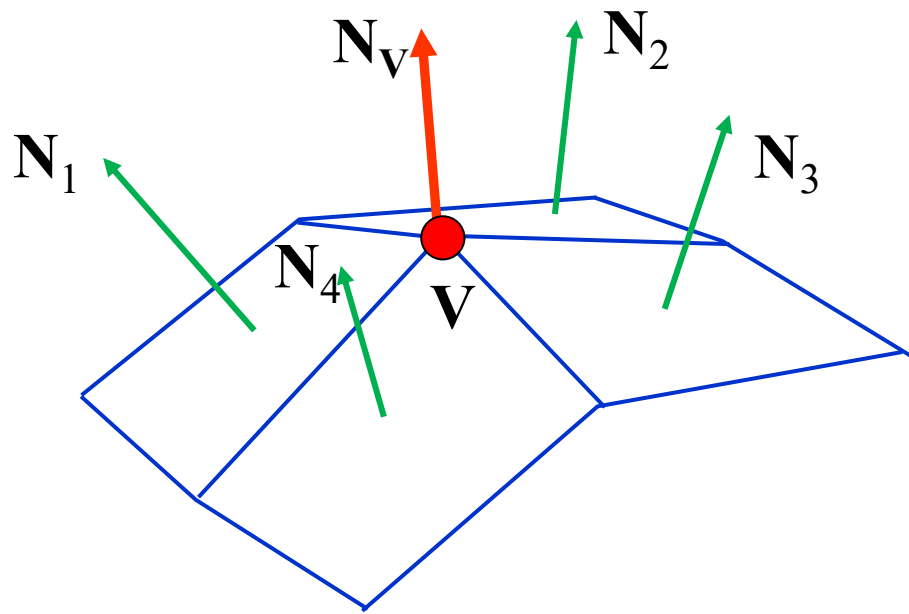
$$I = I_E + K_A I_{AL} + \sum_i (K_D (N \cdot L_i) I_i + K_S (V \cdot R_i)^n I_i)$$

Gouraud Shading

- ❑ To render a polygon, Gouraud surface rendering proceeds as follows:
 1. Determine the average unit **normal** vector **at each vertex** of the polygon
 2. **Apply an illumination model** at each polygon vertex to **obtain** the **light intensity** at that position
 3. **Linearly interpolate** the vertex intensities over the projected area of the polygon



Gouraud Shading



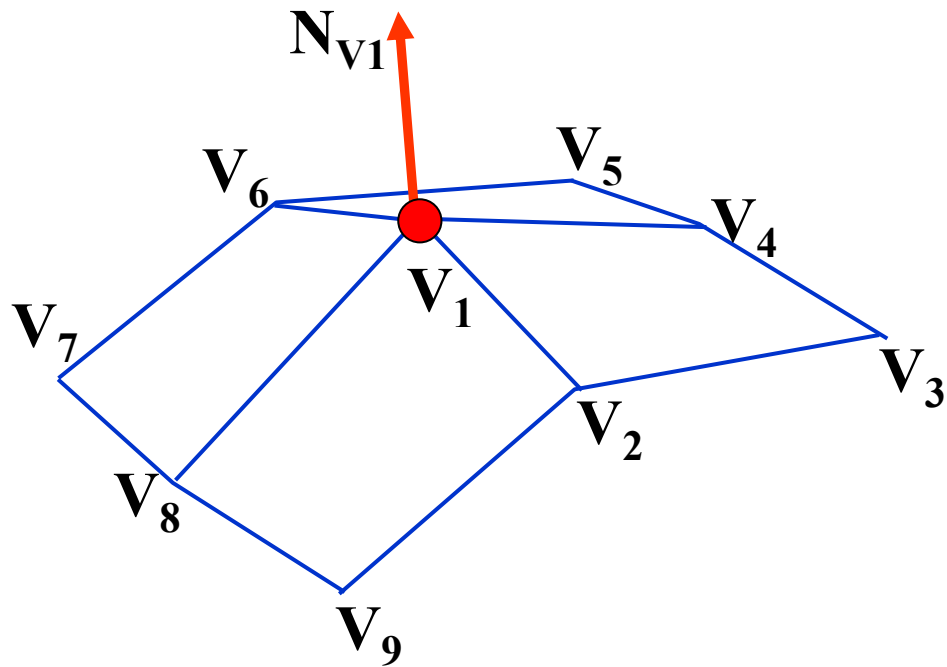
- The average unit normal vector at V is given as:

$$N_v = \frac{N_1 + N_2 + N_3 + N_4}{|N_1 + N_2 + N_3 + N_4|}$$

- or more generally:

$$N_v = \frac{\sum_{i=1}^n N_i}{\left| \sum_{i=1}^n N_i \right|}$$

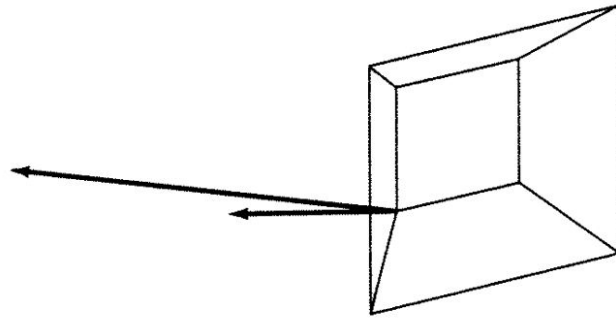
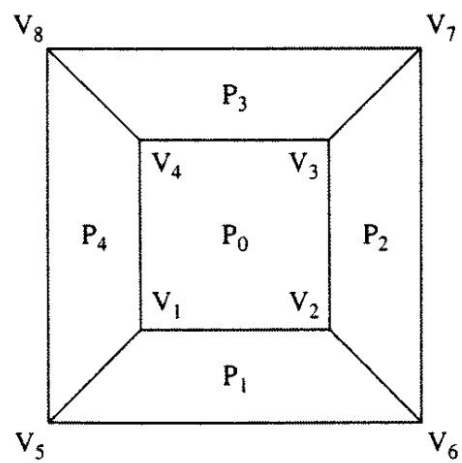
Gouraud Shading



- The average unit normal vector at V is given as:

$$N = V_1V_2 \times V_1V_4 + V_1V_4 \times V_1V_6 + V_1V_6 \times V_1V_8 \\ + V_1V_8 \times V_1V_2$$

$$N_{v1} = \frac{N}{|N|}$$



$$P_0: \quad z - 1 = 0$$

$$P_1: \quad -y + z - 2 = 0$$

$$P_4: \quad -x + z - 2 = 0$$

Consider the polygonal surface shown in Fig. 5–9a. The vertex points are $V_1(-1, -1, 1)$, $V_2(1, -1, 1)$, $V_3(1, 1, 1)$, $V_4(-1, 1, 1)$, $V_5(-2, -2, 0)$, $V_6(2, -2, 0)$, $V_7(2, 2, 0)$, $V_8(-2, 2, 0)$. The surface is a truncated pyramid. The plane equations for the faces labeled P_0, P_1, P_4 surrounding V_1 are

Approximating the normal at V_1

$$\begin{aligned} \mathbf{n}_1 &= (a_0 + a_1 + a_4)\mathbf{i} + (b_0 + b_1 + b_4)\mathbf{j} + (c_0 + c_1 + c_4)\mathbf{k} \\ &= (0 + 0 - 1)\mathbf{i} + (0 - 1 + 0)\mathbf{j} + (1 + 1 + 1)\mathbf{k} \\ &= -\mathbf{i} - \mathbf{j} + 3\mathbf{k} \end{aligned}$$

$$|\mathbf{n}_1| = \sqrt{(-1)^2 + (-1)^2 + (3)^2} = \sqrt{11}$$

$$\frac{\mathbf{n}_1}{|\mathbf{n}_1|} = -0.3\mathbf{i} - 0.3\mathbf{j} + 0.9\mathbf{k}$$

$$\mathbf{V}_1 \mathbf{V}_2 \otimes \mathbf{V}_1 \mathbf{V}_4 = 4\mathbf{k}$$

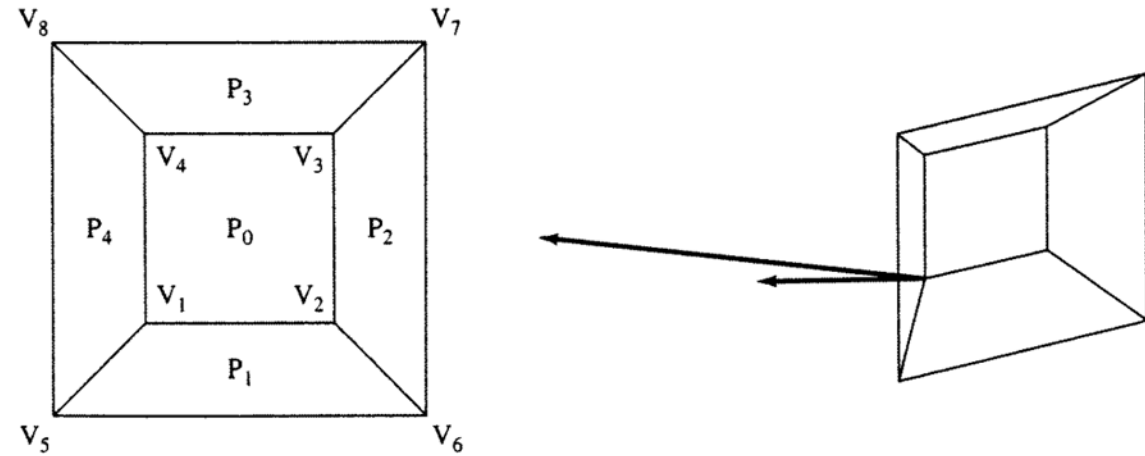
$$\mathbf{V}_1 \mathbf{V}_5 \otimes \mathbf{V}_1 \mathbf{V}_2 = -2\mathbf{j} + 2\mathbf{k}$$

$$\mathbf{V}_1 \mathbf{V}_4 \otimes \mathbf{V}_1 \mathbf{V}_5 = -2\mathbf{i} + 2\mathbf{k}$$

$$\mathbf{n}_1 = -\frac{2}{3}\mathbf{i} - \frac{2}{3}\mathbf{j} + \frac{8}{3}\mathbf{k}$$

$$|\mathbf{n}_1| = \frac{1}{3} \sqrt{(-2)^2 + (-2)^2 + (8)^2} = \frac{1}{3} \sqrt{72}$$

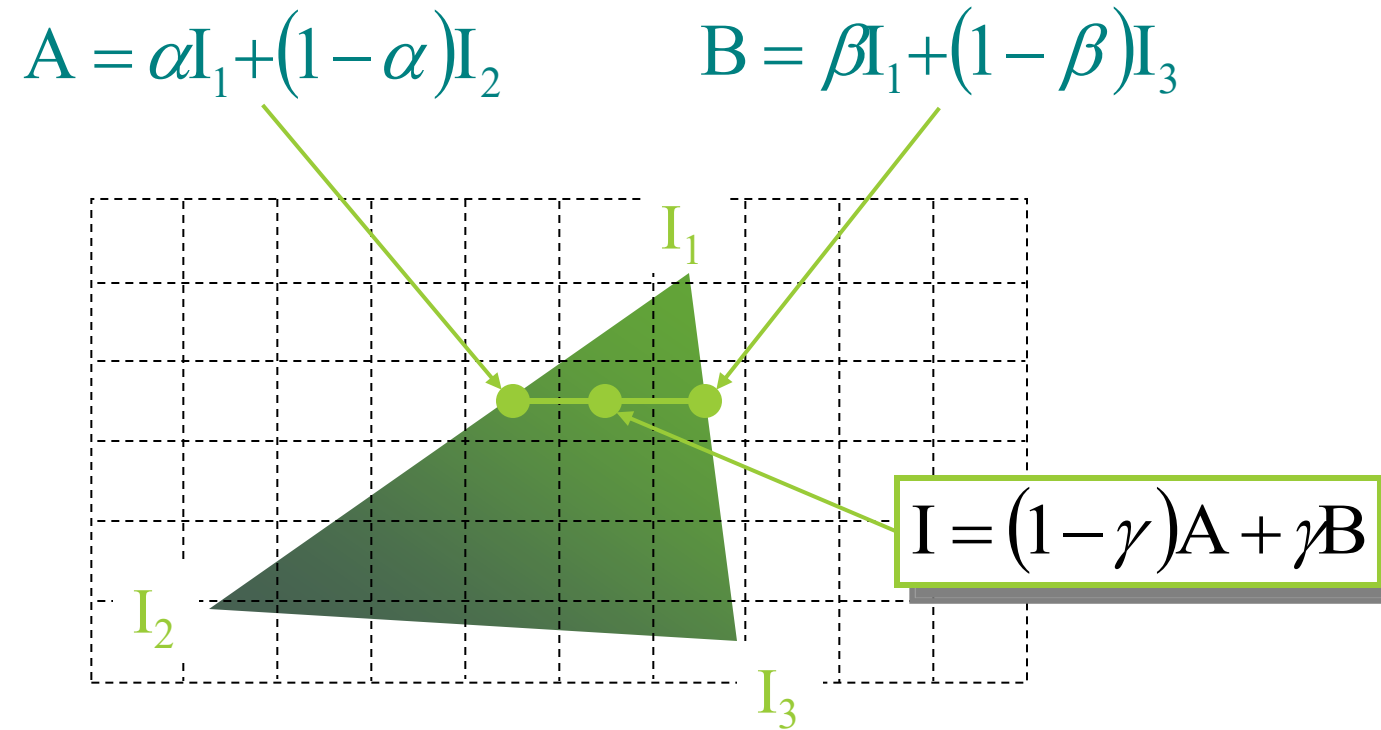
$$\frac{\mathbf{n}_1}{|\mathbf{n}_1|} = -0.24\mathbf{i} - 0.24\mathbf{j} + 0.94\mathbf{k}$$



Notice that both the direction and the magnitude of the normalized and unnormalized surface normals are different for different approximation techniques. This is shown in Fig. 5–9b. Consequently, an illumination model yields subtly different results depending on the technique used to approximate the surface normal.

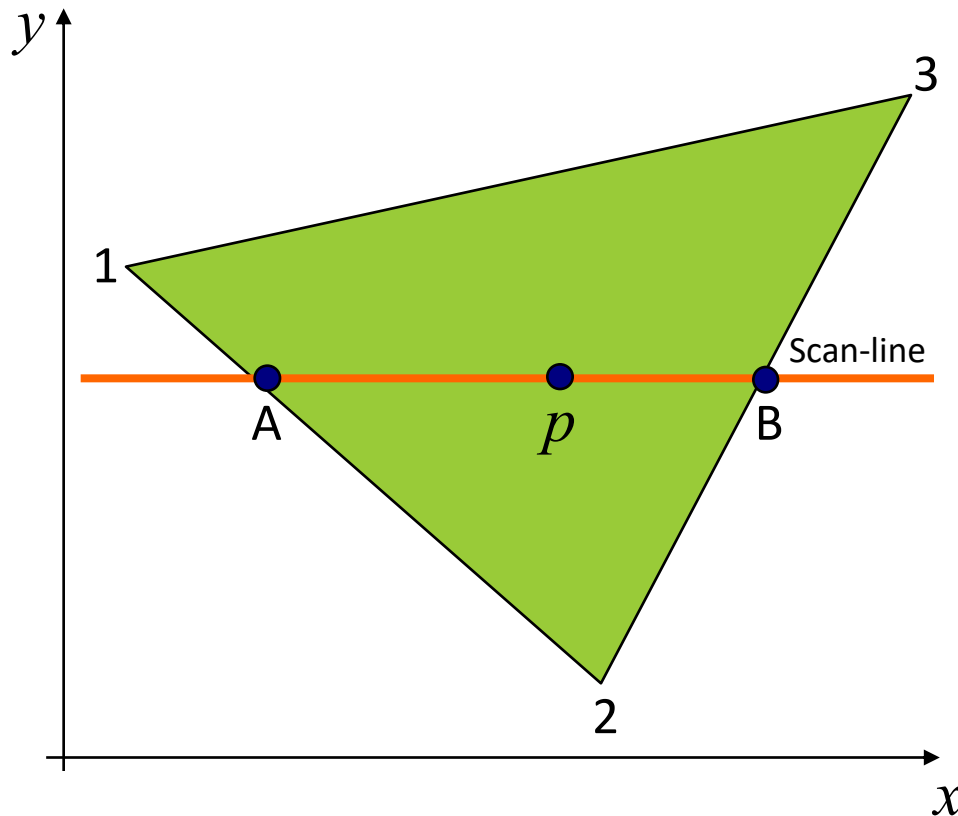
Gouraud Shading

- Bilinearly Interpolate Colors at Vertices Down and Across Scan Lines



Gouraud Shading

- Illumination values are bilinearly interpolated across each scan-line



$$I_A = \frac{y_A - y_2}{y_1 - y_2} I_1 + \frac{y_1 - y_A}{y_1 - y_2} I_2$$

$$I_B = \frac{y_B - y_2}{y_3 - y_2} I_3 + \frac{y_3 - y_B}{y_3 - y_2} I_2$$

$$I_p = \frac{x_B - x_p}{x_B - x_A} I_A + \frac{x_p - x_A}{x_B - x_A} I_B$$

Gouraud Shading

- ❑ Illumination values are bilinearly interpolated across each scan-line

The intensity calculation along the scan line can be performed incrementally.
For two pixels at t_1 and t_2 on the scan line

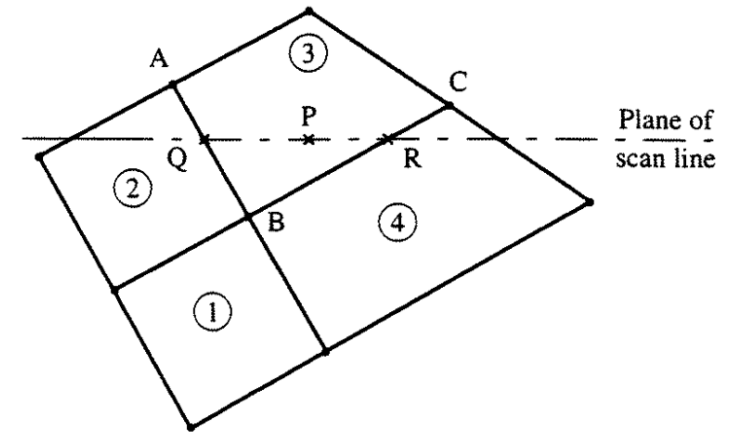
$$I_{P_2} = t_2 I_Q + (1 - t_2) I_R$$

and

$$I_{P_1} = t_1 I_Q + (1 - t_1) I_R$$

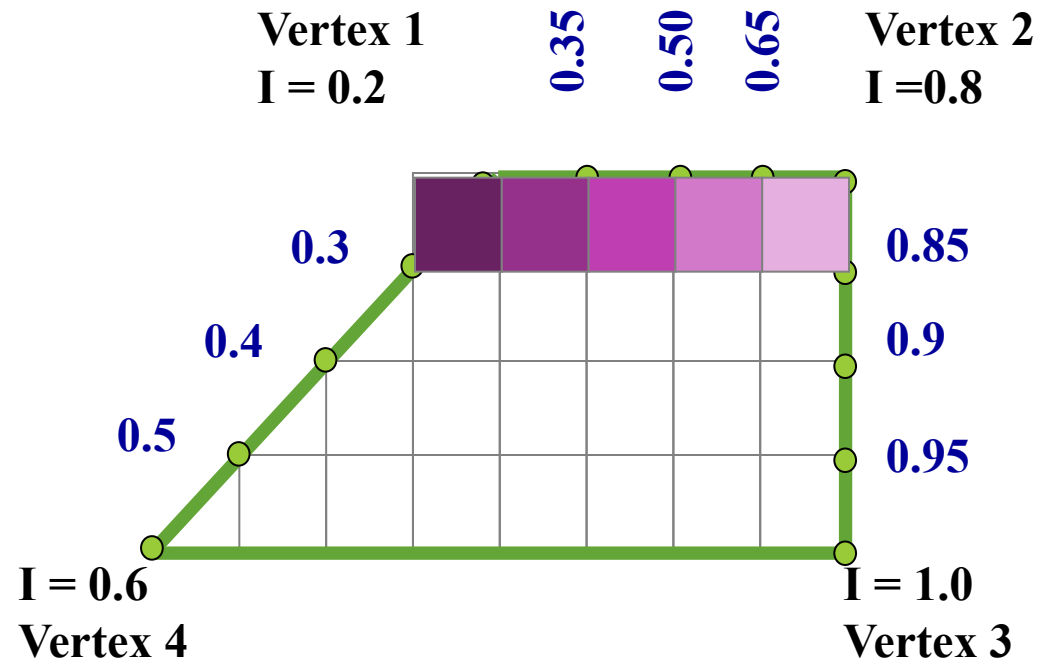
Subtracting yields

$$I_{P_2} = I_{P_1} + (I_Q - I_R)(t_2 - t_1) = I_{P_1} + \Delta I_{QR} \Delta t$$



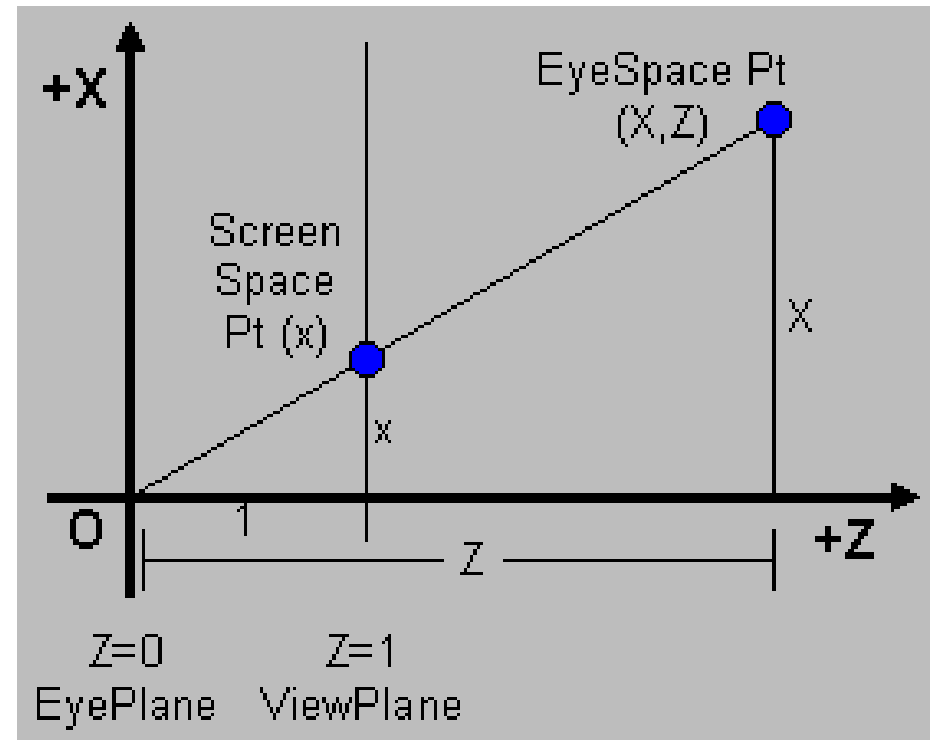
Gouraud Shading

An Example



Linear Interpolation Concerns

- ❑ Perspective projection complicates linear interpolation.
 - Relationship between screen space distance and eye space distance is nonlinear.
 - Therefore, relationship between interpolation in the two spaces is also nonlinear.
 - Thus, screen space linear interpolation of colors (and texture coordinates) results in incorrect values.
- ❑ Note: potential homework / test problem!



Perspectively-correct Interpolation

- ❑ Could interpolate in eye space, then project every interpolated point.
 - Way too much work!
- ❑ Can we interpolate in screen space and correct for perspective nonlinearity?
 - Yes!

Perspectively-correct Interpolation

- ❑ For a detailed derivation, see:
 - <http://www.cs.unc.edu/~hoff/techrep/persp/persp.html>
- ❑ Here, we skip to the punch line:
 - Given two eye space points, E_1 and E_2 .
 - Can lerp in eye space: $E(T) = E_1(1-T) + E_2(T)$.
 - T is eye space parameter, t is screen space parameter.
 - To see relationship, express in terms of screen space t :
 - $E(t) = [(E_1/Z_1)*(1-t) + (E_2/Z_2)*t] / [(1/Z_1)*(1-t) + (1/Z_2)*t]$

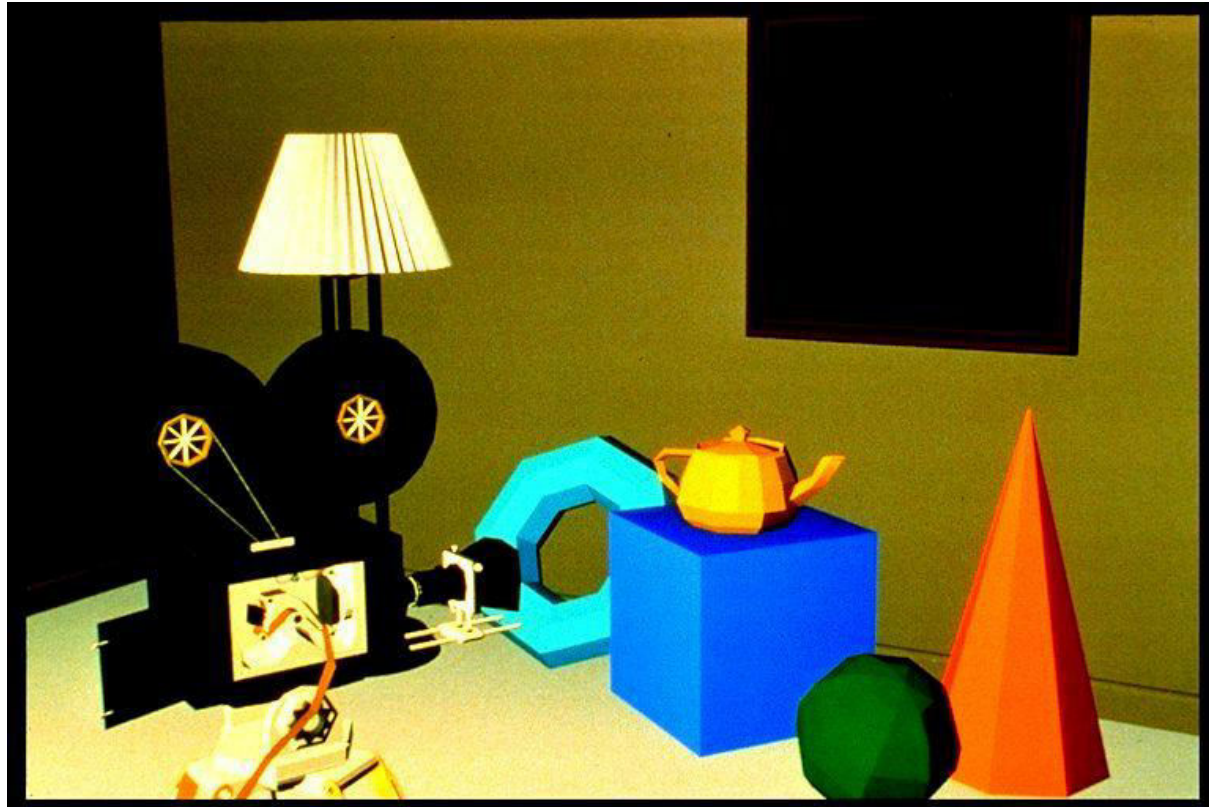
Perspectively-correct Interpolation

□ $E(t) = [(E_1/Z_1)*(1-t) + (E_2/Z_2)*t] / [(1/Z_1)*(1-t) + (1/Z_2)*t]$

- $E_1/Z_1, E_2/Z_2$ are projected points.
- Because Z_1, Z_2 are depths corresponding to E_1, E_2 .

- Looking closely, can see that interpolation along an eye space edge = interpolation along projected edge in screen space divided by the interpolation of $1/Z$.

Flat Vs Gouraud Rendering



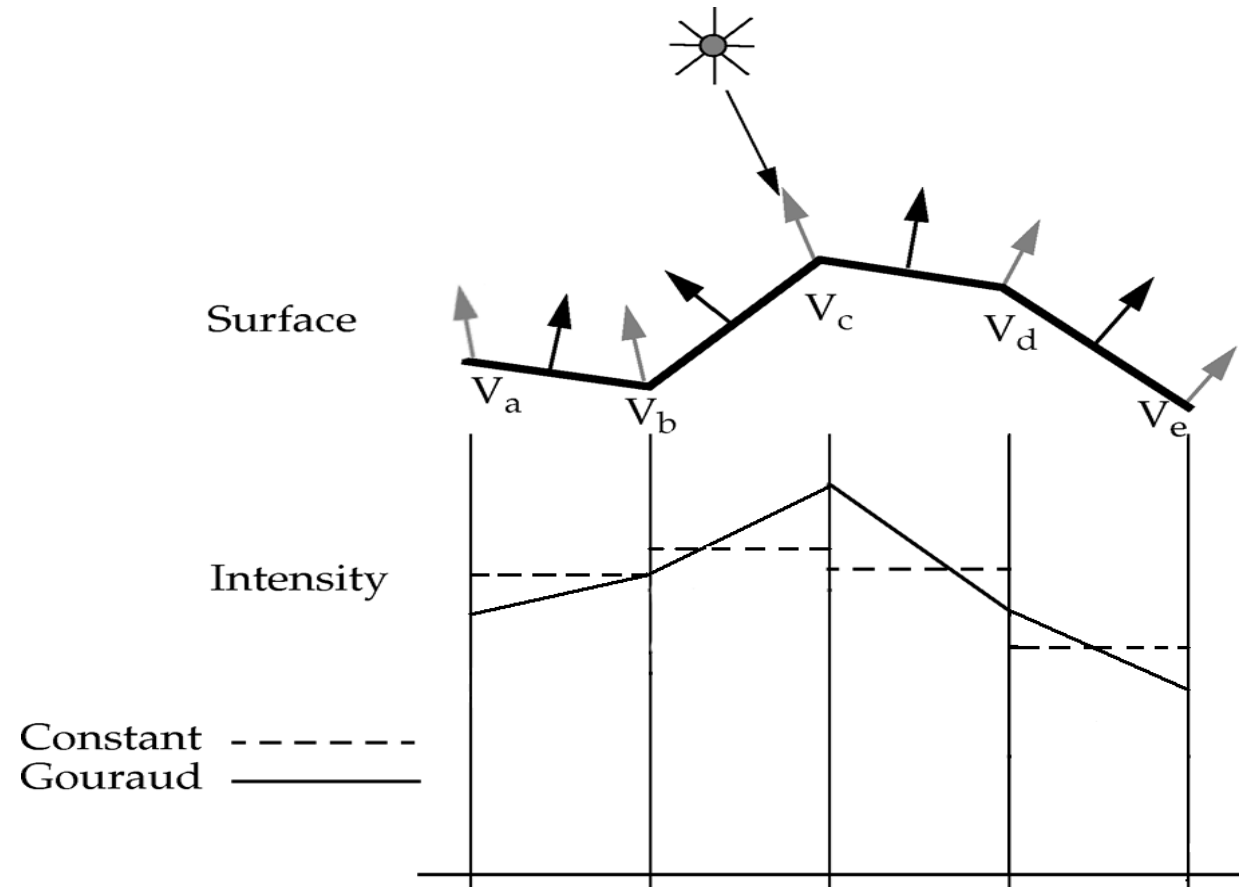
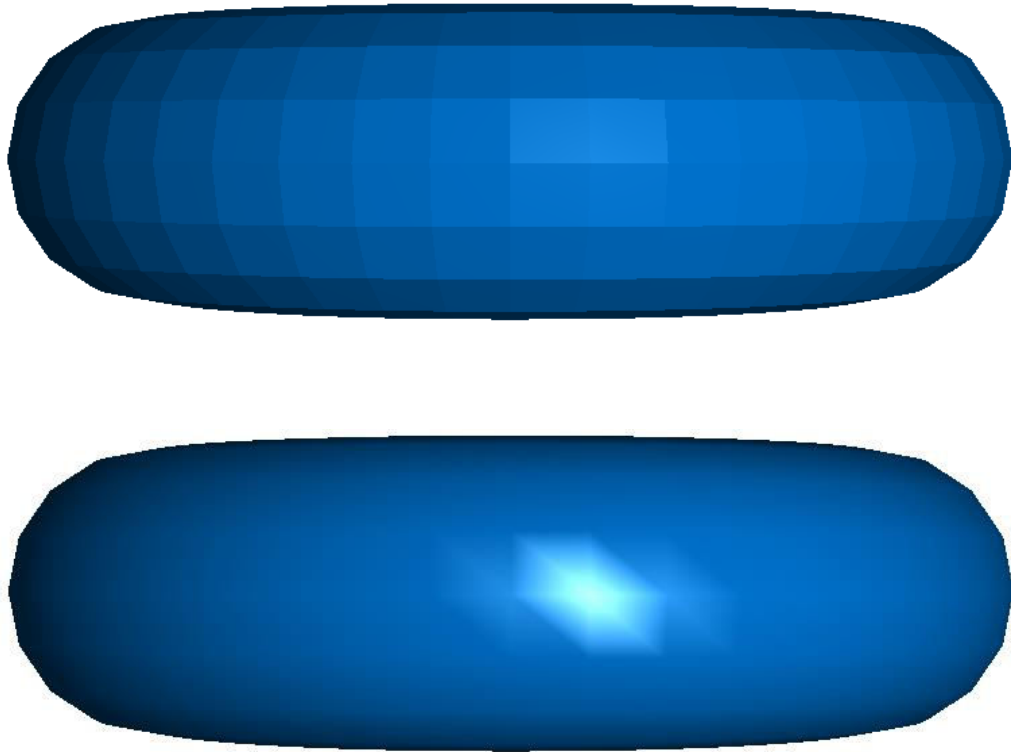
Flat Surface Rendering



Gouraud Surface Rendering

Gouraud Shading

- ❑ Much better result for curved surfaces



Gouraud Shading - Drawbacks

- ❑ Polygon edges are still visible
- ❑ Brightness is modelled as a linear function, but that's not really accurate
- ❑ Real highlights are small and bright, and drop off sharply
 - If polygons are too large, highlights get distorted and dimmed (notice the funny shape)
- ❑ How to avoid these artifacts?



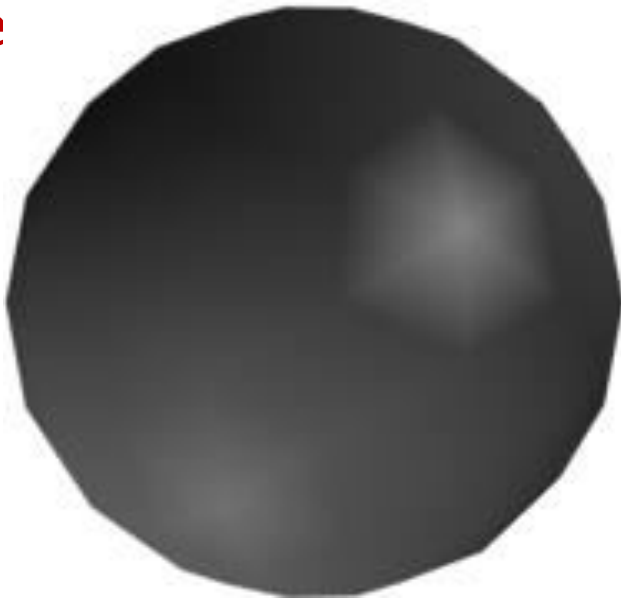
Gouraud Shading - Drawbacks

❑ It has a problem with

specular reflections

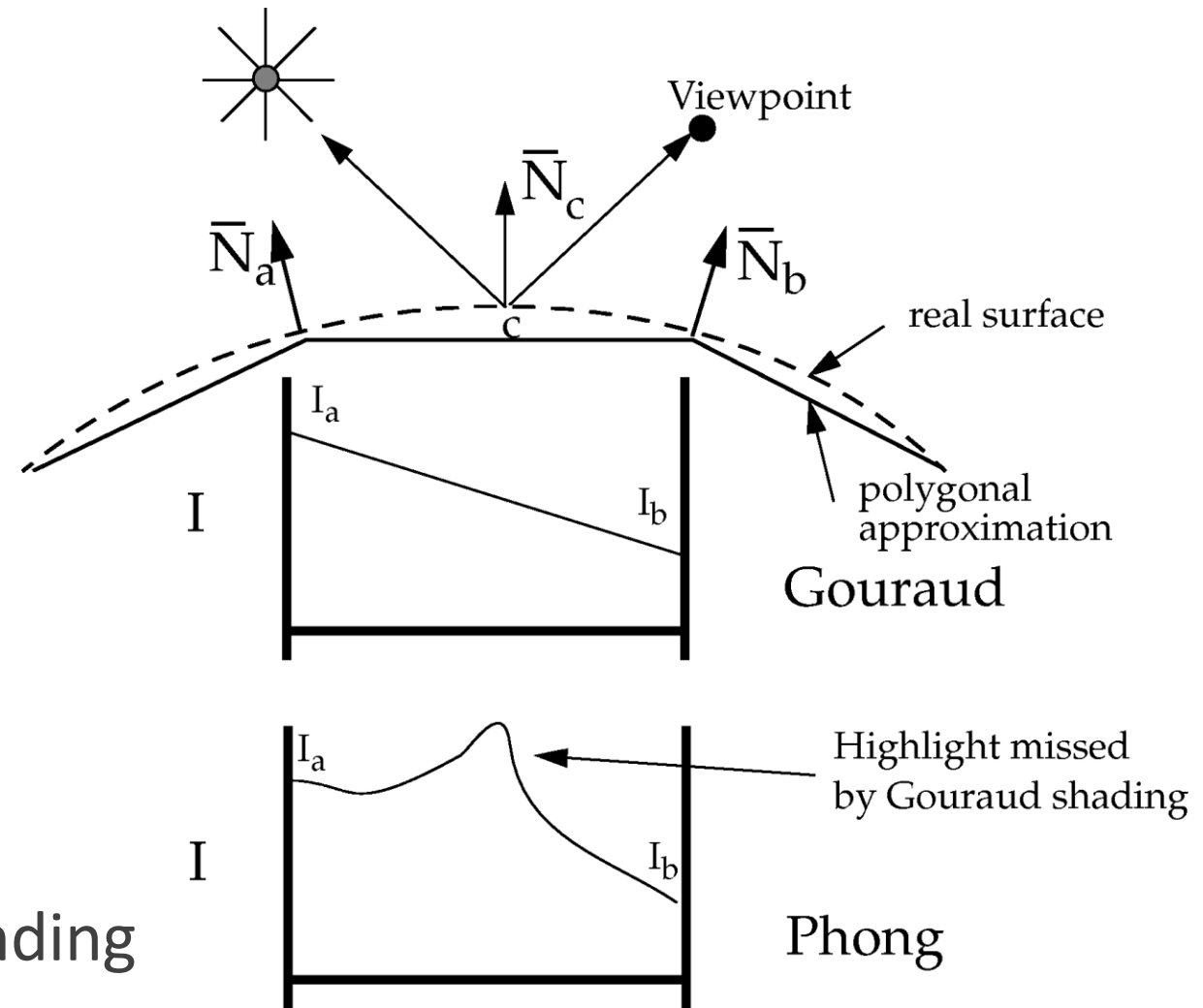
■ Completely miss

■ interpolate



❑ Linear interpolation still gives Mach banding

■ Silhouettes are still not smooth

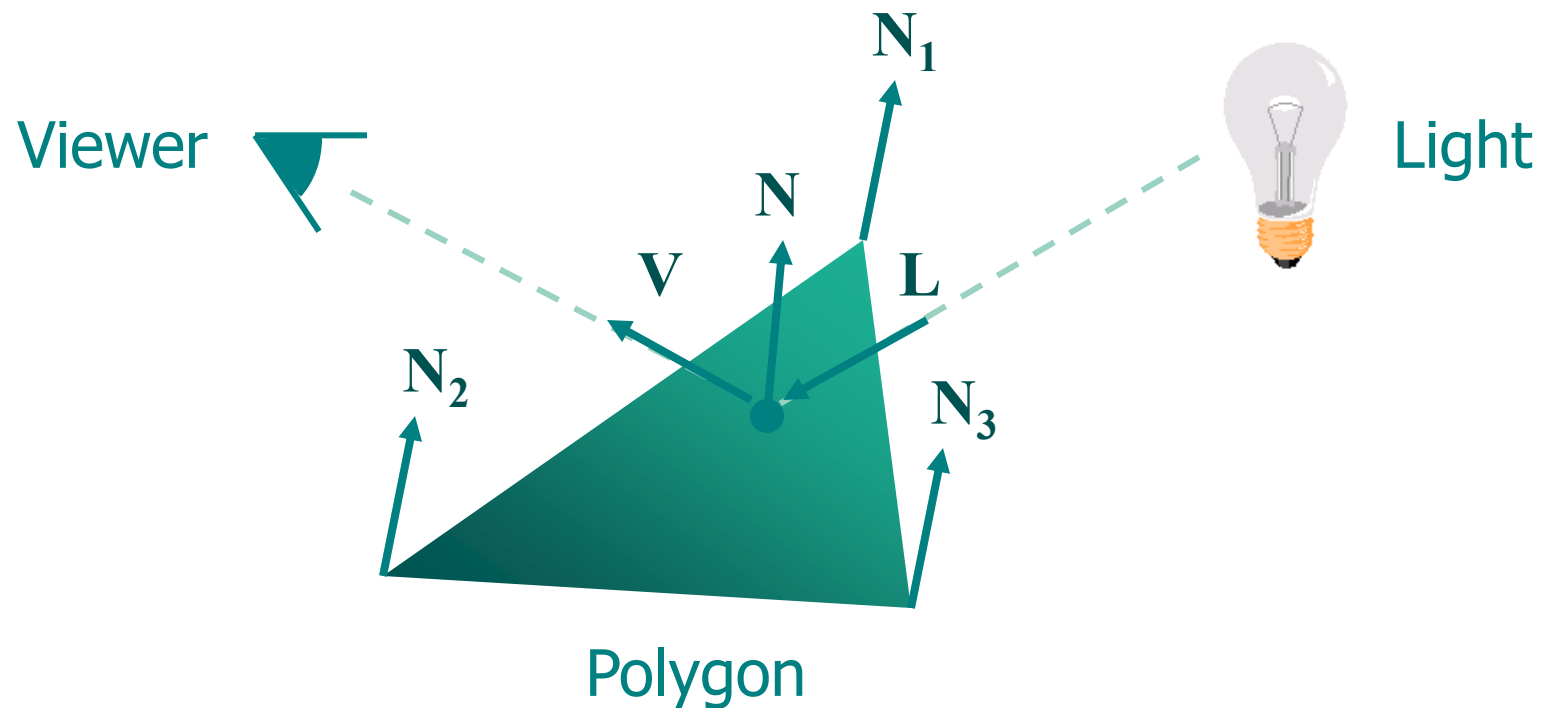


Phong Shading

- ❑ A more accurate interpolation based approach for rendering a polygon was developed by Phong Bui Tuong
- ❑ Basically the Phong surface rendering model (or **normal-vector interpolation rendering**) interpolates normal vectors instead of intensity values

Phong Shading

- ❑ One Lighting Calculation per Pixel
 - Approximate surface normals for points inside polygons by bilinear interpolation of normals from vertices



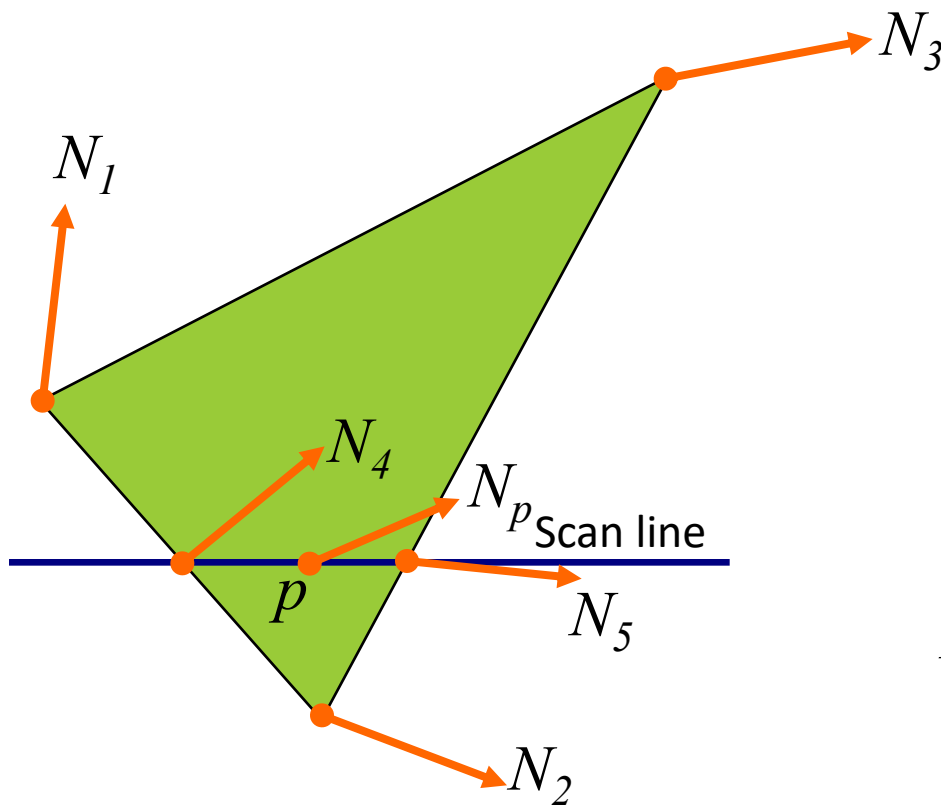
Phong Shading

❑ To render a polygon, Phong surface rendering proceeds as follows:

1. Determine the average unit normal vector at each vertex of the polygon
2. Linearly **interpolate** the vertex **normals** over the projected area of the polygon
 - Normalize it.
 - (Interpolation of unit vectors does not preserve length).
3. Apply an illumination model at positions along scan lines to calculate pixel intensities using the interpolated normal vectors

Phong Shading

- Bilinearly Interpolate Normals at Vertices Down and Across Scan Lines



$$N_4 = \frac{y_4 - y_2}{y_1 - y_2} N_1 + \frac{y_1 - y_4}{y_1 - y_2} N_2$$

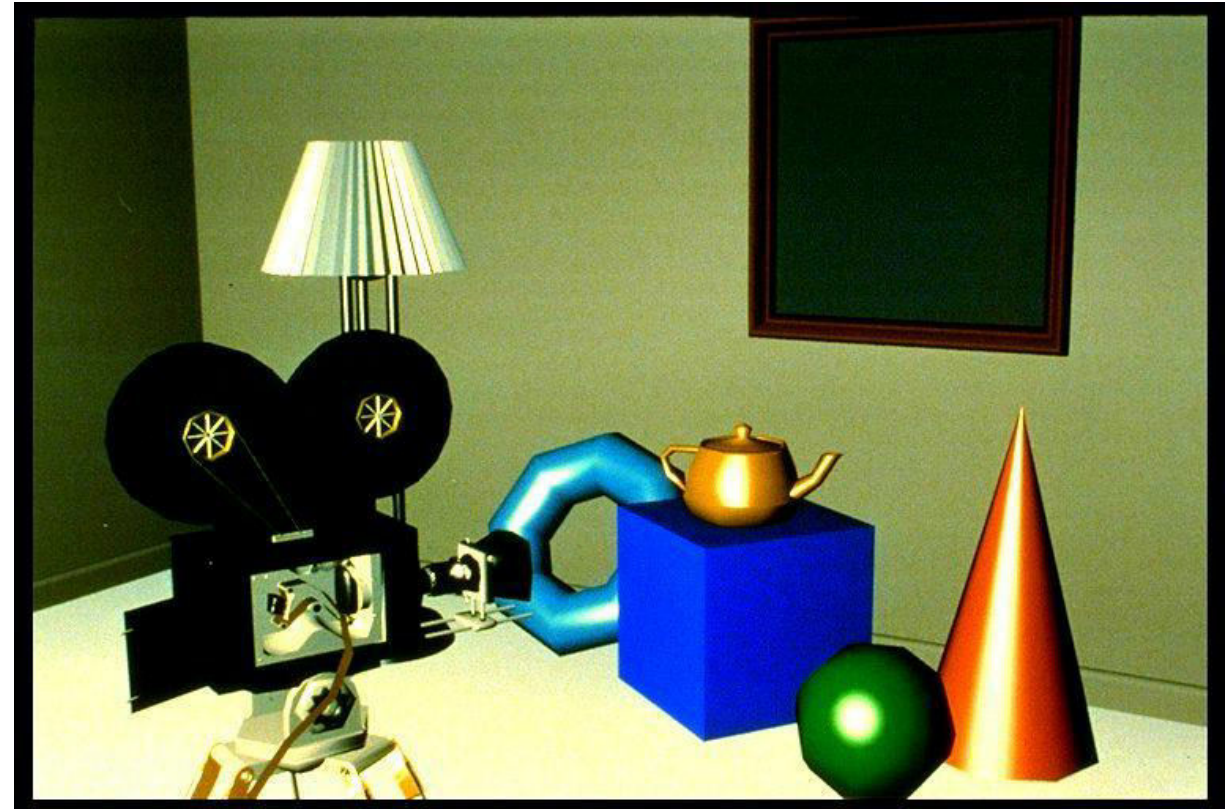
$$N_5 = \frac{y_5 - y_2}{y_3 - y_2} N_3 + \frac{y_3 - y_5}{y_3 - y_2} N_2$$

$$N_p = \frac{y_p - y_5}{y_4 - y_5} N_4 + \frac{y_4 - y_p}{y_4 - y_5} N_5$$

Gouraud Vs Phong Surface Rendering

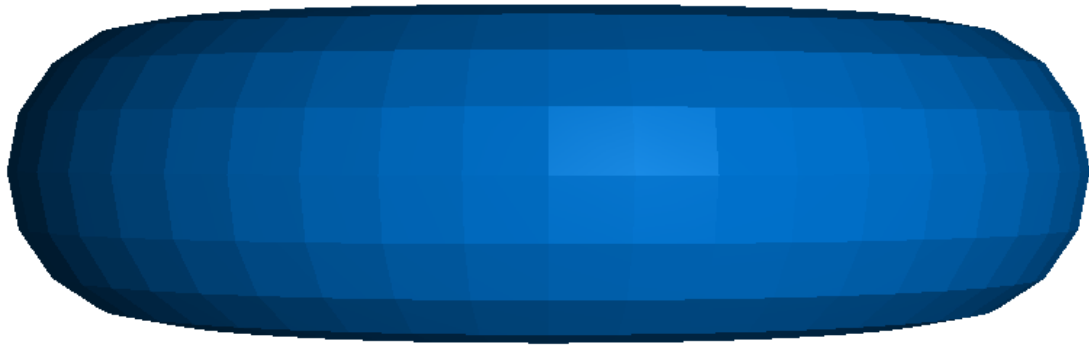


Gouraud Surface Rendering

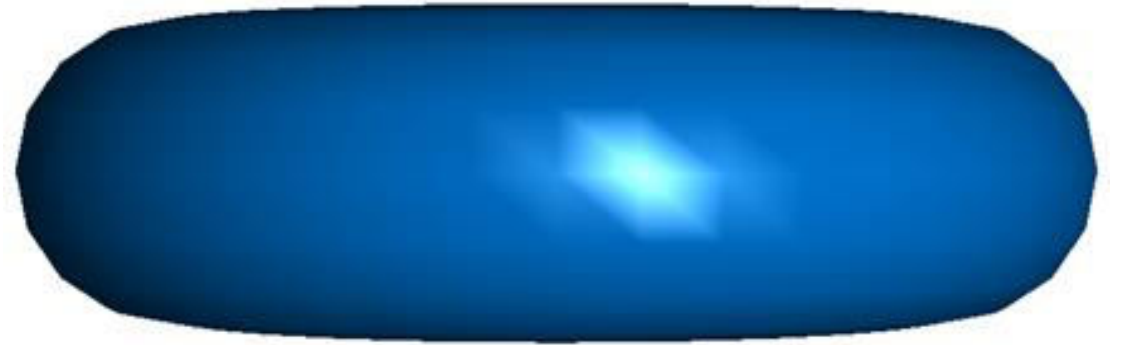


Phong Surface Rendering

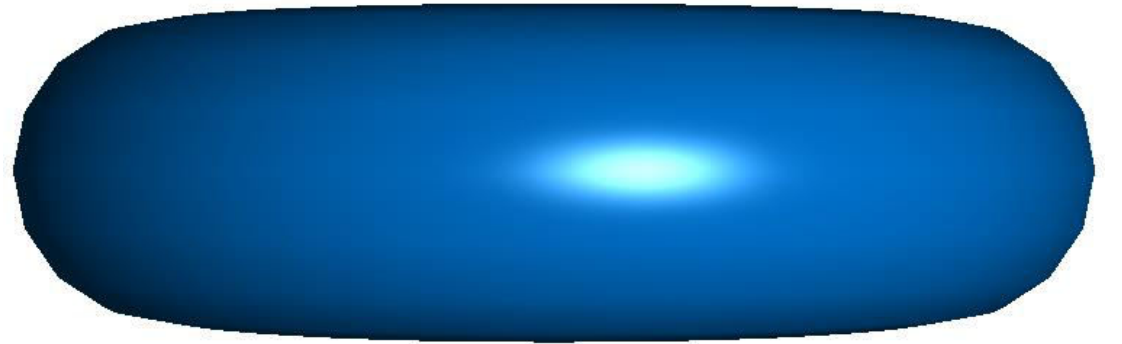
Phong Shading



Flat



Gouraud



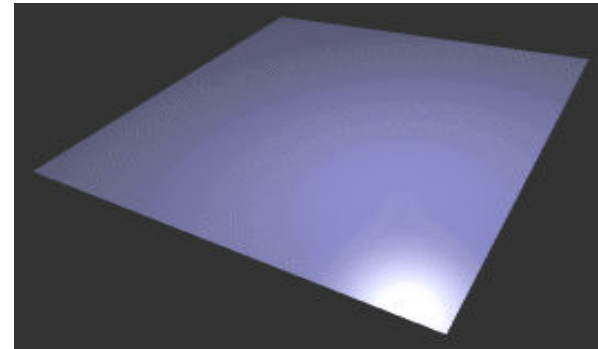
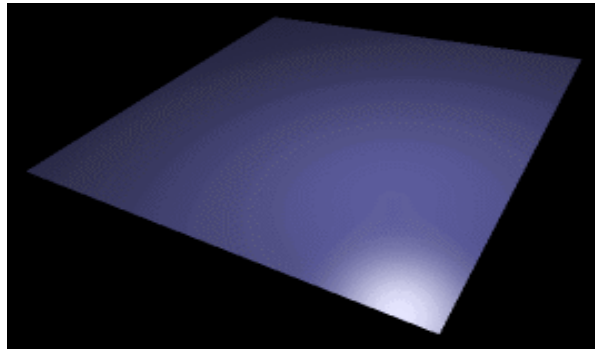
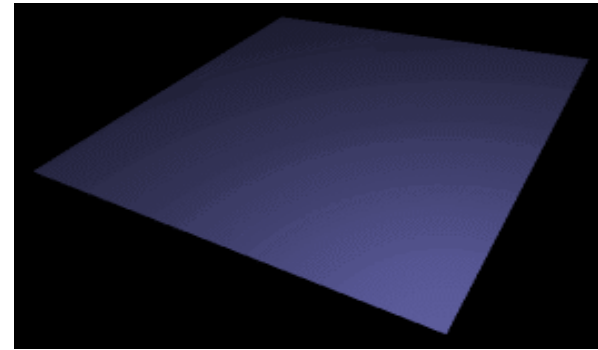
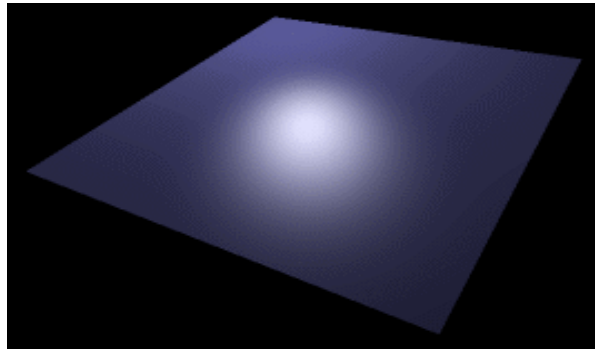
Phong

Phong Shading

- ❑ Even better result for curved surfaces
- ❑ No errors at high lights
- ❑ No Mach banding
- ❑ Phong shading is much slower than Gouraud shading as the lighting model is reevaluated so many times
- ❑ There are **fast Phong** surface rendering approaches that can be implemented iteratively
- ❑ Typically, implemented as part of a visible surface detection technique
- ❑ **Not supported in OpenGL**

Phong vs Gouraud Shading

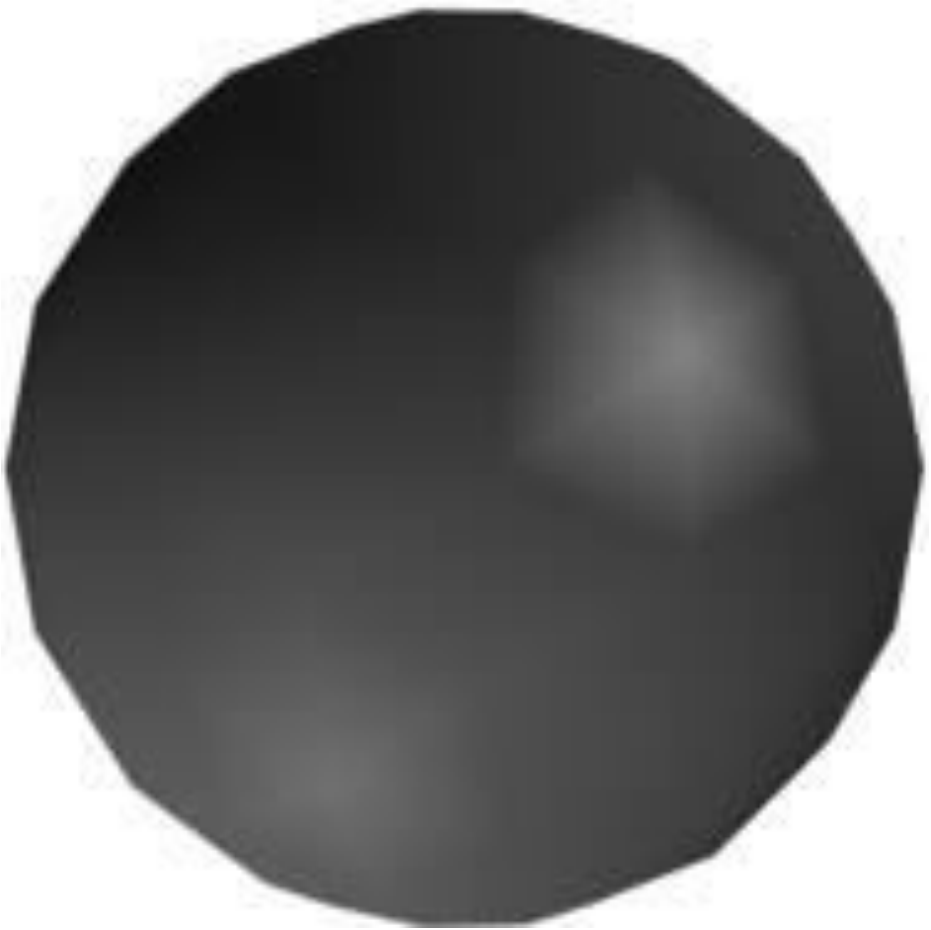
- ❑ If a highlight does not fall on a vertex Gouraud shading may miss it completely, but Phong shading does not.
- ❑ if highlight falls on vertex, Gouraud shading will spread the highlight over the polygon



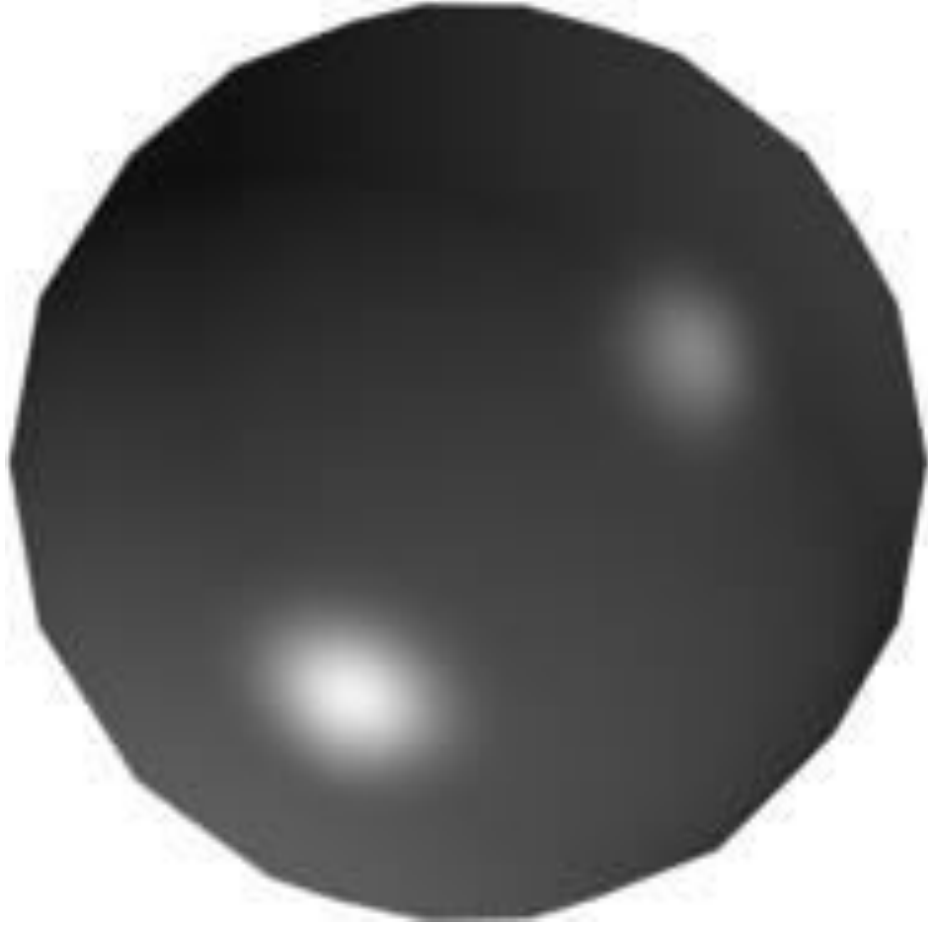
Gouraud Tea Pot Example

Phong Tea Pot Example

Phong vs Gouraud Shading



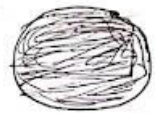
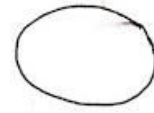
Gouraud



Phong

Phong vs Gouraud Shading – zero marks

(a) "Phong shading is more realistic than Gouraud shading, but expensive" – Yes, This statement is right. In the Gouraud shading this can be overcome the problem of flat shading. We know that colour is important to understand realistic view and the 3D view. ~~some~~ But shading is also important. In the ~~plane~~ object surface is completely plain, and there are no shading here, we can't understand the 3D object. We can understand when it will be 3D view. In the Gouraud shading, we can't understand the original view of a plane on object

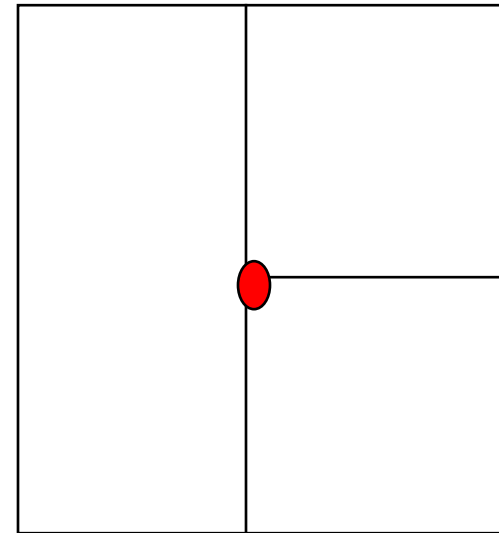
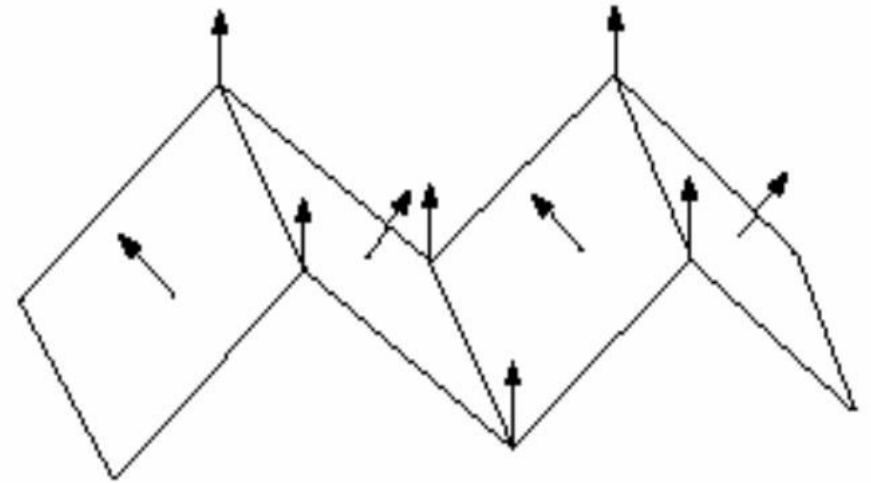


In gouraud shading, it can't make the real view of an object. But in phong shading, it overcome the problem of gouraud shading. It makes the object more realistic to the viewer.

Although phong shading is more realistic, but the cost of the phong shading is much more than gouraud shading. In the gouraud shading, it can't be more realistic but respectably cheaper than phong shading.

Interpolative Shading artifacts

- ❑ Vertex normal does not always reflect the curvature of the surface adequately
- ❑ Incorrect Vertex normals – no variation in shade
 - Appear less flat than actual
- ❑ The shading at the T-junction are different when calculated from different triangles
 - shared by right polygons and not by one on left
 - Shading discontinuity

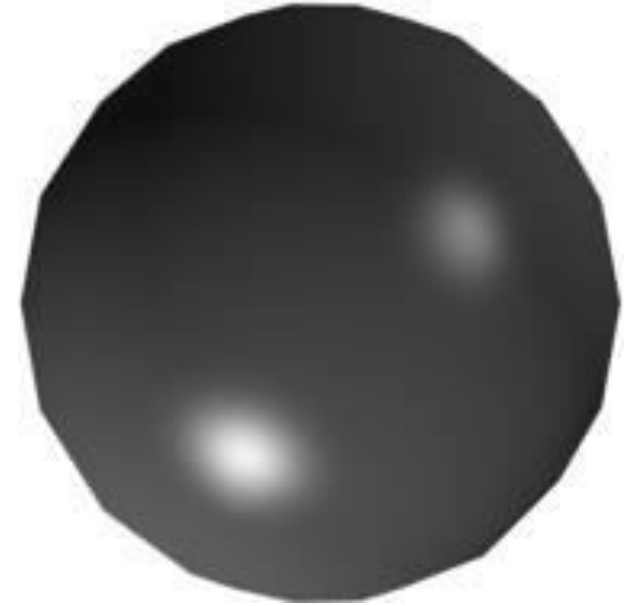
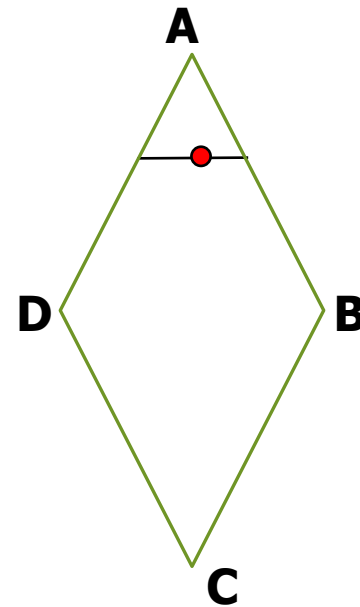
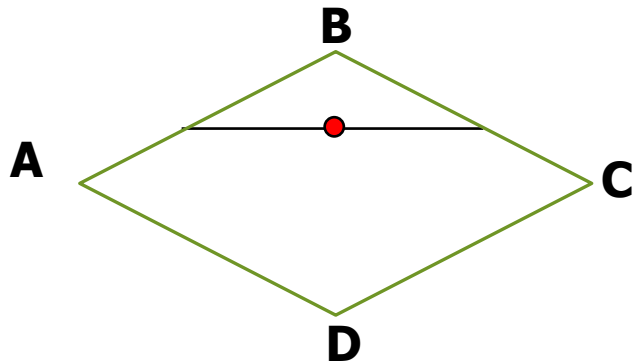


Interpolative Shading artifacts – Mach Bands

- ❑ Common in flat shading since shading is discontinuous at edges
- ❑ Also present in Gouraud shading
 - Gradient of the shading may change suddenly
- ❑ Phong shading reduces it significantly
 - But cannot be eliminated
 - At sharp changes in surface gradient

Interpolative Shading artifacts

- ❑ Polygonal silhouette – edge is always polygonal
- ❑ Perspective distortion – interpolation is in screen space and hence foreshortening takes place
- ❑ In both cases finer polygons can help !
- ❑ Orientation dependence - small rotations cause problems



Phong

Fast Phong Surface Rendering

It significantly reduces the amount of computations at each point by approximating some of the illumination components (G. Bishop and D.M. Weimer, 1986).

Recall that rendering is done for projected polygons.

Consider an object described by triangles.

We interpolate the normal at (x, y) as

$\mathbf{N}(x, y) = \mathbf{A}x + \mathbf{B}y + \mathbf{C}$, where $\mathbf{A}$, $\mathbf{B}$ and $\mathbf{C}$ are determined from the equalities

$$\mathbf{N}(x_k, y_k) = \mathbf{A}x_k + \mathbf{B}y_k + \mathbf{C}, \quad k = 1, 2, 3.$$

Fast Phong Surface Rendering

Considering the first five terms in Taylor expansion of $I_{\text{diffuse}}(x, y)$ yields

$$I_{\text{diffuse}}(x, y) = T_5x^2 + T_4xy + T_3y^2 + T_2x + T_1y + T_0.$$

The various T_i , $1 \leq i \leq 5$, are functions of $a, b, \dots, i$ calculated before, they are fixed for the entire triangle and can be stored in registers.

Fast Phong Surface Rendering

Since the progression of scan line is $x \rightarrow x + 1$ and $y \rightarrow y + 1$, $I_{\text{diffuse}}(x, y)$ is fully calculated for the first point and then only two additions per pixel suffice.

The method can be extended to account specular reflections terms $(\mathbf{N} \cdot \mathbf{H})^{n_s}$.

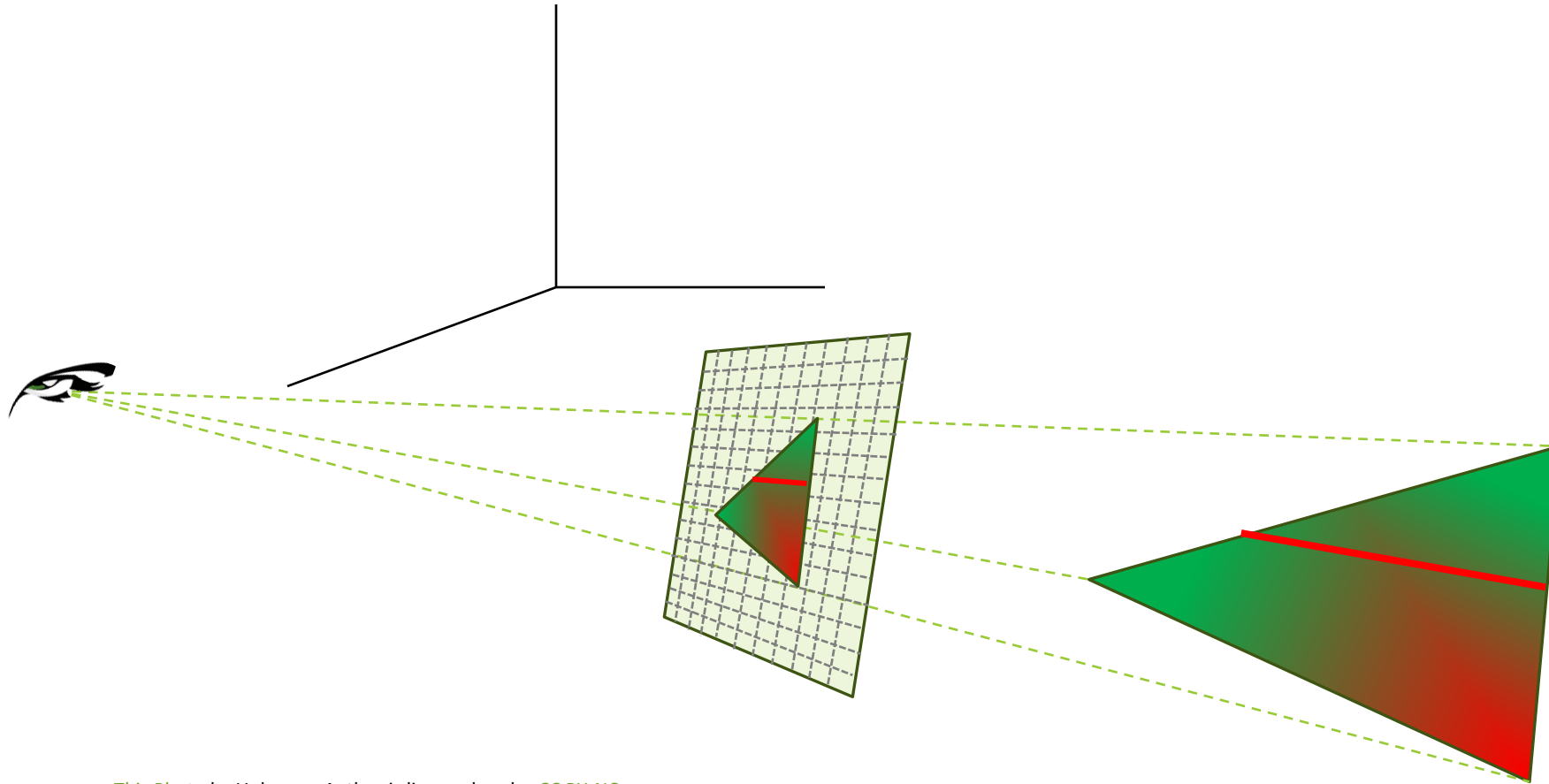
Other Types of Per-pixel Shading

- ❑ Ray tracing.
 - Doesn't use Gouraud or Phong shading.
 - Each pixel uses own ray to determine color.
 - Can apply arbitrary lighting model.
 - Classical (Whitted) ray tracing uses Phong model.
 - Since ray tracing determines colors based on intersections, don't have to use polygonal geometry.
 - Thus, can potentially use exact normals, rather than interpolation.

Summary

- ❑ For realistic rendering of polygons we need interpolation methods to determine lighting positions
- ❑ Flat shading is fast, but unrealistic
- ❑ Gouraud shading is better, but does not handle specular reflections very well
- ❑ Phong shading is even better, but can be slow

Problems - DIY



[This Photo](#) by Unknown Author is licensed under [CC BY-NC](#)

Shading

SURFACE RENDERING
METHODS

Contents

- Today we will start to look at rendering methods used in computer graphics
 - Flat surface rendering
 - Gouraud surface rendering
 - Phong surface rendering

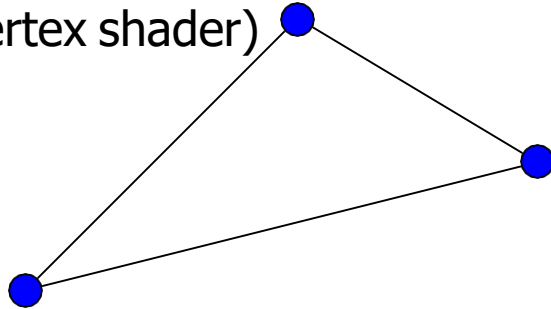
Shading

- ❑ Surface rendering means ***a procedure for applying a lighting model to obtain*** pixel intensities for all the projected surface positions in a scene.
- ❑ A surface-rendering algorithm uses the intensity calculations from an illumination model to determine the light intensity for all projected pixel positions for the various surfaces in a scene.
- ❑ Surface rendering can be performed by applying the illumination model to every visible surface point

Shading?

- After triangle is rasterized (converted to pixels)
 - Per-vertex lighting calculation means color at vertices is accurate, known (red dots)
- Shading: Graphics hardware figures out color of interior pixels (blue dots)
- How? Assume linear change => interpolate

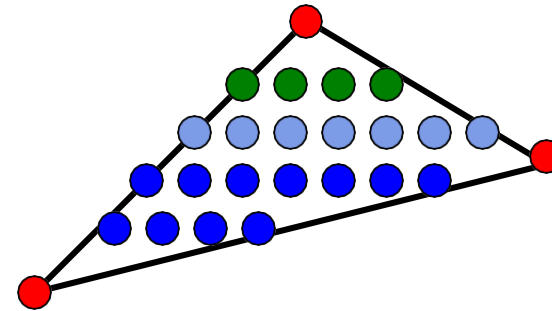
Lighting
(calc at vertices
in vertex shader)

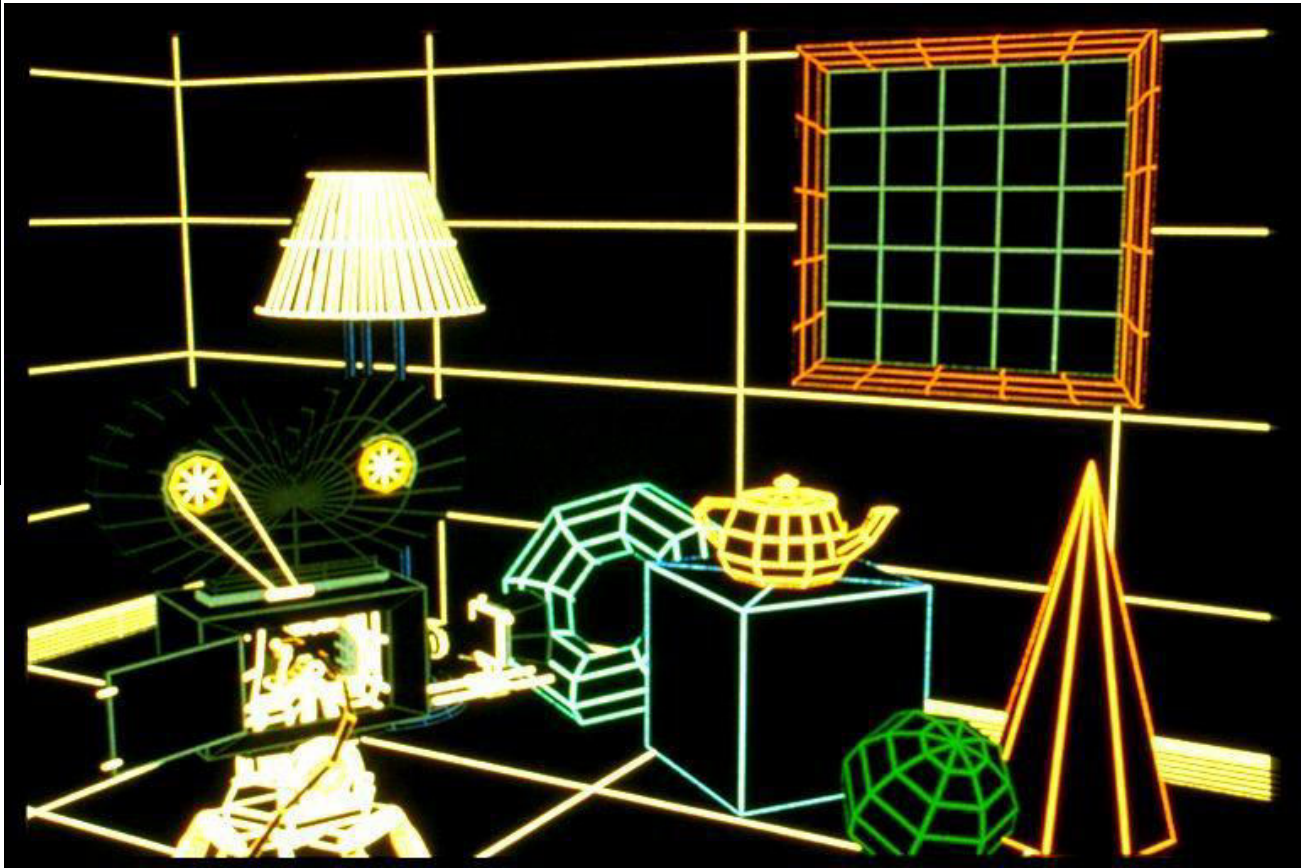
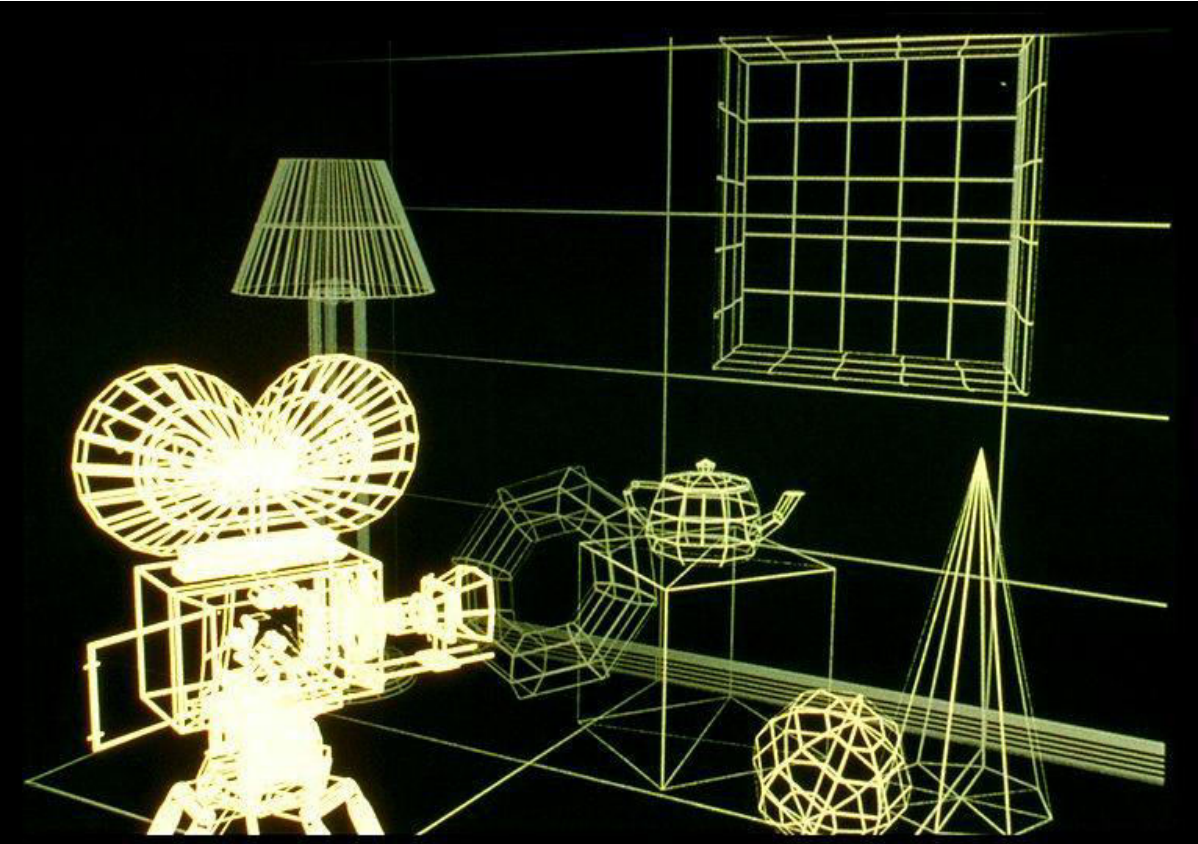


Rasterization
Find pixels belonging
to each object

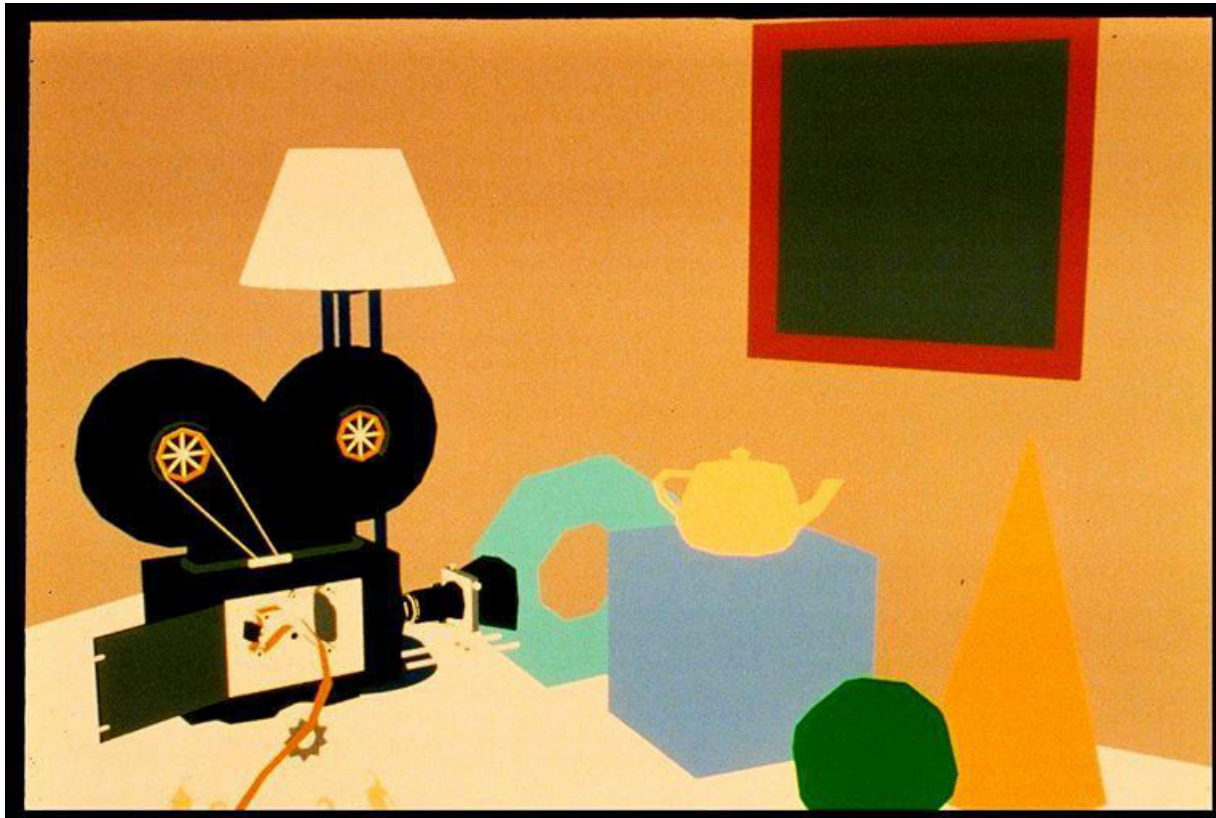


Shading
(done in hardware
during rasterization)

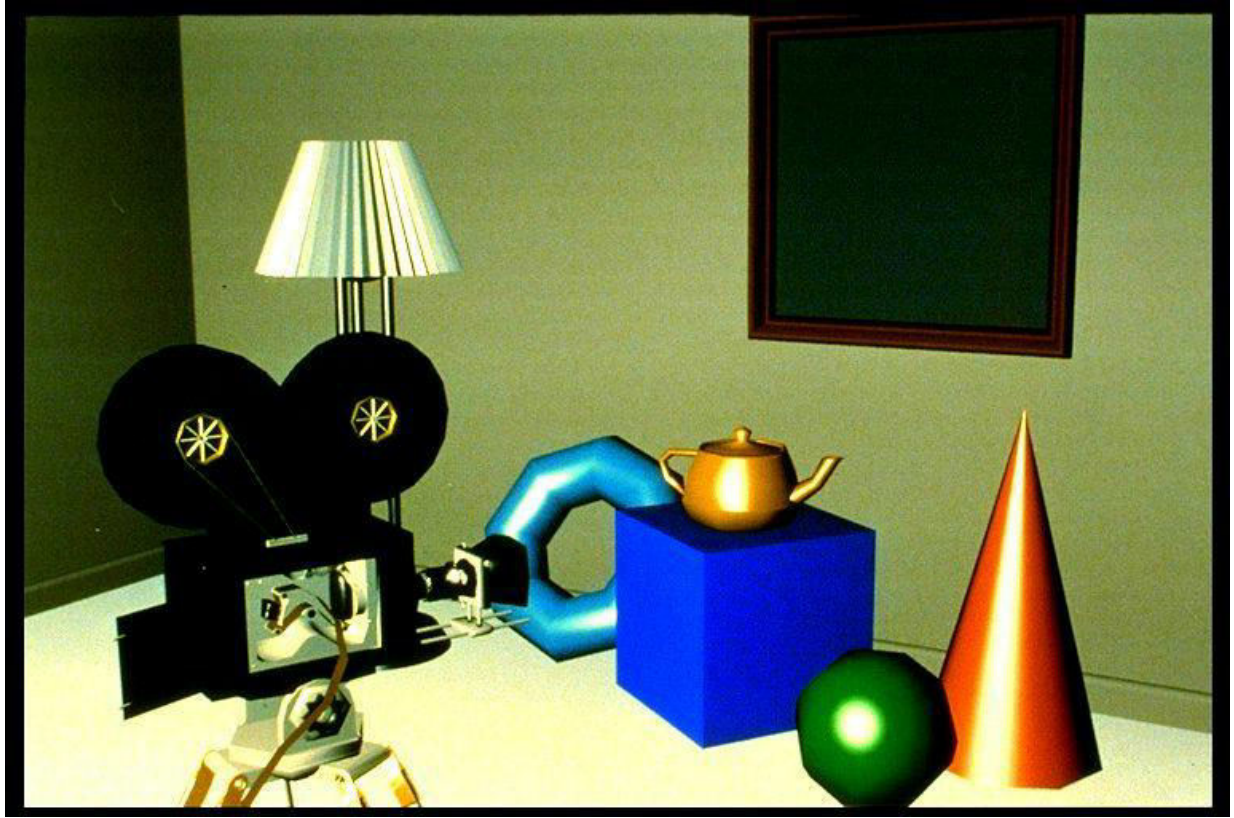




No Surface Rendering Vs Surface Rendering



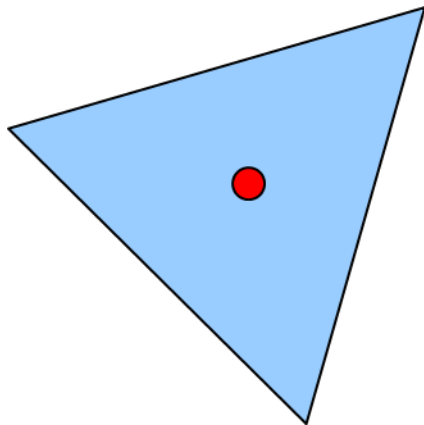
Object Rendering



With Surface Rendering

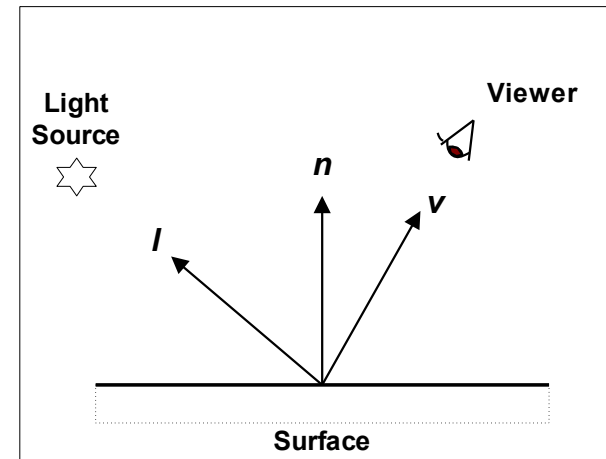
Flat Surface Rendering

- ❑ Simplest method, same color is assigned to all surface positions
- ❑ Illumination at a single point (usually center) on the surface is calculated and used for the entire surface
- ❑ Surface rendering is extremely fast, but can be unrealistic
 - Highlights not visible,
 - Facetted appearance, increased by Mach banding effect.

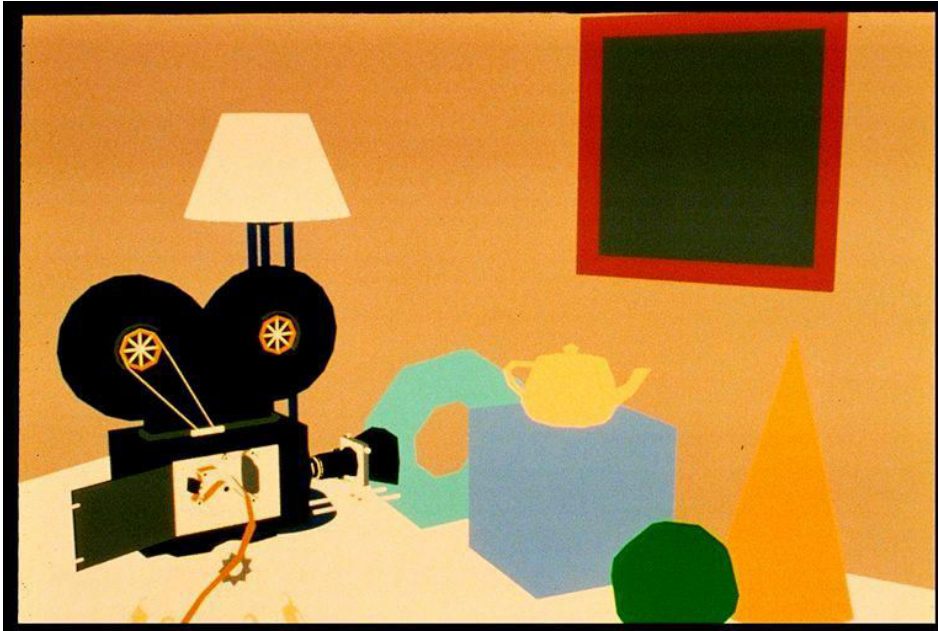


Flat Surface Rendering

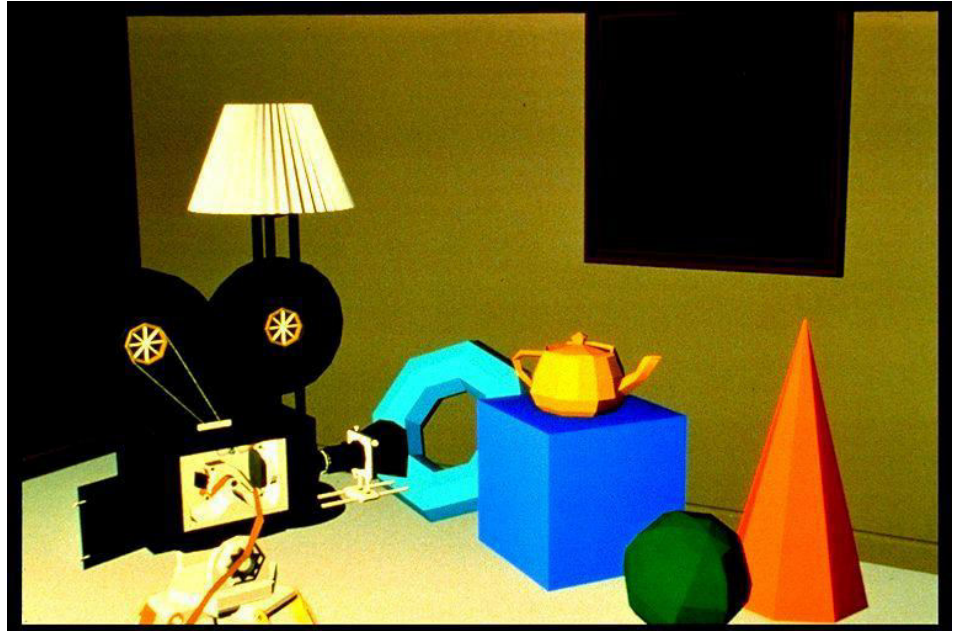
- ❑ Assumptions - For each surface
 - Light source at infinity is constant $\vec{n} \cdot \vec{l}$
 - Viewer at infinity is constant $\vec{n} \cdot \vec{v}$
 - The polygon represents the actual surface being modeled
- ❑ Ok if:
 - Object consists of planar faces, and
 - Light sources are far away, and
 - Eye point is far away,
- ❑ or
 - Polygons are about a pixel in size.



No Surface Rendering Vs Flat Surface Rendering



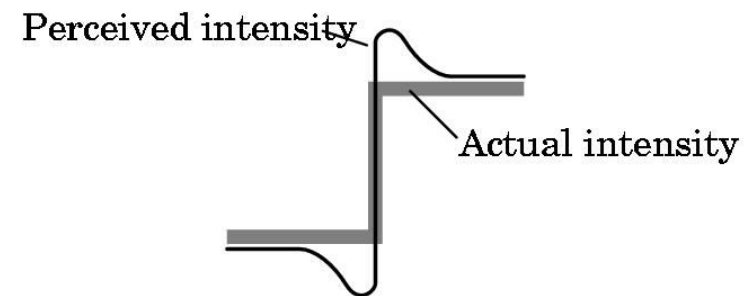
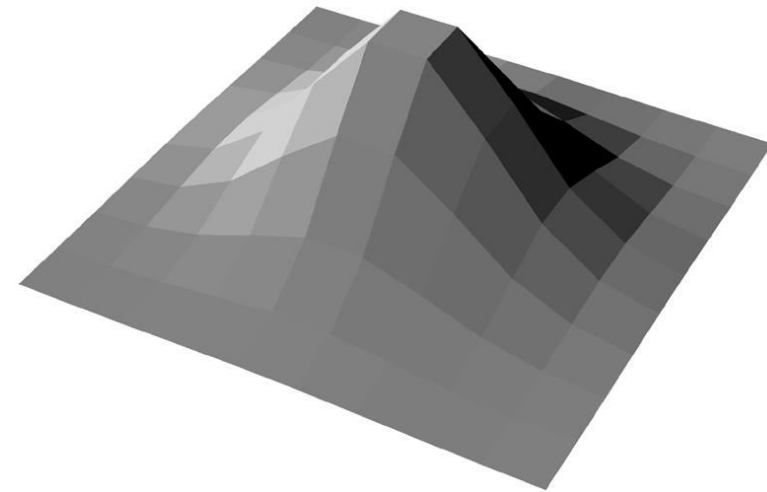
No Surface Rendering



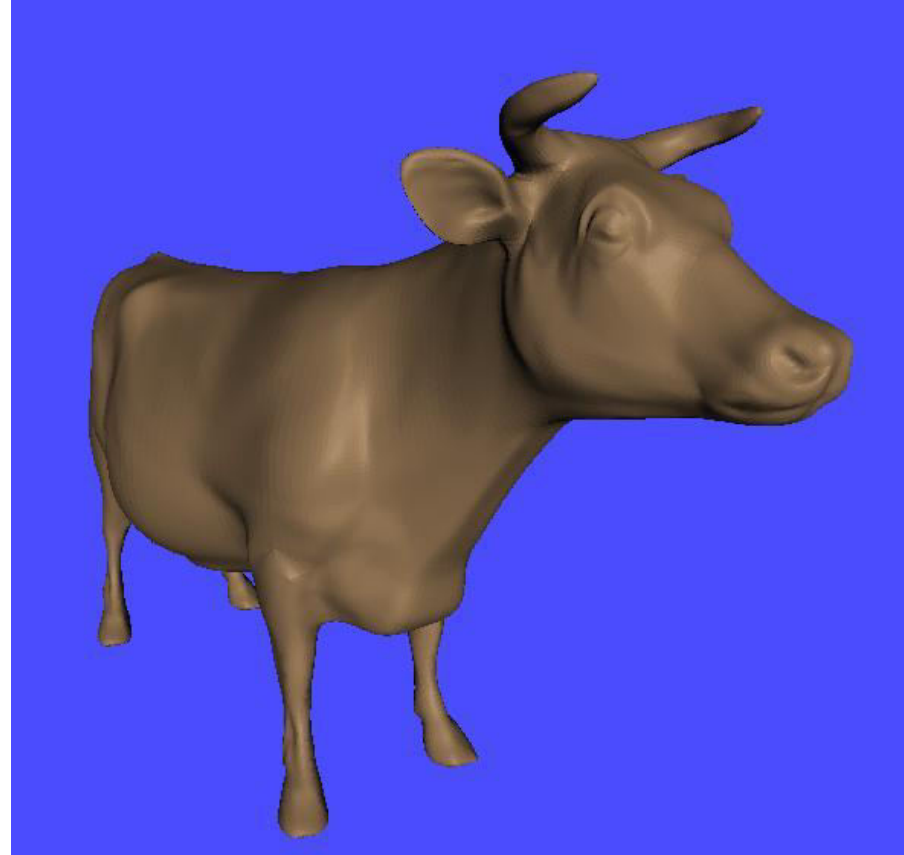
Flat Surface Rendering

Flat shading drawbacks

- ❑ The human visual system enhances edges
- ❑ We see stripes (known as Mach Bands) along edges
- ❑ Much like a sharpening convolution!
- ❑ How to avoid?



Overcoming Flat Shading Limitations



- ❑ Just add lots and lots of polygons – however, this is SLOW!

Interpolative Shading

- ❑ Calculate color at each vertex
- ❑ Interpolate color in interior points
- ❑ Compute during scan conversion (rasterization)
- ❑ Much better image
- ❑ More expensive to calculate
- ❑ Enable with *`glShadeModel(GL_SMOOTH);`*

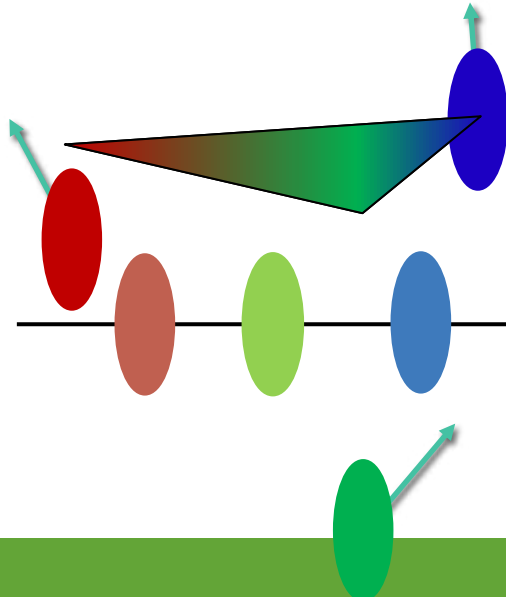
Gouraud Surface Rendering

- ❑ Gouraud surface shading was developed in the 1970s by Henri Gouraud
- ❑ Worked at the University of Utah along with Ivan Sutherland and David Evans
- ❑ Often also called **intensity-interpolation surface rendering**
- ❑ Intensity levels are calculated at each vertex and interpolated across the surface



Gouraud Surface Rendering (cont...)

- ❑ To render a polygon, Gouraud surface rendering proceeds as follows:
 1. Determine the average unit **normal** vector **at each vertex** of the polygon
 2. **Apply an illumination model** at each polygon vertex to **obtain the light intensity** at that position
 3. **Linearly interpolate** the vertex intensities over the projected area of the polygon



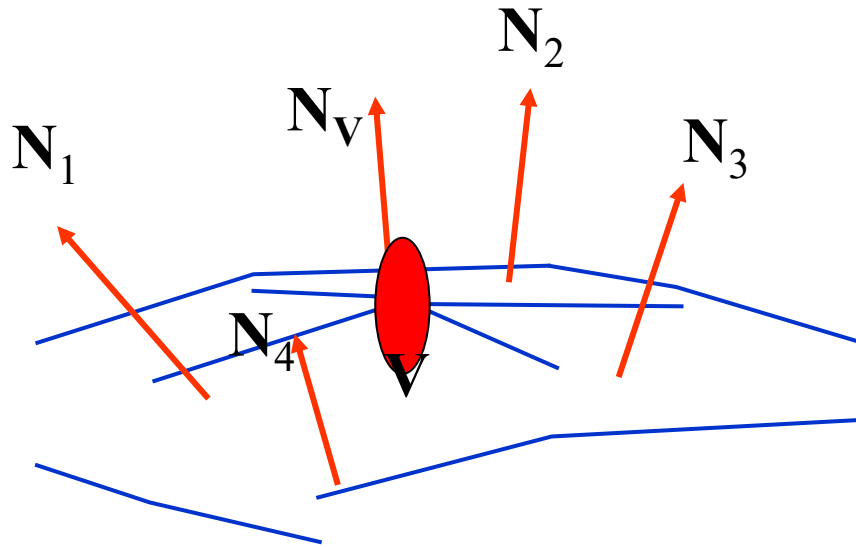
Gouraud Surface Rendering (cont...)

- The average unit normal vector at V is given as:

$$N_v = \frac{N_1 + N_2 + N_3 + N_4}{|N_1 + N_2 + N_3 + N_4|}$$

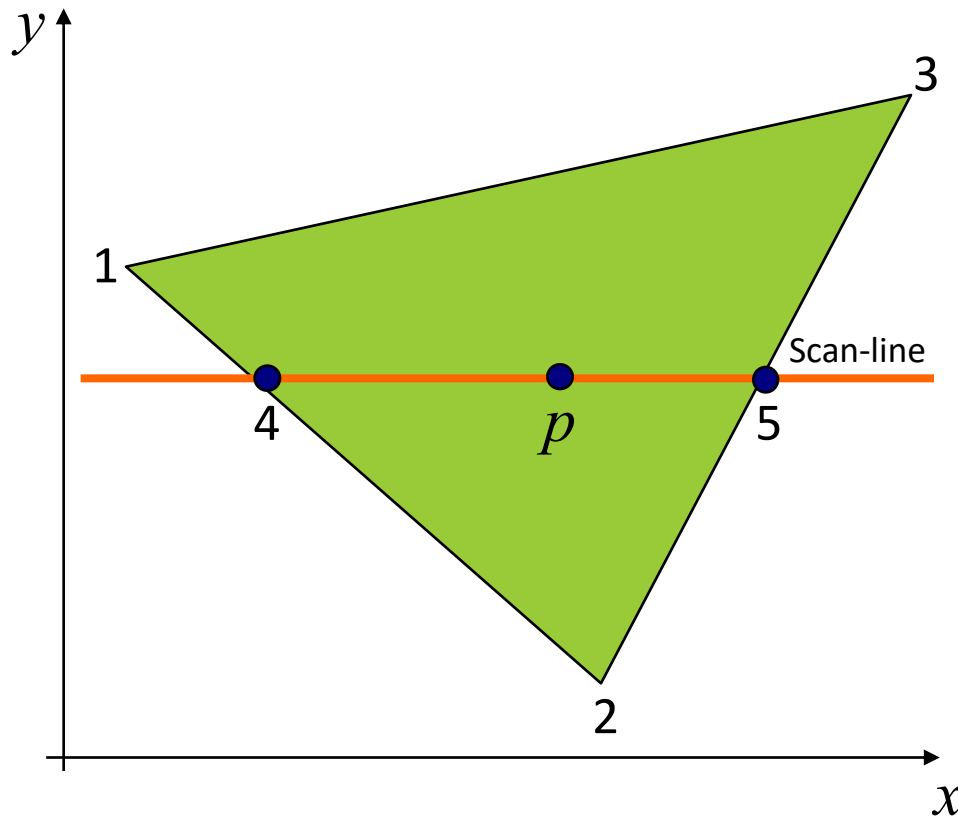
- or more generally:

$$N_v = \frac{\sum_{i=1}^n N_i}{\left| \sum_{i=1}^n N_i \right|}$$



Gouraud Surface Rendering (cont...)

- Illumination values are linearly interpolated across each scan-line



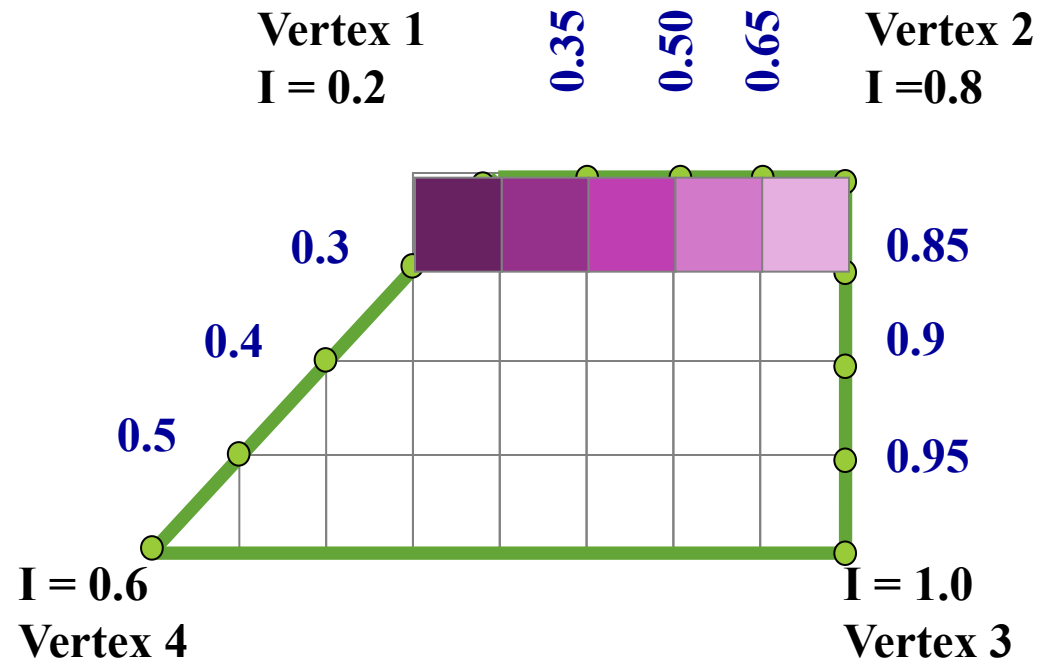
$$I_4 = \frac{y_4 - y_2}{y_1 - y_2} I_1 + \frac{y_1 - y_4}{y_1 - y_2} I_2$$

$$I_5 = \frac{y_5 - y_2}{y_3 - y_2} I_3 + \frac{y_3 - y_5}{y_3 - y_2} I_2$$

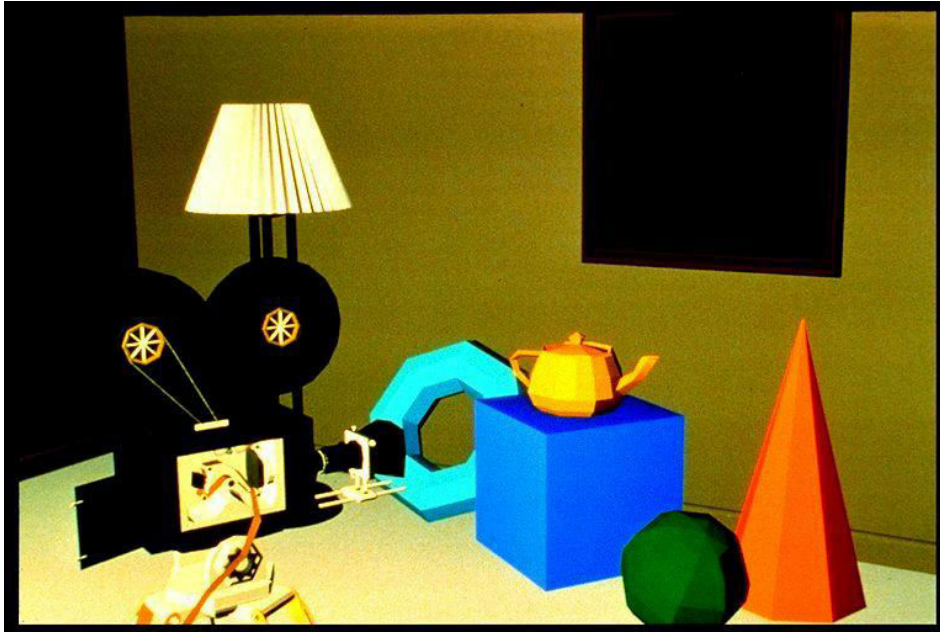
$$I_p = \frac{x_5 - x_p}{x_5 - x_4} I_4 + \frac{x_p - x_4}{x_5 - x_4} I_5$$

Gouraud Shading

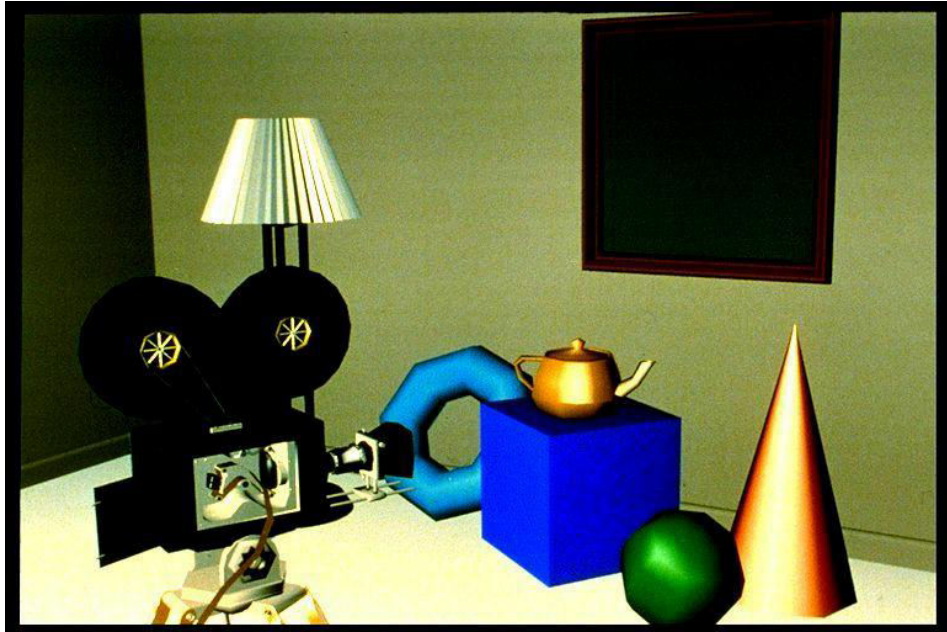
An Example



Flat Vs Gouraud Rendering



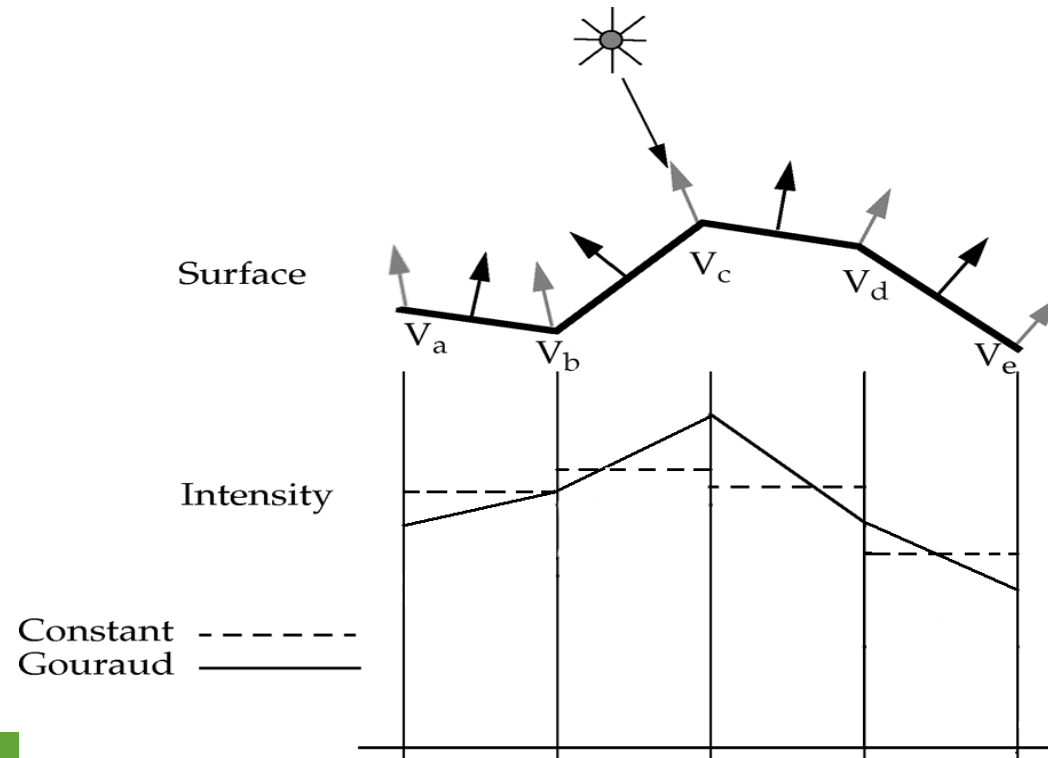
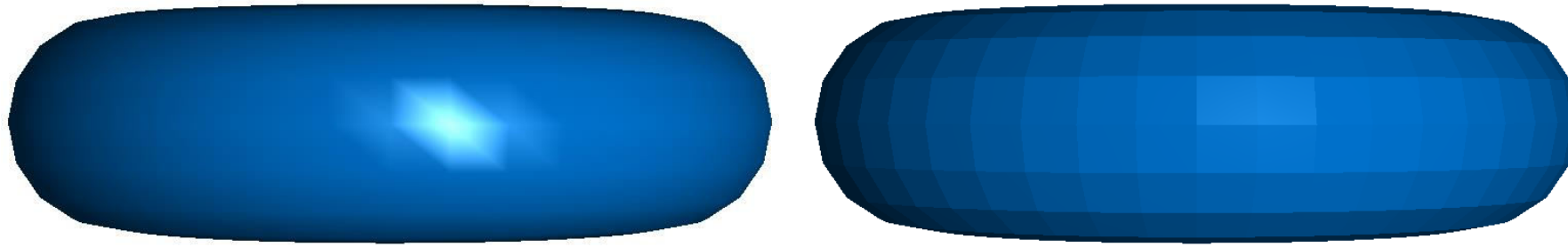
Flat Surface Rendering



Gouraud Surface Rendering

Advantages of Gouraud Surface Rendering

- ❑ Much better result for curved surfaces



Gouraud drawbacks

- ❑ Polygon edges are still visible
- ❑ Brightness is modelled as a linear function, but that's not really accurate
- ❑ Real highlights are small and bright and drop off sharply
 - If polygons are too large, highlights get distorted and dimmed (notice the funny shape)
- ❑ How to avoid these artifacts?

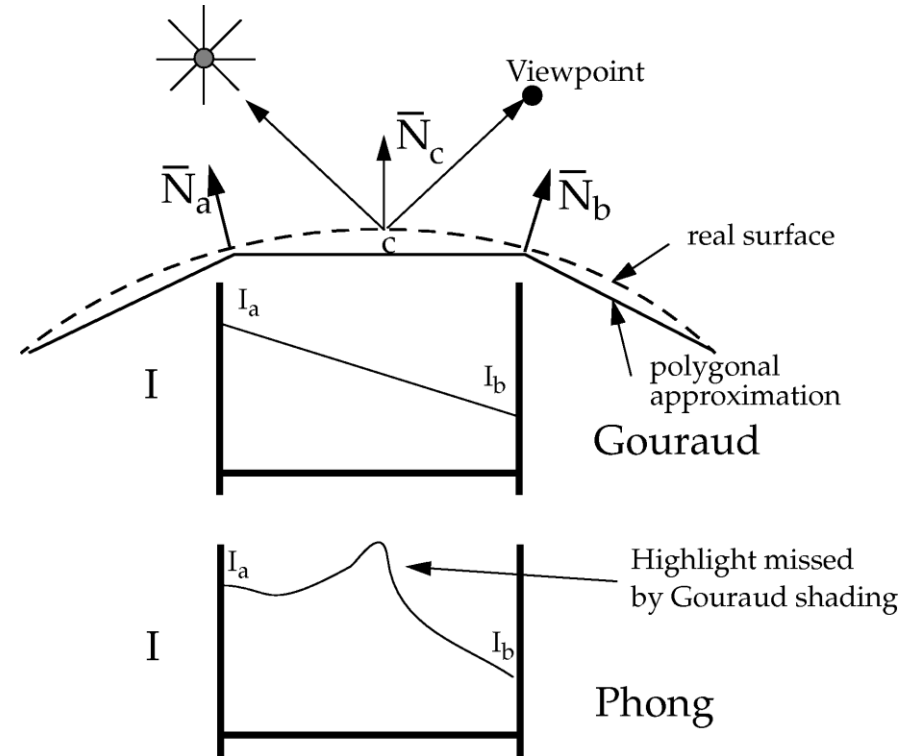
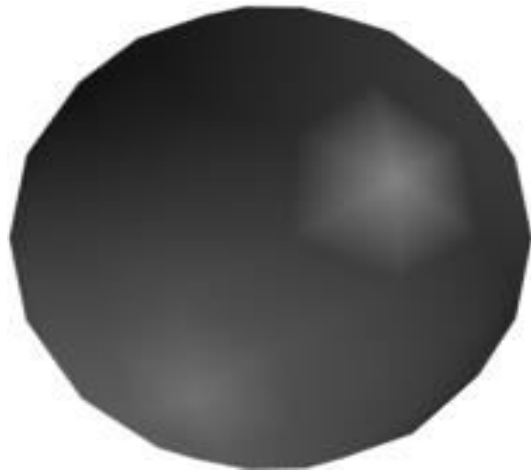


Problems With Gouraud Shading

❑ It has a problem with

specular reflections

- Completely miss
- interpolate



❑ Linear interpolation still gives Mach banding

- Silhouettes are still not smooth

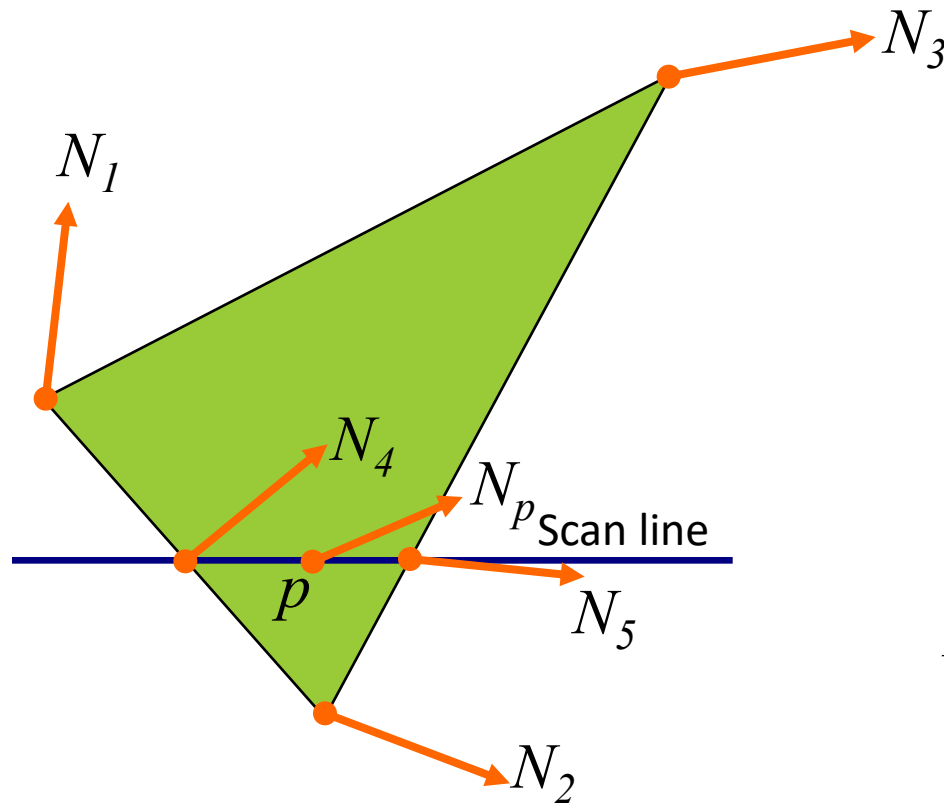
Phong Surface Rendering

- ❑ A more accurate interpolation based approach for rendering a polygon was developed by Phong Bui Tuong
- ❑ Basically the Phong surface rendering model (or **normal-vector interpolation rendering**) interpolates normal vectors instead of intensity values

Phong Surface Rendering (cont...)

- ❑ To render a polygon, Phong surface rendering proceeds as follows:
 1. Determine the average unit normal vector at each vertex of the polygon
 2. Linearly interpolate the vertex normals over the projected area of the polygon
 3. Apply an illumination model at positions along scan lines to calculate pixel intensities using the interpolated normal vectors

Phong Surface Rendering (cont...)

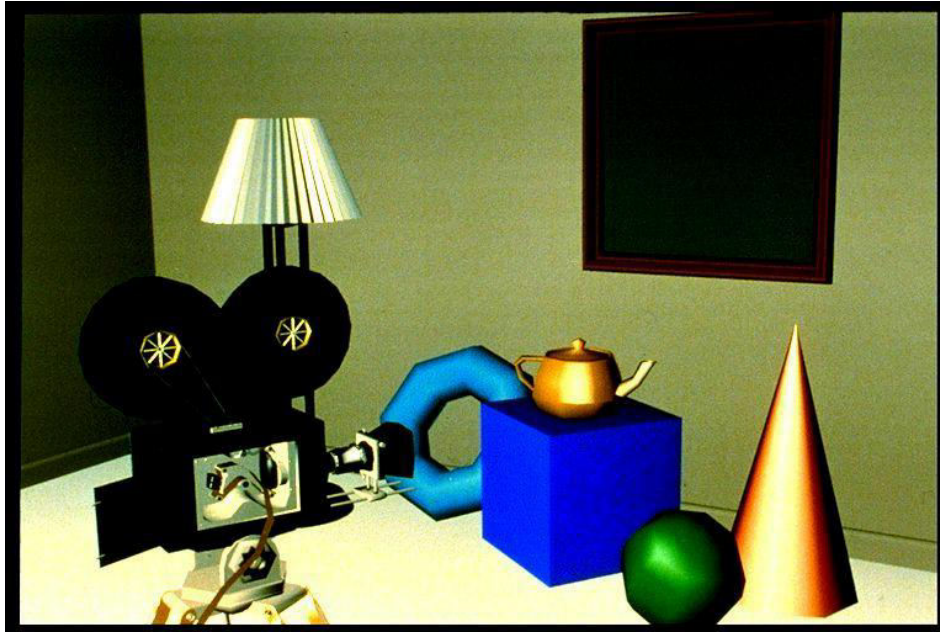


$$N_4 = \frac{y_4 - y_2}{y_1 - y_2} N_1 + \frac{y_1 - y_4}{y_1 - y_2} N_2$$

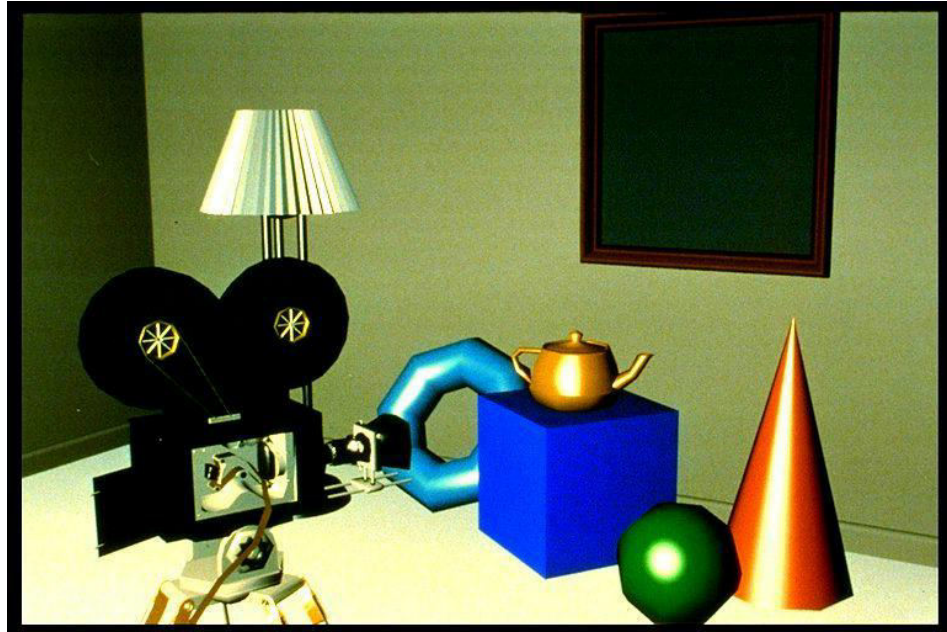
$$N_5 = \frac{y_5 - y_2}{y_3 - y_2} N_3 + \frac{y_3 - y_5}{y_3 - y_2} N_2$$

$$N_p = \frac{y_p - y_5}{y_4 - y_5} N_4 + \frac{y_4 - y_p}{y_4 - y_5} N_5$$

Gouraud Vs Phong Surface Rendering

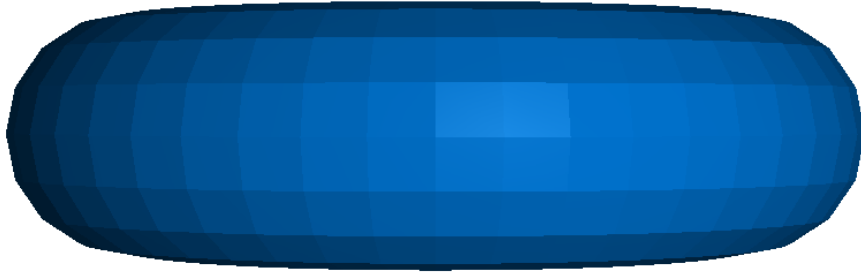


Gouraud Surface Rendering

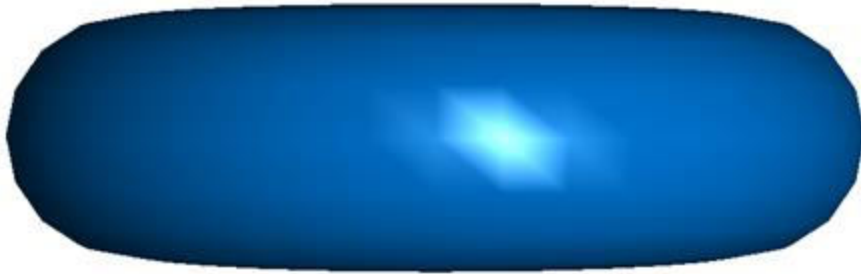


Phong Surface Rendering

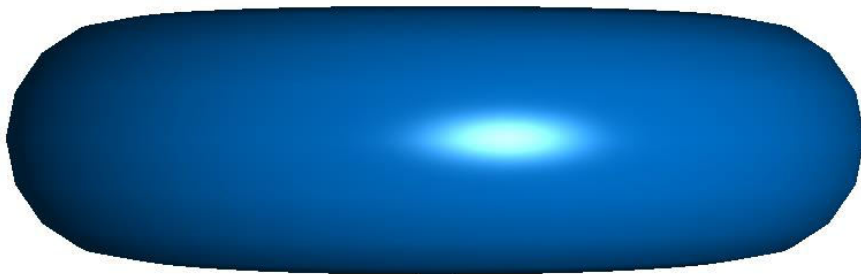
Phong Shading



Flat



Gouraud



Phong

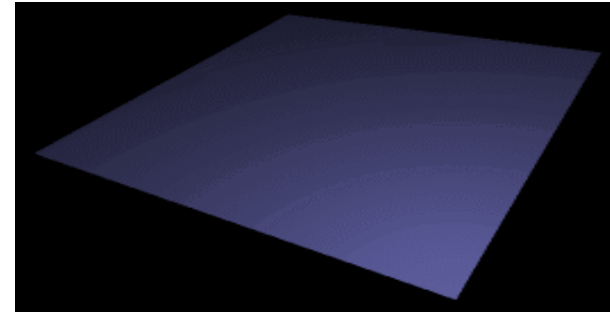
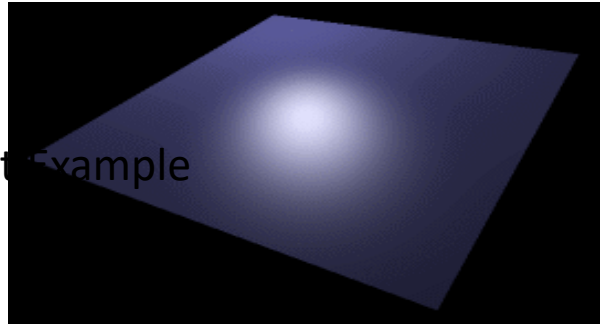
Phong Surface Rendering

- ❑ Even better result for curved surfaces
- ❑ No errors at high lights
- ❑ No Mach banding
- ❑ Phong shading is much slower than Gouraud shading as the lighting model is reevaluated so many times
- ❑ There are fast Phong surface rendering approaches that can be implemented iteratively
- ❑ Typically, implemented as part of a visible surface detection technique
- ❑ **Not supported in OpenGL**

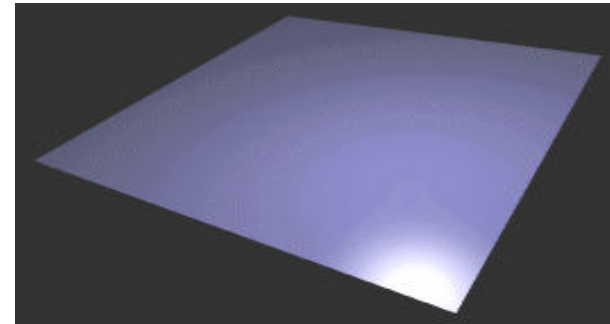
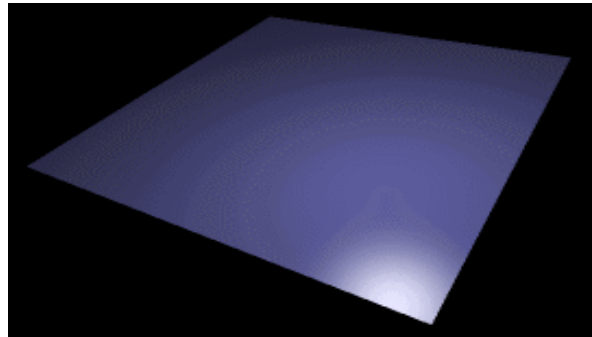
Phong vs Gouraud Shading

- ❑ If a highlight does not fall on a vertex Gouraud shading may miss it completely, but Phong shading does not.
- ❑ if highlight falls on vertex, Gouraud shading will spread the highlight over the polygon

Gouraud Tea Pot Example



Phong Tea Pot Example



Summary

- ❑ For realistic rendering of polygons we need interpolation methods to determine lighting positions
- ❑ Flat shading is fast, but unrealistic
- ❑ Gouraud shading is better, but does not handle specular reflections very well
- ❑ Phong shading is even better, but can be slow